

грокаем

Глубокое обучение

с подкреплением

Мигель Моралес



grokking

Deep Reinforcement Learning

Miguel Morales

Foreword by Charles Isbell, Jr.



MANNING
SHELTER ISLAND

играем

Глубокое обучение с подкреплением

Мигель Моралес



Санкт-Петербург • Москва • Минск

2023

ББК 32.813+32.973.23-018
УДК 004.89+004.85
М79

Моралес Мигель

М79 Прокаем глубокое обучение с подкреплением. — СПб.: Питер, 2023. — 464 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-3944-6

Мы учимся, взаимодействуя с окружающей средой, и получаемые вознаграждения и наказания определяют наше поведение в будущем. Глубокое обучение с подкреплением привносит этот естественный процесс в искусственный интеллект и предполагает анализ результатов для выявления наиболее эффективных путей движения вперед. Агенты глубокого обучения с подкреплением могут способствовать успеху маркетинговых кампаний, прогнозировать рост акций и побеждать гроссмейстеров в Го и шахматах.

Давайте научимся создавать системы глубокого обучения на примере увлекательных упражнений, сопровождаемых кодом на Python с подробными комментариями и понятными объяснениями. Вы увидите, как работают алгоритмы, и научитесь создавать собственных агентов глубокого обучения с подкреплением, используя оценочную обратную связь.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.813+32.973.23-018
УДК 004.89+004.85

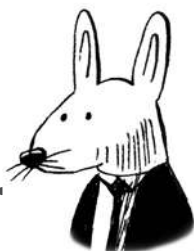
Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1617295454 англ.
ISBN 978-5-4461-3944-6

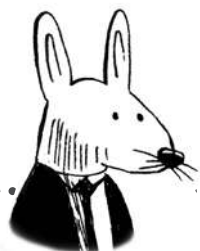
© 2020 by Manning Publications Co. All rights reserved
© Перевод на русский язык ООО «Прогресс книга», 2023
© Издание на русском языке, оформление ООО «Прогресс книга», 2023
© Серия «Библиотека программиста», 2023

Краткое содержание



Предисловие	10
Вступление	12
Благодарности	14
О книге	17
От издательства	19
Об авторе	20
Глава 1. Введение в глубокое обучение с подкреплением	21
Глава 2. Математические основы обучения с подкреплением	52
Глава 3. Баланс краткосрочных и долгосрочных целей	86
Глава 4. Баланс между сбором и использованием информации	119
Глава 5. Оценка поведения агента	153
Глава 6. Улучшение поведения агентов	189
Глава 7. Более действенные и эффективные способы достижения целей	224
Глава 8. Введение в ценностно ориентированное глубокое обучение с подкреплением	259
Глава 9. Более стабильные ценностно ориентированные методы	296
Глава 10. Ценностно ориентированные методы с эффективным использованием выборок	332
Глава 11. Методы градиента политик и «актер-критик»	362
Глава 12. Продвинутое методы «актер-критик»	401
Глава 13. Путь к сильному искусственному интеллекту	437

Оглавление



Предисловие	10
Вступление	12
Благодарности	14
О книге	17
Для кого эта книга	17
Структура издания	17
О коде	18
От издательства	19
Об авторе	20
 Глава 1. Введение в глубокое обучение с подкреплением	 21
Что такое глубокое обучение с подкреплением	22
Прошлые, настоящее и будущее глубокого обучения с подкреплением	35
Целесообразность глубокого обучения с подкреплением	43
Определение четких обоюдных ожиданий	46
Подведем итоги	50
 Глава 2. Математические основы обучения с подкреплением	 52
Элементы обучения с подкреплением	54
MDP: двигатель среды	65
Подведем итоги	84

Глава 3. Баланс краткосрочных и долгосрочных целей	86
Цель агента, принимающего решения	87
Планирование оптимальных последовательностей действий	99
Подведем итоги	117
Глава 4. Баланс между сбором и использованием информации	119
Проблема интерпретации оценочной обратной связи	121
Стратегическое исследование	139
Подведем итоги	151
Глава 5. Оценка поведения агента	153
Учимся прогнозировать ценность политик	155
Прогноз на основе нескольких шагов	173
Подведем итоги	187
Глава 6. Улучшение поведения агентов	189
Анатомия агентов обучения с подкреплением	190
Оптимизация политик поведения	198
Разделение поведения и обучения	208
Подведем итоги	222
Глава 7. Более действенные и эффективные способы достижения целей	224
Улучшение политик с помощью достоверных целей	226
Агенты, которые взаимодействуют, обучаются и планируют	237
Подведем итоги	257
Глава 8. Введение в ценностно ориентированное глубокое обучение с подкреплением	259
Тип обратной связи, который используют агенты глубокого обучения с подкреплением	261
Введение в аппроксимацию функций для обучения с подкреплением	268
NFQ: первая попытка реализовать ценностно ориентированное глубокое обучение с подкреплением	273
Подведем итоги	294

Глава 9. Более стабильные ценностно ориентированные методы	296
DQN: делаем RL похожим на контролируемое обучение	297
Двойная DDQN: борьба с завышением прогнозов функций ценности действий	314
Подведем итоги	329
Глава 10. Ценностно ориентированные методы с эффективным использованием выборок	332
Дуэльная DDQN: архитектура нейросети, рассчитанная на обучение с подкреплением	333
PER: приоритетное воспроизведение полезного опыта	346
Подведем итоги	360
Глава 11. Методы градиента политик и «актер-критик»	362
REINFORCE: обучение политик на основе результатов	363
VPG: формирование функции ценности	374
A3C: параллельное обновление политики	380
GAE: надежное прогнозирование преимущества	387
A2C: синхронное обновление политик	391
Подведем итоги	399
Глава 12. Продвинутые методы «актер-критик»	401
DDPG: аппроксимация детерминированной политики	403
TD3: лучшие оптимизации для DDPG	410
SAC: максимизация ожидаемой выгоды и энтропии	417
PPO: ограничение этапа оптимизации	425
Подведем итоги	435
Глава 13. Путь к сильному искусственному интеллекту	437
Важные темы, которые были рассмотрены, и те, которые остались без внимания	439
Углубленные аспекты AGI	451
Что дальше?	458
Подведем итоги	461

*Данелл, Авроре, Соломону и тем, кто будет после них.
Быть вместе с вами словно получать +1
за каждый временной шаг.
(Могу заверить, что +1 — это наивысшая награда.)
Я вас люблю!*

Предисловие



В этой книге вы узнаете об обучении с подкреплением (reinforcement learning, RL). Его сложно понять и объяснить по ряду причин. Во-первых, это довольно специфический подход — здесь много математики. Усвоить основные знания по этой теме и не увязнуть в них — уже непростая задача.

Во-вторых, обучение с подкреплением изначально предполагает некоторое логическое несоответствие, так как это одновременно и способ осмысления задач принятия решений, и набор инструментов для решения этих задач. Под способом осмысления я имею в виду то, что RL служит основой для принятия решений: помимо прочего, в нем обсуждаются сигналы состояния и подкрепления. Говоря о наборе инструментов, я подразумеваю, что при обсуждении RL мы зачастую пользуемся такими терминами, как *марковские процессы принятия решений* и *алгоритм Беллмана*. На удивление легко спутать способ осмысления с математическими инструментами, которые мы используем.

Наконец, RL можно по-разному реализовать. Поскольку это способ осмысления, мы можем говорить о нем как о чем-то абстрактном, но его также можно воплотить в коде или, если уж на то пошло, в виде нейронов.

Для изучения RL и глубоких нейронных сетей нужны разные фундаментальные знания — это два интересных направления исследований, которые развивались независимо друг от друга. Объяснить и то и другое в контексте средств разработки — непростая задача.

Не стоит забывать, что для понимания обучения с подкреплением нужно не только овладеть инструментами и их реализацией в глубоких нейросетях, но и перенять определенный способ осмысления RL. Иначе вы не сможете делать обобщения за рамками тех примеров, которые изучаете. Повторюсь, преподавать RL сложно, а в обучении глубокому RL очень много нюансов, которые могут свести весь процесс на нет. Чтобы всего этого избежать, мы обращаемся к книге Мигеля Моралеса.

Эта книга очень удачно составлена. Здесь простым языком объясняется, что такое машинное обучение (machine learning, ML), глубокое обучение (deep learning, DL) и обучение с подкреплением. Читателю дается обобщенная информация о положении дел в области искусственного интеллекта, возможностей методик глубокого обучения с подкреплением и способа осмысления ML, RL и DL. Книга может стать для вас учебным пособием, справочником и (как для меня) источником вдохновения.

Простота изложения материала меня совершенно не удивляет. Я знаю Мигеля не один год. Он прошел путь от студента на курсах по машинному обучению до преподавателя. Он был ведущим ассистентом на моем курсе по обучению с подкреплением и принятию решений в онлайн-магистратуре Технологического института Джорджии на протяжении многих семестров. За это время Мигель успел поработать с тысячами студентов. Я наблюдал за его ростом как профессионала, исследователя и педагога. Он помог улучшить мой курс по RL и продолжает его совершенствовать, чтобы сделать опыт обучения с подкреплением более глубоким для студентов. Мигель прирожденный учитель.

Я пишу этот текст в благодарность за его работу. Я счастлив, что могу сотрудничать с ним, и рад, что у него появилась возможность написать эту книгу. Приятного чтения. Думаю, вы многому научитесь. Я и сам кое-что узнал.

*Чарльз Исбелл-мл. (Charles Isbell, Jr.),
профессор и декан факультета в колледже
вычислительной техники
при Технологическом институте Джорджии*

Вступление



Обучение с подкреплением — это захватывающая область машинного обучения, способная внести серьезный вклад в развитие человечества. Несколько технологий уже повлияли на историю нашего мира, начиная с появления колеса, электричества и заканчивая Интернетом. Каждое технологическое открытие стимулирует последующий прогресс: без электричества не было бы персональных компьютеров, без компьютеров не было бы Интернета, а без Интернета не было бы поисковых систем.

Для меня самый удивительный аспект RL и искусственного интеллекта — это не столько возможное появление других разумных существ (что само по себе довольно волнующе), сколько то, что идет после. Я считаю, что, будучи прочным фундаментом для автономной оптимизации определенных задач, обучение с подкреплением способно изменить мир. Помимо автоматизации, создание разумных машин может вывести понимание человеческого интеллекта на уровень, который нам сейчас недоступен. Возможно, зная, как найти оптимальные решения для любых задач, мы сможем понять алгоритм, который занимается поиском этих решений. Я думаю, что, научившись создавать разумных существ, люди сами могут стать умнее.

Но до этого еще далеко. Чтобы воплотить в жизнь эти смелые мечты, мы должны привлечь к процессу больше умов. Обучение с подкреплением уже довольно давно находится в зачаточном состоянии, поэтому впереди у нас много работы. Эта книга — мой вклад в то, чтобы DL и RL в целом занималось больше людей.

Базовая структура RL интуитивно ясна, но большинство материалов на эту тему новичку будет сложно понять. Я не ставил перед собой задачу сделать сборник примеров или справочник по теории обучения с подкреплением. Мне хотелось создать ресурс, который смог бы объединить теорию и практику. Как вы вскоре сами увидите, я не брезгую математическими уравнениями: если вы хо-

тите постичь какую-то область исследований, без них не обойтись. Но даже если вас интересует только практическая сторона дела, для разработки качественных решений на основе RL все равно понадобятся некоторые базовые знания. Я не полагаюсь только на уравнения, ведь не все, кто интересуется обучением с подкреплением, любят математику. Некоторым комфортнее работать с кодом и конкретными примерами, поэтому в книге представлено и много практики.

При написании этого пособия большая часть усилий была направлена на преодоление разрыва между теорией и практикой: доступное изложение базовой информации я сопровождаю примерами кода, уделяя много внимания деталям. Тем, кому сложно дается теория, здесь будет легче понять язык, на котором разговаривают ведущие исследователи. А тем, кто любит читать уравнения, но не может преобразовать их в код, наоборот, будет проще понять практическую сторону обучения с подкреплением.

Я буду искренне рад, если вам удастся насладиться книгой и почерпнуть из нее много полезной информации. Надеюсь, вы овладеете искусством глубокого обучения с подкреплением и сможете внести свой вклад в это фантастическое сообщество, которое я так полюбил. Как я уже упоминал, написание книги было бы невозможным без последних технологических инноваций. Но будущее этой отрасли зависит только от вас, поэтому дерзайте!

Благодарности



Я хочу поблагодарить сотрудников Технологического института Джорджии за то, что они взяли на себя риск и ввели первую онлайн-магистерскую программу компьютерных наук для получения качественного последипломного образования. Если бы не эти ребята, я бы, наверное, никогда не написал эту книгу.

Хочу выразить благодарность профессору и декану Чарльзу Исбеллу и профессору Майклу Литтману (Michael Littman) за создание замечательного курса по обучению с подкреплением. Я особенно признателен декану Исбеллу, который дал мне большой простор для профессионального роста и изучения RL. Кроме того, свой подход к преподаванию обучения с подкреплением (с разделением задачи на три вида обратной связи) я перенял именно у профессора Литтмана. Спасибо за ваши наставления.

Спасибо яркому преподавательскому составу курса CS 7642 в Технологическом институте Джорджии за совместную работу над тем, чтобы помочь студентам узнать больше и приятно провести время с нами. Хочу выразить отдельную благодарность Тиму Бейлу (Tim Bail), Пушкару Колхе (Pushkar Kolhe), Крису Серрано (Chris Serrano), Фарруку Рахману (Farrukh Rahman), Вахе Хагопяну (Vahe Hagopian), Куинну Ли (Quinn Lee), Таке Хасегаве (Taka Hasegawa), Чжу Тяньхангу (Tianhang Zhu) и Дону Джейкобу (Don Jacob). Мы с вами отлично сработались.

Спасибо тем, кто ранее внес существенный вклад в этот курс. Я многое для себя почерпнул из нашего общения: Алек Фейерштейн (Alec Feuerstein), Валькирия Фелсо (Valkyrie Felso), Эдриен Экоффет (Adrien Ecoffet), Каушик Субраманиян (Kaushik Subramanian) и Эшли Эдвардс (Ashley Edwards). Хочу поблагодарить наших студентов за их вопросы, которые помогли мне определить пробелы в знаниях тех, кто пытается освоить RL. Я писал эту книгу, думая о вас. Особой признательности заслуживает анонимный студент, который по-

советовал мне выбрать Manning в качестве издательства для этой книги. Я до сих пор не знаю, кто это был, но спасибо ему.

Хочу поблагодарить сотрудников Lockheed Martin за обратную связь в ходе написания этой книги. Особая благодарность Крису Астеду (Chris Aasted), Джулии Куок (Julia Kwok), Тейлор Лопес (Taylor Lopez) и Джону Хэддону (John Haddon), который одним из первых прочел мой самый ранний черновик. Его отзыв помог вывести мои писательские навыки на новый уровень.

Спасибо ребятам из Manning за то, что предоставили мне условия, благодаря которым эта книга стала реальностью. Спасибо Брайану Сойеру (Brian Sawyer) за то, что связался со мной и открыл передо мной двери; Берту Бейтсу (Bert Bates) за то, что с самого начала задал направление и помог мне сосредоточиться на преподавании; Кэндес Уэст (Candace West) за то, что помогла мне превратиться из «никого» в «кого-то»; Сюзанне Клайн (Susanna Kline) за то, что помогла мне ускорить работу, когда я был занят; Дженнифер Стаут (Jennifer Stout) за моральную поддержку в конце пути; Ребекке Рейнхарт (Rebecca Rinehart) за тушение пожаров; Эллу Кринкеру (Al Krinker) за отзывы, послужившие руководством к действию, и за помощь в отделении полезной информации от шума; Матко Хрватину (Matko Hrvatin) за усердную работу над выпусками MEAP и за дополнительную мотивацию в написании этой книги; Кэндис Гиллхулли (Candace Gillhoolley) за продвижение этой книги, Степану Журековичу (Stjepan Jureković) за то, что вытащил меня; Ивану Мартиновичу (Ivan Martinovic) за важный отзыв, который помог улучшить текст; Лори Вейдерт (Lori Weidert) за то, что дважды подготовила эту книгу к изданию; Дженнифер Хоул (Jennifer Houle) за терпение к изменениям в дизайне; Кэти Петито (Katie Petito) за кропотливую работу над деталями; Кэти Теннант (Katie Tennant) за последние штрихи. Я знаю, что здесь не все: спасибо вам за ваш тяжелый труд.

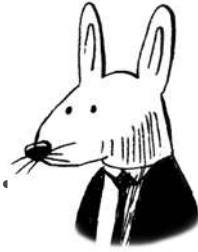
Всем рецензентам — Эллу Рахими (Al Rahimi), Алену Коньюоту (Alain Couniot), Альберто Чарланти (Alberto Ciarlanti), Дэвиду Финтону (David Finton), Дониёру Улмасову (Doniyor Ulmasov), Эдиссону Реинозо (Edisson Reinozo), Эзре Джоелу Шредеру (Ezra Joel Schroeder), Хэнку Мейссе (Hank Meisse), Хао Лю (Hao Liu), Айку Оконкво (Ike Okonkwo), Жи Мей (Jie Mei), Джульену Похи (Julien Pohie), Киму Фолку Йоргенсену (Kim Falk Jørgensen), Марку-Филиппе Хагету (Marc-Philippe Huget), Михаэлю Халлеру (Michael Haller), Мишель Кломп (Michel Klomp), Начо Орменьо (Nacho Ormeño), Робу Пачеко (Rob Pacheco), Себастьяну Майеру (Sebastian Maier), Себастьяну Заба (Sebastian Zaba), Свамнатану Субраманияну (Swaminathan Subramanian), Тайлеру Коваллису (Tyler Kowallis), Урсину Стауссу (Ursin Stauss) и Сяоху Чжу (Xiaohu Zhu) — спасибо за то, что помогли сделать эту книгу лучше.

Хочу поблагодарить работников Udacity за то, что дали мне возможность поделиться своей страстью к глубокому обучению со своими студентами и записать лекции по алгоритму «актер-критик» для их программы Deep Reinforcement Learning Nanodegree. Отдельное спасибо Алексис Кук (Alexis Cook), Мэту Леонарду (Mat Leonard) и Луису Серрано (Luis Serrano).

Я благодарен членам сообщества RL за то, что помогли мне прояснить этот текст и лучше понять материал. Отдельное спасибо Дэвиду Сильверу (David Silver), Сергею Левину (Sergey Levine), Хадо ван Хасселту (Hado van Hasselt), Паскалю Пупарту (Pascal Poupart), Джону Шульману (John Schulman), Питеру Аббелю (Pieter Abbeel), Челсе Финн (Chelsea Finn) и Владу Мниху (Vlad Mnih) за их лекции; Ричу Саттону (Rich Sutton) за предоставление материалов эталонного учебника в этой области; Джеймсу Макглашану (James MacGlashan) и Джошуа Ахиму (Joshua Achiam) за их кодовую базу, онлайн-ресурсы и подсказки, когда я не знал, где искать ответ. Я хочу поблагодарить Дэвида Ха (David Ha) за то, что он подсказал мне, куда двигаться дальше.

Выражаю особую благодарность Сильвии Море (Silvia Mora) за помощь в подготовке иллюстраций к книге и участие почти во всех моих проектах.

Наконец, спасибо моей семье, которая служила мне опорой на протяжении всего проекта. Я знал, что написать книгу будет непросто, и позже в этом убедился. Но, несмотря ни на что, мои жена и дети всегда были рядом, ожидая моих 15-минутных перерывов каждые два часа. Спасибо тебе, Соло, за все моменты счастья. Спасибо тебе, Роза, за твою любовь и красоту. И спасибо тебе, Данелл, моей прекрасной жене, за то, что ты есть. Ты мой верный напарник в этой интересной игре под названием жизнь. Я так рад, что встретил тебя.



Цель этой книги — устранить разрыв между теорией и практикой глубокого обучения с подкреплением. Она рассчитана на читателей, знакомых с методами машинного обучения, которые хотят разобраться в обучении с подкреплением. Сначала я познакомлю вас с основами DRL. Затем дам подробный анализ соответствующих алгоритмов и подходов. В конце вы познакомитесь с передовыми методиками, которые могут иметь большое значение в будущем.

Для кого эта книга

Моя книга будет по душе тем, кого не пугают исследовательская работа, код на Python и интуитивно понятные разъяснения. Здесь есть немного математики и много интересных наглядных примеров, способствующих обучению. Даже если вы знаете только Python, этого будет достаточно, чтобы почерпнуть много полезной информации. Хотя и предполагается, что вы уже знакомы с основами глубокого обучения, здесь приводится краткий обзор нейронных сетей, обратного распространения и связанных с ними методик. В целом книга самодостаточна, и, если вы хотите поэкспериментировать с агентами искусственного интеллекта и освоить глубокое обучение с подкреплением, она вам в этом поможет.

Структура издания

Книга состоит из 13 глав, разделенных на две части.

Первая часть охватывает главы 1–5. В первой главе вы познакомитесь с областью глубокого обучения с подкреплением и узнаете наш дальнейший план действий. Во второй будет представлена структура для разработки задач, понятных агентам RL. В третьей главе подробно рассматриваются алгоритмы

решения тех задач, в которых агенту известны тенденции окружающего мира, а в четвертой — тех, в которых агент не знает о них. В главе 5 вы познакомитесь с методами решения проблемы прогнозирования, которые лежат в основе передовых методов RL.

Во вторую часть входят главы 6–13. В шестой главе представлены методы выполнения задач управления, алгоритмы оптимизации правил только на основе метода проб и ошибок. Глава 7 рассказывает о более продвинутых методах RL, в том числе о тех, что используют планирование для повышения эффективности выборки. В главе 8 речь пойдет об аппроксимации функций в RL за счет реализации простого алгоритма на основе нейронных сетей. В главе 9 подробно обсуждаются передовые методы применения аппроксимации функций для выполнения задач обучения с подкреплением. В главе 10 вы найдете несколько передовых техник для дальнейшего усовершенствования методик, изученных ранее. В главе 11 представлена другая методика использования моделей DRL, эффективность которой доказана в многочисленных тестах. В главе 12 подробно рассматриваются продвинутые методы DRL, передовые алгоритмы и распространенные способы решения реальных задач. В главе 13 можно найти обзор передовых направлений развития исследований RL, которые выглядят наиболее перспективно в контексте сильного искусственного интеллекта.

О коде

В книге вы найдете много примеров исходного кода: в виде листингов с заголовком «Я знаю Python» и в самом тексте. Исходный код отформатирован с помощью **моноширинного шрифта**, чтобы отделить его от обычного текста, и оснащен подсветкой синтаксиса, чтобы вам было легче его читать.

Оригинальный исходный код зачастую переформатирован: в него добавлены переводы строк, в нем переименованы переменные и откорректированы отступы с учетом доступного на страницах книги места. Иногда, очень редко, код содержит оператор продолжения строки в Python (`\`), который сигнализирует о том, что выражение продолжается на следующей строке.

Кроме того, из листингов с исходным кодом были удалены комментарии, а сам код описывается в тексте книги. Аннотации в нем указывают на важные понятия.

Код примеров из этой книги доступен для скачивания на сайте Manning: <https://www.manning.com/books/grokking-deep-reinforcement-learning> — и на странице GitHub: <https://github.com/mimoralea/gdrl>.

От издательства



Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

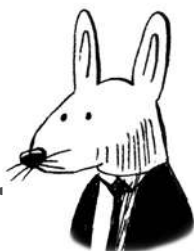
Поскольку эта книга черно-белая, в ней сложно корректно показать цвета линий на графиках. Цветные оригинальные иллюстрации вы можете скачать на нашем сайте по ссылке <https://clck.ru/34VaFw>. Они пронумерованы согласно страницам, на которых расположены.



Об авторе



Мигель Моралес (Miguel Morales) занимается обучением с подкреплением в отделе автономных ракетных и противопожарных систем компании Lockheed Martin в Денвере, штат Колорадо. Он занимает должность внештатного преподавателя в Технологическом институте Джорджии в рамках курса по обучению с подкреплением и принятию решений. Мигель работал в Udacity, где курировал проекты по машинному обучению, руководил обучающим курсом Nanodegree в области беспилотных автомобилей и участвовал в составлении курса по глубокому обучению с подкреплением. Он окончил Технологический институт Джорджии со степенью магистра компьютерных наук по специализации в области интерактивного интеллекта.



В этой главе

- ✓ Вы узнаете, что такое глубокое обучение с подкреплением и чем оно отличается от других подходов к машинному обучению.
- ✓ Познакомитесь с последними достижениями в сфере глубокого RL и узнаете, как оно может помочь в решении разных задач.
- ✓ Узнаете, чего ожидать от этой книги и как извлечь из нее максимум пользы.

Я представляю время, когда мы будем для роботов тем же, чем сейчас собаки являются для людей, и болею за машины.

*Клод Шеннон, отец информационного
века, выдающийся ученый в области
искусственного интеллекта*

Люди хотят быть счастливыми. Каждое наше действие, от выбора еды на завтрак до продвижения по карьерной лестнице, обусловлено стремлением привнести в жизнь приятные моменты. Это могут быть эгоцентричные удовольствия или благородные цели, то, что приносит немедленное или долгосрочное удовлетворение. Главное, что мы считаем это важным и ценным, ведь в каком-то смысле эти мгновения придают нашей жизни смысл.

Наша способность испытывать чувства от подобных моментов связана с интеллектом — способностью приобретать и применять знания и навыки. Люди, которых общество считает умными, могут отказываться не только от немедленного удовлетворения в пользу долгосрочных целей, но и от хорошего, гарантированного будущего в пользу лучшего, но неопределенного. Целей, которые дольше материализуются и обладают неизвестной долгосрочной ценностью, обычно сложнее всего достичь. Преодолеть все эти трудности могут исключительные люди — интеллектуалы и лидеры, признанные в обществе.

Из книги вы узнаете о подходе глубокого обучения с подкреплением, связанном с созданием компьютерных программ, способных достигать целей, требующих интеллекта. В первой главе вы познакомитесь с основами этого подхода и получите советы о том, как извлечь из моего пособия максимум пользы.

Что такое глубокое обучение с подкреплением

Глубокое обучение с подкреплением (deep reinforcement learning, DRL) — это подход к искусственному интеллекту на основе машинного обучения, направленный на создание компьютерных программ, способных выполнять задачи, требующие интеллекта. Отличительная черта программ DRL — обучение методом проб и ошибок на основе обратной связи, которая является одновременно последовательной, оценочной и выборочной за счет использования мощной аппроксимации нелинейных функций.

Давайте разберем это определение по частям, но не слишком увлекайтесь деталями, ведь у вас впереди еще целая книга, чтобы погрузиться в глубокое обучение с подкреплением. Эта глава — введение в материал, который вы будете изучать далее. Периодически мы будем возвращаться к ней при рассмотрении подробностей в последующих главах.

Цель этой книги — дать вам полное, всестороннее понимание этого определения. После прочтения вы сможете объяснить, почему я выбрал именно эти слова и именно эти формулировки, а пока просто расслабьтесь и прочтите первую главу.

Глубокое обучение с подкреплением — это подход к искусственному интеллекту на основе машинного обучения

Искусственный интеллект (ИИ) — это раздел информатики, связанный с созданием программ, способных демонстрировать разумное поведение. Традиционно любое программное обеспечение (ПО), отображающее такие когнитивные способности, как восприятие, поиск, планирование и обучение, считается частью ИИ. Вот несколько примеров:

- страницы, возвращаемые поисковой системой;
- маршрут, прокладываемый GPS-навигатором;
- умный помощник с распознаванием голоса и синтетической речью;
- список рекомендаций на сайтах интернет-магазинов;
- функция «следуй за мной» в дронах.

Области искусственного интеллекта

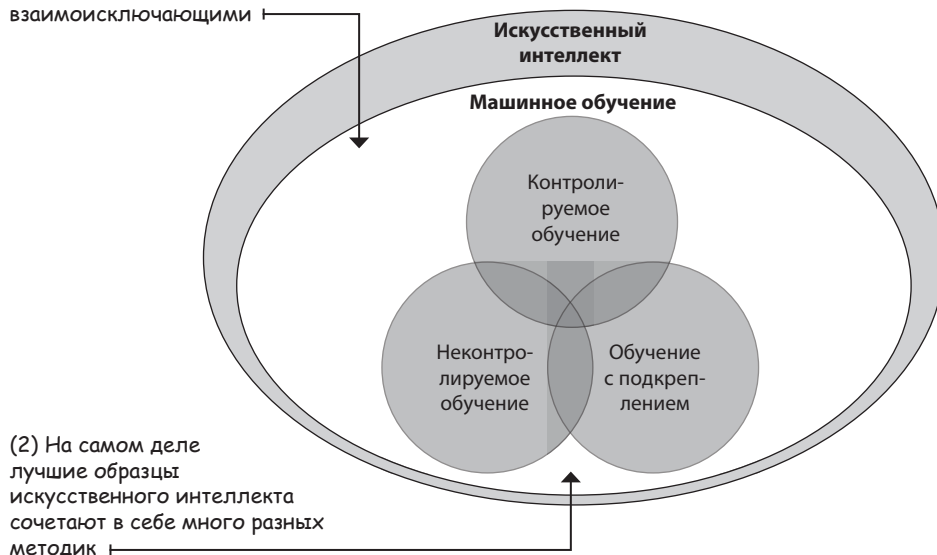


Любая программа, проявляющая интеллект, относится к ИИ, но не все примеры искусственного интеллекта могут обучаться. *Машинное обучение* — это область ИИ, посвященная созданию ПО для выполнения задач, требующих

интеллекта, через обучение на основе данных. У ML есть три основных направления: контролируемое, неконтролируемое и обучение с подкреплением.

Основные направления машинного обучения

(1) Все эти разделы важны
и не являются
взаимоисключающими



(2) На самом деле
лучшие образцы
искусственного интеллекта
сочетают в себе много разных
методик

Контролируемое обучение (supervised learning, SL) предполагает использование промаркированных данных. В процессе SL человек решает, какие данные нужно собрать и как их пометить. Цель этого направления ML — обобщение. Классический пример — приложение для распознавания цифр, написанных от руки: человек собирает изображения с рукописными цифрами и учит модель правильно распознавать и категоризировать эти цифры. Ожидается, что обученная модель сможет обобщать и категоризировать новые изображения с такими цифрами.

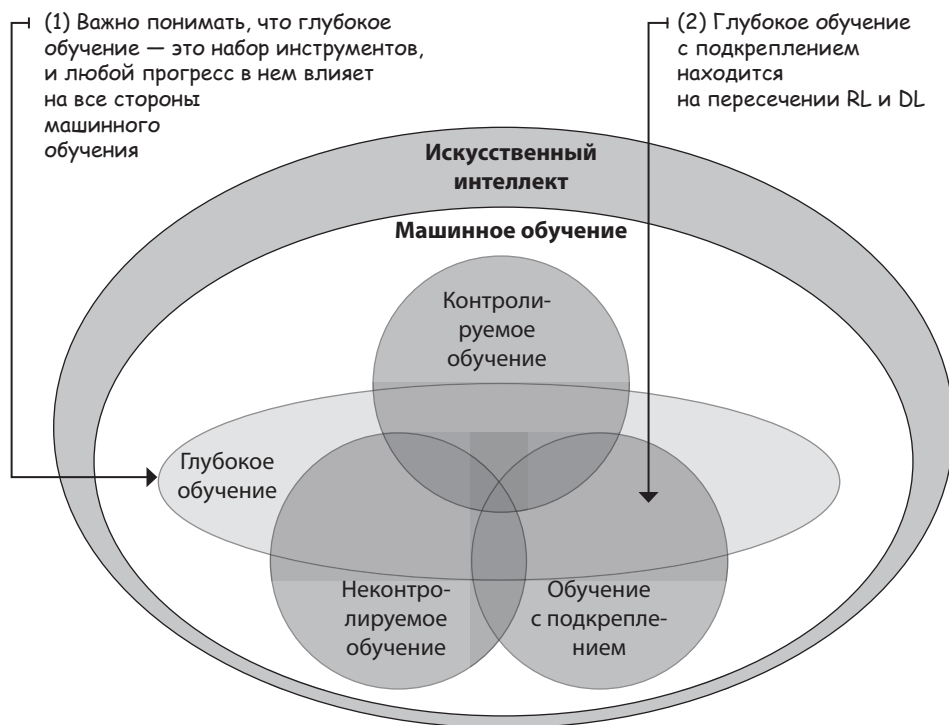
Неконтролируемое обучение (unsupervised learning, UL) подразумевает использование непромаркированных данных. Несмотря на то что данные больше не нуждаются в метках, методы по их сбору, которые использует компьютер, все еще должны разрабатываться человеком. Цель UL — сжатие. Классический пример — приложение для сегментации клиентов: человек собирает сведения о клиентах и учит модель объединять их в кластеры, которые сжимают информацию, раскрывая базовые закономерности.

Обучение с подкреплением проводится методом проб и ошибок. В задачах этого типа человек не маркирует данные, не собирает их и не участвует в разработке механизма их сбора. Цель RL — действие. Классический пример — агент

для игры в Pong, который взаимодействует с эмулятором аркадного автомата Pong и учится, выполняя действия и наблюдая за их последствиями. Обученный агент должен уметь действовать таким образом, который позволит ему успешно играть в Pong.

Относительно новый действенный подход к ML, *глубокое обучение*, включает использование многоуровневой аппроксимации нелинейных функций, обычно в виде нейронных сетей. DL не самостоятельный раздел ML, поэтому принципиально не отличается от описанных выше методов. Это набор техник и методов использования нейронных сетей для выполнения задач ML: будь то SL, UL или RL. DRL — это всего лишь подход к решению задач RL с использованием DL.

Глубокое обучение — это мощный инструмент



Суть в том, что DRL — это подход к решению задачи. Область ИИ определяет следующую задачу: создание разумных машин. Один из способов ее решения — DRL. На страницах книги вы встретите сравнения между RL и другими видами машинного обучения, но в этой главе приводятся только определения и исторический экскурс в ИИ в целом. Важно отметить, что область DRL — это часть RL, поэтому при упоминании RL я имею в виду и DRL. Но при необходимости я провожу между ними различие.

Глубокое обучение с подкреплением предназначено для создания компьютерных программ

По сути, в DRL мы занимаемся сложными последовательными задачами принятия решений в условиях неопределенности. Но эта тема исследуется и в других областях. Например, *теория управления* (control theory, CT) изучает пути управления сложными известными динамическими системами. В CT динамика систем, которыми мы пытаемся управлять, известна заранее. *Исследование операций* (operations research, OR) тоже посвящено принятию решений в условиях неопределенности. Но задачи в этой области обычно гораздо шире, чем в DRL. *Психология* изучает человеческое поведение. Это отчасти та же «сложная последовательная задача о принятии решений в условиях неопределенности».

Синергия между схожими направлениями

(1) Все эти направления (как и многие другие) изучают сложное последовательное принятие решений в условиях неопределенности

(2) В результате между ними возникает синергия. Например, обучение с подкреплением и оптимальное управление способствуют исследованию модельно-ориентированных методов

(3) Точно так же обучение с подкреплением и исследование операций способствуют изучению обширных задач

(4) Недостаток этого подхода в неоднородности обозначений, определений и т. д., из-за чего новичкам сложно сориентироваться



Подытожим: вы имеете дело с областью, на развитие которой влияет много других направлений. Это здорово, но возможна несогласованность терминологий, обозначений и т. д. Я предпочитаю подходить к этой проблеме в кон-

тексте компьютерных наук, поэтому посвятил свою книгу созданию программ, решающих сложные задачи принятия решений в условиях неопределенности. В связи с этим на ее страницах встречаются примеры кода.

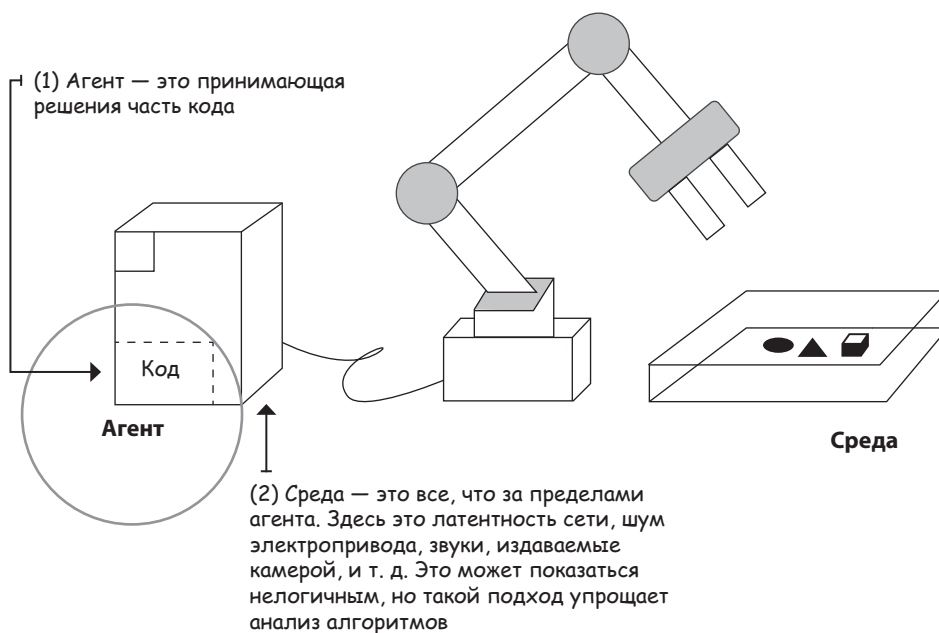
В DRL такие программы называются *агентами*. Они занимаются только принятием решений. Это значит, что рука робота, обученного поднимать объекты, не входит в состав агента. Агент — это только код, принимающий решения.

Агенты глубокого обучения с подкреплением могут выполнять задачи, требующие наличия интеллекта

Противоположность агента — *среда* — все, что находится за пределами агента и вне его полного контроля. Давайте снова представим себе робота, которого вы учите поднимать предметы. Объекты, которые нужно поднять, поднос, на котором они лежат, ветер и все остальное, что не принадлежит лицу, принимающему решения, — это часть среды. Значит, рука робота тоже относится к среде, так как не входит в состав агента. Агент может принять решение о движении рукой, но само движение создает шум, поэтому рука — это часть среды.

Поначалу это четкое разделение между агентом и средой может показаться нелогичным, но агенту отводится лишь одна роль: принятие решений. Все, что происходит после, относится к среде.

Граница между агентом и средой

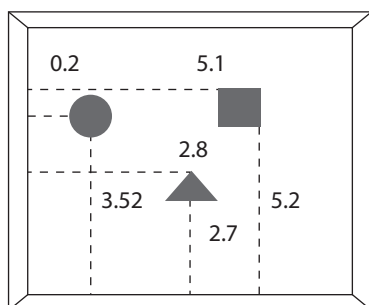


Ниже приведен общий обзор всех компонентов DRL. Подробнее о них вы узнаете в главе 2.

Среда — это набор переменных, относящихся к задаче. Например, в случае с ранее упомянутым роботом в ее состав входят такие переменные, как местоположение, скорость и направление движения руки. Переменные и все их возможные значения называют *пространством состояний*, где состояние — это отдельный экземпляр, набор значений, которые принимают переменные.

Интересно, что у агентов обычно нет доступа ко всему состоянию среды. Та часть состояния, которую агент может наблюдать, называется *наблюдением*. Наблюдения зависят от состояний, но представляют собой то, что может видеть агент. Например, в случае с роботизированной рукой агент может иметь доступ только к изображениям из камеры. У каждого объекта есть определенное местоположение, но агенту ничего не известно об этой конкретной части состояния. Вместо этого он воспринимает основанные на состояниях наблюдения. В научной литературе, в том числе и в этой книге, наблюдения и состояния часто используются как синонимы. Заранее прошу прощения за такую непоследовательность. Просто помните о различиях и делайте поправку на лексику — это главное.

Состояния и наблюдения

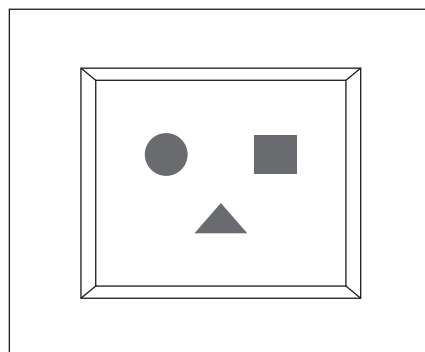


Состояние:
реальное местоположение

(1) Состояния — это точная и полная информация о решаемой задаче

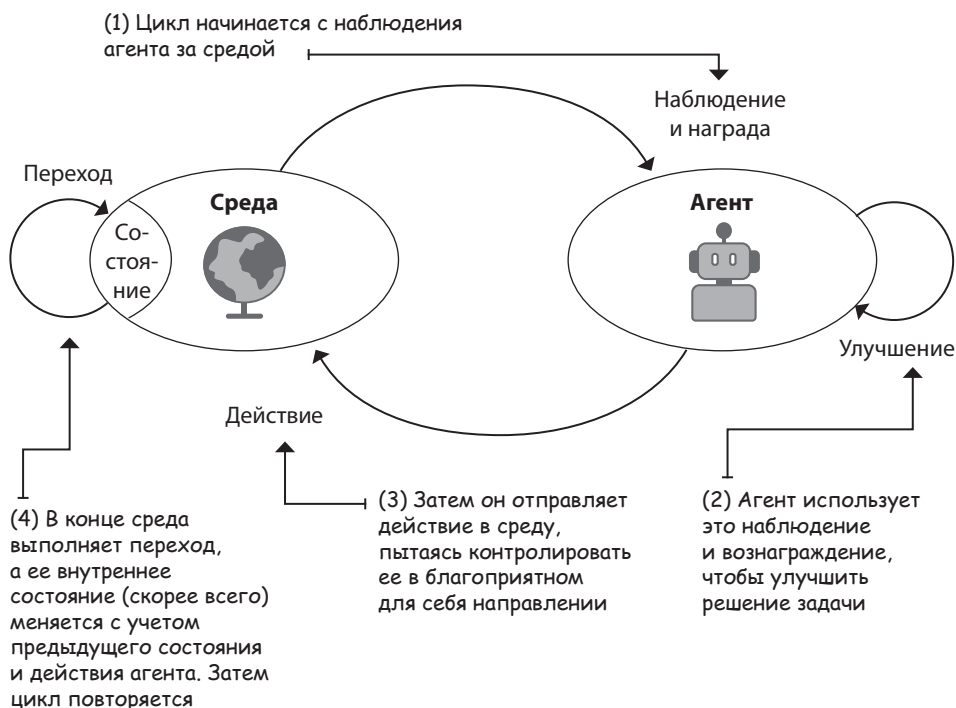
Наблюдение:
просто изображение

(2) Наблюдение — это информация, которую получает агент. Она может быть искаженной или неполной



В каждом состоянии среда предоставляет агенту набор действий, выполнив которые он может повлиять на нее. В ответ на действия агента среда может менять состояния. За эту связь отвечает *функция перехода*. Среда может возвращать и сигнал вознаграждения. За эту связь отвечает *функция вознаграждения*. Совокупность этих двух функций называется *моделью среды*.

Цикл обучения с подкреплением



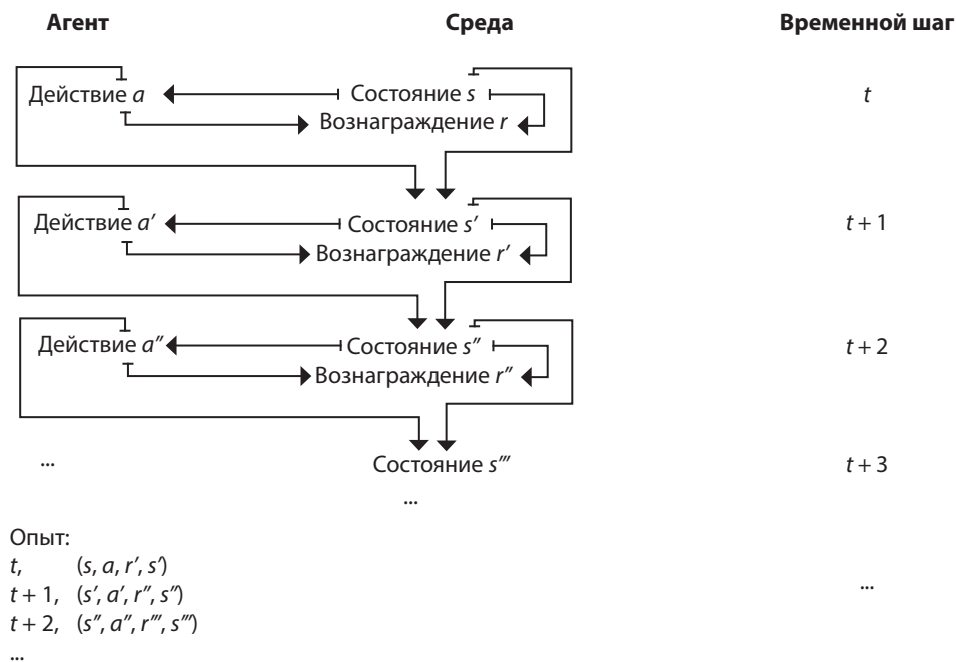
Обычно у среды есть четко определенная задача, которая формируется через функцию вознаграждения. Сигналы этой функции могут быть одновременно последовательными, оценочными и выборочными. Для достижения поставленной цели агент должен продемонстрировать интеллект или хотя бы когнитивные способности, связанные с ним: долгосрочное мышление, сбор информации и обобщение.

Работа агента проходит в три этапа: взаимодействие со средой, оценка ее поведения и улучшение ответов. Агент может быть предназначен для изучения связей между наблюдениями и действиями (такие связи называют *правилами* или *стратегиями*) или для изучения влияния *модели* среды. А с помощью *функций ценности* его можно научить оценивать и будущее вознаграждение.

Агенты глубокого обучения с подкреплением улучшают свое поведение методом проб и ошибок

Взаимодействие между агентом и средой продолжается несколько циклов — *временных шагов*. На каждом временном шаге агент наблюдает за средой, выполняет действие и получает новое наблюдение и награду. Совокупность состояния, действия, награды и нового состояния называется *опытом*. Любой опыт открывает возможность для обучения и улучшения производительности.

Кортежи опыта

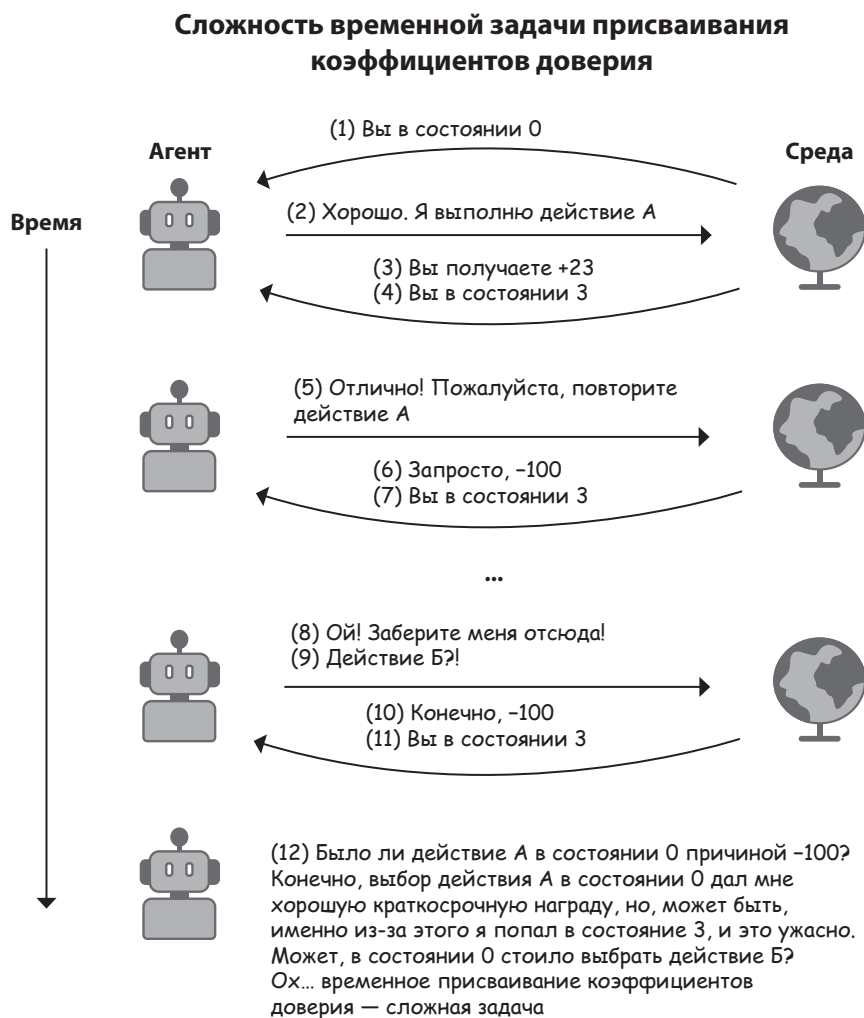


Задача, которую пытается выполнить агент, может иметь естественное завершение или не иметь его. Задачи с естественным завершением, такие как игры, называются *эпизодическими*, а без него, такие как обучение движению вперед, — *непрерывными*. Последовательность временных шагов от начала до завершения эпизодической задачи называется *эпизодом*. Чтобы научиться выполнять задачу, агенту может понадобиться несколько временных шагов и эпизодов. Агенты обучаются методом проб и ошибок: они пытаются что-то сделать, наблюдают результат, делают вывод, пробуют что-то другое и т. д.

Этот цикл мы подробно рассмотрим в главе 4 на примере среды с одношаговыми эпизодами. Начиная с главы 5, вы будете иметь дело со средами, которые требуют больше одного цикла взаимодействия в каждом эпизоде.

Агенты глубокого обучения с подкреплением учатся на последовательной обратной связи

У выполняемого агентом действия могут быть отложенные последствия. Вознаграждение может быть скудным и проявляться только через несколько временных шагов. Поэтому агент должен уметь учиться на последовательной обратной связи. Она лежит в основе так называемой *временной задачи присваивания коэффициентов доверия* — в определении того, какое состояние и/или действие привело к получению вознаграждения. Когда у задачи есть временная составляющая, а у действия — отложенные последствия, наградам сложно присвоить коэффициенты доверия.

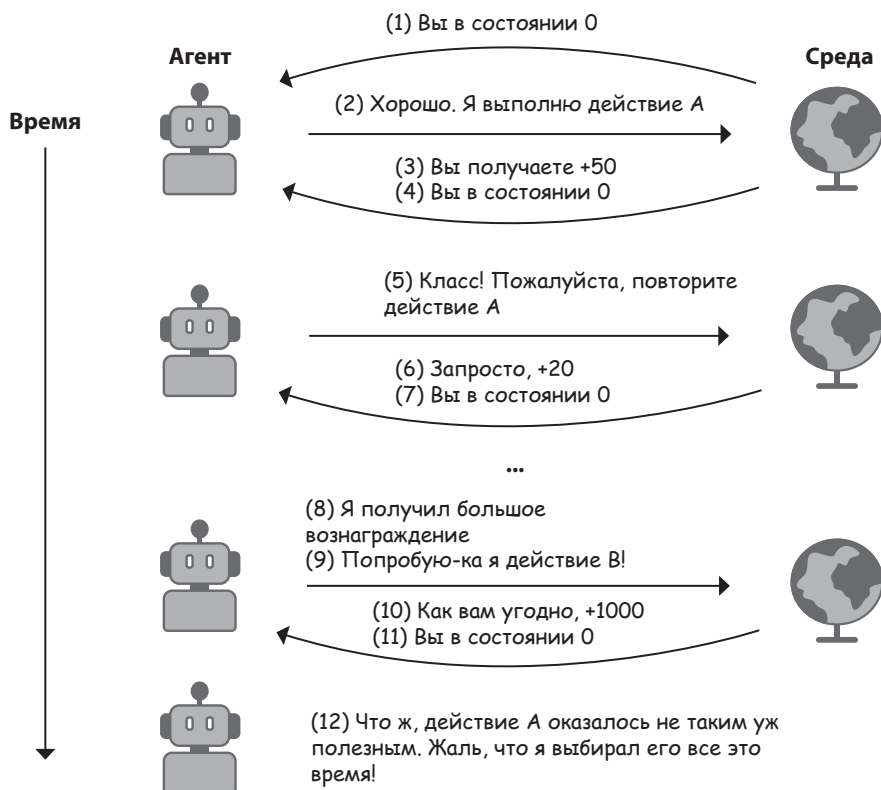


В главе 3 мы отдельно изучим все тонкости последовательной обратной связи. То есть ваши программы будут учиться на одновременно последовательной, контролируемой (в отличие от оценочной) и исчерпывающей (в отличие от выборочной) обратной связи.

Агенты глубокого обучения с подкреплением учатся на оценочной обратной связи

Агенту может быть недостаточно полученного вознаграждения — оно может не повлиять на процесс обучения. Награда может указывать на качество, а не на корректность: то есть она может не нести в себе информации о других потенциальных вознаграждениях. В связи с этим агент должен быть способен учиться на *оценочной обратной связи*. Такая обратная связь порождает потребность в исследовании. Агент должен уметь находить баланс между сбором новой информации и использованием уже имеющейся. Это называют *компромиссом между разведкой и эксплуатацией*.

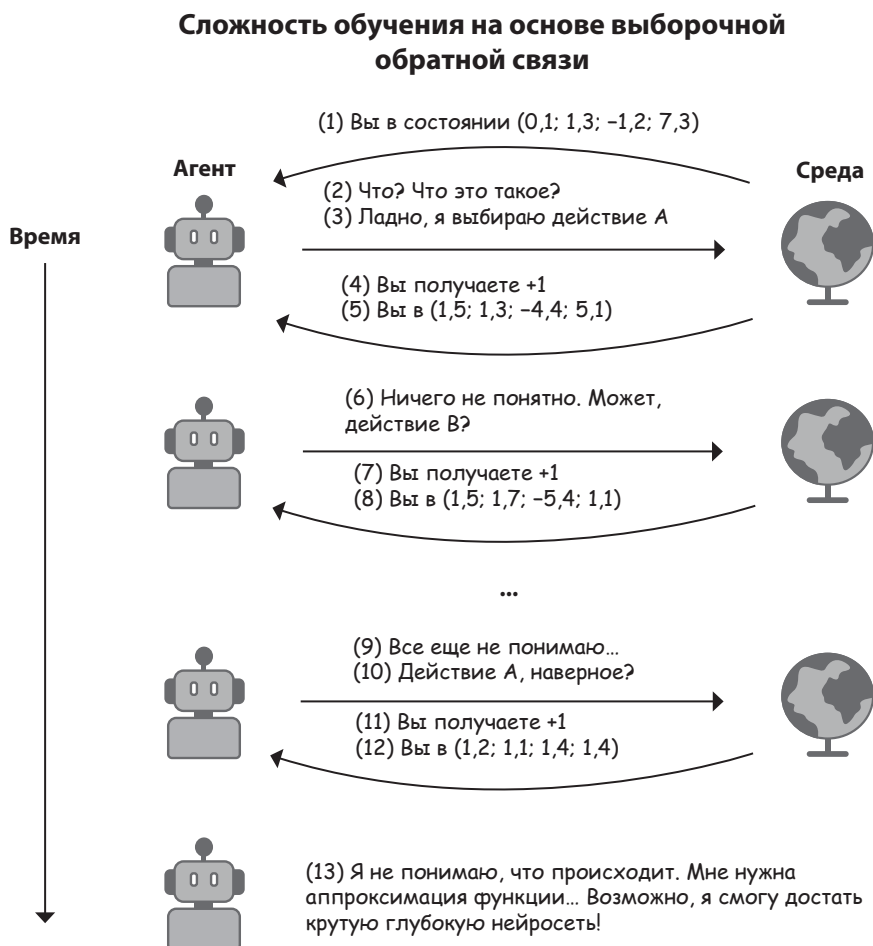
Сложность поиска компромисса между исследованием и эксплуатацией



В главе 4 мы отдельно изучим все тонкости оценочной обратной связи. То есть ваши программы будут обучаться на одновременно одинарной (в отличие от последовательной), оценочной и исчерпывающей (в отличие от выборочной) обратной связи.

Агенты глубокого обучения с подкреплением учатся на выборочной обратной связи

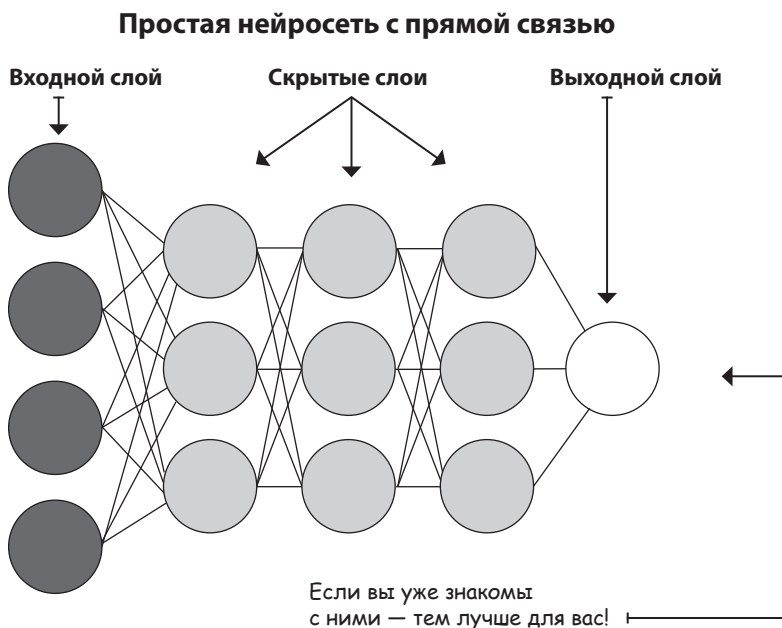
Получаемая агентом награда — просто образец. В действительности у агента нет доступа к функции вознаграждения. К тому же состояние и пространство действий обычно довольно большие или даже бесконечные, что затрудняет обучение с использованием рассеянной и слабой обратной связи. Поэтому агент должен быть способен обобщать и учиться на выборочной обратной связи.



Агенты для аппроксимации правил называются *ориентированными на правила*, для аппроксимации функций ценности — *ценностно ориентированными*, для аппроксимации моделей — *модельно-ориентированными*, а агенты для аппроксимации и правил, и функций ценности называются «*актеры-критики*». Агенты могут предназначаться для аппроксимации одного из этих компонентов или сразу нескольких.

Агенты глубокого обучения с подкреплением используют мощную аппроксимацию нелинейных функций

Агент может аппроксимировать функции с помощью разных методов и подходов, от деревьев принятия решений до SVM и нейросетей. Но в этой книге мы ограничимся только последними. В конце концов, именно они делают RL глубоким. Такое решение подходит не для всех задач: нейросети требовательны к данным и сложны для интерпретации — помните об этом. Но на сегодня это один из самых действенных способов аппроксимации функций, который часто показывает непревзойденную производительность.



Искусственная нейронная сеть (ИНС) — это многоуровневый аппроксиматор нелинейных функций, отдаленно напоминающий биологические нейросети в мозге животного. ИНС — это не алгоритм, а структура, состоящая из нескольких слоев математических преобразований, применяемых к входным значениям.

Главы 3–7 посвящены только задачам, в которых агенты обучаются на исчерпывающей (а не выборочной) обратной связи. В главе 8 мы впервые рассмо-

трим полную задачу DRL: использование нейросетей для обучения агента на выборочной обратной связи. Помните, что связь, на которой обучаются агенты DRL, одновременно последовательная, оценочная и выборочная.

Прошлое, настоящее и будущее глубокого обучения с подкреплением

Для приобретения навыков не обязательно углубляться в историю, но, зная ее, вы сможете лучше вникнуть в контекст изучаемой темы. Это может повысить вашу мотивацию и улучшить ваши навыки. Ознакомившись с историей ИИ и DRL, вы поймете, чего можно ожидать от этой перспективной технологии в будущем. Иногда мне кажется, что такое количество внимания ИИ идет только на пользу, привлекая людей. Но, когда пора приниматься за работу, ажиотаж утихает, и это проблема. Я не против того, чтобы люди восторгались ИИ, но мне хочется, чтобы их ожидания были реалистичными.

Новейшая история искусственного интеллекта и глубокого обучения с подкреплением

История DRL началась очень давно. Еще в древности люди задумывались о возможности существования разумных созданий, помимо людей. Но отправной точкой можно считать работы Алана Тьюринга (Alan Turing) в 1930–1950 годах, проложившие путь к современной информатике и ИИ и послужившие основой для последующих научных изысканий в этой области.

Самый известный пример его трудов — тест, который предлагает стандартный подход к оценке компьютерного интеллекта: если в ходе сеанса вопросов/ответов наблюдателю не удастся отличить компьютер от человека, первый считается разумным. Несмотря на свою примитивность, тест Тьюринга позволил целым поколениям размышлять о возможности создания разумных машин, определив цель, на которую могут ориентироваться исследователи.

Формальное начало ИИ как академической дисциплины можно отнести к Джону Маккарти (John McCarthy), влиятельному исследователю ИИ, который внес заметный вклад в эту область. В 1955 году Маккарти впервые предложил термин «искусственный интеллект», в 1956-м — возглавил конференцию по ИИ, в 1958-м — изобрел язык программирования Lisp, а в 1959-м — стал соучредителем лаборатории MIT, которая занимается исследованием ИИ. Несколько десятилетий он публиковал важные научные работы, способствовавшие развитию ИИ как области научных исследований.

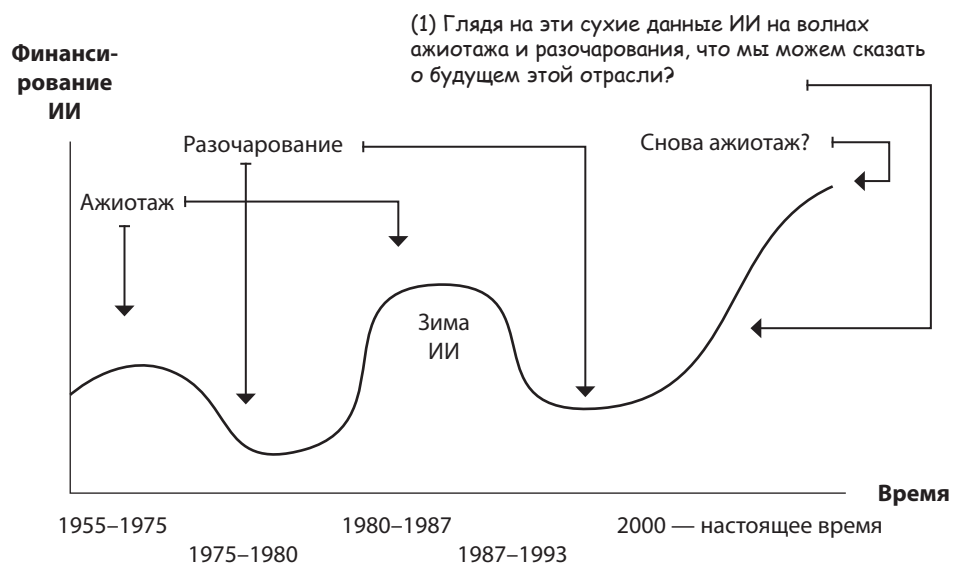
Зимы искусственного интеллекта

Вся та работа и прогресс, которые наблюдались на ранних этапах развития ИИ, вызывали большой интерес, но не обошлось и без серьезных неудач. Известные исследователи высказывались о том, что человекоподобный компьютерный

интеллект появится в течение нескольких лет, но этого так и не произошло. Ситуация усугубилась, когда знаменитый ученый Джеймс Лайтхилл (James Lighthill) составил отчет, в котором раскритиковал положение дел в академическом исследовании ИИ. Все это привело к началу длинного периода, на протяжении которого исследования в этой области испытывали сокращение финансирования и общественного интереса, — к *зиме искусственного интеллекта*.

На протяжении многих лет в области изучения ИИ сохранялась такая картина: исследователи добивались успехов, что порождало чрезмерный оптимизм и завышенные ожидания со стороны общества и в итоге приводило к сокращению финансирования от правительства и отраслевых партнеров.

Модель финансирования ИИ на протяжении многих лет



Текущее положение дел в сфере искусственного интеллекта

Скорее всего, мы переживаем еще один весьма оптимистичный период в истории ИИ, поэтому должны сохранять бдительность. Те, кто применяет ИИ на практике, понимают, что это мощный инструмент, но некоторые люди считают ИИ волшебным ящиком, который может принять любую задачу и выдать лучшее возможное решение. Это не так. Кто-то даже выражает реальные опасения по поводу того, что у ИИ может пробудиться сознание. Вот что сказал Эдсгер В. Дейкстра (Edsger W. Dijkstra) по этому поводу: «Вопрос о том, может ли компьютер мыслить, ничуть не интересней вопроса о том, может ли подводная лодка плавать».

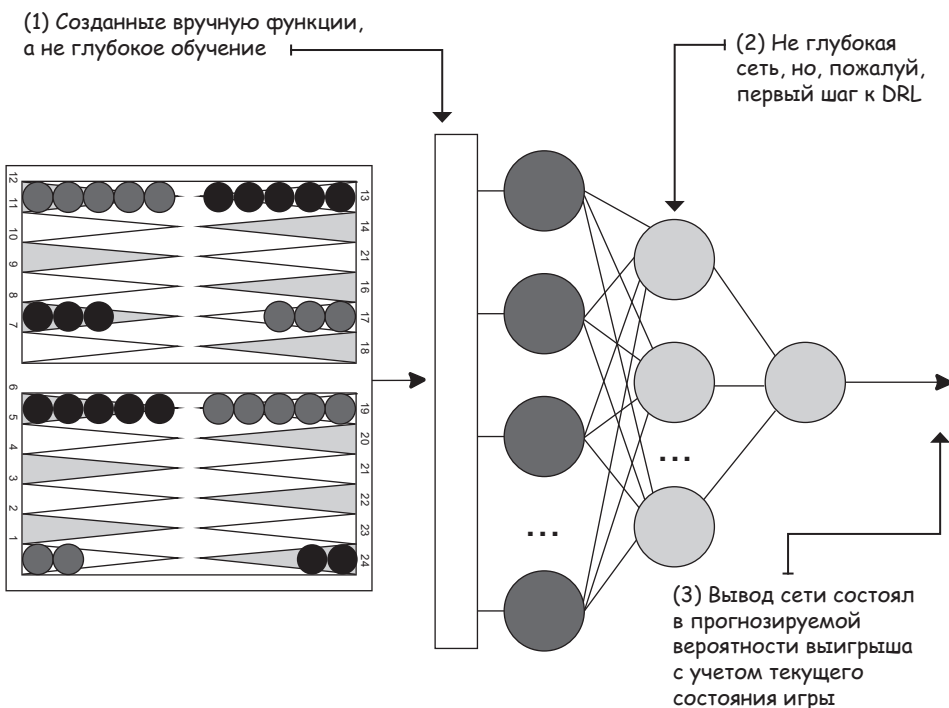
Но если не брать во внимание эту «голливудскую» версию ИИ, последний прогресс в этой области вселяет оптимизм. На сегодня самые влиятельные

компании в мире делают крупные инвестиции в исследование искусственного интеллекта. Google, Facebook, Microsoft, Amazon и Apple вкладывают много средств в эту область и отчасти обязаны ей своей высокой прибыльностью. Существенные и стабильные инвестиции создали идеальные условия для текущих темпов исследования ИИ. Ученые имеют доступ к самым мощным компьютерам и огромным объемам данных. Команды ведущих исследователей работают совместно над одними и теми же задачами. Область ИИ стала более стабильной и продуктивной. Мы наблюдаем один успех за другим, и в ближайшем будущем эта тенденция лишь усилится.

Прогресс глубокого обучения с подкреплением

Искусственные нейросети начали применять для выполнения задач RL в 1990-х. Классический пример — программа для игры в нарды, TD-Gammon, созданная Джеральдом Тезауро (Gerald Tesauro) и др. Чтобы освоить нарды, она сначала научилась самостоятельно оценивать позиции на доске с помощью RL. И хотя методики, реализованные в TD-Gammon, — это не совсем DRL, программа стала одним из первых случаев успешного применения ИНС для решения сложных задач RL.

Структура TD-Gammon



В 2004 году Эндрю Ын (Andrew Ng) и др. разработали автономный вертолет, который самостоятельно обучался трюкам высшего пилотажа, часами наблюдая за полетами опытных пилотов. При разработке была применена методика *обратного обучения с подкреплением*, при которой агент учится на демонстрациях специалистов. В том же году Нейт Кохл (Nate Kohl) и Питер Стоун (Peter Stone) применили категорию методов DRL, известную как *градиент политик*, чтобы создать робота, который играет в футбол, для турнира RoboCup. Для обучения агента движению вперед ученые использовали RL. Спустя всего три часа этот робот научился двигаться вперед быстрее, чем любой другой с той же аппаратной начинкой.

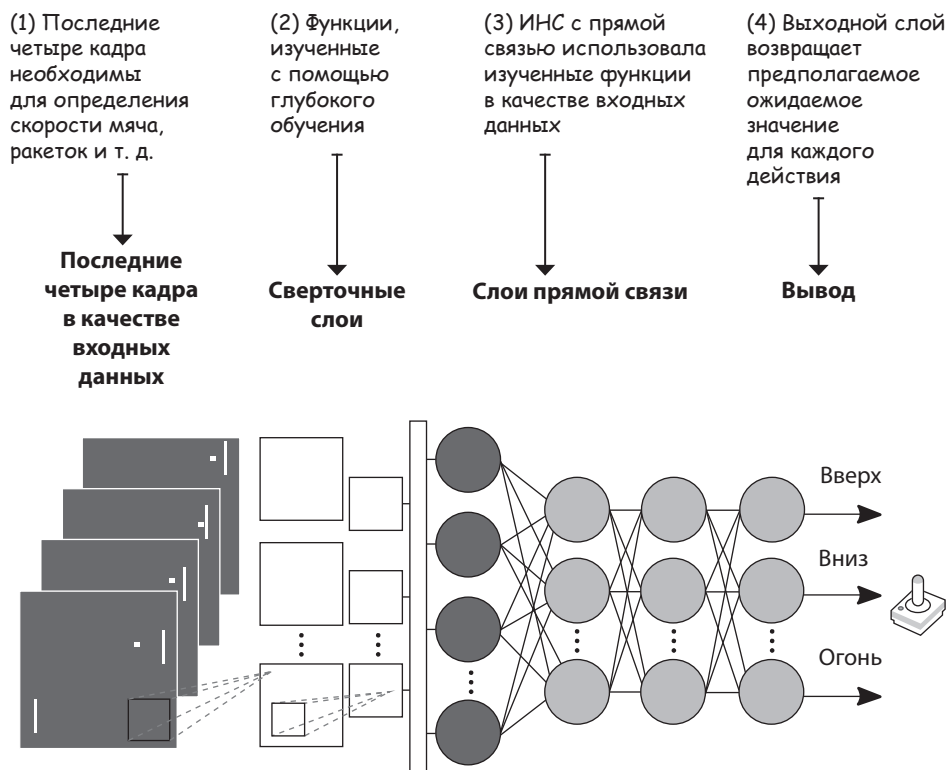
В 2000-х были и другие успехи, но по-настоящему область DRL начала развиваться только с 2010 года, когда произошел всплеск популярности глубокого обучения. В 2013 и 2015 годах Мних (Mnih) и др. опубликовали несколько научных работ с описанием алгоритма DQN (Deep Q-Learning), который учился играть в приставку Atari по одним лишь пикселям на экране. Используя сверточную нейросеть (convolutional neural network, CNN) и единый набор гиперпараметров, алгоритм DQN превзошел профессиональных игроков в 22 играх из 49.

Это достижение положило начало революции в сообществе DRL: в 2014 году Сильвер (Silver) и др. выпустили алгоритм градиента по детерминированным политикам (deterministic policy gradient, DPG), а уже через год Лилликрап (Lillicrap) и др. представили его улучшенную, глубокую версию (deep deterministic policy gradient, DDPG). В 2016 году Шульман (Schulman) и др. предложили методы оптимизации стратегий в доверительной области (trust region policy optimization, TRPO) и обобщенной оценки преимущества (generalized advantage estimation, GAE). Сергей Левин (Sergey Levine) и др. опубликовали управляемый поиск политик¹ (Guided Policy Search, GPS), а Сильвер и др. показали AlphaGo в этом же году и AlphaZero в следующем. В этот период появилось много других алгоритмов: двойные глубокие Q-сети (double deep Q-networks, DDQN), приоритетное воспроизведение опыта (prioritized experience replay, PER), оптимизация проксимальной политики (proximal policy optimization, PPO), «актер-критик» с воспроизведением опыта (actor-critic with experience replay, ACER), асинхронное преимущество «актер-критик» (asynchronous advantage actor-critic, A3C), преимущество «актер-критик» (advantage actor-critic, A2C), «актер-критик» с использованием области доверия с коэффициентом Кронекепа (actor-critic using Kronecker-factored trust region, ACKTR), Rainbow («Радуга»), Unicorn («Единогор»)

¹ В книге встречаются термины policy и strategy. Чтобы не возникало путаницы, первый обозначен как политика, второй — стратегия. — *Примеч. пер.*

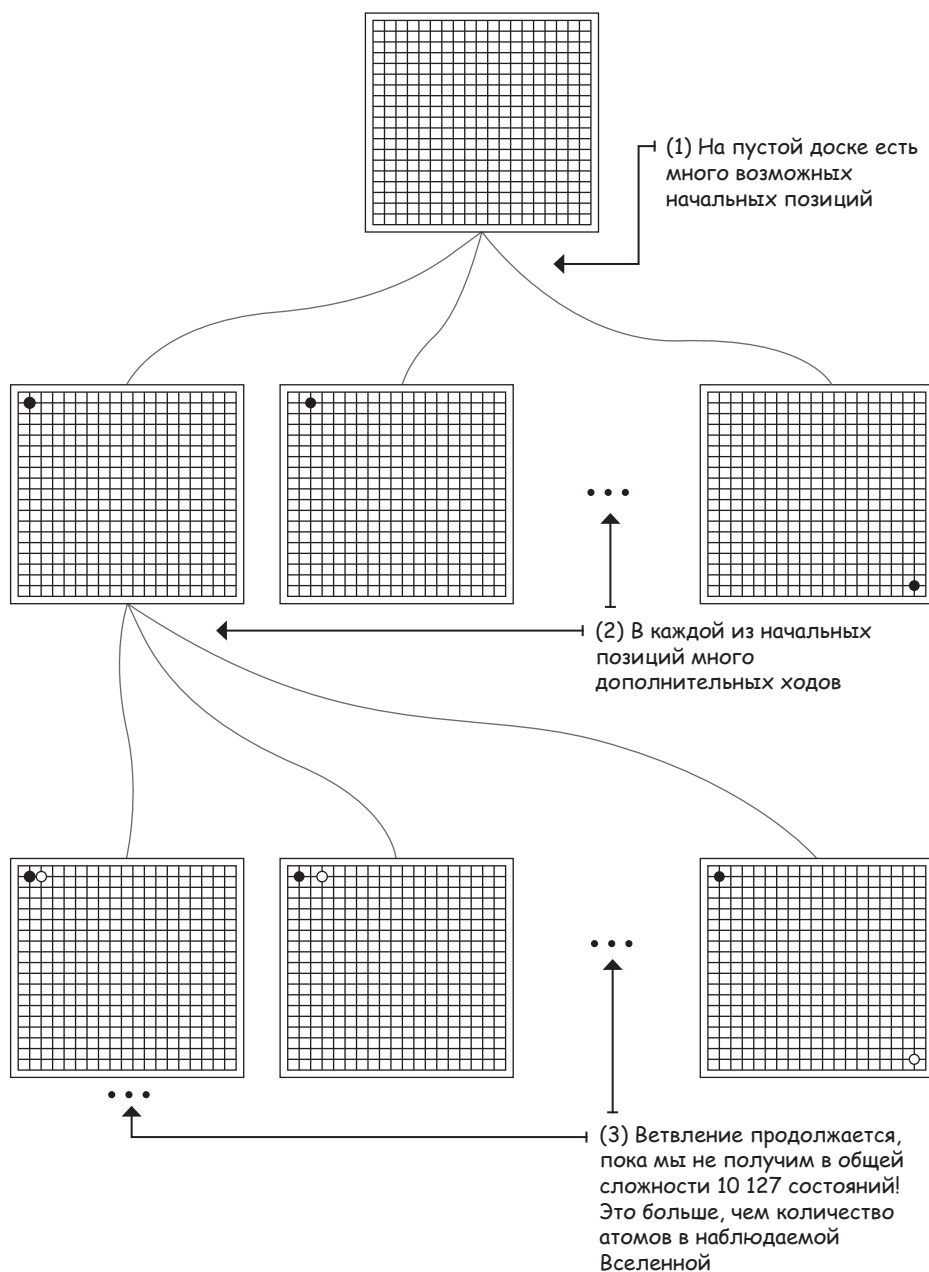
(это, кстати, настоящие названия) и т. д. В 2019 году Ориол Виньялс (Oriol Vinyals) и др. показали агент AlphaStar, способный обыгрывать профессиональных игроков в StarCraft II. Спустя несколько месяцев Якуб Пахоцки (Jakub Pachocki) и др. наблюдали за тем, как их команда ботов для игры в Dota 2 под названием Five стала первым ИИ, победившим чемпионов мира по киберспорту.

Структура сети Atari DQN



Благодаря прогрессу DRL за последние два десятилетия мы прошли путь от нард с 10^{20} состояниями полной информации до игры го или, что еще лучше, StarCraft II, в которых таких состояний 10^{170} и 10^{270} соответственно. Сложно представить более удачный момент для того, чтобы начать знакомство с этой областью. Подумайте только, что может произойти за следующие 20 лет! Хотите быть причастны к этому? Область DRL сейчас на подъеме, и я ожидаю, что она продолжит бурно развиваться.

Игра го: огромный коэффициент ветвления



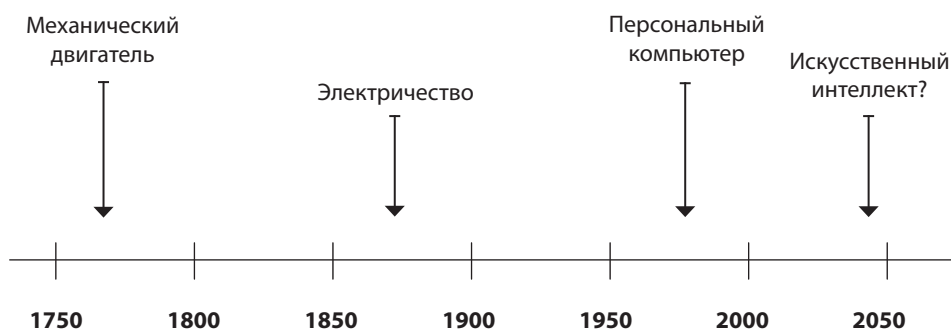
Перспективы

Я считаю, что ИИ — это область с неограниченным потенциалом для позитивных изменений, независимо от того, что говорят паникеры. Начало промышленной революции в 1750-х вызвало настоящий хаос: мощные механизмы приходили на смену однообразному физическому труду, безжалостно вытесняя людей с их рабочих мест. Это не могло не волновать: машины работали быстрее, эффективней и экономили средства. «Они лишат нас работы. Как мы теперь будем зарабатывать на жизнь?» Все опасения оправдались. Но факт в том, что многие из этих видов работ были не только изнурительными, но и опасными.

Спустя век после промышленной революции стало очевидно, что долгосрочные последствия этих изменений пошли обществу на пользу. Люди, у которых раньше было лишь несколько рубашек и пара штанов, могли купить намного больше вещей по более низким ценам. Да, изменения давались тяжело, но в конечном счете они сделали мир лучше.

Появление персональных компьютеров в 1970-х ознаменовало начало цифровой революции. Затем наши жизни изменил Интернет. Благодаря ему мы получили большие данные (big data) и облачные вычисления. На этой благодатной почве возросло машинное обучение в его нынешнем виде. Возможно, в следующие пару десятилетий будет сложно принять изменения, которые повлечет за собой ИИ, и его влияние на общество, но долгосрочные последствия легко перевесят любые препятствия, возникающие на этом пути. Я ожидаю, что в обозримом будущем людям даже не придется работать, чтобы себя прокормить, одеться или иметь крышу над головой, так как все эти вещи будут автоматически производиться искусственным интеллектом. Мы будем жить в изобилии.

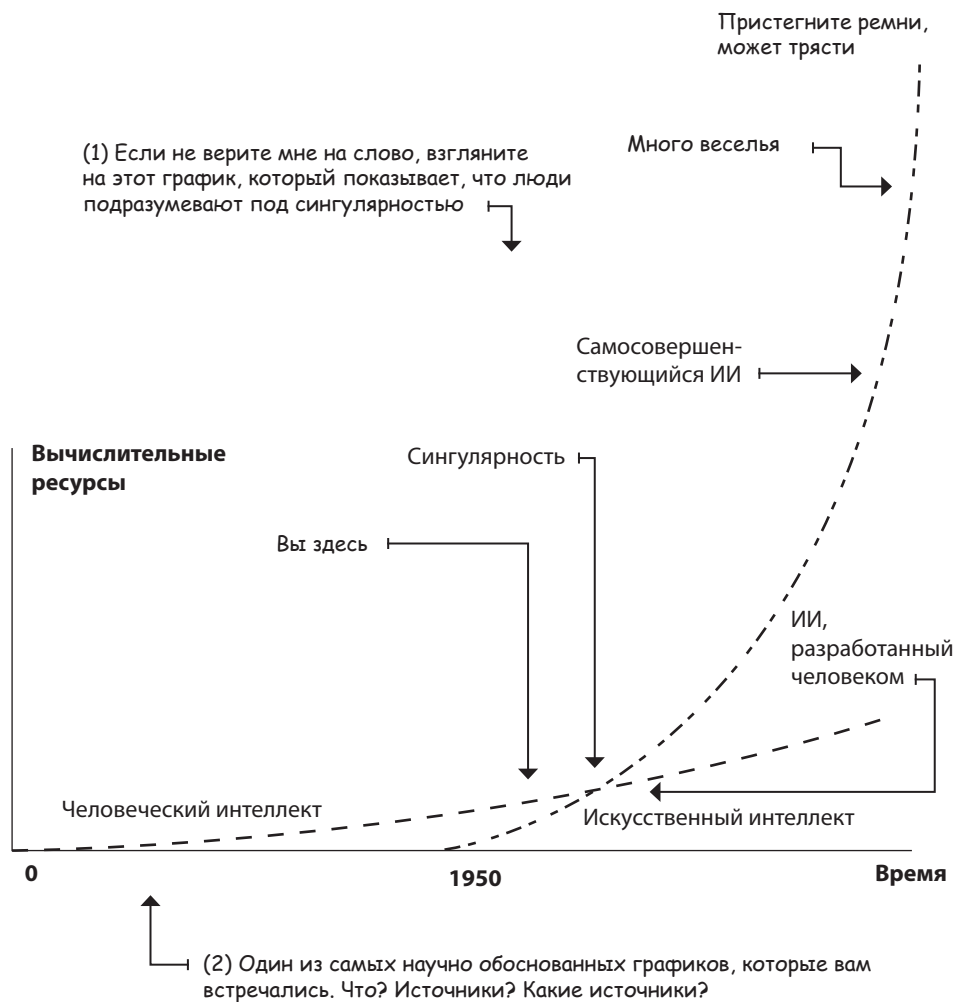
Производственные революции



↑ (1) Революции неизбежно нарушают баланс, сложившийся в промышленности и обществе. Но в долгосрочной перспективе они приносят изобилие и прогресс

Поскольку мы продолжаем повышать уровень машинного интеллекта, некоторые исследователи считают, что это может привести к появлению ИИ, превосходящего человеческий. На этом этапе мы можем столкнуться с таким явлением, как *сингулярность*: искусственный интеллект, превосходящий человеческий, позволит совершенствовать технологию ИИ куда более быстрыми темпами, так как у цикла самосовершенствования больше не будет слабого звена — человека. Но здесь следует проявить благоразумие, так как это скорее идеал, чем практический аспект, о котором нужно переживать.

Сингулярность может возникнуть через несколько десятилетий



Конечно, мы должны осознавать последствия развития ИИ и нам нужно стремиться сделать его безопасным, но на сегодняшний день сингулярность не проблема. В этой книге вы узнаете о множестве реальных проблем, свойственных DRL в его текущем состоянии. Именно на них и следует сосредоточиться.

Целесообразность глубокого обучения с подкреплением

Любую задачу ML можно сформулировать в контексте DRL, но по многим причинам это не всегда уместно. Вы должны понимать плюсы и минусы DRL в целом и уметь определять, для каких задач и условий оно подходит, а для каких — нет.

Преимущества и недостатки

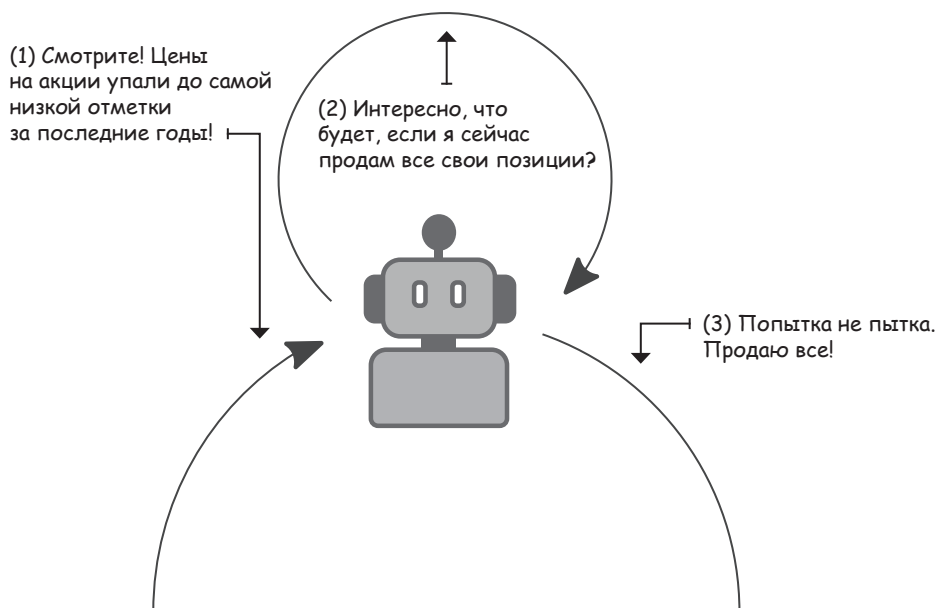
Не углубляясь в технические детали, подумайте о неизбежных плюсах и минусах использования DRL в вашем следующем проекте. Как вы потом поймете, каждый из обсуждаемых здесь аспектов может быть как положительным, так и отрицательным, в зависимости от решаемой задачи. Например, эта область сводится к тому, чтобы передать управление машине. Хорошо это или плохо? Готовы ли вы позволить компьютеру принимать решения за вас? Неспроста в качестве среды исследования DRL часто выбирают игры: обучать агентов в реальных условиях было бы дорого и опасно. Можете себе представить агента для управления автомобилем, который учится не попадать в аварии методом проб и ошибок? В DRL агенты должны ошибаться. Вы можете себе это позволить? Готовы ли вы принять риск отрицательных последствий (с реальным ущербом) для людей? Подумайте об этом, прежде чем браться за свой следующий проект на основе DRL.

Вам нужно подумать и о том, как ваш агент будет познавать среду. Например, в большинстве ценностно ориентированных методов для этого предусмотрен случайный выбор действия. Но есть и методы с более стратегическим подходом к познанию. У каждого варианта свои преимущества и недостатки, и вам часто придется идти на компромисс.

Наконец, обучение с нуля каждый раз может быть сложной задачей, занимающей много времени и ресурсов. Но есть несколько методов, которые помогут вам закрепить ранее полученные знания. Прежде всего, это *трансферное обучение* — перенос знаний из одних задач в другие. Например, чтобы научить робота пользоваться молотком и отверткой, можете взять низкоуровневые действия, освоенные в задаче «поднять молоток», и применить этот опыт для выполнения задачи «поднять отвертку». Для вас это должно звучать логично, ведь людям не приходится заново учиться простым движениям при выполнении

каждой новой задачи. Во время обучения мы формируем иерархии действий, и область *иерархического обучения с подкреплением* пытается имитировать эту черту в агентах DRL.

Агенты глубокого обучения с подкреплением познают окружающий мир. Можете ли вы позволить себе ошибки?



Сильные стороны глубокого обучения с подкреплением

Суть DRL в решении конкретных задач. В отличие от SL, цель которого — обобщение, RL хорошо себя проявляет в четко описанных условиях. Например, у каждой игры Atari есть определенная задача. Агенты DRL плохо справляются с обобщением поведения для решения разных проблем: агент, который обучается игре в Pong, не сможет играть в Breakout. И если вы попытаетесь научить его играть сразу в Pong и Breakout, он, скорее всего, будет показывать плохие результаты в обеих играх. С другой стороны, SL довольно хорошо классифицирует несколько объектов одновременно. Итак, сильная сторона DRL — решение отдельных, четко определенных задач.

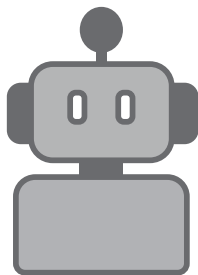
В DRL мы используем методы обобщения для приобретения простых навыков на основе сенсорного ввода. Одни из важнейших улучшений за последние годы связаны с производительностью этих методов, с новыми приемами и рекомендациями по обучению глубоких сетей и т. д. К счастью для нас, большинство достижений в DL открывают новые направления исследований в области DRL.

Слабые стороны глубокого обучения с подкреплением

Конечно, DRL нельзя назвать идеальным. Одна из главных проблем, с которыми вы столкнетесь, в том, что для формирования эффективных стратегий в большинстве задач агентам будут нужны миллионы образцов. Для сравнения: людям для обучения требуется лишь несколько взаимодействий. Оптимизация выборки — это, наверное, один из главных аспектов DRL, нуждающихся в улучшении. Это крайне важная тема, поэтому мы будем периодически к ней возвращаться.

Агентам глубокого обучения с подкреплением нужно много образцов взаимодействия!

Эпизод 2 324 532



(1) Шеф, в этот раз я почти не петлял между полосами. Позвольте мне проехаться еще на одной машине!

Еще одна проблема DRL связана с функциями вознаграждения и пониманием его значения. Если специалист определяет награды, которые агент пытается максимизировать, означает ли это, что мы в каком-то смысле «контролируем» этого агента? И насколько этот фактор положительный? Должны ли появляться вознаграждения как можно чаще, что ускоряет процесс обучения, или как можно реже, что делает решения более захватывающими и уникальными?

У нас, у людей, нет четко определенной награды. Один и тот же человек может увидеть событие как в положительном, так и в отрицательном свете, просто изменив точку зрения. Кроме того, не так-то легко разработать функцию вознаграждения для такой задачи, как ходьба. Что в этом случае важнее: двигаться вперед или не упасть? Так какая же «идеальная» функция вознаграждения для ходьбы?!

Сейчас проводятся интересные исследования в области сигналов вознаграждения. Одно из особенно интригующих направлений называется «*внутренняя мотивация*». Она позволяет агенту исследовать новые действия без какой-либо конкретной причины, просто из любопытства. Агенты с внутренней мотивацией лучше обучаются в средах с редкими наградами, что позволяет находить интересные и уникальные решения. Суть в том, что попытки выполнить задачу, которая не была смоделирована или не имеет четкой функции вознаграждения, приведут к появлению проблем.

Определение четких обоюдных ожиданий

Обсудим еще один момент, который в дальнейшем будет играть существенную роль. Чего ожидать от этой книги? Для меня это очень важно. Я хочу, чтобы вы знали, на что рассчитывать, чтобы дальше не возникало никаких сюрпризов. Мне не хочется создавать у вас иллюзию того, что вы сумеете написать агент, который вас озолотит. Если бы все было так просто, я не стал бы писать эту книгу. Хочу предупредить, что если вы хотите чему-то научиться, то будьте готовы приложить необходимые усилия. Это обоюдный процесс: я постараюсь сделать материал понятным, а вы должны потрудиться над тем, чтобы его усвоить. Если какой-то материал покажется вам лишним и вы решите его пропустить, мы с вами окажемся в проигрыше.

Чего ожидать от этой книги

С помощью книги я хочу превратить вас из любителя ML без опыта в области глубокого обучения с подкреплением в специалиста, способного разрабатывать передовые алгоритмы DRL.

В главах 3–7 вы познакомитесь с агентами, которые могут обучаться на последовательной и оценочной обратной связи, сначала изолированно, а затем в процессе взаимодействия. В главах 8–12 мы погрузимся в ключевые алгоритмы, методы и подходы к DRL. Первые две главы посвящены базовым понятиям DRL в целом, а последняя содержит заключительные замечания.

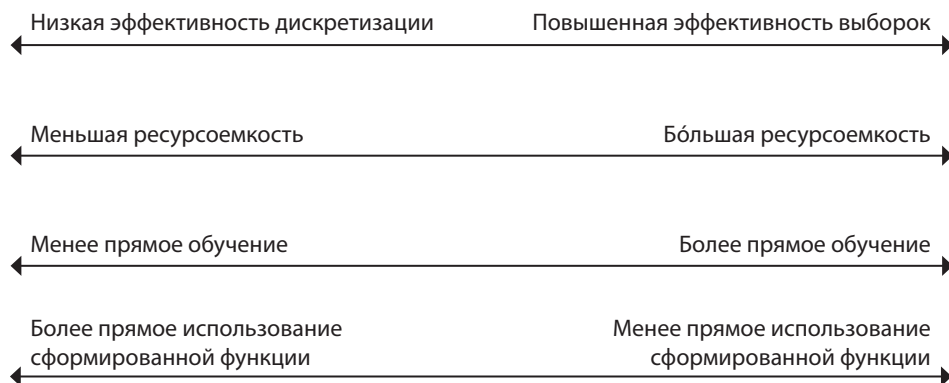
Я рассчитываю на то, что в первой половине книги (главы 3–7) вы освоите «табличное» обучение с подкреплением. То есть задачи RL, в которых можно провести исчерпывающую выборку и в которых вам не понадобятся нейросети или какая-либо аппроксимация функций. Глава 3 посвящена последовательному аспекту RL и временной задаче присваивания коэффициентов доверия. В главе 4 мы проанализируем (снова отдельно) трудности обучения на основе оценочной обратной связи и баланс между исследованием и эксплуатацией. Наконец, вы узнаете о методах, которые могут выполнить эти две задачи одновременно. В главе 5 будут представлены агенты, которые учатся оценивать результаты фиксированного поведения. В шестой главе речь пойдет об обучении, направленном на улучшение поведения, а в седьмой вы изучите методики, повышающие эффективность RL.

Вторая половина (главы 8–12) рассчитана на то, чтобы вы усвоили все тонкости ключевых алгоритмов DRL. Не сомневайтесь, мы как следует углубимся в эту тему. Вы узнаете о множестве разных типов агентов, от ориентированных на политику и ценность до методов «актер-критик». В главах 8–10 подробно рассматривается ценностно ориентированное DRL. В главе 11 вы познакомитесь с DRL, ориентированным на политику и использующим метод «актер-критик», а глава 12 посвящена градиенту по детерминированным по-

литикам (deterministic policy gradient, DPG), мягкому методу «актер-критик» (soft actor-critic, SAC) и оптимизации проксимальной политики (proximal policy optimization, PPO).

Чтобы сделать сравнение и противопоставление агентов одного типа более наглядным, некоторые примеры в этих главах будут повторяться. Вы по-прежнему будете исследовать принципиально разные типы задач, от небольших непрерывных до пространств состояний на основе изображений и от дискретных до непрерывных пространств действий. Но основное внимание в этой книге уделяется не моделированию задач, требующему отдельных навыков, а поиску решений в готовых средах.

Сравнение разных алгоритмических подходов к глубокому обучению с подкреплением



↑ (1) В книге вы познакомитесь со всеми перечисленными алгоритмическими подходами к глубокому обучению с подкреплением. На самом деле меня больше всего интересуют именно алгоритмы, а не задачи. Почему? Потому что в DRL один и тот же алгоритм можно применять к похожим задачам, изменяя гиперпараметры. Изучение алгоритмов будет гораздо полезнее для вас

Как извлечь из книги максимум пользы

К желающим освоить глубокое обучение с подкреплением есть несколько требований. Во-первых, у вас уже должно быть некоторое представление об ML и DL. Во-вторых, вы должны ориентироваться в коде на языке Python и обладать базовыми математическими знаниями. Но, что важнее всего, будьте готовы прилагать усилия.

Я строю свое повествование, основываясь на том, что у читателя уже есть базовые знания об области машинного обучения. Вы должны уметь обучать простые модели SL, возможно, с использованием наборов данных Iris или Titanic; должны быть знакомы с такими понятиями DL, как тензоры и матрицы; у вас должен быть опыт обучения как минимум одной модели DL, например сверточной нейросети (CNN) с набором данных MNIST.

Все темы в этой книге посвящены DRL, и мы не будем рассматривать глубокое обучение отдельно. Есть много полезных ресурсов, которыми вы можете воспользоваться. Но опять же для этого вам нужно знать основы. Если вы уже обучали CNN, можете не волноваться. Если же нет, то прежде, чем переходить ко второй части этой книги, я настоятельно советую выполнить несколько практических упражнений по DL.

Я также рассчитываю на то, что вы ориентируетесь в коде на Python. Это наглядный и простой для восприятия язык программирования. Даже те, кто с ним не знаком, как правило, могут что-то понять, читая написанный на нем код. У вас не должно быть проблем с пониманием Python, так как вам придется читать код на нем. Те, кто будет игнорировать эту часть, многое потеряют.

В этой книге будет много математических уравнений, и это хорошо. Математика — идеальный язык, и ничто не сможет его заменить. Но я ожидаю от вас только умения ориентироваться в уравнениях и желание их читать — ничего больше. Всем уравнениям сопутствуют подробные аннотации, чтобы даже те, кто «не дружит с математикой», могли найти для себя что-то полезное.

Наконец, я ожидаю, что вы не прочь потрудиться — в том смысле, что вы действительно хотите изучить DRL. Если вы решите пропустить математическое уравнение, листинг кода на Python, раздел, страницу, главу или еще что-то, то упустите много полезной информации. Чтобы получить максимум от этой книги, я советую вам читать ее от начала и до конца. Несмотря на разный формат, иллюстрации и врезки — часть основного повествования.

Не забывайте также выполнять представленный здесь исходный код (в следующем разделе вы узнаете, как это делать) и экспериментировать с наиболее интересными для вас примерами.

Среда разработки для глубокого обучения с подкреплением

Вместе с этой книгой вы получаете тщательно проверенную среду разработки и код для воспроизведения результатов. Я создал образ Docker и несколько блокнотов Jupyter, чтобы вам не пришлось возиться с установкой пакетов, на-

стройкой ПО или копированием кода. От вас требуется лишь наличие Docker. Следуйте инструкциям по выполнению кода на странице <https://github.com/mimoralea/gdrl>. Они довольно простые.

Код написан на Python с активным использованием NumPy и PyTorch. Я выбрал PyTorch вместо Keras и TensorFlow, так как эта библиотека кажется мне более подходящей для применяемого языка. Если вы уже работаете с NumPy, использовать PyTorch вам будет несложно, чего нельзя сказать о пакете TensorFlow, который выглядит как совершенно отдельная парадигма программирования. Я не хочу сказать, что какой-то из этих инструментов хуже. Просто, по моему опыту работы с этими двумя библиотеками, PyTorch гораздо лучше подходит для проведения исследований и обучения.

DRL — это алгоритмы, методы, приемы, трюки и т. д., поэтому переписывание библиотек NumPy или PyTorch было бы бессмысленным. Но в то же время мы будем создавать алгоритмы DRL с нуля. Я не стану учить вас использовать библиотеки DRL вроде Keras-RL, Baselines или RLlib. Я хочу, чтобы вы овладели DRL, поэтому код мы будем писать вручную. Я много лет преподаю RL и могу сказать, что у тех, кто пишет код, больше шансов на то, чтобы понять обучение с подкреплением. Но эта книга вовсе не о PyTorch: здесь вы не найдете отдельного обзора этой библиотеки. Мы будем рассматривать только код, написанный с ее использованием. Если вам знакомы некоторые концепции глубокого обучения, вы сможете понять код из этой книги. Не волнуйтесь, вам не нужно изучать PyTorch отдельно. Постепенно я буду давать вам всю необходимую информацию.

Что касается сред для обучения агентов, то мы будем использовать популярный пакет OpenAI Gym и несколько библиотек, которые я разработал для этого пособия. Но мы опять же не станем вникать во все тонкости Gym. Вам достаточно знать, что эта библиотека предоставляет среды для обучения агентов RL. Не забывайте, что нас в первую очередь интересуют алгоритмы RL (решения), а не среды или задачи моделирования, которые, конечно, тоже очень важны.

Если вы знакомы с глубоким обучением, то, скорее всего, знаете, что такое графический процессор (graphics processing unit, GPU). Архитектуры DRL не нуждаются в вычислениях на том уровне, который обычно встречается в моделях DL. Поэтому использовать GPU хоть и полезно, но в нашем случае не нужно.

С другой стороны, в отличие от моделей DL, некоторые агенты DRL сильно нагружают центральный процессор (ЦП) и состоят из множества потоков. Если вы планируете приобрести новый компьютер, обратите внимание на производительность (а точнее, на количество ядер) ЦП. Позже вы увидите, что некоторые алгоритмы активно распараллеливают вычисления, и слабым местом в этом случае будет именно центральный, а не графический процессор. Но примеры кода будут хорошо работать в контейнере независимо от аппаратной начинки вашего компьютера. Если же ваши аппаратные ресурсы

крайне ограничены, можете обратить внимание на облачные платформы. Некоторые сервисы вроде Google Colab предлагают бесплатное оборудование для глубокого обучения.

Подведем итоги

Освоить глубокое обучение с подкреплением нелегко, так как агенты должны учиться на одновременно последовательной, оценочной и выборочной обратной связи. Использование последовательной обратной связи вынуждает агента учиться находить баланс между ближайшими и долгосрочными целями, использование оценочной — учиться балансировать между сбором и использованием информации, а выборочной — обобщать старый и новый опыт.

Обучение с подкреплением относится к искусственному интеллекту — разделу информатики, посвященному созданию программ, которые проявляют интеллект, подобный человеческому. Это общая цель и для многих других дисциплин, таких как теория управления и исследование операций. Один из самых популярных и успешных подходов к искусственному интеллекту — машинное обучение, состоящее из трех основных направлений: обучения с подкреплением, контролируемого и неконтролируемого обучения. Так как глубокое обучение — это один из подходов к ML, у него нет конкретного направления, но его мощь способствует развитию всей сферы.

Глубокое обучение с подкреплением включает в себя использование нескольких слоев мощных аппроксиматоров функций, известных как нейросети, для выполнения сложных последовательных задач принятия решений в условиях неопределенности. Этот подход показывает хорошие результаты в задачах, связанных с управлением, но к принятию критически важных решений без участия человека нужно относиться серьезно. На сегодняшний день глубокое обучение с подкреплением нуждается в более безопасных алгоритмах с повышенной дискретизацией и эффективными стратегиями исследования.

Но будущее DRL довольно безоблачно. По мере взросления этой технологии ее могут поджидать опасности, но сейчас важно лишь то, что у нее большой потенциал. Ступая на этот путь, вы должны чувствовать воодушевление и желание проявить себя с лучшей стороны. Возможность внести свой вклад в потенциальные изменения такого масштаба выпадает нечасто. Вы должны быть благодарны за то, что вам посчастливилось жить в такое время. Давайте же станем частью этих перемен.

На этом этапе вы:

- понимаете, что такое глубокое обучение с подкреплением и как оно соотносится с другими подходами к машинному обучению;
- знаете о последних достижениях в области глубокого обучения с подкреплением и осознали, что его можно применить к широкому спектру задач;
- знаете, чего ожидать от книги и как извлечь из нее максимум пользы.



Не стыдно затвитить

Делитесь своими успехами

В конце каждой главы я буду предлагать несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

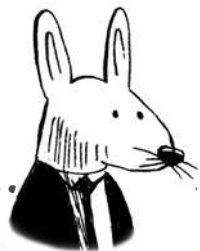
- **#gdrl_ch01_tf01.** Контролируемое, неконтролируемое обучение и обучение с подкреплением — это ключевые направления машинного обучения. Вам нужно знать не только их отличия, но и то, что их объединяет. Запишите краткий сравнительный анализ этих подходов и предложите варианты их совместного использования для выполнения задачи в области ИИ. Все они преследуют одну цель: создание сильного искусственного интеллекта. Мы должны стремиться к тому, чтобы максимально эффективно использовать доступные нам инструменты.
- **#gdrl_ch01_tf02.** Я не удивлюсь, если у вас нет опыта работы в области машинного обучения или компьютерных наук, но вы все равно заинтересованы в материале из этой книги. Для нашего сообщества очень важны исследования процесса принятия решений из других областей. Возможно, вы занимались исследованием операций, психологией, философией, неврологией, теорией управления или экономикой? Почему бы вам не составить список ресурсов, статей, видеороликов на YouTube, книг и прочих материалов и не поделиться им с другими читателями, которые тоже изучают процесс принятия решений?
- **#gdrl_ch01_tf03.** Часть текста в этой главе можно было бы представить более наглядно, с помощью диаграмм, таблиц и т. д. Например, я рассказывал о разных видах агентов обучения с подкреплением (без производных, на основе модели «актер-критик», ценностно, модельно-ориентированный, на основе политик). Попробуйте взять отрезок текста, извлечь из него суть, а после поделитесь своим кратким изложением с миром.
- **#gdrl_ch01_tf04.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>.
#gdrl_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы познакомитесь с основными составляющими обучения с подкреплением.
- ✓ Научитесь представлять задачи последовательного принятия решений в виде сред для обучения с подкреплением с помощью математической модели «марковский процесс принятия решений».
- ✓ Создадите с нуля среды, где в следующих главах агенты обучения с подкреплением будут учиться выполнять задачи.

История человечества — это борьба с враждебным миром. Мы наконец достигли того уровня, который позволяет нам господствовать в нашей среде... Понимание этого факта неизбежно приводит к сдвигу наших взглядов на математику во многих областях от описательного анализа до теории управления.

*Ричард Беллман (Richard Bellman),
американский прикладной математик,
обладатель почетной медали IEEE*

Несмотря на нехватку свободного времени, вы открыли эту книгу и решили прочитать еще одну главу. Тренер оставляет на скамье запасных своего лучшего игрока, игнорируя критику в прессе. Родитель долго и упорно работает над тем, чтобы научить своего ребенка хорошим манерам. Все это примеры сложного последовательного процесса принятия решений в условиях неопределенности.

Я хочу обратить ваше внимание на три слова из последней фразы. Первое, «*сложный*», говорит о том, что агенты могут обучаться в средах с обширными пространствами состояний и действий. Вернемся к примеру с тренером. Зная, что лучшему игроку иногда нужен отдых, вы принимаете решение отправить его на скамью запасных в матче с конкретным соперником, чтобы у него остались силы на другие, более важные игры. Мы обучаемся на выборочной обратной связи, поэтому научиться правильно обобщать не так-то просто.

Второе слово, которое я использовал, — «*последовательный*». У многих задач есть отложенные последствия. Снова вернемся к примеру с тренером. Допустим, тренер отправил своего лучшего игрока в запас для, казалось бы, неважного матча в середине сезона. Но что, если игроки, которых не выпускают на поле, теряют мотивацию и мастерство, что может отразиться на финальных матчах? Иначе говоря, что, если реальные последствия отложены? Дело в том, что назначить коэффициент доверия уже принятым решениям бывает непросто, так как мы обучаемся на последовательной обратной связи.

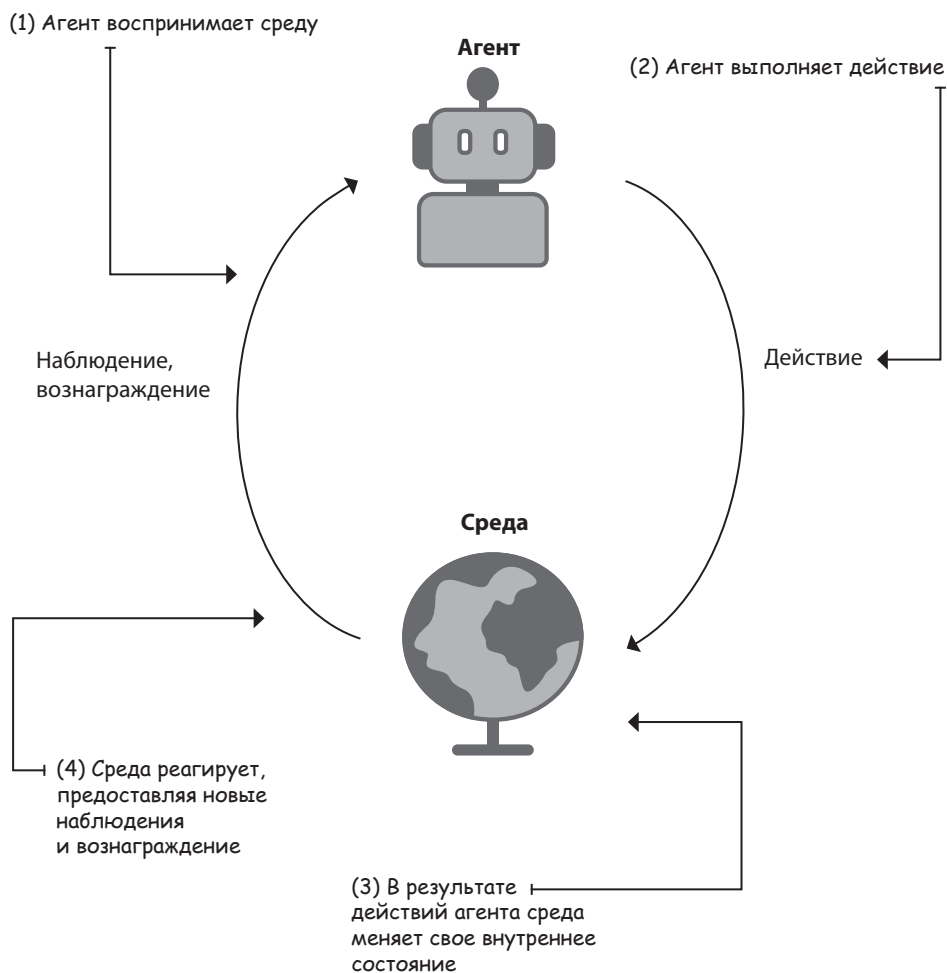
Теперь поговорим о слове «*неопределенность*». Мы не знаем, как на самом деле устроен окружающий мир, и не понимаем, как своими действиями влияем на него: все зависит от нашей интерпретации. Допустим, тренер отправил лучшего игрока в запас, но в следующем матче тот получил травму. А что, если именно из-за решения тренера игрок был физически неподготовлен и это стало причиной травмы? Но, возможно, эта травма мотивирует команду в оставшейся части сезона и приведет к выигрышу в финале? Верно ли поступил тренер? Неопределенность порождает необходимость в дополнительном анализе. Найти правильный баланс между исследованием и эксплуатацией сложно, потому что мы учимся на оценочной обратной связи.

В этой главе вы научитесь представлять такие задачи с помощью модели под названием «*марковский процесс принятия решений*» (Markov decision process, MDP). Общая структура MDP позволяет моделировать практически любую сложную последовательную задачу принятия решений в условиях неопределенности так, чтобы агенты RL могли взаимодействовать и учиться принимать решения только на основе опыта. В третьей главе мы углубимся в детали обучения на основе последовательной обратной связи, а в четвертой — на основе оценочной. В главах 5–7 речь пойдет об обучении на одновременно последовательной и оценочной обратной связи. После этого, в главах 8–12, мы рассмотрим усвоенную информацию в контексте сложных задач.

Элементы обучения с подкреплением

Два основных элемента RL — это *агент* и *среда*. Агент принимает решения и выполняет задачу, а среда — представление этой задачи. Одно из фундаментальных отличий RL от других подходов к машинному обучению в том, что агент и среда взаимодействуют друг с другом: агент пытается повлиять на среду путем выполнения действий, а среда реагирует на действия агента.

Цикл обучения с подкреплением и взаимодействия





Аналогия от Мигеля

Притча о китайском фермере

Есть хорошая притча, иллюстрирующая всю сложность интерпретации одновременно последовательной, оценочной и выборочной обратной связи. Она звучит так.

Китайский фермер покупает лошадь, которая вскоре от него убегает. Сосед говорит: «Как жаль. Плохая новость». Фермер отвечает: «Кто знает, хорошая это новость или плохая».

Вскоре животное возвращается, приведя с собой еще одну лошадь. Сосед восклицает: «Вот так повезло! Хорошая новость!» Фермер отвечает: «Кто знает, хорошая это новость или плохая».

Мужчина отдает вторую лошадь своему сыну, а тот во время езды верхом падает и получает сложный перелом ноги. Сосед говорит: «Очень жаль твоего сына. Это уж точно плохая новость». Фермер отвечает: «Кто знает, хорошая это новость или плохая».

Где-то через неделю представитель императора забирает на войну всех здоровых молодых мужчин, а сын фермера остается дома из-за травмы.

Поэтому кто знает, хорошие новости или плохие?

Интересная история, правда? Часто сложно судить с уверенностью о долгосрочных последствиях происходящих событий и наших собственных действий. Нередко случается так, что несчастье впоследствии приводит к чему-то хорошему, а удача — к чему-то плохому.

Эту историю можно интерпретировать так: красота в глазах смотрящего. Но в обучении с подкреплением мы исходим из того, что между нашими действиями и происходящим в мире есть какая-то связь. Просто ее настолько сложно понять, что люди не могут судить о ней с уверенностью. Но, быть может, это именно то, с чем нам смогут помочь компьютеры. Удивительно, правда?

Помните: когда обратная связь одновременно оценочная, последовательная и выборочная, вы имеете дело со сложной задачей. Глубокое обучение с подкреплением — это вычислительный подход к решению таких задач.

Добро пожаловать в мир глубокого обучения с подкреплением!

Примеры задач, агентов и сред

Ниже перечислены краткие примеры задач, агентов, сред, возможных действий и наблюдений в RL.

- **Задача:** вы учите своего пса команде «Сидеть!». **Агент:** часть вашего мозга, ответственная за принятие решений. **Среда:** ваш пес, его лапы,

лакомства, шумный сосед и т. д. **Действия:** поговорить с псом, ожидать его реакции, помахать рукой, показать лакомство, угостить им собаку, погладить ее. **Наблюдения:** ваш пес обращает на вас внимание, устает, хочет уйти, ваш пес сел по команде.

- **Задача:** ваш пес хочет получить лакомство, которое у вас есть. **Агент:** часть мозга вашего пса, ответственная за принятие решений. **Среда:** вы, ваше лакомство, лапы вашего пса, шумный сосед и т. д. **Действия:** уставиться на хозяина, гавкнуть, прыгнуть на него, попытаться украсть лакомство, бежать, сесть. **Наблюдения:** хозяин продолжает громко разговаривать с псом, показывает лакомство, прячет его, хозяин дает лакомство псу.
- **Задача:** торговый агент инвестирует в фондовый рынок. **Агент:** исполняемый код DRL в памяти и центральном процессоре. **Среда:** ваше интернет-соединение, исполняемый машинный код, цены на акции, геополитическая неопределенность, другие инвесторы, спекулянты и т. д. **Действия:** продать n акций компании y , купить n акций компании y , ждать. **Наблюдения:** рынок на подъеме, рынок идет на спад, экономические трения между двумя влиятельными странами, опасность войны на континенте, глобальная пандемия сеет хаос по всему миру.
- **Задача:** вы за рулем своего автомобиля. **Агент:** часть вашего мозга, ответственная за принятие решений. **Среда:** марка и модель этой машины, другие автомобили, другие водители, погода, дороги, покрышки и т. д. **Действия:** повернуть руль на x , ускориться на y , замедлиться на z , включить фары, обдуть окон, музыку. **Наблюдения:** вы подъезжаете к месту назначения, пробка на главной улице, водитель соседнего автомобиля ведет себя безрассудно, начинается дождь, перед вами едет полицейский.

Как видите, задачи могут быть абсолютно разные: от высокоуровневых, требующих долгосрочного планирования и обширных общих знаний, как в случае с торговлей на фондовом рынке, до низкоуровневых, где геополитические трения не так важны, наподобие вождения автомобиля.

Одну и ту же задачу можно представить с точек зрения разных агентов. В примере с дрессировкой пса на самом деле есть два агента, каждый из которых преследует свои цели и пытается выполнить свою задачу.

Проанализируем каждый из этих компонентов по отдельности.

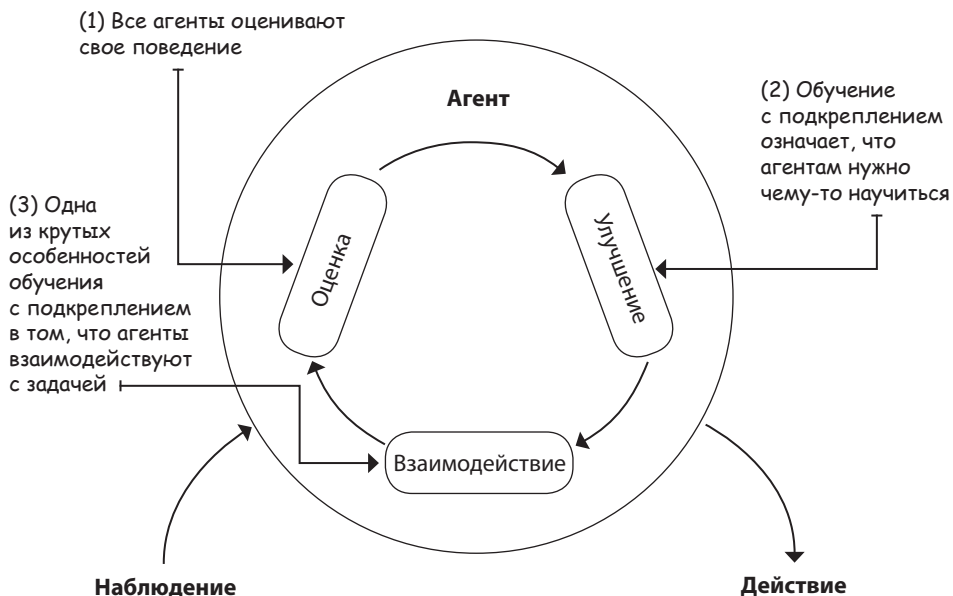
Агент: тот, кто принимает решения

Как я уже упоминал ранее, эта книга почти целиком посвящена агентам. Исключение — эта глава, где речь идет о среде. После ее прочтения нам предстоит создать действенные и эффективные агенты. Для этого мы погрузимся в их внутреннее устройство, структуру их компонентов, процессов и методик.

Пока что вам нужно знать лишь один важный факт об агентах: в общей схеме RL именно они занимаются принятием решений. У них есть свои внутренние компоненты и процессы, что делает каждый агент уникальным и подходящим для решения определенных задач.

При внимательном рассмотрении можно заметить, что большинство агентов проходит три основных этапа, таких как поиск компонента взаимодействия, собирающего данные для обучения, оценка своего текущего поведения и улучшение некоторых аспектов своих внутренних компонентов для повышения общей производительности (по крайней мере попытка это сделать).

Три этапа, через которые проходит любой агент обучения с подкреплением



Подробнее о внутреннем устройстве агентов мы поговорим в следующей главе. А пока поговорим о средах: как они выглядят, как их представлять и моделировать.

Среда: все остальное

Большинство реальных задач, связанных с принятием решений, могут быть выражены в виде среды RL. Для этого их обычно моделируют с помощью *марковского процесса принятия решений* (Markov decision processes, MDP). В RL предполагается, что любая среда основана на MDP. Будь то игра для Atari, фондовый рынок, беспилотный автомобиль, ваша вторая половинка или что-то другое — за любой задачей скрывается MDP (по крайней мере в мире RL).

Среда представлена набором переменных, относящихся к задаче. Сочетание всех потенциальных значений, которые могут принимать эти переменные, называется *пространством состояний*. Состояние — это определенный набор значений, которые хранят переменные в определенный момент времени.

У агентов может быть доступ к реальному состоянию среды или его может не быть, но у них всегда есть возможность наблюдать за средой. Набор переменных, которые агент воспринимает в отдельно взятый момент времени, называется *наблюдением*.

Сочетание всех потенциальных значений, которые могут принимать эти переменные, называется *пространством наблюдений*. Важно отметить, что в сообществе RL термины «состояние» и «наблюдение» считаются синонимами из-за того, что агентам часто (но не всегда) позволено наблюдать за внутренним состоянием среды. Я тоже считаю эти понятия взаимозаменяемыми. Но вы должны понимать, что, несмотря на это, состояния и наблюдения могут отличаться.

В каждом состоянии среда дает агенту набор действий, из которых тот может выбирать. Этот набор часто один и тот же, но не всегда. Набор всех действий во всех состояниях называется *пространством действий*.

Выполняя эти действия, агент пытается повлиять на среду. В ответ на это среда может изменить свое состояние. За это изменение отвечает *функция перехода*.

После выполнения перехода среда генерирует новое наблюдение или может предоставить в качестве ответа сигнал вознаграждения. За это отвечает *функция вознаграждения*.

Набор переходных функций и вознаграждения называют *моделью* среды.



Конкретный пример

Бандитская прогулка

Давайте разберемся в этих понятиях с помощью нашей первой очень простой среды RL. Я создал ее специально для этой книги и назвал «бандитская прогулка» (БП).

БП — это простая грид-среда. Такие часто применяют для исследования алгоритмов RL в виде сетки произвольного размера. У них может быть совершенно любая модель (то есть любые переходные функции и вознаграждения), и они могут сделать доступными любые действия.

Но все они, как правило, позволяют агенту двигаться влево, вниз, вправо, вверх (или на запад, юг, восток, север, что более точно, так как у агента нет направления и он обычно не видит всю сетку, хотя стороны света тоже могут сбить с толку). Каждое действие связано с соответствующим переходом: «влево» вызывает движение влево, «вправо» — движение вправо и т. д. Кроме того, у этих задач обычно полностью наблюдаемые дискретные пространства состояний и наблюдений (то есть состояние равно наблюдению), а местоположение каждой ячейки представлено целым числом. «Прогулка» — это разновидность среды-сетки из одной строки. На самом деле ее обычно называют коридором, но в этой книге я называю все однострочные среды в виде сетки прогулками.

У «бандитской прогулки» (БП) три состояния, два из них конечные. Среды, у которых всего одно пограничное состояние, называются бандитскими, по аналогии с игровыми автоматами — «однорукими бандитами»: у них есть всего один рычаг и, как и настоящие преступники, они способны опустошить ваши карманы.

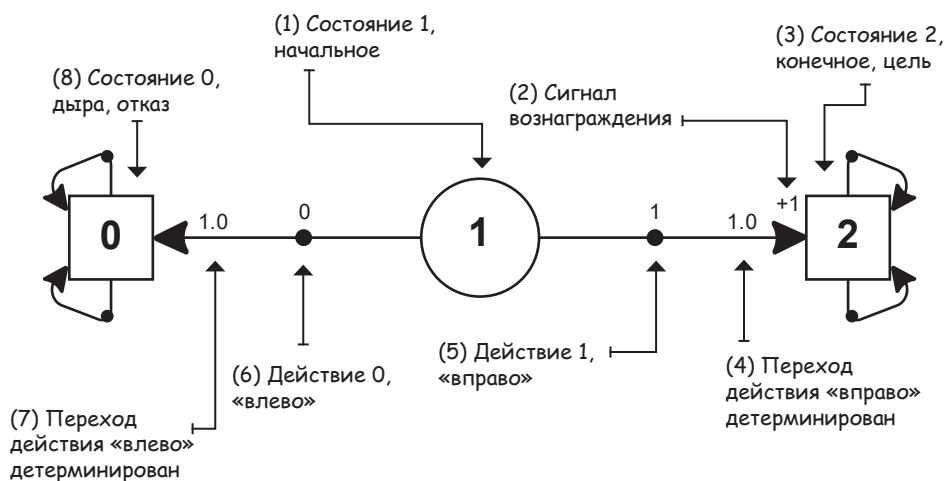
В средах БП доступно всего два действия: «влево» (действие 0) и «вправо» (действие 1). Функция перехода в БП детерминированная: действие «влево» всегда перемещает агента влево, а «вправо» всегда двигает его вправо. Сигнал вознаграждения, +1, возникает при попадании в крайнюю правую ячейку, во всех остальных случаях он равен 0. Агент начинает в средней ячейке.

Среда «бандитская прогулка» (БП)



Графически среду БП можно изобразить так.

Диаграмма «бандитской прогулки»



Я думаю, у вас появились вопросы, но в этой главе вы найдете ответы на них. Например, почему в конечных состояниях есть действия, которые переходят сами в себя: расточительно, не так ли? Может, вас заинтересовало что-то еще? Скажем, что будет, если среда стохастическая? Что вообще такое стохастическая среда?! Читайте дальше.

Эту среду можно представить и в виде таблицы.

Состояние	Действие	Следующее состояние	Вероятность перехода	Сигнал вознаграждения
0 (дыра)	0 (влево)	0 (дыра)	1,0	0
0 (дыра)	1 (вправо)	0 (дыра)	1,0	0
1 (начало)	0 (влево)	0 (дыра)	1,0	0
1 (начало)	1 (вправо)	2 (цель)	1,0	+1
2 (цель)	0 (влево)	2 (цель)	1,0	0
2 (цель)	1 (вправо)	2 (цель)	1,0	0

Интересно, правда? Рассмотрим еще один простой пример.

Конкретный пример

Скользкая бандитская прогулка

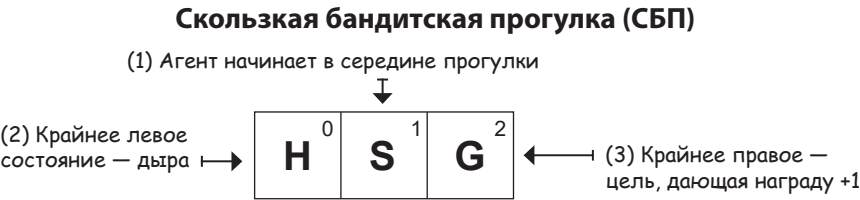
Почему бы нам не сделать эту среду стохастической?

Представьте, что поверхность, по которой вы прогуливаетесь, скользкая и каждое действие может отбросить агента обратно с вероятностью 20 %. Я называю такую среду «скользкой бандитской прогулкой» (СБП).

Это все еще однострочная сетка, прогулка, коридор, где можно двигаться либо влево, либо вправо. Опять же у нас есть три состояния и два действия. Награда та же, что и прежде: +1 при попадании в крайнюю правую ячейку (если только мы не попадаем в нее из нее самой) и 0 — в противном случае.

А вот функция перехода отличается: в 80 % случаев агент переходит в нужную ячейку, а в 20 % — движется в противоположном направлении.

Изображение этой среды будет выглядеть так.



Точно так же, как и в среде БП! Интересно...

Откуда мы знаем, что у действий стохастические последствия? Как представить «скользкую» часть проблемы? Здесь нам помогут графическое и табличное представления.

Среду СБП можно изобразить так.



Видите, как изменилась функция перехода? В среде СБП она стохастическая. Представим среду в виде таблицы, как мы делали ранее.

Состояние	Действие	Следующее состояние	Вероятность перехода	Сигнал вознаграждения
0 (дыра)	0 (влево)	0 (дыра)	1,0	0
0 (дыра)	1 (вправо)	0 (дыра)	1,0	0
1 (начало)	0 (влево)	0 (дыра)	0,8	0
1 (начало)	0 (влево)	2 (цель)	0,2	+1
1 (начало)	1 (вправо)	2 (цель)	0,8	+1
1 (начало)	1 (вправо)	0 (дыра)	0,2	0
2 (цель)	0 (влево)	2 (цель)	1,0	0
2 (цель)	1 (вправо)	2 (цель)	1,0	0

И не нужно ограничиваться размышлениями о средах с дискретными пространствами действий и состояний или даже о «прогулках» (коридорах), «бандитах» (подробнее об этом — в следующей главе) и о сетчатых пространствах. Представление сред в виде MDP — на удивление действенный и интуитивно понятный подход к моделированию сложных последовательных задач, связанных с принятием решений в условиях неопределенности.

Вот еще несколько примеров сред, основанных на MDP.

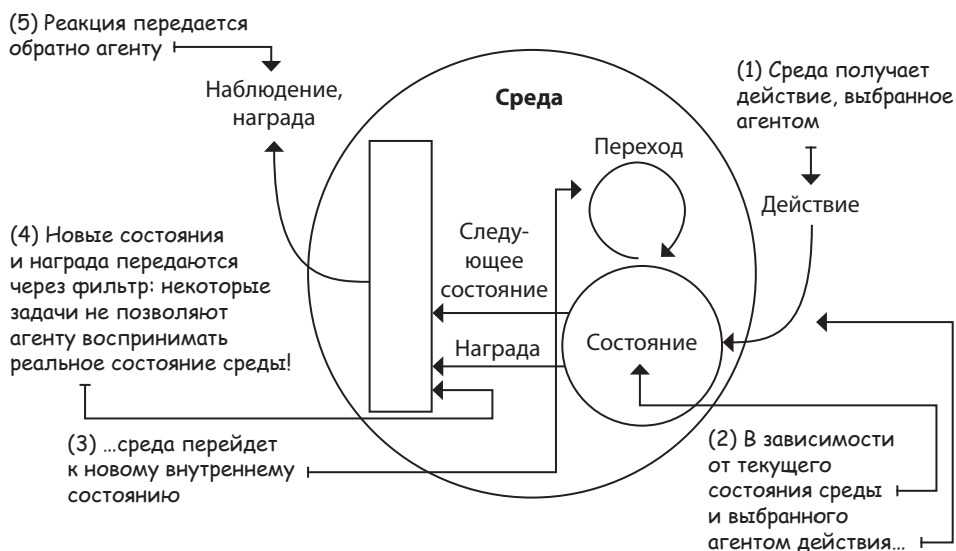
Описание	Пространство наблюдений	Пример наблюдения	Пространство действий	Пример действия	Функция вознаграждения
Горячо, холодно: угадайте случайно выбранное число, используя подсказки	Диапазон значений — 0–3, где 0 — варианты еще не предлагались; 1 — вариант меньше цели; 2 — вариант равен цели; 3 — вариант больше цели	2	Диапазон значений — –2000, 0–2000, 0. Вещественное число, которое угадывает агент	–909,37	Награда — квадрат процентной доли расхождения, на которое агент приблизился к цели
Перевернутый маятник: попробуйте сбалансировать стержень на тележке	Четырехэлементный вектор с диапазонами значений от $[-4,8; -\infty; -4,2; -\infty]$ до $[4,8; \infty; 4,2; \infty]$. Первый элемент — позиция тележки, второй — ее скорость и направление движения, третий — наклон стержня в радианах, четвертый — скорость и направление движения стержня в верхней точке	$[-0,16; -1,61; 0,17; 2,44]$	Диапазон значений — 0–1, 0 означает «толкнуть тележку влево», а 1 — «толкнуть тележку вправо»	0	Награда равна 1 для каждого выполненного шага, в том числе конечного
Лунный посадочный модуль: переместите луноход на посадочную площадку	Восьмиэлементный вектор с диапазонами значений от $[-\infty, -\infty, -\infty, -\infty, -\infty, 0, 0]$ до $[\infty, \infty, \infty, \infty, \infty, 1, 1]$. Первый элемент — позиция x, второй — y, третий — скорость и направление движения по x, четвертый — по y, пятый — наклон модуля, шестой — угловые скорость и направление движения, последние два — булевы значения, сигнализирующие о контакте опор с землей	$[0,36; 0,23; -0,63; -0,10; -0,97; -1,73; 1,0; 0,0]$	Диапазон значений — 0–3: NOP (ничего не делать), включить левый двигатель, главный двигатель, правый двигатель	2	Награда за посадку — 200. Есть награда за спуск к посадочной площадке, за аварию или остановку, за прикосновение каждой опоры к земле и за запуск двигателей

Описание	Пространство наблюдений	Пример наблюдения	Пространство действий	Пример действия	Функция вознаграждения
Pong: отбивайте мяч мимо соперника и не позволяйте мячу пройти мимо вас	Тензор формы 210, 160, 3. Диапазон значений — 0–255. Представляет изображение игрового экрана	[[246, 217, 64], [55, 184, 230], [46, 231, 179], ... [28, 104, 249], [25, 5, 22], [173, 186, 1], ...]]	Диапазон значений — 0–5. Действие 0 — NOP, 1 — огонь, 2 — вверх, 3 — вправо, 4 — влево, 5 — вниз. Заметьте: некоторые действия никак не влияют на игру. В реальности ракетка может двигаться вверх, вниз или оставаться на месте	3	Награда равна 1, когда шарик минует соперника, и -1 , когда ракетка вашего агента промахивается
Гуманоид: заставьте робота бежать как можно быстрее и не упасть	Сорок четыре элемента (или более крупный, в зависимости от реализации) вектор. Диапазон значений — от $-\infty$ до $+\infty$. Отображает положение и скорость суставов робота	[0,6; 0,08; 0,9; 0, 0; 0,0; 0,0; 0,0; 0,0; 0,0; 0,045; 0,0; 0,47... 0,32; 0,0; -0,22... 0,0]	Семнадцатичисленный вектор. Диапазон значений — от $-\infty$ до $+\infty$. Представляет силы, прилагаемые к суставам робота	[$-0,9$; $-0,06$; 0,6; 0,6; 0,6; $-0,06$; $-0,4$; $-0,9$; 0,5; $-0,2$; 0,7; $-0,9$; 0,4; $-0,8$; $-0,1$; 0,8; $-0,03$]	Вознаграждение рассчитывается на основе движения вперед с небольшим штрафом, чтобы стимулировать естественную походку

Заметьте, я не добавил в эту таблицу функцию перехода. Дело в том, что, несмотря на возможность посмотреть код, реализующий динамику для определенных сред, другие реализации недоступны. Например, функция перехода в среде перевернутого маятника представляет собой небольшой Python-файл, определяющий массу тележки и стержня и реализующий основные уравнения, в то время как динамика игр Atari, таких как Pong, скрыта внутри эмулятора Atari и соответствующего файла ПЗУ для конкретной игры.

Отмечу также, что здесь мы пытаемся проиллюстрировать, что среда как-то «реагирует» на действия агента, возможно даже игнорируя их. В конце концов, мы имеем дело с неопределенным внутренним процессом (во всех главах, кроме этой и следующей). Чтобы представить возможность взаимодействия со средой в MDP, нам нужны состояния, наблюдения, действия, переход и функция вознаграждения.

Процесс, через который проходит среда в результате действий агента



Цикл взаимодействия «агент — среда»

У среды, как правило, есть четко определенная задача, цель которой устанавливается через сигнал вознаграждения. Он может быть плотным, разреженным или смешанным. При проектировании среды сигналы вознаграждения позволяют обучать агент так, как вам нужно. Чем они плотнее, тем больше контроля будет над агентом и тем быстрее он будет обучен, но чем больше смещения вы привнесете, тем ниже будет вероятность того, что агент начнет вести себя неожиданно. Чем более разрежены сигналы, тем меньше становится контроля, что повышает вероятность возникновения новых аспектов поведения, но замедляет обучение агента.

Взаимодействие между агентом и средой проходит несколько циклов, каждый из которых называется *временным шагом*. Временной шаг — это произвольная единица времени: миллисекунда, секунда, 1,2563 секунды, минута, день или любой другой интервал и т. п.

На каждом временном шаге агент наблюдает за средой, действует, получает новое наблюдение и вознаграждение. Обратите внимание, что награда может принимать отрицательные значения, но даже в этих случаях в мире RL ее называют наградой. Сочетание наблюдения (или состояния), действия, награды и нового наблюдения (или нового состояния) называется *кортежем опыта*.

У задачи, которую пытается выполнить агент, может быть естественное завершение или его может не быть. Задачи с естественным завершением, такие как игры, называют *эпизодическими*, а без него, такие как обучение движению вперед, — *непрерывными*. Последовательность временных шагов от начала эпизодической задачи до ее завершения называется *эпизодом*. Чтобы научиться выполнять задачу, агенту может понадобиться несколько временных шагов и эпизодов. Сумма вознаграждений, получаемых в рамках одного эпизода, называется *выгодой* (return). Агенты обычно проектируются так, чтобы получить максимум выгоды. К непрерывным задачам часто добавляется ограничение по времени, поэтому они становятся эпизодическими, а агенты могут максимизировать выгоду.

У каждого кортежа опыта есть возможность учиться и повышать производительность. Облегчить процесс обучения агента могут несколько дополнительных компонентов. Агент может быть спроектирован, чтобы изучать сопоставления наблюдений с действиями, называемыми политиками, для обучения отображениям наблюдений за новыми данными и/или вознаграждениями, которые называются моделями. Он может быть создан для формирования связей между наблюдениями (и, возможно, действиями) и ожидаемой наградой (сегментом выгоды). Эти связи называются функциями ценности.

В конце этой главы мы отложим в сторону все, что связано с агентом и взаимодействиями, и сосредоточимся на среде и внутренней модели MDP. В главе 3 мы снова вернемся к агентам, но не затрагивая взаимодействие — оно им не понадобится, ведь у них будет прямой доступ к MDP. В главе 4 мы удалим доступ агента к MDP и снова добавим взаимодействия в уравнение, но это будет в средах с одним состоянием (мы называем их «бандитами»). В главе 5 мы научимся оценивать выгоду в средах с несколькими состояниями, где у агента не будет доступа к MDP. Главы 6 и 7 посвящены оптимизации поведения — основной задаче обучения с подкреплением. В главах 5–7 мы поговорим об обучении агентов в средах, где не нужна аппроксимация функций. Весь дальнейший материал будет полностью сосредоточен вокруг агентов, использующих для обучения нейросети.

MDP: двигатель среды

Давайте спроектируем модели MDP для нескольких сред и посмотрим, из каких компонентов они состоят. Создадим словари Python, представляющие MDP на основе описания задач. В следующей главе мы изучим алгоритмы

планирования MDP. Они помогут нам в разработке решений MDP и позволят найти оптимальные способы выполнения всех задач из этой главы.

Умение самостоятельно создавать среды — важный навык. Но вам будут часто попадаться среды, для которых кто-то уже создал MDP. Кроме того, динамика среды часто скрыта за механизмом моделирования и слишком сложна для детального изучения, а иногда может быть совершенно недоступной или скрытой в силу определенных обстоятельств. На самом деле, чтобы освоить устойчивое поведение, агентам RL не нужно знать точную модель MDP задачи, но очень важно понимать MDP в целом, так как при создании агентов обычно подразумевается наличие какой-то внутренней модели MDP, даже если она недоступна.



Конкретный пример

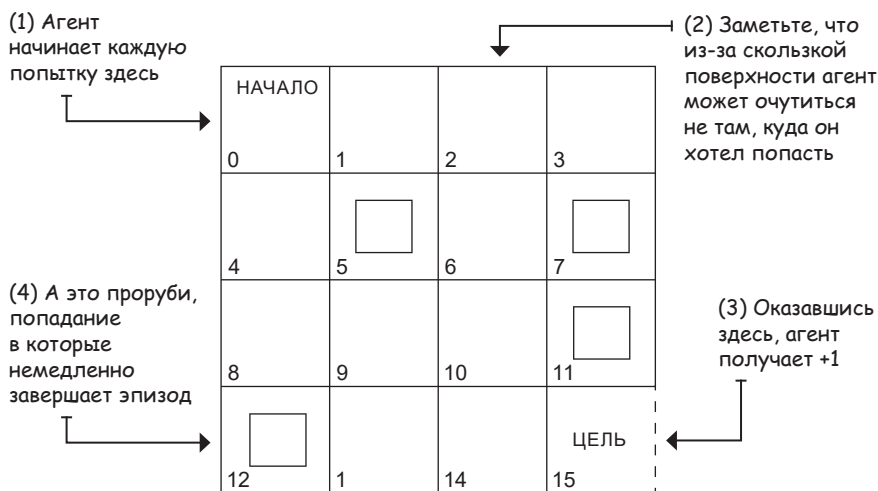
Замерзшее озеро

Еще одна, более сложная задача, для которой в этой главе мы создадим MDP. Эта среда называется «замерзшее озеро» (30).

30 — это простая грид-среда, у которой тоже дискретные пространства состояний и действий. Но на этот раз нам доступно четыре действия: «влево», «вниз», «вправо» и «вверх».

Задача среды 30 похожа на ту, которую мы видели в средах БП и СБП: пройти от начальной ячейки к целевой, избегая попадания в «проруби» (дыры). На СБП это похоже тем, что поверхность 30 скользкая — озеро все-таки замерзшее. Но сама среда по размеру гораздо больше. Рассмотрим ее диаграмму.

Среда «замерзшее озеро» (30)



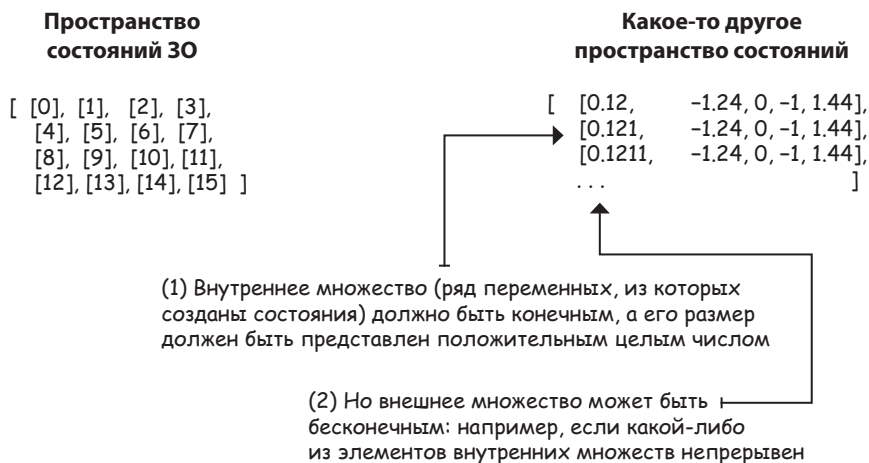
ЗО представляет собой сетку 4×4 (16 ячеек с идентификаторами 0–15). В каждом новом эпизоде агент появляется в ячейке НАЧАЛО. За попадание в ячейку ЦЕЛЬ выдается награда +1, в противном случае — 0. Поскольку поверхность скользкая, агент попадает в целевую ячейку только в каждом третьем случае. Остальные две третьих распределяются равномерно в ортогональных направлениях. Например, если агент решит переместиться вниз, вероятность того, что это произойдет, равна 33,3 %; в других 33,3 % случаев он переместится влево, и еще в 33,3 % — вправо. Вокруг озера есть ограждение, поэтому, если агент попытается выйти за него, он будет возвращен в ячейку, которую хотел покинуть. В озере есть четыре проруби, попадание в одну из которых равнозначно проигрышу.

Готовы ли вы приступить к формированию представления этой динамики? Нам нужен словарь Python с моделью MDP в том виде, в котором мы ее только что описали. Начнем с создания этой модели.

Состояния среды

Состояние — это уникальная и автономная конфигурация задачи. Набор всех возможных состояний называется *пространством состояний* и обозначается как множество S . Пространство состояний может быть конечным или бесконечным. Заметьте, что пространство состояний отличается от набора переменных, составляющих одно состояние, который должен быть конечным и иметь постоянный размер. В целом пространство состояний — это множество множеств. Внутреннее множество должно быть конечным и иметь постоянный размер, так как содержит ряд переменных, представляющих состояния. Внешнее может быть бесконечным, в зависимости от типов элементов внутренних множеств.

Пространство состояний: множество множеств

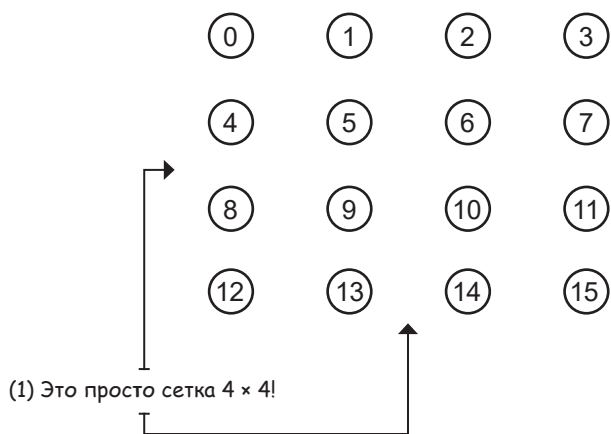


В средах БП, СБП и ЗО состояние — это единственная переменная с идентификатором ячейки, где находится агент в отдельно взятый момент времени. Идентификатор ячейки с агентом — это дискретная переменная. Но переменные состояния могут иметь какой угодно тип, и их количество не ограничено. Это может быть евклидово расстояние, состоящее из непрерывной переменной и бесконечного пространства состояний: например, 2,124; 2,12456; 5,1; 5,1239458 и т. д. Состояние может быть определено и несколькими переменными: например, количеством ячеек между агентом и целью по осям X и Y . В этом случае оно было бы представлено двумя дискретными переменными, что сделало бы пространство состояний конечным. Но мы можем сочетать переменные разных типов: например, одна может быть дискретной, другая — непрерывной, а третья — булевой.

При таком представлении состояний для сред БП, СБП и ЗО размер пространства состояний равен 3, 3 и 16. Поскольку у нас есть 3, 3 или 16 ячеек, в которых может находиться агент в любой момент, у нас получается 3, 3 и 16 возможных состояний в каждом из пространств. Мы можем назначить идентификаторы каждой ячейке, начиная с нуля и двигаясь слева направо и сверху вниз.

В случае с ЗО мы назначаем идентификаторы от нуля до 15, двигаясь слева направо и сверху вниз. Но вы можете сделать это иначе: в случайном порядке, в зависимости от расстояния и т. д. Главное, чтобы они оставались неизменными на протяжении обучения. Но это представление вполне адекватно и работает без проблем, поэтому мы остановимся именно на нем.

Состояния в ЗО содержат одну переменную, обозначающую идентификатор ячейки, в которой находится агент на любом заданном временном шаге



В случае с MDP состояния полностью наблюдаемы: мы можем видеть внутреннее состояние среды на каждом временном шаге — то есть наблюдения

и состояния совпадают. *Частично наблюдаемый марковский процесс принятия решений* (partially observable Markov decision process, POMDP) — более общий подход к моделированию сред, в котором агенту вместо состояния доступны только наблюдения (хотя сами они зависят от внутреннего состояния среды). Отмечу, что для сред БП, СБП и ЗО мы создаем MDP, поэтому агент сможет наблюдать за внутренним состоянием среды.

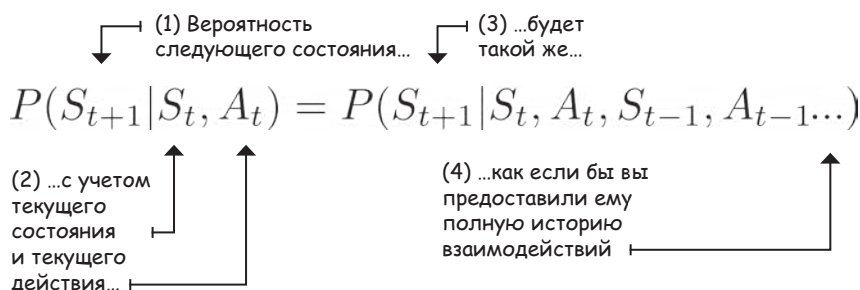
Состояние должно содержать все переменные среды, необходимые для того, чтобы сделать его независимым от всех остальных состояний. Для определения следующих возможных состояний в среде ЗО достаточно знать лишь текущее состояние агента — вам не нужен доступ к истории посещения ячеек. Вы и так знаете, что из ячейки 2 агент может переместиться только в ячейку 1, 3, 6 или 2, и на это никак не влияет то, посещал ли агент эти ячейки ранее.

Вероятность следующего состояния при текущем состоянии и действии не зависит от истории взаимодействий. Это отсутствие памяти в MDP называется *марковским свойством*: вероятность перехода из одного состояния s в другое в двух отдельных случаях при одном и том же действии a одинакова независимо от всех предыдущих состояний или действий, возникших до этого.



Я знаю математику

Марковское свойство



Но зачем это нам? Что ж, в средах, которые мы рассматривали до сих пор, все не так очевидно и этот аспект не слишком важен. Но большинство агентов RL (и DRL) опираются на марковское свойство, поэтому вы должны передать агенту необходимые переменные, чтобы сделать его как можно более устойчивым (полноценная реализация марковского свойства непрактична, если вообще возможна).

Например, разрабатывая агент, который должен обучиться посадке космического корабля, вам нужно сделать так, чтобы он получил все переменные с местоположением и векторами движения. Только местоположения будет недостаточно для безопасного приземления. Кроме того, мы предполагаем, что

у агента нет памяти, поэтому информация, которую нам нужно ему предоставить, не может ограничиваться лишь трехмерными координатами его местоположения относительно посадочной площадки.

Но, как вы знаете, ускорение для скорости — это то же самое, что скорость для местоположения, — производная. И вам, наверное, известно, что при вычислении производных можно не останавливаться на ускорении. Как далеко мы должны зайти, чтобы сделать модель MDP полностью марковской? Это больше похоже на искусство, чем на науку: чем больше переменных вы добавляете, тем дольше обучается агент. С другой стороны, чем меньше переменных, тем выше вероятность того, что агент получает недостаточно информации, что затрудняет процесс его обучения. Вернемся к примеру с космическим кораблем. В этом случае обычно достаточно местоположений и скоростей, а в грид-средах агенту хватит только идентификатора ячейки, в которой он находится.

Множество состояний в MDP обозначается как S^+ . У S^+ есть подмножество *начальных состояний*, S^i . Чтобы начать взаимодействие с MDP, мы извлекаем из S^i новое состояние с учетом распределения вероятностей. Распределение может быть любым, но оно должно оставаться неизменным на протяжении всего обучения: то есть вероятности не должны меняться на протяжении всех эпизодов обучения и при оценке агента значения.

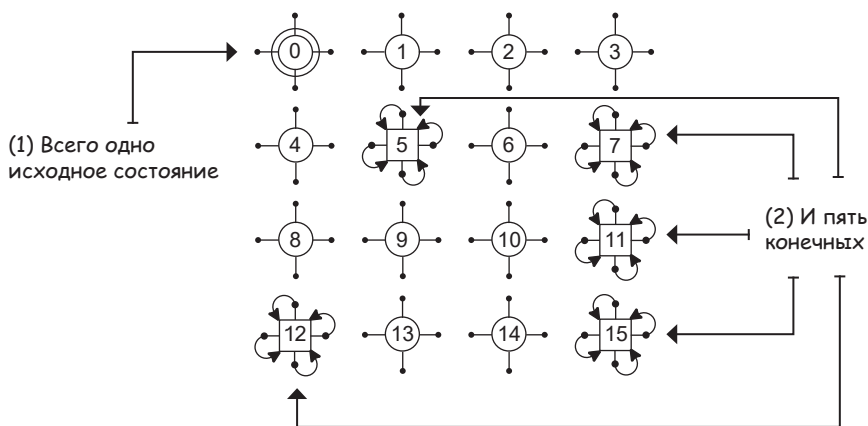
Есть уникальное состояние, которое называют *поглощающим* или *терминальным (конечным)*; множество всех нетерминальных состояний обозначается как S . И хотя создание одного поглощающего состояния (состояния *приемника*), в которое переходят все остальные, — обычная практика, ее применяют не всегда. Чаще всего вам придется сталкиваться именно со множественными терминальными состояниями, и это нормально. Не имеет значения, если вы заставите все поглощающие состояния вести себя предсказуемо, если все это будет работать.

Предсказуемо? Да. У поглощающего состояния есть особое свойство: все доступные в нем действия должны переходить к нему самому и эти переходы не должны давать никакого вознаграждения. Обратите внимание, что я имею в виду переходы из терминального состояния, а не в него.

Очень часто бывает, что вознаграждение в конце эпизода не равно нулю. Например, партию в шахматы можно выиграть, проиграть или сыграть вничью. Логичными сигналами вознаграждения будут +1, -1 и 0 соответственно. Это просто соглашение о совместимости, позволяющее всем алгоритмам приходить к общему решению, чтобы при любых доступных действиях терминальное состояние переходило само в себя с вероятностью 1 и наградой 0. Иначе вы рискуете получить бесконечные суммы и алгоритмы, несовместимые между собой. Помните, как у сред БП и СБП были именно такие состояния?

Например, в среде 3О есть всего одно исходное состояние (состояние 0) и пять терминальных (или пять состояний, переходящих в одно и то же поглощающее, — как вам больше нравится). Уточню, что в иллюстрациях и примерах кода я использую общепринятую модель с несколькими терминальными состояниями (5, 7, 11, 12 и 15). Опять же каждое из них — отдельное терминальное состояние.

Состояния в среде «замёрзшее озеро»



Действия: механизм влияния на среду

Модели MDP предоставляют набор действий A , который зависит от состояния. То есть в определенном состоянии некоторые действия могут быть недоступны. На самом деле A — это функция, которая принимает состояние в качестве аргумента, например, $A(s)$ возвращает множество действий, доступных в s . Но это множество можно сделать и постоянным для всего пространства состояний — в таком случае в каждом состоянии будут доступны все действия. Чтобы запретить действие в заданном состоянии, мы можем обнулить все переходы для пары «состояние — действие». Еще можно установить все переходы из состояния s и действия a в состояние s — так действие a не будет иметь никаких последствий.

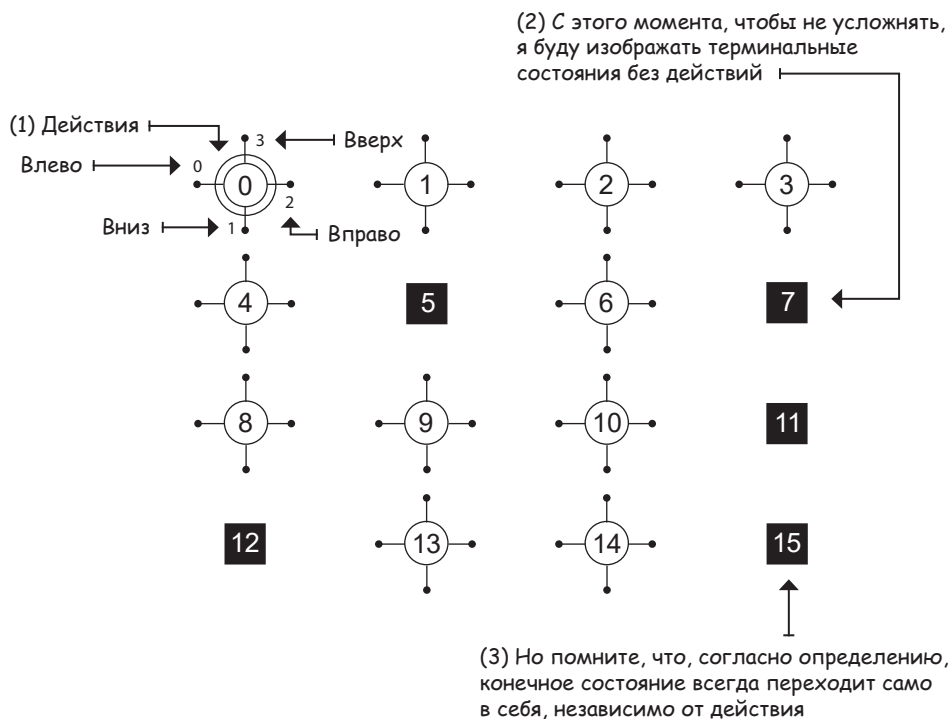
Как и пространство состояний, пространство действий может быть конечным или бесконечным, а набор переменных отдельного действия может содержать больше одного элемента и должен быть конечным. Но, в отличие от состояния, действие может включать в себя разное количество переменных. Число действий, доступных в состоянии, может меняться в зависимости от него, но для удобства большинство сред спроектировано с одинаковым количеством действий во всех состояниях.

Среда заранее делает набор всех доступных действий известным. Агенты могут выбирать их детерминированно или стохастически. Это не то же самое, что детерминированная или стохастическая реакция среды на действия агентов. Оба утверждения верны, но отличаются друг от друга. Здесь я указываю на то, что агенты могут выбирать действия либо из таблицы поиска, либо из распределений вероятности каждого отдельного состояния.

В средах БП, СБП и ЗО действия — это одиночки, показывающие направление, в котором пытается двигаться агент. В ЗО у всех состояний есть

четыре допустимых действия — «вверх», «вниз», «вправо» и «влево», для каждого из которых предусмотрена одна переменная, а размер области действия равен 4.

У среды «замерзшее озеро» есть четыре простых действия-движения



Функция перехода: последствия действий агента

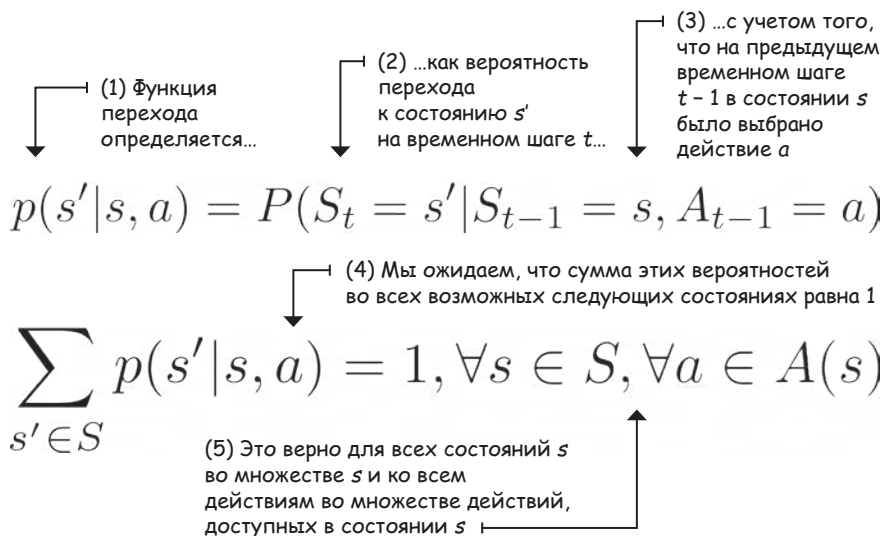
Изменение среды в ответ на действия называется *переходной вероятностью* или просто *функцией перехода* и обозначается как $T(s, a, s')$. Функция перехода T отображает кортеж перехода s, a, s' в вероятность: то есть вы передаете состояния s , действие a и следующее состояние s' , а функция возвращает соответствующую вероятность перехода из s в s' при выполнении действия a . Эту функцию можно представить как $T(s, a)$. В этом случае она будет возвращать следующие состояния для своих ключей и вероятности для своих значений.

Обратите внимание, что T тоже описывает вероятностное распределение $p(\cdot | s, a)$, определяющее, как система будет изменяться в рамках цикла взаимодействия при выборе действия a в состоянии s . При интегрировании по следующим состояниям s' , как и при любом распределении вероятностей, сумма этих вероятностей должна равняться единице.



Я знаю математику

Функция перехода



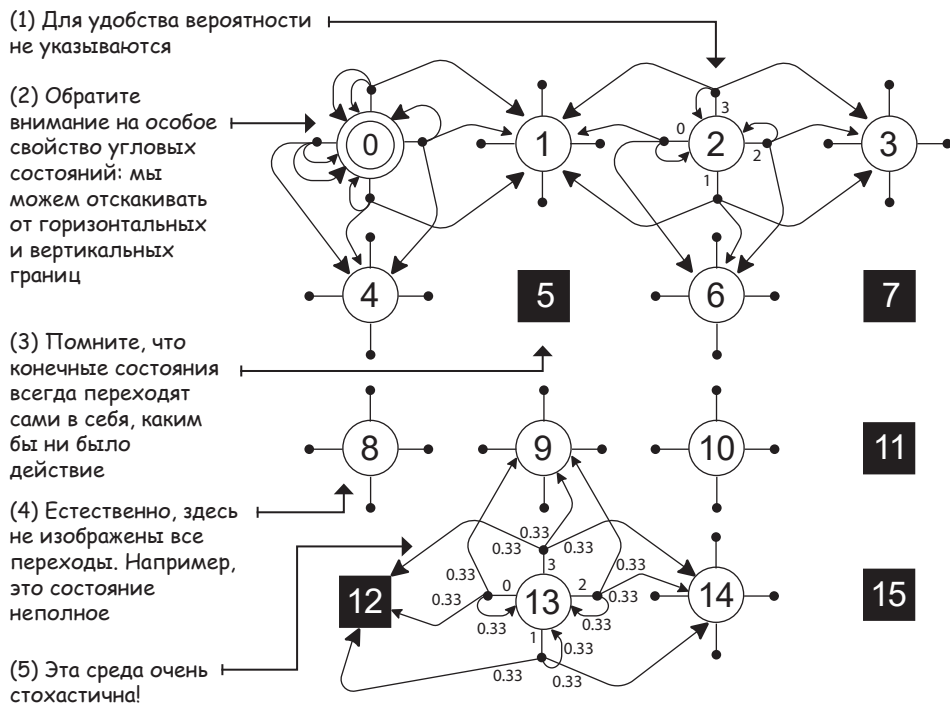
Среда БП была детерминированной. То есть вероятность следующего состояния s' с учетом текущего состояния s и действия a всегда равнялась 1 — всегда существовало одно возможное следующее состояние s' . Среда СБП и ЗО стохастические, значит, вероятность перехода в следующее состояние s' с учетом текущего состояния s и действия a меньше 1. Есть несколько возможных следующих состояний s' .

Одно из ключевых допущений многих алгоритмов RL (и DRL) — то, что это распределение стационарное. То есть, несмотря на то что переходы могут иметь высокую степень стохастичности, вероятностное распределение может не меняться на протяжении обучения или оценки. Как и в случае с марковским свойством, предположение о стационарности зачастую ослабляется до определенной степени. Но для большинства агентов важно, чтобы среды, с которыми они взаимодействуют, хотя бы казались стационарными.

Если говорить о среде ЗО, то мы знаем, что с вероятностью 33,3 % перейдем в нужную нам ячейку (состояние), а в остальных 66,6 % случаев переход будет выполнен в одном из ортогональных направлений. Еще есть шанс вернуться в состояние, из которого выполняется переход, если оно находится рядом с границей.

Чтобы вам было понятнее, я отметил на следующем изображении функцию перехода для всех действий, относящихся только к состояниям 0, 2, 5, 7, 11, 12, 13 и 15. Это подмножество состояний позволяет проиллюстрировать все возможные переходы, не отвлекаясь на лишние детали.

Функция перехода в среде «замерзшее озеро»



Даже эта упрощенная диаграмма может выглядеть немного запутанно. Попробуйте посмотреть на нее с такой стороны: для согласованности у каждого действия в нетерминальных состояниях есть три отдельных перехода (некоторые действия в угловых состояниях могут быть представлены только двумя переходами, но опять же я хочу быть последовательным) — один в нужную ячейку и два в ортогональных направлениях.

Сигнал вознаграждения: кнут и пряник

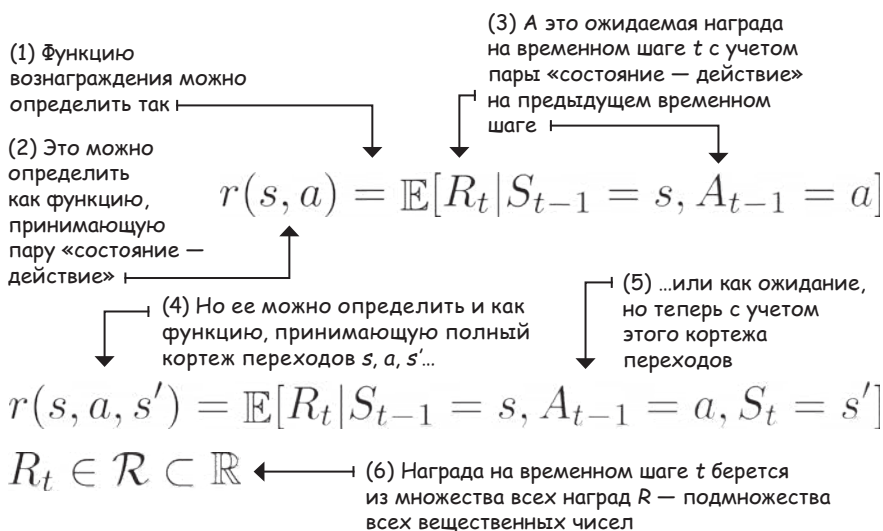
Функция вознаграждения R сопоставляет кортеж переходов s, a, s' со скалярным значением. Она назначает переходу числовой сигнал положительности. Когда сигнал больше нуля, его можно считать прибылью или наградой. В большинстве задач есть хотя бы один положительный сигнал — например, выигрыш шахматной партии или прибытие в пункт назначения. Но награда может быть и отрицательной, ее можно считать затратами, наказанием или штрафом. В робототехнике добавление стоимости временного шага — обычная практика, так как мы хотим не просто достичь цели, а сделать это за определенное количество временных шагов. Следует уточнить один момент: скалярное значение, возвращаемое функцией вознаграждения, будь оно положительным или отрицательным, всегда называется *вознаграждением*. Те, кто занимается RL, мыслят позитивно.

Еще важно подчеркнуть, что в зависимости от наших потребностей функцию вознаграждения можно представлять как явно, $R(s, a, s')$, так и в следующем виде: $R(s, a)$ или даже $R(s)$. Иногда нам нужно вознаграждать агента с учетом состояния, а иногда лучше использовать сочетание состояния и действия. Но самый явный способ представления функции вознаграждения — использование тройного значения: состояния, действия и следующего состояния. Так мы можем вычислить маргинализацию по следующим состояниям в $R(s, a, s')$, чтобы получить $R(s, a)$, и маргинализацию по действиям в $R(s, a)$, чтобы получить $R(s)$. С другой стороны, получив $R(s)$, мы не можем воссоздать $R(s, a)$ или $R(s, a, s')$, а из $R(s, a)$ нельзя получить $R(s, a, s')$.



Я знаю математику

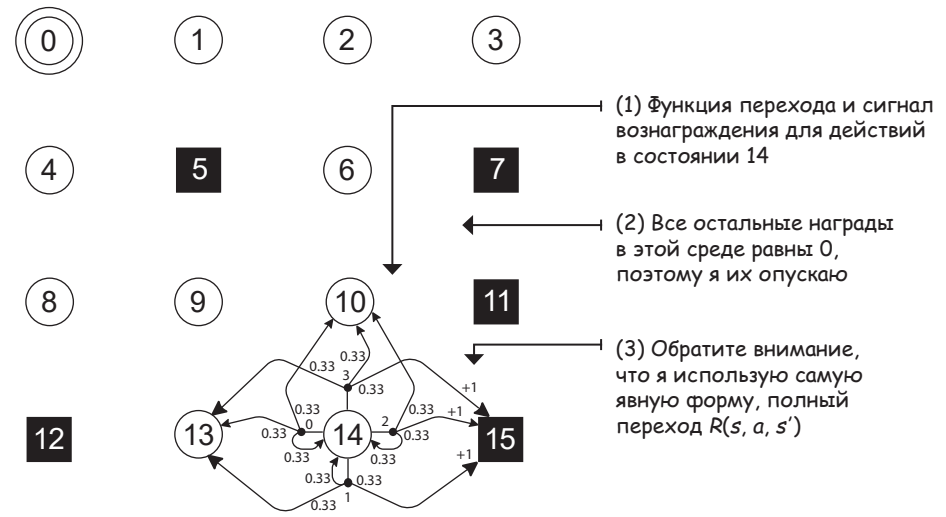
Функция вознаграждения



В среде 30 функция награды возвращает +1, если мы попадаем в состояние 15 и 0 в остальных случаях. Опять-таки, чтобы упростить следующее изображение, я указал сигнал вознаграждения только для переходов к конечному состоянию (15), которые дают ненулевую награду.

В ячейку 15 можно попасть всего тремя способами. (1) Выбор действия «вправо» в состоянии 14 переместит туда агент с вероятностью 33,3 % (переходы в состояние 10 и обратно в 14 тоже имеют вероятность по 33,3 % каждый). Но выбор действий «вверх» (2) и «вниз» (3) в состоянии 14 тоже может случайно привести к переходу агента в ячейку 15 с вероятностью 33,3 %. Видите, чем отличаются действия и переходы? Интересно наблюдать за тем, как стохастичность все усложняет, правда?

Сигнал награды для состояний с ненулевыми переходами вознаграждения



Будет нелишним развернуть функции перехода и вознаграждения в таблицу. Ниже приведен формат, который я рекомендую для большинства задач. В этой таблице перечислена лишь часть переходов (строк), чтобы проиллюстрировать упражнение. Обратите внимание, что я перечисляю их явно, хотя некоторые можно было бы сгруппировать или представить по-другому (как в случае с угловыми ячейками).

Состояние	Действие	Следующее состояние	Вероятность перехода	Сигнал вознаграждения
0	Влево	0	0,33	0
0	Влево	0	0,33	0
0	Влево	4	0,33	0
0	Вниз	0	0,33	0
0	Вниз	4	0,33	0
0	Вниз	1	0,33	0
0	Вправо	4	0,33	0
0	Вправо	1	0,33	0
0	Вправо	0	0,33	0

Состояние	Действие	Следующее состояние	Вероятность перехода	Сигнал вознаграждения
0	Вверх	1	0,33	0
0	Вверх	0	0,33	0
0	Вверх	0	0,33	0
1	Влево	1	0,33	0
1	Влево	0	0,33	0
1	Влево	5	0,33	0
1	Вниз	0	0,33	0
1	Вниз	5	0,33	0
1	Вниз	2	0,33	0
1	Вправо	5	0,33	0
1	Вправо	2	0,33	0
1	Вправо	1	0,33	0
2	Влево	1	0,33	0
2	Влево	2	0,33	0
2	Влево	6	0,33	0
2	Вниз	1	0,33	0
...	...	...	...	...
14	Вниз	14	0,33	0
14	Вниз	15	0,33	1
14	Вправо	14	0,33	0
14	Вправо	15	0,33	1
14	Вправо	10	0,33	0
14	Вверх	15	0,33	1
14	Вверх	10	0,33	0
...	...	...	...	...
15	Влево	15	1,0	0
15	Вниз	15	1,0	0
15	Вправо	15	1,0	0
15	Вверх	15	1,0	0

Горизонт: оптимальное решение со временем меняется

В MDP можно представить и время. *Временной шаг*, также называемый эпохой, циклом, итерацией или даже взаимодействием, — это глобальные часы, синхронизирующие все стороны и дискретизирующие время. Наличие часов позволяет выполнить несколько типов задач. *Эпизодическая* задача — та, в которой есть конечное количество временных шагов либо потому, что часы останавливаются, либо потому, что агент достигает конечного состояния. Есть и непрерывные задачи, которые никогда не заканчиваются: у них нет конечных состояний, поэтому число временных шагов не ограничено. В таких задачах агент нужно останавливать вручную.

Эпизодические и непрерывные задачи можно определить и с точки зрения агента. Мы называем это *горизонтом планирования*. Он может быть *конечным*, если агент знает, что спустя какое-то время задача закончится. Например, мы можем заставить агента завершить работу в среде «замерзшее озеро» за 15 шагов. Особая разновидность такого горизонта планирования — *жадный горизонт*. Он есть как в БП, так и в СБП: эпизод заканчивается сразу после одного взаимодействия. По сути, у всех «бандитских» сред горизонты жадные.

Еще есть *бесконечный горизонт* — когда у агента нет заранее определенно-го ограничения временных шагов и он вынужден планировать свою работу в бесконечном цикле. Такая задача все еще может быть эпизодической и, следовательно, завершаться, но с точки зрения агента ее горизонт планирования бесконечен. Это называется *проблемой бесконечного горизонта*. Агент планирует работать бесконечно, но в любой момент сама среда может прекратить взаимодействие.

В задачах с высокой вероятностью того, что агент попадет в бесконечный цикл и никогда не прекратит работу, часто предусматривают специальное конечное состояние, основанное на количестве временных шагов: жесткий лимит с использованием переходной функции. Такие случаи требуют особого обращения с конечным состоянием на основе ограничения временных шагов. У среды перевернутого маятника (см. главы 8, 9 и 10) как раз такой вид искусственного терминального шага, и позже вы научитесь работать с этими особыми случаями.

Среды БП, СБП и ЗО — примеры эпизодических задач, так как у них есть конечные состояния, включая четко определенные цель и отказ. У ЗО бесконечный горизонт планирования: агент рассчитывает на бесконечное количество шагов, но взаимодействие может завершиться в любой момент. Мы не будем добавлять ограничение по времени для этой среды, поскольку агент с высокой степенью вероятности завершит работу естественным путем, это очень стохастическая среда. В RL такие задачи встречаются чаще всего.

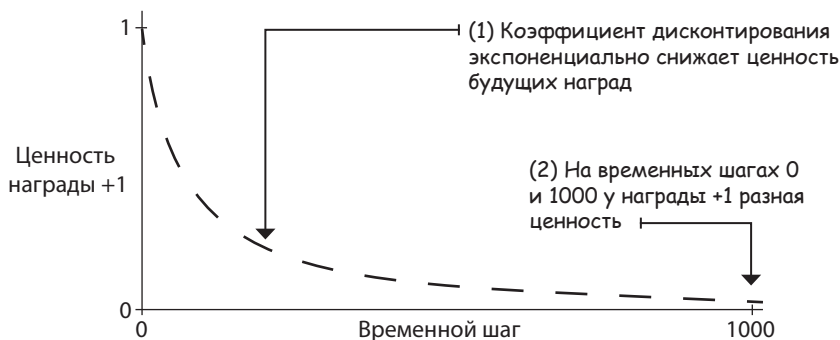
Последовательность временных шагов от начала эпизодической задачи и до самого ее конца называют *эпизодом*, *испытанием*, *периодом* или *стадией*. В бесконечных горизонтах планирования эпизод — это совокупность всех взаимодействий между начальным и конечным состояниями.

Дисконтирование: неопределенное будущее менее значимо

Поскольку в задачах с бесконечным горизонтом могут возникать бесконечные последовательности временных шагов, важно найти способ со временем снижать значение вознаграждения: то есть нам нужно как-то донести до агента, что получить +1 лучше раньше, чем позже. Для экспоненциального уменьшения значений будущих наград мы обычно используем положительные вещественные числа меньше 1. Чем позже мы получаем награду, тем ниже будет ее ценность.

Это называется *коэффициентом дисконтирования* или *гаммой*. Такой коэффициент позволяет регулировать ценность награды в зависимости от времени. Чем позже мы получаем вознаграждение, тем менее привлекательным оно становится для расчетов. Еще одна важная причина широкого использования коэффициента дисконтирования — сокращение дисперсии возвращаемых значений. Наше будущее не определено: чем дальше мы в него заглядываем, тем больше стохастичности накапливаем и тем шире разброс наших оценочных значений. Коэффициент дисконтирования помогает снизить влияние будущих наград на функцию ценности, что стабилизирует процесс обучения для большинства агентов.

Зависимость ценности награды от коэффициента дисконтирования и времени



Интересно, что гамма — это часть определения MDP: задача, а не агент. Но зачастую сложно найти информацию о том, какое значение гаммы подходит для заданной среды. Опять же это связано с тем, что она используется и как гиперпараметр для сокращения дисперсии, поэтому ее регулирование поручают агенту.

С помощью гаммы можно привить агенту чувство «срочности». Чтобы было проще понять, представьте, что я пообещал заплатить вам 1000 долларов, как только вы прочитаете эту книгу, но каждый день эта награда будет уменьшаться вдвое (то есть гамма равна 0,5). В этом случае вы бы, наверное, дочитали ее уже сегодня. Но если гамма равна 1, вы получите полную сумму независимо от того, когда вы закончите.

В средах БП и СБП подходит гамма 1, но в ЗО обычно применяется значение 0,99, поэтому мы выберем именно его.



Я знаю математику

Коэффициент дисконтирования (гамма)

(1) Сумма всех наград, полученных на протяжении эпизода, называется выгодой

$$\text{└───────────} \rightarrow G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

(2) Но также можно использовать и коэффициент дисконтирования, чтобы получить пониженную выгоду, которая снизит ценность наград, получаемых ближе к концу эпизода

$$\text{└} \rightarrow G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T$$

(3) Это уравнение можно упростить так

$$\text{────────────────────────────────} \rightarrow G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

(4) Теперь взгляните на это интересное рекурсивное определение. В следующей главе мы разберем эту формулу подробнее

$$\text{────────────────────────────────} \rightarrow G_t = R_{t+1} + \gamma G_{t+1}$$

Дополнения к MDP

Как уже упоминалось, у модели MDP есть много дополнений, позволяющих выполнять разные виды задач RL. Список ниже не исчерпывающий, но он поможет вам понять, насколько обширна эта область. Помните, что все эти типы обычно обозначают аббревиатурой MDP. То, что вы здесь видите, — лишь верхушка айсберга.

- Частично наблюдаемый марковский процесс принятия решений (partially observable Markov decision process, POMDP). Когда агент не может видеть полное состояние среды.
- Факторизованный марковский процесс принятия решений (factored Markov decision process, FMDP). Позволяет представить функции перехода и вознаграждения более компактно для поддержки представления крупных MDP.
- Марковский процесс принятия решений непрерывного времени/действия/состояния (continuous time/action/state Markov decision process). Когда время, действия, состояние или любое их сочетание непрерывные.
- Реляционный марковский процесс принятия решений (relational Markov decision process, RMDP). Допускает сочетание вероятностных и реляционных знаний.
- Полумарковский процесс принятия решений (semi-Markov decision process, SMDP). Допускает наличие абстрактных действий, на завершение которых может потребоваться несколько шагов.

- Децентрализованный марковский процесс принятия решений (decentralized Markov decision process, Dec-MDP). Позволяет нескольким агентам взаимодействовать для увеличения общего вознаграждения.



Я знаю Python

MDP «бандитская прогулка» (БП)

```

P = {
  0: {
    0: [(1.0, 0, 0.0, True)],
    1: [(1.0, 0, 0.0, True)]
  },
  1: {
    0: [(1.0, 0, 0.0, True)],
    1: [(1.0, 2, 1.0, True)]
  },
  2: {
    0: [(1.0, 2, 0.0, True)],
    1: [(1.0, 2, 0.0, True)]
  }
}

```

(1) Ключи внешнего словаря — это состояния

(2) Ключи внутреннего словаря — действия

(3) Значение внутреннего словаря — список всех возможных переходов для заданной пары «состояние — действие»

(4) У кортежей переходов четыре значения: вероятность перехода, следующее состояние, награда и флаг, определяющий терминальность следующего состояния

(5) MDP можно загрузить и вот так

```

# import gym, gym_walk
# P = gym.make('BanditWalk-v0').env.P

```



Я знаю Python

MDP «скользящая бандитская прогулка» (СБП)

```

P = {
  0: {
    0: [(1.0, 0, 0.0, True)],
    1: [(1.0, 0, 0.0, True)]
  },
  1: {
    0: [(0.8, 0, 0.0, True), (0.2, 2, 1.0, True)],
    1: [(0.8, 2, 1.0, True), (0.2, 0, 0.0, True)]
  },
  2: {
    0: [(1.0, 2, 0.0, True)],
    1: [(1.0, 2, 0.0, True)]
  }
}

```

(1) Взгляните на конечные состояния. Здесь это 0 и 2

(2) Так мы создаем стохастические переходы. Состояние 1, действие 0

(3) Это переходы, доступные после выполнения действия 1 в состоянии 1

(4) Вот так вы можете загрузить «скользящую бандитскую прогулку» в блокнот Jupyter (обязательно проверьте все пошагово!)

```

# import gym, gym_walk
# P = gym.make('BanditSlipperyWalk-v0').env.P

```



Я знаю Python

MDP «замёрзшее озеро» (30)

```

P = {
    0: [
        0: [(0.6666666666666666, 0, 0.0, False),
            (0.3333333333333333, 4, 0.0, False)
        ],
        3: [
            (0.3333333333333333, 1, 0.0, False),
            (0.3333333333333333, 0, 0.0, False),
            (0.3333333333333333, 0, 0.0, False)
        ],
    ],
    14: [
        1: [
            (0.3333333333333333, 13, 0.0, False),
            (0.3333333333333333, 14, 0.0, False),
            (0.3333333333333333, 15, 1.0, True)
        ],
        2: [
            (0.3333333333333333, 14, 0.0, False),
            (0.3333333333333333, 15, 1.0, True),
            (0.3333333333333333, 10, 0.0, False)
        ],
        3: [
            (0.3333333333333333, 15, 1.0, True),
            (0.3333333333333333, 10, 0.0, False),
            (0.3333333333333333, 13, 0.0, False)
        ],
    ],
    15: [
        0: [(1.0, 15, 0, True)],
        1: [(1.0, 15, 0, True)],
        2: [(1.0, 15, 0, True)],
        3: [(1.0, 15, 0, True)]
    ]
}

# import gym
# P = gym.make('FrozenLake-v0').env.P

```

(1) Вероятность попадания в состояние 0 при выборе действия 0 в состоянии 0

(2) Вероятность попадания в состояние 4 при выборе действия 0 в состоянии 0

(3) Вероятности можно группировать, как показано здесь

(4) Их можно записывать и явно, как в этих двух строках. Оба варианта рабочие

(5) Чтобы упростить этот пример, я максимально сократил код

(6) Полную модель MDP 30 можно найти в блокноте

(7) Ненулевую награду дает только состояние 14. У трех из четырех действий есть всего один переход, который ведет к состоянию 15, попадание в которое приносит награду +1

(8) Состояние 15 — конечное

(9) Опять же, вы можете загрузить MDP и вот так

Собираем все воедино

К сожалению, в реальности есть много разных определений MDP. Более того, в некоторых источниках POMDP описывают как MDP без каких-либо объяснений. Все это запутывает новичков. Поэтому сейчас я хочу прояснить несколько моментов.

Во-первых, код на Python, который вы видели выше, — это не полная модель MDP, а лишь набор переходных функций и сигналов вознаграждения, из которых мы можем легко вывести пространства состояний и действий. Эти фрагменты кода я взял из нескольких пакетов со средами, которые разработал для книги, а ZO — часть пакета OpenAI Gym, о котором я писал в первой главе. Некоторые дополнительные компоненты MDP, которых нет в приведенных выше словарях (такие как распределение начальных состояний S_0 , которое берется из множества начального состояния S^i), автоматически предоставляются фреймворком Gym и не показаны в этих примерах. Более того, другие отсутствующие элементы, такие как коэффициент дисконтирования γ и горизонт H , не отображаются в предыдущем словаре и не предоставляются даже фреймворком OpenAI Gym. Как я уже писал, коэффициенты дисконтирования обычно считаются гиперпараметрами, как бы мы к этому ни относились. А горизонт считают бесконечным.

Но не стоит волноваться. Во-первых, для расчета оптимальных политик MDP из этой главы (который мы произведем в следующей главе) нам нужен только показанный ранее словарь с функцией перехода и сигналом вознаграждения. Из них мы можем вывести пространства состояний и действий, а я дам коэффициенты дисконтирования. Мы будем считать горизонт бесконечным, и нам не понадобится распределение начальных состояний. Кроме того, основная цель этой главы — дать вам общее представление об элементах MDP и POMDP. Процесс построения MDP, которым вы будете заниматься в дальнейшем, не сильно отличается от описанного здесь. Но, чтобы у нас не возникало недопонимания, позвольте мне дать определения MDP и POMDP.



Я знаю математику

MDP и POMDP

$$\hookrightarrow MDP(S, A, T, R, S_0, \gamma, H)$$

(1) У моделей MDP есть пространство состояний S , пространство действий A , функция перехода T , сигнал вознаграждения R . Еще у них есть распределение начальных состояний S_0 , коэффициент дисконтирования γ и горизонт H

$$\hookrightarrow POMDP(S, A, T, R, S_0, \gamma, H, O, \epsilon)$$

(2) Чтобы определить POMDP, нужно добавить пространство наблюдений O и вероятность результата ϵ (вероятность показа наблюдения o в состоянии s_t). Все очень просто

Подведем итоги

Я знаю, что эта глава насыщена новыми терминами, но так и было задумано. Лучшее резюме для нее находится выше — там, где дается определение MDP. Вернитесь к последним двум уравнениям и попробуйте вспомнить, что означает каждая буква. Если у вас это получится, то будьте уверены: вы запомнили всю необходимую информацию и готовы двигаться дальше.

Основная задача обучения с подкреплением предполагает взаимодействие агента со средой, в которой он существует. Основываясь на этой информации, мы можем смоделировать множество других задач. Марковский процесс принятия решений (MDP) — это математическая модель для представления сложных задач, связанных с принятием решений в условиях неопределенности.

Он состоит из совокупности состояний системы, действий для каждого состояния, функции перехода, сигнала вознаграждения, горизонта, коэффициента дисконтирования и распределения начального состояния. Состояние описывает конфигурацию среды. Действия позволяют агенту взаимодействовать с ней. Функция перехода сообщает, как среда развивается и реагирует на действия агента. Сигнал вознаграждения определяет поставленную перед агентом цель. Горизонт и коэффициент дисконтирования вводят понятие времени в контекст взаимодействия.

Пространство состояний (множество всех возможных состояний) может быть конечным или бесконечным. Но число переменных, которые входят в отдельно взятое состояние, ограничено. Состояния могут быть полностью наблюдаемыми, но в более общей разновидности MDP — POMDP — они наблюдаемы частично. Это значит, что агент может наблюдать не все состояние системы целиком, а его видоизмененную версию, которую мы называем наблюдением.

Пространство действий — это множество действий, которое может варьироваться от состояния к состоянию, но на практике для всех состояний принято использовать одно и то же множество. Действия, как и состояния, могут содержать больше одной переменной. Эти переменные могут быть дискретными и постоянными.

Функция перехода связывает (следующее) состояние с парой «состояние — действие» и определяет вероятность попадания в это будущее состояние с учетом заданной пары. Сигнал вознаграждения в более общем виде сопоставляет кортеж переходов s, a, s' в скаляр и определяет успешность перехода. Сочетание переходной функции и сигнала вознаграждения составляют модель среды и считаются стационарными. Это значит, что на протяжении эпизода вероятность остается неизменной.

На этом этапе вы:

- знакомы с составляющими задачи обучения с подкреплением и понимаете, как они взаимодействуют;
- знаете, что такое марковский процесс принятия решений, из чего он состоит и как работает;
- умеете представлять последовательные задачи принятия решений в виде MDP.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

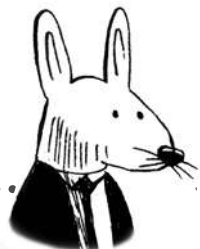
- **#gdr1_ch02_tf01.** Создание среды обучения — важный навык, которому можно посвятить целую книгу. Не хотите попробовать создать свою грид-среду? Можете взглянуть на «прогулочный» код из этой главы (<https://github.com/mimoralea/gym-walk>) или на другие примеры сеточных сред (<https://github.com/mimoralea/gym-aima>, <https://github.com/mimoralea/gym-bandits>, https://github.com/openai/gym/tree/master/gym/envs/toy_text). Теперь создайте пакет для Python с новой грид-средой! Не ограничивайтесь простыми действиями движения, вы можете создавать действия с «телепортацией» и многое другое. Помимо агента, в среду можно добавить парочку других существ. Можете создать небольших монстров, которых ваш агент должен избегать. Импровизируйте: ваши возможности не ограничены.
- **#gdr1_ch02_tf02.** Можете попробовать создать так называемую «среду спортзала» (Gym). Во-первых, выясните, что это такое. Далее изучите следующие пакеты Python (<https://github.com/openai/mujoco-py>, <https://github.com/openai/atari-py>, <https://github.com/google-research/football> и другие, доступные на странице <https://github.com/openai/gym/blob/master/docs/environments.md>). После этого попытайтесь разобраться в том, как другие разработчики предоставляют доступ к механизмам симуляции в виде сред Gym. В завершение создайте такую среду для любого механизма симуляции на ваш выбор. Это непростое упражнение!
- **#gdr1_ch02_tf03.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdr1_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы узнаете о проблемах обучения на основе последовательной обратной связи и о том, как правильно сбалансировать краткосрочные и долгосрочные цели.
- ✓ Разработаете алгоритмы, способные находить лучшие стратегии поведения в последовательных, связанных с принятием решений задачах, смоделированных в виде MDP.
- ✓ Подберете оптимальные политики для всех сред, для которых вы создали модели MDP в предыдущей главе.

При подготовке к битве я всегда убеждался в бессмысленности планов и важности планирования.

*Дуайт Эйзенхауэр (Dwight Eisenhower),
пятизвездный генерал армии
и 34-й президент США*

В предыдущей главе вы создали MDP для сред БП, СБП и ЗО. Модели MDP — это двигатели сред RL. Они определяют задачу: описывают, как агент взаимодействует со средой через пространства состояний и действий, цель агента через функцию вознаграждения, реакцию среды на действия агента через переходную функцию и влияние времени на поведение агента (через коэффициент дисконтирования).

В этой главе речь пойдет об алгоритмах выполнения задач на основе MDP. Вначале мы обсудим, какое задание стоит перед агентом и почему для решения MDP недостаточно простых планов. Затем поговорим о двух базовых алгоритмах решения MDP в рамках метода под названием «*динамическое программирование*»: об *итерации ценностей* (value iteration, VI) и *итерации политик* (policy iteration, PI).

Вскоре вы заметите, что эти методы не совсем «честные»: они нуждаются в полном доступе к MDP и зависят от знания динамики среды, которое нам не всегда удастся получить. Но базовые подходы, которые вы изучите, все равно пригодятся вам в работе с более продвинутыми алгоритмами. В целом VI и PI лежат в основе практически любого алгоритма RL (и DRL).

Вы также заметите, что, когда у агента есть полный доступ к MDP, нет никакой неопределенности, потому что вы можете вычислить ожидания напрямую из динамики и вознаграждений. Это избавляет нас от необходимости проведения исследований: то есть нам не нужно искать баланс между исследованием и эксплуатацией. Нет нужды во взаимодействии, поэтому вам не придется прибегать к обучению методом проб и ошибок. И все это из-за того, что обратная связь, которую мы используем для обучения в этой главе, не оценочная, а контролируемая.

Помните, в DRL агенты обучаются на одновременно последовательной (а не одинарной), оценочной (а не контролируемой) и выборочной (а не исчерпывающей) обратной связи. В этой главе я решил избавиться от сложностей, свойственных обучению на оценочной и выборочной обратной связях, и рассмотреть последовательную отдельно. Так мы будем обучаться на последовательной, контролируемой и исчерпывающей обратной связи.

Цель агента, принимающего решения

Сначала может показаться, что цель агента — это поиск последовательности действий, которые максимизируют выгоду: совокупность наград (дисконтированных или нет — в зависимости от значения гаммы), полученных на протяжении эпизода или всей жизни агента (в зависимости от задачи).

Чтобы конкретизировать эти понятия, рассмотрим новую среду.



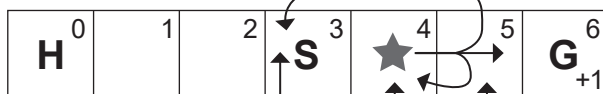
Конкретный пример

Скользкая прогулка пять (СПП)

«Скользкая прогулка пять» (СПП) — это однострочная грид-среда («прогулка»), которая, как и «замёрзшее озеро», стохастическая и содержит пять нетерминальных состояний (и два конечных — всего семь).

Среда «скользящая прогулка пять»

(1) Это стохастическая среда, поэтому, даже выбрав действие «вправо», агент может переместиться влево!



(4) В 16,66 % случаев агент движется в противоположном направлении

(3) В 33,33 % случаев агент остается на месте

(2) В 50 % случаев действие заканчивается успехом

Агент начинает в S . H — это дыра, а G — цель, несущая вознаграждение +1.



Я знаю математику

Выгода G

(1) Выгода — это сумма всех наград, полученных начиная с шага t и заканчивая конечным шагом T

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

(2) Как уже упоминалось, мы можем объединить выгоду и время с помощью коэффициента дисконтирования (гаммы). Тогда это будет дисконтированная выгода, где награда ценится тем больше, чем раньше она получена

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T$$

(3) Мы можем упростить и обобщить это уравнение вот так

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

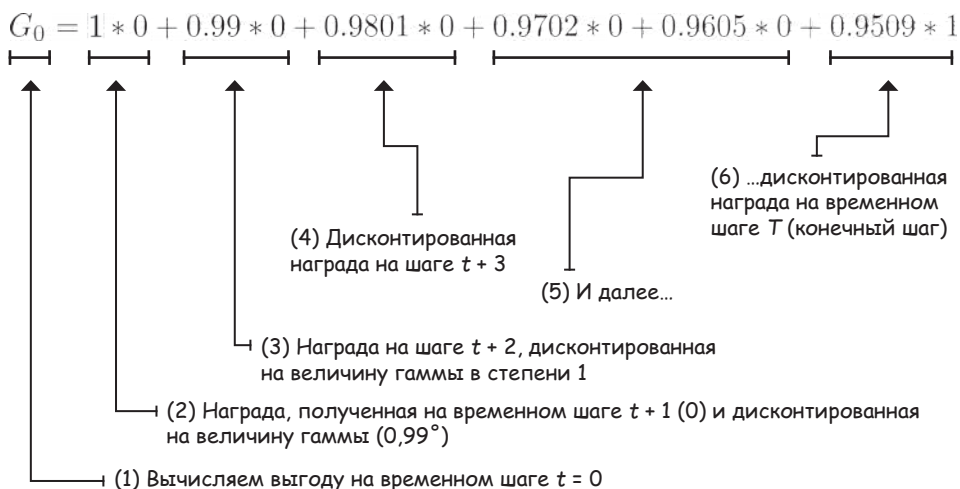
(4) Полюбуйтесь этим рекурсивным определением G

$$G_t = R_{t+1} + \gamma G_{t+1}$$

Выгода — это своего рода взгляд назад: «сколько мы получили» на прошлом временном шаге, но ее можно считать и «будущей наградой», взглядом вперед. Представьте, к примеру, следующий эпизод в среде СПП: состояние 3 (награда 0), состояние 4 (награда 0), состояние 5 (награда 0), состояние 4 (награда 0), состояние 5 (награда 0), состояние 6 (награда +1). Это можно записать сокращенно как 3/0, 4/0, 5/0, 4/0, 5/0, 6/1. Какова выгода этого эпизода/траектории?

Что ж, если использовать дисконтирование, процесс вычисления будет выглядеть так.

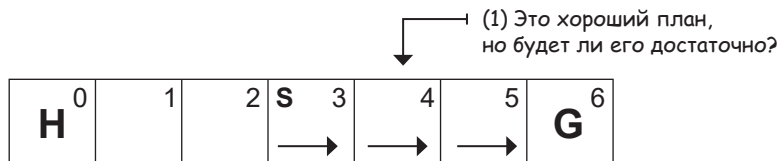
Дисконтированная выгода в среде «скользящая прогулка пять»



Если не применять дисконтирование, выгода будет равна 1 для этой и всех остальных траекторий, которые заканчиваются в крайней правой ячейке (состояние 6), и 0 для всех, которые заканчиваются в крайней левой (состояние 0).

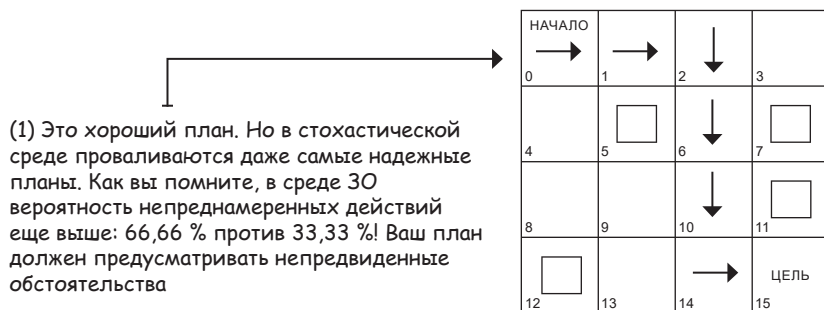
Мы видим, что в среде СПП движение вправо — лучший вариант. Поэтому может показаться, что агенту нужно найти некий *план* — последовательность действий от состояния НАЧАЛО до состояния ЦЕЛЬ. Но это работает не всегда.

Надежный план в среде СПП



В среде 3О план выглядел бы так.

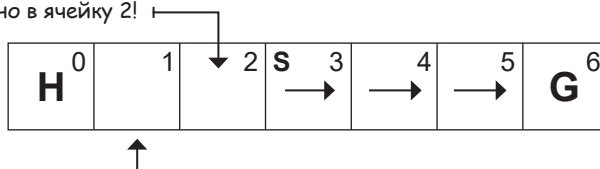
Надежный план в среде 3О



Но этого недостаточно! У планов есть слабое место: они не учитывают стохастичность среды. Из-за стохастичности СПП и 3О выбранные действия не всегда дают ожидаемый результат. Что, если из-за стохастичности среды наш агент попадет в ячейку, не предусмотренную планом?

Потенциальный изъян в нашем плане

(1) Допустим, агент действовал по плану, но уже после первого перехода был послан обратно в ячейку 2!



(2) Ну а теперь что? Вы не запланировали действие для состояния 2. Может, вам нужен план Б? В? Г?

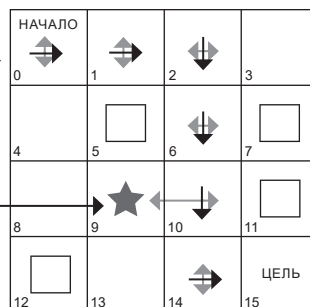
То же самое происходит и в среде 3О.

В стохастических средах одних лишь планов недостаточно

(1) Здесь я демонстрирую действие и его возможные последствия. Заметьте, непредвиденные обстоятельства возникают с вероятностью 66,66 %!

(2) Представьте, что агент действует по плану, но из состояния 10 среда отправляет его в 9, даже если он выбирает на первый взгляд правильное действие «вниз»

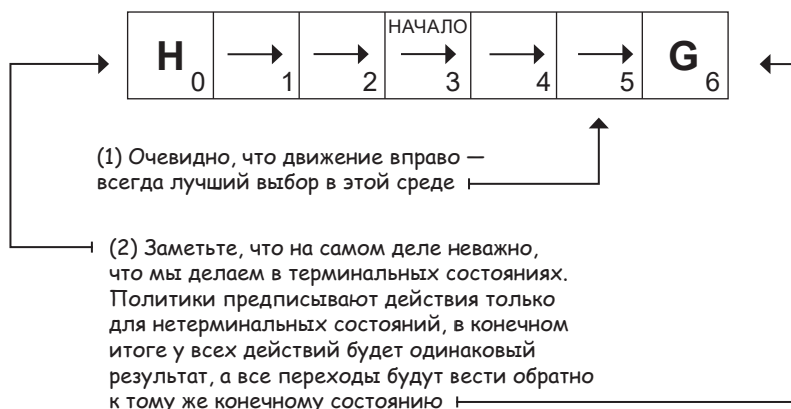
(3) Нам нужен план для каждого возможного состояния. Универсальный. Нам нужна политика



Агенту нужно выработать *политику* — универсальный план, который охватывает все возможные состояния. Это то, что нам нужно планировать для каждого возможного состояния.

Политики бывают стохастическими и детерминированными: первые возвращают для заданного состояния распределение «действие — вероятность», а вторые — просто действия. Пока мы будем иметь дело со вторыми. Они представляют собой таблицу поиска, которая связывает действия с состояниями.

Оптимальная политика в среде СПП



В среде СПП оптимальная политика всегда сводится к движению вправо, вне зависимости от состояния. Отлично. Но многие вопросы по-прежнему остаются без ответа. Например, какую награду ожидать от этой политики? Ведь, несмотря на то что мы знаем, какие действия нужно предпринять, среда может отправить наш агент в противоположном направлении прямоком в дыру, даже если мы всегда движемся к цели. Вот почему одной выгоды недостаточно. Агент действительно стремится максимизировать ожидаемую награду, что подразумевает возврат с учетом стохастичности среды.

Нам нужен и метод автоматического поиска оптимальных политик, так как в случае с ЗО, к примеру, совершенно неочевидно, какая из политик оптимальная!

В поиске оптимального поведения могут помочь несколько внутренних компонентов агента. Для заданной среды может существовать несколько политик. В некоторых средах даже может быть несколько оптимальных. Еще есть функции ценности, которые помогают отслеживать ожидаемую выгоду. Для любой модели MDP существует всего одна оптимальная функция ценности, но в целом их может быть много.

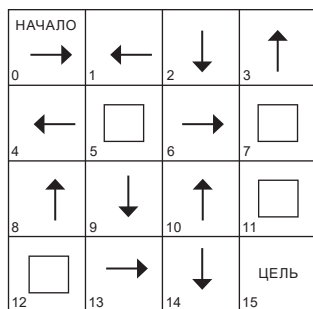
Рассмотрим все внутренние компоненты агента RL, которые помогают ему обучаться и находить подходящие политики. Разберем несколько примеров для наглядности.

Политики: предписания действий для каждого состояния

Агент должен найти стратегию (обозначим ее π) с учетом стохастичности среды «замерзшее озеро» (и большинства задач обучения с подкреплением). Политика — это функция, которая предписывает действия для заданного не-терминального состояния. Помните, стратегии могут быть стохастичными: либо в отношении самого действия, либо с использованием вероятностного распределения по действиям. Но об этом позже.

Вот пример простой политики.

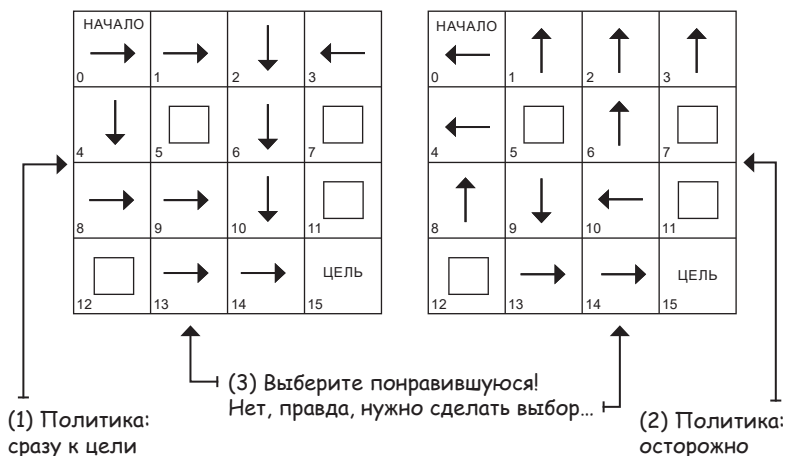
Случайно сгенерированная политика



(1) Политика, сгенерированная случайно, с равномерным распределением. Пока ничего особенного...

Глядя на эту политику, вы можете задаться вопросом: «Насколько она хороша?» Если бы у нас были определенные критерии оценки, можно было бы спросить, насколько эта стратегия лучше другой?

Как сравнивать стратегии?

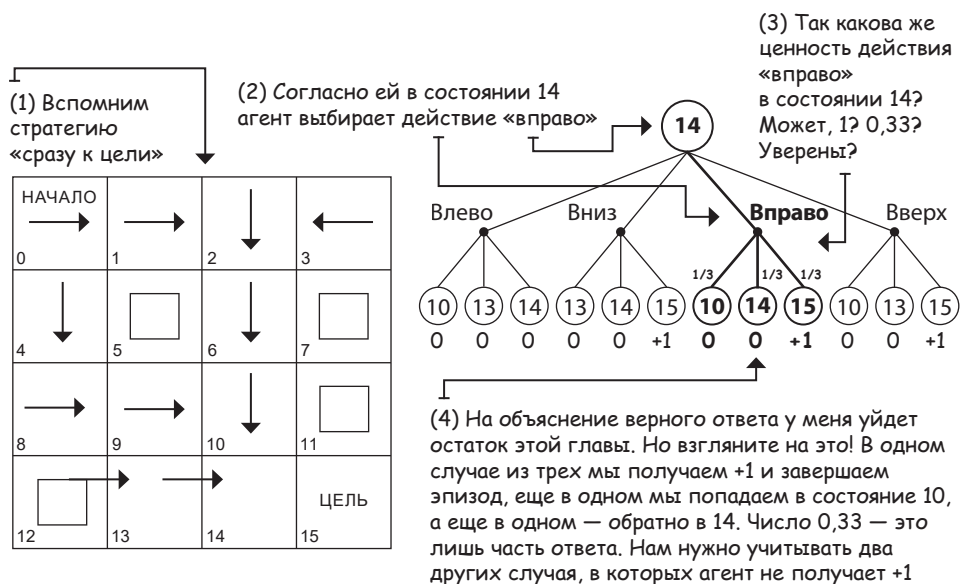


Функция оценки состояния: чего ожидать?

В сравнении политик нам может помочь оценка состояний каждой из них. То есть при наличии стратегии и MDP у нас должна быть возможность вычислить ожидаемую выгоду, начиная с каждого отдельно взятого состояния (нас в основном интересует состояние НАЧАЛО). Как определить ценность нахождения в том или ином состоянии? Например, если наш агент находится в состоянии 14 (слева от ЦЕЛИ), можно ли считать эту позицию лучше по сравнению с состоянием 13 (слева от 14)? И если да, то насколько? Что еще более важно, какая стратегия принесет лучший результат: «сразу к цели» или «осторожно»?

Проверим это на примере «сразу к цели». Какова ценность нахождения в состоянии 14 в рамках этой политики?

Насколько выигрышным будет нахождение в состоянии 14 при использовании политики «сразу к цели»?



Хорошо, вычислить ценность состояния 14 в рамках политики «сразу к цели» не так-то просто, учитывая зависимость от ценности других состояний (в данном случае 10 и 14), которая нам тоже неизвестна. Это похоже на проблему курицы и яйца. Двигаемся дальше.

Мы определили выгоду как сумму наград, получаемых агентом на определенной траектории движения. Теперь эту выгоду можно вычислить безотносительно выбранной агентом политики: достаточно суммировать все полученные награды. Но нас интересует ожидаемая выгода (в состоянии 14) при использовании заданной политики π . Помните, мы находимся в стохастической среде,

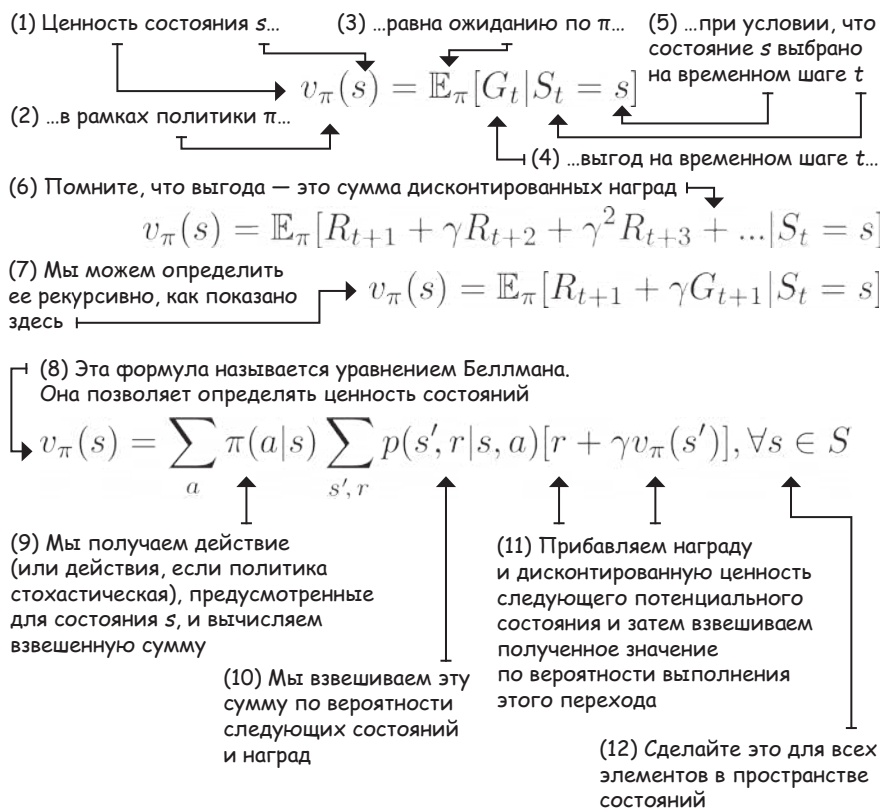
поэтому должны учитывать все возможные способы, которыми среда может отреагировать на нашу политику! Вот в чем заключается ожидание.

Теперь определим ценность состояния s при соблюдении политики π : это выгода, которая ожидается, если агент действует в соответствии с политикой π , начиная с состояния s . Вычислив это значение для каждого состояния, вы получите функцию ценности состояний или функцию ценности (или V-функцию — от англ. value function). Она отражает ожидаемую выгоду при соблюдении политики π , начиная с состояния s .



Я знаю математику

Функция ценности состояний V



Потрясающие уравнения. Немного запутанные, учитывая рекурсивные зависимости, но все равно интересные. Обратите внимание, как ценность состояния может рекурсивно зависеть от ценности многих других состояний, у которых, в свою очередь, тоже могут быть аналогичные зависимости, в том числе и от исходного состояния!

К рекурсивным отношениям между соседними состояниями мы еще вернемся в следующем разделе, где будут рассматриваться алгоритмы, способные последовательно решать эти уравнения и получать функции ценности любых стратегий в среде ЗО (или любой другой).

А пока продолжим исследовать другие компоненты, которые часто встречаются в агентах RL. Далее в этой главе мы узнаем, как рассчитать эти значения. Заметьте, что функцию ценности состояний нередко называют функцией ценности, V -функцией или просто $V_\pi(s)$. В этих терминах можно запутаться, но вы к ним привыкнете.

Функция ценности действий: чего ожидать при выборе определенного действия?

Еще один ключевой аспект, которым нужно интересоваться, касается не столько ценности состояния, сколько ценности выбора действия a в состоянии s . Сравнение полученных результатов поможет в выборе подходящего действия.

Например, заметьте, что, следуя политике «сразу к цели», при нахождении в состоянии 14 мы переходим вправо, а следуя политике «осторожно» — вниз. Какое из этих действий лучше? Точнее, какое из них лучше в рамках каждой политики? Какова ценность перехода вниз по сравнению с переходом вправо при выборе политики «сразу к цели» и какова ценность перехода вправо по сравнению с переходом вниз при выборе политики «осторожно»?

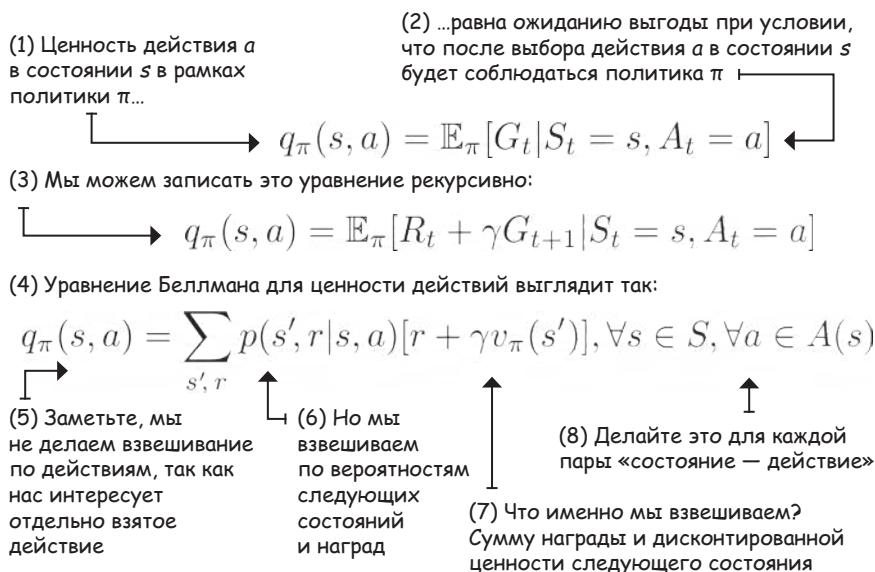
Сравнив разные действия в рамках одной стратегии, мы сможем выбрать лучшие и, следовательно, сделать нашу политику более оптимальной. Именно для этого предназначена *функция ценности действий*, известная как Q -функция или $Q^\pi(s, a)$: она возвращает ожидаемую выгоду при условии, что агент соблюдает политику π после выбора действия a в состоянии s .

На самом деле, если речь идет об улучшении стратегий (его часто называют задачей управления), нам нужны функции ценности действий. Задумайтесь: если у вас нет MDP, как вы можете решить, какое действие предпринять, просто зная ценность всех состояний? Q -функция, в отличие от V -функций, в некоторой степени учитывает динамику среды и позволяет оптимизировать политики при отсутствии MDP. Подробнее об этом мы поговорим в следующих главах.



Я знаю математику

Функция ценности действий Q



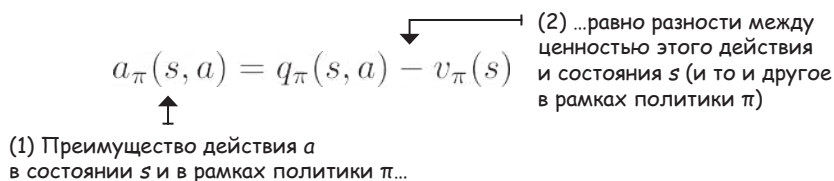
Функция преимущества действий: насколько лучше будет выбрать это действие?

Этот тип функции ценности основан на предыдущих двух. *Функция преимущества действий*, известная также как *функция преимущества*, *A-функция* или $A^{\pi}(s, a)$, возвращает разность между функцией ценности действий a в состоянии s и функцией оценки состояния s в рамках политики π .



Я знаю математику

Функция преимущества действий A



Функция преимущества показывает, насколько выгоднее предпринять действие a вместо следования политике π : преимущество выбора действия a по сравнению с действием по умолчанию.

Взгляните на разные функции ценности для (упрощенной) политики в среде СПП. Помните: показатели ценности зависят от политики. Иначе говоря, функция $Q_\pi(s, a)$ предполагает соблюдение политики π (в следующем примере это «всегда влево») сразу после выбора действия a в состоянии s .

Функции ценности состояний, действий и преимущества действий



(1) Обратите внимание, что $Q_\pi(s, a)$ позволяет нам улучшить политику π , показывая наиболее ценное действие в соответствии с ней

Оптимальность

Политики, функции ценности состояний, оценки действий и функции преимущества действий — это компоненты, с помощью которых мы описываем, оцениваем и оптимизируем поведение. Выбор лучших из доступных называется *оптимальностью*.

Оптимальная политика способна получить для каждого состояния ожидаемую выгоду, большую или равную той, которую дает любая другая политика.

Оптимальная функция оценки состояния обладает максимальной ценностью для всех политик и состояний, а ценности действий — для всех политик и пар «состояние — действие». Оптимальная функция преимущества действий работает по тому же принципу. Любопытно, что она меньше или равна нулю для всех пар «состояние — действие», так как ни у какого действия не может быть преимущества по сравнению с оптимальной функцией ценности состояний.

Кроме того, у модели MDP может быть несколько оптимальных стратегий, но только одна оптимальная функция ценности состояний, одна — оценки действий и одна — преимущества состояний.

Вы могли заметить, что при наличии оптимальной V-функции модель MDP позволяет выполнить одношаговый поиск оптимальной Q-функции и сформировать на основе полученных результатов оптимальную политику. Но если у вас есть оптимальная Q-функция, вам вообще не нужна MDP, ведь с ее помощью можно найти оптимальную V-функцию, если просто взять максимум действий. Точно так же можно получить оптимальную политику.



Я знаю математику

Уравнения оптимальности Беллмана



Планирование оптимальных последовательностей действий

V-функции позволяют отслеживать ценность состояний, Q-функции — ценность пар «состояние — действие», A-функции показывают преимущество выбора определенного действия. Для всего этого у нас есть уравнения, с помощью которых можно оценить текущие политики: получить из них функции ценности, найти среди них оптимальную и тем самым выбрать оптимальные стратегии.

Теперь, когда мы обсудили формулировку задачи обучения с подкреплением и определили цель, к которой стремимся, мы можем приступить к изучению методов ее достижения. Последовательное вычисление уравнений из прошлого раздела — один из самых распространенных способов выполнения задач RL и получения оптимальных политик в ситуациях, когда известна динамика среды, MDP. Давайте рассмотрим и остальные методы.

Оценка политик

Ранее мы уже обсуждали сравнение стратегий и установили, что политика π лучше или равна политике π' , если ее ожидаемая выгода выше или равна той, которой обладает π' во всех состояниях. Но прежде, чем мы сможем использовать это определение, мы должны разработать алгоритм оценки произвольной политики. Он называется *итеративной (последовательной) оценкой политик* или просто *оценкой политик*.

Этот алгоритм заключается в вычислении V-функции для заданной политики путем сканирования пространства состояний и последовательной оптимизации оценок. Тип алгоритмов, принимающих стратегию и возвращающих функцию ценности, предназначен для выполнения задачи *прогнозирования*: вычисления значения ценности заранее определенной политики.



Я знаю математику

Уравнение оценки политик

- (1) Алгоритм оценки политик состоит из итеративной аппроксимации V-функции, принадлежащей той политике, которую мы оцениваем. Алгоритм сходится по мере того, как k стремится к бесконечности
- (2) Инициализируем $v_0(s)$ для всех $s \in \mathcal{S}$, выбранных произвольно, и используем 0, если состояние s конечное. Затем увеличиваем k и постепенно повышаем значения в соответствии с приведенным ниже уравнением
- (3) Вычисляем ценность состояния s в виде взвешенной суммы награды и дисконтированной ожидаемой ценности следующего состояния s'

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_k(s')]$$

С помощью этого уравнения мы можем постепенно аппроксимировать истинную V-функцию произвольной политики. Алгоритм итеративной оценки политики гарантированно сходится к функции ее ценности при достаточном количестве итераций, то есть по мере приближения к бесконечности. На практике же мы используем небольшое пороговое значение для проверки изменений функции ценности, которую аппроксимируем. Как только эти изменения становятся меньше порогового значения, мы останавливаемся.

Посмотрим, как этот алгоритм работает в среде СПП для стратегии «всегда влево».

Начальные вычисления при оценке политики

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_k(s') \right]$$

(1) У нас детерминированная политика, поэтому эта часть равна 1

(2) Пусть гамма будет равна 1

(3) Политика «всегда влево» → π

H	←	←	←	←	←	G
0	1	2	3	4	5	6

НАЧАЛО

Состояние 5, итерация 1 (инициализировано как 0 на итерации 0):

$$v_1^{\pi}(5) = p(s'=4 | s=5, a=Left) * [R(5, Left, 4) + v_0^{\pi}(4)] +$$

$$+ p(s'=5 | s=5, a=Left) * [R(5, Left, 5) + v_0^{\pi}(5)] +$$

$$+ p(s'=6 | s=5, a=Left) * [R(5, Left, 6) + v_0^{\pi}(6)]$$

(4) Ага, это ценность состояния 5 после одной итерации значений политики ($v_1^{\pi}(5)$)

$$v_1^{\pi}(5) = 0.50 * (0 + 0) + 0.33 * (0 + 0) + 0.166 * (1 + 0) = 0.166$$

Теперь нужно вычислить ценность всех состояний, 0–6, и в конце перейти к следующей итерации. Обратите внимание, что для вычисления $V_2^{\pi}(s)$ вам придется использовать значения, полученные из прошлой итерации, $V_1^{\pi}(s)$. Такой подход к вычислению одного оценочного значения из другого называется «бэггинг» (bootstrapping) и широко применяется в RL (включая DRL).

Важно отметить, что значения k здесь — это итерации ценностей, а не взаимодействия со средой. Речь идет не об эпизодах, которые завершил агент, а о процессе выбора действий и наблюдении за средой. Значения k нельзя назвать временными шагами. Это скорее этапы выполнения итеративного алгоритма оценки политик. Вычислите еще несколько таких оценок. В следующей таблице показаны результаты, которые вы должны получить.

k	$V^n(0)$	$V^n(1)$	$V^n(2)$	$V^n(3)$	$V^n(4)$	$V^n(5)$	$V^n(6)$
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0.1667	0
2	0	0	0	0	0.0278	0.2222	0
3	0	0	0	0.0046	0.0463	0.2546	0
4	0	0	0.0008	0.0093	0.0602	0.2747	0
5	0	0.0001	0.0018	0.0135	0.0705	0.2883	0
6	0	0.0003	0.0029	0.0171	0.0783	0.2980	0
7	0	0.0006	0.0040	0.0202	0.0843	0.3052	0
8	0	0.0009	0.0050	0.0228	0.0891	0.3106	0
9	0	0.0011	0.0059	0.0249	0.0929	0.3147	0
10	0	0.0014	0.0067	0.0267	0.0959	0.318	0
...	...	...	...	...	...	...	...
104	0	0.0027	0.011	0.0357	0.1099	0.3324	0

Полученная функция ценности состояний сообщает нам следующее.

Прежде всего, мы видим, что в начале эпизода в этой среде и при условии использования стратегии «всегда влево» ожидается выгода в размере 0,0357. Это довольно низкий показатель.

Можно также сказать, что, даже когда мы оказываемся в состоянии 1 (крайнее левое нетерминальное состояние), у нас все равно есть шанс, пусть и менее одного процента, оказаться в ячейке ЦЕЛЬ (состояние 6). Если быть точными, вероятность такого исхода составляет 0,27 %. И при этом мы всегда стремимся влево! Интересно.

Примечательно, что из-за стохастичности этой среды вероятность достижения ячейки ЦЕЛЬ равна 3,57 % (как вы помните, действия в этой среде имеют 50%-ный шанс на успех; в 33,33 % случаев ничего не происходит, а еще в 16,66 % агент движется в обратном направлении). Повторюсь, что все это в рамках стратегии «всегда влево». Но действие «влево» может отправлять нас вправо снова, снова и снова, или же влево, вправо, вправо, вправо, вправо и т. д.

Подумайте о том, как сочетаются траектории вероятностей. Обратите внимание на итерации и на то, как значения пошагово распространяются в обратном направлении от награды (переход из состояния 5 в 6). Такое обратное распространение оценок — общая черта алгоритмов RL. Оно будет часто попадаться вам в этой книге.



Я знаю Python

Алгоритм оценки политик

```
def policy_evaluation(pi, P, gamma=1.0, theta=1e-10):
```

(1) Это полная реализация алгоритма оценки политик. Все, что нам нужно, — это политика, которую мы пытаемся оценить, и MDP, на котором она работает. Коэффициент дисконтирования, `gamma`, по умолчанию равен 1, а `theta` — это небольшое число, с помощью которого проверяется сходимость

```
    prev_V = np.zeros(len(P))
```

(2) Здесь мы инициализируем значения V-функции первой итерации с помощью значения 0

```
    while True:
```

(3) Начинаем с «бесконечного» цикла...

```
        V = np.zeros(len(P))
```

(4) Обнуляем значения на текущей итерации

```
        for s in range(len(P)):
```

(5) Затем перебираем все состояния, чтобы аппроксимировать функцию оценки

(6) Как видите, для получения всех возможных переходов используется политика `pi`

```
            for prob, next_state, reward, done in P[s][pi(s)]:
```

(7) У каждого кортежа переходов есть вероятность, следующее состояние, награда и флаг `done`, сигнализирующий о том, конечно ли состояние `next_state`

(8) Мы вычисляем ценность этого состояния, суммируя взвешенную ценность перехода

```
                V[s] += prob * (reward + gamma * \
                               prev_V[next_state] * (not done))
```

(9) Обратите внимание, что флаг `'done'` позволяет гарантировать, что при попадании в конечное состояние ценность следующего состояния равна нулю. Нам не нужны бесконечные суммы

```
            if np.max(np.abs(prev_V - V)) < theta:
```

(10) В конце каждой итерации (проверки состояний) мы убеждаемся, что функции оценки состояния меняются. Если это не так, мы считаем, что они сошлись

```
                break
```

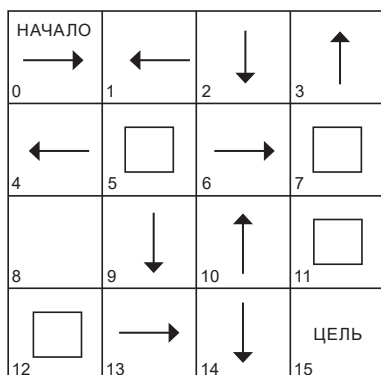
```
    prev_V = V.copy()
```

(11) В завершение мы выполняем «копирование», чтобы подготовиться к следующей итерации или вернуть последнюю функцию оценки состояния

```
    return V
```

Теперь проведем оценку случайно сгенерированной политики, представленной ранее для среды 3О.

Вспомним случайно сгенерированную политику

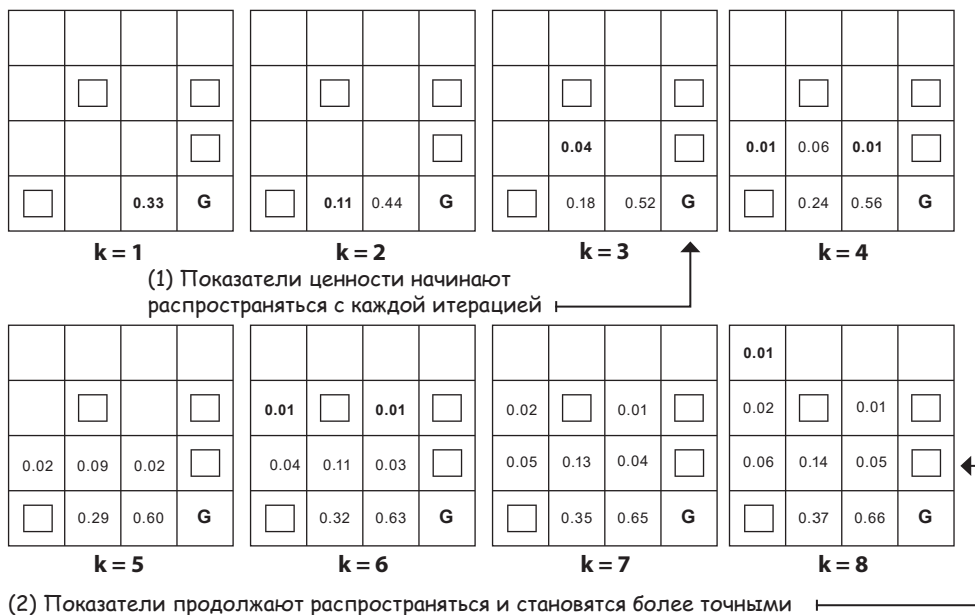


(1) Случайно сгенерированная стратегия...

(2) ...та же, что и прежде. Не нужно лишний раз листать назад!

Ниже показано, как всего после восьми итераций мы получаем точную аппроксимацию V-функции для случайно сгенерированной политики.

Оценка случайно сгенерированной политики для среды 3О



Функция ценности состояний для случайно сгенерированной политики

НАЧАЛО → 0.0955	← 0.0471	↓ 0.0470	↑ 0.0456
← 0.1469	□	→ 0.0498	□
↑ 0.2028	↓ 0.2647	↑ 0.1038	□
□	→ 0.4957	↓ 0.7417	ЦЕЛЬ

(1) После 218 итераций процесс оценки сходится к этим значениям (с использованием минимального изменения ценности $1e-10$ в качестве условия остановки)

Это итоговая функция ценности состояний для данной политики. Обратите внимание, что это всего лишь аппроксимация, так как мы имеем дело с дискретными пространствами состояний и действий, но, несмотря на это, результат можно считать настоящей функцией ценности состояний, когда гамма равна 0,99.

Если вам интересно, как выглядят функции ценности состояний для двух других политик, представленных ранее, взгляните на результаты ниже.

Результаты развития политики

«Сразу к цели»

НАЧАЛО → 0.0342	→ 0.0231	↓ 0.0468	← 0.0231
↓ 0.0463	□	↓ 0.0957	□
→ 0.0940	→ 0.2386	↓ 0.2901	□
□	→ 0.4329	→ 0.6404	ЦЕЛЬ

(1) Функция ценности состояний для этой политики сходится после 66 итераций. Политика достигает цели всего в 3,4 % случаев

«Осторожно»

НАЧАЛО ← 0.4079	↑ 0.3754	↑ 0.3543	↑ 0.3438
← 0.4263	□	↑ 0.1169	□
↑ 0.4454	↓ 0.4840	← 0.4328	□
□	→ 0.5884	→ 0.7107	ЦЕЛЬ

(2) Для этой политики функция ценности состояний сходится после 546 итераций. Политика достигает цели в 53,70 % случаев!

(3) Кстати, эти значения я рассчитал опытным путем, запустив политики 100 раз. Это искаженные значения, но общий смысл должен быть понятен

Похоже, что политика «сразу к цели» не очень-то себя оправдывает в среде 3О! Интересные результаты, не так ли? Но возникает вопрос: есть ли для этой среды политики лучше?

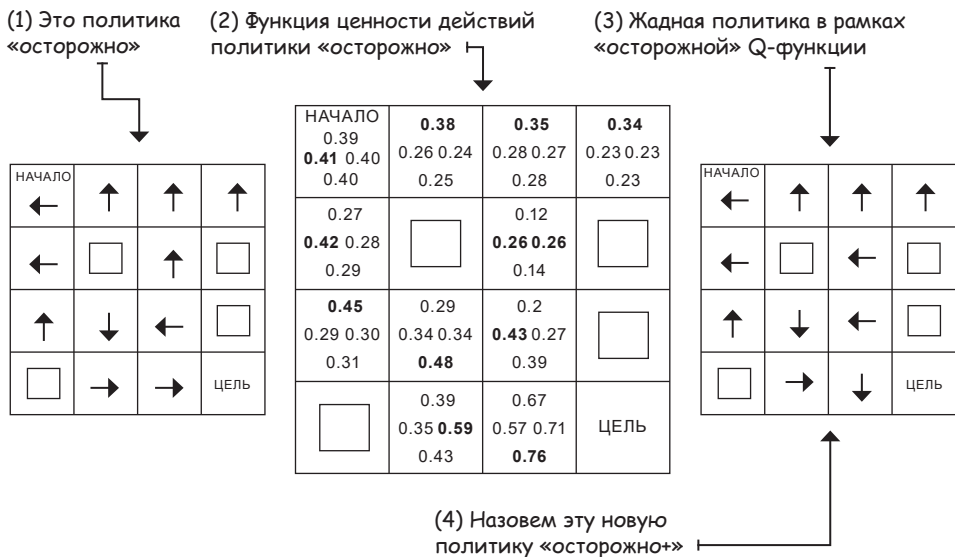
Оптимизация политик: использование оценок

Вы, наверное, уже догадались, зачем мы все это делали. У нас есть способ оценить любую политику. Это уже дает вам некоторую свободу: мы можем оценить многие политики и ранжировать их по V -функции в состоянии НАЧАЛО. Итоговое число говорит вам об ожидаемом совокупном вознаграждении, которое получит рассматриваемая политика, если вы запустите много эпизодов. Круто, правда?

Нет! Это бессмыслица! Зачем генерировать и оценивать кучу случайных политик? Это пустая трата ресурсов, но, что важнее, это не дает никакой гарантии получения все более оптимальных политик. Должен быть способ лучше.

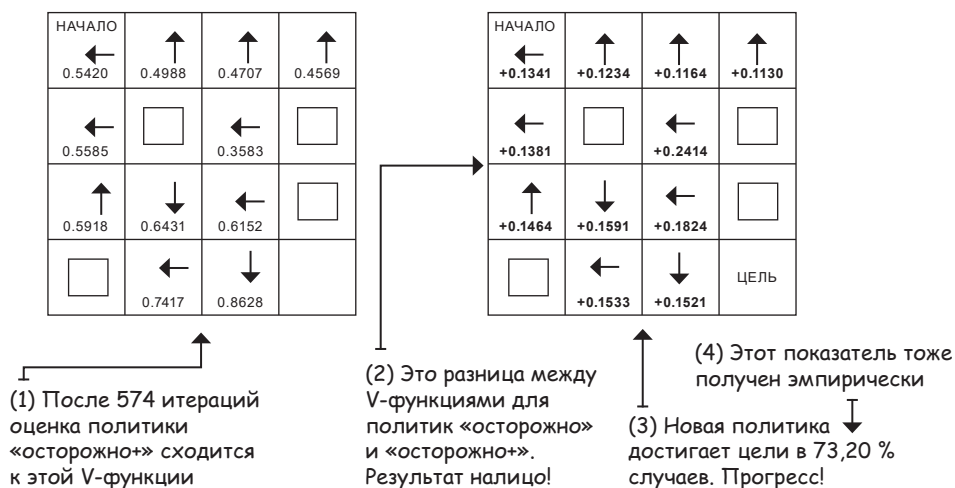
Ключ к решению этой задачи — функция ценности действий, Q -функция. Ее аппроксимацию можно получить с помощью V -функции и MDP. Q -функция даст вам общее представление о ценности всех действий во всех состояниях, что может помочь в улучшении политик. Взгляните на Q -функцию стратегии «осторожно» и на то, как ее можно оптимизировать.

Как Q -функция может помочь с оптимизацией политик?



Обратите внимание: если действовать жадно по отношению к Q -функции нашей политики, получится новая стратегия, «осторожно+». Лучше ли она прежней? Что ж, на этот вопрос нам поможет ответить процесс оценки стратегий. Давайте выясним!

Функция ценности состояний для политики «осторожно»



Новая политика лучше исходной. Отлично! Мы использовали MDP и функцию ценности состояний исходной политики для вычисления ее Q-функции. Затем, действуя жадно для функции ценности действий, мы получили улучшенную политику. Именно для этого и предназначен алгоритм *оптимизации политик*: он вычисляет функцию ценности действий на основе V-функции и MDP, после чего возвращает жадную версию Q-функции исходной политики. Хорошенько подумайте об этом.



Я знаю математику

Уравнение оптимизации политик

- (1) Для улучшения политики мы используем функцию ценности состояний и MDP. Это позволяет заглянуть на шаг вперед и определить, какие действия дают более высокую награду. Это уравнение оптимизации политик
- (2) Мы получаем новую политику π' , выбирая действие с наибольшим вознаграждением
- (3) Как получить это действие?
- $$\pi'(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) \left[r + \gamma v_{\pi}(s') \right]$$
- (4) Нужно вычислить взвешенную сумму всех наград и ценность всех возможных следующих состояний для каждого действия
- (5) Заметьте, что здесь используется действие с наибольшим вознаграждением Q-функции

Вот как алгоритм оптимизации политик выглядит в Python.



Я знаю Python

Алгоритм оптимизации политик

```
def policy_improvement(V, P, gamma=1.0):
```

(1) Это очень простой алгоритм. Он принимает функцию ценности состояний политики, которую нужно улучшить, V , и MDP, P (и гамму при необходимости)

```
    Q = np.zeros((len(P), len(P[0])), dtype=np.float64)
```

(2) Затем присваивает Q -функции значение ноль (теоретически вместо нуля можно использовать случайное значение, но давайте не будем усложнять)

```
    for s in range(len(P)):
        for a in range(len(P[s])):
            for prob, next_state, reward, done in P[s][a]:
```

(3) Проходится по состояниям, действиям и переходам

(4) Флаг, сигнализирующий о том, конечно ли состояние `next_state`

```
                Q[s][a] += prob * (reward + gamma * \
                                   V[next_state] * (not done))
```

(5) Эти показатели мы используем для вычисления Q -функции

```
    new_pi = lambda s: {s:a for s, a in enumerate(
                                   np.argmax(Q, axis=1))}[s]
```

(6) В конце мы получаем жадную политику, извлекая `argmax` Q -функции исходной. Как итог — новая, улучшенная политика

```
    return new_pi
```

Это наталкивает на некоторые вопросы. Есть ли более оптимальная политика? Можем ли мы сделать что-то лучше, чем «осторожно+»? Получится ли у нас оценить политику «осторожно+» и улучшить ее снова? Чтобы с уверенностью на них ответить, нужно сначала попробовать!

Можно ли улучшить политику «осторожно+»

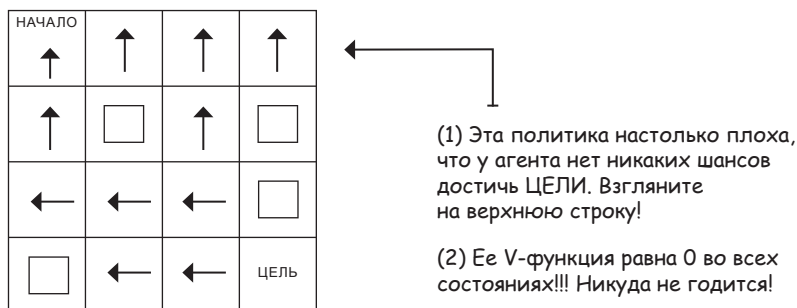


Я провел оценку политики «осторожно+» и затем попробовал ее улучшить. Q-функции «осторожно» и «осторожно+» отличаются, но жадные политики в рамках этих функций идентичны. Проще говоря, на этот раз улучшения не произошло.

Отсутствие улучшения объясняется тем, что «осторожно+» — оптимальная политика в среде 30 (когда гамма равна 0,99). Политика «осторожно» изначально была довольно хороша, поэтому нам понадобилось всего одно улучшение.

Теперь, даже если начать с состязательной политики, специально рассчитанной на плохие результаты, чередование оценки и улучшения сделает ее оптимальной. Не верите? Давайте проверим! Создадим альтернативную политику для среды 30 и посмотрим, что произойдет.

Состязательная политика для среды 30



Итерация политик: улучшение на основе улучшенного поведения

Наша цель: поочередно оценивать и улучшать состязательную политику, пока этап оптимизации не перестанет выдавать ее новые версии. Дело в том, что, если вместо состязательной политики выбрать случайно сгенерированную, алгоритм *интеграции политик* будет выглядеть так.



Я знаю Python

Алгоритм интеграции политик

```
def policy_iteration(P, gamma=1.0, theta=1e-10):
```

(1) Итерация политики проста и требует только MDP (включая гамму)

```
    random_actions = np.random.choice(
        tuple(P[0].keys()), len(P))
```

(2) Первым делом нужно получить случайно сгенерированную политику. Это все, что мы можем здесь сделать. Я создаю список случайных действий и затем привязываю их к состояниям

```
    pi = lambda s: {s:a for s, a in enumerate(
        random_actions)}[s]
```

```
    while True:
```

(3) Прежде чем изменять политику, нужно ее скопировать

```
        old_pi = {s:pi(s) for s in range(len(P))}
```

(4) Получаем функцию ценности состояний для данной политики

```
        v = policy_evaluation(pi, P, gamma, theta)
```

(5) Получаем улучшенную политику

```
        pi = policy_improvement(V, P, gamma)
```

(6) Теперь проверяем, отличается ли новая политика от прежней

```
        if old_pi == {s:pi(s) for s in range(len(P))}:
            break
```

(7) Если да, мы повторяем все заново

(8) Если нет, мы выходим из цикла, возвращая оптимальные политику и функцию ценности состояний

```
    return V, pi
```

Отлично! Давайте опробуем этот алгоритм на состязательной политике и посмотрим, что получится.

Оптимизация состязательной политики 1/2

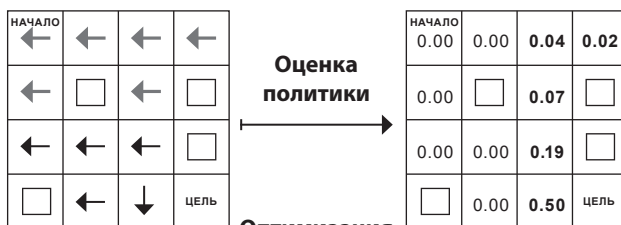
Состязательная политика

Вероятность
успеха: 0,00 %



Оптимизация
политики

Вероятность
успеха: 0,00 %



Оптимизация
политики

Вероятность
успеха: 0,00 %



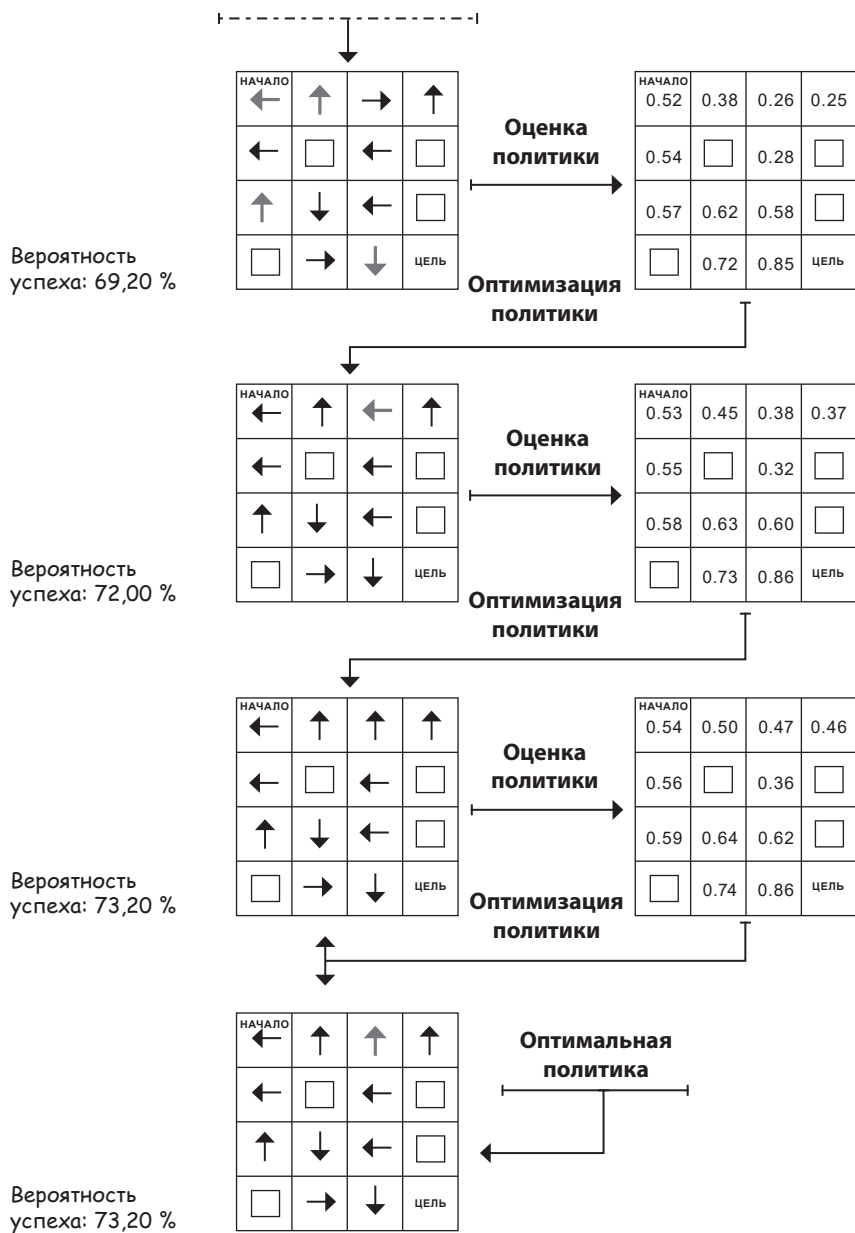
Оптимизация
политики

Вероятность
успеха: 13,60 %



Оптимизация
политики

Оптимизация состязательной политики 2/2

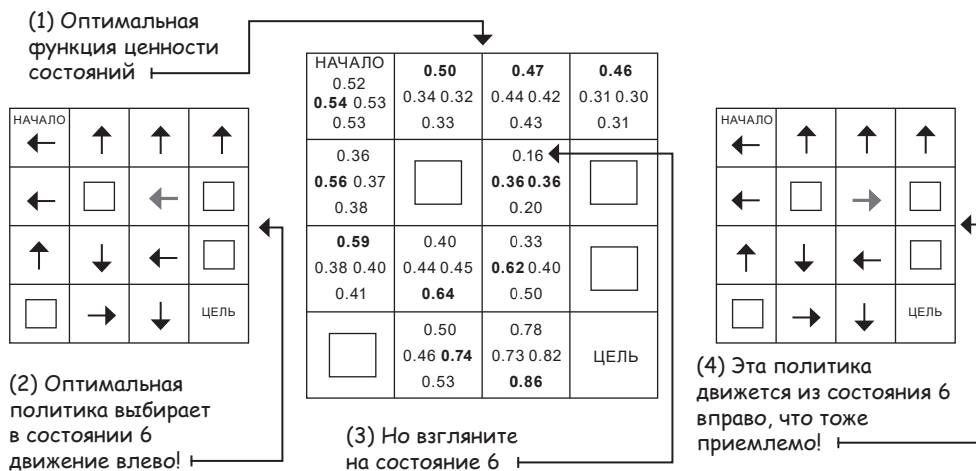


Как уже упоминалось, чередование оценки и улучшения позволяет получить одну из оптимальных политик и оптимальную V-функцию вне зависимости от исходной политики. Я бы хотел сделать несколько замечаний относительно этого утверждения.

Обратите внимание на то, что оптимальная политика лишь «одна из», тогда как оптимальная V-функция единственная. Я специально так выразился. На самом деле я еще раз хочу подчеркнуть это свойство. У MDP может быть несколько оптимальных политик, но только одна оптимальная функция ценности состояний. И это вполне логично.

Функции ценности состояний — это наборы чисел. V-функция может быть только одна (набор самых больших чисел для всех состояний). Но у нее, даже если она оптимальная, могут быть действия, одинаково ценные для данного состояния. В этом случае может быть несколько оптимальных политик, каждая из которых выбирает разные, но одинаково ценные действия. Отличный пример — среда 3О.

У среды 3О есть больше одной оптимальной политики



Кстати, здесь не видно, но ценность всех действий в конечном состоянии одинаковая, нулевая, а значит, у них есть и проблема, аналогичная упомянутой в состоянии 6.

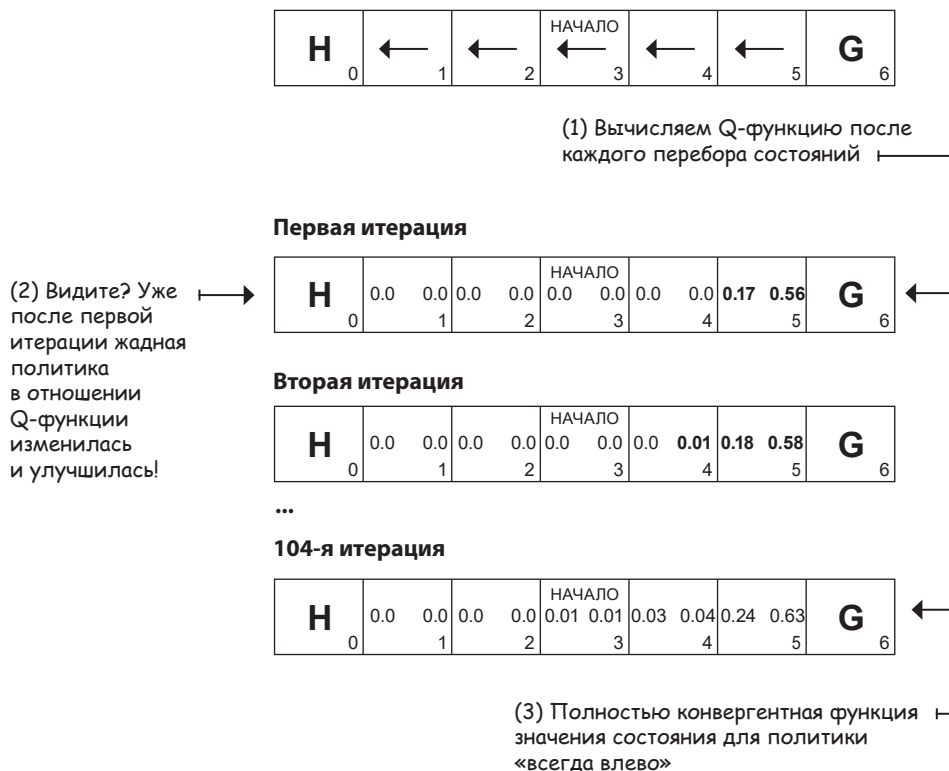
И последнее: процесс итерации гарантированно сходится строго на оптимальной политике. Математически доказано, что он не застрянет в локальном оптимуме. Но с практической точки зрения есть один аспект, с которым нужно быть осторожными. Если у функции ценности состояний есть равнозначные действия (как «влево» и «вправо» в состоянии 6), делать выбор между ними нужно не случайным образом. Иначе процесс оптимизации может выдавать разные политики, даже если никакого реального улучшения нет. С этим разобрались.

Теперь рассмотрим еще один ключевой алгоритм поиска оптимальных V-функций и политик.

Итерация ценностей: улучшение поведения на ранних этапах

Вы, наверное, обратили внимание на то, как происходит оценка политик: значения распространяются последовательно на каждой итерации, но медленно. Давайте посмотрим.

Оценка политики «всегда влево» в среде СПП

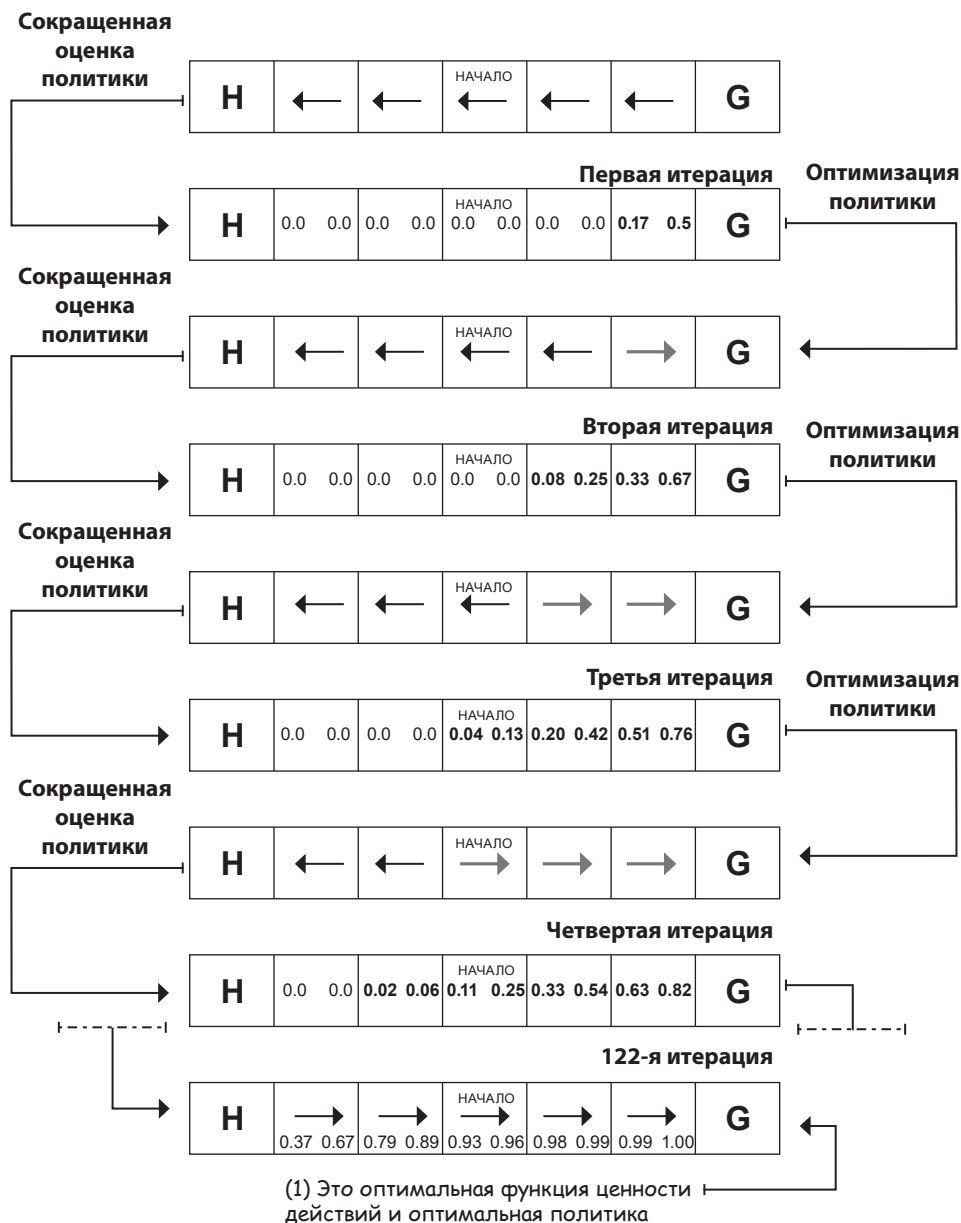


На схеме показан один проход по пространству состояний при оценке политики, за которым следует оценка Q-функции. Для этого мы используем усеченное значение V-функции и MDP на каждой итерации. Оценка помогает увидеть, что жадная политика в отношении ранних аппроксимаций Q-функции улучшается уже после первой итерации. Взгляните на значения Q-функции в состоянии 5 на первой итерации: очевидно, что движение к ЦЕЛИ — это уже улучшение.

Иначе говоря, даже ограничив процесс оценки единственной итерацией, можно улучшить исходную политику. Для этого мы применяем жадную политику в отношении значения Q-функции после первого же прохода по пространству состояний. Это еще один базовый алгоритм RL — *итерация ценностей* (value iteration, VI).

VI можно считать жадной политикой жадности, так как мы вычисляем жадную политику при первой возможности (жадно). VI не ждет точной оценки политики, прежде чем улучшать ее, а усекает фазу оценки политики после однократной проверки состояния. Вот что я имею в виду под жадной политикой жадности.

Жадная политика жадности «всегда влево» в среде СПП



Если в среде СПП вместо состязательной политики «всегда влево» выбрать случайно сгенерированную, алгоритм VI все равно сойдется на оптимальной функции ценности состояний. Этот алгоритм довольно прост, и его можно выразить с помощью одного уравнения.



Я знаю математику

Уравнение итерации ценностей



На самом деле при использовании VI вообще не нужно иметь дело с политиками. У VI нет отдельной стадии оценки, рассчитанной на сходимость. У VI та же цель, что и у PI: найти оптимальную политику для заданной модели MDP. Но алгоритм VI делает это с использованием функций ценности, отсюда и название «итерация ценностей».

Опять же нас интересуют только V- и Q-функция (в зависимости от реализации). Помните: чтобы добиться жадной политики в отношении Q-функции, нужно найти аргументы максимумов (argmax) для действий этой функции. Вместо того чтобы оптимизировать политику через поиск argmax с последующей ее оценкой для получения еще одной V-функции, мы сразу вычисляем максимальную ценность (max вместо argmax) для действий, которые будут применены при следующем проходе по состояниям.

Только в конце алгоритма VI, после того как Q-функция сходится к оптимальным значениям, мы извлекаем оптимальную политику, вычислив argmax для действий Q-функции, как и раньше. Я показал это наглядно во фрагменте кода ниже.

Важно отметить, что, хоть VI и PI и разные алгоритмы, в целом их можно считать двумя разновидностями *обобщенной итерации политик* (generalized policy iteration, GPI). GPI — это базовая концепция обучения с подкреплением, в рамках которой политики улучшаются через аппроксимацию их функций ценности, нацеленную на получение настоящей V-функции для текущей политики. Ждете вы идеальных оценок или нет, это уже детали.



Я знаю Python

Алгоритм итерации ценностей

```
def value_iteration(P, gamma=1.0, theta=1e-10):
```

(1) Наряду с итерацией политик итерация ценностей — это еще один метод получения оптимальных политик. Для этого нам нужна модель MDP (с гаммой). θ — это критерий сходимости: $1e-10$ дает достаточно высокую точность

```
    V = np.zeros(len(P), dtype=np.float64)
```

(2) Сначала нужно определить функцию ценности состояний. Помните, что V -функция должна хорошо работать со случайными числами

```
    while True:
```

(3) Мы входим в этот цикл и обнуляем Q -функцию

(4) Обратите внимание, что это значение здесь должно быть равно нулю, иначе мы получим неправильные результаты

```
        Q = np.zeros((len(P), len(P[0])), dtype=np.float64)
```

(5) Затем для каждого перехода каждого действия в каждом состоянии мы...

```
        for s in range(len(P)):
            for a in range(len(P[s])):
```

(6) ...вычисляем V -функцию...

```
                for prob, next_state, reward, done in P[s][a]:
```

(7) ...здесь используется V — старая усеченная оценка

```
                    Q[s][a] += prob * (reward + gamma * \
                                V[next_state] * (not done))
```

```
        if np.max(np.abs(V - np.max(Q, axis=1))) < theta:
            break
```

(8) После каждого прохода по пространству состояний мы проверяем, продолжает ли изменяться V -функция. Если нет, значит, мы нашли оптимальную и должны покинуть цикл

```
    V = np.max(Q, axis=1)
```

(9) Благодаря этой короткой строчке нам не нужна отдельная стадия оптимизации политики. Это не ее прямой аналог, а, скорее, сочетание улучшения и оценки

```
    pi = lambda s: {s:a for s, a in enumerate(
                                np.argmax(Q, axis=1))}[s]
```

```
    return V, pi
```

(10) Только в конце мы извлекаем оптимальную политику и возвращаем ее вместе с оптимальной функцией ценности состояний

Подведем итоги

Цель агента обучения с подкреплением — максимизировать ожидаемую выгоду, то есть общую награду за несколько эпизодов. Для этого ему нужно использовать политики, которые можно считать универсальными планами. Политики предписывают выбор действий в определенном состоянии. Они могут быть детерминированными (возвращать одиночные действия) и стохастическими (возвращать вероятностные распределения). Для получения политики агент обычно сравнивает несколько итоговых оценок, из которых основные — это функции ценности состояний, ценности действий и преимущества действий.

V-функции обобщают ожидаемую от состояния выгоду и показывают, какую награду должен получить агент в этом состоянии до завершения эпизода. Q-функции обобщают выгоду, ожидаемую от пары «состояние — действие», и позволяют узнать, какая награда ожидается в результате выбора агентом определенного действия в заданном состоянии. Благодаря Q-функциям агент может сравнивать доступные действия, решая тем самым задачи управления. A-функции показывают агенту, насколько лучше будет выбрать определенную пару «состояние — действие» по сравнению с тем, что предлагается по умолчанию. Все эти функции ценности относятся к конкретным политикам, одна из которых может быть оптимальной, и зависят от их соблюдения до окончания эпизода.

Оценка — это метод аппроксимации функции ценности на основе политики и MDP. Оптимизация — это метод извлечения жадной политики из функции ценности и MDP. Итерация — это чередование процессов оценки и оптимизации для извлечения из MDP оптимальной политики. Прежде чем мы сможем получить точную аппроксимацию функции ценности для заданной политики, стадия оценки должна пройти несколько итераций, пока процесс оценки не вернет точный результат. Альтернативный метод, итерация ценностей, сокращает фазу оценки и как можно раньше переходит к фазе оптимизации.

В целом все эти методы можно отнести к обобщенной итерации, которая описывает взаимодействие двух процессов для оптимизации политик: один приближает аппроксимацию функции ценности к ее настоящей версии для текущей политики, а другой оптимизирует текущую политику с помощью аппроксимации ее функции ценности, получая все более оптимальные политики по мере выполнения этого цикла.

На этом этапе вы:

- знаете основную цель агента обучения с подкреплением и то, какими статистическими данными он может обладать в любой момент времени;
- понимаете методы аппроксимации функций ценности на основе политик и оптимизации политик на основе функций ценности;
- умеете искать оптимальные политики в рамках последовательных задач принятия решений, смоделированных с помощью MDP.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

- **#gdr1_ch03_tf01.** У многих грид-сред есть MDP, которые можно выполнить с помощью функций итерации политик и значений из этой главы. Удивлены? Возьмите переменную `env.unwrapped.P` и передайте ее рассмотренным здесь функциям. Можете сделать это в средах, которые мы не использовали: взять среды, созданные кем-то другим, или те, что были созданы вами ранее.
- **#gdr1_ch03_tf02.** Коэффициент дисконтирования γ был введен в предыдущей главе как часть определения MDP. Но мы не до конца разобрали эту важную переменную. Попробуйте выполнить итерацию политик и оценок с разными значениями гаммы и записать сумму наград, которые агент получает в каждом случае, и оптимальные политики. Как они соотносятся? Возможно, у вас есть какая-то интересная информация, которая поможет другим лучше понять роль коэффициента дисконтирования?
- **#gdr1_ch03_tf03.** Итерация политик и оценок — это, по сути, один процесс: мы берем определение модели MDP и находим решение для оптимальных функции оценки и политик. Но есть ли разница в этих подходах? Можете ли вы придумать MDP, сложный для итерации политики и простой для решения итерации ценности и наоборот? Создайте такую среду в виде пакета Python и поделитесь им с нами. Возможно, вы узнали что-то, чем хотите поделиться? Как соотносятся VI и PI?
- **#gdr1_ch03_tf04.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdr1_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы узнаете о трудностях обучения на оценочной обратной связи и научитесь находить правильный баланс между сбором и использованием информации.
- ✓ Разработаете стратегии исследования, накапливающие небольшое количество потерь в задачах, где функция перехода и сигналы вознаграждения неизвестны.
- ✓ Напишете код с агентами, которые методом проб и ошибок учатся оптимизировать свое поведение с учетом опыта в средах с выбором одного из многих вариантов — «многоруких бандитах».

Неопределенность и ожидания приносят радость, в то время как безопасность остается абсолютно бесполезной.

*Уильям Конгрив (William Congreve),
английский драматург и поэт эпохи
классицизма, политический деятель
Либеральной партии Великобритании*

Каким бы мелким и незначительным ни казалось решение, это всегда будет компромисс между сбором и использованием информации. Например, находясь в любимом ресторане, вы можете в очередной раз заказать блюдо, которое вам нравится, или же попробовать что-то новое. Если вам поступит предложение поучаствовать в стартапе в Кремниевой долине, вы можете принять его или остаться в своей текущей компании.

Такие выборы иллюстрируют дилемму исследования и эксплуатации, которая лежит в основе задач обучения с подкреплением. Ее суть в том, чтобы решить, когда приобретать новые знания, а когда использовать уже приобретенные. Не так-то просто понять, достаточно ли нам того, что мы уже знаем. Когда нужно остановиться, а когда стоит научиться чему-то новому? Что лучше: синица в руках или журавль в небе?

Основная проблема в том, что приятные моменты в жизни относительны: чтобы понять их ценность, их нужно сравнивать между собой. Например, могу поспорить, что вы были в восторге, когда вас впервые приняли на работу. Вам могло даже показаться, что это лучшее из всего, что с вами случилось. Но жизнь на этом не останавливается, и вы переживаете еще более волнительные моменты: повышение в должности, получение прибавки к зарплате или женитьба — кто знает!

И в этом главная сложность: даже если вы ранжируете прожитые моменты по степени их «удивительности», вы не можете знать, какой из них самый волнительный в вашей жизни, ведь она полна неопределенностей. В реальном мире у вас нет переходной функции и сигнала вознаграждения, поэтому вам приходится постоянно исследовать окружающий мир. В этой главе вы узнаете, насколько важен процесс познания для агента, взаимодействующего с неопределенной средой и задачами, в которых MDP недоступна для планирования.

Ранее вы узнали о трудностях обучения на последовательной обратной связи и научились балансировать между краткосрочными и долгосрочными целями. Здесь же мы рассмотрим сложности обучения на оценочной обратной связи не в последовательных, а в одинарных средах — *«многоруких бандитах»* (multi-armed bandits, MABs).

MABs изолируют и выявляют задачи обучения на оценочной обратной связи. Мы подробно исследуем разные методики балансирования исследования и эксплуатации в конкретных типах сред: с единственным состоянием и несколькими вариантами, из которых нужно выбрать один. Агенты будут

действовать в условиях неопределенности — у них не будет доступа к MDP. При этом они будут взаимодействовать с одинарными средами без последовательного компонента.

Как вы помните, в DRL агенты обучаются на одновременно последовательной (а не одинарной), оценочной (а не контролируемой) и выборочной (а не исчерпывающей) обратной связи. В этой главе мы избавимся от сложности, которую приносят первый и последний типы обратной связи, и отдельно исследуем все тонкости оценочного типа. Итак, приступим.

Проблема интерпретации оценочной обратной связи

В предыдущей главе при решении задач в среде ЗО мы заранее знали, как она будет реагировать на любые наши действия, какие у этой среды функция перехода и сигнал вознаграждения. Это позволяло нам вычислять оптимальные политики с использованием алгоритмов планирования вроде PI и VI без какого-либо взаимодействия со средой.

Но со знанием MDP заранее все упрощается, возможно, даже слишком. Мы не всегда можем быть уверены в том, что нам будет известна реакция среды на любые наши действия — в жизни все не так просто. Мы можем попытаться изучить эту реакцию, как вы позже сами убедитесь. Наша основная задача: позволить нашим агентам самостоятельно взаимодействовать со средами и, познавая их, вырабатывать оптимальное поведение только на собственном опыте. Это и называется обучением методом проб и ошибок.

Когда агент RL вырабатывает поведение на основе взаимодействия со средой, она каждый раз задает ему один и тот же вопрос: «Что вы хотите делать сейчас?» Для агента, принимающего решения, ответ на этот вопрос — основная задача. Какое действие он должен выбрать на этот раз? Может быть, ему стоит воспользоваться приобретенными знаниями и сделать выбор в пользу действия с наивысшей текущей ценностью? Или лучше изучить действия, которые еще не были опробованы как следует? Но это влечет за собой дополнительные вопросы: «Откуда мы знаем, достаточно ли хороши наши значения?», «Откуда знаем, что действие, которое плохо себя показало, было как следует опробовано?» и т. д.

Вы познакомитесь с более эффективными способами поиска баланса между исследованием и эксплуатацией

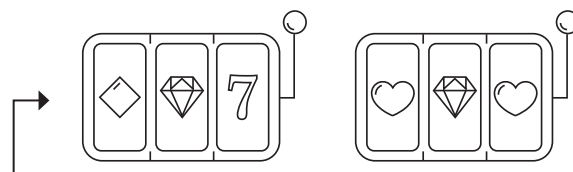


Ключевой момент здесь — то, что на этапе исследования накапливаются знания, которые затем можно эффективно использовать, и любой агент, принимающий решения, всегда стремится сделать это.

Бандиты: задачи принятия решений с одним состоянием

«Многорукые бандиты» (multi-armed bandits, MAB) — это частный случай задачи RL, где размер пространства состояний и горизонта равен 1. У MAB есть несколько действий, одно состояние и жадный горизонт. Можно сказать, что это среда с несколькими вариантами, из которых нужно выбрать один. Название происходит от игровых автоматов (их иногда называют бандитами) с несколькими рычагами на выбор (в реальности выбор происходит между несколькими игральными автоматами).

Задача «многорукого бандита»



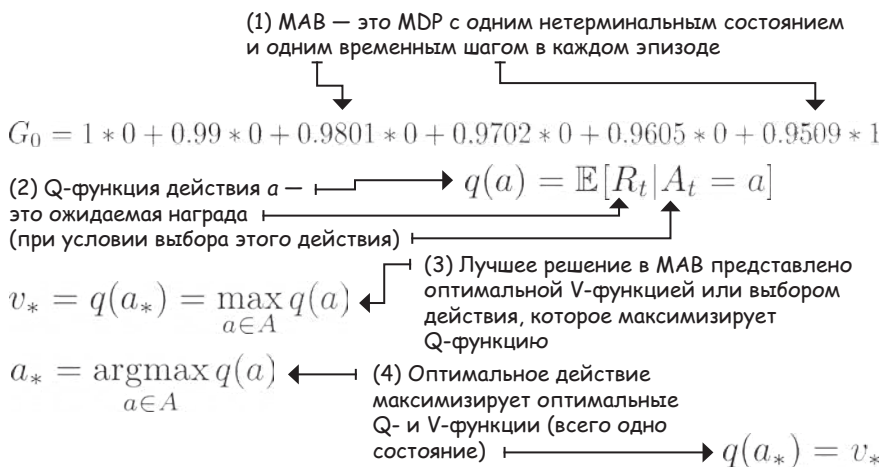
(1) «Двурукий бандит» — это задача принятия решений с двумя вариантами, для оценки которых вы должны опробовать оба. Как лучше всего подойти к поиску компромисса между исследованием и эксплуатацией?

Есть много коммерческих приложений для методов, вытекающих из исследований МАР. Многим компаниям нужно правильно сбалансировать показ рекламы, по которой вы перейдете, и новых объявлений, которые могут вас заинтересовать. Сайты по сбору денежных средств от благотворительных или политических организаций должны балансировать между отображением макета, который привел к наибольшему количеству пожертвований, и новым дизайном, который не использовался часто, но несет в себе потенциал для лучших результатов. Интернет-магазин должен рекомендовать вам самые продаваемые товары, но в то же время предлагать что-то новое. В ходе клинических исследований важно как можно скорее выявить эффект, оказываемый лекарством на пациентов. Изучение баланса между исследованием и эксплуатацией способствует поиску более эффективных решений во многих областях, включая нефтедобычу, видеоигры, поисковые системы и т. д. Нам задачи МАР интересны не столько с точки зрения практического применения, сколько в качестве подходящего метода для нахождения баланса между исследованием и эксплуатацией в агентах RL.



Я знаю математику

Многорукий бандит



Потеря: цена исследования

Цели МАР и RL очень похожи. В RL агенту нужно максимизировать ожидаемую совокупную дисконтированную награду (ожидаемую выгоду). Это значит, что на протяжении эпизода (совокупно) он должен получить как можно

большую (максимальную) награду как можно раньше (более поздние награды дисконтируются сильнее) вопреки стохастичности среды (то есть награда ожидаемая). В средах с несколькими состояниями, где эпизод взаимодействия с агентом состоит из более чем одного временного шага, это оправданно. Но в МАВ в каждом эпизоде можно выбрать только одно действие.

Поэтому мы можем убрать из определения цели RL слова, которые не относятся к МАВ: награда у нас не «совокупная», так как эпизод состоит всего из одного временного шага, и не «дисконтированная», поскольку нет следующих состояний, которые нужно учитывать. Это значит, что цель агента МАВ — максимизировать ожидаемую награду. Обратите внимание, что слово «ожидаемую» осталось, ведь среде присуща стохастичность. Именно это и нужно узнать агенту МАВ: исходное вероятностное распределение сигнала вознаграждения.

Но если мы оставим цель «максимизировать ожидаемое вознаграждение», сравнивать агенты будет непросто. Представьте, что один агент учится увеличивать ожидаемую награду путем выбора случайных действий во всех эпизодах, кроме заключительного, а другой, с более эффективной выборкой, использует разумную стратегию для быстрого определения оптимального действия. Если сравнивать эти агенты лишь по их результатам в заключительном эпизоде (что нередко можно увидеть в RL), их производительность будет одинаково хорошей. Это явно не то, что нам нужно.

Надежный способ достичь поставленной цели в полной мере — увеличить награду, ожидаемую в каждом эпизоде, при этом минимизировав во всех *общие потери*. Для их вычисления мы суммируем разности реальной ожидаемой награды оптимального и выбранного действия в каждом эпизоде. Очевидно, что чем меньше потерь, тем лучше. Заметьте, я называю награду реальной, так как для вычисления потерь нужно иметь доступ к MDP. Это не значит, что ваш агент должен взаимодействовать с моделью MDP, она просто нужна для сравнения эффективности этапов исследования разных агентов.



Я знаю математику

Уравнение общих потерь

$$\mathcal{T} = \sum_{e=1}^E \mathbb{E} [v_* - q_*(A_e)]$$

(1) Чтобы вычислить общую потерю $\mathcal{T}$, для всех эпизодов нужно суммировать...

(2) ...разности оптимальной ценности МАВ и реальной ценности выбранного действия

Подходы к поиску решений в средах МАВ

Есть три основных подхода к поиску решений в МАВ. Самый популярный и простой из них основан на исследовании через внедрение элемента случайности в процесс выбора действий: агент будет в основном использовать имеющиеся знания, но иногда исследовать среду случайным путем. Это семейство методов называется *случайными стратегиями исследования*. Элементарный пример: стратегия, которая большую часть времени выбирает жадное действие, но по достижении порога эпсилон делает равномерно случайный выбор. В связи с этим у нас появляется много вопросов. Например, должно ли значение эпсилон быть одинаковым для всех эпизодов? Нужно ли максимизировать исследование на ранних этапах? Стоит ли периодически увеличивать значение эпсилон, чтобы агент всегда занимался исследованием?

Другой подход к разрешению дилеммы исследования и эксплуатации — это оптимизм. Именно так. Семейство *оптимистических стратегий исследования* дает нам более систематизированный подход, который количественно характеризует неопределенность в задачах принятия решений и отдает предпочтение наиболее неопределенным состояниям. Суть в том, что оптимизм подталкивает агента к выбору неопределенных состояний: он предполагает, что незнакомые ему состояния должны быть самыми выгодными. Это помогает агенту исследовать среду, и, по мере того как он сталкивается с реальностью, его ожидания становятся все ниже и ниже, приближаясь к реальным показателям.

Третий подход — внедрение семейства *стратегий исследования информационного пространства состояний*. Они моделируют информацию о состоянии агента в рамках среды. Благодаря внедрению неопределенности в пространство агент будет по-разному воспринимать исследованные и неисследованные состояния. Это действенный метод, но он может существенно увеличить пространство состояний, что усложнит выполнение задачи.

Здесь мы рассмотрим примеры из первых двух подходов. Мы сделаем это на нескольких разных средах МАВ с разными свойствами, преимуществами и недостатками. Это позволит нам провести глубокое сравнение нескольких стратегий.

Важно отметить, что в средах МАВ оценка Q -функции выглядит довольно просто — это относится ко всем рассматриваемым стратегиям. Среда МАВ одношаговая, поэтому для оценки Q -функции нужно вычислить среднюю награду для каждого действия. Проще говоря, ценность действия a равна общей награде при выборе этого действия, разделенной на количество случаев, в которых оно было выбрано.

Заметьте, что все стратегии, которые мы здесь рассмотрим, оценивают Q -функцию совершенно одинаково. Единственное отличие в том, как каждая из стратегий использует полученное значение для выбора действий.

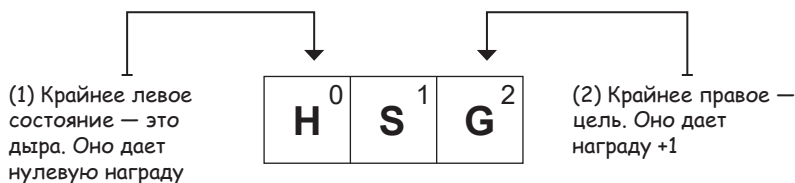


Конкретный пример

«Скользкая бандитская походка» (СБП) возвращается!

Для начала возьмем уже знакомую нам среду MAB: «скользящая бандитская походка» (СБП).

Среда «скользящая бандитская походка»



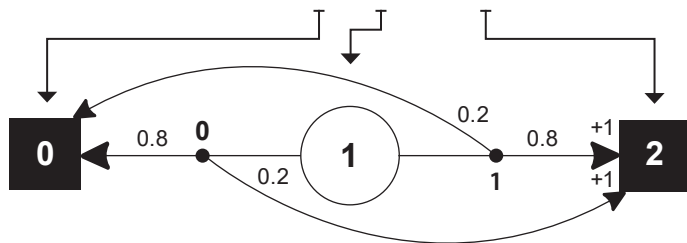
Как вы помните, СБП — это среда в виде сетки с одной строкой, поэтому мы называем ее «походкой». Но у этой «походки» есть одна особенность: агент начинает посредине и любое действие отправляет его прямым в конечное состояние. У этой среды один временной шаг, поэтому она «бандитская».

СБП — это «двурукий бандит», который может предстать перед агентом в виде «двурукого бандита Бернулли». «Бандиты Бернулли» выплачивают награду +1 с вероятностью p и нулевую — с вероятностью $q = 1 - p$. Иначе говоря, сигнал вознаграждения — это распределение Бернулли.

В СБП есть два конечных состояния, одно из которых дает +1, а другое — 0. Если посчитать, можно заметить, что вероятность награды +1 при выборе действия 0 равна 0,2, а при выборе действия 1 — 0,8. Но ваш агент об этом не знает, и мы не станем делиться с ним этой информацией. Нам нужно получить ответы на следующие вопросы: «Насколько быстро наш агент может определить оптимальное действие?», «Сколько всего потерь он накопит, обучаясь максимизировать ожидаемую награду?». Давайте посмотрим.

Диаграмма «скользящей бандитской походки»

(1) Помните: дыра, начало и конечное состояние



Жадная стратегия: всегда эксплуатировать

Первая стратегия, которую мы рассмотрим, не совсем полноценная: это скорее базовый план. Я уже упоминал, что нам нужно исследовать наши алгоритмы, иначе мы рискуем перейти к неподходящим действиям. Поэтому для сравнения обсудим алгоритм, который вообще не занимается исследованием новой информации.

Этот базовый подход называется *жадной стратегией* или *стратегией чистой эксплуатации*. Его суть в том, что мы всегда выбираем действие с наивысшей ожидаемой ценностью. И хотя может случиться так, что первое же выбранное нами действие окажется лучшим, вероятность этого счастливого стечения обстоятельств уменьшается по мере увеличения числа доступных действий.

Чистая эксплуатация в СБП

Первая итерация

→ Агент

(1) Действие — это индекс элемента с наивысшей ценностью (если их больше одного, выбирается первый)

$Q(a)$	$a=0$	$a=1$
	0	0

→ $\text{argmax}(Q) = 0$

→ Среда



Вторая итерация

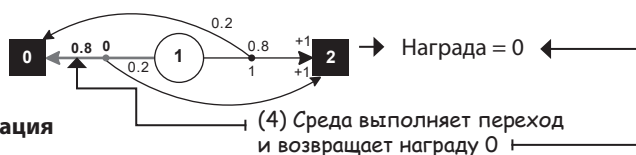
→ Агент

(3) Агент снова выбирает действие 0

$Q(a)$	$a=0$	$a=1$
	1	0

→ $\text{argmax}(Q) = 0$

→ Среда



Третья итерация

→ Агент

(5) Как видите, агент уже застрял на действии 0

$Q(a)$	$a=0$	$a=1$
	0.5	0

→ $\text{argmax}(Q) = 0$

Как и ожидалось, жадная политика сразу же останавливается на первом действии. Если инициализировать Q-таблицу нулями и если в среде нет отрицательных наград, жадная политика всегда будет выбирать первое действие.



Я знаю Python

Стратегия чистой эксплуатации

```
def pure_exploitation(env, n_episodes=5000):
    (1) Почти все стратегии используют одни и те же вычисления
    для оценки Q-значений
    (2) Мы инициализируем Q-функцию и массив count всеми нулями
    Q = np.zeros((env.action_space.n))
    N = np.zeros((env.action_space.n))
    (3) Эти переменные предназначены для ведения статистики, они необязательны
    Qe = np.empty((n_episodes, env.action_space.n))
    returns = np.empty(n_episodes)
    actions = np.empty(n_episodes, dtype=np.int)
    (4) Здесь мы входим в главный цикл и взаимодействуем со средой
    name = 'Pure exploitation'
    for e in tqdm(range(n_episodes),
                  desc='Episodes for: ' + name, leave=False):
        action = np.argmax(Q)
        (5) Все просто: мы выбираем
        действие, которое максимизирует
        ожидаемые нами Q-значения
        (6) Затем мы передаем
        его среде и получаем
        новую награду
        _, reward, _, _ = env.step(action)
        N[action] += 1
        Q[action] = Q[action] + (reward - Q[action])/N[action]
        (7) Мы обновляем счетчики (count) и Q-таблицу
        Qe[e] = Q
        returns[e] = reward
        actions[e] = action
    return name, returns, Qe, actions
    (8) А затем обновляем
    статистические данные
    и начинаем новый эпизод
```

Я хочу обратить ваше внимание на связь между жадной стратегией и временем. Если у вашего агента остается всего один эпизод, жадное поведение — лучший выбор. Зная, что это ваш последний день на земле, вы посвятите его вещам, которые приносят вам больше всего положительных эмоций. В этом в какой-то степени суть жадной политики: агент пытается извлечь максимум с учетом ограниченного количества времени и знаний о среде.

Это вполне разумно, когда у вас осталось мало времени. Не сделать этого было бы недальновидно, ведь вы не пожертвуете немедленным удовлетворением или вознаграждением ради информации, которая позволит вам улучшить долгосрочные результаты.

Случайная стратегия: всегда исследуем

Рассмотрим противоположный подход: стратегию, которая занимается только исследованием, но никогда не использует полученную информацию. Это еще один базовый метод, который называется *случайной стратегией* или *стратегией случайного исследования*. Это просто процесс выбора действий, который совершенно не предусматривает использования имеющихся знаний. Агент заинтересован только в исследовании.

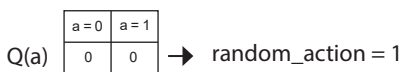
У вас наверняка есть знакомые, которые перед началом нового проекта тратят много времени на подготовку и никогда не принимают поспешных решений? У меня тоже! Они могут провести целые недели за чтением документации. Этап исследования очень важен, но для получения максимальной выгоды он должен быть сбалансирован.

Чистое исследование в СБП

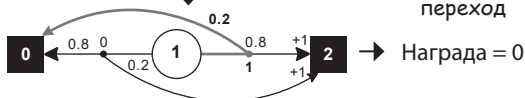
Первая итерация

→ Агент

(1) Агент выбирает действие 1 равномерно случайным образом



→ Среда



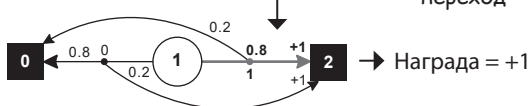
Вторая итерация

→ Агент

(3) Агент снова выбирает действие 1



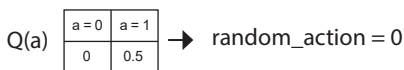
→ Среда



Третья итерация

→ Агент

(6) Агент продолжает выбирать действия случайно, не обращая никакого внимания на вознаграждение!



(5) Теперь агент выбирает действие 0

(7) Заметьте, что при достаточном количестве эпизодов награды будут сходиться к оптимальным значениям

Очевидно, что случайная стратегия тоже далека от идеала и оптимальных результатов она не даст. Нельзя концентрироваться только на исследовании нового или использовании имеющейся информации. Нам нужен алгоритм, который умеет сочетать эти две стадии.



Я знаю Python

Стратегия чистого исследования

```
def pure_exploration(env, n_episodes=5000):
    <...>
    name = 'Pure exploration'
    for e in tqdm(range(n_episodes),
                    desc='Episodes for: ' + name,
                    leave=False):
        <...>
        action = np.random.randint(len(Q))
        <...>
    return name, returns, Qe, actions
```

(1) При чистом исследовании используется тот же шаблонный код, что и прежде, поэтому я его опускаю

(2) Вот как ведет себя наша стратегия чистого исследования: всегда выбирает действие наугад

(3) Для краткости я убрал разделы оценки и статистики



(4) Этот подход не совсем правильно называть стратегией чистого исследования. Это скорее случайная стратегия, поскольку исследовать можно и другими способами, без случайного поведения. Но пусть вас это не отвлекает

В коде приводится одно замечание, на котором я бы хотел остановиться подробнее. Представленная здесь стратегия — лишь один из подходов к исследованию (а именно, случайный). У нее есть много альтернатив. Например, мы можем подсчитывать, сколько раз было выбрано одно действие по сравнению с другими, или сравнивать полученные награды.

Задумайтесь на секунду: использовать информацию можно только одним способом, но исследовать ее можно по-разному. Этап эксплуатации — это не что иное, как выбор действий, которые вы считаете лучшими. Это довольно просто: если вам кажется, что лучше А ничего нет, вы выбираете А.

Этап же исследования намного сложнее. Очевидно, что вам нужно собирать информацию, но как это делать — уже другой вопрос. Вы можете попытаться подтвердить или опровергнуть ваши текущие знания. В основе исследования может лежать как уверенность, так и неопределенность. Список можно продолжать очень долго.

В целом все логично: ваша цель — использование информации, полученной на этапе исследования. Для ее достижения нужно обзавестись знаниями, это понятно. Но получать их можно по-разному, и именно в этом вся сложность.

Эпсилон-жадная стратегия: почти всегда жадная, но иногда случайная

Объединим две базовые стратегии (чистая эксплуатация и чистое исследование), чтобы наш агент мог и собирать информацию, и использовать ее для принятия обоснованных решений. Гибридная стратегия большую часть времени действует жадно, но иногда занимается случайным исследованием.

Такой подход называется *эпсилон-жадной стратегией*, и работает он на удивление хорошо. Почти всегда выбирая действия, которые кажутся вам лучшими, вы получите хорошие результаты, а это «почти» позволяет исследовать неопробованные варианты. Так ваша функция ценности действий получит возможность сойтись на своем реальном значении, что поможет вам получить больше награды в долгосрочной перспективе.

Эпсилон-жадная стратегия в СБП

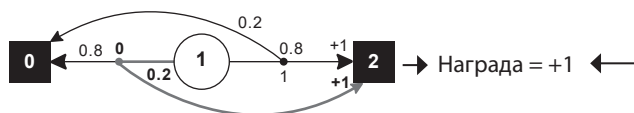
Первая итерация

→ Агент

	a = 0	a = 1
O(a)	0	0

(1) Агент жадно выбирает действие 0
 $\text{argmax}(Q) = 0$

→ Среда



Вторая итерация

→ Агент

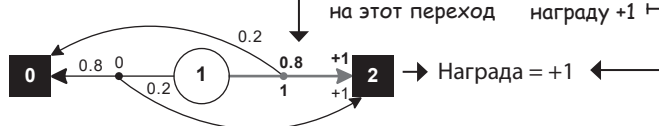
	a = 0	a = 1
Q(a)	1	0

(2) Среда выполняет этот переход и выдает награду +1

(3) Агент выбирает действие 1, на этот раз случайно

$\text{random_action} = 1$

→ Среда



(5) Агент получает награду +1

Третья итерация

→ Агент

(6) Теперь представим, что агент выбирает действие 0 и начинает получать нулевые награды

	a = 0	a = 1
Q(a)	1	1

(7) Сочетание этапов исследования и эксплуатации гарантирует, что агент не заикнется на негативных вознаграждениях
 $\text{argmax}(Q) = 0$



Я знаю Python

Эпсилон-жадная стратегия

```
def epsilon_greedy(env, epsilon=0.01, n_episodes=5000):
    <...>
    name = 'E-greedy {}'.format(epsilon)
    for e in tqdm(range(n_episodes),
                    desc='Episodes for: ' + name,
                    leave=False):
        (2) Эпсилон-жадная стратегия на удивление эффективная, учитывая ее
        простоту. Ее суть в том, что время от времени выбирается случайное
        действие. Первым делом нужно сгенерировать случайное число
        и сравнить его с гиперпараметром epsilon
        if np.random.random() > epsilon:
            (3) Если случайное число больше эпсилон, мы выбираем жадное
            действие с наивысшей ожидаемой ценностью
            action = np.argmax(Q)
            (4) Если нет, продолжаем исследовать, выбирая
            случайные действия
        else:
            action = np.random.randint(len(Q))
        (5) Помните, что это вполне может привести к жадному действию, потому
        что мы выбираем действие случайно из всех доступных, включая жадное.
        На самом деле вероятность, которая используется в исследовании, чуть
        меньше эпсилон, а не равна ей и зависит от количества действий
        <...>
        (6) Убираем оценку и ведение статистики
    return name, returns, Qe, actions
```

Эпсилон-жадная относится к случайным стратегиям исследования, так как при выборе действий используется элемент непредсказуемости. Решение использовать имеющуюся информацию или исследовать новую тоже принимается случайно. Есть и другие виды случайных стратегий исследования: например, функция softmax (о ней мы поговорим немного позже), которая лишена первого этапа случайного принятия решений.

Обратите внимание: если эпсилон равна 0,5 и у вас есть два действия, невозможно сказать, что ваш агент будет заниматься исследованием в 50 % случаев, если под исследованием мы имеем в виду выбор нежадного действия. Также заметьте, что этап исследования в эпсилон-жадной стратегии включает в себя жадное действие. На практике ваш агент будет заниматься исследованием с вероятностью чуть ниже эпсилон, зависящей от количества действий.

Затухающая эпсилон-жадная стратегия: сначала максимизируются исследования, затем эксплуатация

Самые активные исследования логично проводить в начале, когда агент еще недостаточно знаком со средой, а позже, когда у нас уже есть хорошая аппроксимация функции ценности, можно позволить ему чаще использовать полученные знания. Все просто: мы начинаем с большого (меньше или равного 1) значения эпсилон и уменьшаем его с каждым шагом. Такой подход называется *затухающей эпсилон-жадной стратегией* и может принимать разные формы в зависимости от того, как изменяется значение эпсилон. Ниже я покажу вам два примера.



Я знаю Python

Линейно затухающая эпсилон-жадная стратегия

```
def lin_dec_epsilon_greedy(env,
                           init_epsilon=1.0,
                           min_epsilon=0.01,
                           decay_ratio=0.05,
                           n_episodes=5000):
    <...>
    name = 'Lin e-greedy {} {} {}'.format(
        init_epsilon, min_epsilon, decay_ratio)
    for e in tqdm(range(n_episodes),
                  desc='Episodes for: ' + name,
                  leave=False):
        (1) И снова убираем шаблонный код!
        <...>
        (2) Суть этой стратегии в том, что значение эпсилон уменьшается прямо
        пропорционально количеству шагов. Вначале нужно вычислить число
        эпизодов, на протяжении которых эпсилон будет затухать
        до минимального значения
        decay_episodes = n_episodes * decay_ratio

        (3) Теперь вычисляем значение эпсилон для текущего эпизода
        epsilon = 1 - e / decay_episodes
        epsilon *= init_epsilon - min_epsilon
        epsilon += min_epsilon
        epsilon = np.clip(epsilon, min_epsilon, init_epsilon)

        (4) После этого делаем все то же, что и в эпсилон-жадной стратегии
        if np.random.random() > epsilon:
            action = np.argmax(Q)
        else:
            action = np.random.randint(len(Q))
        <...>
    return name, returns, Qe, actions
    (5) Здесь я убрал статистику
```



Я знаю Python

Экспоненциально затухающая эpsilon-жадная стратегия

```
def exp_dec_epsilon_greedy(env,
                           init_epsilon=1.0,
                           min_epsilon=0.01,
                           decay_ratio=0.1,
                           n_episodes=5000):
```

↓ (1) Обратите внимание: код неполный

```
<...>
```

(2) Здесь мы вычисляем экспоненциально затухающие значения эpsilon. Заметьте, все их можно вычислить сразу и затем извлекать из массива внутри цикла

```
    decay_episodes = int(n_episodes * decay_ratio)
    rem_episodes = n_episodes - decay_episodes
    epsilons = 0.01
    epsilons /= np.logspace(-2, 0, decay_episodes)
    epsilons *= init_epsilon - min_epsilon
    epsilons += min_epsilon
    epsilons = np.pad(epsilons, (0, rem_episodes), 'edge')
```

↓ (3) Все остальное выглядит так же, как и раньше

```
    name = 'Exp e-greedy {} {} {}'.format(
        init_epsilon, min_epsilon, decay_ratio)
    for e in tqdm(range(n_episodes),
                  desc='Episodes for: ' + name,
                  leave=False):
        if np.random.random() > epsilons[e]:
            action = np.argmax(Q)
        else:
            action = np.random.randint(len(Q))
    <...>
```

↓ (4) И статистика, конечно же, снова убрана

```
    return name, returns, Qe, actions
```

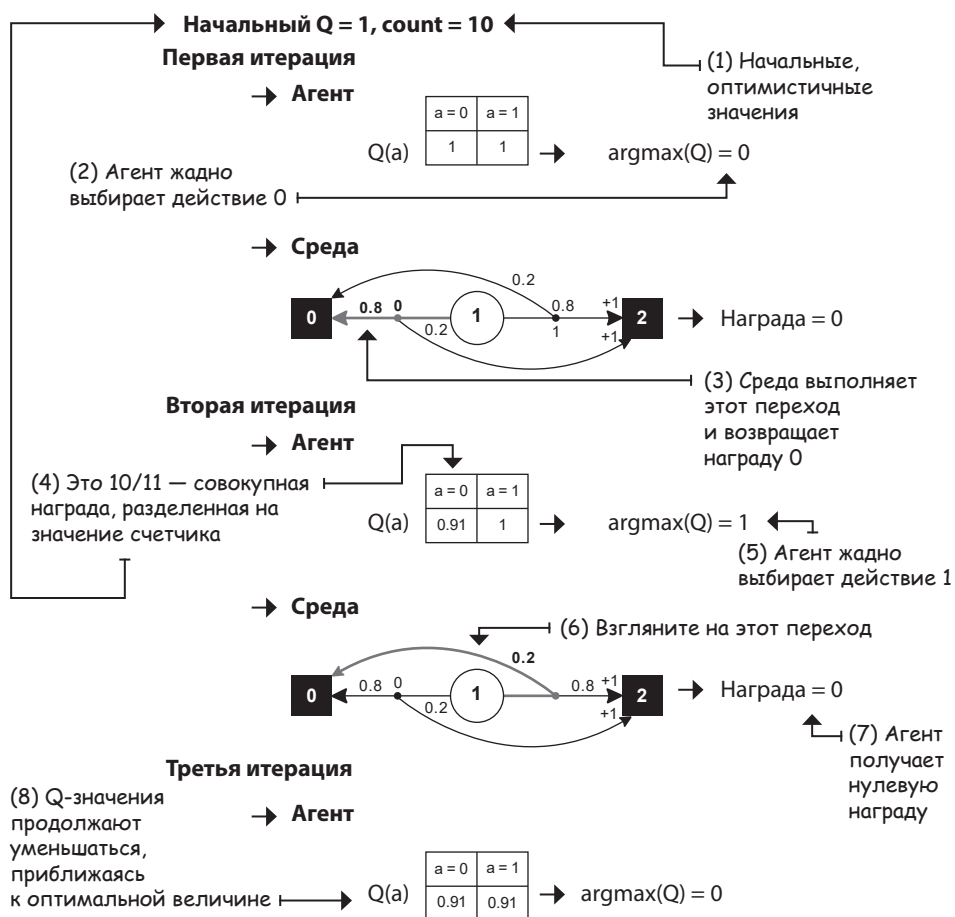
Есть много других способов применения затухания эpsilon: от простого $1/\text{episode}$ до затухающих синусоидальных волн. Есть даже разные реализации одних и тех же линейных и экспоненциальных методов. Суть в том, что вначале агент с более высокой вероятностью должен заниматься исследованием, а позже нужно ввести возможность применять полученную информацию. На ранних этапах высок риск получения неверных оценочных значений. Но со временем агент узнает больше информации, приближая эти значения к реальности, а это значит, что можно начать меньше заниматься исследованием и больше эксплуатацией.

Оптимистичная инициализация: начинаем с веры в то, что все замечательно

Еще один интересный подход к дилемме исследования и эксплуатации: считать недостаточно исследованные действия лучшими из возможных. Этот класс стратегий называется *оптимизмом в условиях неопределенности*. Одна из его разновидностей — оптимистичная стратегия инициализации.

Принцип ее работы прост: мы изначально присваиваем Q -функции высокую ценность и, исходя из этого значения, ведем себя жадно. Нужно прояснить два момента. Во-первых, высокая ценность — это то, к чему у нас нет доступа в RL. Мы вернемся к этому чуть позже, а пока что притворимся, что это число у нас уже есть. Во-вторых, вместе с Q -значениями нам нужно определить счетчики, используя значение больше 1. Если этого не сделать, Q -функция будет меняться слишком быстро, что снизит эффективность стратегии.

Оптимистичная инициализация в СБП





Я знаю Python

Оптимистичная стратегия инициализации

```
def optimistic_initialization(env,
                             optimistic_estimate=1.0,
                             initial_count=100,
                             n_episodes=5000):
    Q = np.full((env.action_space.n),
               optimistic_estimate,
               dtype=np.float64)
    N = np.full((env.action_space.n),
               initial_count,
               dtype=np.float64)
    <...>
    name = 'Optimistic {} {}'.format(optimistic_estimate,
                                     initial_count)
    for e in tqdm(range(n_episodes),
                  desc='Episodes for: ' + name,
                  leave=False):
        <...>
        action = np.argmax(Q)
        <...>
    return name, returns, Qe, actions
```

(1) Стратегия начинается с выбора оптимистичных Q-значений

(2) Обозначаем счетчики, которые будут служить мерой неопределенности: чем больше их значение, тем неопределенней условия

(3) Здесь убран тот же код

(4) После этого мы всегда выбираем действие с наивысшей ожидаемой ценностью, как в стратегии чистой эксплуатации

(5) Удалена еще часть кода

Интересно, правда? Поскольку агент изначально рассчитывает получить больше вознаграждения, чем может получить на самом деле, он исследует среду, пока не найдет источники вознаграждения. С увеличением опыта агент становится менее «наивным»: Q-значения уменьшаются, пока не сойдутся на реальных показателях вознаграждения.

Опять же, инициализируя Q-функцию высоким значением, мы поощряем исследование неопробованных действий. По мере того как агент взаимодействует со средой, наши значения становятся все более низкими и точными, позволяя ему находить и выбирать действия с действительно наивысшей выгодой.

Суть вот в чем: если вы собираетесь быть жадными, то хотя бы проявите оптимизм.



Конкретный пример

Среда «двурукий бандит Бернулли»

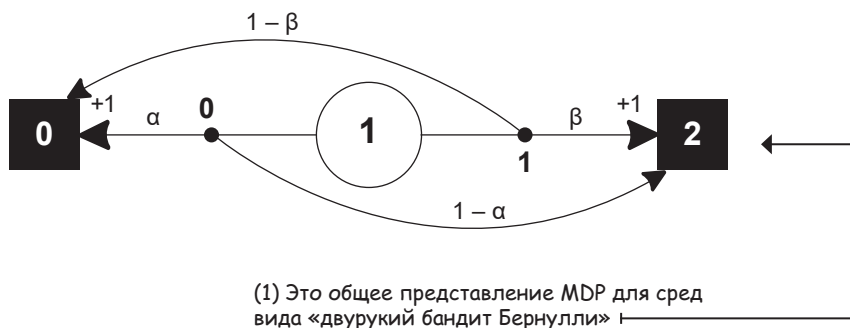
Давайте сравним конкретные реализации стратегий выше на примере нескольких сред вида «двурукий бандит Бернулли».

Среда «двурукий бандит Бернулли» содержит единственное неконечное состояние и два действия. Действие 0 получает награды +1 и 0 с вероятностью α и $1 - \alpha$ соответственно. Действие 1 получает награды +1 и 0 с вероятностью β и $1 - \beta$ соответственно.

Это чем-то напоминает СБП. Среде СБП характерны дополняющие друг друга вероятности: действие 0 дает +1 с вероятностью α , а действие 1 дает +1 с вероятностью $1 - \alpha$. В этой же, «бандитской» среде вероятности независимы и могут даже быть равными.

Ниже представлена диаграмма MDP для «двурукого бандита Бернулли».

Среды вида «двурукий бандит Бернулли»



Очень важно понимать, что эту среду можно представить по-разному. На самом деле в коде я ее реализовал иначе, избавившись от большого количества шума.

Взгляните, к примеру, на два конечных состояния: у одного из них может быть два действия с переходом в себя, но если изобразить это на диаграмме, она станет слишком запутанной.

Из всего этого можно извлечь ценный урок: никто вам не запрещает создавать и представлять среды так, как вам удобно. Здесь нет какого-то единого правильного ответа. Конечно, неправильных методов тоже хватает. Поэтому их и нужно исследовать!

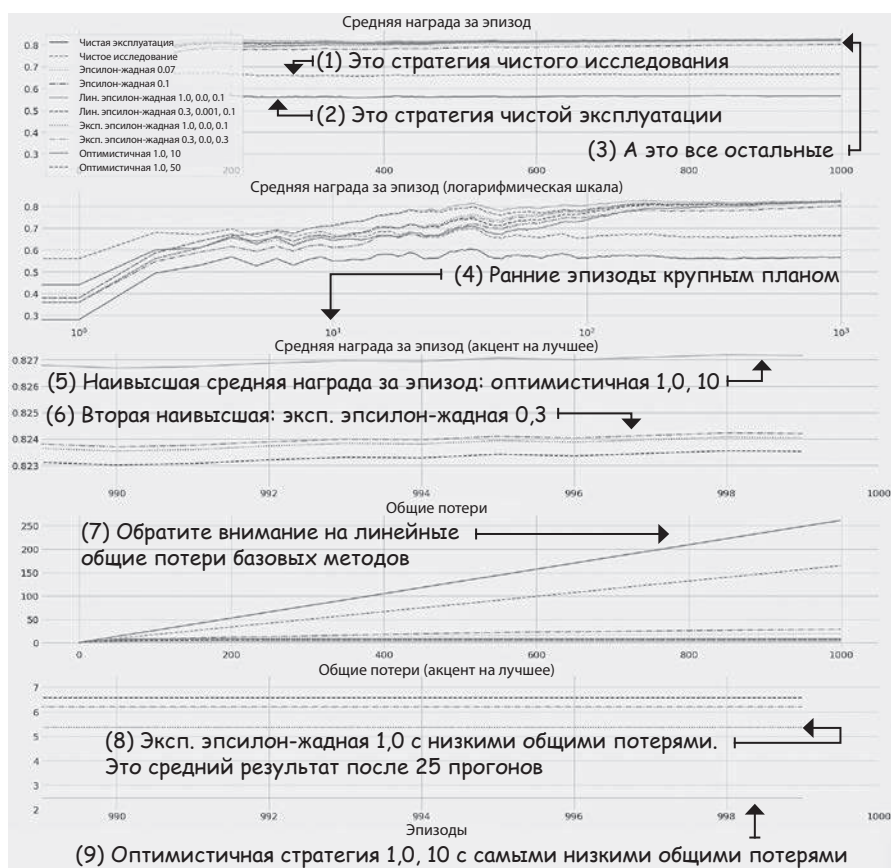
Простите, не удержался.



Посчитаем

Простые стратегии исследования в средах вида «двурукий бандит Бернулли»

Я запустил по два экземпляра гиперпараметров всех представленных до сих пор стратегий: эpsilon-жадной, двух затухающих и оптимистичной, а еще базовых методов с чистыми эксплуатацией/исследованием в пяти средах вида «двурукий бандит Бернулли» с вероятностями α и β , инициализированными равномерно случайно, и пятью начальными значениями. Ниже представлены средние результаты после 25 прогонов.



Лучшие результаты в этом эксперименте показала оптимистичная стратегия с начальными Q -значениями 1,0 и десятью первичными расчетами. Производительность остальных тоже оказалась довольно неплохой, несмотря на то что они не были тщательно оптимизированы, так что это все было просто для развлечения — ничего более. Можете открыть блокнот Jupyter для главы 4 и вдоволь поэкспериментировать.



В деталях

Простые стратегии в средах вида «двурукий бандит Бернулли»

Давайте обсудим детали этого эксперимента.

Во-первых, для генерации пяти разных сред вида «двурукий бандит Бернулли» я использовал пять разных начальных значений (12, 34, 56, 78, 90). Помните: все «бандиты Бернулли» возвращают награду +1 с определенной вероятностью, которая своя у каждой руки.

Ниже перечислены полученные среды и вероятность выгоды в них:

- «двурукий бандит» с начальным значением 12. Вероятность награды: [0,41630234; 0,5545003];
- «двурукий бандит» с начальным значением 34. Вероятность награды: [0,88039337; 0,56881791];
- «двурукий бандит» с начальным значением 56. Вероятность награды: [0,44859284; 0,9499771];
- «двурукий бандит» с начальным значением 78. Вероятность награды: [0,53235706; 0,84511988];
- «двурукий бандит» с начальным значением 90. Вероятность награды: [0,56461729; 0,91744039].

Средняя оптимальная величина этих значений — 0,83.

Все эти стратегии были применены в каждой из перечисленных выше сред с пятью разными начальными значениями (12, 34, 56, 78, 90), чтобы смягчить перепады между результатами и исключить из них фактор случайности. Первым я использовал начальное значение 12 для создания «бандита Бернулли», затем с помощью значений 12, 34 и т. д. я измерил производительность каждой стратегии в среде, созданной с начальным значением 12.

После я создал еще одного «бандита Бернулли» с использованием начального значения 34 и затем оценил в его рамках каждую стратегию, применяя значения 12, 34 и т. д. В итоге я получил средние показатели для пяти сред и пяти начальных значений, что дало 25 разных прогонов для каждой среды.

Я оптимизировал каждую стратегию по отдельности вручную: из примерно десяти комбинаций гиперпараметров я выбрал две лучшие.

Стратегическое исследование

Представьте, что вам нужно написать агент RL, который будет учиться водить автомобиль. Для этого вы решили реализовать эпсилон-жадную стратегию исследования. Вы загрузили свой агент в бортовой компьютер, повернули ключ зажигания, нажали красивую зеленую кнопку, и ваша машина начала свое

исследование. Она подбрасывает воображаемую монету и выбирает случайное действие, скажем движение по противоположной стороне дороги. Нравится вам такое? Мне тоже нет. Надеюсь, этот пример помог показать важность выбора разных стратегий исследования.

Конечно, я немного преувеличил. Никто не позволил бы необученному агенту познавать реальный окружающий мир напрямую. На практике, прежде чем применить RL в настоящих автомобилях, беспилотных аппаратах и т. п., агент сначала проходит предварительное обучение в симуляции и/или использует более эффективные методы выборки.

Но то, о чем я говорил выше, по-прежнему верно. Если подумать, то мы, люди, не познаем окружающий мир случайно. Ну, кроме младенцев. Возможно, неопределенность и вносит в нашу жизнь некоторый элемент случайности, но мы, к примеру, не женимся на первом встречном без причины (если это не Лас-Вегас, конечно). Я бы сказал, что взрослые подходят к процессу исследования более стратегически. Мы понимаем, что для получения долгосрочного удовлетворения приходится жертвовать краткосрочным, и осознанно пытаемся получить информацию. Мы познаем окружающий мир, пробуя то неизведанное, которое может потенциально улучшить нашу жизнь. Возможно, наши стратегии исследования — это сочетание оценок со степенью их неопределенности. Например, вместо любимого блюда, которое мы заказываем каждые выходные, мы можем предпочесть то, которое нам, скорее всего, понравится. Быть может, мы исследуем, основываясь на нашей любознательности или погрешности наших предсказаний. Например, заказав в ресторане интересное блюдо, которое мы еще не пробовали, мы можем неожиданно открыть для себя то, что до этого мы не ели ничего вкуснее. «Погрешность предсказаний» и такой «сюрприз» могут иногда служить стимулами для исследования окружающей среды.

Оставшаяся часть этой главы посвящена более развитым стратегиям исследования. Несколько из них тоже используют элемент случайности, но делают это пропорционально текущим оценкам действий, а некоторые учитывают степень уверенности и неопределенности в этих оценках.

С учетом вышесказанного я еще раз хочу подчеркнуть, что эпсилон-жадная стратегия исследования (и ее затухающие версии) по-прежнему самая популярная: возможно, из-за хорошей производительности, а возможно, из-за ее простоты. Может быть, дело в том, что большинство сред обучения с подкреплением на сегодняшний день существуют только внутри компьютера, а в виртуальном мире вопрос человеческой безопасности стоит не так остро. Подумайте над этим хорошенько. Поиск баланса между исследованием (сбором) новой информации и использованием уже имеющейся — базовый аспект человеческого и искусственного интеллекта, равно как и обучения с подкреплением. Я уверен, что прогресс в этом направлении сильно повлияет на искусственный интеллект, обучение с подкреплением и на все другие области, заинтересованные в этом фундаментальном компромиссе.

Softmax: случайный выбор действий пропорционально их ценности

В стратегиях случайного исследования больше смысла, если они учитывают оценки Q -значения. Если ценность действия низкая, то и вероятность его выбора должна быть ниже. Именно так работает стратегия *softmax*: она выбирает из вероятностного распределения по функции ценности действий так, чтобы вероятность выбора действия была прямо пропорциональна его текущей ожидаемой ценности. Этот подход относится к семейству стратегий случайного исследования, но благодаря внедрению элемента случайности на этапе исследования он имеет отношение и к эпсилон-жадной стратегии. Эпсилон-жадная стратегия делает выбор равномерно случайно из всего набора действий, доступных в заданном состоянии, тогда как *softmax* отдает предпочтение более ценным действиям.

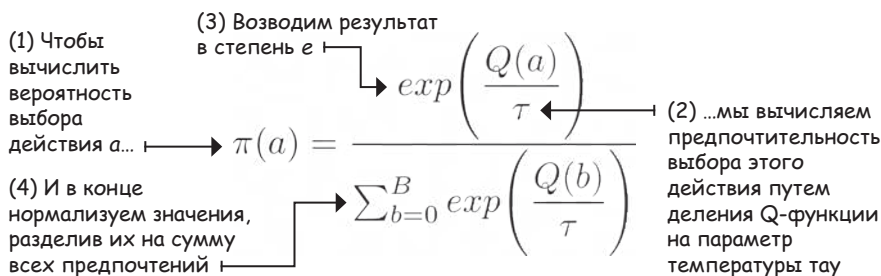
Используя *softmax*, мы фактически делаем ожидаемую ценность действия индикатором предпочтительности. Неважно, каковы показатели: если прибавить к ним одну и ту же константу, вероятностное распределение не изменится. При выборе действий из вероятностного распределения учитывается Q -функция. Разница между ожидаемыми Q -значениями приводит к тому, что агент чаще выбирает действия с наивысшими значениями.

Мы можем добавить гиперпараметр, чтобы регулировать чувствительность к перепадам ожидаемых Q -значений, — *температуру* (отсылка к статистической механике), и по мере приближения его значения к бесконечности предпочтения по Q -значениям становятся равными. По сути, выборка действий происходит равномерно. Но чем ближе температура к нулю, тем вероятней выбор действия с наивысшей ожидаемой ценностью. Мы можем сделать и так, чтобы гиперпараметр затухал линейно, экспоненциально или как-то еще. Но на практике из соображений численной стабильности мы не можем использовать бесконечность или ноль в качестве температуры. Вместо этого мы берем либо очень большие, либо очень маленькие вещественные числа и затем их нормируем.



Я знаю математику

Стратегия исследования *softmax*





Я знаю Python

Стратегия softmax

```
def softmax(env,
            init_temp=1000.0,
            min_temp=0.01,
            decay_ratio=0.04,
            n_episodes=5000):
```

(1) Чтобы было проще, я сократил код

```
<...>
name = 'SoftMax {} {} {}'.format(init_temp,
                                  min_temp,
                                  decay_ratio)

for e in tqdm(range(n_episodes),
              desc='Episodes for: ' + name,
              leave=False):
```

(2) Сначала мы вычисляем линейно затухающую температуру, как делали это с линейно затухающим значением эпсилон

```
    decay_episodes = n_episodes * decay_ratio
    temp = 1 - e / decay_episodes
    temp *= init_temp - min_temp
    temp += min_temp
    temp = np.clip(temp, min_temp, init_temp)
```

(3) Чтобы избежать деления на ноль, нужно убедиться, что min_temp не равно 0. Подробности см. в блокноте Jupyter

(4) Далее мы вычисляем вероятности, применяя функцию softmax к Q-значениям

```
        scaled_Q = Q / temp
        norm_Q = scaled_Q - np.max(scaled_Q)
        exp_Q = np.exp(norm_Q)
        probs = exp_Q / np.sum(exp_Q)
```

(5) Нормализуем для вычислительной устойчивости

```
    assert np.isclose(probs.sum(), 1.0)
    action = np.random.choice(np.arange(len(probs)),
                              size=1,
                              p=probs)[0]
```

(6) Теперь убеждаемся, что мы получили хорошие вероятности, и выбираем действие на их основе

```
_, reward, _, _ = env.step(action)
<...>
```

(7) Здесь я тоже убрал код

```
return name, returns, Qe, actions
```

UCB: не просто оптимизм, а реалистичный оптимизм

В последнем разделе мы рассмотрим оптимистичную стратегию инициализации. Это разумный (и немного философский) подход к поиску баланса между исследованием и эксплуатацией — простейший в семействе оптимистичных стратегий в условиях неопределенности.

У рассмотренного нами алгоритма есть два существенных недостатка. Прежде всего, нам не всегда известна максимальная награда, которую может получить агент в заданной среде. Если сделать начальное ожидаемое Q -значение оптимистичной стратегии намного выше его реального максимума, алгоритм окажется неэффективным, ведь для приближения к фактическим значениям агенту понадобится много эпизодов (в зависимости от гиперпараметра `counts`). Но, что еще хуже, если начальные Q -значения окажутся намного меньше доступного в среде максимума, алгоритм перестанет быть оптимистичным и больше не сможет работать правильно.

Вторая проблема этой стратегии в том виде, в котором мы ее представили, связана с тем, что переменная `counts` — гиперпараметр, который нужно регулировать. Но на самом деле то, что мы пытаемся представить с ее помощью, — это неопределенность оценки, которая не должна быть гиперпараметром. Мы изначально верим в лучший исход и назначаем степень уверенности произвольно. Вместо этого мы можем действовать по тому же принципу оптимистичной инициализации, но при этом использовать статистические методики для вычисления ожидаемой степени неопределенности и применять полученный результат в процессе исследования. Именно это делает стратегия *верхнего доверительного интервала* (upper confidence bound, UCB).

UCB сохраняет оптимизм, но приближает его к реальности: мы не слепо полагаемся на удачу, а учитываем неопределенность ожидаемых оценок. Чем менее определенная оценка Q -значения, тем важнее ее исследовать. Заметьте, что мы больше не верим в то, что оценка будет «максимально возможной», хотя она и может быть таковой! Новый показатель, который нас здесь интересует, — это неопределенность: мы верим в то, что за ней может скрываться что-то полезное.



Я знаю математику

Уравнение верхнего доверительного интервала (UCB)

$$A_e = \underset{a}{\operatorname{argmax}} \left[Q_e(a) + c \sqrt{\frac{\ln e}{N_e(a)}} \right]$$

(1) Чтобы выбрать действие во время эпизода e ...

(2) ...мы суммируем ожидаемые Q -значения...

(3) ...с фактором неопределенности

(4) И затем выбираем действие с наивысшей общей ценностью

Для реализации этой стратегии мы выбираем действие с самой большой суммой ожидаемого Q -значения и степени неопределенности действия U . То есть мы прибавляем верхний доверительный интервал $U_i(a)$ к ожидаемому Q -значению действия a , рассчитывая, что у действия a , выбранного всего несколько раз, высокий показатель U , что поощряет исследование a . Если число попыток высокое, мы добавляем к ожидаемым Q -значениям только небольшое U , так как наша уверенность в их точности растет и их исследование уже не настолько важно.



Я знаю Python

Стратегия верхнего доверительного интервала (UCB)

```
def upper_confidence_bound(env,
                           c=2,
                           n_episodes=5000):
    <...>  (1) Часть кода скрыта для краткости
    name = 'UCB {}'.format(c)
    for e in tqdm(range(n_episodes),
                  desc='Episodes for: ' + name,
                  leave=False):
        if e < len(Q):  (2) Сначала выбираем все
            action = e  действия по одному разу, чтобы
                       избежать деления на ноль
        else:
            (3) Затем переходим к вычислению доверительного интервала
            U = np.sqrt(c * np.log(e) / N)
            (4) В конце мы выбираем действие с самой большой суммой
                ожидаемого значения и степени неопределенности
            action = np.argmax(Q + U)
    <...>  (5) Код ведения статистики
    return name, returns, Qe, actions  для краткости я тоже сократил
```

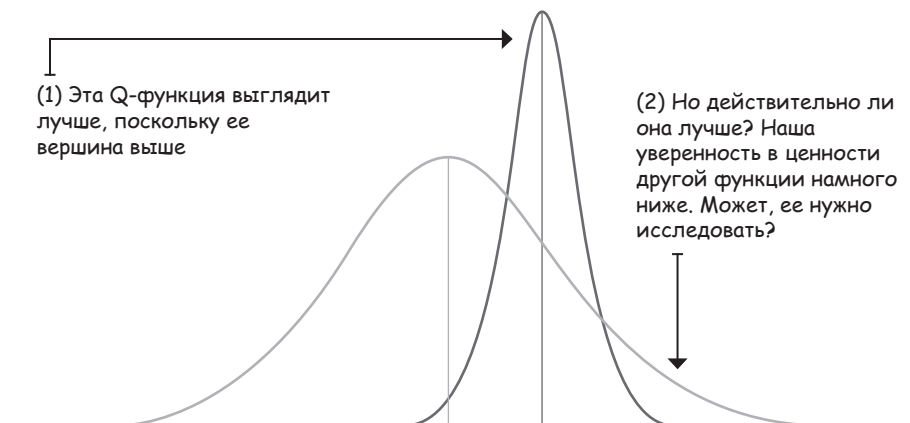
Если построить график U в виде функции от эпизодов и показателей счетчиков, можно заметить, что результат во многом похож на экспоненциально затухающую функцию, но с несколькими отличиями. Здесь затухание получается не плавным, а резким и ранним, с длинным хвостом. Благодаря этому в начальных эпизодах наблюдается высокая степень неопределенности при небольших различиях между действиями. Но чем больше проходит эпизодов и чем дальше инкрементируются счетчики, тем меньше варьируется степень неопределенности. Другими словами, 0 попыток против 100 должны давать более высокое вознаграждение к 0, чем к 100 против 200 попыток. Еще важно отметить, что гиперпараметр с определяет масштаб U : чем больше c , тем больше степень неопределенности и наоборот.

Выборка Томпсона: баланс риска и вознаграждения

Алгоритм UCB — самый распространенный подход к поиску компромисса между исследованием и эксплуатацией, так как он делает минимальные предположения о распределениях, лежащих в основе Q -функции. Но другие методики, вроде байесовских стратегий, способны использовать априорные предположения для формирования обоснованных и применять эти знания. *Выборка Томпсона* основана на сопоставлении вероятностей, что позволяет нам использовать байесовские методики для поиска баланса между исследованием и эксплуатацией.

Простой способ реализовать эту стратегию — отслеживать каждое Q -значение в рамках гауссовского (нормального) распределения. На самом деле в качестве приоритетного можно использовать любое другое распределение: например, для этого часто выбирают бета-распределение. В нашем случае вершина гауссовского распределения — это ожидаемое Q -значение, а гауссовское стандартное отклонение измеряет неопределенность, которая обновляется в каждом эпизоде.

Сравнение двух функций ценности действий, представленных в виде распределений Гаусса



Как понятно из названия, в выборке Томпсона мы выбираем действие из этих нормальных распределений, при этом его ожидаемая ценность должна быть самой высокой. Затем, чтобы обновить стандартное отклонение распределения по Гауссу, мы используем формулу, похожую на ту, что была применена с UCB, согласно которой на ранних стадиях, когда уровень неопределенности высок, стандартное отклонение более важно, что расширяет гауссовское распределение. Но по мере смены эпизодов вершины распределений смещаются в сторону точных оценок. Стандартное отклонение уменьшается, а гауссовское распределение сужается, что повышает вероятность выбора действия рядом с его ожидаемой вершиной.



Я знаю Python

Стратегия выборки Томпсона

```
def thompson_sampling(env,
                      alpha=1,
                      beta=0,
                      n_episodes=5000):
    (1) Код инициализации
    сокращен
    ↓
    <...>
    name = 'Thompson Sampling {} {}'.format(alpha, beta)
    for e in tqdm(range(n_episodes),
                  desc='Episodes for: ' + name,
                  leave=False):
        (2) Здесь мы выбираем числа из распределения по Гауссу. Заметьте, как параметр
        scale, определяющий ширину распределения (стандартное отклонение), сужается
        по мере выбора каждого последующего действия. Еще отмечу, что alpha определяет
        начальную ширину гауссовского распределения, а beta — темпы сужения
        samples = np.random.normal(
            loc=Q, scale=alpha/(np.sqrt(N) + beta))
        (3) Затем мы выбираем действие с самой высокой ценностью
        action = np.argmax(samples)
        <...>
        return name, returns, Qe, actions
    (4) Код ведения
    статистики я тоже скрыл
```

В этой реализации я использовал два гиперпараметра: α для регулирования масштаба гауссовского распределения (или величины начального стандартного отклонения) и β для смещения затухания так, чтобы стандартное отклонение уменьшалось не так быстро. На практике эти гиперпараметры требуют небольшой доработки для примеров из этой главы, потому что, как вы уже знаете, стандартное отклонение, равное всего 5, например, — это почти плоское распределение, охватывающее более чем десять значений. Учитывая, что наши задачи предусматривают награды (и Q -значения) от 0 до 1 и приблизительно от -3 до 3 (этот пример будет рассмотрен дальше), нам в нашем распределении будет достаточно стандартного отклонения, ненамного превышающего 1.

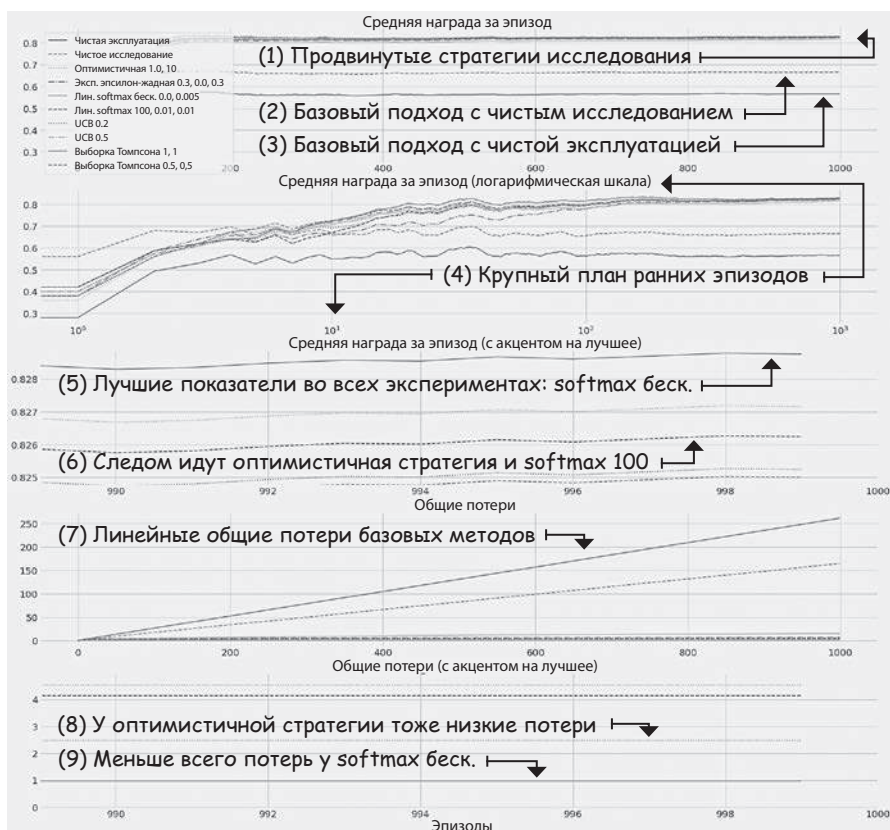
Наконец, я хочу еще раз подчеркнуть, что использование гауссовских распределений — это не самый популярный подход к реализации выборки Томпсона. Пальма первенства в этом случае принадлежит бета-распределениям. Но для решения этих задач я предпочитаю именно распределения по Гауссу, так как они симметричны относительно своих вершин и благодаря своей простоте хорошо подходят для учебных целей. Но вы можете изучить этот вопрос подробнее и поделиться своими знаниями.



Посчитаем

Продвинутые стратегии исследования в средах вида «двурукий бандит Бернулли»

Я запустил по два экземпляра гиперпараметров для каждой из новых стратегий (softmax, UCB и подход Томпсона) и сравнил их с базовыми подходами чистой эксплуатации и чистого исследования, а также с эффективными простыми стратегиями, рассмотренными ранее. Во всех случаях использовались среды вида «двурукий бандит Бернулли». В общей сложности это десять агентов в пяти средах с пятью начальными значениями. То есть 25 прогонов для каждой стратегии. Здесь представлены средние результаты для всех этих прогонов.



Как показывают результаты, продвинутые подходы более эффективны, даже несмотря на то, что оптимистичная стратегия использует знания предметной области, на которые мы не можем полагаться.



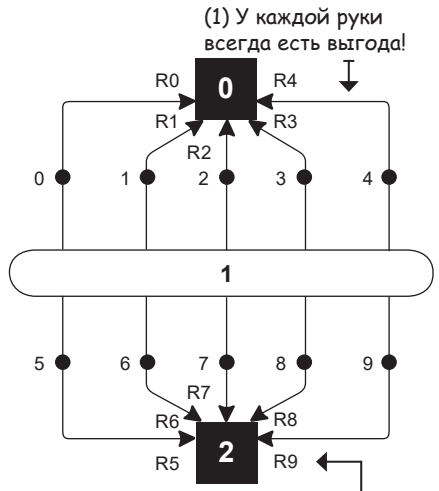
Конкретный пример

Среды вида «десятирукий бандит Гаусса»

В средах вида «десятирукий бандит Гаусса» тоже есть лишь одно неконечное состояние, они относятся к «бандитским» средам. Как вы уже догадались, у них десять рук (или действий) вместо двух, как у «бандитов Бернулли», и другие вероятностные распределения и сигналы вознаграждения. Во-первых, у «бандитов Бернулли» вероятность выигрыша равна p , и, если сделать ее $1 - p$, рука не вернет никакой награды. У «гауссовских бандитов» всегда есть какая-то выгода (если только их выборка не равна 0 — об этом чуть ниже). Во-вторых, у «бандитов Бернулли» двоичный сигнал вознаграждения: вы получаете либо +1, либо 0, а «гауссовские бандиты» вознаграждают агента каждый раз, выбирая награду из распределения по Гауссу.

Чтобы создать среду «десятирукий бандит Гаусса», мы сначала выбираем десять значений из стандартного нормального распределения (с вершиной 0 и дисперсией 1), чтобы получить оптимальную функцию ценности действий $q \times (a_k)$ для всех k (10) рук. Эти значения станут средней величиной сигнала вознаграждения для каждого действия. Чтобы получить награду для действия k в эпизоде e , мы выбираем значение из другого распределения по Гауссу с вершиной $q \times (a_k)$ и дисперсией 1.

Десятирукий бандит Гаусса



Я знаю математику

Функция вознаграждения для «десятирукого бандита Гаусса»

(1) Перед взаимодействием со средой мы должны ее сначала создать, вычислив оптимальную Q-функцию для каждой руки (действия) k

$$\rightarrow a_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s) \leftarrow$$

(3) Когда агент начнет взаимодействовать со средой, нужно выбрать награду R для руки (действия) k в эпизоде e ...

$$\rightarrow R_{k,e} \sim \mathcal{N}(\mu = q^*(a_k), \sigma^2 = 1) \leftarrow$$

(2) Для этого мы делаем выборку из стандартного гауссовского распределения — нормального распределения с вершиной 0 и дисперсией 1

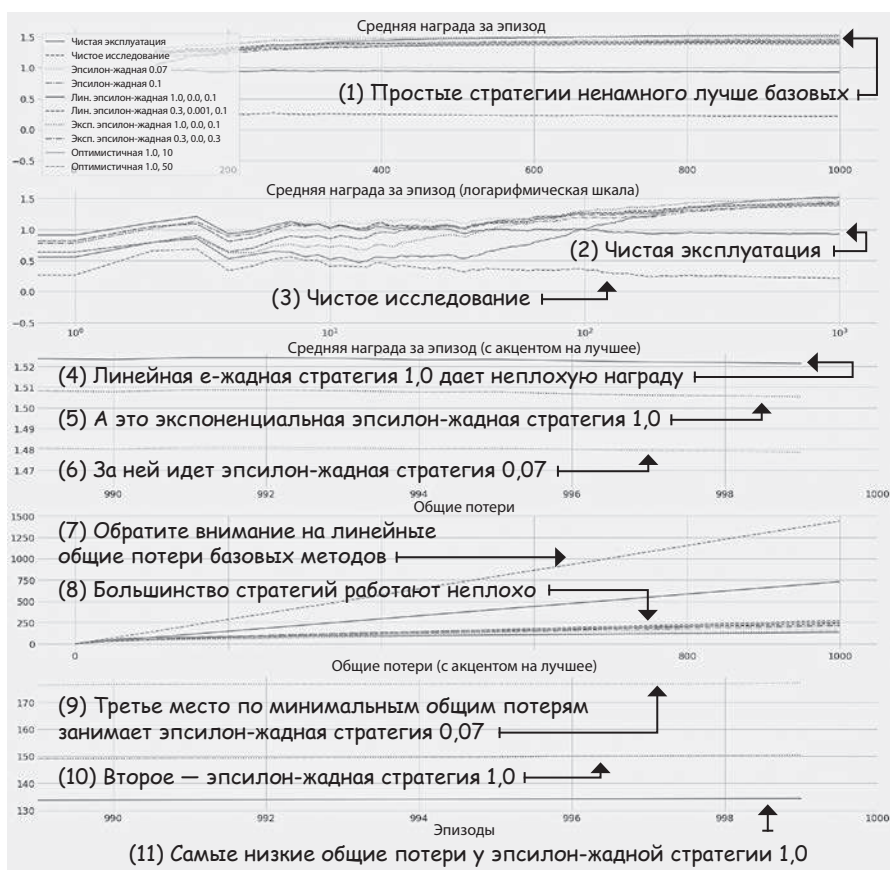
(4) ...мы делаем выборку из распределения по Гауссу с вершиной в оптимальном Q-значении и дисперсией 1



Посчитаем

Продвинутые стратегии исследования в средах вида
«десятирукий бандит Гаусса»

Я запустил те же гиперпараметры простых стратегий, рассмотренных ранее, но теперь в средах вида «десятирукий бандит Гаусса». Конечно, это не совсем чистый эксперимент, так как эти методы тоже могут хорошо работать в данной среде, если их как следует отрегулировать. Но моя цель — показать, что самые продвинутые стратегии по-прежнему хорошо работают со старыми гиперпараметрами, несмотря на изменение среды. Ниже вы сможете в этом убедиться.



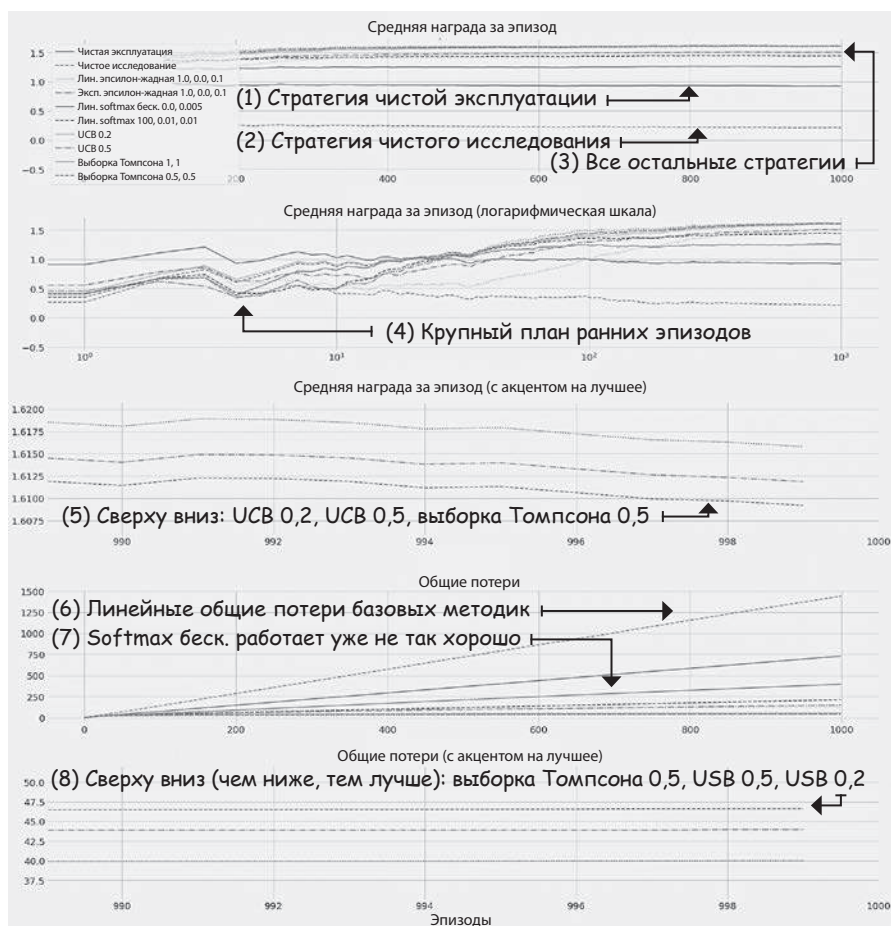
Только посмотрите на это: у простейших стратегий самые низкие общие потери и максимальная ожидаемая награда в пяти разных сценариях!



Посчитаем

Продвинутые стратегии исследования в средах вида «десятирукий бандит Гаусса»

Дальше я протестировал продвинутые стратегии с теми же гиперпараметрами, что и раньше. Я добавил для сравнения две базовые методики и две самые эффективные простые стратегии. Как и во всех предыдущих экспериментах, в общей сложности у меня получилось 25 прогонов.



На этот раз верх одержали продвинутые стратегии. Их общие потери оказались довольно низкими. Теперь откройте блокнот Jupyter и вдоволь поэкспериментируйте! Поделитесь своими результатами с сообществом. Мне не терпится увидеть, к каким результатам вы пришли. Повеселитесь как следует!

Подведем итоги

Обучение на оценочной обратной связи — основная задача обучения с подкреплением, которая делает его уникальным. Получая обратную связь вида +1; +1,345; +1,5; -100; -4, ваш агент не знает, какая модель MDP внутри, поэтому не может определить размер максимальной возможной награды. Ваш агент думает так: «Хорошо, я получил +1, но вдруг под этим камнем прячется +100?» Эта присущая среде неопределенность подталкивает к разработке агентов, которые занимаются исследованием.

Но, как вы уже знаете, к процессу исследования нельзя относиться легкомысленно. По сути, он тратит впустую циклы, которые можно было бы использовать для максимизации награды (то есть для применения имеющихся знаний). Но ваш агент не сможет увеличить вознаграждение (или даже сделать вид, что ему это под силу), если сначала не соберет нужную информацию. В этом и поможет этап исследования. Внезапно ваш агент должен научиться балансировать между исследованием и эксплуатацией: идти на компромисс, искать равновесие между двумя ключевыми, но противоположными аспектами обучения. Все мы стояли перед таким выбором, поэтому эти проблемы должны быть вам знакомы: «лучше синица в руках, чем журавль в небе», но вместе с тем «стремления человека не должны ограничиваться его пониманием». Процесс выбора своего пути тоже может быть интересным. Главное, избегайте крайностей. Ищите баланс!

Определив этот фундаментальный компромисс, мы познакомились с несколькими методами создания агентов (или стратегиями) для балансировки процессов исследования и эксплуатации. Эпсилон-жадная стратегия занимается использованием имеющейся информации в ущерб этапу исследования, который заключается в случайном выборе действия. Затухающие эпсилон-жадные стратегии учитывают то, что вначале агент должен активнее заниматься исследованием, чтобы собрать информацию для дальнейшего принятия верных решений, но вместе с тем он должен как можно быстрее приступить к применению полученных знаний, чтобы не накапливать потери (мера того, насколько далеки наши действия от оптимальных). Затухающие эпсилон-жадные стратегии уменьшают эпсилон с каждым эпизодом по мере того, как наш агент собирает информацию (по крайней мере мы надеемся на то, что он это делает).

Но затем мы познакомились с другими стратегиями, которые пытаются сделать наше «надеюсь» более вероятным. При выборе действий они учитывают не только ценность, но и их неопределенность и потенциал. В число таких стратегий входят оптимистичная инициализация, UCB, выборка Томпсона и softmax. На самом деле последний метод не использует степень неопределенности, а выбирает действия случайно прямо пропорционально их ожидаемой ценности.

На этом этапе вы:

- понимаете, что сложность обучения на основе оценочной обратной связи в том, что агенты не могут видеть базовую MDP, управляющую их средой;
- знаете, что эта проблема побуждает искать баланс между исследованием и эксплуатацией;
- знакомы со многими стратегиями, которые часто используются для решения этой проблемы.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

- **#gdrl_ch04_tf01.** Есть много других методик поиска решений в «бандитских» средах. Изучите другие ресурсы и сообщите нам о подходах, которые кажутся вам важными. Исследуйте байесовский подход к выбору действий и стратегии, основанные на получении информации. Опять же что это такое и какова их роль в контексте RL? Можете ли вы придумать другие интересные стратегии, в том числе и затухающие, способные использовать имеющуюся информацию для снижения темпов исследования у агентов? Представьте, к примеру, агент, в котором эpsilon затухает в зависимости от статистики состояний или каких-то других факторов.
- **#gdrl_ch04_tf02.** Предложите несколько других «бандитских» сред, которые было бы интересно проанализировать. Клонировать мой репозиторий (<https://github.com/mimoralea/gym-bandits> — это, к слову, тоже форк) и добавьте в него еще несколько «бандитских» сред.
- **#gdrl_ch04_tf03.** Где-то посередине между «бандитскими» средами и алгоритмами обучения с подкреплением находится еще один тип сред — *контекстные «бандитские» задачи*. Что это такое? Попробуйте их описать. Не ограничивайтесь одной лишь статьей в блоге: создайте тренировочную среду с «контекстными бандитами». Возможно ли это вообще сделать? Оформите эти среды в виде пакета Python и создайте еще один с алгоритмами, способными находить решения в этих средах.
- **#gdrl_ch04_tf04.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdrl_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы познакомитесь с ценностно ориентированными политиками, изучая одновременно последовательную и оценочную обратную связь.
- ✓ Разработаете алгоритмы оценки политик в средах обучения с подкреплением, где функции перехода и вознаграждения неизвестны.
- ✓ Напишете код для прогнозирования ценности политик в средах, которые полностью демонстрируют задачу обучения с подкреплением.

Я считаю, что большинство человеческих бед вызваны неверным суждением о ценности вещей.

*Бенджамин Франклин (Benjamin Franklin),
отец-основатель Соединенных Штатов,
писатель, политик, изобретатель
и общественный деятель*

Вы уже знаете, как сложно сбалансировать краткосрочные и долгосрочные цели. Все мы сталкиваемся с этой проблемой по несколько раз на дню. Например, чем лучше заняться этим вечером: посмотреть фильм или продолжить чтение книги? Первый вариант сразу же приносит удовлетворение: на протяжении полутора-двух часов вы можете променять нищету на богатство, одиночество на любовь, избыточный вес на стройную фигуру — и все это не переставая жевать попкорн. А вот чтение книги не даст вам особых ощущений сегодня вечером, но в дальнейшем эти ощущения могут (без какой-либо гарантии) оказаться куда сильнее.

Все это плавно подводит нас к другой проблеме, которую мы обсуждали. Если наше долгосрочное удовлетворение будет намного сильнее, то встает вопрос: насколько именно? Можем ли мы это определить? Возможно ли это в принципе? Что ж, в этом вся прелесть жизни: я не знаю, вы не знаете и мы так никогда и не узнаем, если не попробуем. Жизнь не раскрывает вам свою модель MDP — она полна неопределенности. Именно это мы обсуждали в предыдущей главе: поиск баланса между сбором информации и ее использованием.

Но мы рассматривали эту проблему отдельно от последовательного аспекта RL. По сути, мы предполагаем, что у наших действий нет никаких долгосрочных последствий, и сосредотачиваемся на поиске лучшего в текущей ситуации. Например, вы можете озадачиться выбором хорошего фильма или книги, не думая о том, как их прочтение или просмотр повлияют на остаток вашей жизни. Ваши действия не «накапливаются».

Здесь же мы рассмотрим агенты, обучающиеся на одновременно последовательной и оценочной обратной связи: они должны искать баланс не только между краткосрочными и долгосрочными целями, но еще и между исследованием и эксплуатацией. Вернемся к примеру с выбором между фильмом и книгой, где вам нужно решить, что делать сегодня, с учетом того, что все ваши решения имеют накопительный эффект. Как и большинство людей, в условиях неопределенности вы, скорее всего, примете оптимальное решение. Так что же вы выберете: посмотреть фильм или продолжить чтение?

Эта глава посвящена агентам, которые учатся прогнозировать ценность политик, как в методе оценки политик, но на этот раз без MDP. Это обычно называют задачей прогнозирования, так как мы предсказываем функции ценности, которые можно определить как ожидание будущих дисконтированных наград: то есть эти функции содержат значения, зависящие от будущих действий. Мы в каком-то смысле учимся предсказывать будущее. В следующей главе мы займемся оптимизацией политик без моделей MDP. Этот процесс называется задачей управления, так как мы пытаемся улучшить поведение агента. Как позже увидите, это два равнозначных аспекта RL. Специалисты в области машинного обучения обычно придерживаются такого правила: «Модель настолько хороша, насколько хороши данные, на которых ее обучили». В контексте RL это можно перефразировать так: «Политика настолько хороша, насколько хороши ожидаемые значения» или «То, насколько можно улучшить политику, зависит от правильности и точности ее ожидаемых оценок».

Повторюсь, что в DRL агенты обучаются на одновременно последовательной (а не одинарной), оценочной (а не контролируемой) и выборочной (а не исчер-

пывающей) обратной связи. В этой главе мы рассматриваем агенты, которые обучаются на последовательной и оценочной обратной связи. «Выборочность» здесь временно игнорируем. Мы вернемся к ней в главе 8 и повеселимся от души. Я обещаю.

Учимся прогнозировать ценность политик

Как я уже упоминал, эта глава посвящена прогнозированию ценности существующих политик. Когда я впервые столкнулся с этой задачей, то не понял, чем она мотивирована. Как по мне, самый простой способ предсказать ценность политики — это ее многократное тестирование с усреднением результатов.

И это, безусловно, правильный подход и, возможно, даже самый подходящий. Но тогда я упустил из виду то, что для прогнозирования функции ценности есть много других подходов, у каждого из которых свои плюсы и минусы. Многие из этих методов полностью противоположны этому, но есть и компромиссные варианты. В целом мы имеем полный спектр алгоритмов.

В этой главе мы исследуем разные подходы, проанализируем их плюсы и минусы и узнаем, что между ними общего.



С акцентом на RL

Награда, выгода и функция ценности

Награда. Это сигнал вознаграждения, который получает агент за один шаг: он видит состояние, выбирает действие и получает сигнал вознаграждения. Сигнал вознаграждения — это основа RL, но это не то, что агент пытается максимизировать! Он не пытается максимизировать награду! Вы должны понимать, что увеличение награды в рамках одного шага может отрицательно сказаться на долгосрочной выгоде.

Выгода. Это совокупность дисконтированных наград. Выгоду вычисляют от какого-либо состояния и, как правило, до конца эпизода. Ее еще называют общей, совокупной наградой или суммой наград. Выгода обычно дисконтируется, поэтому можно встретить и термины «общая дисконтированная награда», «совокупная дисконтированная награда» и «сумма дисконтированных наград». Но по сути это одно и то же: выгода отражает размер награды, *полученной* агентом за эпизод. Это более надежный показатель производительности, так как в ней есть долгосрочная последовательность — история наград в рамках отдельно взятого эпизода. Но и не это агент пытается максимизировать! В попытке получить максимально большую выгоду мы можем прийти к ненадежной политике, выгода которой может быть высокой в некоторых случаях, но далеко не во всех.

Функция ценности. Это ожидаемая выгода. Конечно же, нам хочется, чтобы выгода была высокой, но это наши *ожидания (в целом)*. Если агент находится в среде с большим разбросом результатов или использует стохастическую политику, в этом нет ничего плохого. В конце концов, он пытается максимизировать общую ожидаемую дисконтированную награду: функцию ценности.



Аналогия от Мигеля

Награда, выгода, функция ценности и реальная жизнь

Как вы принимаете решения в реальной жизни? Выбираете действия, которые приносят наибольшую выгоду лично вам? Или вы принадлежите к числу тех, кто в первую очередь заботится о благе других?

В любом случае вам нечего стыдиться! Эгоизм, как по мне, — отличный сигнал вознаграждения. Он бывает очень полезен и позволяет исследовать много разных вещей. Преследование краткосрочных целей на ранних этапах жизни может оказаться вполне приличной стратегией.

Часто людей порицают за «чрезмерный эгоизм», но мне кажется, что именно так и нужно жить: делайте то, что вам хочется, о чем вы мечтаете, что приносит вам удовлетворение. Следуйте за вознаграждением! Со стороны вы будете выглядеть эгоистичным и жадным, но пусть вас это не волнует.

Со временем вы поймете, что преследование краткосрочных целей, даже если они приносят пользу, — не лучшая стратегия. Вы начнете мыслить шире: если съесть слишком много конфет, у вас заболит живот, а если потратить все деньги на онлайн-шопинг, можно разориться.

Рано или поздно вы начинаете ориентироваться на выгоду. К вам приходит понимание того, что жизнь не ограничивается вашими эгоистичными мотивами. Вы отбрасываете свою жадность, ведь в долгосрочной перспективе она вам только вредит — и вы это понимаете. Но эгоизм остается, так как вы по-прежнему воспринимаете мир сквозь призму награды, только теперь эта награда «суммарная» (то есть выгода). И это тоже неплохо!

В какой-то момент вы осознаете, что мир не вращается вокруг вас и что в жизни есть много факторов, о которых вы раньше не задумывались, которые сложно понять. Вам уже известно: «Что посеешь, то и пожнешь».

Ваше мировоззрение снова меняется: теперь вы преследуете не награду или выгоду, а ценность. Вы стали мудрее! Теперь вы знаете, что, помогая другим, можно многому научиться. Не знаю почему, но это работает. Чем сильнее вы любите свою вторую половинку, тем сильнее она будет любить вас в ответ. Вот так дела! Чем меньше денег вы тратите, тем больше можете потратить. Удивительно. Но заметьте, ваш эгоизм никуда не делся!

Вы начинаете замечать сложные закономерности окружающего мира и понимаете, что хорошее отношение к людям приносит больше всего выгоды, — беспроигрышная ситуация.

Я хочу, чтобы вы четко осознавали разницу между наградой, выгодой и функцией ценности. Подумайте над этим.

Ориентируйтесь на награду!

Затем на выгоду!

А после — на функцию ценности.

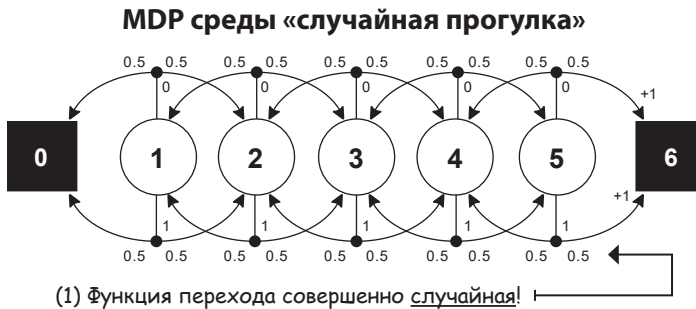


Конкретный пример

Среда «случайная прогулка»

Основная среда, которую мы будем использовать в этой главе, называется «случайная прогулка» (СП). Это однострочная грид-среда с пятью нетерминальными состояниями. Она довольно необычная, поэтому я хочу объяснить ее двумя способами.

С одной стороны, СП — это среда, где при выборе действия «влево» вероятность перемещения влево и вправо одинаковая. То же самое относится к действию «вправо». Проще говоря, агент никак не контролирует свои перемещения! Независимо от выбранного действия вероятность перехода влево или вправо равна 50 %. Это случайная прогулка как-никак. С ума сойти!



Но мне кажется, что это не самое удачное описание СП — возможно, мне просто хочется, чтобы агент на что-то влиял. Какой смысл в изучении методов RL, нацеленных на поиск оптимального подхода к управлению, если среда не позволяет нам ничем управлять?!

Поэтому давайте считать СП средой с детерминированной функцией перехода (то есть, если агент решает перейти влево или вправо, он туда и переходит — все предсказуемо). Теперь предположим, что агент хочет оценить стохастическую политику, которая выбирает действия равномерно случайно: в одной половине случаев она выбирает действие «влево», в другой — «вправо».

Но смысл не меняется: у нас есть «прогулка» по пяти нетерминальным состояниям, во время которой агент перемещается влево и вправо равномерно случайно. Наша задача — оценить общую ожидаемую дисконтированную награду, которую может получить агент в этих условиях.

Первый визит в Монте-Карло: повышение ценности после каждого эпизода

Итак, наша цель — предсказать ценность стратегии — узнать, какой общей награды следует от нее ожидать. Если быть точнее, цель в том, чтобы спрогнозировать функцию ценности состояний $v_{\pi}(s)$ политики π . Самый очевидный подход,

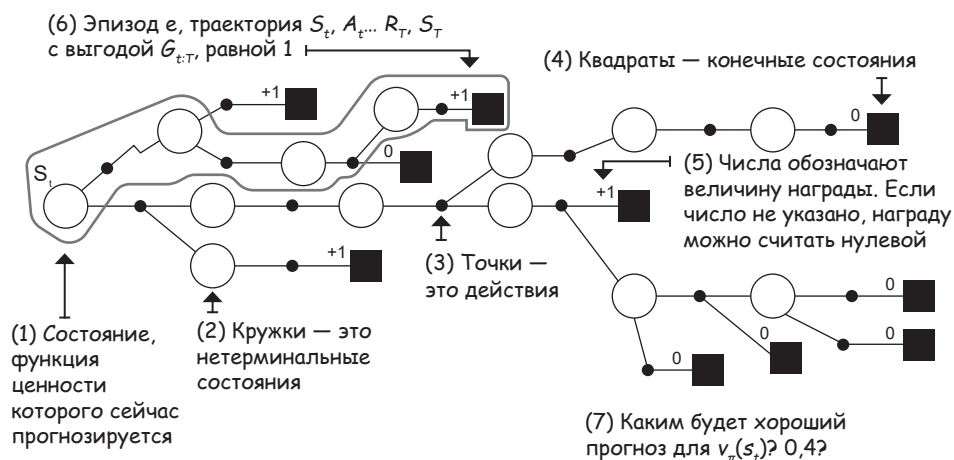
который мы можем здесь применить, — тот, о котором я уже упоминал, — выполнить несколько эпизодов с использованием этой политики, собрать сотни траекторий и затем посчитать средние показатели для каждого состояния. То же самое, что мы делали в «бандитских» средах. Этот способ называется *методом Монте-Карло* (Monte Carlo prediction, MC).

МС легко реализовать. Сначала агент взаимодействует со средой с помощью политики π , пока не попадет в конечное состояние S_T . Сочетание состояния S_t , действия A_t , награды R_{t+1} и следующего состояния S_{t+1} называется *кортежем опыта*, а набор таких кортежей — *траекторией*. Первым делом нужно заставить нашего агента сгенерировать траекторию.

Получив траекторию, вы можете вычислить выгоду $G_{t:T}$ для каждого состояния S_t . Например, если взять состояние S_t , мы начинаем с временного шага t и суммируем/дисконтируем полученные награды $R_{t+1}, R_{t+2}, R_{t+3} \dots R_T$, пока не будет достигнут конец траектории на временном шаге T . Затем мы повторяем этот процесс для состояния S_{t+1} , складывая дисконтированное вознаграждение с временным шагом $t + 1$, пока снова не достигнем T . То же мы делаем для S_{t+2} и всех остальных состояний, кроме S_T , ценность которого по определению — 0. В итоге $G_{t:T}$ будет использовать награды на всех временных шагах, начиная с $t + 1$ и вплоть до завершения эпизода на шаге T . Эти награды дисконтируются с помощью экспоненциально затухающего коэффициента: $\gamma^0, \gamma^1, \gamma^2 \dots \gamma^{T-1}$. Это значит, что мы умножаем награду R на соответствующий коэффициент дисконтирования γ и затем суммируем полученные произведения.

После генерации траектории и вычисления выгоды для всех состояний S_t можно оценить функцию значения состояния $v_\pi(s)$ в конце каждого эпизода e и на последнем временном шаге T . Для этого достаточно посчитать среднюю выгоду, полученную в каждом состоянии s . Проще говоря, мы оцениваем ожидание по средним значениям.

Метод Монте-Карло





Я знаю математику

Обучение методом Монте-Карло

(1) ВНИМАНИЕ: я активно пользуюсь обозначениями, чтобы вы получили полную картину происходящего. Будьте внимательны с тем, что за чем вычисляется. Например, если вам встречается нижний индекс $t:T$, значит, результат получен на временных шагах между t и конечным шагом T . T означает, что вычисления проводятся в конце эпизода на заключительном временном шаге T

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_{t:T} \mid S_t = s]$$

(2) Напомню, что функция ценности действий — это ожидание выгоды. Запомните это определение

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

(3) А выгода — это общая дисконтированная награда

(4) В МС мы первым делом выбираем политику для траектории

(5) На основе этой траектории можно вычислить выгоду для всех посещенных состояний

$$S_t, A_t, R_{t+1}, S_{t+1} \dots R_T, S_T \sim \pi_{t:T}$$

$$T_T(S_t) = T_T(S_t) + G_{t:T}$$

(6) Теперь суммируем выгоду для каждого состояния

$$N_T(S_t) = N_T(S_t) + 1$$

(7) И инкрементируем счетчик (подробнее об этом — чуть позже)

(8) Мы можем оценить ожидание с помощью выборочного среднего, поэтому спрогнозированная функция ценности для заданного состояния равна средней выгоде в этом состоянии

$$V_T(S_t) = \frac{T_T(S_t)}{N_T(S_t)}$$

(9) По мере того как счетчик приближается к бесконечности, прогноз становится все ближе к настоящему значению

$$N(s) \rightarrow \infty \quad V(s) \rightarrow v_{\pi}(s)$$

(10) Обратите внимание на то, что средние значения можно вычислять последовательно. Нам не нужно знать суммарную выгоду для всех состояний. Это более эффективный аналог уравнения в пункте 8

$$V_T(S_t) = V_{T-1}(S_t) + \frac{1}{N_t(S_t)} [G_{t:T} - V_{T-1}(S_t)]$$

(11) Здесь мы заменяем среднее значение тем, которое может быть константой или зависеть от времени

$$V_T(S_t) = V_{T-1}(S_t) + \alpha_t \left[\overbrace{G_{t:T} - V_{T-1}(S_t)}^{\text{МС погрешность}} \right]$$

МС цель

(12) Заметьте, что V вычисляется только в конце эпизода, на временном шаге T , поскольку от него зависит G

Каждый визит в Монте-Карло: другой подход к посещению состояний

Вы, наверное, заметили, что есть два разных способа реализации алгоритма усреднения выгоды. Дело в том, что состояния в рамках одной траектории могут посещаться повторно. Что лучше в таком случае: вычислять выгоду после каждого посещения по отдельности и затем учитывать все эти цели при получении среднего или использовать только первый переход в каждое состояние?

Оба эти подхода допустимы и теоретически обоснованны. Более «стандартная» версия — *первый визит MC* (first-visit MC, FVMC). Ее свойства сходимости легко обосновать, так как траектории не зависят друг от друга и являются одинаково распределенной выборкой $v_{\pi}(s)$. Когда количество выборок достигнет бесконечности, наши прогнозы сойдутся на реальных оценках.

Каждый визит MC (every-visit MC, EVMC) немного отличается: выгода больше не вычисляется отдельно и не распределена одинаково, так как состояния в рамках одной траектории могут посещаться по нескольку раз. Но, к счастью для нас, EVMC тоже гарантированно сходится при наличии бесконечного количества выборок.



По сути

Первый и каждый визит в Монте-Карло

В MC прогноз средней выгоды π равен $v_{\pi}(s)$. В FVMC в рамках каждого эпизода используется всего одна величина выгоды для каждого состояния. EVMC усредняет выгоду с учетом всех посещений состояния, даже если они произошли в одном эпизоде.

0001 Немного истории

Метод первого визита в Монте-Карло

Вы, наверное, уже встречали термин «симулятор Монте-Карло». Сам метод появился в 1940-х годах. Это широкий класс алгоритмов, использующих случайную выборку для оценки. Но «первый» и «каждый визит Монте-Карло» были впервые описаны в 1996 году в научной работе Сатиндера Сингха (Satinder Singh) и Ричарда Саттона (Richard Sutton) *Reinforcement Learning with Replacing Eligibility Traces*.

Сингх и Саттон получили свои докторские степени в области компьютерных наук в Массачусетском университете в Амхерсте, где под руководством профессора Энди Барто стали выдающимися деятелями в сфере RL. Они сделали огромный вклад в эту область и сейчас работают над Google DeepMind в качестве заслуженных ученых-исследователей. Рич защитил докторскую в 1984 году и занимает должность профессора в Альбертском университете, а Сатиндер получил свою степень в 1994 году. Он сейчас профессор в Университете Мичигана.



Я знаю Python

Экспоненциально затухающий график

```
def decay_schedule(init_value, min_value, ← (1) Эта функция
                                decay_ratio, max_steps,      позволяет
                                log_start=-2, log_base=10):    рассчитать все
    decay_steps = int(max_steps * decay_ratio)                значения alpha
    rem_steps = max_steps - decay_steps                        для всего процесса
                                                              обучения
(2) Сначала вычисляем количество этапов затухания ценности
с помощью переменной decay_ratio
(3) Затем вычисляем саму ценность на обратной логарифмической кривой.
Обратите внимание, что затем мы нормализуем значения между 0 и 1
и, наконец, преобразуем точки, чтобы они лежали между init_value и min_value
    values = np.logspace(log_start, 0, decay_steps,
                        base=log_base, endpoint=True)[: -1]
    values = (values - values.min()) / \
              (values.max() - values.min())
    values = (init_value - min_value) * values + min_value
    values = np.pad(values, (0, rem_steps), 'edge')
    return values
```



Я знаю Python

Создание полных траекторий

```
def generate_trajectory(pi, env, max_steps=20):
    done, trajectory = False, []
    while not done:
        state = env.reset()
        for t in count():
            action = pi(state)
            next_state, reward, done, _ = env.step(action)
            experience = (state, action, reward,
                        next_state, done)
            trajectory.append(experience)
        if done:
            break
        if t >= max_steps - 1:
            trajectory = []
            break
        state = next_state
    return np.array(trajectory, np.object)
```

(1) Это простая функция, которая выполняет стратегию и извлекает набор кортежей опыта (траекторий) для последующей обработки

(2) Это позволяет указать максимальное количество шагов, чтобы при желании мы могли сократить длинные траектории



Я знаю Python

Метод Монте-Карло

```
def mc_prediction(pi,  ← (1) Функция mc_prediction
    env,               подходит как для первого,
    gamma=1.0,         так и для каждого визита МС.
    init_alpha=0.5,     Здесь вы видите стандартные
    min_alpha=0.01,     гиперпараметры. Помните,
    alpha_decay_ratio=0.3, что коэффициент
    n_episodes=500,     дисконтирования, гамма,
    max_steps=100,      зависит от среды
    first_visit=True):
```

(2) В качестве темпа обучения, alpha, я использую значения от init_alpha (0,5) до min_alpha (0,01), затухающие на протяжении первых 30 % (alpha_decay_ratio = 0,3) эпизодов (max_episodes = 500). Мы уже обсуждали max_steps в прошлой функции, поэтому я просто передаю этот аргумент. А флаг first_visit позволяет переключаться между FVMC и EVMC

```
    nS = env.observation_space.n
    discounts = np.logspace(  ← (3) Вот это круто.
        0, max_steps, num=max_steps,      Я вычисляю сразу все
        base=gamma, endpoint=False)       возможные коэффициенты
    alphas = decay_schedule(              дисконтирования. Эта
        init_alpha, min_alpha,            функция logspace для
        alpha_decay_ratio, n_episodes)    гаммы 0,99 и max_step 100
    ← возвращает числовой
    ← вектор 100: [1; 0,99;
    ← 0,9801... 0,3697]
```

(4) Здесь я вычисляю все значения alpha!

(5) Здесь мы инициализируем переменные, которые будем использовать внутри основного цикла: текущий прогноз функции ценности состояний V и копия V из каждого эпизода для дальнейшего анализа

```
V = np.zeros(nS)
V_track = np.zeros((n_episodes, nS))
```

(6) Мы перебираем в цикле каждый эпизод. Заметьте, что здесь используется пакет tqdm, который выводит индикатор выполнения. Это полезно для таких нетерпеливых людей, как я. Если вы не из таких, вам он может не понадобиться

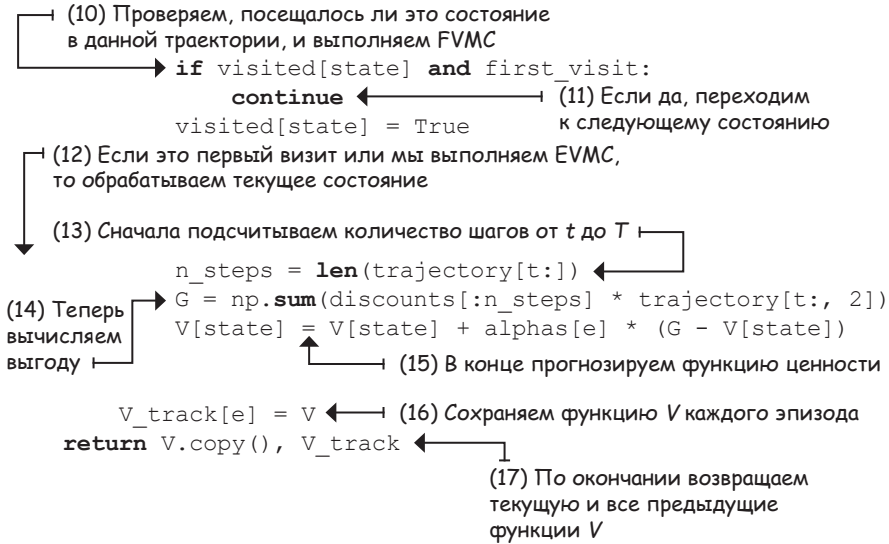
```
for e in tqdm(range(n_episodes), leave=False):
```

```
    trajectory = generate_trajectory(  ← (7) Генерируем
        pi, env, max_steps)           полную
    ← траекторию
```

(8) Инициализируем вектор булевых индикаторов посещения

```
    visited = np.zeros(nS, dtype=np.bool)
    for t, (state, _, reward, _, _) in enumerate(
        trajectory):
```

(9) Теперь перебираем все кортежи опыта в траектории



RL

С акцентом на RL

Инкрементальный и последовательные методы, методы проб и ошибок

Инкрементальные методы. Это постепенное улучшение прогнозов. Пример: алгоритмы динамического программирования, которые постепенно вычисляют ответы. Они не «взаимодействуют» со средой, а получают ответы через последовательное выполнение итераций, инкрементальное. «Бандиты» тоже инкрементальны: они получают хорошие аппроксимации на протяжении ряда эпизодов или попыток. Инкрементально и обучение с подкреплением. В зависимости от конкретного алгоритма значения улучшаются в каждом эпизоде или постепенно на каждом временном шаге.

Последовательные методы. Это обучение в среде с более чем одним нетерминальным (и достижимым) состоянием. Динамическое программирование относится к последовательным методам. «Бандиты» непоследовательны: это MDP с одним состоянием и одним шагом. У действий агента нет долгосрочных последствий. Обучение с подкреплением, несомненно, последовательное.

Методы проб и ошибок. Это обучение через взаимодействие со средой. Динамическое программирование, в отличие от «бандитских» сред и RL в целом, не относится к обучению методом проб и ошибок.

Временное разностное обучение: улучшение прогнозов после каждого шага

Один из важнейших недостатков МС — то, что агенту приходится ждать завершения эпизода, хотя реальную выгоду $G_{t:T}$ можно получить еще до того, как у него появится возможность обновить прогноз функции ценности состояний $V_t(S_t)$. С другой стороны, метод МС обладает довольно приличной сходимостью, так как он обновляет прогноз функции ценности $V_t(S_t)$ в направлении реальной выгоды $G_{t:T}$, что является несмещенной аппроксимацией реальной функции ценности состояний $v_\pi(s)$.

Но, несмотря на высокую степень корректности, фактическая выгода дает не самые точные прогнозы. Ожидаемая функция ценности состояний $v_\pi(s)$, которая из нее получается, обладает высокой дисперсией. Несложно понять почему: фактическая выгода накапливает много случайных событий в рамках одной траектории. Все действия, следующие состояния и награды — это случайные события. Фактическая выгода $G_{t:T}$ вбирает в себя все эти элементы случайности на протяжении многих временных шагов, от t до T . Повторюсь, несмотря на отсутствие смещения, она обладает высокой дисперсией.

Из-за этого метод МС может быть неэффективным с точки зрения выборки. Все эти элементы случайности становятся шумом, который можно подавить только огромным количеством данных, множеством траекторий и выборками реальной выгоды. Чтобы смягчить проблемы, вызванные высокой дисперсией, вместо реальной выгоды $G_{t:T}$ можно использовать прогнозируемую. Задумайтесь: ваш агент уже вычисляет ожидаемую версию $V(s)$ реальной функции ценности состояний $v_\pi(s)$. Как с ее помощью спрогнозировать выгоду хотя бы частично? Подумайте над этим!

Точно! Вы можете использовать выгоду за один шаг, R_{t+1} , и затем, увидев следующее состояние S_{t+1} , предсказать выгоду на следующем шаге $G_{t+1:T}$ с помощью ожидаемой функции ценности состояний $V(S_{t+1})$. На это отношение в уравнениях опираются методы *временных разностей* (temporal-difference, TD). В отличие от МС, они позволяют обучаться на незавершенных эпизодах, используя одношаговую (то есть немедленную награду R_{t+1}) выгоду, а затем оценку выгоды, ожидаемую от следующего состояния, которая представляет собой прогнозируемую функцию оценки следующего состояния $V(S_{t+1})$ — то есть $R_{t+1} + \gamma V(S_{t+1})$. Мы называем это **целью TD**.



По сути

Временное разностное обучение и бэггинг

Методы TD прогнозируют с помощью $v_\pi(s)$. Они делают предположения на основе агрегированных предположений: вместо реальной выгоды используется прогнозируемая. Если быть точнее, эти методы вычисляют и прогнозируют $V_{t+1}(S_t)$ с помощью $R_{t+1} + \gamma V(S_{t+1})$.

Это рабочий подход, так как при этом используется фактическая награда R_{t+1} на отдельно взятом шаге. Сигнал вознаграждения R_{t+1} постепенно «приближает к реальности» наши прогнозы.



Я знаю математику

Уравнения временного разностного обучения

(1) Мы снова начинаем с определения функции ценности состояний... $v_{\pi}(s) = \mathbb{E}_{\pi}[G_{t:T} \mid S_t = s]$

(2) ...и определения выгоды

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

(3) Мы можем видоизменить уравнение выгоды, сгруппировав некоторые члены. Взгляните:

$$\begin{aligned} G_{t:T} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots + \gamma^{T-2} R_T) \\ &= R_{t+1} + \gamma G_{t+1:T} \end{aligned}$$

(4) Теперь та же выгода записана в рекурсивном стиле

(5) С помощью этого нового определения мы можем изменить уравнение функции ценности состояний

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_{t:T} \mid S_t = s]$$

$$= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1:T} \mid S_t = s]$$

$$= \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s]$$

(6) И поскольку выгода, ожидаемая от следующего состояния, — это функция оценки этого состояния, мы получаем следующее

(7) Значит, мы можем спрогнозировать функцию ценности состояний на каждом временном шаге

$$S_t, A_t, R_{t+1}, S_{t+1} \sim \pi_{t:t+1}$$

(8) Берем отдельно взятый шаг взаимодействия...

(9) ...и получаем ожидаемую версию реальной функции ценности состояний $v_{\pi}(s)$ способом, отличным от МС

(10) Важно осознать ключевую разницу: мы теперь прогнозируем $v_{\pi}(s_t)$ на основе прогноза $v_{\pi}(s_{t+1})$. Используем не реальную выгоду, а ожидаемую

$$V_{t+1}(S_t) = V_t(S_t) + \alpha_t \left[\underbrace{R_{t+1} + \gamma V_t(S_{t+1})}_{\text{TD цель}} - V_t(S_t) \right]$$

TD погрешность

(11) У этого метода есть большое преимущество: он позволяет обновлять прогноз функции ценности состояний $v(s)$ на каждом временном шаге



Я знаю Python

Алгоритм временного разностного обучения

```
def td(pi, ← (1) td — это метод прогнозирования. Он принимает
    env,      стратегию pi, среду env, с которой будет
    gamma=1.0, происходить взаимодействие, и коэффициент
    init_alpha=0.5, дисконтирования gamma
    min_alpha=0.01, ← (2) У td есть гиперпараметр alpha,
    alpha_decay_ratio=0.3, регулирующий темп обучения
    n_episodes=500):
    (3) Один из множества способов регуляции скорости обучения — ee
    экспоненциальное затухание. init_alpha и min_alpha — это начальное
    и минимальное значения, а alpha_decay_ratio — доля эпизодов,
    на протяжении которых alpha будет затухать от init_alpha до min_alpha

    nS = env.observation_space.n
    V = np.zeros(nS) ← (4) Инициализируем нужные переменные
    V_track = np.zeros((n_episodes, nS))
    alphas = decay_schedule( ← (5) Вычисляем темпы
        init_alpha, min_alpha, обучения для всех
        alpha_decay_ratio, n_episodes) эпизодов...

    for e in tqdm(range(n_episodes), leave=False):
        (6) ...и циклически перебираем
        n_episodes эпизодов
        (7) Получив исходное состояние,
        мы входим в цикл взаимодействия
        state, done = env.reset(), False
        while not done:
            (8) Сначала нужно выбрать стратегию pi
            для действия, которое будет выполнено
            в этом состоянии
            action = pi(state)

        (9) Затем взаимодействуем со средой в соответствии с нашим
        выбором... Выполняем стратегию в рамках одного шага
        next_state, reward, done, _ = env.step(action)

        (10) Мы можем сразу же вычислить цель, чтобы обновить
        прогноз функции ценности состояний...
        td_target = reward + gamma * V[next_state] * \
            (not done)

        (11) ...а зная цель, можно
        получить погрешность
        td_error = td_target - V[state]
```

```

                                (12) Теперь обновляем  $v(s)$ 
                                ┌──────────────────────────────────────────┐
                                │  $V[state] = V[state] + \text{alphas}[e] * \text{td\_error}$  │
                                └──────────────────────────────────────────┘
                                │
                                │ (13) Не забываем обновить переменную
                                │ state для следующей итерации. Такие
                                │ ошибки сложно искать!
                                │
                                └──────────────────────────────────────────┘
                                │
                                │  $V\_track[e] = V$ 
                                │
                                │ return  $V, V\_track$ 
                                │
                                └──────────────────────────────────────────┘
                                (14) И возвращаем V-функцию и переменную
                                со всеми предыдущими V-функциями

```

RL

С акцентом на RL

Истинная функция ценности, фактическая выгода и прогноз

Истинная функция ценности. Это верная и совершенно точная функция, как будто описанная в пророчестве. Агент прогнозирует ее с помощью выборов. Истинная функция ценности поможет легко предсказать выгоду.

Фактическая выгода. Это полученная, а не прогнозируемая выгода. Агенты могут получать только фактическую выгоду, но они могут ее прогнозировать на основе ожидаемых функций ценности.

Прогнозируемая функция ценности или выгода. Это приблизительный расчет истинной функции ценности или фактической выгоды — аппроксимация, догадка, ожидание. Истинные функции ценности позволяют рассчитать выгоду, а прогнозируемые добавляют к этим значениям смещение.

Хочу подчеркнуть, что цель TD — это смещенная оценка истинной функции ценности состояний $v_{\pi}(s)$, так как для ее прогнозирования используется ее ожидаемая версия. Звучит странно, я знаю. Такой способ обновления прогнозов за счет прогнозирования называется бэггингом (bootstrapping). Принцип его действия во многом напоминает методы динамического программирования из главы 3. Но эти методы выполняют бэггинг с учетом ожидания на отдельно взятом временном шаге, тогда как методы TD делают это с учетом выборки одношаговых ожиданий. Слово «выборка» здесь играет огромную роль.

У такого подхода есть положительная сторона: ожидаемая выгода (цель TD) хоть и является смещенным прогнозом истинной функции ценности состояний $v_{\pi}(s)$, но зато имеет куда более низкую дисперсию по сравнению с фактической выгодой $G_{t:T}$, которая используется в обновлениях Монте-Карло. Это связано с тем, что цель TD зависит только от одного действия, одного перехода и одной награды, поэтому она накапливает намного меньше случайности. Благодаря этому методы TD обычно работают гораздо быстрее методов МС.

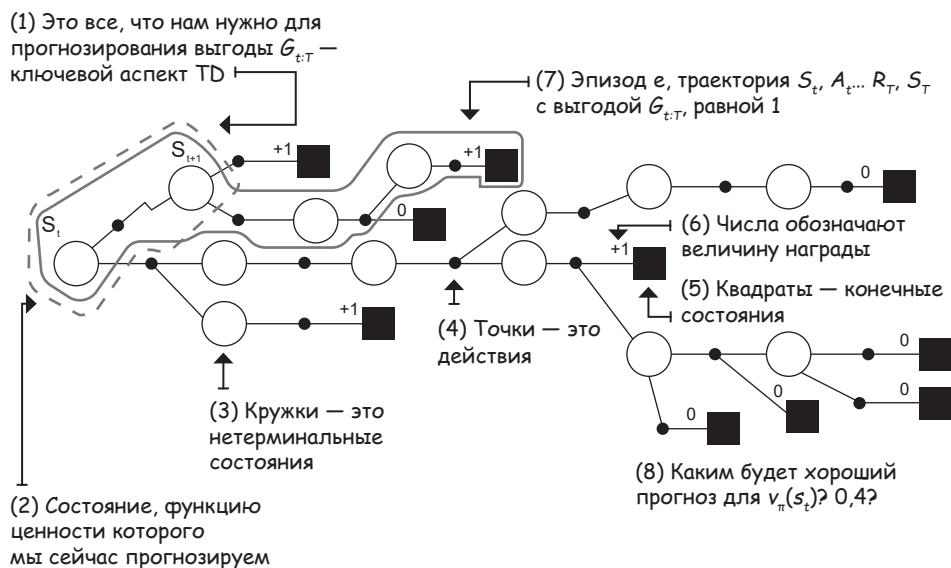
0001 Немного истории

Временное разностное обучение

В 1988 году Ричард Саттон (Richard Sutton) опубликовал научную работу *Learning to Predict by the Methods of Temporal Differences*, где представил методы обучения TD. Среда СП, которую мы используем в этой главе, тоже была впервые предложена там. Благодаря этой научной работе стало понятно, что, в отличие от методов МС, которые вычисляют погрешности на основе различий между прогнозируемой и фактической выгодой, методы TD способны учитывать разность между прогнозами для последовательных временных шагов — отсюда и название «временное разностное обучение».

Обучение TD легло в основу таких методов, как SARSA, Q-обучение, двойное Q-обучение, глубокие Q-сети (deep Q-networks, DQN), двойные глубокие Q-сети (double deep Q-networks, DDQN) и др. Мы изучим их позже.

Метод TD



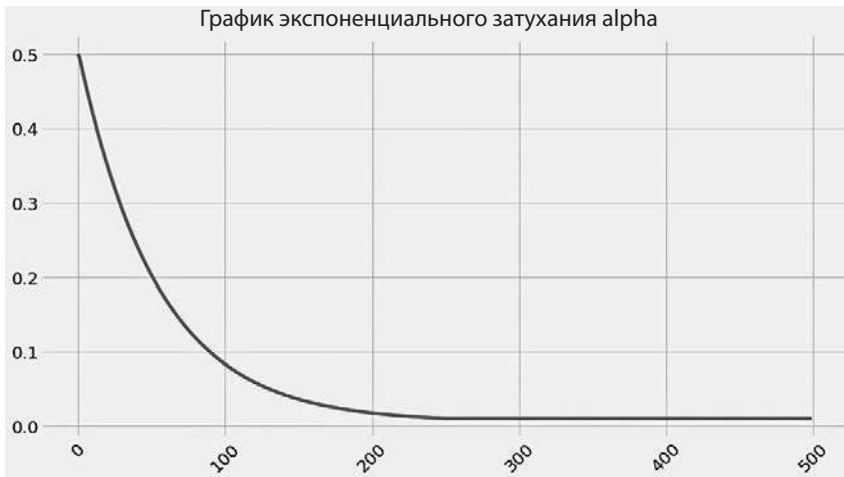


В деталях

FVMC, EVMC и TD в среде СП

Я запустил эти три алгоритма оценки политик в среде СП. В каждом случае оценивалась политика «всегда влево». Как вы помните, эта среда устроена так, что вероятность перехода влево или вправо всегда одинаковая — 50 %. В этом случае неважно, какую политику мы оцениваем.

Я использовал одинаковую конфигурацию темпа обучения, α , во всех трех алгоритмах: α начинается с 0,5 и уменьшается до 0,01 на протяжении 250 эпизодов — это 50 % общего количества эпизодов, 500. Это ключевой гиперпараметр. Обычно в качестве α используют положительную константу меньше 1. Постоянная величина α способствует обучению в нестационарных средах.



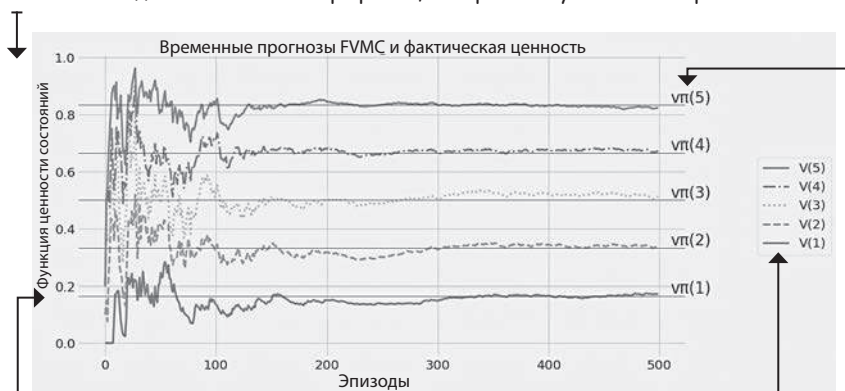
Но я решил сделать величину α затухающей, чтобы продемонстрировать сходимость. То, как она затухает, помогает алгоритмам приблизиться к точке схождения, но, так как я уменьшаю α до нуля, методы сходятся не до конца. В остальном эти результаты должны дать вам общее представление о различиях этих методов.



Посчитаем

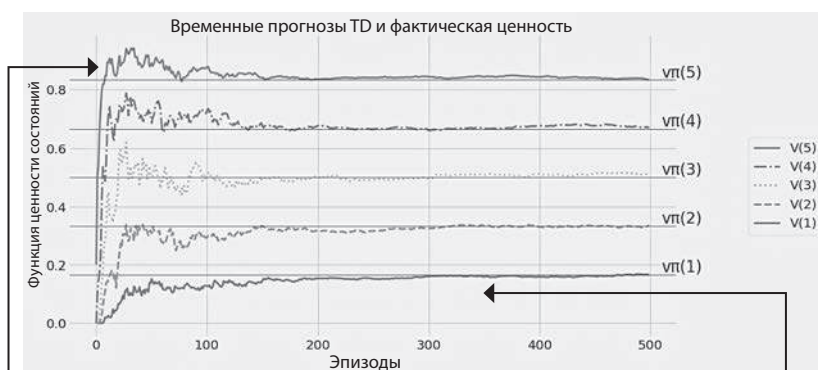
MC и TD почти сходятся на истинной функции ценности состояний

(1) Здесь я показываю только прогнозирование методом FVMC (первый визит Монте-Карло) и TD (временное разностное обучение). В блокноте Jupyter для этой главы можно найти результаты предсказаний EVMC (каждый визит Монте-Карло) и несколько дополнительных графиков, которые могут вас заинтересовать!



(2) Внимательно посмотрите на графики. Это динамика изменения прогнозов функции ценности состояний $v(s)$ для политики «всегда влево» в среде «случайная прогулка». Как видите, оба алгоритма почти сходятся на реальных оценках

(3) Теперь обратите внимание на отличительные тенденции этих алгоритмов. Динамика прогнозирования FVMC очень нестабильна: ожидаемые значения скачут вверх и вниз относительно реальных



(4) У TD динамика более стабильная, но на протяжении большинства эпизодов она смещена относительно центра. Например, $v(s)$ обычно находится выше, чем $v_{\pi}(s)$, а $v(1)$ — ниже, чем $v_{\pi}(1)$. Но если сравнить это с прогнозами FVMC, мы заметим другую тенденцию



Посчитаем

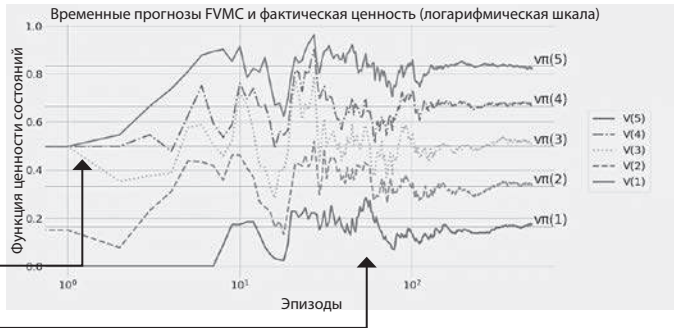
Прогнозы MC содержат много шума, прогнозы TD неточные

$$V_T(S_t) = V_{T-1}(S_t) + \alpha_t \left[\overbrace{G_{t:T} - V_{T-1}(S_t)}^{\text{MC погрешность}} \right]$$

MC цель ←

(1) Если взглянуть поближе на эти тенденции, можно увидеть, что происходит на самом деле: прогнозы MC колеблются вокруг реальной ценности. Это связано с высокой дисперсией целей MC

(2) Но есть и плюсы. Например, вы можете видеть, что все прогнозы приближаются к реальной ценности на очень раннем этапе и колеблются вокруг нее

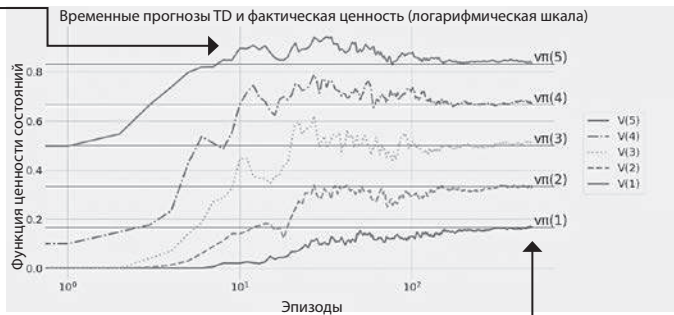


$$V_{t+1}(S_t) = V_t(S_t) + \alpha_t \left[\overbrace{G_{t:t+1} - V_t(S_t)}^{\text{TD погрешность}} \right]$$

TD цель ↑

(3) Прогнозы TD в основном остаются неточными, но зато не так сильно колеблются. Это связано с низкой дисперсией целей TD, которые при этом смещены. Они используют в качестве цели прогнозируемую выгоду

(4) Смещение тоже проявляется: при выборе целей TD жертвует точностью в угоду стабильности. К тому же, прежде чем прогнозы приблизятся к реальной ценности, должно пройти немного больше времени, по крайней мере в этой среде





Посчитаем

Высокая дисперсия целей MC, смещение целей TD

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

(1) Здесь мы видим баланс между дисперсией и смещением у целей MC и TD. Помните: MC в качестве цели использует выгоду, которая накапливает много шума. Из-за этого у целей высокая дисперсия

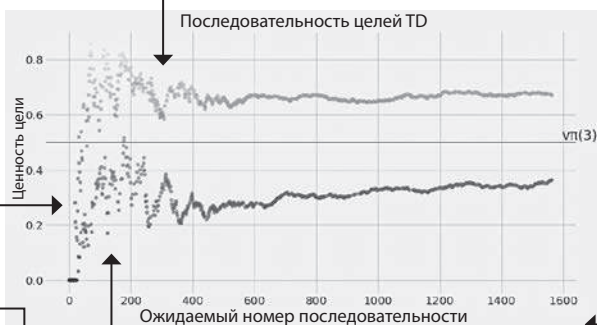
(2) На этих графиках показаны цели для начального состояния в среде СТП. Цели MC (выгода) равны либо 0, либо 1, поскольку эпизод заканчивается либо переходом влево (выгода 0), либо вправо (выгода 1), хотя оптимальное значение равно 0,5!



$$G_{t:t+1} = R_{t+1} + \gamma V_t(S_{t+1})$$

(3) Цели TD вычисляются на основе ожидаемой выгоды. Для прогноза оценок, которые будут получены, начиная со следующего состояния применяется функция ценности. Это помогает сделать вычисления компактнее и получить больше прогнозов в каждом эпизоде (как можно видеть на оси X, у нас ~1600 прогнозов в 500 эпизодах). Но, так как мы используем ожидаемую и неточную функцию, $V(S_{t+1})$, цели TD смещены

(4) Здесь видно, что диапазон целевых значений TD намного ниже. MC колеблется строго от 1 до 0, тогда как у TD этот диапазон находится примерно между 0,7 и 0,3, в зависимости от следующего выбранного состояния. Но, поскольку $V(S_{t+1})$ — это лишь прогноз, значение $G_{t:t+1}$ получается смещенным, нецелевым и неточным



Прогноз на основе нескольких шагов

В этой главе мы рассмотрели два основных алгоритма прогноза функций ценности заданной политики за счет взаимодействия. При использовании МС, прежде чем предсказывать функцию ценности, мы выбираем все действия в среде до завершения эпизода. Эти методы распределяют реальную выгоду (общую дисконтированную награду) между всеми состояниями. Например, если коэффициент дисконтирования меньше 1, а выгода равна либо 1, либо 0, как в случае со средой СП, то цель МС всегда будет равна 0 или 1 в каждом состоянии. Один и тот же сигнал передается обратно в самое начало траектории. Конечно, это не относится к средам с другими коэффициентами дисконтирования или функциями вознаграждения.

Для сравнения: при TD-обучении агент взаимодействует со средой лишь один раз. Он оценивает ожидаемую выгоду, прогнозирует цель, а затем функцию ценности. Методы TD выполняют бэггинг: формируют предположение из предположения. Это значит, что нам не нужно ждать завершения эпизода, чтобы получить реальную выгоду, как в случае с МС. Методы TD используют одноэтапное вознаграждение, прогнозируя ожидаемую выгоду — функцию ценности следующего состояния.

Но есть что-то среднее между этими методами? Здорово, что TD выполняет бэггинг после каждого шага, но что насчет двух шагов? Или трех? Четырех? Сколько шагов мы должны подождать, прежде чем оценить ожидаемую выгоду и получить функцию ценности?

Оказывается, между МС и TD есть целый спектр алгоритмов. В этом разделе мы рассмотрим те, что посередине. Вы увидите, что мы можем регулировать роль бэггинга в определении целей, уравнивая смещение и дисперсию.

Что находится в середине?



Аналогия от Мигеля

У МС и TD разный характер

Я отношусь к алгоритмам МС и TD как к личностям типа А и В соответственно. Чтобы понять, о чем речь, можете поискать в Интернете. Люди типа А ориентируются на результат, ценят свое время и отличаются деловитостью, тогда как представители типа В беззаботные, задумчивые и ведут себя как хиппи. То, что МС использует фактическую выгоду, а TD — прогнозируемую, заставляет задуматься над тем, отвечает ли поведение каждого из этих методов какому-то типу личности. Подумайте об этом. Я уверен, что вам удастся подметить несколько интересных закономерностей, которые помогут вам запомнить, чем МС отличается от TD.

N-шаговое TD-обучение: улучшение прогнозов после нескольких шагов

Вы должны четко понимать, чем мотивирован выбранный подход. У нас есть две крайности: методы Монте-Карло и методы временных разностей. Эффективность каждого метода может меняться в зависимости от обстоятельств. МС — это метод с бесконечным количеством шагов, потому что он продолжается до конца эпизода.

Я знаю, что это звучит странно, но вспомните, какое определение мы дали конечному состоянию во второй главе: состояние, в котором все действия и переходы, возникающие в результате этих действий, возвращаются обратно в исходное состояние без награды. Так агент «застревает» в этом цикле навсегда и, следовательно, выполняет бесконечное количество шагов, не накапливая награды и не обновляя функцию ценности состояний.

Для сравнения, TD — это одношаговый метод: прежде чем выполнять бэггинг и обновлять функцию ценности состояний, он взаимодействует со средой на протяжении одного шага. Эти два подхода можно обобщить в один n -шаговый метод. Вместо одного шага, как в случае с TD, или целого эпизода, как в МС, мы можем вычислять функции ценности с использованием n шагов, абстрагируясь от n . Этот способ называется *n -шаговым методом на основе TD*: он выполняет бэггинг на протяжении n шагов. Интересно, что промежуточное значение n часто работает лучше, чем любое из крайних значений. Видите, очень важно сохранять баланс!



Я знаю математику

N -шаговые уравнения временной разности

$$\longrightarrow S_t, A_t, R_{t+1}, S_{t+1} \dots R_{t+n}, S_{t+n} \sim \pi_{t:t+n}$$

(1) Заметьте, что прежде, чем обновлять $v(s)$ в n -шаговом TD, нужно дождаться завершения n шагов

(2) Значение n может быть любым: не ∞ , как в МС, или 1, как в TD. У вас есть выбор. На самом деле, если ваш агент достигнет конечного состояния, n будет равно или меньше этих показателей, но не больше

$$G_{t:t+n} = R_{t+1} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$$

(3) Здесь видно, как функция ценности состояний обновляется примерно каждые n шагов

$$V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha_t \left[\overbrace{G_{t:t+n} - V_{t+n-1}(S_t)}^{n\text{-шаговая погрешность}} \right]$$

(4) Но после этого можно установить эту цель как обычно $\longrightarrow$ n -шаговая цель



Я знаю Python

N-шаговый метод TD

```
def ntd(pi,
    env,
    gamma=1.0,
    init_alpha=0.5,
    min_alpha=0.01,
    alpha_decay_ratio=0.5,
    n_step=3,
    n_episodes=500):
```

(1) Это моя реализация n -шагового алгоритма TD. Его можно запрограммировать по-разному. В примере изображен лишь один из способов

(2) Здесь мы используем те же гиперпараметры, что и прежде. Заметьте, `n_step` по умолчанию равно 3. Это три шага перед бэггингом — или меньше, если мы попадаем в конечное состояние: тогда бэггинг не произойдет (напомню, что ценность конечного состояния по определению равна нулю)

```
    nS = env.observation_space.n
    V = np.zeros(nS)
    V_track = np.zeros((n_episodes, nS))
```

(3) Уже знакомые нам операции

```
    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio, n_episodes)
```

(4) Заранее вычисляем все значения `alpha`

(5) А вот гибрид MC и TD. Обратите внимание, что при вычислении коэффициента дисконтирования мы проходим не `max_steps` шагов, как в реализации MC, а `max_steps + 1`, чтобы получить n шагов и прогноз на основе бэггинга

```
    discounts = np.logspace(
        0, n_step+1, num=n_step+1, base=gamma, endpoint=False)
```

(6) Заходим в цикл обработки эпизодов

```
    for e in tqdm(range(n_episodes), leave=False):
```

(7) Переменная `path` будет хранить кортежи опыта за последние `n_step` шагов. Это частичная траектория

```
        state, done, path = env.reset(), False, []
```

(8) Продолжаем, пока не дойдем до `done` и пока `path` не будет равно `None`. Вскоре вы увидите зачем

```
        while not done or path is not None:
```

(9) Здесь мы извлекаем первый элемент `path`

```
            path = path[1:]

            while not done and len(path) < n_step:
```

(10) Это блок взаимодействия. По сути, мы накапливаем опыт, пока не дойдем до `done` и пока длина `path` не станет равной `n_step`

```
                action = pi(state)
                next_state, reward, done, _ = env.step(action)
                experience = (state, reward, next_state, done)
                path.append(experience)
                state = next_state
            if done:
                break
```

(11) n здесь может быть равно `n_step`, или меньше, если в `path` будет конечное состояние

```
n = len(path)
est_state = path[0][0]
```

(12) Здесь мы извлекаем не реальное, а прогнозируемое состояние

(13) `rewards` — это вектор всех вознаграждений, полученных между `est_state` и n

```
rewards = np.array(path)[:n, 1]
```

(14) `partial_return` — это вектор дисконтированных наград между `est_state` и n

```
partial_return = discounts[:n] * rewards
```

(15) `bs_val` — это оценка бэггинга. Заметьте, что здесь получено корректное следующее состояние

```
bs_val = discounts[-1] * V[next_state] * (not done)
```

(16) `ntd_target` — это сумма частичной выгоды и оценки бэггинга

```
ntd_target = np.sum(np.append(partial_return,
```

(17) Это погрешность (вычисляется так же, как и во всех предыдущих примерах)

```
bs_val))
ntd_error = ntd_target - V[est_state]
```

(18) Обновление функции ценности состояний

```
V[est_state] = V[est_state] + alphas[e] * ntd_error
```

(19) Здесь мы присваиваем `path` значение `None`, чтобы выйти из цикла обработки эпизодов, если `path` содержит только один кортеж опыта, а флаг `done` в этом кортеже равен `True` (то есть в `state` находится только конечное состояние)

```
if len(path) == 1 and path[0][3]:
    path = None
    V_track[e] = V
return V, V_track
```

(20) Как и прежде, возвращаем `V` и `V_track`

TD(λ) прямого обзора: улучшение прогнозов для всех посещенных состояний

Возникает вопрос: каким должно быть оптимальное значение n ? Когда использовать один шаг, два, три и т. д.? Я уже отмечал, что значения n больше 1 обычно дают лучшие результаты, но до самой выгоды тоже доходить не стоит. Бэггинг помогает, но его смещение приводит к определенным трудностям.



Я знаю математику

TD(λ) прямого обзора

(1) Это довольно запутанное уравнение. Давайте разберем его по частям. Здесь мы используем выгоду на всех n шагах, вплоть до заключительного шага T , взвешивая ее с помощью экспоненциально затухающего значения

(2) T — это переменная, поэтому нам нужно взвешивать реальную выгоду с помощью нормализованного значения, чтобы сумма весов была равна 1

$$G_{t:T}^{\lambda} = (1 - \lambda) \underbrace{\sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n}}_{\text{Суммарная взвешенная выгода шагов от 1 до } T-1} + \underbrace{\lambda^{T-t-1} G_{t:T}}_{\text{Окончательная взвешенная выгода (T)}}$$

(3) Уравнение сводится к тому, что мы вычисляем одношаговую выгоду и взвешиваем ее с помощью следующего коэффициента... $\rightarrow 1 - \lambda$

$$G_{t:t+1} = R_{t+1} + \gamma V_t(S_{t+1})$$

(4) ...затем мы берем двухшаговую выгоду и взвешиваем ее с помощью этого коэффициента $\rightarrow (1 - \lambda)\lambda$

$$G_{t:t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2})$$

(5) То же самое делаем для трехшаговой выгоды и этого коэффициента $\rightarrow (1 - \lambda)\lambda^2$

$$G_{t:t+3} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V_{t+2}(S_{t+3})$$

(6) Повторяем для всех n шагов... $\rightarrow (1 - \lambda)\lambda^{n-1}$

$$G_{t:t+n} = R_{t+1} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$$

(7) ...пока агент не достигнет конечного состояния, и взвешиваем с помощью этого коэффициента $\rightarrow \lambda^{T-t-1}$

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

(8) У такого подхода есть существенный минус: прежде чем вычислять эти значения, вам нужно сделать выборку всей траектории

(9) Получается, что V -функция становится доступной на шаге T ...

$$S_t, A_t, R_{t+1}, S_{t+1} \dots R_T, S_T \sim \pi_{t:T}$$

$$V_T(S_t) = V_{T-1}(S_t) + \alpha_t \left[\underbrace{G_{t:T}^{\lambda}}_{\lambda\text{-выгода}} - \underbrace{V_{T-1}(S_t)}_{\lambda\text{-погрешность}} \right]$$

(10) ...из-за этого

TD(λ): улучшение прогнозов для всех посещенных состояний после каждого шага

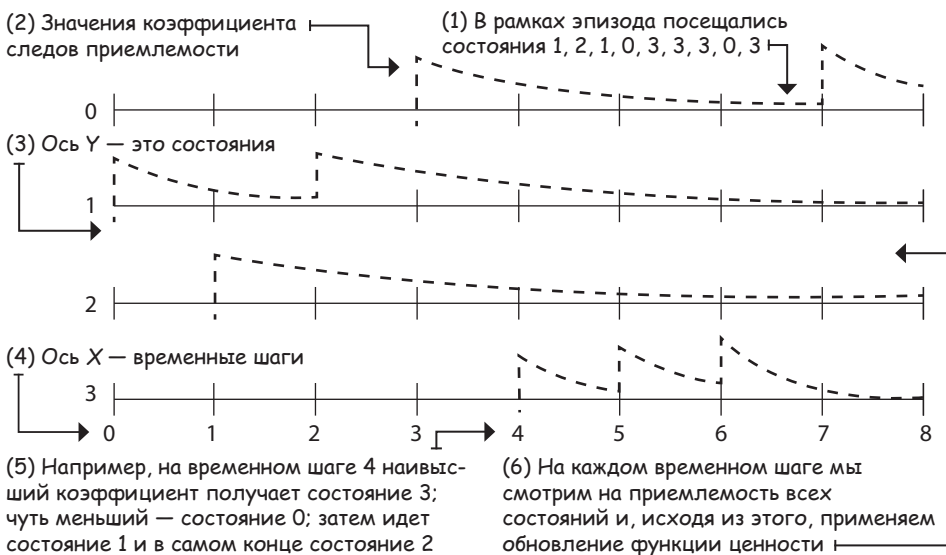
Методы MC находятся под «заклятием временного шага»: для обновления прогноза V-функции им нужно достичь конечного состояния. Заклятие сохраняется даже при использовании n -шагового бэггинга, поскольку обновить прогноз можно только по прошествии n взаимодействий со средой. По сути, вы играете в догонялки с отставанием в n шагов. Например, в пятишаговом бэггинге перед выполнением любых вычислений нам нужно посетить пять состояний (или меньше, если мы достигли конечного) и получить пять вознаграждений. Это чем-то похоже на методы MC.

TD(λ) прямого обзора возвращает нас обратно к MC с точки зрения временных шагов: прежде чем применять соответствующее обновление к прогнозам функции ценности состояний, мы должны ждать завершения эпизода. Но по крайней мере мы можем получить более низкую дисперсию, если нас устраивает смещение.

TD(λ) обратного обзора, сокращенно **TD(λ)**, не только обобщает и объединяет методы MC и TD. Он позволяет балансировать между смещением и дисперсией и применять обновления на каждом временном шаге, прямо как TD.

Механизм, дающий **TD(λ)** такое преимущество, называется *следами приемлемости (eligibility traces)*. Это вектор памяти, который хранит историю недавно посещенных состояний. Суть в том, что на каждом шаге мы отслеживаем состояния, в которых можно выполнить обновление. Мы получаем информацию не только о том, подходит состояние или нет, но и о том, насколько оно подходит. Так мы сможем верно применить соответствующие обновления к подходящим состояниям.

Следы приемлемости для среды с четырьмя состояниями во время восьмизагового эпизода



Например, все следы приемлемости изначально равны 0, и, встречая состояние, вы добавляете 1 к его следу. На каждом временном шаге вычисляется обновление V-функции для всех состояний и умножается на вектор следов приемлемости — так вы сможете обновлять только подходящие состояния. После вектор затухает на λ (коэффициент веса) и γ (коэффициент дисконтирования), чтобы снизить влияние будущих событий подкрепления на предыдущие состояния. Это позволяет последним состояниям получить более высокий коэффициент наград за последние переходы, по сравнению с посещенными на ранних стадиях эпизода, при условии, что λ не равна 1. Иначе это будет похоже на обновление MC, где все состояния, посещенные на протяжении эпизода, получают одинаковый коэффициент (без учета дисконтирования).



Я знаю математику

TD(λ) обратного обзора — TD(λ) со следами приемлемости, или просто TD(λ)

(1) В каждом новом эпизоде мы обнуляем вектор применимости $\mapsto E_0 = 0$

(2) Затем в течение одного цикла мы взаимодействуем со средой $\mapsto S_t, A_t, R_{t+1}, S_{t+1} \sim \pi_{t:t+1}$

(3) Посетив состояние S_t , делаем его подходящим для обновления.

По сути, мы просто инкрементируем его применимость на 1 $\mapsto$

(4) Затем, как и прежде, вычисляем погрешность TD $\mapsto E_t(S_t) = E_t(S_t) + 1$

(5) Но теперь мы обновляем ожидаемую функцию ценности состояний V — то есть всю функцию за раз и на каждом временном шаге! Заметьте, вместо V_t я использую $V_t(S_t)$. Поскольку мы умножаем на вектор применимости, все подходящие состояния получат соответствующий коэффициент

$$\delta_{t:t+1}^{TD}(S_t) = \underbrace{R_{t+1} + \gamma V_t(S_{t+1}) - V_t(S_t)}_{\text{TD цель}}$$

$$V_{t+1} = V_t + \alpha_t \underbrace{\delta_{t:t+1}^{TD}(S_t) E_t}_{\text{TD погрешность}}$$

(6) В конце уменьшаем применимость $\mapsto E_{t+1} = E_t \gamma \lambda$

И еще: когда $\lambda = 0$, TD(λ) ничем не отличается от уже знакомого нам метода TD. В связи с этим TD зачастую обозначают как TD(0). С другой стороны, когда $\lambda = 1$, метод TD(λ) превращается в MC. Ну почти: на самом деле мы получаем MC с обновлениями, которые накапливаются и применяются в конце эпизода. Онлайн-обновления подразумевают, что ожидаемая функция ценности состояний меняется на каждом шаге: прогнозы бэггинга варьируются, что влияет на

сам процесс прогнозирования. Но обычно предполагается, что $TD(1)$ равен MC . Более того, недавно появился метод *true online TD(λ)* — еще одна реализация $TD(\lambda)$, полностью равноценная $TD(0)$ в случае с TD и $TD(1)$ в случае с MC .



Я знаю Python

Алгоритм $TD(\lambda)$ или $TD(\lambda)$ обратного обзора

```
def td_lambda(pi,
              env,
              gamma=1.0,
              init_alpha=0.5,
              min_alpha=0.01,
              alpha_decay_ratio=0.3,
              lambda_=0.3,
              n_episodes=500):
    nS = env.observation_space.n
    V = np.zeros(nS)
    V_track = np.zeros((n_episodes, nS))
    E = np.zeros(nS)
    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio, n_episodes)
    for e in tqdm(range(n_episodes), leave=False):
        E.fill(0)
        state, done = env.reset(), False
        while not done:
            action = pi(state)
            next_state, reward, done, _ = env.step(action)
            td_target = reward + gamma * V[next_state]
            td_error = td_target - V[state]
            E[state] = E[state] + 1
            V = V + alphas[e] * td_error * E
            E = gamma * lambda_ * E
            state = next_state
            V_track[e] = V
    return V, V_track
```

(1) У метода `td_lambda` почти такая же сигнатура, как у всех остальных. Мы видим всего один новый гиперпараметр, `lambda_` (с подчеркиванием, потому что `lambda` — это ключевое слово в Python)

(2) Уже знакомые нам операции

(3) А это кое-что новое: вектор следов приемлемости

(4) Вычисляем `alpha` для всех эпизодов

(5) Здесь мы входим в цикл обработки эпизодов

(6) Обнуляем `E` в начале каждого нового эпизода

(7) Инициализируем переменные

(8) Входим в цикл временных шагов

(9) Сначала взаимодействуем со средой в рамках одного шага и получаем кортеж опыта

(10) Затем с помощью этого кортежа вычисляем погрешность TD (not done)

(11) Увеличиваем применимость состояния на 1

(12) И применяем обновление погрешности ко всем подходящим состояниям (`E`)

(13) `E` затухает...

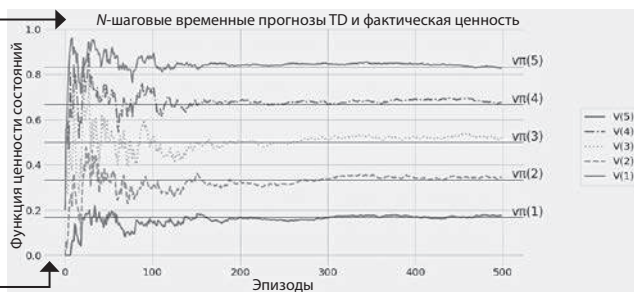
(14) ...а мы движемся дальше



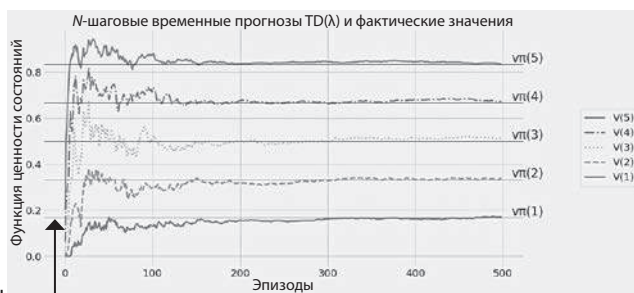
Посчитаем

N -шаговые прогнозы TD и $TD(\lambda)$ в среде СР

(1) Думаю, самые интересные различия и сходства МС, TD, n -шагового TD и $TD(\lambda)$ проявляются, если сопоставить эти методы друг с другом. Я настоятельно советую вам отправиться в репозиторий книги и ознакомиться с блокнотом Jupyter для этой главы. Там вы найдете гораздо больше, чем показано здесь



(2) Но пока я могу отметить, что кривые n -шагового TD напоминают МС: они шумные и центрированные, а $TD(\lambda)$ больше похожи на TD: гладкие и смещенные



(3) Если взглянуть на графики с логарифмической шкалой, можно увидеть, насколько высока дисперсия прогнозов n -шагового TD (по крайней мере выше, чем у $TD(\lambda)$ в этом эксперименте) и как прогнозы со временем отклоняются вверх и вниз относительно реальных оценок, даже несмотря на отсутствие смещения



(4) Значения $TD(\lambda)$ смещены, но выглядят более гладко, чем у МС. Это два любопытных свойства. Сравните их с остальными изученными методами!



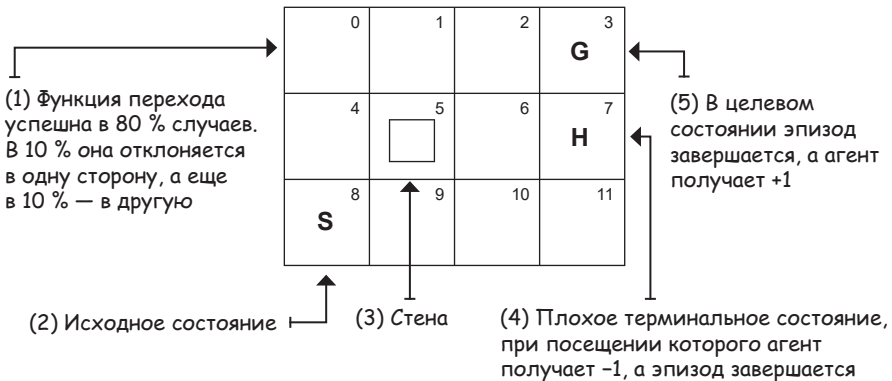


Конкретный пример

Оценка оптимальной политики в грид-среде Рассела и Норвига

Протестируем все эти алгоритмы в другой среде. Вы уже, наверное, не раз сталкивались с ней в прошлом. Она позаимствована из книги Рассела (Russell) и Норвига (Norvig) об ИИ.

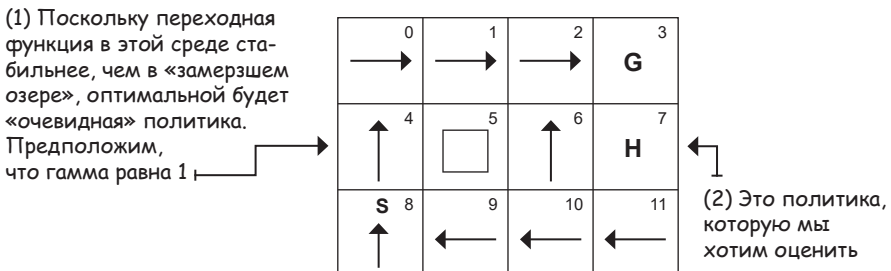
Грид-среда Рассела и Норвига



Это сетка Рассела и Норвига (CPN) размером 3×4 . Начиная в левом нижнем углу, агент должен достигнуть правого верхнего. Здесь, как и в среде «замёрзшее озеро», есть дыра (снизу от цели), а возле начала есть стена. Искажение переходной функции — 20 %: то есть 80 % действий завершаются успешно, а остальные 20 % — безуспешно равномерно в случайном порядке в ортогональном направлении. Функция вознаграждения возвращает штраф $-0,04$ на каждом шаге, за достижение цели агент получает $+1$, а за попадание в дыру — -1 .

Теперь займемся оценкой политики. Так получилось, что я уже добавил оптимальную политику в блокнот Jupyter для главы 3, хотя для ее обсуждения у меня не хватило места. Старайтесь просматривать все блокноты для этой книги.

Оптимальная политика в среде CPN

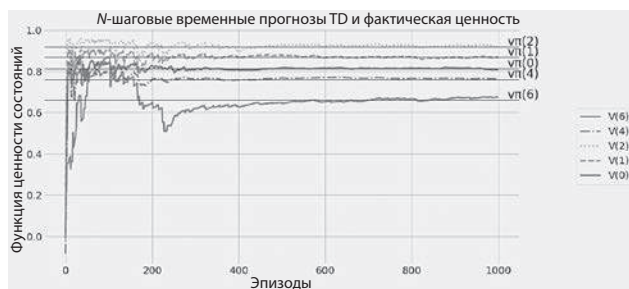
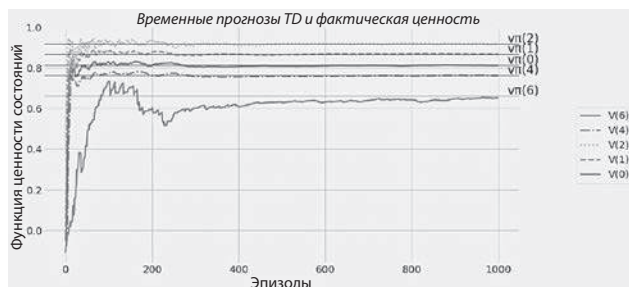
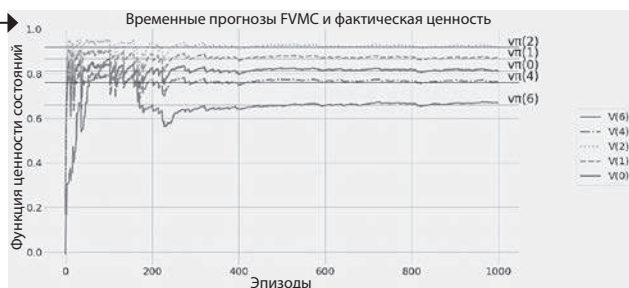




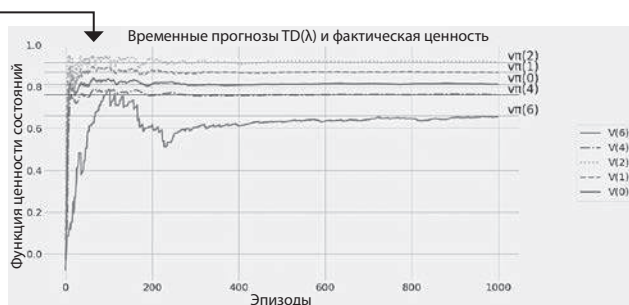
Посчитаем

FVMC, TD, n-шаговый TD и $TD(\lambda)$ в среде CPH

(1) Я запустил те же гиперпараметры, что и раньше, но уже на протяжении 1000 эпизодов (а не 500, как в случае с СТП). Результаты справа — динамика изменения прогнозов V -функци для 5 из 12 случайно выбранных состояний (но с одинаковым начальным значением в каждом графике для удобства сравнения. Выбор был не совсем случайным: я сначала отфильтровал ожидаемые значения, не достигшие порогового значения, 0,1). Все это я сделал, чтобы вам было легче заметить важные тенденции, несмотря на малое количество состояний



(2) Как видите, все четыре алгоритма (в блокноте можно найти пятый!) дают довольно хороший прогноз реальной функции ценности состояний. Если присмотреться, можно заметить, что у TD и $TD(\lambda)$ самые гладкие кривые, а MC и n -шаговый TD отличаются наименьшим смещением



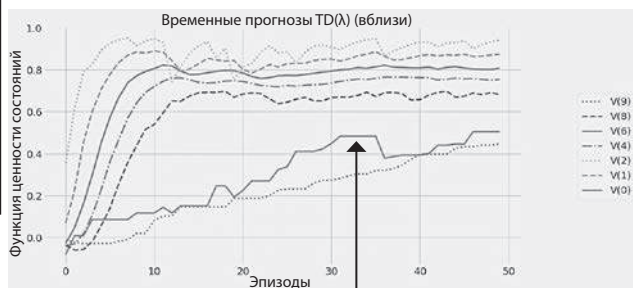
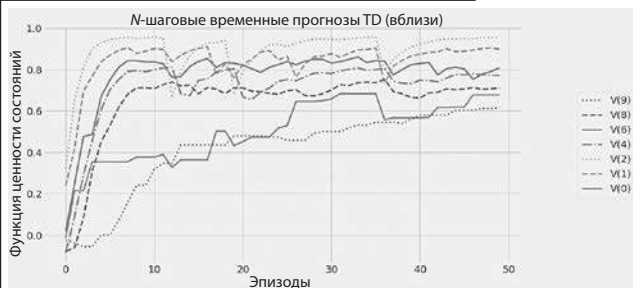
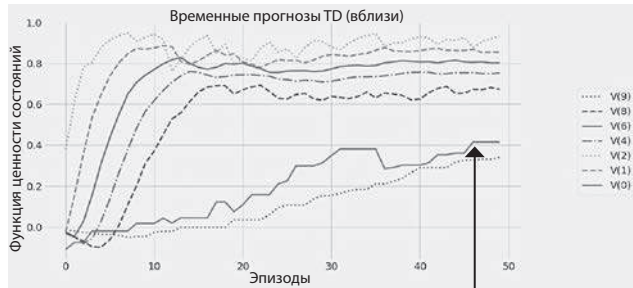
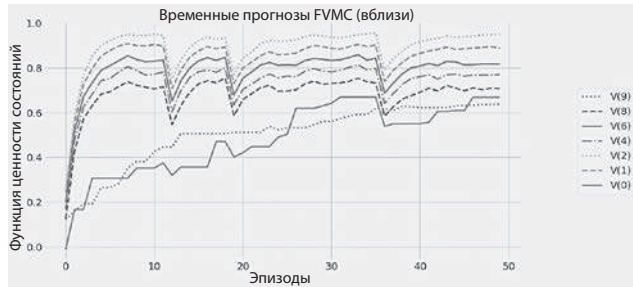


Посчитаем

CRN немного лучше иллюстрирует влияние смещения и дисперсии на прогнозы

(1) Я подумал, что вам будет полезно поближе взглянуть на начальную часть этих кривых. Это не логарифмические графики вроде тех, что я показывал раньше, а отрезок из первых 50 эпизодов. Здесь изображены только значения выше 0,1, но, как вы видите, они охватывают большинство состояний. Функции ценности в состояниях 3, 5 и 7 равны 0, а состояния 10 и 11 далеки от того, чтобы соответствовать оптимальной политике, так как действия в состояниях 9 и 6 направлены влево и вверх, вне состояний 10 и 11.

(2) Внимательно рассмотрите тенденции здесь. Их легче заметить. Например, на графике FVMC мы видим резкие всплески и падения. График TD гладкий, но медленно растущий. Где-то посередине находится n -шаговый TD. Интересно, что $TD(\lambda)$ обладает той же гладкостью, что и TD, но при этом растет быстрее. Например, взгляните на кривую V(6): в районе 25-го эпизода она преодолевает отметку 0,4 и только к 45 возвращается к уровню TD



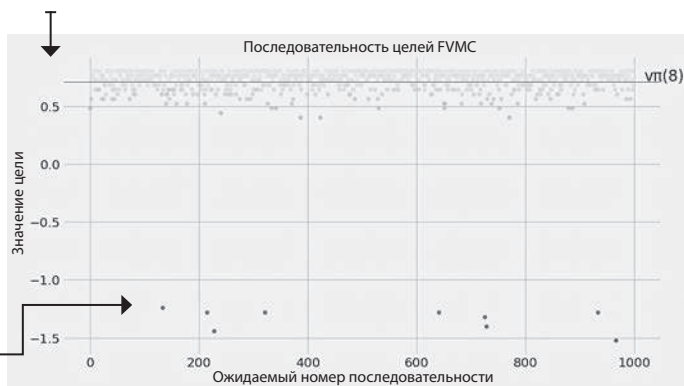


Посчитаем

Цели FVMC и TD в исходном состоянии СРН

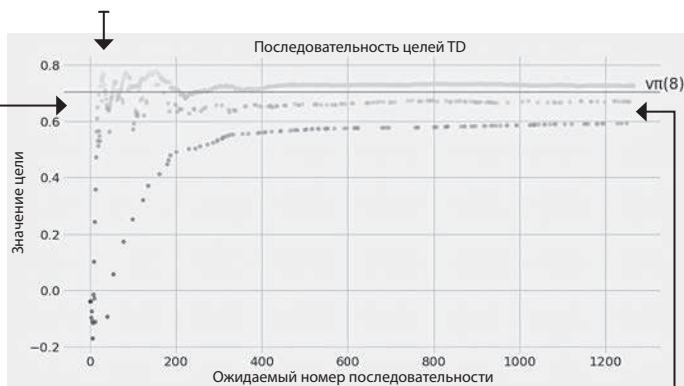
(1) Эти итоговые графики отражают последовательности целевых значений в исходном состоянии. Как и следовало ожидать, цели МС не зависят от номера последовательности, так как отражают реальную выгоду и не предусматривают бэджинга для функции ценности состояний

(2) Вы могли заметить и их высокую дисперсию: большинство из них сосредоточены вверху, но несколько значений находится здесь



(3) У целей TD чуть более тесная связь с последовательностью. Заметьте, что в самом начале они крайне неточные и довольно сильно колеблются, но со временем они накапливаются и становятся более стабильными

(4) Вы можете заметить начало формирования трех прямых. Напомню, что это цели для начального состояния 8. Если взглянуть на стратегию, то можно заметить, что путь вверх к этому состоянию может занимать только три перехода...



(5) ...из них 80 % переносят агент в состояние 4 (вверх), 10 % — обратно в состояние 8 (влево) и еще 10 % — в состояние 9 (вправо). Можете ли вы определить, какому «следующему состоянию» соответствует каждая из прямых? Аргументируйте свой выбор. Поэкспериментируйте!

Подведем итоги

Обучение на последовательной обратной связи — сложный процесс, с которым вы подробно познакомились в главе 3. Мы вместе с вами создали агенты, балансирующие между краткосрочными и долгосрочными целями. Такие методы, как итерация ценностей (value iteration, VI) и итерация политик (policy iteration, PI), лежат в основе RL. В обучении на оценочной обратной связи тоже немало трудностей. Глава 4 была полностью посвящена конкретному типу сред, где агенты должны учиться балансировать между исследованием и эксплуатацией. Эпсилон-жадная стратегия, softmax, оптимистичная инициализация и прочие подобные методики — это тоже неотъемлемая часть RL.

Я хочу, чтобы вы на секунду остановились и еще раз подумали об этих двух компромиссах. Мне встречались учебники на 500 и более страниц, посвященные каждому из них, тогда как у нас на знакомство с ними было всего по 30. Это хорошо или плохо? Если вы хотите разработать новые алгоритмы DRL, чтобы продвинуться вперед, я рекомендую вам изучить эти два компромисса независимо друг от друга. Ищите книги об алгоритмах планирования и «бандитских» алгоритмах, потратьте время и силы на то, чтобы как следует понять эти направления. Вернувшись обратно к RL и увидев все закономерности, вы почувствуете, что сделали огромный скачок вперед. Но если вы хотите просто разобраться в DRL, чтобы реализовать парочку методов для использования в своих проектах, вам будет достаточно того, что вы здесь прочли.

В этой главе вы узнали об агентах, способных работать с одновременно последовательной и оценочной обратной связью. И, повторяюсь, это довольно существенный шаг вперед! Порой даже не всем людям под силу найти двойной баланс между краткосрочными и долгосрочными целями и между исследованием и эксплуатацией! В этой главе мы углубились в изучение задачи прогнозирования — предсказание ценности поведения агента. Для этого мы обсудили метод Монте-Карло и временное разностное обучение. Это два полностью противоположных подхода, а все, что между ними, можно обобщить с помощью n -шагового агента TD. Благодаря простому изменению размера шага можно получить почти любой тип промежуточных агентов. Позже мы рассмотрели $TD(\lambda)$, и вы узнали, что в одном и том же агенте можно сочетать обе крайности и все, что между ними.

Далее речь пойдет о задаче управления — об улучшении поведения агентов. Ранее мы уже разделяли алгоритм итерации политик на две части: оценку политики и ее оптимизацию. Точно так же обучение с подкреплением можно разделить на задачи прогнозирования и управления, что позволит нам углубиться в детали и выработать лучшие методы.

На этом этапе вы:

- понимаете, что трудности обучения с подкреплением вызваны тем, что агенты не могут видеть базовую MDP, управляющую их развивающейся средой;
- знаете, что сочетание этих двух проблем привело к появлению области RL;
- знакомы со многими способами расчета целей для прогнозирования функций ценности состояний.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

- **#gdr1_ch05_tf01.** Ни один метод из этой главы не учитывает ограничение временных шагов, которое вплетено во многие тренировочные среды. Скорее всего, вы даже не понимаете, о чем я. Не волнуйтесь, я все подробно объясню в главе 8. А пока можете взглянуть на этот файл: https://github.com/openai/gym/blob/master/gym/envs/__init__.py. Как видите, во многих средах, включая «замерзшее озеро», есть переменная `max_episode_steps`. Это ограничение временных шагов, наложенное на среду. Подумайте, как оно влияет на алгоритмы из этой главы. Откройте блокнот Jupyter, приложенный к этой книге, и измените алгоритмы так, чтобы они правильно обрабатывали ограничение временного шага, а значения функции ценности были более точными. Поменялись ли в результате функции ценности? Как думаете почему? Если вы все еще не понимаете, о чем речь, продолжайте читать — вы всегда сможете вернуться, когда все прояснится.
- **#gdr1_ch05_tf02.** Сравнение графиков целевых значений метода Монте-Карло и временных разностей дало свои плоды. Чтобы лучше понять, чем отличаются эти два подхода, можно провести более тщательный анализ их целей и сравнить их с целями n -шагового TD(λ). Найдите другие способы сравнения этих методов прогнозирования. Используйте графики и прочие визуальные приемы!
- **#gdr1_ch05_tf03.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdr1_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы узнаете о способах оптимизации политик при обучении на одновременно последовательной и оценочной обратной связи.
- ✓ Разработаете алгоритмы поиска оптимальных политик в средах обучения с подкреплением, где функции перехода и вознаграждения неизвестны.
- ✓ Напишете код для агентов, способных приходить от случайного поведения к оптимальному только с помощью своего опыта и принятия решений, и обучите их в разных средах.

Если вам кажется, что цель недостижима, не меняйте ее — меняйте свой план действий.

*Конфуций, китайский учитель,
редактор, политик и философ периода
Весен и Осеней в китайской истории*

Ранее вы изучали обособленное и взаимосвязанное обучение на основе двух из трех типов обратной связи, с которыми должен иметь дело агент обучения с подкреплением: последовательной, оценочной и выборочной. В главе 2 вы научились представлять последовательные задачи принятия решений с помощью *марковского процесса принятия решений* (Markov decision process, MDP). В главе 3 вы узнали, как эти задачи можно выполнять с помощью алгоритмов, которые извлекают политики из MDP. В главе 4 вы научились выполнять простые задачи управления с несколькими вариантами, из которых нужно выбрать один, «многорукими бандитами», когда агенту недоступно представление MDP. Наконец, в главе 5 мы объединили эти два вида задач управления: то есть мы имели дело с последовательными и неопределенными задачами, но при этом нас интересовал только прогноз функций ценности. Мы выполнили так называемую задачу прогнозирования, состоящую в оценке политик или в определении предполагаемой выгоды.

В этой главе вы познакомитесь с агентами, выполняющими задачи управления. Для этого нам нужно будет изменить две составляющие. Во-первых, мы будем прогнозировать не функции ценности состояний, $V(s)$, а функции ценности действий, $Q(s, a)$. Основная причина в том, что Q -функции, в отличие от V -функций, позволяют нам узнавать ценность действий без использования MDP. Во-вторых, мы можем применять полученные ожидаемые Q -значения для улучшения политик. Это похоже на то, чем мы занимались в алгоритме итерации политик: мы оцениваем, оптимизируем, затем оцениваем улучшенную стратегию, потом улучшаем ее дальше и т. д. Как уже отмечалось в главе 3, этот процесс называется *обобщенной итерацией политик* (generalized policy iteration, GPI) и с его помощью можно создать архитектуру, для которой подходит практически любой алгоритм обучения с подкреплением, включая передовые агенты DRL.

Общий план главы выглядит так: сначала я подробно расскажу об архитектуре обобщенной итерации политик, а потом вы познакомитесь с разными видами агентов для выполнения задач управления, включая специальные версии методов Монте-Карло и временного разностного обучения. Вы узнаете об агентах, в которых обучение отделено от поведения. По сути, в этой главе вы будете заниматься разработкой агентов для выполнения задач методом проб и ошибок, которые будут искать оптимальные политики только на основе своего взаимодействия со средой.

Анатомия агентов обучения с подкреплением

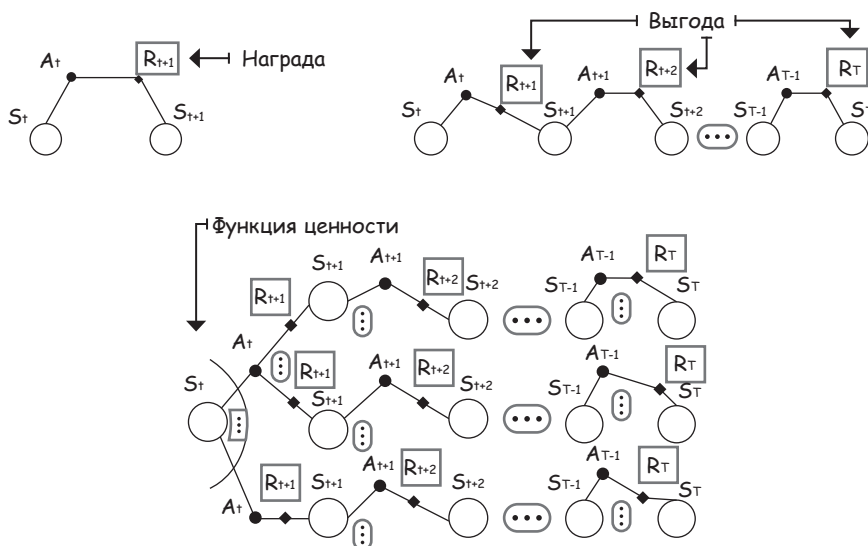
Здесь я хочу изобразить модель, под которую подходят если не все агенты обучения с подкреплением, то точно их большинство. Во-первых, каждый агент RL накапливает опыт — либо взаимодействуя со средой, либо обращаясь к ее модели, которую он сформировал. Во-вторых, он учится что-то оценивать или прогнозировать: модель среды, политику, функцию ценности или просто выгоду. В-третьих, он пытается улучшить политику — в конце концов, в этом вся суть обучения с подкреплением.



Освежим в памяти

Награда, выгода и функция ценности

Самое время вспомнить некоторые понятия. Важно помнить, чем отличаются награда, выгода и функция ценности, чтобы у вас не возникало трудностей при чтении этой главы и вы могли разрабатывать агенты, которые подбирают оптимальные политики в процессе обучения методом проб и ошибок. Давайте повторим пройденное.



Награда — это числовой сигнал, указывающий на качество перехода. Находясь в состоянии S_t , ваш агент выбирает действие A_t . В результате среда меняется, возвращает награду $R_t + 1$ и переходит в новое состояние S_{t+1} . Награда отображает положительность перехода, выполняемого на каждом временном шаге эпизода.

Выгода — это сумма всех наград, полученных за эпизод. Ваш агент получает награды R_{t+1}, R_{t+2} и т. д., вплоть до R_T , получаемую перед переходом в конечное состояние S_T . Выгода часто определяется как дисконтированная сумма, где приоритет отдается наградам, полученным на ранних этапах эпизода (в зависимости от коэффициента дисконтирования, конечно). Строго говоря, дисконтированная сумма — это более общее определение выгоды, так как коэффициент дисконтирования, равный 1, превращает ее в обычную сумму.

Функция ценности — это ожидаемая выгода, где ожидание вычисляется как сумма всех возможных оценок, каждая из которых умножена на вероятность ее получения. Можете считать ее средним бесконечным числом выборов. Ожидаемая выгода похожа на выборку бесконечного числа экземпляров выгоды и вычисление их среднего значения. При вычислении выгоды после выбора действия ожидание отражает функцию ценности действий для пары «состояние — действие» $Q(s, a)$. Если вы проигнорируете предпринятое действие и будете считать от состояния s , оно станет функцией значения состояния $V(s)$.

Агенты собирают образцы опыта

Одна из особенностей RL в том, что агенты обучаются методом проб и ошибок. Они взаимодействуют со средой и одновременно собирают данные. Необычный аспект в том, что сбор данных и обучение на них — это два отдельных процесса. Вскоре вы увидите, что обучение на основе данных отличается от оптимизации политики с их помощью. В RL обучение осуществляется в три этапа: сбор данных, обучение и улучшение. Например, агент, который довольно хорошо справляется со сбором данных, может плохо на них обучаться, и наоборот. Наверное, у каждого был приятель, который ужасно вел конспекты, но затем получал хорошие оценки, тогда как остальные студенты записывали каждое слово, но не могли похвастаться тем же уровнем успеваемости.

Когда вы узнали о методах динамического программирования в главе 3, я упомянул, что итерации значений и политик нужно называть не RL, а методами планирования, поскольку они не собирают данные. Им не нужно взаимодействовать со средой, так как ее модель, MDP, предоставляется заранее.

RL

С акцентом на RL

Задачи планирования и обучения

Задачи планирования. Это задачи, где доступна модель среды, что избавляет нас от потребности в обучении. Их можно выполнять с помощью таких методов планирования, как итерация ценности и политик. Такие задачи нацелены на поиск подходящих политик, а не на их формирование. Представьте, что я дал вам карту и попросил найти лучший маршрут из точки А в точку Б. В этом случае вам не нужно ничему учиться — достаточно ограничиться планированием.

Задачи обучения. Это задачи, в которых нужно учиться на выборках, поскольку модель среды либо недоступна, либо не может быть создана в принципе. Основная трудность таких задач в том, что мы даем оценки на основе выборок с высокой дисперсией и, следовательно, низким качеством. На таких сложно учиться. Выборки могут быть смещенными, если их взяли не из того распределения, которое оценивается, или когда из одних оценок выводятся другие, что может сделать их ошибочными. Представьте, что на этот раз карты у вас нет. Как бы вы нашли лучший маршрут? Вероятно, методом проб и ошибок.

Чтобы алгоритм считался стандартным методом RL, должен быть аспект взаимодействия со средой, с задачей, которую мы пытаемся выполнить. Большинство агентов RL накапливают опыт самостоятельно, что отличает их от методов контролируемого обучения, где предоставляется готовый набор данных. Агенты RL должны сами выбирать подходящие наборы, и это еще одна трудность, которую им приходится преодолевать. Это объясняется тем, что RL зачастую сводится к выполнению интерактивных задач обучения.



С акцентом на RL

Интерактивные и неинтерактивные задачи обучения

Неинтерактивные задачи обучения. Это тип задач, в которых не нужно или нельзя взаимодействовать со средой. Обучение происходит на сгенерированных заранее данных. Цель этих задач — на основе предоставленных выборок найти политику или что-то другое. Например, обратное RL направлено на воссоздание функции вознаграждения на основе экспертных характеристик. При обучении на демонстрации агент пытается получить из этой воссозданной функции вознаграждения политику. Цель методов поведенческого клонирования (разновидность имитационного обучения) — переход от образцов экспертных характеристик к самим политикам с помощью контролируемого обучения.

Интерактивные задачи обучения. Это тип задач, где обучение и взаимодействие чередуются. У них есть интересная особенность: обучающийся управляет и процессом сбора данных. Оптимальное обучение на выборках и поиск выборок для оптимального обучения — это два разных процесса.

Агенты что-то оценивают

Агент может много что делать с собранными данными. Некоторые, к примеру, учатся прогнозировать ожидаемую *выгоду* или функции ценности. Ранее вы познакомились с разными способами, которыми это можно делать: от методов Монте-Карло до TD, от EVMC до FVMC, от n -шаговых методов до λ -выгоды. Цели, которые используются для прогноза функции ценности, тоже можно вычислять по-разному.

Но на основе накопленного опыта агенты могут формировать не только функции ценности, но и модели среды. В следующей главе вы увидите, что агенты RL, ориентированные на модели, используют собранные данные для формирования переходных функций и вознаграждения. Создавая модель среды, агент может оценивать дальнейшее состояние и награду. Более того, с помощью этих прогнозов можно спланировать последовательность действий, как это делают методы динамического программирования. Или мы можем взять синтетические данные, сгенерированные при взаимодействии со сформированными моделями, чтобы научиться чему-то другому. Основная идея в том, что агенты могут быть предназначены для изучения моделей среды.

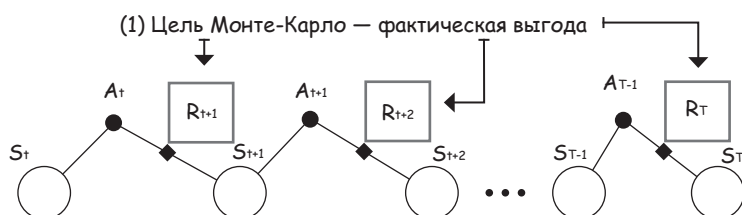
Кроме того, агенты могут заниматься улучшением политик, напрямую используя ожидаемую выгоду. В следующих главах вы познакомитесь с методами на основе градиента политик, основанных на аппроксимации функций, которые могут использовать как реальную выгоду (в простейшем случае), так и спрогнозированные функции ценности. Еще отмечу, что агенты могут оценивать сразу несколько параметров, и это самый распространенный подход, так как большинство агентов всегда что-то оценивают.



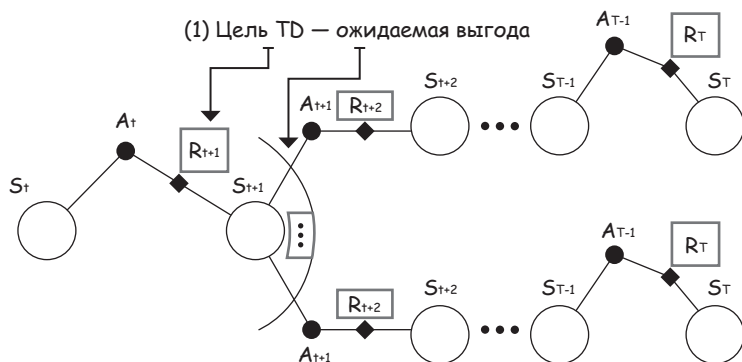
Освежим в памяти

Цели Монте-Карло и временных разностей

Еще раз подчеркну, что функции ценности можно прогнозировать по-разному, но все методы постепенно приближают свои прогнозы к цели, снижая погрешность. Общее уравнение, которое лежит в основе большинства методов обучения, выглядит так: *прогноз* = *прогноз* + *шаг* × *погрешность*. Погрешность — это просто разность между полученной целью и ее текущей ценностью (*цель* – *ценность*). Два основных противоположных способа вычисления этих целей: метод Монте-Карло и временное разностное обучение.



Цель Монте-Карло — фактическая выгода, ничего более. Процесс прогнозирования заключается в корректировке ожидаемой функции ценности на основе наблюдаемой средней выгоды вместо ожидаемой (как если бы мы могли усреднить бесконечные выборки).



Целевой показатель временной разницы состоит из расчетной выгоды. Помните бэггинг? По сути, он заключается в использовании прогнозируемой выгоды, которую мы ожидаем получить в будущих состояниях, для оценки ожидаемой выгоды в текущем. Именно этим занимается TD: получением одной догадки из другой. Цель TD формируется с помощью отдельно взятой награды и прогнозируемой выгоды, ожидаемой в следующем состоянии. Для этого используется текущий прогноз функции ценности.

Агенты улучшают политики

И последнее: большинство агентов занимаются оптимизацией политики. Этот этап зависит от типа обучаемого агента и того, что он пытается оценивать. Например, если агент прогнозирует функцию ценности, то в большинстве случаев он оптимизирует целевую политику, которая неявно закодирована в ней, то есть ту, которую он формирует. Преимущество оптимизации целевой политики в том, что она приводит к улучшению политики сбора данных, что, в свою очередь, повышает качество самих данных, которые агент будет собирать в будущем. Если целевая и поведенческая политики совпадают, оптимизация соответствующей функции ценности будет улучшать качество собираемой в дальнейшем информации.

Если же политика представлена явно, а не функцией ценности (как в случае с градиентом политик и методами «актер-критик»), агент может оптимизировать ее с помощью фактической выгоды. Еще он может прогнозировать выгоду улучшенной политики с помощью функций ценности. Наконец, в модельно-ориентированном RL оптимизировать политики можно по-разному. Например, с использованием сформированной модели среды, чтобы спланировать последовательность действий. В этом случае на этапе планирования совершенствуется неявная политика. Вместо этого можно использовать модель функции ценности, которая косвенно определяет политику. Или можно получить политику напрямую из модели. Основная идея здесь в том, что все агенты пытаются оптимизировать политику.



С акцентом на RL

Жадная, эпсилон-жадная и оптимальная политики

Жадная политика. Эта политика всегда выбирает действия, которые, как ожидается, должны приносить самую высокую выгоду в каждом состоянии. Важно понимать, что жадная она по отношению к функции ценности. Именно поэтому я использовал фразу «как ожидается». Суть в том, что, когда кто-то говорит «жадная политика», вы должны спросить: «Жадная относительно чего?» Например, жадная политика по отношению к случайной функции ценности — довольно плохая политика.

Эпсилон-жадная политика. Эта политика зачастую выбирает действия с самой высокой *ожидаемой* выгодой в каждом состоянии. Как и вышеописанная политика, жадная она по отношению к определенной функции ценности. Всегда обращайте внимание на то, о какой именно функции ценности идет речь.

Оптимальная политика. Эта политика всегда выбирает действия, фактически приносящие самую высокую ожидаемую выгоду в каждом состоянии. Если жадная политика *может* быть оптимальной, то оптимальная политика *всегда* будет жадной. Надеюсь, вы сразу же переспросили: «Жадная относительно чего?» Хорошая работа! Оптимальная политика жадная по отношению к уникальной, оптимальной функции ценности.

Обобщенная итерация политик

Еще один простой подход, который часто используют для описания архитектуры алгоритмов обучения с подкреплением, — *обобщенная итерация политик* (generalized policy iteration, GPI). Ее суть в том, что непрерывное взаимодействие оценки и оптимизации политик приближает их к оптимуму.

Как вы уже знаете, алгоритм итерации политик объединял в себе два этапа: оценку и улучшение. На первом этапе мы берем любую политику и оцениваем ее: прогнозируем ее функцию ценности. На этапе улучшения мы используем эту оценку (функцию ценности) для получения улучшенной политики. Когда взаимодействие между этими этапами больше не приносит никаких изменений, мы можем считать, что они стабилизировались и что политика и функция ценности оптимальны.

Вслед за итерацией политик мы рассмотрели еще один алгоритм — итерацию ценности. Как и предыдущий, он состоит из этапов оценки и улучшения политик. Но главное отличие в том, что этап оценки включает лишь одну итерацию. Иначе говоря, в результате оценки стратегии нельзя получить реальную функцию ценности. Наш прогноз к ней стремится, но не достигает цели. Но даже с этим сокращенным этапом оценки принцип обобщенной итерации политик, примененный к итерации ценности, тоже позволяет получить оптимальные функцию ценности и политику.

Главное здесь то, что оценка политик, как и алгоритмы из предыдущей главы, состоит из сбора и прогнозирования функций ценности. И, как вы уже знаете, политики можно оценивать по-разному, способов прогнозирования функции ценности для них тоже много. Точно так же есть и разные подходы к этапу оценки в обобщенной итерации политик.

Кроме того, улучшить политику — значит изменить ее так, чтобы сделать ее более жадной для функции ценности. В контексте итерации это значит, что ее нужно сделать совершенно жадной для функции ценности оцениваемой политики. Это можно сделать только при наличии MDP-среды. Но методам оценки политик, с которыми мы познакомились ранее, она не нужна, и за это приходится платить. Например, мы не можем сделать политику совершенно жадной, так как наши агенты должны исследовать среду. Поэтому мы делаем стратегию не абсолютно жадной, а просто «жаднее», оставляя пространство для исследования. Такое частичное улучшение применялось в главе 4, когда мы использовали разные стратегии исследования для работы с прогнозами.

На этом все. Большинство алгоритмов RL следуют этому шаблону GPI: при наличии отдельных этапов оценки и улучшения нам остается только выбирать подходящие методы.



Аналогия от Мигеля

Обобщенная итерация политик и почему важно прислушиваться к критике

Обобщенная итерация политик (generalized policy iteration, GPI) похожа на взаимоотношения критиков и артистов: оценка политики дает нужную для ее улучшения обратную связь точно так же, как полезные отзывы критиков помогают артистам совершенствовать свое мастерство.

Как однажды сказал Бенджамин Франклин, «критики — наши друзья, они указывают нам на наши ошибки». Он был умным человеком, который использовал GPI для самосовершенствования. Используйте полезные отзывы критиков, чтобы стать лучше. Это просто! По этому принципу работают многие крупнейшие мировые компании. Как вы думаете, что означает «решения на основе данных»? Это принцип, по которому для эффективного улучшения политик сначала применяется подходящий процесс их оценки — точно так же работает и GPI! Как говаривал Норман Винсент Пил, у большинства из нас одна проблема: мы лучше погибнем от восхищения, чем примем спасение от критики. Позвольте критикам вам помочь.

Но будьте осторожны! То, что критики могут помочь, вовсе не означает, что они всегда правы и что нужно слепо следовать их советам. То же самое относится и к оценке политик! Великий творец должен внимательно прислушиваться к реакции публики, собирая самые полезные отзывы и принимая их всерьез, только если он уверен в их правильности, ведь, в конце концов, мир принадлежит тем, кто творит, а не тем, кто критикует.

Лучше всего об этом выразился Теодор Рузвельт: «Важен не критик и не тот, кто объясняет, на чем споткнулся сильный или где ошибся действующий. Честь и слава принадлежат тому, чье лицо покрыто пылью, потом и кровью, тому, кто доблестно стремится к победе, кто ошибается и вновь начинает все с нуля, потому что не бывает усилий без заблуждений и ошибок. Кто действительно стремится действовать, кто испытал великий подъем духа и преданность своему делу, тот выбрал себе достойный путь. В лучшем случае он познает триумф великих свершений, а в худшем — вкус дерзновенного риска. В любом случае он никогда не окажется среди тех холодных и робких душ, которым неведомы ни победа, ни поражение».

Позже мы изучим метод «актер-критик», и вы поймете, к чему я все это написал (хотите вы этого или нет)! Артисты и критики помогают друг другу, но об этом позже.

То, что шаблоны принятия оптимальных решений используются повсеместно, действительно впечатляет. Знания, полученные при изучении DRL, помогут вам в жизни, что в итоге подтолкнет вас к созданию лучших агентов.

Круто, правда?

Оптимизация политик поведения

Ранее вы научились выполнять задачи прогнозирования: обучать агенты наиболее точно предсказывать функцию ценности заданной политики. Это полезный навык, но сам по себе он не помогает агенту выполнять конкретные задачи. В этом разделе вы узнаете чуть больше о задачах управления, что поможет обучить агенты улучшению политик. Эта новая возможность позволит им вырабатывать оптимальное поведение методом проб и ошибок, превращая любые произвольные политики в оптимальные. По прочтении этой главы вы сможете разрабатывать агенты, способные выполнять любые задачи MDP. Единственное ограничение в том, что у модели MDP должны быть дискретные пространства состояний и действий.

Чтобы показать агенты, мы применим уже известный вам принцип GPI: мы выберем алгоритмы для этапа оценки политик из изученных в предыдущей главе, а для этапа улучшения — из тех, что были представлены в главе 4. Благодаря взаимодействию этих двух процессов нам будет достаточно выбрать алгоритмы оценки и улучшения политик — все остальное агент сделает за нас.



С акцентом на RL

Задачи прогнозирования и управления, этапы оценки и улучшения политик

Задача прогнозирования. Оценка политик, прогнозирование функций ценности для заданной политики. Прогнозирование функций ценности — это обучение предсказыванию выгоды. Функции ценности состояний прогнозируют выгоду, ожидаемую от перехода в определенные состояния, а ценности действий — ожидаемую от пары «состояние — действие».

Задача управления. Поиск оптимальных политик. Задачи управления обычно решают по принципу GPI, когда конкурирующие процессы оценки и улучшения постепенно приближают политику к оптимальной. Методы RL зачастую сочетают в себе прогноз ценности действий, оптимизацию политик и стратегии выбора действий.

Оценка политик. Это алгоритмы выполнения задач прогнозирования. Отмечу, что есть метод динамического программирования под названием «оценка политик», но этот же термин применим и ко *всем алгоритмам*, выполняющим задачу прогнозирования.

Улучшение политик. Это алгоритмы улучшения исходных политик за счет повышения их жадности к функции ценности оригинала. Заметьте, что улучшение политики само по себе не решает задачу управления. Для этого такой процесс обычно сочетают с этапом оценки политик. Процесс улучшения — это просто произведение вычислений, оптимизирующих политику с учетом результатов ее оценки.



Конкретный пример

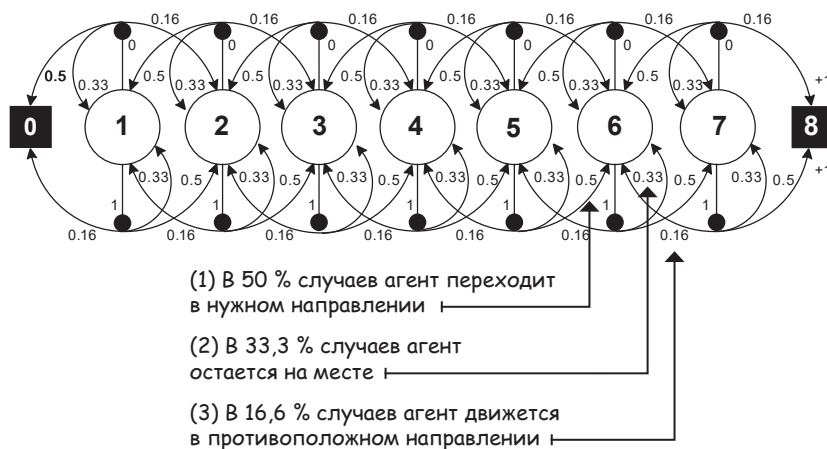
Среда «скользящая прогулка семь»

В этой главе мы будем использовать среду «скользящая прогулка семь» (СПС). Это однострочная грид-среда с семью нетерминальными состояниями. Ее отличительная черта — то, что она скользящая, — последствия действий стохастические. Если агент решает перейти влево, у него это может получиться, но есть вероятность того, что он переместится вправо или останется на месте.

Рассмотрим MDP для этой среды. Но помните, что у агента нет никакого доступа к информации о вероятностях переходов. Ему неизвестна динамика этой среды. Этот пример я использую только в ознакомительных целях.

Помните, что агент изначально не видит никаких связей между состояниями: он не знает, что состояние 3 находится посередине «прогулки» или что оно окружено состояниями 2 и 4, он даже не знает, что такое «прогулка»! Ему неизвестно, что действие 0 приводит к переходу влево, а 1 — вправо. Откройте блокнот Jupyter и самостоятельно поэкспериментируйте с этой средой, чтобы лучше понять, о чем речь. Суть в том, что агент видит только идентификаторы состояний (0, 1, 2 и т. д.) и выбирает одно из двух действий: 0 или 1.

MDP среды «скользящая прогулка семь»



Среда СПС похожа на «случайную прогулку» (СП) из предыдущей главы, но отличается наличием контроля. Как вы помните, СП — это среда, где вероятность перехода влево при выборе действия «влево» та же, что и у перехода вправо, и наоборот. Вы ничего не контролируете. Эта же среда, несмотря на существенные искажения, позволяет агенту выбирать действия, влияющие на производительность. К тому же у нее семь нетерминальных состояний, а не пять, как в СП.

Управление Монте-Карло: оптимизация политик после каждого эпизода

Попробуем создать алгоритм управления для оценки политик с помощью метода Монте-Карло. Допустим, для оптимизации политики используется тот же шаг, что мы применяли в алгоритме для ее итерации. То есть этап улучшения делает политику жадной для функции ценности оцениваемой стратегии. Можно ли так получить алгоритм, который помогает находить оптимальные политики только на основе взаимодействия? Нет. Чтобы этот подход работал, нам нужно внести два изменения.

Во-первых, мы должны позаботиться о том, чтобы наш агент прогнозировал функцию ценности действий $Q(s, a)$, а не состояний $V(s, a)$, прогнозируемую в предыдущей главе. Проблема V -функции в том, что в отсутствие MDP она не позволяет узнать, какое оптимальное действие доступно в определенном состоянии. Проще говоря, этап улучшения политик не будет работать.

Во-вторых, нам нужно убедиться, что наш агент исследует среду. Проблема в том, что мы больше не используем MDP для оценки политик. Формируя прогнозы на основе выборок, мы получаем ценность всех пар «состояние — действие», которые мы посетили. Но может быть так, что часть оптимальных состояний еще не посещалась.

Нам нужно прогнозировать функции ценности состояний

0	0.25	0.5	1	0
---	------	-----	---	---

(1) Здесь показаны действия «влево»/«вправо» и V -функция. Можете ли вы определить лучшую политику?

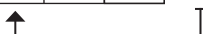


- (2) Что, если действие «влево» переносит нас вправо в 70 % случаев?
 (3) Какая же политика будет лучшей на этот раз?
 (4) Видите?! Одной лишь V -функции недостаточно

Нам нужно исследовать

0	←	←	←	←	+1
---	---	---	---	---	----

(1) Представьте, что это наша начальная детерминированная политика



- (2) Откуда вы знаете, какое действие лучше: «влево» или «вправо», если оцениваете только «влево»?
 (3) Видите?! Вашему агенту нужно исследовать среду

Поэтому будем использовать метод FVMC для этапа оценки и эpsilon-жадную стратегию выбора действий для этапа улучшения. И все — мы получаем полноценный алгоритм RL, которому не нужна модель и в котором мы оцениваем политики методом первого визита в Монте-Карло, оптимизируя их с помощью эpsilon-жадной стратегии выбора действий.

Точно так же, как в итерации ценности мы сокращали этап оценки политик, мы можем сократить метод прогнозирования Монте-Карло. Вместо того чтобы

прогнозировать функцию ценности для одной политики с помощью метода MC на протяжении нескольких эпизодов, как в предыдущей главе, мы можем завершить этап прогнозирования уже после первого шага и оценки траектории выборок, сразу перейдя к оптимизации политики. Мы будем чередовать одношаговое прогнозирование MC и одношаговый затухающий эпсилон-жадный выбор действий.

Рассмотрим наш первый метод управления на основе MC с помощью RL. У нас есть три функции:

- `decay_schedule` вычисляет затухающие значения в соответствии с переданными аргументами;
- `generate_trajectory` развертывает политику в среде для целого эпизода;
- `mc_control` — это полноценная реализация метода управления на основе MC.



Я знаю Python

Экспоненциальное затухание

```
def decay_schedule(
    init_value, min_value,
    decay_ratio, max_steps,
    log_start=-2, log_base=10):
```

(1) График затухания, который мы будем использовать для альфы и эпсилон, такой же, как мы применяли в предыдущей главе для альфы. Но в этот раз — подробнее

(2) Мне эта функция нравится тем, что мы передаем ей начальное, минимальное значения и долю шагов `max_steps`, на протяжении которых значения будут затухать от начального до минимального

`decay_steps = int(max_steps * decay_ratio)`

(3) Переменная `decay_steps` — это индекс, по достижении которого прекращается затухание значений, а `min_value` продолжается, пока не достигнет `max_steps`

(4) `rem_steps` — это разность

`rem_steps = max_steps - decay_steps`

(5) Я вычисляю значения с помощью метода `logspace`, начиная с переменной `log_start`, которая по умолчанию равна `-2`, и заканчивая нулем. Количество нужных мне значений в этом пространстве равно `decay_steps`, а `base` — переменной `log_base`, значение по умолчанию которой — `10`. Заметьте, я поменял порядок следования этих значений на обратный!

`values = np.logspace(
 log_start, 0, decay_steps,
 base=log_base, endpoint=True)[:::-1]`

(6) Поскольку значения могут не заканчиваться точно на `0`, учитывая, что это логарифм, я использую диапазон от `0` до `1`, чтобы полученная кривая была гладкой и аккуратной

`values = (values - values.min()) / \
 (values.max() - values.min())`

(7) Теперь мы можем выполнить линейное преобразование и получить точки между `init_value` и `min_value`

`values = (init_value - min_value) * values + min_value`

(8) Эта функция `pad` повторяет крайнее правое значение `rem_step` раз

`values = np.pad(values, (0, rem_steps), 'edge')`

`return values`



Я знаю Python

Создание траекторий исследовательской политики

```
def generate_trajectory(
    select_action, Q, epsilon,
    env, max_steps=200):
    (2) Сначала инициализируем флаг done
    и список кортежей опыта trajectory
    done, trajectory = False, []

    while not done:
        (3) Затем начинаем циклический перебор,
        пока флаг done не будет в значении true

        state = env.reset()
        (4) Мы сбрасываем среду
        для взаимодействия в новом эпизоде

        for t in count():
            (5) Затем начинаем отсчитывать шаги t

            (6) После используем переданную функцию select_action
            для выбора действия
            action = select_action(state, Q, epsilon)

            (7) С помощью этого действия мы переходим в следующее
            состояние среды и получаем полный кортеж опыта
            next_state, reward, done, _ = env.step(action)
            experience = (state,
                        action,
                        reward,
                        next_state,
                        done)
            (8) Добавляем кортеж
            в список trajectory
            trajectory.append(experience)

            if done:
                (9) Если мы попали в конечное состояние
                и флаг done теперь в значении true, мы
                выходим из цикла и возвращаем результат
                break

            (10) Если же счетчик шагов t
            в текущей траектории достигнет
            максимально допустимого
            значения, мы очищаем список
            trajectory, выходим из цикла
            и пытаемся получить другую
            траекторию
            if t >= max_steps - 1:
                trajectory = []
                break

            state = next_state
            (11) Не забываем обновить состояние

    return np.array(trajectory, np.object)
    (12) В конце возвращаем
    траекторию в формате NumPy
    для упрощения обработки данных
```



Я знаю Python

Метод управления Монте-Карло

```
def mc_control(env,
               gamma=1.0,
               init_alpha=0.5,
               min_alpha=0.01,
               alpha_decay_ratio=0.5,
               init_epsilon=1.0,
               min_epsilon=0.1,
               epsilon_decay_ratio=0.9,
               n_episodes=3000,
               max_steps=200,
               first_visit=True):
```

(1) Функция `mc_control` похожа на `mc_prediction`. Но у них есть два основных различия: на этот раз мы прогнозируем функцию ценности действий Q и нам нужно исследовать среду

(2) Обратите внимание, что в определении функции используются значения эпсилон для выбора графика затухания в условиях случайного исследования

```
    nS, nA = env.observation_space.n, env.action_space.n

    discounts = np.logspace(
        0, max_steps,
        num=max_steps, base=gamma,
        endpoint=False)
```

(3) Мы заранее вычисляем значения коэффициентов дисконтирования. Важно: мы используем `max_steps`, так как это максимальная длина траектории

```
    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio,
        n_episodes)
```

(4) С помощью переданных значений мы заранее вычисляем альфу

```
    epsilons = decay_schedule(
        init_epsilon, min_epsilon,
        epsilon_decay_ratio,
        n_episodes)
```

(5) Наконец, повторяем то же самое для эпсилон и получаем массив, который будет использоваться на протяжении всего сеанса обучения

```
    pi_track = []
    Q = np.zeros((nS, nA), dtype=np.float64)
    Q_track = np.zeros((n_episodes, nS, nA), dtype=np.float64)
```

(6) Здесь мы просто инициализируем переменные, включая Q -функцию

(7) Это эпсилон-жадная стратегия, но эпсилон затухает по окончании каждого эпизода, а не шага

```
    select_action = lambda state, Q, epsilon: \
        np.argmax(Q[state]) \
        if np.random.random() > epsilon \
        else np.random.randint(len(Q[state]))
```

```

for e in tqdm(range(n_episodes), leave=False):
    (8) Здесь мы входим в цикл обработки эпизодов в n_episodes
    итераций. Напомню, что tqdm просто выводит подходящий
    индикатор выполнения — ничего особенного

    trajectory = generate_trajectory(select_action,
    (9) В каждом новом эпизоде e мы генерируем Q,
    новую траекторию с политикой исследования, epsilons[e],
    которая определяется функцией select_action. env,
    Ограничиваем длину траектории до max_steps → max_steps)

    (10) Теперь мы отслеживаем посещения пар «состояние —
    действие» — это еще одно важное отличие от метода mc_prediction

    visited = np.zeros((nS, nA), dtype=np.bool)
    for t, (state, action, reward, _, _) in enumerate(\
        trajectory):
        (11) Обратите внимание, что траектории здесь обрабатываются
        после прекращения взаимодействия со средой

        if visited[state][action] and first_visit:
            continue ← (12) Здесь мы проверяем
            посещения пар
            «состояние — действие»
            и в зависимости от этого
            решаем, что делать
            дальше
            visited[state][action] = True

        (13) Переходим к вычислению выгоды,
        как делали это с методом прогнозирования,
        но на этот раз мы используем Q-функцию
        n_steps = len(trajectory[t:])
        G = np.sum(discounts[:n_steps] * trajectory[t:, 2])
        Q[state][action] = Q[state][action] + \
        alphas[e] * (G - Q[state][action])
        (14) Обратите внимание на то, как здесь используются альфы

    (15) Теперь остается только сохранить значения для анализа
    Q_track[e] = Q
    pi_track.append(np.argmax(Q, axis=1))
    V = np.max(Q, axis=1)
    pi = lambda s: {s:a for s, a in enumerate(\
        np.argmax(Q, axis=1))}[s]
    (16) В конце мы извлекаем функцию
    ценности состояний и жадную политику
    return Q, V, pi, Q_track, pi_track

```

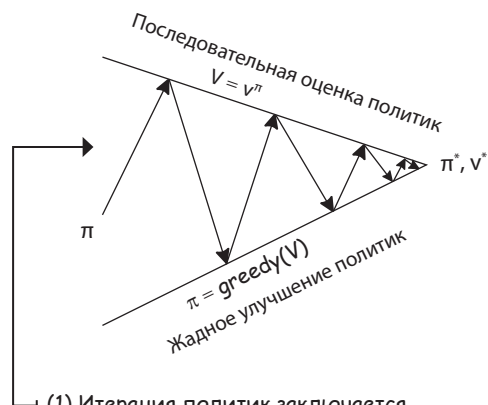
SARSA: улучшение политик после каждого шага

Как уже обсуждалось ранее, один из недостатков методов Монте-Карло в том, что они работают только в промежутках между эпизодами. Это значит, что, прежде чем оптимизировать наши прогнозы функции ценности, мы должны

дождаться достижения конечного состояния. Но при оценке политики метод Монте-Карло довольно легко заменить временным разностным прогнозированием. Используя TD вместо MC, мы получаем другой алгоритм — *агент SARSA*.

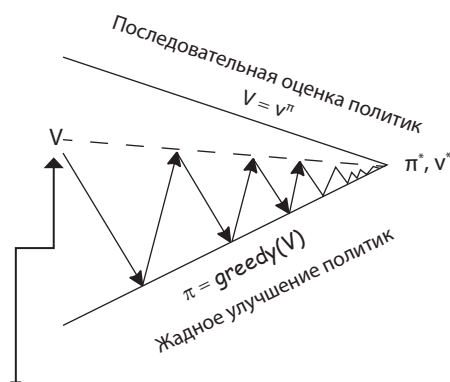
Сравнение методов планирования и управления

Итерация политик



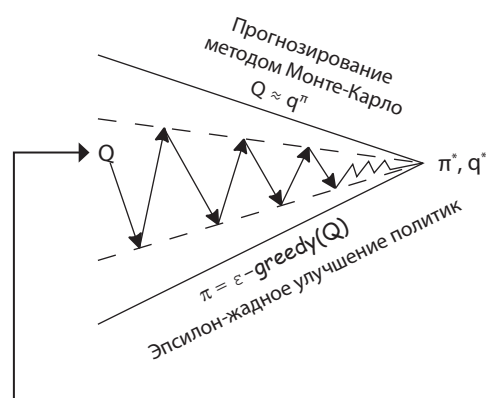
(1) Итерация политик заключается в полном сходимости чередующихся процессов последовательной оценки политики и ее жадного улучшения

Итерация ценностей



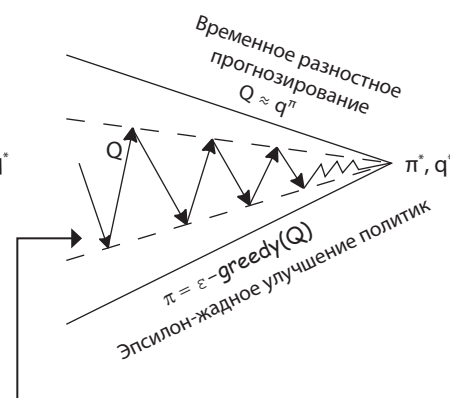
(2) Итерация ценности начинается с произвольной функции ценности, и у нее сокращенный этап оценки политик

Метод Монте-Карло



(3) Метод управления на основе MC оценивает Q-функцию. Вслед за сокращенным этапом прогнозирования идет эpsilon-жадное улучшение политик

SARSA



(4) SARSA очень похож на метод управления на основе MC, только для оценки политики используется сокращенное прогнозирование методом TD



Я знаю Python

Агент SARSA

```
def sarsa(env,
          gamma=1.0,
          init_alpha=0.5,
          min_alpha=0.01,
          alpha_decay_ratio=0.5,
          init_epsilon=1.0,
          min_epsilon=0.1,
          epsilon_decay_ratio=0.9,
          n_episodes=3000):
```

(1) Агент SARSA — это прямая модификация TD для выполнения задач управления: по сути, это TD с двумя существенными изменениями. Во-первых, этот алгоритм прогнозирует функцию ценности действий Q, а во-вторых, предусматривает исследовательский этап улучшения политики

(2) Мы используем эпсилон точно так же, как в `mc_control`

(3) Вначале мы создаем несколько полезных переменных. Помните: `pi_track` сохранит жадную политику в каждом эпизоде

```
    nS, nA = env.observation_space.n, env.action_space.n
    pi_track = []
```

(4) Затем мы создаем Q-функцию. Я использую тип `np.float64` для максимальной точности, но, наверное, это лишнее

```
    Q = np.zeros((nS, nA), dtype=np.float64)
    Q_track = np.zeros((n_episodes, nS, nA), dtype=np.float64)
```

(5) `Q_track` сохранит спрогнозированную Q-функцию в каждом эпизоде

(6) `select_action` — это, как и раньше, эпсилон-жадная стратегия

```
    select_action = lambda state, Q, epsilon: \
        np.argmax(Q[state]) \
        if np.random.random() > epsilon \
        else np.random.randint(len(Q[state]))
```

(7) В SARSA не нужно заранее рассчитывать все коэффициенты дисконтирования, потому что мы не будем использовать полную выгоду. Нас интересует только ожидаемая выгода, поэтому дисконтирование можно вычислять в режиме реального времени

```
    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio,
        n_episodes)
```

(8) Заметьте, что все альфы вычисляются заранее. Вызов этой функции возвращает вектор с соответствующими значениями альфы

(9) Функция `select_action` сама по себе не затухающая стратегия. Мы заранее вычисляем затухающие значения эпсилона, поэтому наш агент будет использовать затухающую эпсилон-жадную стратегию

```
    epsilons = decay_schedule(
        init_epsilon, min_epsilon,
        epsilon_decay_ratio,
        n_episodes)
```

```

for e in tqdm(range(n_episodes), leave=False):
    (10) Теперь мы внутри цикла обработки эпизодов
        (11) Каждый эпизод начинается со сброса среды и флага done
        state, done = env.reset(), False
        action = select_action(state, Q, epsilons[e])
    (12) Выбираем действие (возможно, для исследования)
        для начального состояния
        while not done:
            (13) Повторяем, пока
                    не дойдем до терминального
                (14) Первым делом войдем в среду
                        и получим кортеж опыта
                next_state, reward, done, _ = env.step(action)
                next_action = select_action(next_state,
                                            Q,
                                            epsilons[e])
            (15) Заметьте, перед выполнением любых
                    вычислений нам нужно получить действие
                    для следующего шага
                td_target = reward + gamma * \
                            Q[next_state][next_action] * (not done)
            (16) Вычисляем td_target, используя следующую пару
                    «состояние — действие». В конечных состояниях мы
                    используем небольшой трюк: умножаем результат
                    на выражение (not done), обнуляя так будущие состояния
            (17) Теперь вычисляем td_error как разность цели
                    и фактического прогноза
                td_error = td_target - Q[state][action]
            (18) Наконец, обновляем Q-функцию, немного
                    приближая прогнозы к погрешности
                Q[state][action] = Q[state][action] + \
                                    alphas[e] * td_error
            (19) Обновляем состояние и действие для следующего шага
                state, action = next_state, next_action
                (20) Сохраняем Q-функцию и жадную
                        политику для анализа
                Q_track[e] = Q
                pi_track.append(np.argmax(Q, axis=1))
                V = np.max(Q, axis=1)
                pi = lambda s: {s:a for s, a in enumerate(\
                                    np.argmax(Q, axis=1))}[s]
            (21) В конце вычисляем и возвращаем ожидаемую
                    оптимальную V-функцию и ее жадную политику
        return Q, V, pi, Q_track, pi_track

```



С акцентом на RL

Задачи и методы пакетного, офлайн- и онлайн-обучения

Задачи и методы пакетного обучения. Обычно пакетное обучение — это либо тип задач обучения, где образцы опыта статические и доступны заранее, либо тип методов, оптимизированных для синхронного использования целых наборов кортежей опыта. Методы пакетного обучения обычно рассматриваются в контексте неинтерактивных задач — в частности, пакетных. Но их также можно применять и к интерактивным. Например, методы растущего пакетного обучения могут собирать информацию, «накапливая» пакетные данные. Кроме того, задачи пакетного обучения не обязательно выполнять с помощью пакетных методов — точно так же, как методы пакетного обучения позволяют выполнять не только пакетные задачи.

Задачи и методы офлайн-обучения. Как правило, офлайн-обучение — это либо постановка задачи, где предусмотрена симуляция сбора данных (в противовес настоящей онлайн-среде), либо методы обучения *постфактум* — например, между эпизодами. Отмечу, что в офлайн-методах обучение и взаимодействие тоже могут чередоваться, но производительность улучшается только после получения выборки. Это похоже на растущие пакетные методы, описанные выше, с той лишь разницей, что старые выборки обычно отбрасываются: то есть данные не *накапливаются*. Например, методы МС нередко причисляют к офлайн-методам, так как они чередуют обучение и взаимодействие на уровне отдельных эпизодов. Обучение и взаимодействие — два отдельных этапа. МС можно считать как интерактивным, так и офлайн-методом.

Задачи и методы онлайн-обучения. Онлайн-обучение — это либо одновременное обучение и взаимодействие с динамической системой, как у роботов, либо методы обучения на новейшем опыте, собираемом после каждого временного шага.

Заметьте, что офлайн- и онлайн-обучение зачастую используются в разных контекстах. Я видел, как эти термины применялись для обозначения неинтерактивных и интерактивных методов, но, как я уже писал, с их помощью могут разделять обучение в симуляторе и в реальной динамичной системе.

Здесь используются определения, принятые среди таких исследователей в области RL, как Ричард Саттон (Richard Sutton) (книга 2018 года), Дэвид Сильвер (David Silver) (лекции 2015 года), Хадзо ван Хасселт (Hado van Hasselt) (лекции 2018 года), Майкл Литтман (Michael Littman) (научная работа 2015 года) и Чабба Сепесвари (Csaba Szepesvari) (книга 2009 года).

Просто имейте в виду, что у этих терминов несколько трактовок.

Разделение поведения и обучения

Вернемся к уравнению обновления TD для функций ценности состояний. Помните, что в качестве цели в нем используется $R_{t+1} + \gamma V(S_{t+1})$. Но если посмотреть к уравнению обновления TD для функций ценности действий,

$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$, можно заметить несколько дополнительных возможностей. Взгляните на используемое действие и подумайте, что из этого следует. Чем его можно заменить? Одним из важнейших достижений в области обучения с подкреплением была разработка алгоритма *Q-обучения* — метода бэггинга, который не использует модель и не зависит от действий агента, а напрямую аппроксимирует оптимальную политику без учета опыта, на котором она основана. Да, это значит, что агент теоретически может вести себя непредсказуемо и при этом найти оптимальные функцию ценности и политики. Как это возможно?

Q-обучение: действуем оптимально, даже если не хотим

Алгоритм SARSA — это что-то вроде «разведки боем». Агент исследует ту же политику, которую он использует для получения опыта. Такой тип обучения называют *стратегическим*, или *онлайн-обучением* (on-policy). Придерживаясь его, мы учимся на своих ошибках. Но хочу отметить, что при этом используются только текущие ошибки. Что, если мы хотим учиться на более ранних ошибках? Или на чужих? Стратегические методы этого не позволяют. А вот *нестратегическое*, или *офлайн-обучение* (off-policy), позволяет как бы «учиться на чужих ошибках». Агент исследует политику, не связанную с полученным им опытом. Здесь применяются две политики: поведенческая (для генерации опыта и взаимодействия со средой) и целевая, которую мы формируем. SARSA — стратегический метод, а Q-обучение — нестратегический.



Я знаю математику

Уравнения обновлений в SARSA и Q-обучении

(1) Единственное отличие SARSA от Q-обучения в том, какое действие используется при вычислении цели

(2) Это уравнение обновления SARSA

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t \left[\underbrace{R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})}_{\text{Цель SARSA}} - Q(S_t, A_t) \right]$$

Погрешность SARSA

(3) Для вычисления цели оно использует действие, предпринятое в следующем состоянии

(4) А это уравнение Q-обучения

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t \left[\underbrace{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}_{\text{Цель Q-обучения}} - Q(S_t, A_t) \right]$$

Погрешность Q-обучения

(5) Оно использует действие с самой высокой ожидаемой ценностью в следующем состоянии, независимо от того, какое на самом деле было выбрано



Я знаю Python

Агент Q-обучения

```
def q_learning(env,
               gamma=1.0,
               init_alpha=0.5,
               min_alpha=0.01,
               alpha_decay_ratio=0.5,
               init_epsilon=1.0,
               min_epsilon=0.1,
               epsilon_decay_ratio=0.9,
               n_episodes=3000):
```

(1) Заметьте, что у агентов SARSA и Q-обучения одинаковый начальный код

(2) Я даже использую одинаковые гиперпараметры в обоих алгоритмах

(3) Это несколько полезных переменных

```
    nS, nA = env.observation_space.n, env.action_space.n
    pi_track = []
```

(4) Q-функция и переменная для последующего анализа

```
    Q = np.zeros((nS, nA), dtype=np.float64)
    Q_track = np.zeros((n_episodes, nS, nA), dtype=np.float64)
```

(5) Та же эpsilon-жадная стратегия выбора действий

```
    select_action = lambda state, Q, epsilon: \
        np.argmax(Q[state]) \
        if np.random.random() > epsilon \
        else np.random.randint(len(Q[state]))
```

(6) Вектор со всеми альфами, который будет использоваться при обучении

```
    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio,
        n_episodes)
```

(7) Вектор со всеми значениями эpsilon для нужного нам затухания

```
    epsilons = decay_schedule(
        init_epsilon, min_epsilon,
        epsilon_decay_ratio,
        n_episodes)
```

(8) Перебираем эпизоды

```
    for e in tqdm(range(n_episodes), leave=False):
```

```

└───> state, done = env.reset(), False ◀───┐
(9) Сбрасываем среду, получаем исходное состояние и присваиваем
флагу done значение false
(10) Входим в цикл взаимодействия для онлайн-обучения (шаги)
└───> while not done: ◀───┐ (11) Продолжаем цикл, пока
не достигнем конечного состояния
и пока не будет установлен флаг done
(12) Сначала выбираем действие
для текущего состояния. Обратите внимание
на использование значений эпсилонa
└───> action = select_action(state, Q, epsilons[e])
└───> next_state, reward, done, _ = env.step(action)
(13) Входим в среду и получаем полный кортеж опыта (s, a, s', r, d)

(14) Вычисляем цель TD. Q-обучение — это особый
алгоритм. Он пытается сформировать оптимальную
функцию ценности действий q* даже при использовании
затухающей эпсилон-жадной политики исследования,
как здесь. Это называется нестратегическим обучением
└───┐
td_target = reward + gamma * \ ◀───┐
└───┐ Q[next_state].max() * (not done)
(15) И снова благодаря not done максимальная оценка следующего состояния
обнуляется при попадании в конечное. Важно, чтобы после «смерти» агент
не ожидал никакой награды!!!
(16) Далее мы вычисляем погрешность TD как разность прогнозируемого
значения и цели
└───┐
└───> td_error = td_target - Q[state][action]
└───> Q[state][action] = Q[state][action] + \
alphas[e] * td_error
(17) После этого мы немного приближаем
Q-функцию для пары «состояние — действие»
к погрешности
state = next_state ◀───┐ (18) Обновляем состояние
Q_track[e] = Q ◀───┐ (19) Сохраняем Q-функцию и политику
pi_track.append(np.argmax(Q, axis=1))
(20) На выходе получаем V-функцию
и итоговую политику
V = np.max(Q, axis=1) ◀───┐
pi = lambda s: {s:a for s, a in enumerate(\
np.argmax(Q, axis=1))}[s]
return Q, V, pi, Q_track, pi_track

```



Аналогия от Мигеля

Люди тоже учатся стратегически и нестратегически

Стратегическое обучение — это изучение политики, которая используется для принятия решений. Считайте, что это «разведка боем». Нестратегическое обучение — это исследование политики, которая не используется для принятия решений. Иначе говоря, «обучение на чужом опыте»: мы учимся делать что-то, но сами не применяем эти навыки. Оба подхода важны для принятия оптимальных решений. Любопытно, что мы довольно быстро можем определить, какой из этих методов предпочитает тот или иной человек.

Мой сын, например, предпочитает стратегическое обучение. Когда он не может понять, что делать с игрушкой, я подхожу и пытаюсь ему помочь, но он начинает капризничать, пока я не оставлю его в покое. Он пробует снова и снова, и в конце концов у него все получается. Мой сын любит учиться на своем опыте, а не на чужом. Стратегическое обучение — это простой и стабильный способ обучения.

А вот моя дочь, напротив, придерживается нестратегического типа. Прежде чем за что-то браться, она может понаблюдать за мной. Я показываю ей, как рисовать дом, и затем она пробует сама.

Но имейте в виду, что эта аналогия слегка притянута за уши. Имитационное и нестратегическое обучение — это не одно и то же. Нестратегическое обучение — это использование опыта из одной области в другой. Например, навыки бега можно применять для игры в футбол. Проще говоря, вы делаете одно, а изучаете что-то другое. Уверен, вы можете вспомнить несколько примеров такого подхода: скажем, вы могли читать книгу о живописи и готовить себе ужин. В нестратегическом обучении важен источник опыта. Если целевая политика отличается от поведенческой, то обучение можно считать нестратегическим.

Не торопитесь определять, какой из этих методов лучше. Помните, что в контексте RL у обоих есть свои плюсы и минусы. С одной стороны, стратегическое обучение стабильное и интуитивно понятное. Если вам нужно научиться хорошо играть на фортепиано, почему бы не заниматься именно на нем?

С другой стороны, использование альтернативных источников, помимо своего опыта, может быть полезным. В конце концов, у нас есть и другие дела. Возможно, медитация научит вас чему-то, что пригодится в игре на фортепиано и улучшит навыки. Методы нестратегического обучения позволяют черпать знания из разных источников (и/или навыков), но они зачастую обладают высокой дисперсией и поэтому медленней сходятся.

Помните также, что нестратегическое обучение — один из трех элементов, которые в совокупности гарантированно приводят к расхождению, два других — бэггинг и аппроксимация функций. Они плохо сочетаются друг с другом. С первыми двумя мы уже познакомились, а о третьем вы узнаете чуть позже.



С акцентом на RL

Жадность в пределе с бесконечным исследованием и теория стохастической аппроксимации

Жадность в пределе с бесконечным исследованием (greedy in the limit with infinite exploration, GLIE) — это набор требований, которым должен соответствовать алгоритм стратегического RL, такой как Монте-Карло или SARSA, чтобы гарантировать сходимость в оптимальной политике. Требования следующие:

- все пары «состояние — действие» должны исследоваться бесконечно часто;
- при схождении политика должна становиться жадной.

Это значит, что эпсилон-жадная стратегия исследования, к примеру, должна медленно приближать эпсилон к нулю. Если это делать слишком быстро, первое условие может быть не выполнено, а слишком медленное затухание будет затягивать схождение.

Отмечу, что к алгоритмам нестратегического RL, таким как Q-обучение, применимо только первое из этих двух требований, поскольку в нестратегическом обучении для выбора действий используется не та политика, которую мы исследуем. Например, Q-обучение требует только, чтобы все пары «состояние — действие» обновлялись в достаточной степени, что обеспечивается первым условием.

Можете ли вы быть уверены в том, что простая стратегия исследования (например, эпсилон-жадная) удовлетворяет этому требованию, — это уже другой вопрос. В простых грид-средах и дискретных пространствах действий и состояний эпсилон-жадный подход, скорее всего, будет работать. А что насчет сложных сред, где случайного поведения было бы недостаточно?

Есть еще один набор требований к общей сходимости, основанных на теории стохастической аппроксимации и применимых ко всем этим методам. Поскольку мы обучаемся на выборках, которым свойственна определенная дисперсия, прогнозы будут сходиться, только если приблизить темп обучения, альфа, к нулю:

- сумма темпов обучения должна быть бесконечной;
- сумма темпов обучения, возведенных в квадрат, должна быть конечной.

Это значит, что ваши темпы обучения должны затухать, не достигая нуля. Например, при использовании $1/t$ или $1/e$ скорость обучения изначально умеренно велика, чтобы гарантировать, что алгоритм не следует строго только за одной выборкой, но постепенно становится достаточно низкой, чтобы вы были уверены в том, что он находит сигнал за шумом.

Об этих свойствах сходимости полезно знать при работе над теорией алгоритмов RL, но на практике, в зависимости от решаемой задачи, скорость обучения обычно устанавливается достаточно малой константой. Кроме того, важно понимать, что маленькая константа лучше подходит для нестационарных сред, широко распространенных в реальном мире.



С акцентом на RL

Стратегическое и нестратегическое обучение

Стратегическое обучение. Относится к методам, которые пытаются оценить или улучшить политику, используемую для принятия решений. Он довольно прост. У вас есть одна политика, генерирующая поведение. Ваш агент оценивает это поведение и с учетом этого выбирает аспекты для оптимизации. Агент учится оценивать и улучшать ту же политику, с помощью которой он генерирует данные.

Нестратегическое обучение. Относится к методам, которые пытаются оценить или улучшить политику, отличную от той, которая используется для генерации данных. Этот подход посложнее. У вас есть две политики, одна из которых собирает данные, накапливает опыт, вырабатывает поведение. Ваш агент использует все это для оценки, улучшения и исследования другой политики: не той, которую он использует для генерации данных.

Двойное Q-обучение: максимальная ожидаемая ценность для прогнозирования максимума

Q-обучение нередко дает слишком оптимистичный прогноз функции ценности. Вы только подумайте: на каждом шаге мы берем максимальную функцию ценности действий, ожидаемую в следующем состоянии, но в действительности нам нужно реальное значение максимальной функции ценности действий для следующего состояния. Иначе говоря, для оценки максимума мы выбираем максимальные ожидаемые значения.

Этот неточный способ прогнозирования создает существенную проблему: данные оценки бэггинга, с помощью которых формируются цели TD, часто смещены. Использование максимальной среди всех смещенных ценностей в качестве прогноза максимума влечет за собой *смещение максимизации*.

Все просто. Представьте себе функцию ценности действий с нулевой фактической ценностью и смещенными прогнозами у каждой: одни положительные, другие отрицательные. Например, 0,11; 0,65; -0,44; -0,26 и т. д. Мы знаем, что максимальная фактическая ценность равна нулю, но максимальная ожидаемая — 0,65. Если бы мы поочередно выбирали ценность то с положительным, то с отрицательным смещением, проблема не стояла бы так остро. Но мы всегда выбираем максимум, поэтому ожидаемая ценность получается высокой, несмотря на самое большое смещение (наибольшую погрешность). Если делать это снова и снова, погрешность будет только накапливаться.

Все мы встречали чрезмерно позитивных людей, которые забывают о том, что не все то золото, что блестит. На мой взгляд, это одна из причин, по которой

многие люди советуют не подпитывать шумиху вокруг ИИ: преувеличение часто вредит нам самим и, чтобы показывать хорошие результаты, его следует избегать.



Я знаю Python

Агент двойного Q-обучения

```
def double_q_learning(env,
                      gamma=1.0,
                      init_alpha=0.5,
                      min_alpha=0.01,
                      alpha_decay_ratio=0.5,
                      init_epsilon=1.0,
                      min_epsilon=0.1,
                      epsilon_decay_ratio=0.9,
                      n_episodes=3000):
```

(1) Как и ожидалось, двойное Q-обучение принимает точно такие же аргументы, как и обычное

(2) Начинаем с уже знакомых нам переменных

```
    nS, nA = env.observation_space.n, env.action_space.n
    pi_track = []
```

(3) Но кое-что сразу же бросается в глаза: мы используем две функции ценности, Q1 и Q2. Это нечто вроде перекрестной проверки: прогноз одной Q-функции поможет нам проверить прогноз другой. Но проблема в том, что опыт теперь разделяется между двумя разными функциями, что замедляет обучение

```
    Q1 = np.zeros((nS, nA), dtype=np.float64)
    Q2 = np.zeros((nS, nA), dtype=np.float64)
    Q_track1 = np.zeros((n_episodes, nS, nA), dtype=np.float64)
    Q_track2 = np.zeros((n_episodes, nS, nA), dtype=np.float64)
```

(4) Остальной код на этой странице довольно прост, и вы уже должны в нем уверенно ориентироваться. Параметры `select_action`, `альфа` и значения `эпсилона` вычисляются так же, как и прежде

```
    select_action = lambda state, Q, epsilon: \
        np.argmax(Q[state]) \
        if np.random.random() > epsilon \
        else np.random.randint(len(Q[state]))

    alphas = decay_schedule(init_alpha,
                           min_alpha,
                           alpha_decay_ratio,
                           n_episodes)

    epsilons = decay_schedule(init_epsilon,
                             min_epsilon,
                             epsilon_decay_ratio,
                             n_episodes)
```

(5) Возвращаемся в цикл
обработки эпизодов

```
└─ for e in tqdm(range(n_episodes), leave=False):
```

(6) Каждый новый эпизод начинается
со сброса среды и получения
начального состояния

```
└─ state, done = env.reset(), False
   while not done: ───────────────────┘
```

(7) Все повторяется, пока мы не дойдем до терминального состояния
(при этом флаг done должен принимать значение true)

(8) На каждом шаге выбираем действие
с помощью нашей функции select_action

```
└─ action = select_action(state,
                           (Q1 + Q2)/2.,
                           epsilons[e])
```

(9) Но здесь есть кое-что любопытное: мы используем среднее значение
из двух Q-функций!!! Здесь также подошла бы их сумма — результаты
были бы такие же

```
└─ next_state, reward, done, _ = env.step(action)
```

(10) Затем мы передаем действие среде и получаем кортеж опыта

(11) Теперь все начинает меняться: мы бросаем монету, чтобы
определить обновление для Q1 или Q2

```
└─ if np.random.randint(2):
    argmax_Q1 = np.argmax(Q1[next_state])
```

(12) Выбираем лучшее действие по мнению Q1...

(13) ...но саму ценность для вычисления цели TD берем из Q2

```
└─ td_target = reward + gamma * \
    Q2[next_state][argmax_Q1] * (not done)
```

(14) Заметьте, ценность предписана функцией Q1, но взята из Q2

(15) Теперь вычисляем погрешность TD на основе прогноза Q1

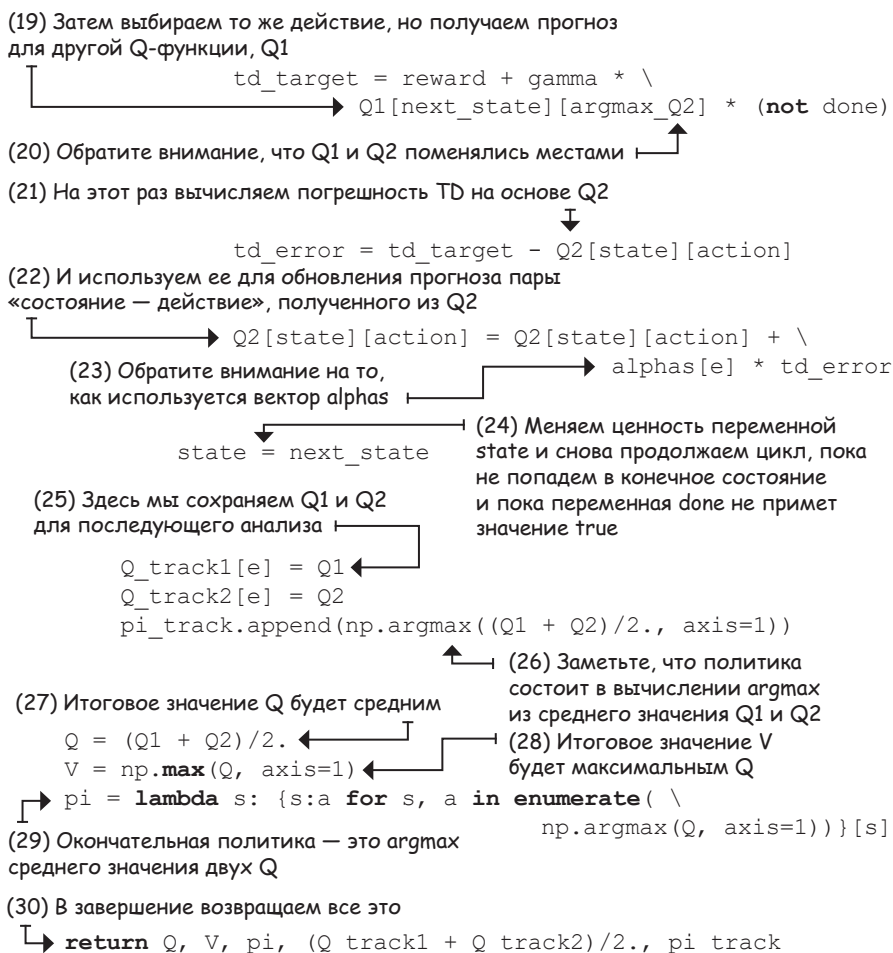
```
└─ td_error = td_target - Q1[state][action]
```

(16) С помощью полученной погрешности приближаем наш прогноз к этой цели

```
└─ Q1[state][action] = Q1[state][action] + \
```

(17) Теперь, если случайное число равно 0 (в 50 %
случаев), мы обновляем другую Q-функцию, Q2

```
└─ else:
    (18) Но это, по сути, зеркальное отражение другого
    обновления. Мы получаем argmax для Q2
    argmax_Q2 = np.argmax(Q2[next_state])
```

Один из способов борьбы со смещением максимизации — отслеживание прогнозов двух Q-функций. На каждом временном шаге мы выбираем действие с помощью одной из них и определяем самое высокое ожидаемое значение согласно этой Q-функции. Но, чтобы получить прогноз действия, мы используем другую Q-функцию. Так мы снижаем вероятность того, что у нас всегда будет положительное смещение.

Чтобы выбрать действие в заданной среде, мы используем среднее значение или сумму двух Q-функций в текущем состоянии. Например, максимум по $Q_1(S_{t+1}) + Q_2(S_{t+1})$. Такой подход называется *двойным обучением*, а реализующий

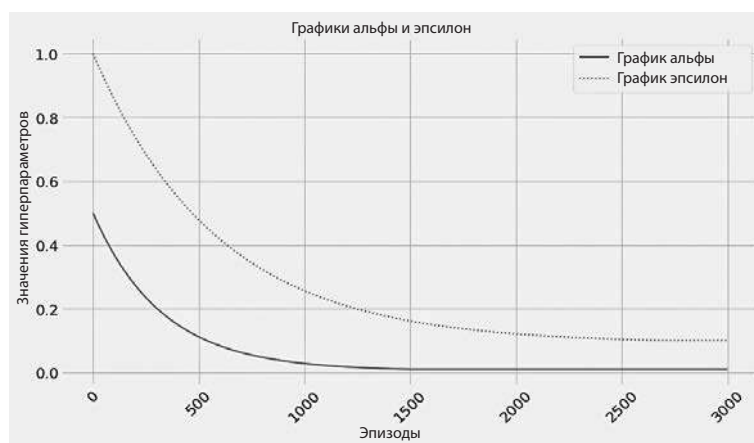
его алгоритм известен как *двойное Q-обучение*. Чуть позже вы познакомитесь с алгоритмом глубокого обучения с подкреплением под названием «*двойные глубокие Q-сети*» (double deep Q-networks, DDQN), где применяется одна из разновидностей метода двойного обучения.



В деталях

FVMC, SARSA, Q-обучение и двойное Q-обучение в среде СПС

Сравним все алгоритмы, с которыми мы только что познакомились, в среде «скользящая прогулка семь».



Как видите, я использовал во всех алгоритмах одинаковые гиперпараметры, значения гаммы, альфы и эпсилон и соответствующие графики затухания. Помните: если альфа не стремится к нулю, алгоритм не может полностью сойтись. У меня альфа затухает до 0,01 — этого достаточно для такой простой среды. Для полной сходимости значение эпсилон тоже должно стремиться к нулю, но обычно так делают редко. На самом деле во многих продвинутых реализациях эпсилон вообще не затухает и обладает постоянной ценностью. Здесь он затухает до 0,1.

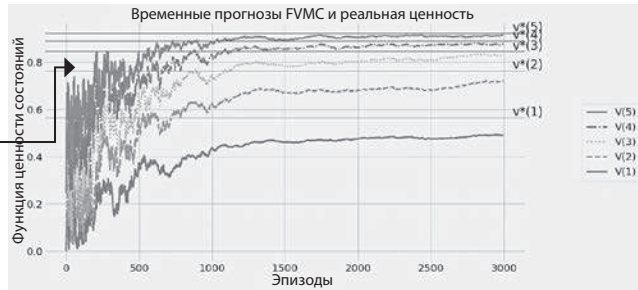
И еще кое-что: в этих прогонах для всех алгоритмов задано одно число эпизодов — 3000. Вы увидите, что некоторые алгоритмы не сходятся за это количество шагов, но это не означает, что они не сходятся в принципе. Кроме того, некоторые среды в блокноте Jupyter для этой главы, такие как «замёрзшее озеро», завершают работу по прохождении определенного количества шагов: то есть, если ваш агент не успевает завершить каждый эпизод за 100 шагов, ему присваивается флаг done. Это создает некоторые проблемы, которые мы обсудим в следующих главах. Но, пожалуйста, не забывайте заглядывать в блокнот! Я думаю, вам понравится экспериментировать.



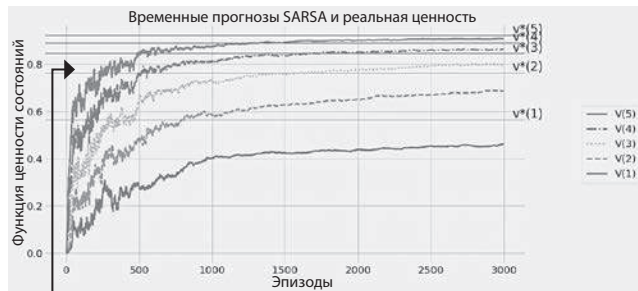
Посчитаем

Похожие тенденции среди методов бэггинга и стратегических методов

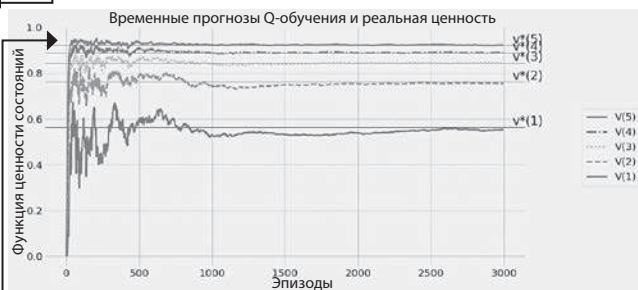
(1) Это управление методом первого визита в Монте-Карло. Как видите, значения сильно колеблются, как в алгоритме ϵ -прогнозирования. Кроме того, эти алгоритмы используют одну и ту же стратегию выбора действий. Единственная разница в методе, применяемом для оценки политик. Круто, правда?!



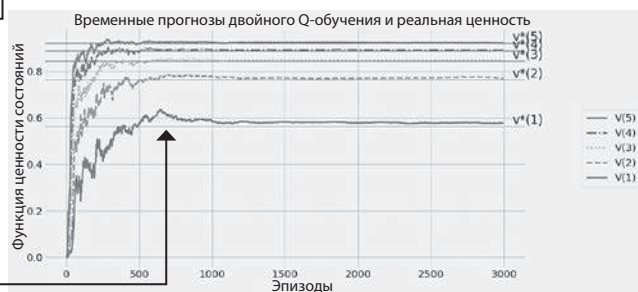
(2) SARSA и MC — стратегические методы, но первый использует бэггинг, а второй — нет. В этих экспериментах видно, что у SARSA более низкая дисперсия по сравнению с MC, но для достижения оптимальных значений ему нужно почти столько же времени



(3) Q-обучение — это нестратегический метод с использованием бэггинга. Как видите, прогнозируемая ценность очень быстро приближается к фактической. С другой стороны, прогнозы зачастую оказываются завышенными и довольно резко колеблются



(4) А вот двойное Q-обучение приближается к оптимальной функции ценности состояний немного медленнее обычного, но делает это более стабильно. Прогнозы по-прежнему слегка завышены, но не бесконтрольно





Посчитаем

Анализ политик, исследованных в среде СПС

(1) Вот несколько интересных графиков, которые помогут лучше понять рассмотренные алгоритмы. Выше отмечалось, что Q-обучение первым достигает оптимальной ценности, но при этом дает завышенные прогнозы. Как это влияет на результативность? На этом графике можно видеть, что двойное Q-обучение достигает 100 % результативности раньше обычного. Отмечу, что под результативностью я подразумеваю попадание в целевое состояние, которое в СПС представлено крайней правой ячейкой



(2) Что насчет средней выгоды, получаемой агентом во время обучения? Какова его производительность по сравнению с агентом, применяющим оптимальную политику? Что ж, двойное Q-обучение опять приходит к оптимуму первым. Это усредненные результаты для пяти разных начальных значений. У них много шума, но тенденция должна сохраняться



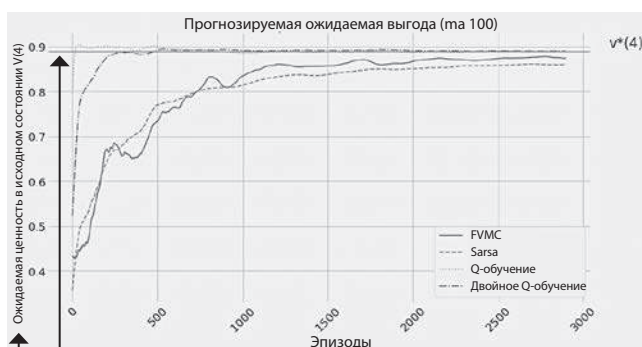
(3) Наконец, мы можем взглянуть на скользящие средние потери, которые, напомним, отражают отличие от оптимума и показывают величину потерянной агентом награды (что может быть оправданно, так как это произошло в ходе обучения). И снова двойное Q-обучение показывает лучшую производительность



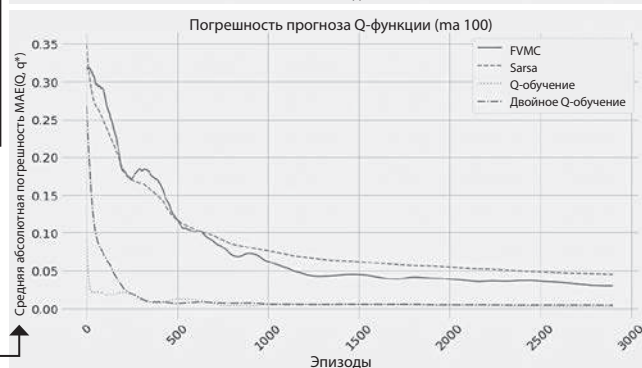
Посчитаем

Анализ функций ценности, исследованных в среде СПС

(1) Эти графики тоже довольно любопытные. Я показываю скользящее среднее прогнозируемой ожидаемой выгоды по 100 эпизодам — то есть сколько агент ожидает получить за эпизод (от начального до конечного состояния) по сравнению с тем, какую выгоду он должен был бы ожидать при использовании оптимальной V-функции в исходном состоянии



(2) На следующем графике показана погрешность прогноза функции ценности состояний, V-функции. Это среднее абсолютное отклонение от соответствующих оптимумов. Видите, насколько быстро значения Q-обучения снижаются почти к нулю, хотя самую низкую погрешность первым начинает показывать двойное Q-обучение. SARSA и FVMC показывают равные результаты в этой простой среде



(3) Наконец, здесь показана погрешность прогноза функции ценности действий, Q-функции. Этот график отличается от прошлых двух, где я проводил вычисления только между прогнозируемым максимальным действием и оптимумом. Здесь представлена средняя абсолютная погрешность для всех действий

Подведем итоги

В этой главе вы применили на практике все то, что вам удалось изучить на текущий момент. Мы познакомились с алгоритмами улучшения политик методом проб и ошибок. Они обучаются одновременно на последовательной и оценочной обратной связи: то есть агенты учатся одновременно балансировать как между краткосрочными и долгосрочными целями, так и между сбором и применением информации. Но, в отличие от предыдущей главы, где они ограничивались выполнением задач прогнозирования, здесь мы обучали их выполнять задачи управления.

Вы познакомились с некоторыми базовыми концепциями и узнали, что задача прогнозирования состоит в оценке политик, тогда как задача управления — в их оптимизации. Вы увидели, что для выполнения задач прогнозирования используются методы оценки политик, подобные тем, которые мы изучили в предыдущей главе. А вот задачу управления, как ни странно, нельзя выполнить только с помощью этих методов. В дополнение к ним нужно применять методы оценки, позволяющие агенту учиться прогнозировать Q -функции по одним лишь выборкам.

Эта глава была посвящена принципу обобщенной итерации политик (generalized policy-iteration, GPI), который заключается во взаимодействии между методами оценки и улучшения. Первый согласует функцию ценности с оцениваемой политикой, тогда как второй нарушает эту согласованность, но делает политику более оптимальной. Согласно GPI, взаимодействие между этими двумя процессами приводит к постепенному улучшению политики до тех пор, пока она и функция ценности не станут оптимальными. Теория обучения с подкреплением поддерживает этот шаблон. Это говорит о том, что мы действительно можем найти оптимальные политики и V -функции в дискретных пространствах состояний и действий при соблюдении лишь нескольких требований. Еще вы узнали, что теории GLIE и стохастической аппроксимации применимы к алгоритмам RL на разных уровнях.

Вы изучили еще много всего: например, стратегические и нестратегические методы, онлайн- и офлайн-обучение и т. д. Двойное Q -обучение и двойное обучение в целом — это важные методики, от которых мы будем отталкиваться в будущем. В следующей главе мы рассмотрим продвинутые методы выполнения задач управления. С усложнением среды возникает необходимость в других методах формирования оптимальных политик. Далее мы обсудим самые действенные и эффективные из них: те, которые не только решают задачи, но и делают это с помощью меньших выборок по сравнению с методами из этой главы.

На этом этапе вы:

- знаете, что большинство агентов работают по принципу обобщенной итерации политик;
- понимаете, что GPI выполняет задачу управления за счет оценки и улучшения политик;
- знакомы с несколькими агентами, выполняющими задачи управления по принципу GPI.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

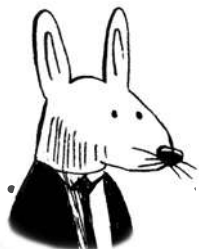
- **#gdr1_ch06_tf01.** Все алгоритмы из этой главы используют две ключевые переменные: темп обучения (альфа) и коэффициент дисконтирования (гамма). Проанализируйте их. Как они взаимодействуют друг с другом? Как влияют на общую награду, полученную агентом, и на результативность политики?
- **#gdr1_ch06_tf02.** Еще один момент, на который нужно обратить внимание: во всех методах использовалась одна стратегия исследования — экспоненциально затухающая эпсилон-жадная. Но была ли она лучшей из возможных? Как бы вы применили другие стратегии из главы 4? Не хотите создать и проверить свою? Попробуйте изменить гиперпараметры, относящиеся к исследованию среды, и посмотрите, что будет с результатами. У вас не должно возникнуть с этим никаких проблем. Откройте блокноты Jupyter для этой книги и отредактируйте несколько гиперпараметров, а затем полностью измените стратегию исследования и расскажите нам, что у вас получилось.
- **#gdr1_ch06_tf03.** Вы, наверное, уже заметили, что алгоритмы в этой главе неправильно используют ограничение временных шагов. Попробуйте найти причину и внесите исправления. Изменились ли результаты? Повысилась ли эффективность агентов? Что улучшилось: прогнозирование оптимальной функции ценности, поиск оптимальной политики или все сразу? Если улучшения есть, то насколько они существенные? Можете вернуться к этому аспекту по прочтении главы 8. Поделитесь своими наблюдениями.
- **#gdr1_ch06_tf04.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdr1_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы научитесь создавать более эффективные агенты обучения с подкреплением для достижения оптимальной производительности при взаимодействии со сложными средами.
- ✓ Научитесь создавать агенты, которые более эффективно достигают целей за счет извлечения максимальной пользы из накопленного опыта.
- ✓ Улучшите агенты из предыдущих глав так, чтобы они как можно эффективнее использовали собираемые ими данные и быстрее оптимизировали свою производительность.

Производительность — это умение делать вещи правильно, а эффективность — умение делать правильные вещи.

*Питер Друкер (Peter Drucker),
основоположник современной
теории менеджмента и обладатель
Президентской медали Свободы*

Здесь мы займемся улучшением агентов из предыдущей главы в двух направлениях. Во-первых, мы применим λ -выгоду (см. главу 5) для соответствия требованиям принципа обобщенной итерации политик. Вы изучите ее использование как для стратегических методов, так и для нестратегических. В сочетании со следами приемлемости это позволяет ускорить распространение коэффициентов доверия между подходящими парами «состояние — действие» по сравнению со стандартными методами. Благодаря этому ожидаемая функция ценности быстрее приближается к фактическим значениям.

Во-вторых, мы рассмотрим алгоритмы, которые используют образцы опыта для изучения модели среды, марковского процесса принятия решений (MDP). Так эти методы извлекают максимум пользы из собранных данных и зачастую достигают оптимума быстрее остальных. Алгоритмы, которые пытаются изучить модель среды, относят к категории *модельно-ориентированного обучения с подкреплением*.

Важно отметить, что, хотя мы исследуем эти направления по отдельности, ничто не мешает вам попытаться объединить их. Возможно, именно это вам и следует сделать после прочтения этой главы. А теперь перейдем к подробностям.

RL

С акцентом на RL

Планирование, RL без модели и модельно-ориентированное RL

Планирование. Алгоритмы, которым для формирования политики нужна модель среды. Методы планирования могут ориентироваться на пространство состояний, используя его для поиска политики, или на пространство всех возможных планов (как в случае с генетическими алгоритмами). В этой книге мы уже имели дело с такими алгоритмами, как итерация ценности и политик.

RL без модели. Алгоритмы, которые не используют модель среды. Их особенность в том, что для формирования политики им не нужна карта, модель или MDP. Вместо этого они обучаются методом проб и ошибок. В этой книге мы уже рассмотрели несколько алгоритмов RL без модели, включая MC, SARSA и Q-обучение.

Модельно-ориентированное RL. Алгоритмы, которые могут обучаться, но не требуют модели среды для создания политики. Их отличие в том, что изначально они могут обойтись и без модели, но, если она есть, они способны эффективно ее использовать. Что более важно, они могут попытаться сами сформировать модель в ходе взаимодействия со средой. В этой главе мы познакомимся с несколькими из них, включая Dyna-Q и выборку траектории.

Улучшение политик с помощью достоверных целей

Первый способ улучшения, который мы обсудим в этой главе, — использование более достоверных целей в наших методах оценки политик. Ранее мы уже рассматривали методы оценки политик, использующие разные цели для прогнозирования функций ценности. Вы познакомились с подходами Монте-Карло и TD и узнали о цели под названием λ -выгода, основанной на взвешенном сочетании наград, полученных во всех посещенных состояниях.

TD(λ) — это метод прогнозирования, который использует λ -выгоду для оценки политик. Но, как вы помните, при решении задач управления нужно использовать метод оценки для прогнозирования функций и метод улучшения для исследования среды. Сейчас мы обсудим методы управления, похожие на SARSA и Q-обучение, но использующие λ -выгоду.



Конкретный пример

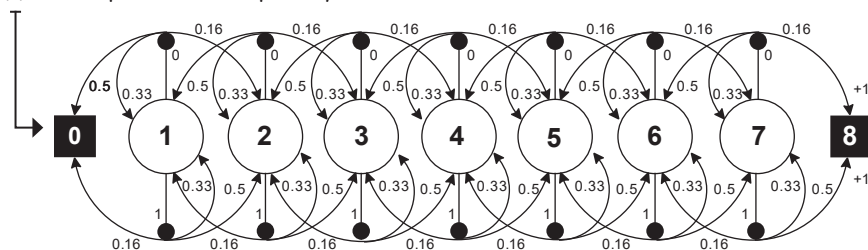
Среда «скользящая прогулка семь»

Для знакомства с алгоритмами мы используем ту же среду, что и в предыдущей главе, — «скользящая прогулка семь» (СПС). Но в конце главы мы опробуем эти методы в куда более сложных средах.

Как вы помните, СПС — это однострочная грид-среда с семью нетерминальными состояниями. Ее отличительная черта — то, что она скользящая, — последствия действий стохастические. Если агент решает перейти влево, у него это может получиться, но есть вероятность того, что он переместится вправо или останется на месте.

MDP среды «скользящая прогулка семь»

(1) Та же среда, что и в предыдущей главе



Напомню: выше приведен MDP для этой среды. Но имейте в виду, что у агента нет никакого доступа к информации о вероятности переходов. Динамика среды остается неизвестной. Агенту изначально ничего не известно и об отношениях между состояниями.

SARSA(λ): улучшение политик после каждого шага на основе многошаговых прогнозов

SARSA(λ) — это улучшенная версия исходного агента SARSA, с единственной разницей в том, что вместо цели на основе одношагового бэггинга (цели TD, как в SARSA) используется λ -выгода. Все остальное то же самое. Seriously! Видите, зная основы, проще воспринимать сложные концепции.

Теперь я бы хотел подробнее остановиться на концепции следов (или признаков) приемлемости из главы 5. Тип признаков, которые я использовал, *накопительный*. Но есть и другие способы получения признаков состояния или пар «состояние — действие», которые возвращают награду.

В этом разделе мы детально обсудим накопительные признаки и применим их для выполнения задачи управления. Мы исследуем и другой вид признаков, *заменяющий*, и используем его вместе с накопительным в агенте SARSA(λ).

0001 Немного истории

Появление агентов SARSA и SARSA(λ)

В 1994 году Гэвин Раммери (Gavin Rummery) и Махесан Нираньян (Mahesan Niranjana) опубликовали научную работу *Online Q-Learning Using Connectionist Systems*, где был представлен алгоритм, который они называли «модифицированным ассоциативным Q-обучением». В 1996 году Сингх (Singh) и Саттон (Sutton) переименовали его в SARSA. Это название основано на последовательности из пяти событий, которую использует этот алгоритм ($S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$). Люди часто интересуются происхождением этих терминов, и, как вы вскоре увидите, исследователи в области RL могут быть довольно находчивыми в выборе названий.

Забавно, что в 1995 году, еще до того, как произошло это публичное и «несанкционированное» переименование, Гэвин в своей докторской диссертации *Problem Solving with Reinforcement Learning* извинился перед Саттоном за использование термина «модифицированное ассоциативное Q-обучение», зная, что тот предпочитал SARSA. Саттон продолжил называть этот алгоритм SARSA, и в итоге этот вариант прижился в сообществе RL. Кстати, в той же диссертации Гэвина был впервые представлен агент SARSA(λ).

Защитив докторскую в 1995 году, Гэвин стал программистом и затем ведущим разработчиком в компании, занимавшейся созданием игр из цикла Tomb Raider. У него сложилась успешная карьера в сфере игровой разработки.

Махесан, который занял место скоростно скончавшегося научного руководителя Гэвина, пошел по более традиционному, академическому карьерному пути. После окончания университета в 1990 году он выступает в роли лектора и профессора.

Чтобы применить накопительные признаки к задаче управления, достаточно внести одно изменение: вместо посещенных состояний нужно отслеживать посещенные пары «состояние — действие». Поэтому вместо вектора применимости будет использоваться матрица применимости.

Механизм заменяющих признаков тоже довольно прост. Он заключается в использовании для следов приемлемости максимального значения, 1, то есть информация о применимости будет накапливаться не бесконечно, а только пока не достигнет единицы. Преимущество этого подхода в том, что ваши признаки не станут слишком большими, даже если агент «застрянет» в цикле. Основная идея следующая: при посещении пары «состояние — действие» заменяющим признакам присваивается 1, после чего они затухают в соответствии со значением λ , как в случае с методом с накопительными признаками.

0001 Немного истории

История появления механизма следов приемлемости

Общая концепция механизма следов приемлемости впервые была выдвинута в 1972 году Гарри Клопфом (A. Harry Klopff) в его научной работе *Brain Function and Adaptive Systems — A Heterostatic Theory*. Он описал, как синапсы могли бы стать подходящими для применения изменений («применимыми») после событий закрепления. Гарри Клопф высказал следующее предположение: «Когда нейрон срабатывает, все его возбуждающие и подавляющие синапсы, которые были активны во время суммирования потенциалов, спровоцировавших реакцию, *пригодны* для внесения изменений в их коэффициенты пропуска».

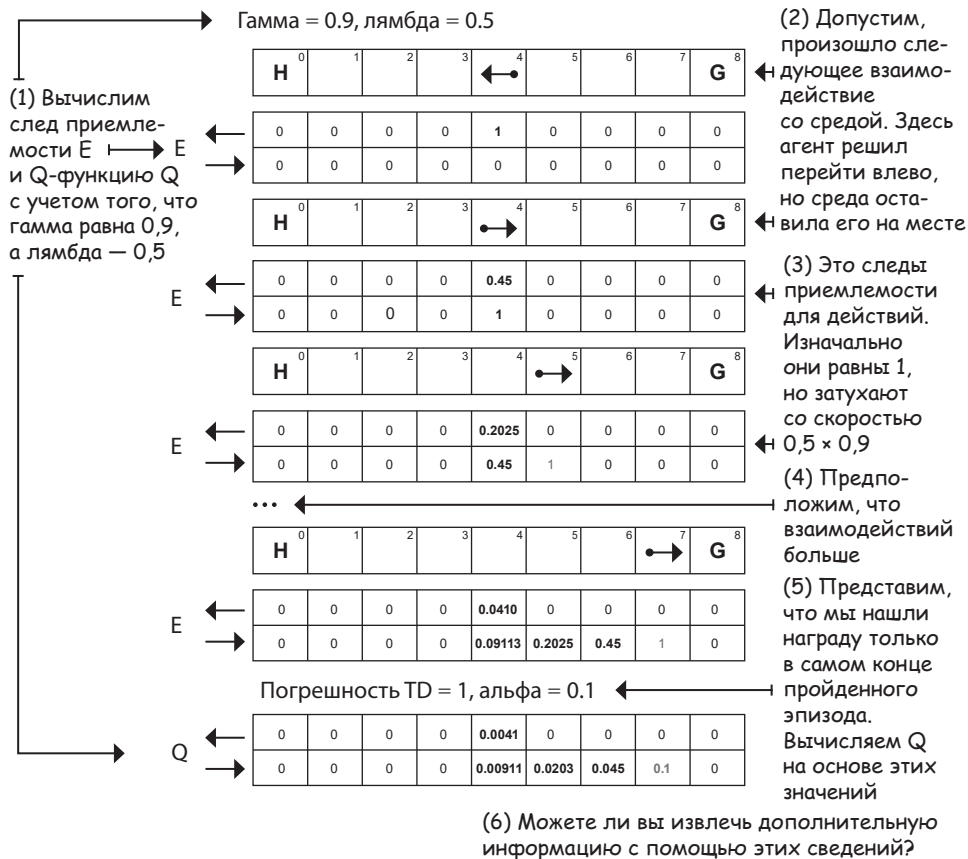
В контексте RL механизм следов приемлемости был впервые упомянут в диссертации Ричарда Саттона (1984), где он предложил накопительные признаки, с которыми вы познакомились в этой книге (их еще называют традиционными).

А вот заменяющие признаки впервые были представлены в 1996 году в научном труде Сатиндера Сингха и Ричарда Саттона *Reinforcement Learning with Replacing Eligibility Traces*. Позже мы к нему вернемся.

Сингх и Саттон обнаружили несколько любопытных фактов. Во-первых, оказалось, что механизм заменяющих признаков, в сравнении с накопительными, делает обучение быстрее и надежнее. Во-вторых, накопительным свойственно смещение, а заменяющим — нет. Но самое интересное то, что между TD(1), MC и следами приемлемости есть определенная связь.

Обнаружилось, что TD(1) с заменяющими признаками связан с FVMC, а TD(1) с накопительными — с EVMC. Более того, офлайн-версия TD(1) с заменяющими признаками оказалась идентична методу FVMC. Мир тесен!

Накопительные признаки в среде СПС



По сути

Эвристические показатели частоты и новизны в механизме накопительных признаков

Накопительный признак сочетает в себе эвристические показатели частоты и новизны. Когда агент выбирает пару «состояние — действие», ее признак инкрементируется на 1. Теперь представьте, что в среде есть цикл и агент выбирает ту же пару несколько раз. Следует ли считать эту пару более «ответственной» за дальнейшие полученные вознаграждения?

Накопительные признаки помогают отслеживать значения больше 1, а замещающие — нет. Признаки позволяют объединять эвристики частоты (как часто выбирается пара «состояние — действие») и новизны (как давно она выбиралась), неявно закодированные в механизме признаков.

Заменяющие признаки в среде СПС

(1) Та же траектория, что и прежде: поменялись только след приемлемости E и Q-функция Q

Гамма = 0.9, лямбда = 0.5

	0	1	2	3	4	5	6	7	8
	H				←				G

E	←	0	0	0	0	1	0	0	0	0
→		0	0	0	0	0	0	0	0	0

	0	1	2	3	4	5	6	7	8
	H				→				G

E	←	0	0	0	0	0	0	0	0	0
→		0	0	0	0	1	0	0	0	0

	0	1	2	3	4	5	6	7	8
	H				→				G

E	←	0	0	0	0	0	0	0	0	0
→		0	0	0	0	0.45	1	0	0	0

...

	0	1	2	3	4	5	6	7	8
	H							→	G

E	←	0	0	0	0	0	0	0	0	0
→		0	0	0	0	0.09113	0.2025	0.45	1	0

Q ←
→ Погрешность TD = 1, альфа = 0.1

	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0.00911	0.0203	0.045	0.1	0	0

(4) Снова попытайтесь воссоздать последовательность выбранных агентом действий. Это несложно

(2) Здесь «заменяются» признаки других действий в том же состоянии. В состоянии 4 признак «влево» заменяется признаком «вправо»

(3) В конце мы действительно получаем другую Q-функцию, более логичную. Но у прошлой тоже есть свои достоинства



Я знаю Python

Агент SARSA(λ)

```
def sarsa_lambda(env,
                 gamma=1.0,
                 init_alpha=0.5,
                 min_alpha=0.01,
                 alpha_decay_ratio=0.5,
                 init_epsilon=1.0,
                 min_epsilon=0.1,
                 epsilon_decay_ratio=0.9,
                 lambda_=0.5,
                 replacing_traces=True,
                 n_episodes=3000):
```

(1) Агент SARSA(λ) — это что-то среднее между методами SARSA и TD(λ)

(2) Это гиперпараметр `lambda_` (с _ в конце: в Python `lambda` — ключевое слово)

(3) Переменная `replacing_traces` устанавливает алгоритм для переключения между заменяющим и накопительным признаками

```
    nS, nA = env.observation_space.n, env.action_space.n
    pi_track = []
```

(4) Используем те же переменные, что и прежде...

(5) ...включая Q-функцию и матрицу отслеживания

```
    Q = np.zeros((nS, nA), dtype=np.float64)
    Q_track = np.zeros((n_episodes, nS, nA),
                      dtype=np.float64)
```

(6) Это следы приемлемости, которые позволяют нам отслеживать состояния, подходящие для обновления

```
    E = np.zeros((nS, nA), dtype=np.float64)
```

```
    select_action = lambda state, Q, epsilon: \
        np.argmax(Q[state]) \
        if np.random.random() > epsilon \
        else np.random.randint(len(Q[state]))
```

(7) Остальной код, включая функцию `select_action`, векторы с альфой и значениями эпсилона, не изменился

```
    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio, n_episodes)

    epsilons = decay_schedule(
        init_epsilon, min_epsilon,
        epsilon_decay_ratio, n_episodes)
```

```

for e in tqdm(range(n_episodes), leave=False):
    (8) В каждом новом эпизоде мы обнуляем
        применимость каждого состояния
    E.fill(0)
    (9) Затем мы как обычно сбрасываем среду и флаг done
    state, done = env.reset(), False
    (10) Выбираем действие в исходном состоянии
    action = select_action(state, Q, epsilons[e])

    while not done:
        (11) Входим в цикл взаимодействия

        (12) Передаем действие среде и получаем кортеж опыта
        next_state, reward, done, _ = env.step(action)
        next_action = select_action(next_state,
                                   Q,
                                   epsilons[e])
        (13) Выбираем действие для следующего
            состояния, используя Q-таблицу и эпсилон,
            соответствующие текущему эпизоду
        td_target = reward + gamma * \
                    Q[next_state][next_action] * (not done)
        td_error = td_target - Q[state][action]
        (14) Как и в алгоритме SARSA, вычисляем цель TD и погрешность TD
        (15) Инкрементируем признак пары «состояние — действие»
            и, если он заменяющий, урезаем его до 1
        E[state][action] = E[state][action] + 1
        if replacing_traces: E.clip(0, 1, out=E)

        (16) Обратите внимание: мы применяем погрешность TD
            сразу ко всем применимым парам «состояние — действие».
            Несмотря на использование целой Q-таблицы, эпсилон в большинстве
            случаев будет равен 0 (или больше 0 для применимых пар)
        Q = Q + alphas[e] * td_error * E
        E = gamma * lambda_ * E
        (17) Уменьшаем показатели применимости

        state, action = next_state, next_action

        (18) Обновляем переменные
        Q_track[e] = Q
        (19) Сохраняем Q и pi
        pi_track.append(np.argmax(Q, axis=1))

    (20) В конце обучения извлекаем и возвращаем V и pi
    V = np.max(Q, axis=1)
    pi = lambda s: {s:a for s, a in enumerate(\
                    np.argmax(Q, axis=1))}[s]
    return Q, V, pi, Q_track, pi_track

```




Аналогия от Мигеля

Накопительные/заменяющие признаки и диета

Пару месяцев назад у моей дочери начались проблемы со сном. Она просыпалась по несколько раз за ночь и очень громко плакала, но не хотела объяснять, в чем проблема.

Спустя несколько таких ночей мы с женой решили, что с этим нужно что-то делать, и попытались найти «признак» проблемы, чтобы присвоить ему определенный «кредит доверия». В общем, решили разобраться, в чем дело.

Нам пришлось провести небольшое расследование (если у вас есть дети, вы понимаете, о чем речь). Мы перебрали много разных вариантов, чтобы определить проблему, и где-то через неделю из всех возможных причин осталась всего одна — еда. Мы знали, что наша дочь не могла уснуть из-за употребления каких-то продуктов, но определить, что это, нам не удалось. Я заметил, что в течение дня она ела много углеводных продуктов с глютенем, включая хлопья, макароны, крекеры и хлеб, а ближе к вечеру налегала на фрукты.

Накопительный признак, который я взял на вооружение, указывал на углеводы. «Конечно! — думал я. — Всем известно, что глютен — это зло. К тому же моя дочь употребляет его весь день». Если посчитать, сколько раз в совокупности она ела что-то с глютенем, вывод будет напрашиваться сам собой. Поэтому от этого компонента мы избавились.

К нашему удивлению, мы избавились от проблемы лишь наполовину — наша дочь все еще чувствовала дискомфорт. Через несколько дней моя жена вспомнила, что в детстве ела бананы по ночам, что доставляло ей немало проблем. Я не мог в это поверить, ведь бананы — это фрукты. Они должны приносить только пользу, ведь так? Но после того, как мы отказались от бананов, наша дочь начала хорошо спать. Поразительно, но факт!

Возможно, если бы вместо накопительного признака я использовал заменяющий, мы бы не так сильно винили все те углеводы, которые моя дочь употребляла в течение дня, и смогли бы заметить, что бананы перед сном сыграли свою роль.

Суть в том, что накопительные признаки могут «преувеличивать» некоторые факторы, если они встречаются достаточно часто, тогда как заменяющие придают частоте событий более умеренное значение, что помогает агенту учитывать редкие, но недавние события.

Но не стоит делать поспешных выводов. Как это обычно бывает в жизни (и в RL), у всех инструментов есть свое назначение, и не нужно сразу от них отказываться. Я лишь описываю доступные варианты, а выбор подходящих методик для достижения ваших целей остается за вами.

Q(λ) Уоткинса: снова отделяем поведение от обучения

Конечно же, у λ -алгоритмов есть нестратегическая версия для выполнения задач управления — Q(λ). Это расширенный вариант Q-обучения, который использует λ -выгоду для удовлетворения требований к оценке политик в соответствии с принципом обобщенной итерации. Мы вносим всего одно изменение: используем λ -выгоду вместо цели TD для нестратегического управления (той, которая выбирает для следующего состояния действие с максимальной ценностью). Q-обучение можно расширить до следов приемлемости двумя способами, но здесь представлена только оригинальная версия, которую обычно называют Q(λ) Уоткинса.

0001 Немного истории

Внедрение Q-обучения и Q(λ) агентов

В 1989 году Крис Уоткинс (Chris Watkins) представил методы Q-обучения и Q(λ) в своей докторской диссертации *Learning from Delayed Rewards*, которая положила начало развитию современной теории обучения с подкреплением.

Q-обучение до сих пор остается одним из самых популярных алгоритмов RL благодаря своей простоте и производительности. Q(λ) теперь называют Q(λ) Уоткинса, так как есть и другая версия Q(λ), разработанная Цзином Пенгом (Jing Peng) и Рональдом Уильямсом (Ronald Williams) в 1993–1996 годах (Q(λ) Пена).

В 1992 году Крис и Питер Даян (Chris, Peter Dayan) опубликовали научную работу, *Technical Note Q-learning*, где доказали теорему сходимости для Q-обучения. Они показали, что Q-обучение сходится в оптимальной функции ценности действий с вероятностью 1 при условии, что все пары «состояние — действие» представлены дискретно и выбираются по несколько раз.

К сожалению, почти сразу после этого Крис прекратил заниматься исследованиями в области RL. Свою карьеру он продолжил в лондонских хедж-фондах и посещал исследовательские лаборатории, включая группу, возглавляемую Яном Лекуном (Yann LeCun), продолжая работать над задачами, связанными с ИИ, но это уже не имело прямого отношения к RL. Последние 22 с лишним года Крис читает лекции по искусственному интеллекту в Лондонском университете.

В 1991 году, завершив работу над своей докторской диссертацией *Reinforcing Connectionism: Learning the Statistical Way* (обратите внимание на слово connectionism («коннекционизм») — именно так в то время назывались нейросети), Питер работал научным сотрудником, в том числе с Джеффри Хинтоном (Geoff Hinton) в Университете Торонто. Он был научным руководителем Демиса Хасабиса (Demis Hassabis), сооснователя DeepMind. Питер неоднократно занимал должность директора в разных исследовательских лабораториях. Его последнее место работы — Институт Макса Планка.

В 2018 году он стал членом Лондонского королевского общества — это одно из престижнейших званий, присуждаемых в Великобритании.



Я знаю Python

Агент $Q(\lambda)$ Уоткинса

```
def q_lambda(env,
             gamma=1.0,
             init_alpha=0.5,
             min_alpha=0.01,
             alpha_decay_ratio=0.5,
             init_epsilon=1.0,
             min_epsilon=0.1,
             epsilon_decay_ratio=0.9,
             lambda_=0.5,
             replacing_traces=True,
             n_episodes=3000):
```

(1) Агент $Q(\lambda)$ — это смесь Q -обучения и методов $TD(\lambda)$

(2) А вот и гиперпараметры λ и replacing_traces

(3) Полезные переменные

```
    nS, nA = env.observation_space.n, env.action_space.n
    pi_track = []
```

(4) Q -таблица

```
    Q = np.zeros((nS, nA), dtype=np.float64)
    Q_track = np.zeros((n_episodes, nS, nA), dtype=np.float64)
```

(5) Матрица следов приемлемости для всех пар «состояние — действие»

```
    E = np.zeros((nS, nA), dtype=np.float64)
```

(6) Уже знакомый нам код

```
    select_action = lambda state, Q, epsilon: \
        np.argmax(Q[state]) \
        if np.random.random() > epsilon \
        else np.random.randint(len(Q[state]))

    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio, n_episodes)

    epsilons = decay_schedule(
        init_epsilon, min_epsilon,
        epsilon_decay_ratio, n_episodes)

    for e in tqdm(range(n_episodes), leave=False):
```

(7) $Q(\lambda)$ — это нестратегический метод, поэтому E нужно использовать осторожно. Мы формируем жадную политику, но при этом руководствуемся исследовательской. Как и прежде, E вначале заполняется нулями

```
        E.fill(0)
```

(8) Сбрасываем среду и done

```
└─> state, done = env.reset(), False
```

(9) Обратите внимание: мы заранее выбираем действие, как в SARSA, хотя в Q-обучении мы этого не делали. Нам нужно проверить, насколько жадным будет наше следующее действие!

```
└─> action = select_action(state,
                           Q,
                           epsilons[e])
```

(10) Входим в цикл взаимодействия

```
└─> while not done:
```

(11) Проходимся по среде и накапливаем опыт

```
next_state, reward, done, _ = env.step(action)
```

(12) Выбираем next_action, как в SARSA!

```
└─> next_action = select_action(next_state,
                               Q,
                               epsilons[e])
```

(13) И с помощью него убеждаемся, что действие на следующем шаге по-прежнему соответствует жадной политике

```
└─> next_action_is_greedy = \
    Q[next_state][next_action] == Q[next_state].max()
```

(14) На этом шаге мы все еще вычисляем цель TD с помощью max — как в обычном Q-обучении

```
td_target = reward + gamma * \
    Q[next_state].max() * (not done)
```

(15) Используем цель TD

для вычисления погрешности TD

```
└─> td_error = td_target - Q[state][action]
```

(16) Снова вычисляем погрешность TD на основе цели и текущего прогноза пары «состояние — действие». Заметьте, это state, а не next_state!

```
└─> td_error = td_target - Q[state][action]
```

(17) Еще один подход к методам управления на основе заменяющих признаков: обнуление оценок всех действий в текущем состоянии с последующей инкрементацией текущего действия

```
└─> if replacing_traces: E[state].fill(0)
```

(18) Инкрементируем степень применимости текущей пары «состояние — действие» на 1

```
└─> E[state][action] = E[state][action] + 1
```

```

└── Q = Q + alphas[e] * td_error * E

```

(19) Как и прежде, умножаем всю матрицу следов приемлемости на погрешность и темп обучения в эпизоде e , а затем приближаем всю Q -таблицу к этой погрешности. Так мы в той или иной степени ослабляем сигнал для всех посещенных состояний

```

if next_action_is_greedy:
    E = gamma * lambda_ * E
else:
    E.fill(0)
└──

```

(20) Это тоже важно. Если действие, которое мы выполним в следующем состоянии (мы его уже выбрали), окажется жадным, то матрица применимости, как обычно, подлежит затуханию. Если же нет, ее нужно обнулить, так как мы больше не будем формировать жадную политику

(21) В конце шага делаем следующие состояние и действие текущими

```

└── state, action = next_state, next_action

```

```

Q_track[e] = Q
pi_track.append(np.argmax(Q, axis=1))

```

(23) Сохраняем V и итоговое π

```

└── V = np.max(Q, axis=1)
    pi = lambda s: {s:a for s, a in enumerate(\

```

(22) Сохраняем Q и π

```

np.argmax(Q, axis=1))}[s]

```

(24) В завершение возвращаем все эти значения

```

└── return Q, V, pi, Q_track, pi_track

```

Агенты, которые взаимодействуют, обучаются и планируют

В главе 3 мы обсуждали такие алгоритмы планирования, как итерация ценности (value iteration, VI) и итерация политик (policy iteration, PI), которым нужна модель среды, MDP. Они вычисляют оптимальные политики на основе готовых данных. А вот в предыдущей главе вы узнали о методах обучения с подкреплением без модели. Ознакомившись с тем, что я написал, вы могли подумать, что они лучше методов планирования. Но так ли это?

Преимущество безмодельного RL, по сравнению с методами планирования, в том, что ему не нужен доступ к MDP. Зачастую модели MDP сложно получить заранее, а иногда их вообще невозможно создать. Представьте себе игру го с 10^{170} возможными состояниями или StarCraft II с 10^{1685} . Это немаленькие числа, даже без учета пространств действий и переходной функции! Отсутствие необходимости в готовой модели MDP — это практическое преимущество.

Но давайте на секунду задумаемся: что, если нам не будет нужен MDP заранее, но мы сможем сформировать его, взаимодействуя со средой? Представьте: прогуливаясь по незнакомым улицам, вы подсознательно строите карту у себя в голове. Спустя некоторое время вы находите кофейню, заказываете чашечку кофе и уже знаете, как вернуться назад. Навык генерации карт местности должен быть интуитивно понятен. Способен ли агент обучения с подкреплением на нечто подобное?

В этом разделе мы поговорим об агентах, которые взаимодействуют со средой по примеру безмодельных методов, но формируют на основе этих взаимодействий модели среды, MDP. Умение формировать карты зачастую позволяет агентам использовать меньше выборок опыта для изучения оптимальных политик. Такие методы относят к *модельно-ориентированному обучению с подкреплением*. Отмечу, что в научной литературе VI и PI могут называть как методами планирования, так и модельно-ориентированными методами. Я предпочитаю первый вариант, так как для того, чтобы сделать что-то полезное, им нужна модель MDP. SARSA и Q-обучение — безмодельные, так как не требуют готовой модели и не формируют ее сами. Методы из этого раздела — модельно-ориентированные, потому что они хоть и не требуют доступа к модели MDP, но могут ее сами создать и использовать (если не саму модель, то ее аппроксимацию).

RL

С акцентом на RL

Модели выборки и дистрибутивные модели

Модели выборки. Модели среды, генерирующие единую выборку вероятностей разных переходов, — вы делаете выборку перехода из модели.

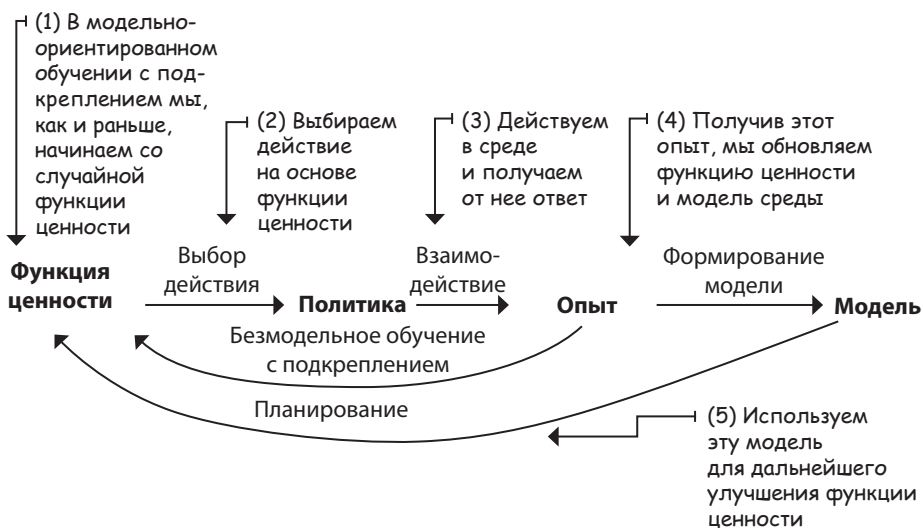
Дистрибутивные модели. Модели среды, генерирующие вероятностное распределение переходных функций и вознаграждения.

Dyna-Q: формирование моделей выборки

Одна из самых известных в этом направлении архитектур, *Dyna-Q*, состоит из двух методов, переплетающихся между собой: безмодельного метода RL, такого как Q-обучение, и планирования вроде итерации по значениям. При этом для улучшения функции ценности действий используется опыт, накопленный при взаимодействии со средой и полученный из сформированной модели.

В Dyna-Q мы отслеживаем функции перехода и вознаграждения в виде трехмерных тензоров, индексированных по состоянию, действию и следующему состоянию. Тензор перехода отсчитывает, сколько раз нам встречался трехэлементный кортеж (s, a, s') , обозначающий количество переходов в состояние s' из s при выборе действия a . Тензор вознаграждения хранит среднюю награду, полученную в трехэлементном кортеже (s, a, s') . Это помогает определить, какой награды следует ожидать, когда выбор действия a в состоянии s переносит нас в s' .

Структура модельно-ориентированного обучения с подкреплением



0001 Немного истории

Внедрение агента Dyna-Q

Идеи, связанные с методами модельно-ориентированного обучения, начали зарождаться много лет назад. Их приписывают сразу нескольким исследователям, но в основе архитектуры Dyna лежат три научных работы.

Первая, опубликованная в 1981 году, *An Adaptive Network that Constructs and Uses an Internal Model of Its World*, принадлежит Ричарду Саттону и Эндрю Барто (Andrew Barto). Вслед за ней в 1990 и 1991 годах Саттон опубликовал еще две работы: *Integrated Architectures for Learning, Planning and Reacting Based on Approximating Dynamic Programming* и *Dyna, an Integrated Architecture for Learning, Planning and Reacting*. В последней была представлена общая архитектура Dyna, которая и привела к появлению агента Dyna-Q.



Я знаю Python

Агент Dyna-Q

```
def dyna_q(env,
           gamma=1.0,
           init_alpha=0.5,
           min_alpha=0.01,
           alpha_decay_ratio=0.5,
           init_epsilon=1.0,
           min_epsilon=0.1,
           epsilon_decay_ratio=0.9,
           n_planning=3,
           n_episodes=3000):
```

(1) Агенты Dyna-Q и Q-обучения похожи, но первый формирует модель среды и использует ее для улучшения прогнозов

(2) Гиперпараметр `n_planning` определяет количество обновлений прогнозов, полученных из сформированной модели

(3) Первая часть алгоритма в основном остается неизменной

```
    nS, nA = env.observation_space.n, env.action_space.n
    pi_track = []
```

(4) Мы присваиваем Q-функции нулевое значение и т. д.

```
    Q = np.zeros((nS, nA), dtype=np.float64)
    Q_track = np.zeros((n_episodes, nS, nA), dtype=np.float64)
```

(5) Но теперь мы создаем одну функцию для отслеживания переходной функции

(6) И еще одну — для отслеживания сигнала вознаграждения

```
    T_count = np.zeros((nS, nA, nS), dtype=np.int)
    R_model = np.zeros((nS, nA, nS), dtype=np.float64)
```

(7) Как обычно, инициализируем `select_action`, векторы с альфой и значениями эpsilon

```
    select_action = lambda state, Q, epsilon: \
        np.argmax(Q[state]) \
        if np.random.random() > epsilon \
        else np.random.randint(len(Q[state]))

    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio, n_episodes)

    epsilons = decay_schedule(
        init_epsilon, min_epsilon,
        epsilon_decay_ratio, n_episodes)
```



```

for e in tqdm(range(n_episodes), leave=False):

    (8) Каждый новый эпизод начинается со сброса среды и получения
        начального состояния. Мы присваиваем флагу done значение false
        и входим в цикл пошагового взаимодействия
    state, done = env.reset(), False
    while not done:

        (9) Выбираем действие, как в оригинальном
            Q-обучении (только внутри цикла)
        action = select_action(state, Q, epsilons[e])

        (10) Входим в среду и получаем кортеж опыта
        next_state, reward, done, _ = env.step(action)

        (11) Теперь начинаем формировать модели!
            Инкрементируем счетчик переходов
            для триплета state-action-next_state,
            сигнализируя еще об одном полном переходе
        T_count[state][action][next_state] += 1

        (12) Пытаемся вычислить инкрементальное среднее значение сигнала
            вознаграждения. Получаем разность
        r_diff = reward - \
            R_model[state][action][next_state]

        (13) С помощью этой разности и счетчика
            переходов формируем сигнал вознаграждения
        R_model[state][action][next_state] += \
            (r_diff / T_count[state][action][next_state])

        (14) Как обычно, вычисляем цель TD с помощью
            Q-обучения (нестратегично, с использованием max)...
        td_target = reward + gamma * \
            Q[next_state].max() * (not done)

        (15) ...и погрешность TD с помощью цели TD и текущего прогноза
        td_error = td_target - Q[state][action]
        Q[state][action] = Q[state][action] + \
            alphas[e] * td_error

        (16) Наконец, обновляем Q-функцию

        (17) И прежде, чем перейти к этапам планирования,
            мы создаем резервную копию переменной next_state
        backup_next_state = next_state

```

```

for _ in range(n_planning):
    if Q.sum() == 0: break
    visited_states = np.where( \
        np.sum(T_count, axis=(1, 2)) > 0)[0]
    state = np.random.choice(visited_states)
    actions_taken = np.where( \
        np.sum(T_count[state], axis=1) > 0)[0]
    action = np.random.choice(actions_taken)
    probs = T_count[state][action] / \
        T_count[state][action].sum()
    next_state = np.random.choice( \
        np.arange(nS), size=1, p=probs)[0]
    reward = R_model[state][action][next_state]
    td_target = reward + gamma * \
        Q[next_state].max()
    td_error = td_target - Q[state][action]
    Q[state][action] = Q[state][action] + \
        alphas[e] * td_error
    state = backup_next_state
    Q_track[e] = Q
    pi_track.append(np.argmax(Q, axis=1))
    V = np.max(Q, axis=1)
    pi = lambda s: {s:a for s, a in enumerate( \
        np.argmax(Q, axis=1))}[s]
    return Q, V, pi, Q_track, pi_track

```

(18) Вначале нужно убедиться, что Q-функция уже обновлялась, иначе нам было бы нечего планировать

(19) Затем выбираем одно из состояний, посещенных агентом

(20) Выбираем действие, которое было выполнено в этом состоянии

(21) Используем матрицу счетчиков, чтобы вычислить вероятности следующего состояния и того, что идет за ним

(22) Используем модель вознаграждения в качестве награды

(23) И обновляем Q-функцию с помощью этого смоделированного опыта!

(24) В конце каждого шага планирования делаем следующее состояние текущим

(25) Все остальное без изменений



Посчитаем

Модельно-ориентированные методы формируют функции перехода и вознаграждения (нижний переход)

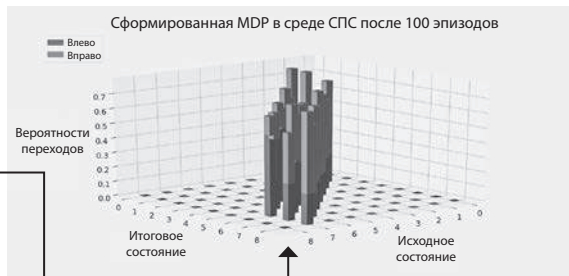
(1) Взгляните на первую диаграмму. Это модель, которую сформировал алгоритм Дупа-Q после первого эпизода. У нее есть очевидные недостатки, но мы завершили лишь один эпизод. Если использовать модель, сформированную на раннем этапе, могут возникнуть проблемы: при выборе данных из некорректной модели будет возникать смещение



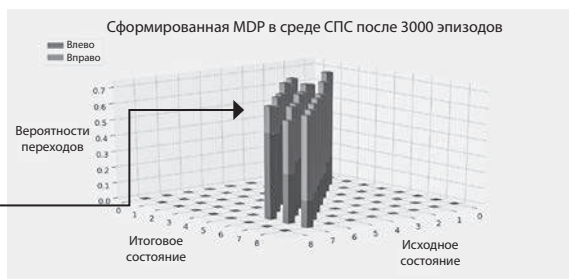
(2) Только после десяти эпизодов модель начинает как следует формироваться. На второй диаграмме вы уже должны видеть, как сходятся вероятности. Ось справа — это исходное состояние, а слева — конечное. Действия обозначены разными оттенками, а высота столбцов соответствует вероятности каждого перехода



(3) После 100 эпизодов вероятности уже довольно близки к реальной модели. Так как это простая среда, агент может быстро накопить опыт для построения MDP



(4) Как видите, вероятности получились достаточно точными и корректно описывают MDP. Вы знаете, что при выборе действия «вправо» в состоянии 7 вы в 50 % случаев окажетесь в состоянии 8, в 30 % в состоянии 7 и еще в 20 % в состоянии 6



Выборка траектории: планирование ближайшего будущего

В Дупа-Q мы формируем модель, как было описано ранее, корректируем функции ценности действий, как в обычном Q-обучении, и выполняем в конце алгоритма несколько итераций планирования. Заметьте, если убрать из кода этапы формирования модели и планирования, получится тот же алгоритм Q-обучения, с которым мы познакомились в предыдущей главе.

На этапе планирования мы выбираем только из уже посещенных пар «состояние — действие», чтобы агент не тратил ресурсы на те, о которых у модели нет никакой информации. Из этих пар мы равномерно случайно выбираем состояние, после чего точно так же (равномерно случайно) выбираем одно из ранее совершенных действий. В конце мы получаем выборку со следующим состоянием и наградой, используя вероятности переходов для данной пары «состояние — действие». Но вам не кажется, что что-то здесь не так? Мы планируем, используя случайное состояние!

Не был бы этот метод более эффективным, выбери мы состояние, которое ожидаем встретить во время текущего эпизода? Подумайте об этом. Что бы вы предпочли: приоритетное планирование ближайших дня, недели, месяца и года или планирование случайного события, которое «может» произойти в вашей жизни? Представьте, что вы разработчик ПО. Что бы вам хотелось планировать больше: чтение книги по программированию и работу над сторонним проектом или свою карьеру в области здравоохранения? Планирование ближайшего будущего — более разумный подход. *Выборка траектории* — это метод модельно-ориентированного RL, который занимается именно этим.



По сути

Выборка траектории

Если Дупа-Q выбирает данные из сформированной равномерно случайно модели MDP, то выборка траектории собирает траектории — то есть переходы и награды, которые можно встретить в ближайшем будущем. Вы планируете свою неделю, а не случайный период в своей жизни. Звучит логично, не правда ли?

Традиционный подход к выборке траектории заключается в выборе данных из всех состояний, от начального до конечного, с использованием стратегической траектории. Иными словами, мы выбираем действия из одной и той же поведенческой политики на заданном временном шаге.

Но не закидывайтесь только на этом подходе — экспериментируйте. Например, моя реализация начинает выбирать данные в текущем состоянии вместо начального и доходит до конечного за заранее определенное количество шагов. Так выборка получается жадной для текущих прогнозов.

Но вы можете попробовать что-то другое. Выборкой траектории можно назвать любой метод, который состоит в выборе из траектории каких-то данных.



Я знаю Python

Агент выборки траектории

```
def trajectory_sampling(env, ← (1) Выборка траектории
                           gamma=1.0,                похожа на Dyna-Q,
                           init_alpha=0.5,            за исключением
                           min_alpha=0.01,            нескольких деталей
                           alpha_decay_ratio=0.5,
                           init_epsilon=1.0,
                           min_epsilon=0.1,
                           epsilon_decay_ratio=0.9,
                           max_trajectory_depth=100,
                           n_episodes=3000):
    ↓
    (2) Вместо n_planning мы используем
    max_trajectory_depth, чтобы
    ограничить длину траектории

    (3) Большая часть алгоритма не отличается от Dyna-Q
    ↓
    nS, nA = env.observation_space.n, env.action_space.n
    pi_track = []

    (4) Q-функция и т. д.
    ↓
    Q = np.zeros((nS, nA), dtype=np.float64)
    Q_track = np.zeros((n_episodes, nS, nA), dtype=np.float64)

    (5) Создаем ту же переменную для моделирования переходной функции...
    ↓
    (6) ...и еще одну — для сигнала вознаграждения
    T_count = np.zeros((nS, nA, nS), dtype=np.int) ←
    R_model = np.zeros((nS, nA, nS), dtype=np.float64)

    (7) Функция select_action, векторы
    с альфой и значениями эпсилона
    остаются без изменений
    ↓
    select_action = lambda state, Q, epsilon: \
        np.argmax(Q[state]) \
        if np.random.random() > epsilon \
        else np.random.randint(len(Q[state]))

    alphas = decay_schedule(
        init_alpha, min_alpha,
        alpha_decay_ratio, n_episodes)

    epsilons = decay_schedule(
        init_epsilon, min_epsilon,
        epsilon_decay_ratio, n_episodes)
```

```

for e in tqdm(range(n_episodes), leave=False):
    (8) И снова каждый новый эпизод начинается со
        сброса среды и получения начального состояния.
        Вновь присваиваем флагу done значение false,
        чтобы войти в цикл пошагового взаимодействия
    state, done = env.reset(), False
    while not done:
(9) Выбираем действие
    action = select_action(state, Q, epsilons[e])
(10) Входим в среду и получаем кортеж опыта
    next_state, reward, done, _ = env.step(action)
    (11) Формируем модель, как в Dyna-Q:
        инкрементируем счетчик переходов
        для триплета state-action-next_state,
        сигнализируя о полном переходе
    T_count[state][action][next_state] += 1
(12) Снова вычисляем инкрементальное среднее значение сигнала
    вознаграждения. Сначала получаем разность
    r_diff = reward - \
        R_model[state][action][next_state]
(13) Затем используем ее и счетчик переходов для формирования
    сигнала вознаграждения
    R_model[state][action][next_state] += \
        (r_diff / T_count[state][action][next_state])
(14) Как обычно, вычисляем цель TD
    td_target = reward + gamma * \
        Q[next_state].max() * (not done)
(15) Погрешность TD на основе цели TD и текущего прогноза
    td_error = td_target - Q[state][action]
    Q[state][action] = Q[state][action] + \
        alphas[e] * td_error
(16) Затем обновляем Q-функцию
(17) И прежде, чем перейти к этапам планирования, мы создаем резервную
    копию переменной next_state
    backup_next_state = next_state
    for _ in range(max_trajectory_depth):
(18) Заметьте, на этот раз в ходе планирования используется
    переменная max_trajectory_depth

```

(19) По-прежнему проверяем, отличается ли чем-то Q-функция, чтобы не проводить лишних вычислений

```
└───> if Q.sum() == 0: break
```

(20) Выбираем действие либо стратегически, либо нестратегически (с помощью жадной политики)

```
└───> # action = select_action(state, Q, epsilons[e])
      action = Q[state].argmax()
```

(21) Если мы не выполнили этот переход, планирование будет беспорядочным, поэтому выходим

```
└───> if not T_count[state][action].sum(): break
```

(22) В противном случае получаем вероятности next_state и выбираем соответствующее действие из модели

```
└───> probs = T_count[state][action] / \
      T_count[state][action].sum()
      next_state = np.random.choice( \
      np.arange(nS), size=1, p=probs)[0]
```

(23) Получаем награду, предписанную моделью «награда — сигнал»

```
└───> reward = R_model[state][action][next_state]
```

(24) Продолжаем обновлять Q-функцию

будто бы с использованием реального опыта

```
└───> td_target = reward + gamma * \
      Q[next_state].max()
      td_error = td_target - Q[state][action]
      Q[state][action] = Q[state][action] + \
      alphas[e] * td_error
```

```
└───> state = next_state
```

(25) Отмечу, что переменная state обновляется прямо перед возвращением в начало цикла и продолжением стратегического планирования

```
state = backup_next_state ← (26) После цикла
                             планирования мы
                             восстанавливаем значение
                             state и возобновляем
                             реальное взаимодействие
```

(27) Все остальное как обычно

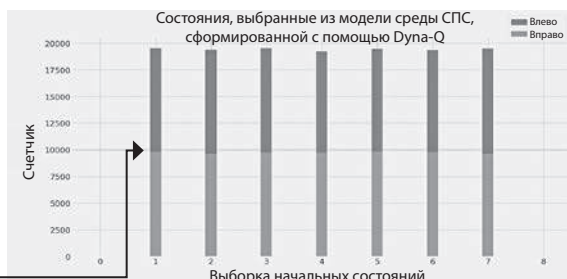
```
└───> Q_track[e] = Q
      pi_track.append(np.argmax(Q, axis=1))
      V = np.max(Q, axis=1)
      pi = lambda s: {s:a for s, a in enumerate( \
      np.argmax(Q, axis=1))}[s]
      return Q, V, pi, Q_track, pi_track
```



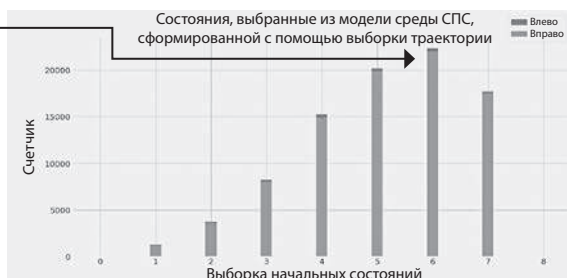
Посчитаем

Дyna-Q и выборка траектории по-разному собирают данные из модели

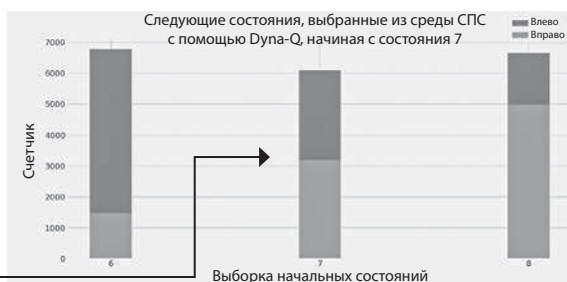
(1) На первой диаграмме представлены состояния, отобранные на этапе планирования Dyna-Q, и действия, выбранные в этих состояниях. Как видите, Dyna-Q выбирает случайно не только состояния, но и действия, которые в них выполняются



(2) Выборка траекторий отбирает данные иначе. Как вы помните, в среде СПС единственное состояние с ненулевой наградой (состояние 8) находится справа в самом конце и дает +1. Жадная политика выборки траектории отбирает данные из модели в попытке оптимизировать жадный выбор действий. Именно поэтому отобранные состояния смещены в сторону цели (состояние 8). То же самое происходит с отобранными действиями. Как видите, на протяжении всего эксперимента действие «вправо» выбирается намного чаще, чем «влево»



(3) Чтобы показать особенности разных стратегий выборки, я создал диаграмму конечных состояний после выбора действия в состоянии 7, слева от цели. Как мы уже видели, Dyna-Q отбирает данные случайно, поэтому полученные вероятности отражают MDP



(4) А вот выборка траекторий попадает в цель намного чаще, поэтому получает от модели больше ненулевых наград





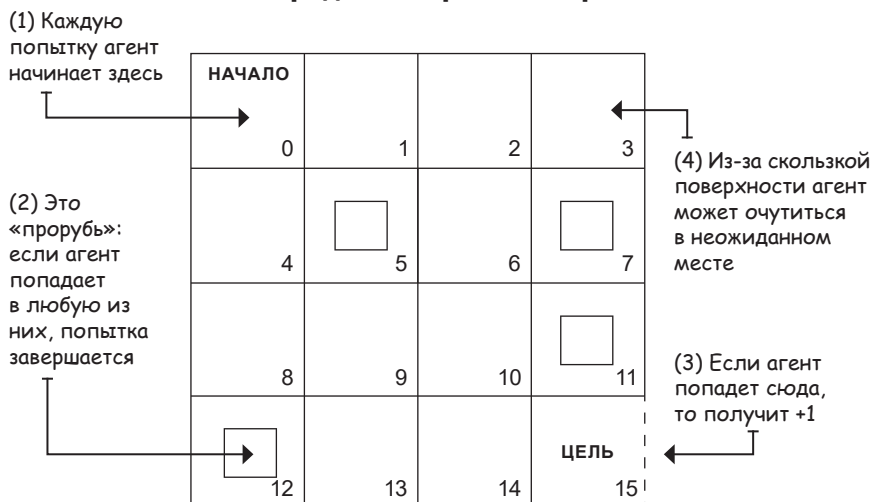
Конкретный пример

Среда «замёрзшее озеро»

В главе 2 мы создали MDP для среды «замёрзшее озеро» (30). Как вы помните, это простая грид-среда, у которой дискретные пространства состояний и действий, с 16 состояниями и четырьмя действиями.

Агент должен добраться до цели из начальной ячейки и не угодить в «прорубь». Конкретно в этой реализации 30 нам нужно пройти из состояния 0 в 15. Но озеро покрыто льдом, что делает его скользким, очень.

Среда «замёрзшее озеро»



Среда 30 — это сетка 4×4 с 16 ячейками, которые представляют состояния 0–15, размещенные слева направо и сверху вниз. Состояние 0 — единственное в начальном распределении состояний. Это значит, что в каждом новом эпизоде агент появляется в НАЧАЛЕ. Состояния 5, 7, 11, 12 и 15 являются конечными: когда агент в них попадает, эпизод завершается. Состояния 5, 7, 11 и 12 — это «проруби», а 15 — ЦЕЛЬ. «Проруби» и цель отличаются только функцией вознаграждения. Переход в ЦЕЛЬ (состояние 15) дает награду +1, тогда как любой другой в этой среде — 0.

Агент, конечно, будет пытаться выполнить этот результативный переход, а для этого ему нужно избегать попадания в «прорубь». Сложность этой среды в том, что у действий здесь стохастические последствия, поэтому агент попадает туда, куда хотел, лишь в каждом третьем случае. Две другие трети распределены поровну по двум ортогональным направлениям. При попытке покинуть грид-среду агент будет возвращен в ту ячейку, из которой он хотел переместиться.



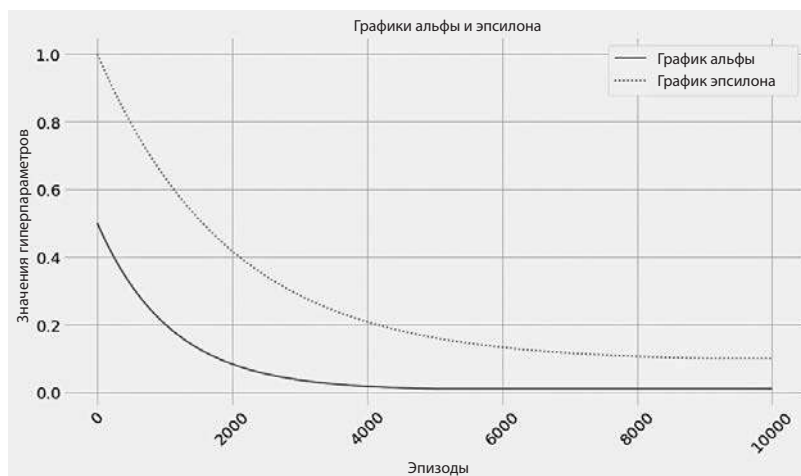
В деталях

Значения гиперпараметров для среды «замёрзшее озеро»

Среда 3O более сложная, чем, скажем, СПС. Поэтому одно из важнейших изменений, которое нам нужно сделать, — это увеличить количество эпизодов взаимодействия агента со средой.

В среде СПС взаимодействие ограничено 3000 эпизодами, а в среде 3O — 10 000. Это простое изменение автоматически корректирует и график затухания как для альфы, так и для эpsilon.

Изменение значения параметра `n_episodes` с 3000 на 10 000 автоматически меняет масштабы исследования и обучения, которые выполняет агент. Альфа теперь затухает с 0,5 до минимального значения 0,01 после завершения 50 % от общего числа эпизодов (5000), а эpsilon — от 1,0 до минимального значения 0,1 после завершения 90 % от общего числа эпизодов (9000).



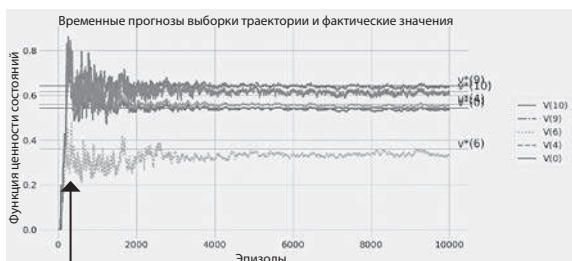
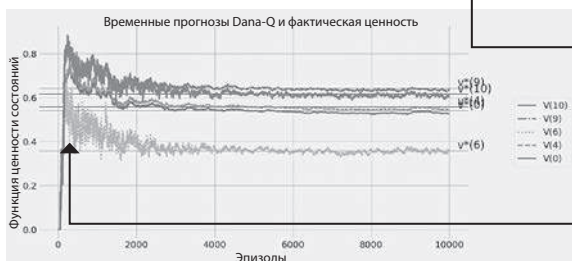
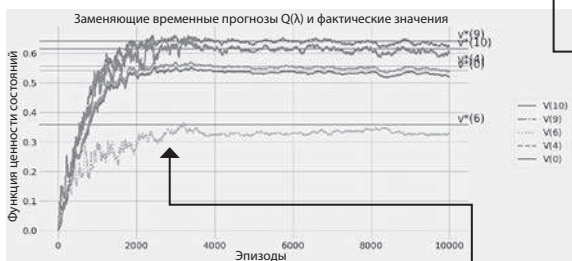
Важно отметить, что я использую гамму 0,99 и что среда 3O, если ее симулировать с помощью OpenAI Gym, автоматически получает временное ограничение благодаря оболочке Gym Wrapper, которая следит за тем, чтобы агент завершал каждый эпизод не более чем за 100 шагов.

Строго говоря, эти два решения (гамма и временное ограничение) влияют на оптимальные политику и функцию ценности, которые сформирует агент. К ним нужно относиться серьезно. Советую вам поэкспериментировать со средой 3O в блокноте Jupyter для главы 7. Попробуйте разные значения гаммы (1, 0,5, 0) и избавьтесь от временного ограничения с помощью атрибута `unwrapped` экземпляра среды (например, `env = env.unwrapped`). Подумайте, как эти два аспекта влияют на итоговые политики и функции ценности, которые находит агент.



Посчитаем

Методы модельно-ориентированного RL приближают значения к фактическим за меньшее количество эпизодов.



(1) Проведите такой любопытный эксперимент: позвольте стандартным агентам SARSA и Q-обучения взаимодействовать с этой средой и сравните результаты. Посмотрите, как агенту SARSA(λ) не удастся спрогнозировать оптимальную функцию ценности состояний. Помните, что в этих

графиках горизонтальными линиями обозначены оптимальные V-функции для нескольких состояний — в данном случае для 0, 4, 6, 9 и 10

(2) Агент Q(λ) нестратегический. В отличие от SARSA(λ) он приближает прогнозы оптимальной V-функции к реальным значениям. Важно понимать, что здесь все сводится к количеству шагов. Уверен, что при достаточном числе эпизодов SARSA(λ) рано или поздно сойдется на реальных значениях

(3) Агент Dyna-Q приближается к реальным значениям даже быстрее, чем Q(λ). Но обратите внимание на стремительное повышение погрешности в начале обучения. Так произошло потому, что сначала модель некорректная, а Dyna-Q случайно отбирает состояния из нее, даже если они посещались нечасто

(4) Я использую жадную реализацию выборки траектории, поэтому агент отбирает состояния, которые будут посещены с высокой вероятностью. Именно благодаря этому мы наблюдаем в TS повышенную стабильность



Посчитаем

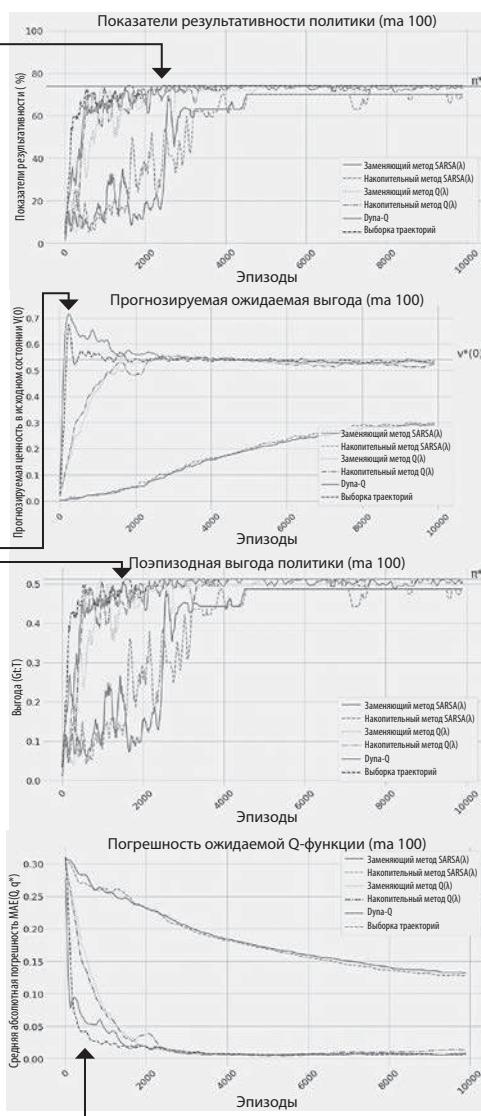
И признаки, и модельно-ориентированные методы эффективно обрабатывают накопленный опыт

(1) Теперь давайте обсудим, как предыдущие итоговые данные связаны с результативностью. Как вы видите на первом графике, все алгоритмы, кроме SARSA(λ), достигают того же уровня результативности, что и оптимальная политика. Кроме того, похоже, что методы модельно-ориентированного RL достигают этих показателей раньше остальных, но ненамного. Напомню, что под «результативностью» здесь подразумевается количество раз, за которые агенту удастся достичь цели (в среде 3O это состояние равно 15)

(2) На втором графике показана прогнозируемая ожидаемая выгода для начального состояния. Обратите внимание на то, как в начале обучения оба модельно-ориентированных метода демонстрируют стремительный рост погрешности. Выборка траекторий стабилизируется немного раньше, чем Dupa-Q, но всплеск все равно велик. Методы Q(λ) приближаются к оптимуму без заметных всплесков и достаточно рано, тогда как SARSA(λ) вообще не достигает желаемых показателей на протяжении всего обучения

(3) На третьем графике показана фактическая выгода, усредненная по 100 эпизодам. Как видите, модельно-ориентированные методы и Q(λ) получают ожидаемую выгоду примерно после 2000 эпизодов. Агенту SARSA(λ) не удается этого сделать на протяжении всего обучения. Но я уверен, что рано или поздно у него это получилось бы

(4) На последнем графике изображена средняя абсолютная погрешность для Q-функции. Как видите, модельно-ориентированные методы быстрее всего приближают погрешность к нулю. Хотя по прошествии 2000 эпизодов они почти не отличаются от методов Q(λ). Методы SARSA(λ) снова медленно приближаются к оптимуму



Конкретный пример

Среда «замерзшее озеро 8 × 8»

Давайте немного поднажмем и протестируем эти алгоритмы в более сложной среде.

Эта среда называется «замерзшее озеро 8 × 8» (3O 8 × 8). Логично, что это сетка размером 8 на 8, которая по своим свойствам похожа на 3O. Состояние 0 — начальное и находится в левом верхнем углу; состояние 63, ЦЕЛЬ, — конечное и находится в правом нижнем углу. Стохастичность последствий действий та же, вероятность перехода в выбранную ячейку — всего 33,33 %, а остальные 66,66 % поровну поделены между ортогональными направлениями.

Среда «замерзшее озеро 8 × 8»

(1) 3O 8 × 8 похоже на обычное «замерзшее озеро», но намного больше по размеру и поэтому сложнее



НАЧАЛО							
0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	ЦЕЛЬ 63

Главное отличие этой среды в том, что в ней намного больше «прорубей», которые, естественно, находятся в других местах: это состояния 19, 29, 35, 41, 42, 46, 49, 52, 54 и 59. В общей сложности десять!

Как и в оригинальной среде 3O, в 3O 8 × 8 правильная политика позволяет агенту достигать конечного состояния в 100 % эпизодов. Но в реализации OpenAI Gym агенты, формирующие оптимальные политики, не находят эти конкретные оптимумы из-за гаммы и временного ограничения. Задумайтесь: учитывая стохастичность этих сред и ограниченное время, осторожная политика может завершить эпизод без награды. А если значение гаммы меньше 1, то с каждым новым шагом награда будет все меньше влиять на выгоду. Значит, осторожные политики — не всегда хороший выбор, поэтому агент не занимается их формированием. Помните, что политика, которую мы ищем, должна не просто достигать цели в 100 % случаев, но и делать это за 100 шагов в среде 3O и за 200 в среде 3O 8 × 8. А для этого нужно идти на риски.



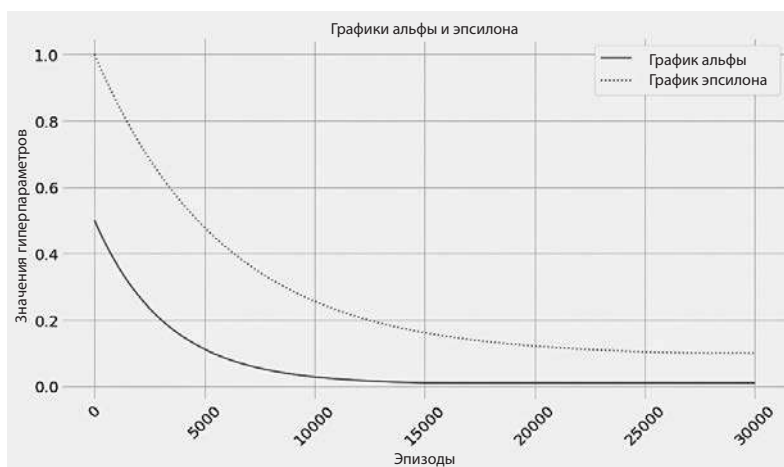
В деталях

Значения гиперпараметров для среды «замёрзшее озеро 8×8 »

«Замёрзшее озеро 8×8 » — самая сложная среда с дискретными пространствами состояний и действий из тех, что мы обсуждали. И на то есть ряд причин. Прежде всего, у $30 \times 8 \times 8$ самое большое количество состояний — 64. Но особую сложность представляет то, что у нее всего одна ненулевая награда.

Это значит, что агент может быть уверен в правильности своих действий только после того, как впервые попадет в терминальное состояние. Помните, что это происходит случайно! Найдя переход с ненулевой наградой, агент (SARSA или Q-обучение, но только обычные версии, без лямбды) обновит ценность только того состояния, из которого он перешел к ЦЕЛИ, находясь за шаг до награды. Угадайте, что ему нужно сделать, чтобы вернуть эту функцию ценности еще на шаг назад? Правильно, он должен случайно попасть в предпоследнее состояние. Но это касается только версий без лямбды. В SARSA(λ) и Q(λ) передача оценок зависит от значения λ . Во всех примерах из этой главы лямбда равна 0,5, что в какой-то степени вынуждает агента передавать значения примерно до середины траектории (в зависимости от типа используемых признаков).

Удивительно, но единственное изменение, которое претерпели эти агенты, заключается в количестве эпизодов, на протяжении которых мы позволяем им взаимодействовать со средой. Если в среде СПС взаимодействие длится 3000 эпизодов, то в среде 30 этот показатель достигает 10 000, а в $30 \times 8 \times 8$ — 30 000. Это значит, что альфа затухает от своего начального значения, 0,5, до минимума, 0,01, по завершении 50 % от общего числа эпизодов, что теперь составляет 15 000. Эпсилон затухает от начального значения, 1,0, до минимума, 0,1, после окончания 90 % эпизодов, что теперь равно 27 000.

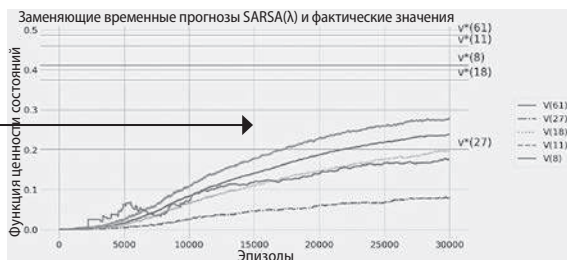




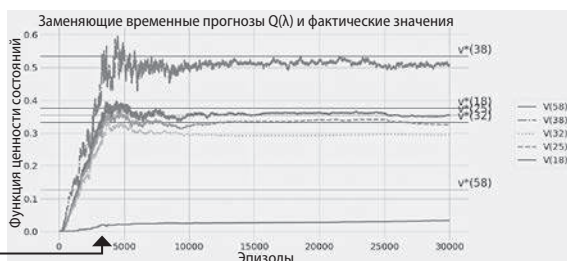
Посчитаем

Стратегические методы замедляются, а нестратегические, модельно-ориентированные методы и признаки держат темп

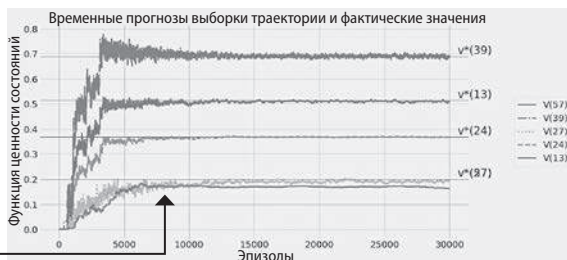
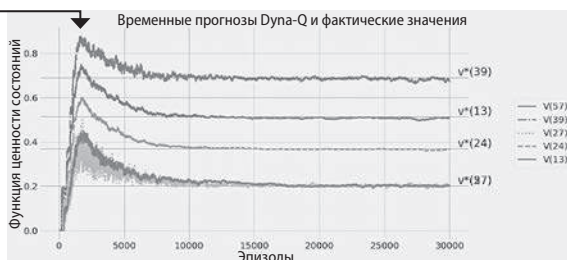
(1) Результаты показывают почти те же тенденции. Наверное, агент SARSA(λ) оказался слишком медленным, чтобы рассматривать его всерьез. Как я уже упоминал, возможная причина в том, что это стратегический алгоритм. Как видите, ни один из прогнозов даже не приближается к оптимальным значениям



(2) А вот прогнозы агента Q(λ) отражают фактические значения. Но есть один подвод: оптимальные значения на этих графиках не учитывают ограничение временных шагов, с которым сталкивается агент в ходе взаимодействия. Это должно повлиять на прогнозы



(3) У агента Dyna-Q есть большое преимущество. Это метод модельно-ориентированного RL, поэтому все шаги взаимодействия, выполняемые перед попаданием в конечное состояние, помогают формировать MDP. Когда агент впервые находит награду, стадия планирования методов модельно-ориентированного RL быстро распространяет значения

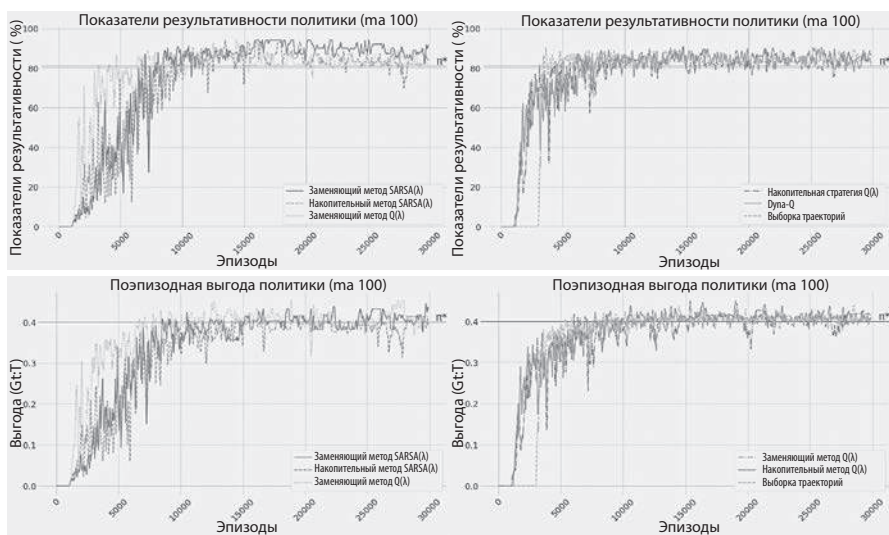


(4) Агент выборки траектории показывает уже знакомую нам тенденцию: прогнозы коррелируют с оптимальными значениями и, что более важно, в начале нет огромного всплеска, вызванного некорректностью модели. У прогнозов, полученных этим методом, намного более стабильная кривая



Посчитаем

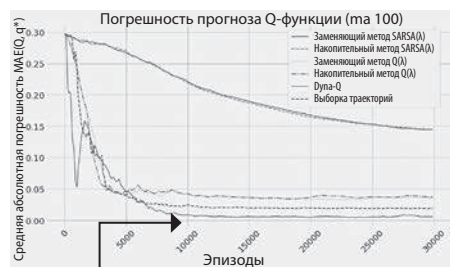
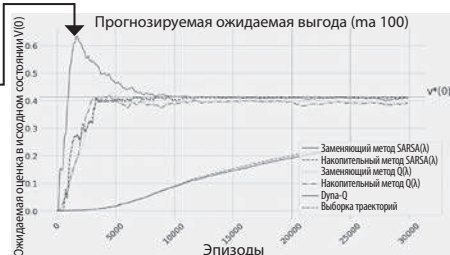
У некоторых модельно-ориентированных методов большие всплески погрешности, которые нужно учитывать



(1) Мне пришлось разделить графики результативности политик и эпизодической выгоды

(2) На графике справа видно, что у прогнозируемой ожидаемой выгоды Dyna-Q большая погрешность, тогда как агенты выборки траектории и Q(λ) ведут себя гораздо стабильнее. Еще здесь можно увидеть, что агенты SARSA(λ) отклоняются слишком сильно

(3) Погрешность прогнозирования Q-функции у всех агентов примерно одинаковая. Хотя вы можете заметить, что у Dyna-Q она самая низкая. Как думаете почему? Помните, моя реализация выборки траектории генерирует только выборки жадных траекторий? Это значит, что по прошествии нескольких эпизодов некоторые состояния не будут обновлены (или посещены). В то же время такие методы, как Dyna-Q, делают выбор равномерно случайно, благодаря чему обновляются многие пары «состояние — действие», даже если они не влияют на производительность политики



Подведем итоги

В этой главе вы узнали, как сделать RL более действенным и эффективным. Под действенностью я подразумеваю то, что представленные здесь агенты способны выполнять задачи с ограниченным числом эпизодов для взаимодействия. Это было бы сложно сделать с помощью других методов, вроде обычной версии SARSA, Q-обучения или даже метода Монте-Карло. Вам бы точно было непросто совладать со средой $30 \times 8 \times 8$ всего за 30 000 эпизодов. Вот что я имел в виду: агентам удастся достигать желаемых результатов.

Мы исследовали более эффективные алгоритмы: эффективные в отношении данных. То есть агенты из этой главы могут извлечь из тех же данных больше пользы, чем если бы они применяли другие методы. SARSA(λ) и Q(λ), к примеру, способны интегрировать награды в прогнозы функции ценности намного быстрее, чем их обычные версии. Откорректировав гиперпараметр λ , вы даже можете назначать коэффициенты доверия всем состояниям, посещенным в ходе эпизода. Значение 1 для λ не всегда оптимальное, но у вас хотя бы будет возможность использовать SARSA(λ) и Q(λ).

Вы узнали о таких методах модельно-ориентированного RL, как Dyna-Q и выборка траектории. Они тоже эффективны, но в другом смысле. Эти методы используют выборки для формирования модели среды. И если в каждой из миллиона выборок ваш агент попадает в состояние s' при выборе действия a в состоянии s , то почему бы не использовать эту информацию для улучшения функций ценности и политик? Продвинутое модельно-ориентированное обучение с подкреплением нередко применяется в средах, где сбор образцов опыта довольно затратен. Например, в областях, где недоступна ускоренная симуляция, таких как робототехника, или где оборудование требует существенных финансовых вложений.

На оставшихся страницах этой книги мы займемся обсуждением нюансов, связанных с аппроксимацией нелинейных функций и обучением с подкреплением. Все знания, которые вы уже получили, вам пригодятся. Разница лишь в том, что вместо использования векторов и матриц для хранения функций ценности и политик мы займемся контролируемым обучением и аппроксимацией функций. Как вы помните, агенты DRL обучаются на одновременно последовательной (а не одинарной), оценочной (а не контролируемой) и выборочной (а не исчерпывающей) обратной связи. Пока мы еще не затрагивали тему «выборочности»; наши агенты всегда могли посещать все состояния или пары «состояние — действие». Начиная со следующей главы, мы сосредоточимся на задачах, где недоступны исчерпывающие выборки.

На этом этапе вы:

- знаете, как разрабатывать агенты RL, которые лучше справляются с достижением поставленных целей;
- умеете разрабатывать агенты RL, которые более эффективно используют выборки данных;
- знаете, что делать с одновременно последовательной и оценочной обратной связью.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

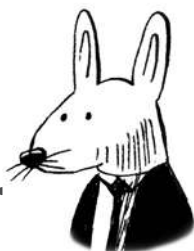
- **#gdrl_ch07_tf01.** В среде «замерзшее озеро 8×8 » тестировались только алгоритмы из этой главы. Если вам интересно, как в ней себя поведут другие алгоритмы, можете сами это проверить! Скопируйте их из блокнота Jupyter для главы 6 в блокнот текущей главы и сравните полученные результаты.
- **#gdrl_ch07_tf02.** Есть много других передовых алгоритмов, рассчитанных на грид-среды. Составьте список наиболее интересных для вас и опубликуйте его.
- **#gdrl_ch07_tf03.** Реализуйте один алгоритм из своего списка и еще один из чужого. Если до вас этот хештег еще никто не использовал, реализуйте сразу два алгоритма из своего списка.
- **#gdrl_ch07_tf04.** Есть базовый алгоритм под названием «*приоритетный обход*». Расскажите о нем подробнее! Не забудьте поделиться своей реализацией: добавьте ее в блокнот для этой главы и сравните с другими представленными здесь алгоритмами.
- **#gdrl_ch07_tf05.** Создайте среду вроде «замерзшего озера 8×8 », но немного сложнее. Например, «замерзшее озеро 16×16 ». Протестируйте в ней все алгоритмы из этой главы и посмотрите, как они себя ведут. Поделитесь с нами результатами.
- **#gdrl_ch07_tf06.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdrl_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы узнаете о базовых проблемах обучения агентов RL с использованием аппроксиматоров нелинейных функций.
- ✓ Создадите агент DRL, который может выполнять разные задачи при обучении с нуля и с минимальным изменением гиперпараметров.
- ✓ Определите плюсы и минусы использования ценностно ориентированных методов для выполнения задач обучения с подкреплением.

Человеческое поведение проистекает из трех основных источников: желаний, эмоций и знаний.

*Платон, древнегреческий философ,
основатель академии в Афинах*

Мы с вами проделали немалый путь, и теперь вы действительно готовы к тому, чтобы прогнуть глубокое обучение с подкреплением. В главе 2 вы научились представлять задачи так, чтобы агенты могли выполнить их с помощью MDP. В главе 3 разработали алгоритмы для выполнения этих задач — то есть агенты, способные вырабатывать оптимальное поведение в задачах, связанных с последовательным принятием решений. В главе 4 вы познакомились с алгоритмами выполнения задач с одношаговыми моделями MDP без доступа к ним. Это неопределенные задачи: агенты не могут обращаться к MDP, поэтому учатся находить оптимальное поведение методом проб и ошибок. В главе 5 мы объединили эти два вида задач (последовательные и неопределенные), чтобы исследовать агенты, которые учатся оценивать политики. Эти агенты занимались не поиском оптимальных политик, а их оценкой и прогнозированием функций ценности, и их результаты были точны. В главе 6 вы узнали об агентах, которые находят оптимальные политики в задачах, связанных с последовательным принятием решений в условиях неопределенности. Такие агенты начинают с произвольной политики и превращают ее в оптимальную, просто взаимодействуя со средой и целенаправленно накапливая опыт для дальнейшего обучения. В главе 7 были представлены агенты, которые тоже искали оптимальные политики, но делали это, извлекая максимум пользы из накопленного опыта.

Глава 2 — это основа для всех остальных глав в этой книге. Глава 3 посвящена алгоритмам планирования, связанным с последовательной обратной связью. В главе 4 основное внимание уделяется «бандитским» алгоритмам для работы с оценочной обратной связью. А в главах 5, 6 и 7 речь идет об алгоритмах RL, рассчитанных на одновременно последовательную и оценочную обратную связь. Такие задачи называют *табличным обучением с подкреплением*. Начиная с этой главы, мы будем детально рассматривать глубокое обучение с подкреплением.

Мы сделаем экскурс в глубокие нейросети и посмотрим, как с их помощью можно выполнять задачи RL. Глубокое обучение с подкреплением позволяет по-разному использовать возможности подобных им аппроксиматоров крайне нелинейных функций. Речь идет о ценностно ориентированных, модельно-ориентированных, безградиентных методах вида «актер-критик», основанных на политиках. Здесь мы подробно рассмотрим самый первый из них.

Виды алгоритмических подходов из этой книги



(1) В следующих трех главах мы сосредоточимся на этом

Тип обратной связи, который используют агенты глубокого обучения с подкреплением

Глубокое обучение с подкреплением подразумевает создание агентов, способных обучаться одновременно на оценочной, последовательной и выборочной обратной связи. Я не устаю это подчеркивать, потому что вам нужно понимать, что это значит.

В главе 1 я упомянул, что глубокое обучение с подкреплением — это решение *сложных задач, связанных с последовательным принятием решений в условиях неопределенности*. Вы тогда, наверное, подумали: «Сколько непонятных слов!» Но, как я и обещал, у каждого слова есть определенное значение. В главе 3 вы узнали, что такое задачи, связанные с последовательным принятием решений, а в главе 4 — задачи в условиях неопределенности. В главах 5, 6 и 7 мы занимались рассмотрением задач последовательного принятия решений в условиях неопределенности. Здесь же мы наконец сделаем эти задачи «сложными». Давайте воспользуемся этим вступительным разделом, чтобы в последний раз обсудить три вида обратной связи, которые использует агент глубокого обучения с подкреплением.



По сути

Виды обратной связи в глубоком обучении с подкреплением

	Последовательная (а не одинарная)	Оценочная (а не контролируемая)	Выборочная (а не исчерпывающая)
Контролируемое обучение	×	×	✓
Планирование (глава 3)	✓	×	×
«Бандиты» (глава 4)	×	✓	×
Табличное обучение с подкреплением (главы 5, 6, 7)	✓	✓	×
Глубокое обучение с подкреплением (главы 8, 9, 10, 11, 12)	✓	✓	✓

Агенты глубокого обучения с подкреплением работают с последовательной обратной связью

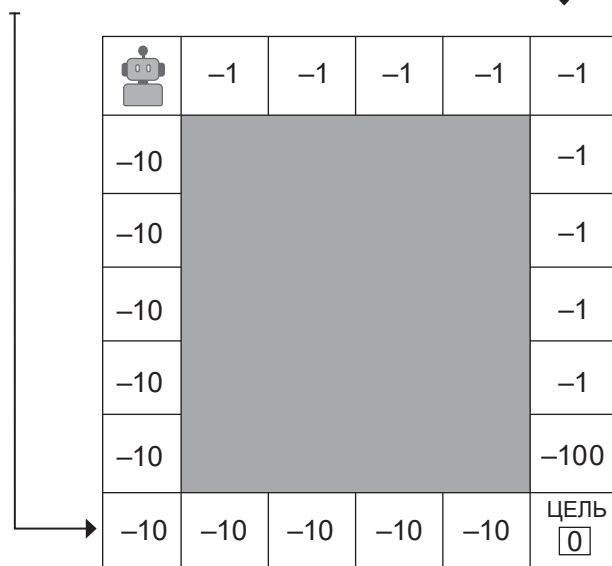
Агентам глубокого обучения с подкреплением приходится иметь дело с последовательной обратной связью. Одна из главных трудностей здесь в том, что информация может поступать с задержкой.

Представьте себе шахматную партию: в начале вы сделали несколько плохих ходов, но их последствия проявляются только к концу игры, когда (и если) вы потерпите поражение.

Интерпретировать источник отложенной обратной связи непросто. Последовательная обратная связь порождает проблему присваивания коэффициентов доверия, состоящую в определении того, какое состояние, действие или их сочетание привело к получению награды. Временной аспект и отложенные последствия действий затрудняют процесс присваивания коэффициентов за полученные награды.

Последовательная обратная связь

(1) Взгляните: в этой среде один маршрут выглядит явно лучше другого даже после нескольких шагов



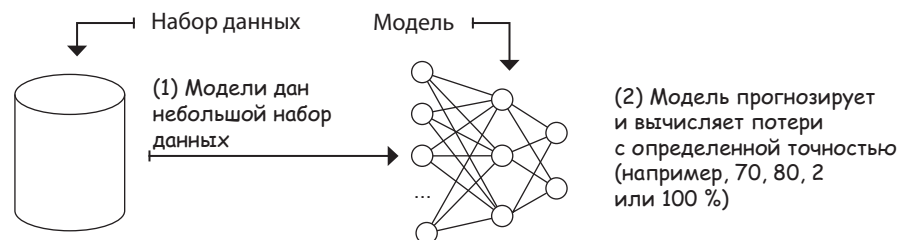
(2) Но прежде, чем завершить этот «лучший на первый взгляд» путь, агент понесет большие потери

(3) В этом вся трудность последовательной обратной связи и одна из причин, почему для выбора действий мы используем функции ценности, а не только награды

Если не последовательная, то какая?

Противоположность отложенной обратной связи — это немедленная. То есть противоположность последовательной — одинарная. В задачах, где она используется (контролируемое обучение или «многорукие бандиты»), у решений нет долгосрочных последствий. Возьмем, к примеру, задачу категоризации: присвоение изображению верной или неверной категории не повлияет на будущую производительность. Скажем, изображения в следующей модели не будут отличаться независимо от того, правильно она классифицировала предыдущую партию или нет. В DRL такая последовательная зависимость есть.

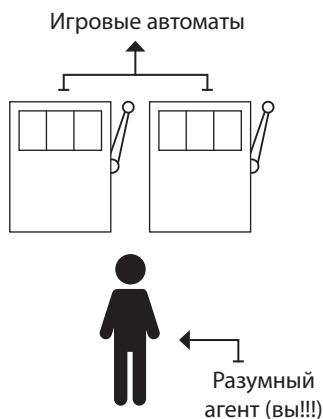
Задача категоризации



(3) Но набор данных не зависит от успешности действий модели. Ей будет предоставлена небольшая случайная выборка вне зависимости от ее производительности. Иначе говоря, долгосрочных последствий нет

В задачах с «бандитами» тоже нет долгосрочных последствий, хотя причина здесь не столь очевидна. «Бандиты» — это одношаговые модели MDP, в которых эпизоды завершаются после выбора первого же действия. Поэтому дальнейшие действия не влияют на производительность агента в эпизоде.

Двурукий бандит



Примечание: предполагается, что вероятность победы в игровых автоматах стационарна. Значит, после того, как вы потянете за рычаг, вероятность выигрыша не изменится. Настоящие автоматы, скорее всего, работают иначе

(1) Когда вы посещаете казино, ваша цель — найти автомат с самой высокой вероятностью выигрыша и попытаться на нем свою удачу

(2) В «бандитских» задачах мы предполагаем, что вероятность выигрыша остается неизменной после каждой игры, что делает задачу разовой

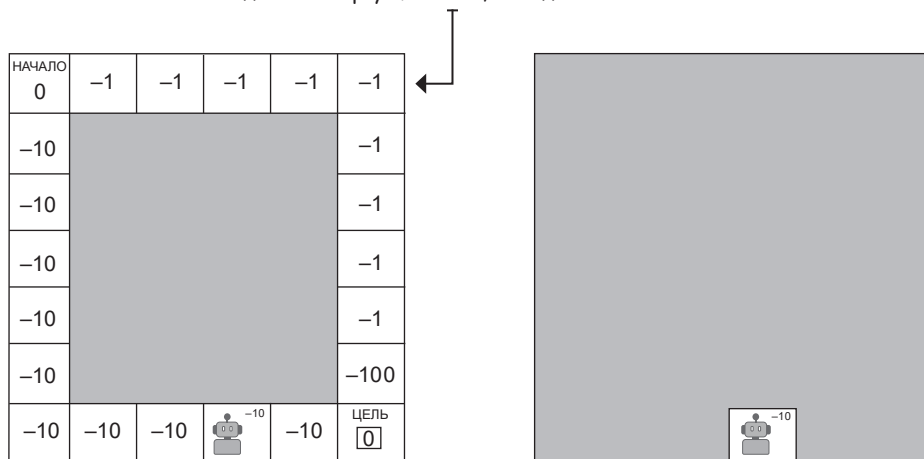
Агенты глубокого обучения с подкреплением работают с оценочной обратной связью

Второе свойство обратной связи, которое мы изучили, — ее оценочность. Оценочная обратная связь используется и в глубоком, и в табличном обучении с подкреплением, равно как и в «бандитских» задачах. Ее отличает то, что из-за неопределенности среды ее положительность относительна. Мы не знаем, как на самом деле устроена среда: у нас нет доступа ни к переходной функции, ни к сигналу вознаграждения.

Поэтому нам приходится исследовать окружение. Но проблема в том, что в процессе мы теряем возможность извлечь пользу из полученных ранее знаний, что повышает вероятность потерь. Так мы сталкиваемся с необходимостью в поиске компромисса между исследованием среды и использованием информации — это неизбежный побочный эффект неопределенности. Без доступа к модели среды мы должны либо собирать новые данные, либо улучшать поведение на основе имеющихся.

Оценочная обратная связь

(1) Чтобы понять проблемы оценочной обратной связи, нужно учитывать, что агенты не видят всю карту целиком, как здесь



(2) Они видят только текущие состояние и награду, как здесь

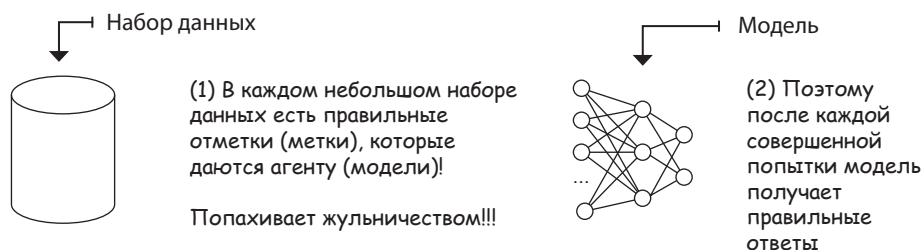
(3) -10 — это плохо или хорошо?

Если не оценочная, то какая?

Противоположность оценочной обратной связи — контролируемая. В задаче классификации ваша модель находится под контролем: то есть в ходе обучения вашей модели предоставляются правильные метки для каждой выборки. Агенту

не нужно гадать. В случае ошибки ваша модель сразу получает правильный ответ. Очень удобно!

Контролируемая категоризация



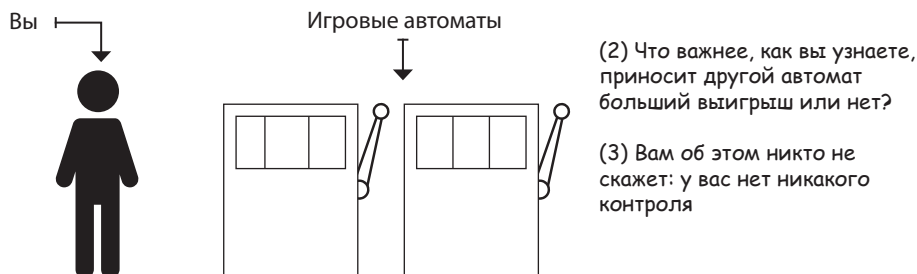
(3) Но мы-то с вами знаем, что жизнь правильных ответов не дает

То, что алгоритму обучения даются верные ответы, делает эту обратную связь намного проще по сравнению с оценочной. Есть четкая разница между задачами контролируемого обучения и задачами на основе оценочной обратной связи, такими как «многорукие бандиты», табличное и глубокое обучение с подкреплением.

«Бандитские» задачи могут обходиться без последовательной обратной связи, но для обучения им нужна оценочная. Это главная проблема, которая решается в «бандитских» задачах. Работая с оценочной обратной связью, агент должен искать баланс между исследованием и эксплуатацией. Если же обратная связь одновременно оценочная и последовательная, то существенных изменений нужно еще больше. Алгоритмы должны уравнивать не только краткосрочные и долгосрочные цели, но еще и сбор и применение данных. В табличном обучении с подкреплением и в DRL агенты обучаются на одновременно последовательной и оценочной обратной связи.

«Бандиты» работают с оценочной обратной связью

(1) Вы тянете за первый рычаг и получаете 10 \$. Это хорошо или плохо? Что, если другой автомат выдал бы вам 50 \$? Что, если следующие 500 попыток будут приносить вам всего по 1 \$?



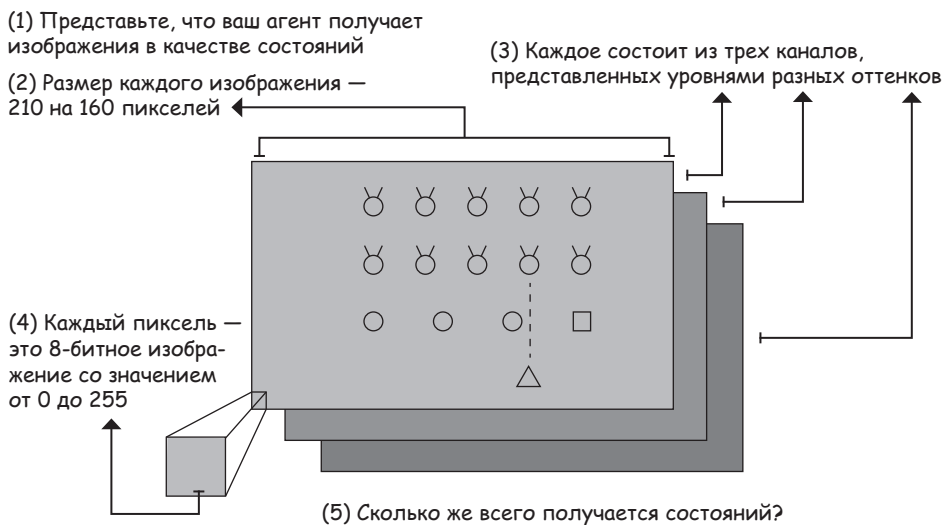
Агенты глубокого обучения с подкреплением работают с выборочной обратной связью

Что отличает глубокое обучение с подкреплением от табличного, так это сложность задач. В DRL агентам крайне редко удастся собрать всю возможную обратную связь. Им приходится делать обобщения на основе собранной и, оттачиваясь от этого, принимать разумные решения.

Подумайте над этим: никто не может получить от жизни исчерпывающую обратную связь. Вы не можете быть одновременно врачом, юристом и инженером, по крайней мере если хотите достичь хоть каких-то успехов в любом из этих направлений. Чтобы принимать удачные решения, определяющие ваше будущее, вы должны использовать накопленный опыт. Это просто. Вам легко давалась математика в средней школе? Отлично! Выбирайте специальность, связанную с математикой. Вы были склонны к творчеству? Тогда выбирайте соответствующее направление. Обобщение позволяет внести ясность в ваш будущий жизненный путь, помогая вам находить закономерности, делать предположения и сопоставлять факты. Так вам будет легче улучшить свою жизнь.

К слову, контролируемое обучение работает с выборочной обратной связью. И в этом его основная задача: научиться обобщать с учетом новых выборок. Этим не занимаются ни «многорукие бандиты», ни задачи табличного обучения с подкреплением.

Выборочная обратная связь



(6) $(255^3)^{210 \times 160} = (16\,581\,375)^{33\,600} = \text{много!}$

(7) Чтобы развлечься, я посчитал это в Python и получил число длиной 242 580 цифр. Для сравнения: обозримая Вселенная содержит от 10^{78} до 10^{82} атомов — это максимум 83-значное число

Если не выборочная, то какая?

Исчерпывающая обратная связь — противоположность выборочной. Чтобы полностью исследовать среду, у агента должен быть доступ ко всем возможным выборкам. Например, агентам табличного обучения с подкреплением и «бандитам» нужно изучить среду только до тех пор, пока накопленной информации не будет достаточно для оптимальной производительности. Сбор исчерпывающей обратной связи — еще одна причина того, почему в табличном обучении с подкреплением гарантируется оптимальная сходимость. В небольших грид-средах с ограниченными пространствами состояний и действий нередко предполагается наличие «бесконечных данных» или возможности посещения каждой пары «состояние — действие» бесконечно часто.

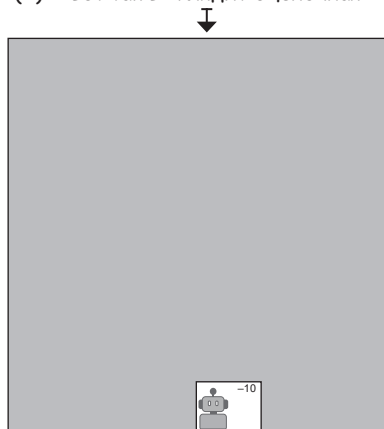
Последовательная, оценочная и исчерпывающая обратная связь

(1) Напоминаю, так выглядит последовательная обратная связь

↓

НАЧАЛО 0	-1	-1	-1	-1	-1
-10					-1
-10					-1
-10					-1
-10					-1
-10					-100
-10	-10	-10	 -10	-10	ЦЕЛЬ 0

(2) А вот так выглядит оценочная



(3) Но если у вас есть дискретное количество состояний и действий, вы можете полностью изучить среду. В небольших пространствах состояний и действий все просто как с теоретической, так и с практической стороны. Но чем больше эти пространства, тем очевиднее становится необходимость в аппроксимации функций

Это плоскость, с которой мы еще не работали. До сих пор в книге рассматривались задачи табличного обучения с подкреплением. Оно было основано на оценочной, последовательной и исчерпывающей обратной связи. Но что, если нам нужно выполнить более сложную задачу, где нельзя рассчитывать на то, что агент сможет полностью исследовать среду? Что, если пространство состояний многомерное, как в случае с игровой доской го, предусматривающей 10^{170} состояний? А что насчет игр Atari с $(255^3)^{210 \times 160}$ состояниями и частотой выборки 60 Гц? Что, если пространство состояний среды содержит непрерывные переменные, определяющие, например, углы сгибания руки робота? Что насчет задач с многомерными и непрерывными состояниями или действиями? Всем этим занимается область глубокого обучения с подкреплением.

Введение в аппроксимацию функций для обучения с подкреплением

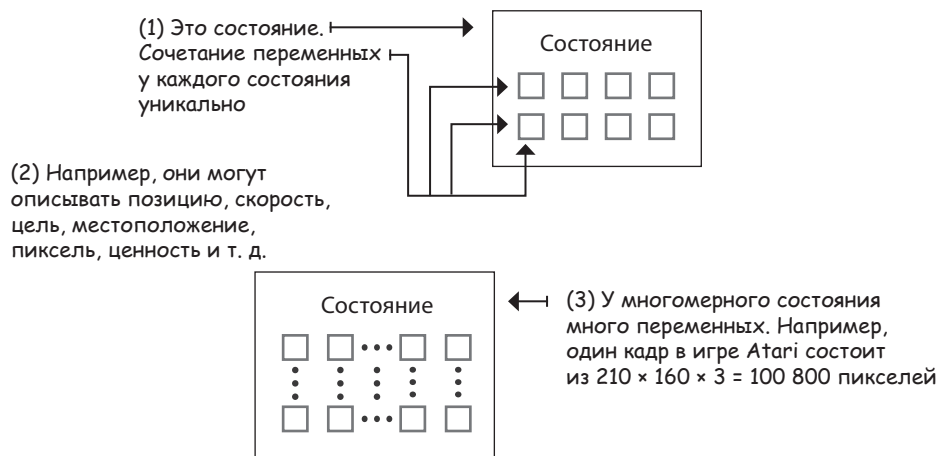
Сначала важно понять роль аппроксимации функций в обучении с подкреплением. Очень легко запутаться в многочисленных терминах и выбрать самое популярное решение. «Глубокое обучение» звучит более захватывающе, чем «аппроксимация нелинейных функций», что, по сути, одно и то же. Это человеческая природа. Такое случается со мной и, уверен, не только. Наша цель — избавиться от всего лишнего и упростить наше мышление.

В этом разделе я обосную применение аппроксимации функций для выполнения задач обучения с подкреплением. Этот подход, наверное, больше относится к функциям ценности, а не к RL в целом, но мотивы, лежащие в основе его использования, применимы ко всем видам DRL.

У задач обучения с подкреплением могут быть многомерные пространства состояний и действий

Основной недостаток табличного обучения с подкреплением в том, что представлять функции ценности в сложных задачах в виде таблиц непрактично. У сред могут быть многомерные пространства состояний, то есть в каждом состоянии может быть огромное количество переменных. Например, игры Atari можно назвать многомерными, учитывая изображение 210 на 160 пикселей и три цветовых канала. Независимо от принимаемых пикселями значений мы подразумеваем под многомерностью количество переменных, составляющих отдельное состояние.

Многомерные пространства состояний



У задач обучения с подкреплением могут быть непрерывные пространства состояний и действий

У среды могут быть непрерывные переменные, способные принимать бесконечное количество значений. Уточню: пространства состояний и действий могут быть многомерными с дискретными переменными, низкоразмерными с непрерывными и т. д.

Даже если переменные не непрерывные (а поэтому не бесконечно большие), они все равно могут принимать настолько большое количество значений, что исследовать их без аппроксимации функций было бы непрактично. Это относится к играм Atari, в которых каждый пиксель принимает 256 значений (от 0 до 255). Пространство состояний ограничено, но из-за его большого размера для обучения нужна аппроксимация функций.

Но иногда даже пространства состояний с низкой размерностью могут быть бесконечно большими. Представьте задачу, где пространство состояний содержит лишь координаты робота: x , y , z (три переменные). Конечно, его размерность будет довольно низкой. Но что, если каждая из этих переменных предоставляется как непрерывная, то есть может обладать бесконечно высокой точностью? Значения могут иметь вид 1,56; 1,5683; 1,5683256 и т. д. Как в таком случае составить таблицу, где учитываются все три значения? Можно, конечно, попытаться дискретизировать пространство состояний, но давайте лучше сэкономим ваше время и перейдем сразу к сути: вам нужна аппроксимация функций.

Непрерывные пространства состояний



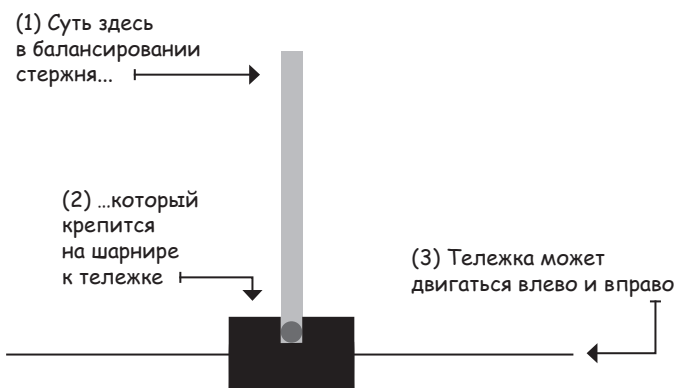


Конкретный пример

Среда «перевернутый маятник»

«Перевернутый маятник» — это классическая среда для обучения с подкреплением. У пространства состояний низкая размерность, но оно непрерывное, что отлично подходит для разработки алгоритмов: обучение проходит быстро, но вызывает кое-какие трудности, преодолеть которые поможет аппроксимация функций.

Это среда «перевернутый маятник»



Состояние этой среды содержит четыре переменные:

- позиция тележки на колее (ось X) в диапазоне от $-2,4$ до $2,4$;
- скорость движения тележки вдоль колеи (ось X) в диапазоне от $-\infty$ до ∞ ;
- угол наклона стержня в диапазоне примерно от -40° до $+40^\circ$;
- скорость движения вершины стержня в диапазоне от $-\infty$ до ∞ .

В каждом состоянии доступно два действия:

- действие 0 применяет к тележке силу -1 (толкает ее влево);
- действие 1 — силу $+1$ (толкает ее вправо).

Конечное состояние достигается, если:

- стержень отклоняется от вертикального положения более чем на 12° ;
- центр тележки отдаляется от центра колеи больше чем на $2,4$ единицы;
- количество шагов в эпизоде достигает 500 (подробнее об этом позже).

Функция вознаграждения равна $+1$ на каждом временном шаге.

Преимущества аппроксимации функций

Уверен, вы понимаете, что, работая в средах с многомерными или непрерывными пространствами состояний, нет никакой причины отказываться от использования аппроксимации функций. Ранее мы обсуждали алгоритмы планирования и обучения с подкреплением, где функции ценности представлены в виде таблиц.



Освежим в памяти

Алгоритмы «итерация ценности» и «Q-обучение» представляют функции ценности в виде таблиц

Итерация ценности — это метод, который берет модель MDP и вырабатывает для нее подходящую политику, вычисляя оптимальную функцию ценности состояний, v^* . Для этого изменяющаяся функция ценности состояний, v , отслеживается на протяжении какого-то количества итераций. Этот метод представляет ожидаемую V-функцию в виде индексированного по состояниям вектора значений, который хранится в поисковой таблице для извлечения и обновления прогнозов.

Функция ценности состояний

(1) Функция ценности

индексруется

по состояниям и возвращает значение, дающее награду, ожидаемую в заданном состоянии

Состояние	0	1	2	3	4	5
V	-3.5	1.4	0.2	1.1	-1.5	3.4

Алгоритм Q-обучения не нуждается в MDP и не использует функцию ценности состояний. Вместо этого он прогнозирует значения оптимальной функции ценности действий, q^* . Она представлена не вектором, а матрицей — двумерной таблицей, проиндексированной по состояниям и действиям.

Функция ценности действий





По сути

Аппроксимация функций может повысить эффективность наших алгоритмов

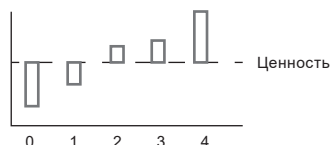
В среде «перевернутый маятник» мы будем использовать обобщение: оно повышает эффективность применения накопленного опыта. С помощью аппроксимации функций агент изучает и использует закономерности, требуя при этом меньше данных (и, возможно, работая быстрее).

Функция ценности состояний с аппроксимацией и без

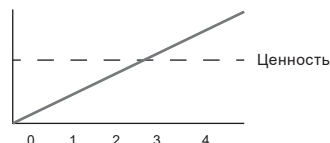
(1) Представьте себе такую функцию ценности состояний

$$V = [-2.5, -1.1, 0.7, 3.2, 7.6]$$

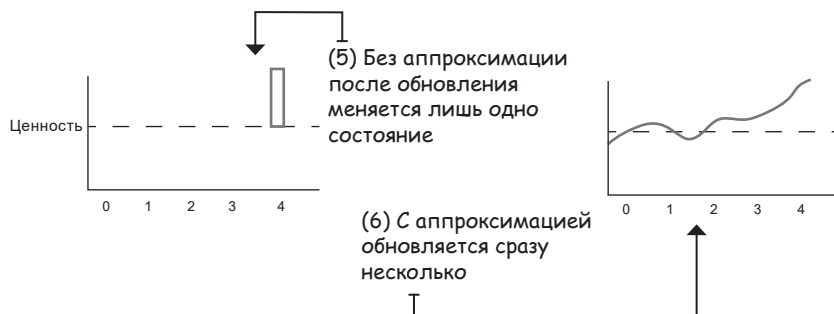
(2) Без аппроксимации все значения независимы



(3) С аппроксимацией мы можем исследовать и использовать внутренние отношения между состояниями



(4) Преимущество аппроксимации функции особенно очевидно, если изобразить все на графиках после первого же обновления



(7) Конечно, это упрощенный пример, но он помогает показать, что происходит. Чем отличались бы «реальные» примеры?

Прежде всего, если мы аппроксимируем функцию ценности действий Q , нам нужно добавить еще одно измерение.

К тому же аппроксиматоры нелинейных функций, такие как нейросети, позволяют находить более сложные связи

Если неспособность выполнять задачи с выборочной обратной связью делает итерацию по значениям и Q -обучение непрактичными, то отсутствие обобщения делает их неэффективными. Если вкратце, то таблицы можно было бы как-то адаптировать к средам с непрерывными пространствами состояний, но за это пришлось бы дорого заплатить. Дискретизация оценок, к примеру, позволила бы использовать таблицы для хранения функций ценности. Но даже здесь мы бы потеряли все преимущества обобщения.

Например, в среде «перевернутый маятник» аппроксимация функций могла бы нашим агентам заметить закономерность, связанную с расстоянием по оси X . Агенты узнали бы, что находиться на расстоянии 2,35 единицы от центра немного опасней, чем на расстоянии 2,2. Мы знаем, что диапазон по оси X ограничен значением 2,4. Это еще одна причина использовать обобщение, и ее нельзя недооценивать. У функций ценности часто есть внутренние связи, которые агент может исследовать и использовать. Аппроксиматоры функций, такие как нейросети, способны эти связи обнаруживать.



По сути

Причины использования аппроксимации функций

Аппроксимация функций позволяет выполнять задачи, которые нельзя было бы выполнить по-другому. При этом решения получаются более эффективными.

NFQ: первая попытка реализовать ценностно ориентированное глубокое обучение с подкреплением

Итерация нейронной аппроксимации Q (neural fitted Q , NFQ) — это, наверное, один из первых алгоритмов, который успешно применяет аппроксимацию функций на основе нейросетей для выполнения задач обучения с подкреплением.

Остаток этой главы мы посвятим обсуждению составляющих большинства алгоритмов ценностно ориентированного глубокого обучения с подкреплением. Воспринимайте это как возможность самостоятельно выбирать те или иные части алгоритма. Например, при рассмотрении функции потерь вместе с NFQ я предлагаю несколько альтернатив. Я не всегда выбираю компоненты, которые использовались в оригинальной версии. Кроме того, при обсуждении методов оптимизации, будь то корень из среднего квадратов градиентов (RMSprop) или адаптивная оценка инерции (Adam), я объясняю, чем продиктован мой выбор, и даю контекст, чтобы вы могли сами выбирать то, что вам лучше подходит.

Надеюсь, вы заметили, что моя цель — не только рассказать вам об этом конкретном алгоритме, но и показать разные среды, где вы можете опробовать разные подходы. Многие алгоритмы RL работают по принципу «подключи и работай», обратите на это внимание.

Первый этап принятия решений: выбор функции ценности для аппроксимации

Аппроксимацию функций ценности с помощью нейросетей можно реализовать по-разному. Начнем с того, что есть много разных функций ценности, которые мы могли бы аппроксимировать.



Освежим в памяти

Функции ценности

Мы знаем следующие функции ценности:

- функция ценности состояний $v(s)$;
- функция ценности действий $q(s, a)$;
- функция преимущества действий $a(s, a)$.

Как вы помните, функция ценности состояний $v(s)$ хотя и полезна для многих целей, сама по себе недостаточна для решения задачи управления. Наличие $v(s)$ позволяет узнать, какую ожидаемую общую дисконтированную награду можно получить в состоянии s и с последующим использованием политики π . Но для определения дальнейшего действия с помощью V -функции нужен доступ к MDP среды: так вы сможете заглядывать на шаг вперед и учитывать все потенциальные следующие состояния после выбора каждого действия.

Еще вы, скорее всего, помните, что функция ценности действий $q(s, a)$ позволяет выполнять задачи управления, поэтому она больше подходит для нашей среды «перевернутый маятник», где мы хотим узнать ценность всех действий во всех состояниях, чтобы балансировать стержнем за счет управления тележкой. Если бы у нас были значения пар «состояние — действие», мы могли бы определить действия, которые приводят либо к получению новой информации (если они исследовательские), либо к максимизации ожидаемой выгоды (если они жадные).

Важно: мы хотим спрогнозировать не какую-нибудь, а оптимальную функцию ценности действий. Но, как мы уже знаем благодаря принципу обобщенной итерации политик, обучение может быть стратегическим и нестратегическим. В первом случае мы используем эпсилон-жадную политику и прогнозируем ее значения напрямую, а во втором — жадную по отношению к текущим прогнозам, которая затем становится оптимальной.

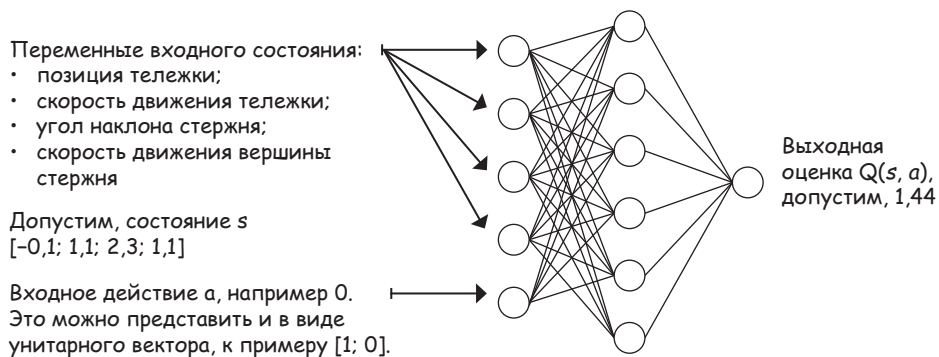
Напоследок напомним, что мы познакомились с функцией преимущества действий $a(s, a)$, которая помогает различать ценность разных действий и наглядно показывает, насколько лучше наш выбор по сравнению со средним действием.

О том, как использовать функции $v(s)$ и $a(s)$, мы поговорим через несколько глав. А пока сосредоточимся на прогнозировании функции ценности действий $q(s, a)$, как в Q -обучении. Аппроксимация ожидаемой Q -функции обозначается как $Q(s, a; \theta)$: это значит, что прогнозы Q параметризованы с помощью θ (весов нейросети), s (состояния) и a (действия).

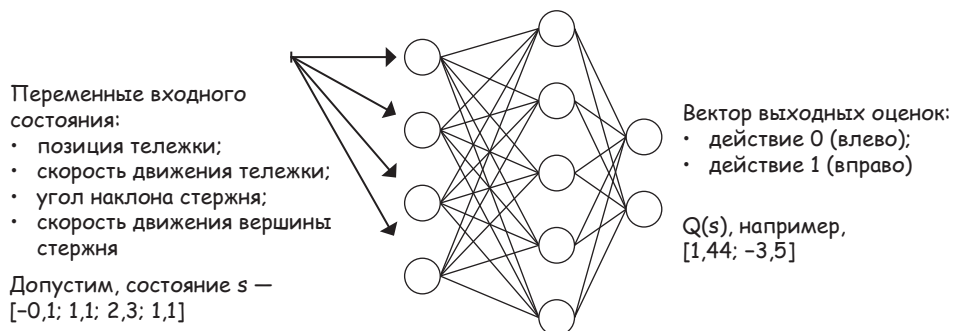
Второй этап принятия решений: выбор архитектуры нейросетей

Мы решили сосредоточиться на функции ценности действий $Q(s, a; \theta)$. Выше я упомянул, что ее нужно параметризовать с помощью θ , s и a , но делать это не обязательно. Следующий компонент, который мы обсудим, — архитектура нейросетей.

Архитектура «состояние — действие на входе, значения на выходе»



Архитектура «состояние на входе, значения на выходе»



В ходе реализации агента Q -обучения вы могли заметить, что матрица, хранящая функцию ценности действий, индексировалась по парам «состояние — действие». Нейросеть с простой архитектурой принимает на вход состояние (в среде «перевернутый маятник» это четыре переменные) и действие, которые нужно оценить, и выдает на выходе один узел, представляющий Q -значение этой пары «состояние — действие».

В среде «перевернутый маятник» эта архитектура работала бы хорошо. Но есть более эффективный подход: на вход нейросети подается только состояние (в нашем случае четыре переменные), а на выходе получаются Q -значения всех действий в этом состоянии (в среде «перевернутый маятник» их два). Этот метод явно лучше при использовании стратегий исследования, включая эпсилон-жадную и softmax, так как для получения оценок всех действий в любом заданном состоянии достаточно сделать один проход вперед.

Это высокопроизводительная реализация, которая особенно хорошо себя проявляет в средах с большим количеством действий.

В нашей реализации NFQ мы используем архитектуру «состояние на входе, значения на выходе» — в среде «перевернутый маятник — это четыре входных и два выходных узла».



Я знаю Python

Полносвязная Q-функция (состояние на входе, значения на выходе)

```
class FCQ(nn.Module):
    def __init__(self,
                  input_dim,
                  output_dim,
                  hidden_dims=(32, 32),
                  activation_fc=F.relu):
        (1) Здесь мы просто определяем
        входной слой: мы принимаем
        input_dim и возвращаем первый
        элемент вектора hidden_dims
        super(FCQ, self).__init__()
        self.activation_fc = activation_fc
        self.input_layer = nn.Linear(input_dim,
                                     hidden_dims[0])
        self.hidden_layers = nn.ModuleList()
        (2) Создаем скрытые слои. Заметьте, насколько гибок этот класс: он позволяет
        изменять число слоев и модулей в каждом из них. Если передать переменной
        hidden_dims другой кортеж, скажем (64, 32, 16), этот класс создаст сеть с тремя
        скрытыми слоями, состоящими из 64, 32 и 16 модулей соответственно
        for i in range(len(hidden_dims)-1):
            hidden_layer = nn.Linear(
                hidden_dims[i], hidden_dims[i+1])
            self.hidden_layers.append(hidden_layer)
        self.output_layer = nn.Linear(
            hidden_dims[-1], output_dim)
        (3) Теперь соединяем
        последний скрытый
        слой с выходным
        def forward(self, state):
            (4) В функции forward мы сначала
            принимаем необработанное состояние
            и превращаем его в тензор
            x = state
            if not isinstance(x, torch.Tensor):
                x = torch.tensor(x,
                                device=self.device,
                                dtype=torch.float32)
            (5) Пропускаем
            его сначала
            через входной
            слой, а затем
            через функцию
            активации
            x = x.unsqueeze(0)
            x = self.activation_fc(self.input_layer(x))
            (6) Повторяем
            это для всех
            скрытых слоев
            for hidden_layer in self.hidden_layers:
                x = self.activation_fc(hidden_layer(x))
            x = self.output_layer(x)
            (7) Наконец, доходим до выходного слоя.
            Заметьте, мы возвращаем вывод напрямую,
            без применения к нему функции активации
            return x
```

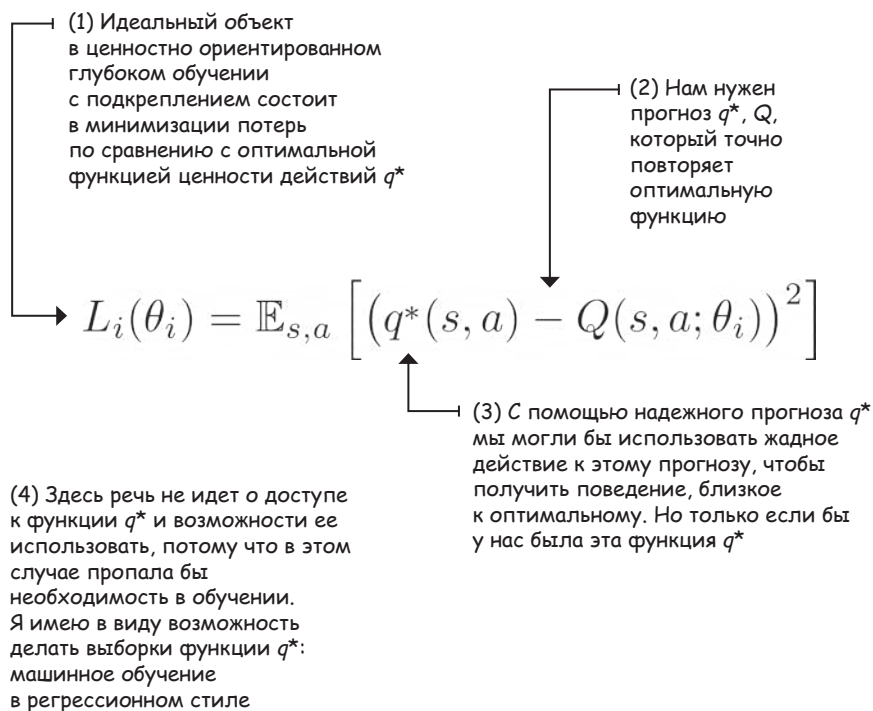
Третий этап принятия решений: выбор функции для оптимизации

Представьте на секунду, что среда «перевернутый маятник» относится к задачам контролируемого обучения. Допустим, у вас есть набор данных с состояниями в качестве ввода и функцией ценности в качестве меток. Какую ценностную функцию вы бы для этого выбрали?



Я знаю математику

Идеальный объект



Конечно, в качестве меток для формирования оптимальной функции ценности действий нам бы хотелось иметь соответствующие оптимальные Q -значения (обратите внимание на то, что строчное q означает реальные значения, а заглавное — прогнозы) для входной пары «состояние — действие». Именно это нам и дает оптимальная функция ценности действий $q^*(s, a)$.

Если бы у нас был доступ, мы бы ею воспользовались, но если у нас есть доступ к ее выборке, мы можем минимизировать потери ее аппроксимированной версии, и на этом все.

Но оптимальная функция ценности действий — это не то, что нас интересует.



Освежим в памяти

Оптимальная функция ценности действий

(1) Напомню, определение оптимальной функции ценности действий выглядит так

$$q^*(s, a) = \max_{\pi} \mathbb{E}_{\pi} [G_t | S_t = s, A_t = a], \forall s \in S, \forall a \in A(s)$$

(2) Это говорит нам о том, что она представляет собой...

(3) ...политику, которая дает...

(4) ...максимальную ожидаемую выгоду...

(5) ...для каждого действия и в каждом состоянии

Но это всего лишь недостижимая мечта. Почему? Что ж, начнем с того, что у нас нет оптимальной функции ценности действий $q^*(s, a)$. К тому же мы даже не можем получить ее выборку, так как у нас нет и оптимальной политики.

К счастью, для поиска хороших политик можно использовать принципы обобщенной итерации политик, согласно которым процессы оценки и улучшения должны чередоваться. Но гарантий сходимости в этом случае нет, так как мы аппроксимируем нелинейную функцию. Это Дикий Запад в мире глубокого обучения.

В нашей реализации NFQ мы применяем именно этот подход. Вначале берется произвольно инициализированная функция ценности действий (неявная стратегия). Затем проводится оценка политики путем выбора из нее действий (см. главу 5), после чего она улучшается с помощью стратегии исследования (например, эpsilon-жадной), как в главе 4. Процесс повторяется, пока не будет достигнута нужная производительность (см. главы 6 и 7).



По сути

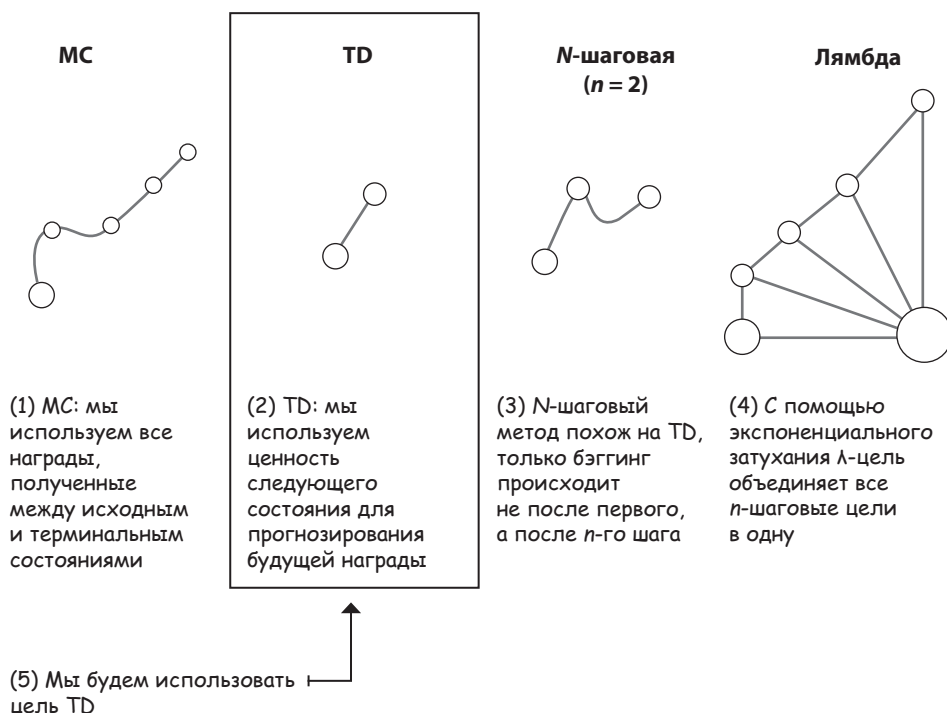
Мы не можем использовать идеальный объект

Мы не можем использовать идеальный объект: у нас нет доступа к оптимальной функции ценности действий или даже к оптимальной политике, из которой можно делать выборки. Вместо этого мы должны чередовать процессы оценки политики (выбирая из нее действия) и ее оптимизации (используя стратегию исследования — например, эpsilon-жадную). Это принцип обобщенной итерации политик из главы 6.

Четвертый этап принятия решений: выбор целей для оценки политики

Политику можно оценивать по-разному. Например, можно использовать разные *цели* для прогнозирования Q -функции политики π . Основные из тех, которые вы уже изучили, — цель Монте-Карло (MC), цель временных разностей (TD), n -шаговая цель и λ -цель.

Цели MC, TD, n -шаговые и λ



Использование любой из этих целей принесет положительный результат, но на этот раз мы не станем ничего усложнять и воспользуемся целью TD в нашей реализации.

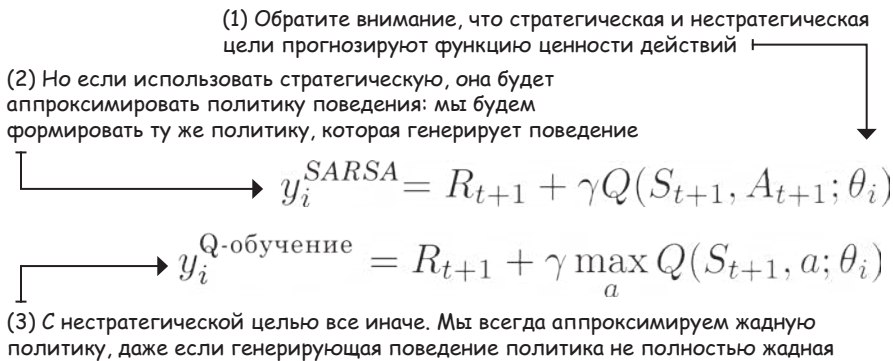
Напомним, что цели TD могут быть стратегическими и нестратегическими, в зависимости от того, как происходит бэггинг. Есть два основных способа выполнения бэггинга для целей TD: мы можем использовать либо функцию ценности действия, которое агент выберет в следующем состоянии, либо оценку действия в следующем состоянии с лучшим прогнозом.

В научной литературе стратегическую версию этой цели нередко называют целью SARSA, а нестратегическую — целью Q -обучения.



Я знаю математику

Стратегические и нестратегические цели TD

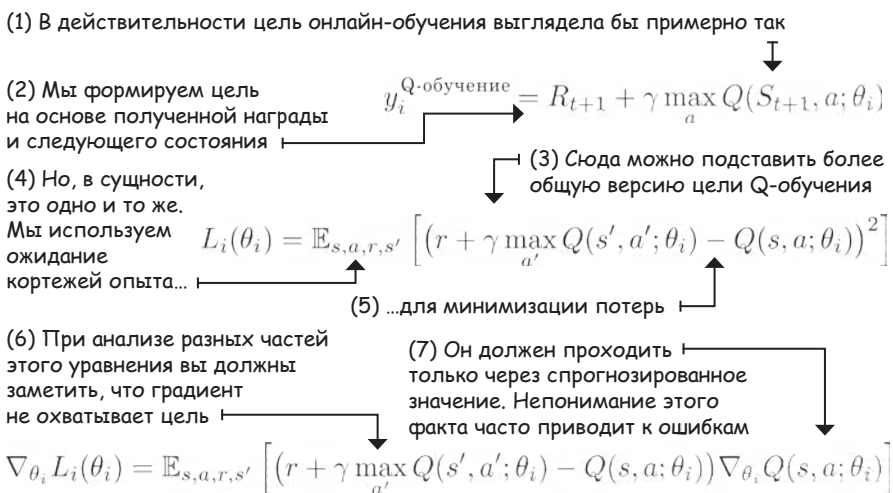


В нашей реализации NFQ будет использоваться та же нестратегическая цель TD, которую мы применяли в алгоритме Q-обучения. На этом этапе для получения целевой функции нужно вычесть цель Q-обучения из оптимальной функции ценности действий $q^*(s, a)$, которую мы представили ранее в качестве уравнения идеального объекта.



Я знаю математику

Цель Q-обучения, нестратегическая цель TD





Я знаю Python

Цель Q-обучения

```

q_sp = self.online_model(next_states).detach()
max_a_q_sp = q_sp.max(1)[0].unsqueeze(1)
max_a_q_sp *= (1 - is_terminals)
target_q_s = rewards + self.gamma * max_a_q_sp
q_sa = self.online_model(states).gather(1, actions)

```

(1) Переменная `next_states` хранит набор значений `next_state`

(2) Метод `detach` играет здесь важную роль. Мы не должны пропускать значения через эту модель. Мы просто вычисляем цели

(3) Получаем максимальное значение следующего состояния, `max_a`

(4) Метод `unsqueeze` добавляет в вектор еще одну размерность, чтобы последующие операции работали с нужными нам элементами

(5) Конечные состояния обнуляются. Это важный шаг, о котором часто забывают

(6) Отмечу, что `is_terminals` — это набор флагов `is_terminal`, которые всего лишь определяют, конечное ли состояние `next_state`

(7) Теперь вычисляем цель

(8) Получаем текущий прогноз $Q(s, a)$. Теперь мы готовы создать нашу функцию потерь

Обратите внимание на две проблемы, которые часто встречаются в DRL-реализациях алгоритмов на основе целей TD.

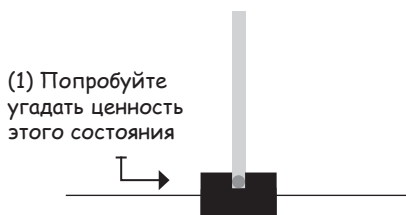
Во-первых, важно убедиться, что обратному распространению подлежат только спрогнозированные значения. Позвольте мне объяснить. Вы уже знаете, что в контролируемом обучении есть предсказанные значения, которые мы получаем из формируемой нами модели, и реальные, в виде предоставленных заранее констант. В RL реальные значения зачастую зависят от предсказанных: они берутся из модели.

Например, при формировании цели TD используется награда, которая является константой, и дисконтированное значение следующего состояния, предоставленное моделью. Но заметьте, что это значение тоже не фактическое, что порождает самые разные проблемы, которые позднее нам придется решать. Еще обратите внимание, что спрогнозированное значение берется из нейросети. Его нужно превратить в константу. В PyTorch для этого достаточно вызвать метод `detach`. Вернитесь к двум прошлым примечаниям и попробуйте как

следует разобраться в этих аспектах. Они очень важны для реализации надежных алгоритмов DRL.

Во-вторых, важно отметить еще одну проблему, связанную с тем, как среды на основе OpenAI Gym обращаются с конечными состояниями. Функция `step`, которую OpenAI Gym использует для взаимодействия со средой, возвращает после каждого шага удобный флаг, определяющий, попал агент в конечное состояние или нет. С помощью этого флага агент может принудительно свести ценность конечных состояний к нулю. Как вы помните, это делается для предотвращения отклонения функций ценности. Мы ведь знаем, что ценность жизни после смерти равна нулю.

Какова ценность этого состояния?



(1) Попробуйте угадать ценность этого состояния

(2) Подсказка: оно выглядит довольно неплохо! Стержень находится в строго вертикальном положении, и среда, похоже, уравновешена. Возможно, тележку лучше толкнуть вправо, но следующее за этим состояние не выглядит особенно важным. Ценность двух действий не сильно отличается

Сложность в том, что некоторые среды OpenAI Gym, такие как «перевернутый маятник», содержат оберточный код, который принудительно завершает эпизод после какого-то количества временных шагов. В среде `CartPole-v0` этот лимит равен 200 временным шагам, а в `CartPole-v1` — 500. Оберточный код не позволяет агентам задерживаться в одном эпизоде слишком долго. Это может быть полезно, но из-за него у нас могут возникнуть проблемы. Подумайте: какой будет ценность строго вертикального положения стержня на временном шаге 500? Ведь если он не наклонен и мы получаем +1 за каждый шаг, то фактическая ценность этого состояния бесконечна. Но поскольку после 500 временных шагов агент завершает эпизод и получает флаг конечного состояния, то, если быть невнимательным, в ходе бэггинга получится ноль. Серьезность такой ошибки сложно переоценить. Есть два популярных способа, которыми можно с ней справиться. В процессе бэггинга вместо нуля можно выбрать значение

следующего состояния, которое спрогнозировала нейросеть, но для этого должно быть выполнено одно из двух условий: (1) достижение максимального количества временных шагов в среде или (2) наличие ключа `TimeLimit.truncated` в словаре `info`. Я покажу второй вариант.



Я знаю Python

Правильная обработка терминальных состояний

```

new_state, reward, is_terminal, info = env.step(action)
is_truncated = 'TimeLimit.truncated' in info and \
               info['TimeLimit.truncated']
is_failure = is_terminal and not is_truncated
experience = (state, action, reward, new_state, float(is_failure))

```

(1) Собираем кортеж опыта как обычно

(2) Затем ищем ключ `TimeLimit.truncated`

(3) Определение отрицательного исхода выглядит так

(4) Если эпизод завершился неудачно, добавляем флаг `is_terminal`. В противном случае выполняем бэггинг с оценкой `new_state`

Пятый этап принятия решений: выбор стратегии исследования

Нам нужно решить, какой шаг оптимизации будет использоваться в ходе обобщенной итерации политик. Мы уже обсуждали это в главах 6 и 7: метод оценки политики, такой как MC или TD, нужно чередовать с методом ее улучшения (например, затухающим эpsilon-жадным), относящимся к исследованию.

В главе 4 мы рассмотрели много разных способов нахождения баланса между этапами исследования среды и использования данных. Сейчас нам подошел бы почти любой из них. Но, чтобы не усложнять, мы будем использовать эpsilon-жадную стратегию.

Обратите внимание, что здесь мы обучаем нестратегический алгоритм. А значит, у нас есть две политики: одна генерирует поведение (здесь это эpsilon-жадная), а другую, жадную (которая станет оптимальной), мы формируем.

У нестратегических алгоритмов обучения из главы 6 есть одна интересная особенность: для определения политики можно использовать любое поведение. Главное, чтобы оно позволило в достаточной степени исследовать все пары «состояние — действие». В нашей реализации NFQ я использую эpsilon-жадную стратегию, которая во время обучения в 50 % случаев выбирает действия случайно. Но для оценки агента я использую действие, жадное для сформированной функции ценности действий.



Я знаю Python

Эпсилон-жадная стратегия исследования

```
class EGreedyStrategy():
    <...>
    def select_action(self, model, state):
        with torch.no_grad():
            q_values = model(state).cpu().detach()
            q_values = q_values.data.numpy().squeeze()
            if np.random.rand() > self.epsilon:
                action = np.argmax(q_values)
            else:
                action = np.random.randint(len(q_values))
        <...>
        return action
```

(1) Функция `select_action` эpsilon-жадной стратегии начинается с извлечения Q-значений для состояния `s`

(2) Я делаю эти значения совместимыми с NumPy и убираю дополнительную размерность

(3) Затем берем случайное число и, если оно больше эpsilon, используем жадное поведение

(4) В противном случае выбираем случайное действие

(5) Примечание: я всегда обращаюсь к модели, чтобы посчитать статистические показатели. Но если вас заботит производительность, этого делать не стоит

Шестой этап принятия решений: выбор функции потерь

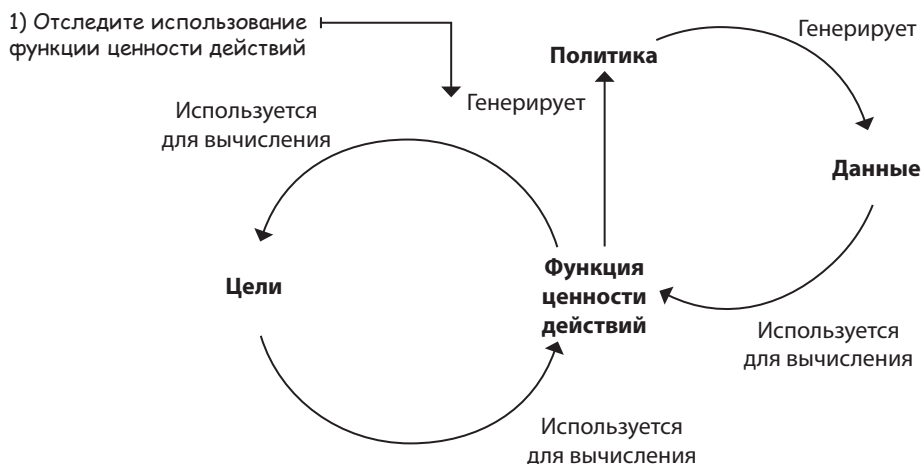
Функция потерь определяет, насколько хорошие прогнозы выдает наша нейросеть. В контролируемом обучении ее легче интерпретировать: с помощью набора прогнозов и соответствующих фактических значений функция потерь вычисляет отклонение, определяющее, насколько хорошо сеть справилась с этим набором.

Это отклонение можно вычислять по-разному, но здесь я отдаю предпочтение простым методам и использую один из самых распространенных: MSE

(mean squared error — среднеквадратичная ошибка, или потери L2). Но снова напомним, что одна из сложностей обучения с подкреплением, по сравнению с контролируемым, в том, что наши реальные значения основаны на прогнозах, которые берутся из сети.

MSE (или потери L2) — это квадрат средней разности между спрогнозированными и реальными значениями. В нашем случае мы прогнозируем значения Q-функции, которые берутся прямо из нейросети, — здесь все сходится. Но в качестве реальных значений используются... верно, цели TD (значения следующих состояний), зависящие от прогнозов, которые дает та же сеть.

Циклическая зависимость функции ценности действий



Как вы уже догадались, эта циклическая зависимость нежелательна: она игнорирует несколько предположений, на которые опираются задачи контролируемого обучения. Эти предположения будут рассмотрены в этой главе, а проблемы, возникающие в случае их несоблюдения, — в следующей.

Седьмой этап принятия решений: выбор метода оптимизации

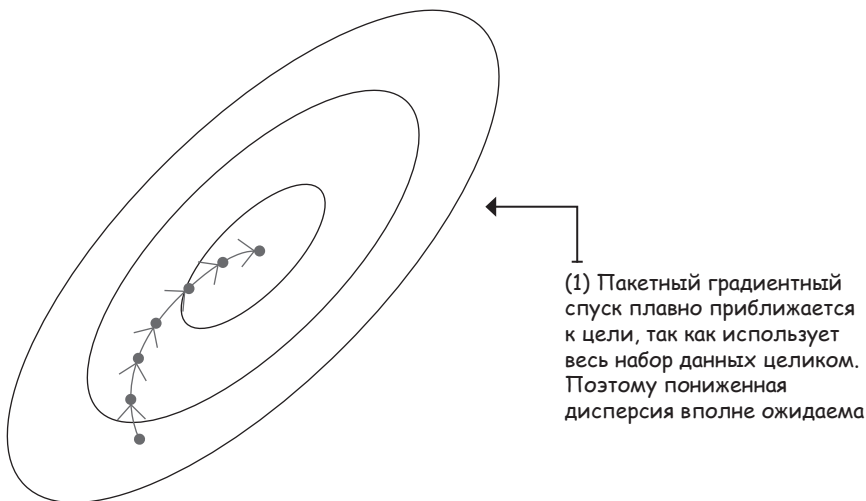
Градиентный спуск — это метод стабильной оптимизации при выполнении нескольких условий: данные должны быть независимы и одинаково распределены (identically distributed, IID), а цели — стационарны. Но в обучении с подкреплением нельзя гарантировать выполнение ни одного из них, поэтому выбор надежного метода оптимизации для минимизации функции потерь — это зачастую ключевой фактор, от которого зависит достижение сходимости.

Если представить функцию потерь в виде ландшафта с долинами, холмами и равнинами, метод оптимизации будет играть роль походной стратегии для поиска интересующих нас участков — обычно самой низкой или высокой точки в этом ландшафте.

Классический метод оптимизации контролируемого обучения называется *пакетным градиентным спуском*. Этот алгоритм берет сразу весь набор данных и вычисляет его градиент, постепенно двигая этот градиент вперед. Процесс повторяется до достижения сходимости. Возвращаясь к аналогии с ландшафтом, градиент — это сигнал, показывающий, куда нам нужно двигаться. Пакетный градиентный спуск не сильно распространен, поскольку обрабатывать огромные наборы данных целиком непрактично. Когда в наборе миллионы выборок, этот алгоритм становится слишком медленным, чтобы применять его в реальных условиях. К тому же в обучении с подкреплением у нас изначально нет никакого набора данных, поэтому в нашем случае этот метод тоже не годится.

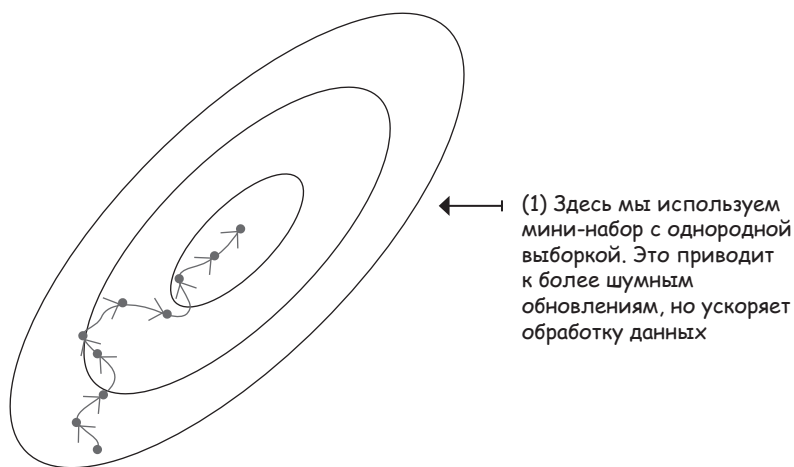
Есть аналогичный метод, способный работать с меньшими наборами данных, — *мини-пакетный градиентный спуск*. Он обрабатывает данные по частям.

Пакетный градиентный спуск



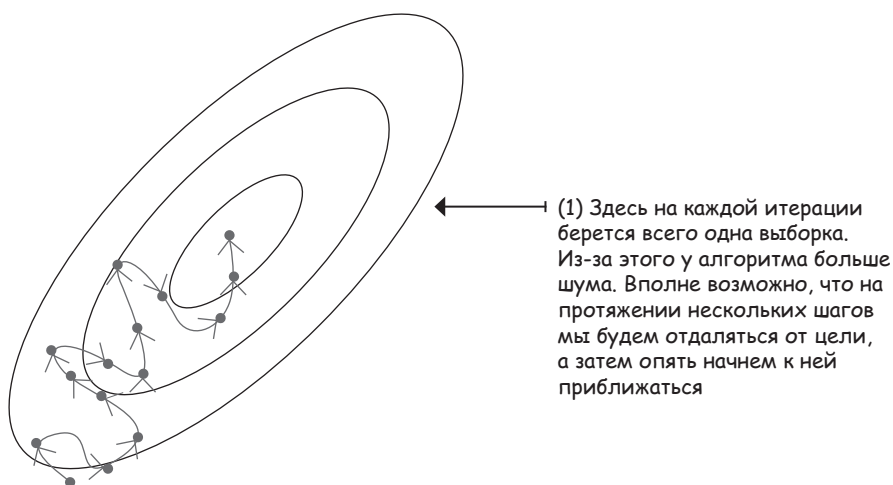
Мы обрабатываем мини-набор выборок в поиске потерь, затем передаем результаты в обратном направлении, чтобы вычислить градиент, а после корректируем веса в сети, чтобы она лучше прогнозировала значения в этом наборе. Мини-пакетный градиентный спуск позволяет регулировать размер мини-наборов, что делает возможной обработку больших наборов данных.

Мини-пакетный градиентный спуск



Мини-набор можно сделать такого же размера, как весь набор данных. В этом случае вы вернетесь к пакетному градиентному спуску. С другой стороны, в качестве мини-набора на каждом шаге можно использовать отдельную выборку, тогда мы получим алгоритм *«стохастический градиентный спуск»*.

Стохастический градиентный спуск



Чем больше набор, тем меньше будет дисперсия шагов метода оптимизации. Но если сделать этот набор слишком большим, темпы обучения замедлятся.

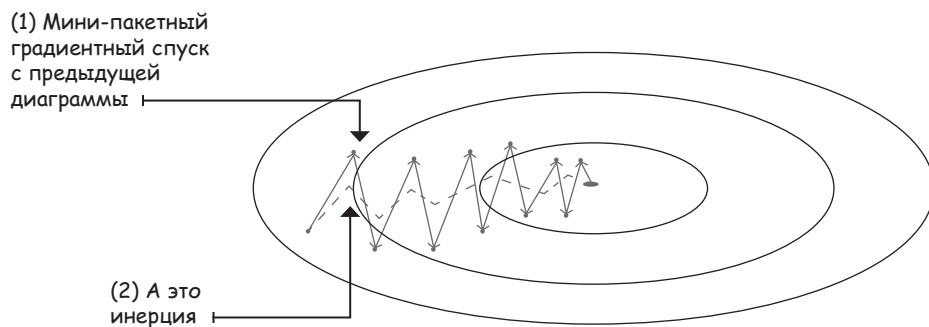
Это два крайних случая, в которых такой метод становится непрактичным. Поэтому размер мини-наборов обычно варьируется от 32 до 1024.

Зигзагообразный мини-пакетный градиентный спуск



У алгоритма градиентного спуска есть улучшенная версия — *градиентный спуск с инерцией* (для краткости будем называть ее просто *инерцией*). По сути, это мини-пакетный градиентный спуск, который приближает веса в сети не к самому градиенту, а к его скользящему среднему.

Сравнение мини-пакетного градиентного спуска и инерции



У инерции есть альтернатива — *корень из среднего квадратов градиентов* (root mean square propagation, RMSprop). Этот метод тоже снижает колебания и движется к цели по более прямому пути, но делает это по-своему.

Если инерция приближается к скользящему среднему градиента, то RMSprop выбирает более консервативный подход, меняя масштаб градиента прямо

пропорционально скользящему среднему его магнитуды — то есть корню из среднего квадратов градиента, или, проще говоря, средней магнитуде последних градиентов.



Аналогия от Мигеля

Методы оптимизации в ценностно ориентированном глубоком обучении с подкреплением

Чтобы представить себе RMSprop, подумайте о том, как меняется крутизна поверхности вашей функции потерь. Если поверхность спускается в плоскую долину от больших градиентов к маленьким, скользящая средняя магнитуда градиентов будет выше, чем у самого последнего градиента: размер шага уменьшается, что снижает колебания и отклонение.

Если изначально маленькие градиенты (как в случае с почти плоской поверхностью) существенно вырастают (как при спуске), их средняя магнитуда будет небольшой. Это увеличит размер шага и ускорит процесс обучения.

Последний метод оптимизации, который я бы хотел обсудить, — *адаптивная оценка инерции* (adaptive moment estimation, Adam). Это что-то среднее между RMSprop и инерцией. Метод Adam пошагово приближается к вектору движения градиентов, как в случае с инерцией, но в то же время масштабирует обновления прямо пропорционально скользящему среднему магнитуды градиентов, как это делает RMSprop. Таким образом, оптимизация Adam является более агрессивной по сравнению с RMSprop, но менее агрессивной, чем инерция.

На практике алгоритмы Adam и RMSprop хорошо подходят для методов ценностно ориентированного глубокого обучения с подкреплением. Они оба используются в следующих главах. Но, как вы вскоре заметите, для ценностно ориентированных методов я все же предпочитаю RMSprop. Это стабильный алгоритм, который не так чувствителен к гиперпараметрам, что особенно важно в ценностно ориентированном DRL.

0001 Немного истории

Внедрение алгоритма NFQ

Алгоритм NFQ был предложен в 2005 году Мартином Ридмиллером (Martin Riedmiller) в научной работе *Neural Fitted Q Iteration First Experiences with a Data Efficient Neural Reinforcement Learning Method*. После 13 лет работы профессором в ряде европейских университетов Мартин стал научным сотрудником в Google DeepMind.



В деталях

Полный алгоритм нейронной аппроксимации Q (NFQ)

Итак, мы выбрали такие методы, как:

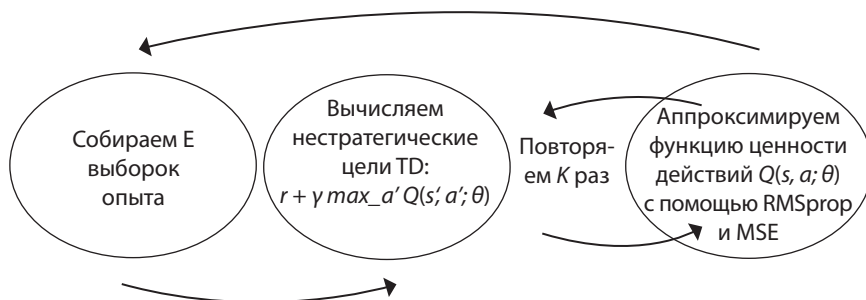
- аппроксимация функции ценности действий $Q(s, a; \theta)$;
- использование архитектуры «состояние на входе, значения на выходе» (узлы: 4, 512 128, 2);
- оптимизация Q -функции для аппроксимации ее оптимальной версии $q^*(s, a)$;
- использование нестратегических целей TD ($r + \gamma \max_a Q(s', a'; \theta)$) для оценки политик;
- использование эpsilon-жадной стратегии (эпсилон равен 0,5) на этапе улучшения политик;
- использование среднеквадратической ошибки (MSE) для нашей функции потерь;
- использование RMSprop как оптимизатора с темпом обучения 0,0005.

NFQ состоит из трех основных этапов.

- Сбор E коротких опытов: (s, a, r, s', d) . Мы используем 1024 выборки.
- Вычисление нестратегических целей TD: $r + \gamma \max_a Q(s', a'; \theta)$.
- Корректировка функции ценности действий $Q(s, a; \theta)$ с помощью MSE и RMSprop.

Алгоритм повторяет шаги 2 и 3 K раз и затем возвращается к шагу 1. Вот что делает его подходящим — вложенный цикл. Мы будем использовать 40 шагов аппроксимации K .

NFQ

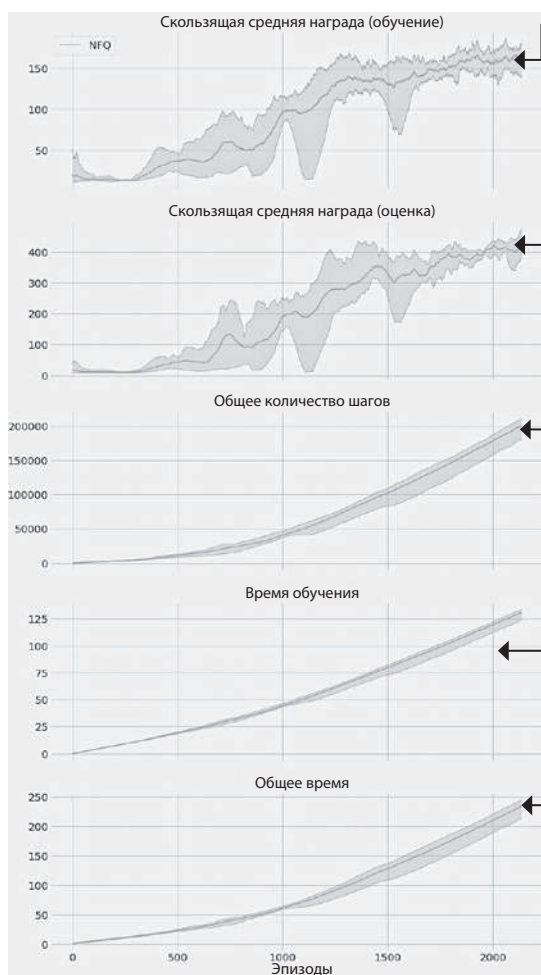




Посчитаем

NFQ проходит среду «перевернутый маятник»

Несмотря на то что NFQ далеко не передовой метод ценностно ориентированного глубокого обучения с подкреплением, в таких довольно простых средах, как «перевернутый маятник», он показывает достойную производительность.



(1) Заметьте, что на этапе обучения награда никогда не достигает 500 за эпизод. Дело в том, что мы используем эпсилон 0,5. Такой высокий темп обучения помогает находить более точные функции ценности, но показывает худшую производительность

(2) На втором графике показана средняя награда на этапе оценки. Во время этих шагов агент показывает лучшую производительность, на которую способен

(3) Главная проблема метода NFQ в том, что для достижения достойной производительности ему нужно слишком много шагов: то есть с точки зрения эффективности выборки NFQ работает плохо. Для получения хороших результатов ему нужно много выборок. Он не извлекает из каждой максимальной пользу

(4) Следующие два графика относятся к времени. Мы видим, что для прохождения среды алгоритму NFQ нужно в среднем 80 секунд. Время обучения не включает в себя шаги оценки, ведение статистики и т. д.

(5) Это общее время прохождения среды от начала и до конца

Что может пойти не так

У нашего алгоритма есть две проблемы. Во-первых, поскольку мы используем мощный аппроксиматор функций, мы можем обобщать пары «состояние — действие». Это отлично, но также означает, что нейросеть корректирует значения всех похожих состояний одновременно.

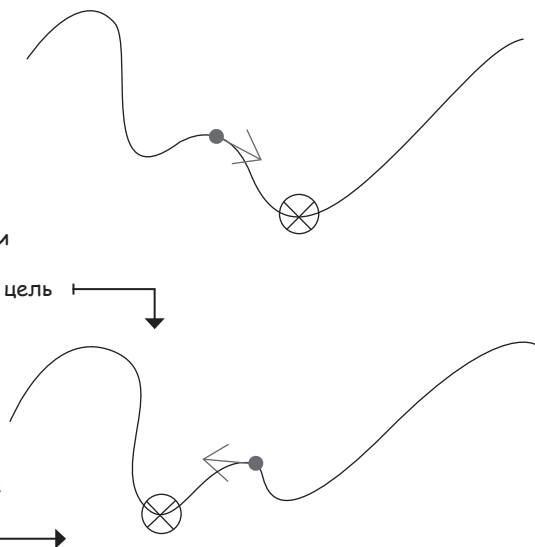
Теперь задумайтесь: наши целевые значения зависят от значений следующего состояния, а оно, в свою очередь, аналогично состояниям, для которых мы корректируем значения в первую очередь. Иначе говоря, мы создаем для наших обновлений в процессе обучения нестационарную цель. Поскольку мы обновляем веса аппроксимированной Q -функции, цель тоже изменяется и делает наше последнее обновление неактуальным. Так обучение быстро становится нестабильным.

Нестационарная цель

(1) Сначала оптимизация идет по плану, приближая нас к цели

(2) Но по мере оптимизации прогнозов оптимизируется (то есть изменяется) и наша цель

(3) Теперь у нашего метода оптимизации могут возникнуть трудности

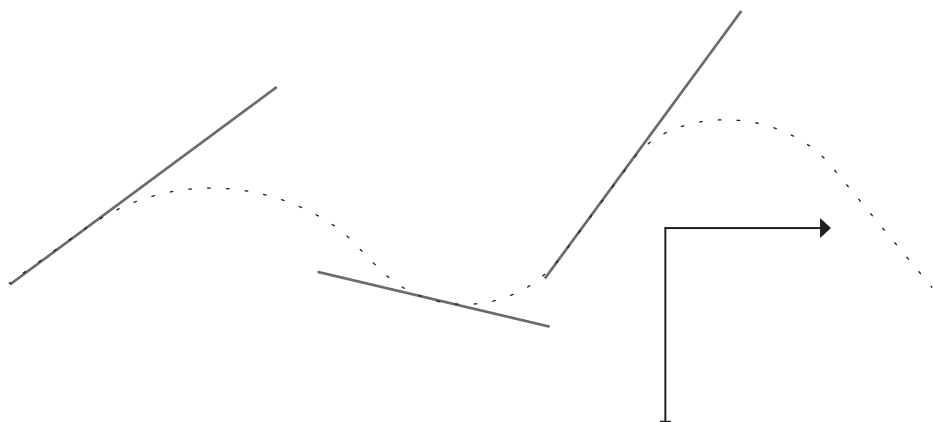


Вторая проблема в том, что мы используем мини-набор из 1024 выборок, собираемых в режиме реального времени, и обновляем с его помощью нейросеть. Как вы понимаете, эти выборки коррелированы, учитывая, что большинство из них исходят из одной траектории и политики. Это значит, что сеть обучается на одних мини-наборах похожих друг на друга выборок и позже использует другие. Эти выборки коррелируют в пределах каждого отдельно взятого мини-набора, но отличаются от предыдущих мини-наборов, особенно если они собирались с помощью другой, более старой политики.

Мы не соблюдаем условия IID, и это плохо, так как в методы оптимизации заложено предположение о том, что выборки данных, которые они используют для обучения, независимы и одинаково распределены. Но наше обучение проходит в почти противоположных условиях: выборки в нашем распределении нельзя считать независимыми, ведь исход нового состояния зависит от текущего.

Кроме того, наши выборки распределены неравномерно, так как внутренний процесс генерации данных (наша политика) со временем меняется, поэтому наше распределение данных варьируется. Наша политика, ответственная за генерацию данных, периодически обновляется и, как мы надеемся, улучшается. Каждый раз, когда это происходит, мы получаем новый и другой опыт. Методы оптимизации позволяют ослабить требования IID, но задачи обучения с подкреплением обязаны их полностью соблюдать, поэтому нам нужно что-то с этим сделать.

Данные коррелируют с временем



(1) Представьте, что мы генерируем эти элементы данных в рамках одной траектории. Пусть ось Y — это положение тележки вдоль колеи, а X — шаги траектории. Как видите, элементы данных в соседних временных шагах с высокой степенью вероятности оказываются похожими, из-за чего наш аппроксиматор функций будет отклоняться к этой локальной области

В следующей главе мы попробуем разобраться с этими двумя проблемами. Первым делом улучшим NFQ с помощью алгоритма DQN, который и лег в основу революции глубокого обучения с подкреплением. Затем обсудим и другие методы оптимизации оригинальной версии DQN, которые были предложены за эти годы. А еще мы поговорим о двойном DQN и в главе 10 проведем сравнение между DQN и PER.

Подведем итоги

В этой главе мы в общих чертах рассмотрели, как выборочная обратная связь взаимодействует с последовательной и оценочной. В этом нам помог простой агент глубокого обучения с подкреплением для аппроксимации Q -функции, которая в прошлых главах была в виде таблицы с возможностью поиска. Эта глава была введением в методы ценностно ориентированного глубокого обучения с подкреплением.

Вы узнали, чем отличаются многомерные и непрерывные пространства действий и состояний. Многомерность указывает на большое количество значений, из которых состоит отдельное состояние, а непрерывность — на наличие как минимум одной переменной, которая может принимать бесконечное количество значений. Вы узнали, что задачи принятия решений могут содержать одновременно многомерные и непрерывные пространства, что делает аппроксимацию нелинейных функций такой захватывающей.

Мы выяснили, что аппроксимация функций полезна не только для прогнозирования ожидаемых значений на основе нескольких выборок, но и для формирования внутренних связей в пространствах состояний и действий. С помощью хорошей модели можно предсказывать значения, для которых у нас нет никаких выборок, и использовать весь накопленный опыт.

Мы подробно рассмотрели разные компоненты для создания агентов глубокого обучения с подкреплением. Вы узнали, что аппроксимировать можно разные функции ценности, от функций ценности состояний $v(s)$ до функций ценности действий $q(s, a)$. Это можно делать с помощью нейросетей с разными архитектурами, такими как «пара «состояние — действие» на входе», «значения на выходе» и более эффективная «состояние на входе, значения на выходе». Вы научились использовать цель TD, которую применяли в Q -обучении, для нестратегического управления. Теперь вы знаете о существовании разных целей обучения сетей. Мы рассмотрели стратегии исследования, функции потерь и методы оптимизации. Вы узнали, что агенты глубокого обучения с подкреплением совместимы с выбранными нами методами оптимизации и минимизации потерь. Вы познакомились с такими стабильными методами оптимизации, как RMSprop и Adam.

Наконец, вы научились объединять все эти компоненты в алгоритм «нейронная аппроксимация Q » и узнали о проблемах, присущих методам ценностно ориентированного глубокого обучения с подкреплением. Вы узнали, что такое предположения PD и стационарность целей и что несерьезное отношение к этим двум аспектам чревато проблемами.

На этом этапе вы:

- понимаете, что можно извлечь из одновременно последовательной, оценочной и выборочной обратной связи;
- умеете выполнять задачи RL с непрерывными пространствами состояний;
- знакомы с компонентами методов ценностно ориентированного DRL и присущими им проблемами.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и, я надеюсь, вы им воспользуетесь.

- **#gdr1_ch08_tf01.** Между табличным и глубоким обучением с подкреплением есть несколько аспектов, которые можно исследовать. С помощью этого хештега поделитесь своими впечатлениями о методах дискретизации состояний и мозаичного кодирования. Что это такое? Возможно, есть и другие методы, о которых нам стоит узнать?
- **#gdr1_ch08_tf02.** Изучите применение аппроксимации линейных функций вместо глубоких нейросетей. Можете ли вы сравнить ее с другими методами аппроксимации функций? Какие из них показывают лучшие результаты?
- **#gdr1_ch08_tf03.** Мы познакомились с градиентным спуском — методом оптимизации, который будет использоваться до конца этой книги. Но знаете ли вы, что нейросети можно оптимизировать иначе? Исследуйте разные пути оптимизации нейронных сетей, такие как метод черного ящика, генетические алгоритмы и другие, менее популярные подходы. Поделитесь своими находками, создайте и опубликуйте блокнот Jupyter с результатами ваших исследований.
- **#gdr1_ch08_tf04.** В начале этой главы я представил улучшенный подход к Q-обучению на основе аппроксимации функций. Но важно знать и об упрощенном подходе, который не дал желаемых результатов. Внесите минимальные изменения в метод Q-обучения, чтобы сделать его совместимым с нейросетями. У вас должно получиться Q-обучение с динамически накапливаемым опытом, с которым вы познакомились в главе 6. Поделитесь с нами результатами своего исследования.
- **#gdr1_ch08_tf05.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

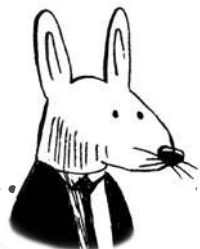
Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdr1_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.

9

Более стабильные ценностно ориентированные методы



В этой главе

- ✓ Вы внесете улучшения в методы из предыдущей главы, делая их более стабильными и менее подверженными отклонению.
- ✓ Исследуете передовые методы ценностно ориентированного глубокого обучения с подкреплением и компоненты для их улучшения.
- ✓ Выполните задачу с перевернутым маятником с меньшим количеством выборок и более надежными, последовательными результатами.

Пусть шаг твой будет медленным и ровным, чтобы ты не споткнулся.

*Токугава Иэясу (Tokugawa Ieyasu),
основатель и первый сёгун сёгуната Токугава,
один из трех объединителей Японии*

Ранее вы познакомились с ценностно ориентированным глубоким обучением с подкреплением. Мы разработали алгоритм NFQ, который помог решить две самые распространенные проблемы ценностно ориентированных методов. Во-первых, данные в RL не независимые и одинаково распределенные, а наоборот. Опыт зависит от генерирующей его политики, и они не распределены одинаково, так как последняя меняется в процессе обучения. Во-вторых, цели, которые мы используем, не стационарны. Для стабильной производительности методам оптимизации нужна постоянная цель, что можно проследить на примере контролируемого обучения. У нас есть набор данных с готовыми метками в виде констант, и наши методы оптимизации используют эти фиксированные цели для стохастической аппроксимации функции, генерирующей данные. А вот в RL используются такие цели, как TD, основанные на награде и дисконтированной ожидаемой выгоде в следующем состоянии. Но эту выгоду предсказывает нейросеть, которую мы оптимизируем и которая изменяется при каждом выполнении шагов оптимизации. В результате цель постоянно сдвигается, что делает процесс обучения нестабильным.

Для борьбы с этими проблемами NFQ использует метод пакетной обработки. Сгруппировав достаточное количество выборок, мы получаем возможность оптимизировать их одновременно. Чем больше набор, тем легче собрать разнообразные выборки. Это позволяет соблюсти требование IID. Для соблюдения условия стационарности цели NFQ использует один мини-набор на нескольких последовательных шагах оптимизации. Как вы помните, в NFQ в каждом эпизоде E мы приводим нейросеть в соответствие с одним и тем же набором K раз. Это позволяет методу оптимизации двигаться к цели более плавно. Формирование набора и аппроксимация модели на протяжении нескольких итераций похожи на то, как работают методы контролируемого обучения, где мы собираем набор данных и постепенно обучаемся.

NFQ показывает неплохие результаты, но мы не станем на этом останавливаться. Определившись с проблемами, мы можем использовать для их решения более подходящие средства. В этой главе мы исследуем алгоритмы, которые помогут справиться с этими и другими задачами, повышая стабильность ценностно ориентированных методов.

DQN: делаем RL похожим на контролируемое обучение

Первый алгоритм, который мы обсудим в этой главе, — *глубокая Q-сеть* (deep Q-network, DQN). Это один из самых популярных алгоритмов DRL, поскольку с него начался ряд исследований, определивших историю развития обучения с подкреплением. DQN первым превзошел человека в серии тестов на основе игр Atari, в которых агенты обучаются на обычных пиксельных изображениях.

Со временем в DQN было внесено большое количество улучшений. И хотя на сегодня оригинальная версия не лучший выбор, этот алгоритм по-прежнему один из лучших агентов DRL с точки зрения производительности. А все благодаря улучшениям, многие из которых вы найдете в этой книге.

Проблемы ценностно ориентированного глубокого обучения с подкреплением

Важно четко определить две самые распространенные проблемы, которые постоянно возникают в ценностно ориентированном глубоком обучении с подкреплением: нарушение условия IID и стационарность целей.

В контролируемом обучении мы заранее получаем полный набор данных. Мы его обрабатываем, перемешиваем и разделяем на группы для дальнейшего обучения. Второй этап — один из ключевых в этом процессе. Перетасовка позволяет нашему методу оптимизации избежать смещений, связанных с переобучением, снизить дисперсию, ускорить сходимость и в целом сформировать более обобщенное представление внутреннего процесса генерации данных. К сожалению, в обучении с подкреплением данные обычно собираются динамически, в результате чего выборки, сгенерированные на временных шагах t и $t + 1$, связаны. Более того, по мере улучшения политики изменяется и внутренний процесс сбора данных, из-за чего новые данные локально коррелируются и распределяются неравномерно.



По сути

Данные не распределены независимо и равномерно (IID)

Первая проблема — несоблюдение условий IID в отношении данных. Методы оптимизации разрабатываются исходя из того, что выборки в наборе данных, на котором мы обучаемся, независимы и равномерно распределенные.

Но мы уже знаем, что наши выборки нельзя считать независимыми: они берутся из последовательности, временного ряда, траектории. Выборка на временном шаге $t + 1$ зависит от выборки на временном шаге t . Они коррелируют, и с этим ничего не поделаешь: это естественно для динамического обучения.

Но выборки еще и распределены неодинаково, так как зависят от политики, генерирующей действия. Как мы знаем, она со временем меняется, и нам это играет на руку, ведь мы хотим, чтобы политика улучшилась. Но вместе с ней будет улучшаться и распределение выборок (посещенных пар «состояние — действие»).

Кроме того, цели в контролируемом обучении берутся прямо из набора данных и служат фиксированными значениями, неизменными на протяжении всего процесса обучения. В RL, и особенно в динамическом, цели меняются с каждым шагом сети. В ходе каждого обновления мы оптимизируем ожидаемую функцию ценности, меняя ее форму и, возможно, всю ее целиком. А значит, меняются и целевые значения, поэтому использованные цели становятся

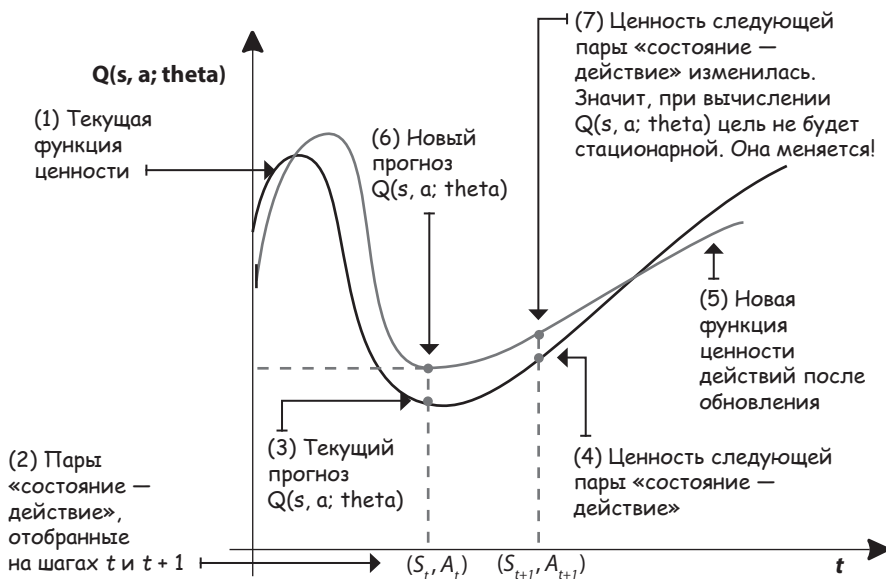
недействительными. Поскольку они берутся из сети, мы можем исходить из того, что они уже неактуальны или как минимум смещены, и для этого нам их даже не нужно использовать.



По сути

Нестационарность целей

Ниже показана задача с нестационарными целями — теми, которые используются для обучения сети, но та же сеть участвует в их вычислении. В результате функция изменяется с каждым обновлением, изменяя и цели.



В NFQ мы смягчаем эту проблему, используя пакет и подгоняя сеть к небольшому фиксированному набору данных для нескольких итераций. Мы собираем небольшой набор данных, вычисляем цели, оптимизируем сеть несколько раз и идем дальше за новыми выборками. Если набор большой, обновления нейросети будут охватывать множество точек на функции, делая изменения еще стабильней.

Алгоритм DQN помогает ответить на вопрос «Как сделать RL больше похожим на контролируемое обучение?». Только подумайте об изменениях, которые вам бы пришлось внести, чтобы зафиксировать цели и привести данные к требованиям PID.

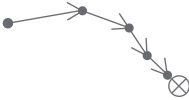
Использование целевых сетей

Чтобы сделать целевые значения более стационарными, можно использовать отдельную сеть, которая будет зафиксирована на протяжении нескольких шагов, и выделить ее для вычисления целей. В DQN такая сеть называется *целевой*.

Оптимизация Q-функции без целевой сети

(1) Сначала все выглядит нормально. Мы просто преследуем цель 	(2) Но по мере улучшения нашей Q-функции она будет двигаться 
(3) Затем ситуация усугубляется 	(4) Движущиеся цели могут спровоцировать отклонение 

Аппроксимация Q-функции с целевой сетью

(1) Допустим, мы зафиксировали цель на несколько шагов 	(2) Так оптимизатор может плавно к ней двигаться 
(3) В итоге мы обновляем цель и повторяем все заново 	(4) Это позволяет алгоритму стабильно развиваться 

Фиксируя цели с помощью целевой сети, мы смягчаем проблему «погони за собственным хвостом» за счет искусственного создания нескольких мелких задач контролируемого обучения, представленных агенту последовательно. Мы закрепляем наши цели и целевую сеть на протяжении одинакового количества шагов. Это повышает наши шансы на сходимость, но не до оптимальных значений, так как в условиях аппроксимации нелинейной функции таких вовсе не существует. Что важнее, это сильно снижает вероятность отклонения, которое нередко происходит в методах ценностно ориентированного глубокого обучения с подкреплением.



Я знаю математику

Обновление градиента с помощью целевой сети

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s,a,r,s'} \left[(r + \gamma \max_{a'} Q(s', a'; \theta_i) - Q(s, a; \theta_i)) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$

(1) Единственная разница между этими двумя уравнениями в «возрасте» весов нейросети

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s,a,r,s'} \left[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta_i)) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$

(2) Целевая сеть — это «старый» экземпляр нейросети, который мы замораживаем на определенное количество шагов. Теперь у обновления градиента есть время, чтобы догнать «замершую» цель, которая в этом состоянии получается намного устойчивей. Это повышает стабильность обновлений

Важно отметить, что на практике вместо двух разных сетей мы используем два набора весов для одной нейросети. Мы применяем и одну архитектуру модели и обновляем веса целевой сети с высокой частотой, чтобы сравнять их с весами динамической, которая улучшается на каждом шаге. К сожалению, высота этой частоты зависит от задачи. Обычно веса целевой сети замораживают на протяжении 10–10 000 шагов — опять же в зависимости от задачи (речь идет о временных шагах, а не об эпизодах. Будьте внимательны). Если используется сверточная сеть, как в играх Atari, частота 10 000 шагов будет приемлемой. Но в более простых средах, таких как «перевернутый маятник», уместнее будет частота 10–20 шагов.

Благодаря использованию целевых сетей процесс обучения больше не может заикливаться: мы фиксируем цели на протяжении определенного числа шагов, позволяя весам динамической сети двигаться к целям более последовательно, после чего в результате обновления меняется задача оптимизации. Так мы стабилизируем процесс обучения, но замедляем его, так как он больше не использует актуальные значения — замороженные веса целевой сети могут отставать на 10 000 шагов. Эти гиперпараметры обязательно нужно корректировать, чтобы обеспечить баланс между стабильностью и скоростью.



Я знаю Python

Использование целевой и динамической сетей в DQN

```
def optimize_model(self, experiences):
    states, actions, rewards, \
        next_states, is_terminals = experiences
    batch_size = len(is_terminals)

    (1) Заметьте, что для получения
    прогнозов следующего состояния
    мы обращаемся к целевой сети
    q_sp = self.target_model(next_states).detach()

    (2) Мы берем максимальное из этих
    значений и следим, чтобы терминальные
    состояния обрабатывались как следует
    max_a_q_sp = q_sp.max(1)[0].unsqueeze(1)
    max_a_q_sp *= (1 - is_terminals)

    (3) Создаем цели TD
    target_q_sa = rewards + self.gamma * max_a_q_sp

    (4) Запрашиваем текущий динамический прогноз
    q_sa = self.online_model(states).gather(1, actions)

    (5) На основе этих оценок создаем погрешности
    td_error = q_sa - target_q_sa
    value_loss = td_error.pow(2).mul(0.5).mean()
    self.value_optimizer.zero_grad()
    value_loss.backward()
    self.value_optimizer.step()

    (6) Вычисляем потери
    и оптимизируем
    динамическую сеть

def interaction_step(self, state, env):
    action = self.training_strategy.select_action(
        self.online_model, state)

    (7) Обратите внимание
    на использование динамической
    модели при выборе действий
    new_state, reward, is_terminal, _ = env.step(action)
    <...>
    return new_state, is_terminal

    (8) Вот как целевая (отстающая)
    сеть синхронизируется
    с динамической (актуальной)

def update_network(self):
    for target, online in zip(
        self.target_model.parameters(),
        self.online_model.parameters()):
        target.data.copy_(online.data)
```

Использование крупных сетей

Проблему нестационарности можно смягчить и за счет использования сетей большего размера. В таких сетях повышается вероятность обнаружения незначительных различий между состояниями, что позволяет ослабить эффект наложения пар «состояние — действие». Чем больше сеть, тем слабее эффект наложения — чем слабее эффект, тем менее очевидна корреляция между соседними выборками. Благодаря этому целевые значения и текущие прогнозы выглядят более независимыми.

«Эффект наложения», в моем понимании, — это когда нейросеть может считать два разных состояния одинаковыми (или похожими), даже если для них все еще нужны разные действия. Наложение состояний появляется, когда сети недостает разрешающей способности. Нейросети пытаются искать сходства, которые можно обобщить, — это их назначение. Но слишком маленькая сеть может привести к неверному обобщению из-за риска заикнуться на простых закономерностях, которые легко найти.

Одна из причин использования целевых сетей — то, что с их помощью становится проще различать коррелирующие состояния. Применение же более мощной сети помогает выявлять тонкие различия.

Но чем мощнее нейросеть, тем дольше ее нужно обучать. Ей нужно не только больше данных (времени взаимодействия), но и больше вычислений (времени обработки). Использование целевых сетей — более надежный способ борьбы с проблемой нестационарности, но я хочу обучить вас всем доступным приемам. Вам будет полезно знать, как эти два свойства агента (размер сети и применение целевых сетей с частотой обновлений) связаны и как они влияют на итоговую производительность.



По сути

Способы борьбы с нестационарностью целей в обучении с подкреплением

Еще раз напомним, что для смягчения проблемы нестационарности мы можем:

- создать целевую сеть, которая дает временно стационарные целевые значения;
- создать крупную сеть, способную находить небольшие различия между похожими (например, временно коррелирующими) состояниями.

Целевые сети хорошо работают и неоднократно доказывали свою эффективность. Использование больших сетей — это скорее эмпирический, а не научно доказанный подход, который будет работать в любых ситуациях. Поэкспериментируйте с блокнотом Jupyter для этой главы. Вы сможете легко менять значения и проверять свои гипотезы.

Воспроизведение опыта

В наших экспериментах с NFQ мы используем мини-набор с 1024 выборками и с его помощью обучаемся в течение 40 итераций, чередуя вычисление новых целей и оптимизацию сети. Эти 1024 выборки временно коррелируют между собой, так как все принадлежат одной траектории, а каждый эпизод в среде «перевернутый маятник» состоит не больше чем из 500 шагов. Одно из возможных улучшений — использование методики *воспроизведения опыта*. Для этого нам нужна структура данных, которую часто называют буфером или памятью воспроизведения: она будет хранить выборки для определенного числа шагов (больше чем 1024), что позволит собирать данные из мини-наборов с использованием уже накопленного опыта. Наличие буфера воспроизведения позволяет улучшить два важных аспекта работы агента. Во-первых, он может использовать в процессе обучения более разнообразный мини-набор для выполнения обновлений. Во-вторых, агенту больше не нужно приводить модель к одному мини-набору на протяжении нескольких итераций. Адекватная выборка из большого буфера воспроизведения дает медленно движущуюся цель, поэтому агент может выполнять выборку и обучение на каждом временном шаге с меньшим риском расхождения.

0001 Немного истории

Кто придумал воспроизведение опыта

Впервые воспроизведение опыта было предложено в научной работе Лун-цзы Линя (Long-Ji Lin) *Self-Improving Reactive Agents Based On Reinforcement Learning, Planning and Teaching*, опубликованной аж в 1992 году! Еще когда нейросети называли «коннекционизмом»... Тяжелые времена!

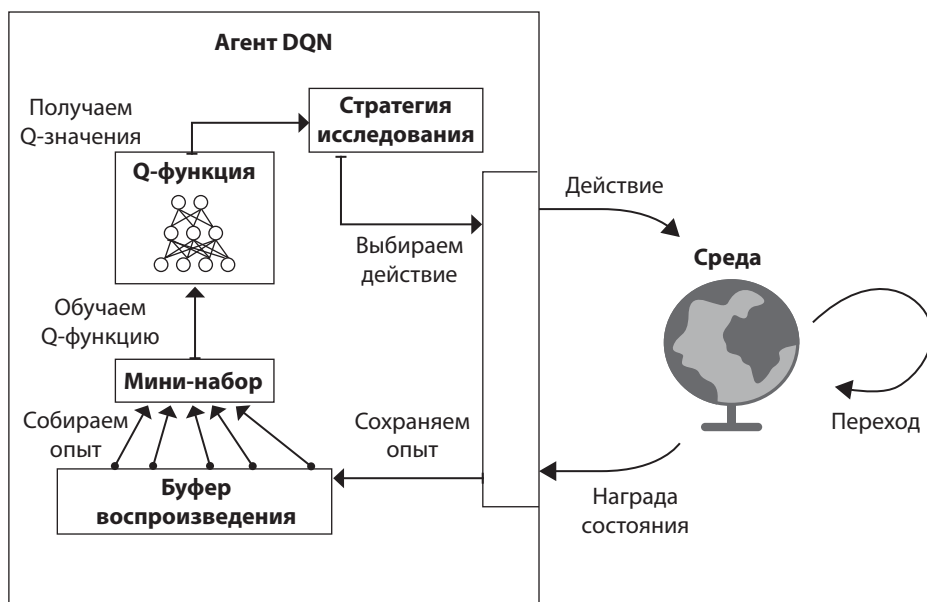
Защитив докторскую в Университете Карнеги-Меллона, доктор Линь поработал в разных компаниях на разных технических должностях. Сейчас он главный научный сотрудник в Signifyd, возглавляющий команду, которая работает над системой прогнозирования и предотвращения мошенничества в Интернете.

У воспроизведения опыта есть несколько преимуществ. Собирая данные случайно, мы повышаем вероятность появления низкой дисперсии обновлений нашей нейросети. Когда мы использовали набор данных в NFQ, большинство выборок в нем были похожими и коррелировали между собой. Обновление на основе похожих выборок приводит к концентрированному изменению ограниченной области функции, что может сделать магнитуду обновлений слишком большой. Но если данные собираются равномерно случайно из крупного буфера, наши обновления сети будут шире распределяться и лучше отражать реальную функцию ценности.

Использование буфера воспроизведения создает впечатление, что наши данные соответствуют IID и что метод оптимизации стабилен. Выборки выглядят независимыми и одинаково распределенными, потому что берутся сразу из нескольких траекторий и даже политик.

Сохраняя опыт и единообразно его сэмплируя, мы позволяем данным, поступающим в метод оптимизации, выглядеть независимыми и равномерно распределенными. Чтобы буфер хорошо работал, у него должен быть довольно большой объем, от 10 000 до 1 000 000 выборок в зависимости от задачи. По достижении максимального размера перед добавлением новой выборки нужно сначала удалить самую старую.

DQN с буфером воспроизведения



К сожалению, при работе с многомерными наблюдениями реализация становится немного сложнее, так как плохо реализованный буфер воспроизведения может очень быстро исчерпать доступную физическую память. Например, в средах, ориентированных на изображения, где каждое состояние представлено стеком из четырех последних фреймов (как в играх Atari), у вашего ПК, скорее всего, не хватит памяти для хранения сразу 1 000 000 выборок. В среде «перевернутый маятник» эта проблема стоит не так остро. Во-первых, нам не нужно 1 000 000 выборок, так как мы используем буфер в 50 000. Во-вторых, состояния имеют вид четырехэлементных векторов, поэтому трудностей с обеспечением хорошей производительности быть не должно.



Я знаю математику

Обновление градиента с использованием буфера воспроизведения

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s,a,r,s'} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$

(1) Единственная разница между этими двумя уравнениями в том, что опыт, который используется при обучении, не накапливается динамически, как раньше, а собирается из буфера D равномерно случайно

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right]$$

(2) Это полное градиентное обновление для DQN — то, которое называется Nature DQN (DQN с целевой сетью и буфером воспроизведения)

Но благодаря использованию буфера воспроизведения ваши данные внешне соответствуют IID, а цели выглядят более стационарными, чем на самом деле. Агент обучается на равномерно сформированных мини-наборах данных, поэтому опыт, накапливаемый в RL динамически, становится похожим на наборы данных в традиционном контролируемом обучении, выполняющим условия IID и обладающим фиксированными целями. Конечно, при добавлении новых и удалении старых выборок данные все еще изменяются, но делают это медленно и в какой-то мере незаметно для нейросети и метода оптимизации.



По сути

Воспроизведение опыта делает так, что данные внешне соответствуют IID, а цели выглядят более стационарными

Воспроизведение опыта — лучшее решение, когда данные не соответствуют IID.

Это простой подход, проверенный временем. По мере того как ваш агент динамически собирает кортежи опыта $e_t = (S_t, A_t, R_{t+1}, S_{t+1})$, мы вставляем их в структуру данных D , которую называют *буфером воспроизведения*, — например, $D = \{e_1, e_2, \dots, e_M\}$. M отражает размер буфера и обычно находится в пределах от 10 000 до 1 000 000 в зависимости от задачи.

После агент обучается на мини-наборе, который формируется из буфера (обычно равномерно случайно) так, чтобы вероятность применения каждой выборки была одинаковой. Хотя, как вы позже увидите, выборки могут быть распределены иначе. Поэтому будьте внимательны. Подробнее об этом — в главе 10.



Я знаю Python

Простой буфер воспроизведения

```

class ReplayBuffer():
    def __init__(self,
                  m_size=50000,
                  batch_size=64):
        self.ss_mem = np.empty(shape=(m_size), dtype=np.ndarray)
        self.as_mem = np.empty(shape=(m_size), dtype=np.ndarray)
        <...>

    def store(self, sample):
        s, a, r, p, d = sample
        self.ss_mem[self._idx] = s
        self.as_mem[self._idx] = a
        <...>

        self._idx += 1
        self._idx = self._idx % self.m_size

        self.size += 1
        self.size = min(self.size, self.m_size)

    def sample(self, batch_size=None):
        if batch_size == None:
            batch_size = self.batch_size
        idxs = np.random.choice(
            self.size, batch_size, replace=False)

        experiences = np.vstack(self.ss_mem[idxs]), \
                       np.vstack(self.as_mem[idxs]), \
                       np.vstack(self.rs_mem[idxs]), \
                       np.vstack(self.ps_mem[idxs]), \
                       np.vstack(self.ds_mem[idxs])

        return experiences

    def __len__(self):
        return self.size

```

(1) Это простой буфер воспроизведения с максимальным размером 50 000 и размером выборки 64, которые используются по умолчанию

(2) Инициализируем пять массивов для хранения состояний, действий, наград, следующих состояний и флагов done. Скрыто для краткости

(3) Инициализируем несколько переменных для хранения и сбора данных

(4) Сохранение новой выборки начинается с развертывания переменной образца с последующим присвоением соответствующего значения каждому элементу массива

(5) Опущено для краткости

(6) `_idx` указывает на следующий изменяемый индекс, поэтому мы увеличиваем его и убеждаемся, что он возвращается по достижении максимального размера (конец буфера)

(7) Размер тоже увеличивается с сохранением каждой новой выборки, но не обнуляется, а просто перестает расти

(8) Функция `sample` начинается с определения размера набора. Если ничего не передано, используем значение по умолчанию 64

(9) Выбираем `batch_size` идентификаторов от 0 до size

(10) Извлекаем выборки из буфера с помощью их идентификаторов...

(11) ...и возвращаем их

(12) Это удобная функция для возврата правильного размера буфера при вызове `len(buffer)`

Использование разных стратегий исследования

Исследование — это неотъемлемая часть обучения с подкреплением. В алгоритме NFQ используется эpsilon-жадная стратегия исследования — выбор случайных действий с вероятностью эpsilon. Мы берем число из равномерного распределения (0, 1). Если оно меньше фиксированного гиперпараметра (эpsilon), ваш агент выбирает действие равномерно случайно (выбор может пасть и на жадное действие), а если больше — он действует жадно.

Для экспериментов с DQN я добавил в блокнот Jupyter главы 9 некоторые стратегии исследования из главы 4, адаптировав их для использования вместе с нейросетями. Ниже они приведены повторно. Обязательно ознакомьтесь со всеми блокнотами и используйте их в своих экспериментах.



Я знаю Python

Линейно затухающая эpsilon-жадная стратегия исследования

```
class EGreedyLinearStrategy():
    <...>
    def _epsilon_update(self):
        self.epsilon = 1 - self.t / self.max_steps
        self.epsilon = (self.init_epsilon - self.min_epsilon) * \
            self.epsilon + self.min_epsilon
        self.epsilon = np.clip(self.epsilon,
                               self.min_epsilon,
                               self.init_epsilon)
        self.t += 1
    return self.epsilon

(4) В методе select_action
используем модель и состояние
def select_action(self, model, state):
    self.exploratory_action = False
    with torch.no_grad():
        q_values = model(state).cpu().detach()
        q_values = q_values.data.numpy().squeeze()

        if np.random.rand() > self.epsilon:
            action = np.argmax(q_values)
        else:
            action = np.random.randint(len(q_values))

    self._epsilon_update()
    self.exploratory_action = action != np.argmax(q_values)
    return action
```

(1) Здесь мы начинаем с высокого значения эpsilon, снижая его линейно

(2) Ограничиваем эpsilon диапазоном от начального до минимального значения

(3) Эта переменная отслеживает, сколько раз обновлялось значение эpsilon

(5) Я всегда извлекаю значения q для ведения журнала

(6) Берем случайное число из равномерного распределения и сравниваем его с эpsilonом

(7) Если оно больше, вычисляем $\arg\max$ для q_values , если нет, выбираем случайное действие

(8) Обновляем эpsilon, устанавливаем переменную для ведения журнала и возвращаем выбранное действие



Я знаю Python

Экспоненциально затухающая эpsilon-жадная стратегия исследования

```
class EGreedyExpStrategy():
    <...>

    def _epsilon_update(self):
        (1) Единственное отличие экспоненциально
            затухающей стратегии в том, что эpsilon
            теперь затухает по экспоненте
        self.epsilon = max(self.min_epsilon,
                           self.decay_rate * self.epsilon)
        return self.epsilon

        (2) Это еще один способ экспоненциального затухания
            эpsilon, на этот раз на основе экспоненциальной
            функции. Значения эpsilon во многом останутся без
            изменений, только затухание будет в другом масштабе
        # def _epsilon_update(self):
        #     self.decay_rate = 0.0001
        #     epsilon = self.init_epsilon * np.exp( \
        #                                     -self.decay_rate * self.t)
        #     epsilon = max(epsilon, self.min_epsilon)
        #     self.t += 1
        #     return epsilon

    def select_action(self, model, state):
        self.exploratory_action = False
        with torch.no_grad():
            q_values = model(state).cpu().detach()
            q_values = q_values.data.numpy().squeeze()

        (3) Здесь используется та же функция select_action, что и в прошлой стратегии.
            Заметьте, что я выбираю значения из q_values на каждом шаге, только чтобы
            собрать информацию, которой хочу с вами поделиться. Если вас заботит
            производительность, этого лучше не делать. Более быстрая реализация
            обращалась бы к сети только при выборе жадного действия

        if np.random.rand() > self.epsilon:
            action = np.argmax(q_values)
        else:
            action = np.random.randint(len(q_values))
            self._epsilon_update()

        (4) Здесь переменная exploratory_action используется
            для вычисления количества исследовательских действий
            за эпизод. Это нужно для ведения статистики
        self.exploratory_action = action != np.argmax(q_values)
        return action
```



Я знаю Python

Стратегия исследования softmax

```

class SoftMaxStrategy():
    <...>
    def _update_temp(self):
        temp = 1 - self.t / (self.max_steps * self.explore_ratio)
        temp = (self.init_temp - self.min_temp) * \
            temp + self.min_temp

```

(1) Один из параметров здесь — температура: чем ближе ее значение к 0, тем ярче выражаются различия между значениями, что делает процесс выбора действий более жадным. Температура затухает линейно

```

        temp = np.clip(temp, self.min_temp, self.init_temp)
        self.t += 1
        return temp

```

(2) После линейного затухания температуры мы ограничиваем ее значение, чтобы оно находилось в допустимом диапазоне

```

def select_action(self, model, state):
    self.exploratory_action = False
    temp = self._update_temp()
    with torch.no_grad():

```

(3) Заметьте, что в стратегии softmax нельзя избежать извлечения q_values из модели, так как действия напрямую зависят от значений

```

        q_values = model(state).cpu().detach()
        q_values = q_values.data.numpy().squeeze()

```

(4) После извлечения значений мы хотим сделать различия между ними ярко выраженными (если только температура не равна 1)

```

        scaled_qs = q_values/temp

```

(5) Нормализуем их, чтобы избежать переполнения операции exp

```

        norm_qs = scaled_qs - scaled_qs.max()
        e = np.exp(norm_qs)
        probs = e / np.sum(e)
        assert np.isclose(probs.sum(), 1.0)

```

(6) Вычисляем экспоненту

(7) Преобразуем в вероятности

(8) На основе этих вероятностей выбираем действие. Обратите внимание, как мы присваиваем переменную probs аргументу функции p

```

        action = np.random.choice(np.arange(len(probs)),
                                   size=1, p=probs)[0]

```

(9) И снова, каким было действие: жадным или исследовательским?

```

        self.exploratory_action = action != np.argmax(q_values)
        return action

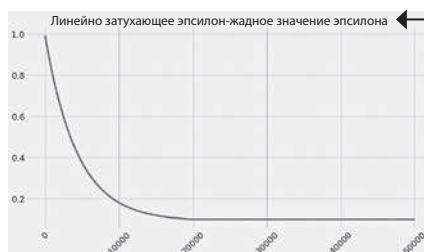
```



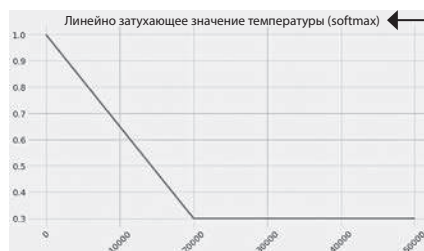
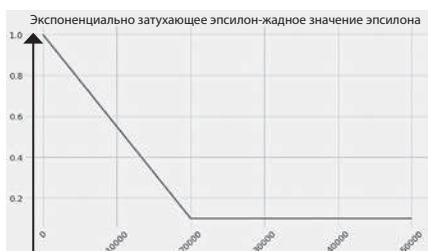
В деталях

Стратегии исследования сильно влияют на производительность

(1) В NFQ мы использовали эpsilon-жадную стратегию с постоянным значением 0,5. Именно так! В половине случаев мы вели себя жадно, а в другой половине выбирали действия равномерно случайно. Учитывая, что в этой среде всего два действия, вероятность выбора жадного — 75 %, а нежадного — 25 %. Заметьте, что в большом пространстве действий вероятность жадного выбора будет ниже. В блокноте Jupyter я пометил эту фактическую вероятность как $\epsilon \times 100$, что означает «частота выбора исследовательского действия за последние 100 шагов»



(2) В DQN и во всех остальных ценностно ориентированных алгоритмах из этой и следующих глав я использую экспоненциально затухающую эpsilon-жадную стратегию, потому что она простая и хорошо работает. Но вы можете попробовать и другие, более продвинутые. Я заметил, что даже небольшие изменения в гиперпараметрах сильно влияют на производительность. Убедитесь в этом сами



(3) Это графики затухания остальных стратегий исследования из блокнота к главе 9. Я настоятельно советую вам поэкспериментировать с ним, используя разные гиперпараметры и стратегии. Глубокое обучение с подкреплением не ограничивается только алгоритмами



В деталях

Полный алгоритм глубокой Q-сети (DQN)

Компоненты и параметры в нашей реализации DQN похожи на те, что были в NFQ:

- аппроксимация функции ценности действий $Q(s, a; \theta)$;
- использование архитектуры «состояние на входе, значения на выходе» (узлы: 4, 512 128, 2);
- оптимизация Q-функции для аппроксимации ее оптимальной версии $q^*(s, a)$;
- использование нестратегических целей TD ($r + \gamma \max_a Q(s', a'; \theta)$) для оценки политик;
- использование среднеквадратической ошибки (MSE) для нашей функции потерь;
- использование RMSprop в качестве оптимизатора с темпом обучения 0,0005.

Особенность этой реализации DQN в том, что теперь мы:

- используем экспоненциально затухающую стратегию на этапе улучшения, уменьшая эпсилон с 1,0 до 0,3 примерно за 20 000 шагов;
- используем буфер воспроизведения размером 320–50 000 выборок и мини-наборы размером 64;
- используем целевую сеть, которая обновляется раз в 15 шагов.

DQN состоит из трех основных этапов.

- Сбор кортежа опыта $(S_t, A_t, R_{t+1}, S_{t+1}, D_{t+1})$ и добавление его в буфер воспроизведения.
- Выбор из буфера случайного мини-набора и вычисление нестратегических целей TD для всего набора данных: $r + \gamma \max_a Q(s', a'; \theta)$.
- Корректировка функции ценности действий $Q(s, a; \theta)$ с помощью MSE и RMSprop.

0001

Немного истории

История создания алгоритма DQN

Алгоритм DQN впервые был предложен в 2013 году Владимиром Мнихом (Vladimir Mnih) в научной работе *Playing Atari with Deep Reinforcement Learning*, где описывалась версия DQN с воспроизведением опыта. В 2015 году была опубликована другая научная работа, *Human-Level Control Through Deep Reinforcement Learning*, где алгоритм DQN применялся вместе с целевыми сетями (это полная версия DQN, с которой вы познакомились).

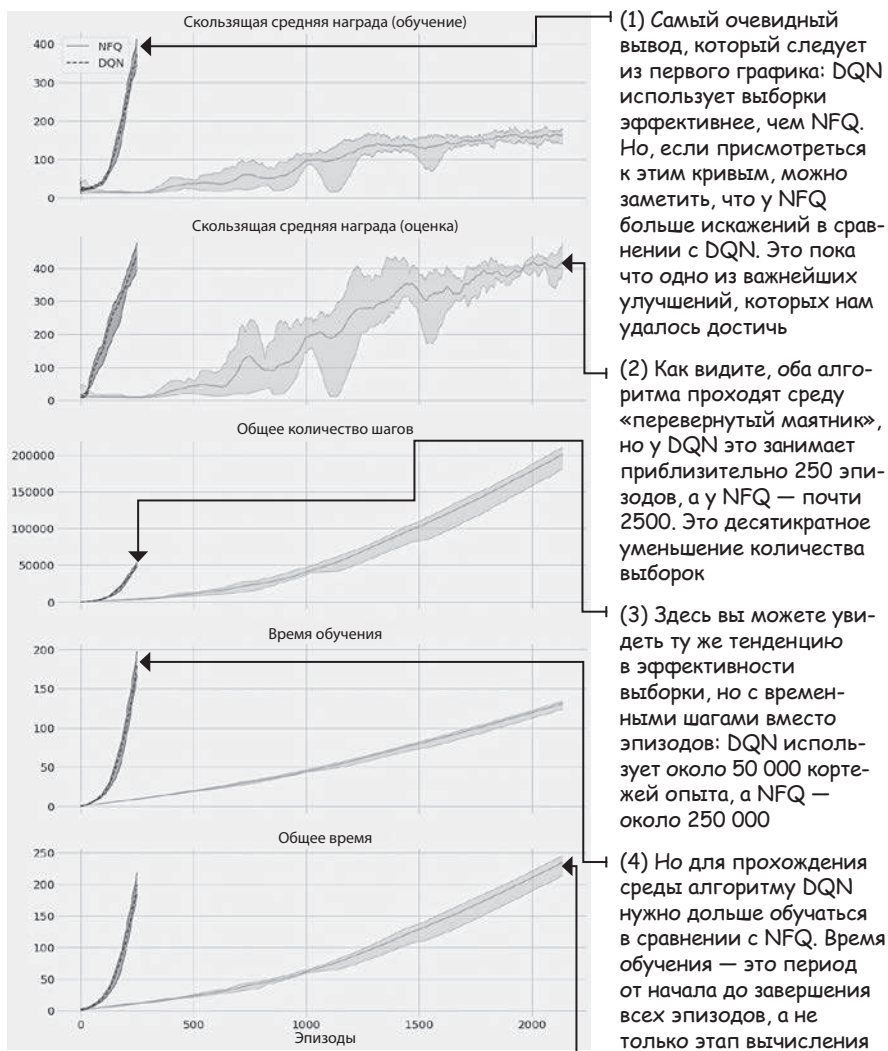
Влад защитил свою докторскую диссертацию под руководством Джеффри Хинтона (Geoffrey Hinton) (одного из отцов глубокого обучения) и сейчас работает научным сотрудником в Google DeepMind. Он получил признание за изобретение DQN, а в 2017 году его включили в список 35 лучших изобретателей по версии MIT Technology Review.



Посчитаем

DQN проходит среду «перевернутый маятник»

Самая примечательная особенность этих результатов — то, что для выполнения задачи алгоритму NFQ нужно намного больше выборок, чем DQN, последний более эффективен в этом отношении. Но и тот и другой тратят примерно одинаковое количество времени как на обучение (вычисление), так и в целом.



(5) Что касается общего времени (которое включает время обучения, вычисление статистики, шаги значений и т. д.), то оба алгоритма тратят около пяти минут

Двойная DDQN: борьба с завышением прогнозов функций ценности действий

В этом разделе представлено одно из главных улучшений DQN за годы существования этого алгоритма — *двойные глубокие Q-сети* (двойные DQN, или DDQN). Это двойное обучение нашего агента DQN. Его легко реализовать, и в результате получают агенты, по своей производительности стабильно превосходящие DQN. Важные изменения похожи на те, которые мы применяли для получения двойного Q-обучения, но с несколькими различиями, которые нужно обсудить.

Проблема завышения прогнозов, вторая попытка

Как вы помните из главы 6, Q-обучению свойственно завышать прогнозы функций ценности действий. У нашего агента DQN та же проблема, так как мы используем ту же нестратегическую цель TD с оператором *max*. Суть проста: мы берем максимальное из ожидаемых значений. Ожидаемые значения обычно смещены: некоторые оказываются больше реальных, некоторые меньше, но так или иначе они отклоняются. Поскольку мы всегда выбираем максимальное из них, мы начинаем склоняться к завышению значений, даже если они неправильные. У нашего алгоритма положительное смещение, что плохо сказывается на его производительности.



Аналогия от Мигеля

Проблема излишнего оптимизма

Мне нравились суперпозитивные люди, пока я не узнал о двойной DQN. Нет, правда, представьте, что вы встретили крайне оптимистичного человека — назовем его DQN. Итак, DQN полон оптимизма. У него богатый жизненный опыт, включающий серьезные неудачи и огромный успех. Но у DQN есть одна особенность: за что бы он ни брался, он всегда надеется на самый положительный исход. Плохо это или хорошо?

Однажды DQN решил сходить в местное казино. Усевшись за игровой автомат, он сразу же сорвал джекпот. С присущим ему оптимизмом DQN немедленно откорректировал свою функцию ценности, размышляя так: «Посещение казино — довольно прибыльное занятие (стало быть, у $Q(s, a)$ высокое значение), так как любой может подойти к автомату (следующее состояние s') и получить все деньги [$\max_a' Q(s', a')$]».

Но у такого мышления есть и минусы. Для начала, DQN не играет в автоматы при каждом посещении казино. Он любит пробовать (исследовать) что-то новое: рулетку, покер, блек-джек (пробует разные действия). Иногда зал с автоматами закрыт на ремонт (среда переносит его в другое место). Кроме того, в большинстве случаев игровые автоматы не приносят DQN ожидаемой награды (среда стохастическая). Не зря их называют бандитами — не теми, о которых вы подумали, а другими... Впрочем, неважно.

Разделение выбора и оценки действий

Чтобы понять, что такое положительное смещение и как с ним бороться при аппроксимации функций, можно обсудить оператор *max*, который применяется в вычислении цели. *max* Q-функции — это то же самое, что Q-функция действия *argmax*.



Освежим в памяти

Напомните, что такое *argmax*?

Функция *argmax* — это аргументы максимизации. Значение *argmax* функции ценности действий, *argmax* Q-функции, $\text{argmax}_a Q(s, a)$ — это индекс действия с максимальным значением в заданном состоянии *s*.

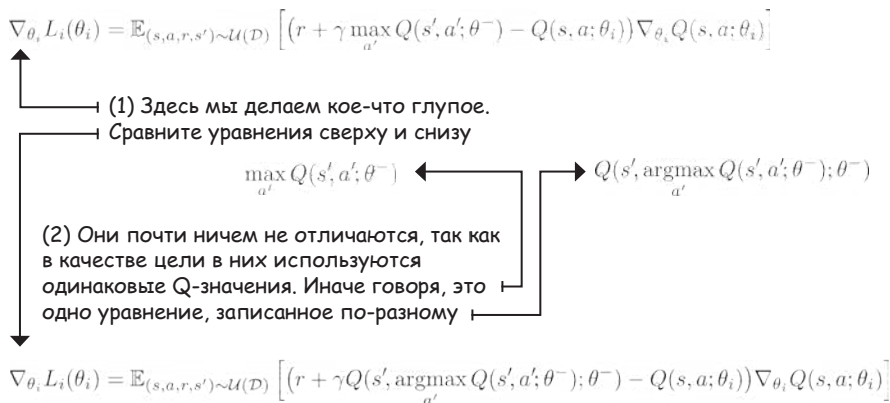
Например, если у вас есть $Q(s)$ со значениями $[-1; 0; -4; -9]$ для действий 0–3, $\text{max}_a Q(s, a)$ будет 0 (максимальная оценка), а $\text{argmax}_a Q(s, a)$ — 1 (индекс максимального значения).

Разберем предыдущее предложение о *max* и *argmax*. Обратите внимание, что при переходе от обычного Q-обучения к двойному мы совершали почти те же изменения. Но здесь речь идет об аппроксимации функций, поэтому нам нужно действовать осторожно. На первый взгляд этот разбор может показаться глупым, но он поможет понять, как бороться с проблемой.



Я знаю математику

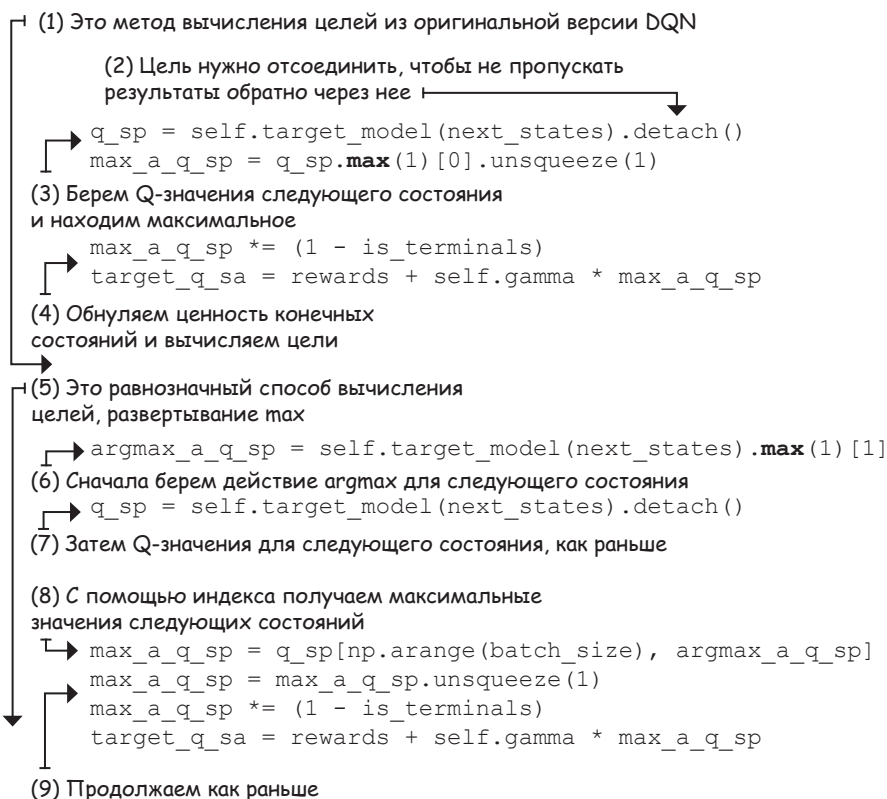
Развертывание *argmax*





Я знаю Python

Развертывание *max* в DQN



Все это значит, что функция *max* фактически спрашивает сеть: «Какое значение у самого ценного действия в состоянии *s*?»

Но это два вопроса в одном. Сначала мы вычисляем *argmax*, словно спрашивая: «Какое действие самое ценное в состоянии *s*?» А затем используем ответ, чтобы получить саму ценность: «Каково значение этого действия (с самой высокой ценностью) в состоянии *s*?»

Одна из проблем в том, что мы задаем оба эти вопроса одной Q-функции, ответы которой смещены в одном и том же направлении. Проще говоря, аппроксиматор функций ответит: «Мне кажется, это действие самое ценное в состоянии *s*, и вот его значение».

Решение

Чтобы снизить вероятность положительного смещения, можно использовать два экземпляра функции ценности действий, как в главе 6.

Если у вас несколько источников прогнозов, можете задать один вопрос первому, а другой — второму. Это чем-то напоминает голосование, или разрезание торта: «Я режу, ты выбираешь», или получение дополнительной консультации у другого врача.

В двойном обучении один источник прогнозов выбирает индекс действия, которое он считает самым ценным, а другой присваивает этому действию значение.



Освежим в памяти

Процедура двойного обучения

Мы выполняли эту процедуру в рамках табличного RL в главе 6 с помощью агента двойного Q-обучения:

- создаем две функции ценности действий — Q_A и Q_B ;
- подбрасываем монету, чтобы решить, какую из них нужно обновить. Например, Q_A , если выпадет орел, и Q_B , если решка;
- если выпал орел, обновляем Q_A . Выбираем из Q_B индекс действия, которое нужно оценить, и *оцениваем* его с помощью прогноза из Q_A . Затем, как обычно, переходим к обновлению Q_A и оставляем Q_B в покое;
- если выпала решка, обновляем Q_B , делая все наоборот. Выбираем из Q_A индекс действия, которое нужно оценить, и *оцениваем* его с помощью прогноза из Q_B . Затем обновляем Q_B и оставляем Q_A в покое.

Но если в точности повторить эту же процедуру, то при использовании аппроксимации функций (для DQN) возникнут ненужные издержки. У нас получится четыре сети: две динамические для обучения (Q_A , Q_B) и еще две целевые, по одной для Q_A и Q_B .

Кроме того, это замедляет процесс обучения: сети будут обучаться по очереди, поэтому на каждом шаге будет улучшаться только одна из них. Это, несомненно, расточительство.

Все же выполнить эту процедуру лучше, чем не выполнять ее вообще, несмотря на дополнительные накладные расходы. Но, к счастью, у нее есть более практичный вариант, который применяется в DQN и приносит существенное улучшение без траты лишних ресурсов.

Более практичное решение

Вместо того чтобы смириться с накладными расходами, которые плохо сказываются на скорости обучения, мы можем воспользоваться другой, целевой сетью, ведь она у нас уже есть. Она будет применяться лишь для перекрестной проверки прогнозов, а динамическая — только для обучения.

Выбирать, какая сеть должна заниматься выбором действий, а какая — их оценкой, нужно осторожно. Изначально мы добавили целевую сеть для стабилизации обучения за счет отказа от преследования движущейся цели. Чтобы продвинуться в этом направлении, важно убедиться, что для ответа на первый вопрос используется сеть, которую мы обучаем (то есть динамическая). Проще говоря, сначала, для получения индекса лучшего действия, мы используем динамическую сеть, а затем, для оценки выбранного ранее, — целевую.

Именно в таком порядке, ведь это логично. Используя целевую сеть для оценки прогнозов, мы получаем нужную нам стабильность за счет того, что целевые значения заморожены. Если бы мы реализовали все наоборот, значения брались бы из динамической сети, которая постоянно меняется, обновляясь на каждом временном шаге.

Выбор действия, его оценка



0001 Немного истории

Появление алгоритма DDQN

Алгоритм DDQN был предложен в 2015 году Хаддо ван Хасселтом (Hado van Hasselt) вскоре после выхода оригинальной версии DQN 2015 года (иногда ее называют Nature DQN, потому что она была опубликована в научном журнале Nature, а иногда — Vanilla DQN, так как это было первое из длинного ряда улучшений, внесенных впоследствии).

В 2010 году Хаддо разработал алгоритм двойного Q-обучения, который состоял в применении двойного обучения к табличному RL. Его обсуждением и реализацией мы занимались в главе 6.

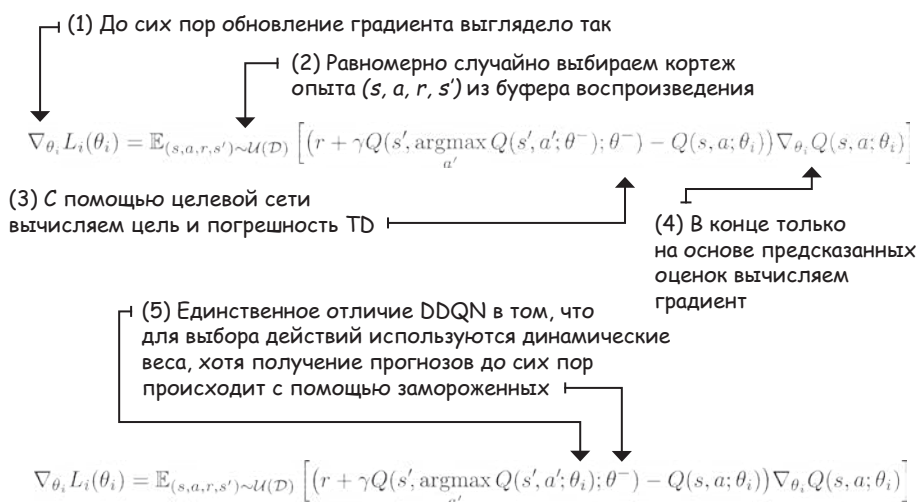
DDQN (Double DQN — двойная Q-сеть) — это первая из множества модификаций, предложенных за годы до улучшения DQN. В 2015 году, когда она только появилась, ей удалось показать лучшие на тот момент результаты в области задач на основе игр Atari.

Хаддо защитил свою докторскую диссертацию по искусственному интеллекту в Утрехтском университете (Нидерланды). Спустя несколько лет он стал научным сотрудником в Google DeepMind.



Я знаю математику

Обновление градиента DDQN





Я знаю Python

DDQN

```
def optimize_model(self, experiences):
    states, actions, rewards, \
        next_states, is_terminals = experiences
    batch_size = len(is_terminals)
```

(1) Здесь мы используем динамическую сеть для получения индекса самого ценного действия в следующем состоянии (`argmax`). Заметьте, мы не отсоединяем значения `argmax`, так как их нельзя дифференцировать. `max(1)[1]` возвращает уже отсоединенный индекс `max`

```
        #argmax_a_q_sp = self.target_model(next_states).max(1)[1]
    argmax_a_q_sp = self.online_model(next_states).max(1)[1]
```

(2) Извлекаем Q-значения следующего состояния, предоставленные целевой сетью

```
    q_sp = self.target_model(next_states).detach()
```

(3) Выбираем Q-значение по индексу, предоставленному динамической сетью

```
    max_a_q_sp = q_sp[np.arange(batch_size), argmax_a_q_sp]
```

(4) Как обычно, подготавливаем цели

```
    max_a_q_sp = max_a_q_sp.unsqueeze(1)
    max_a_q_sp *= (1 - is_terminals)
    target_q_sa = rewards + (self.gamma * max_a_q_sp)
```

(5) Получаем текущие прогнозы. Обратите внимание, что здесь проходят градиенты

```
    q_sa = self.online_model(states).gather(1, actions)
    td_error = q_sa - target_q_sa
    value_loss = td_error.pow(2).mul(0.5).mean()
    self.value_optimizer.zero_grad()
    value_loss.backward()
    self.value_optimizer.step()
```

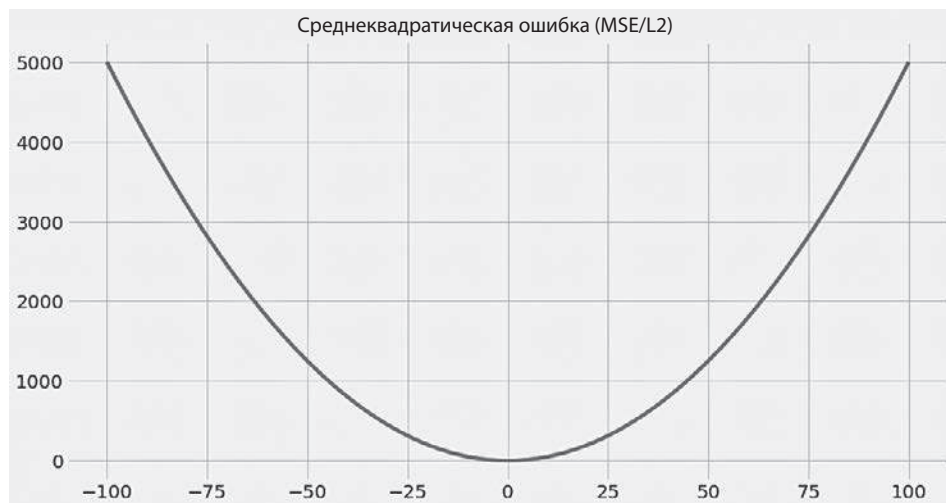
(6) Вычисляем потери и переводим оптимизатор на следующий шаг

```
def interaction_step(self, state, env):
    action = self.training_strategy.select_action(
        self.online_model, state)
    (7) Продолжаем использовать динамическую сеть для выбора действий
    new_state, reward, is_terminal, _ = env.step(action)
    return new_state, is_terminal

def update_network(self):
    (8) Процесс обновления целевой
    for target, online in zip(
        self.target_model.parameters(),
        self.online_model.parameters()):
        target.data.copy_(online.data)
```

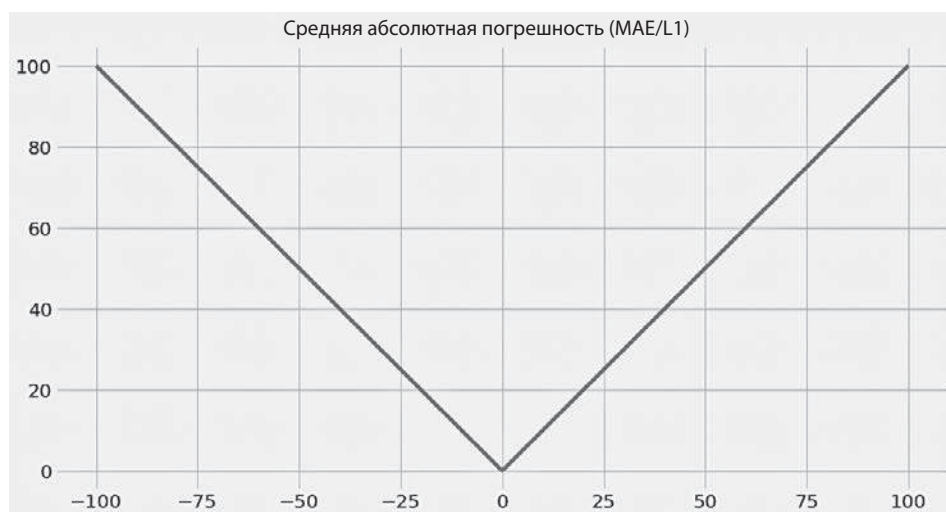

Менее прихотливая функция потерь

Ранее мы использовали в качестве нашей функции потерь метод L2, известный как *среднеквадратическая ошибка* (MSE). Все из-за его распространенности и простоты. Для задач вроде среды «перевернутый маятник» это оптимальный вариант. Но я пытаюсь научить вас всем тонкостям алгоритмов, не ограничиваясь простыми примерами их применения. Вам нужно знать о разных доступных вариантах, которые помогут выполнять более сложные задачи.



Широкое использование функции MSE объясняется ее простотой, логичностью и результативностью. Но в контексте обучения с подкреплением у нее есть одна проблема: она реагирует на крупные погрешности сильнее, чем на мелкие. В контролируемом обучении это оправданно, ведь там изначально берутся реальные значения, которые остаются неизменными до конца. Это значит, что за совершенно неправильную модель агент должен быть «наказан» сильнее, чем просто за неточную.

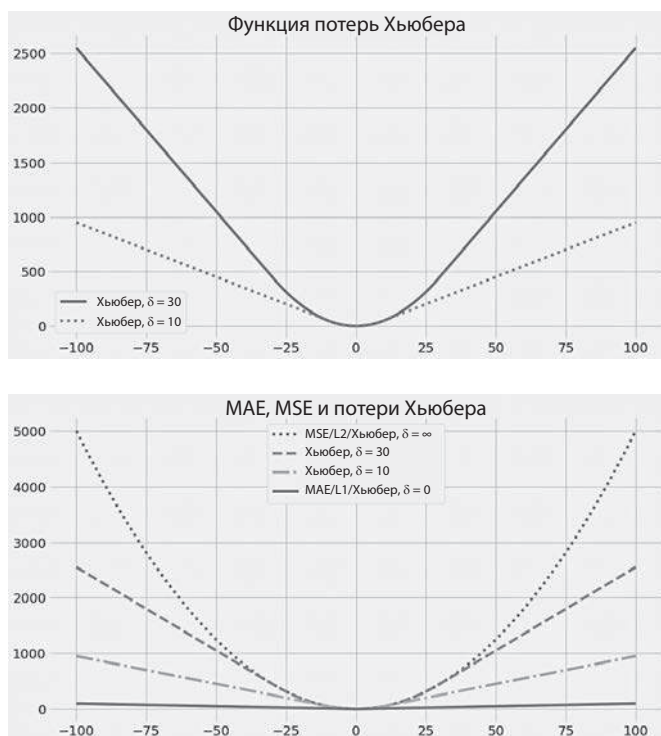
Но, как уже не раз упоминалось, в обучении с подкреплением у нас нет этих реальных значений, а те, которые мы используем для обучения нашей сети, зависят от самого агента. Это в корне меняет наш взгляд на вещи. К тому же цели постоянно меняются, даже при использовании целевых сетей. В обучении с подкреплением большие ошибки — это ожидаемое явление, которое мы приветствуем. Если подумать, то мы не обучаем агенты: они учатся сами по себе.



Есть и другая, не настолько прихотливая функция потерь, которая лучше справляется с резко отклоняющимися значениями, — *средняя абсолютная погрешность*, также известная как MAE или потери L1. Это средняя абсолютная разность между спрогнозированными и реальными значениями, то есть между ожидаемой функцией ценности действий и целью TD. Учитывая, что MAE линейная, а не квадратичная функция, как MSE, мы можем ожидать, что у нее лучше получится обрабатывать крупные ошибки тем же способом, что и мелкие. В нашем случае это может пригодиться, так как во время обучения наша функция ценности действий должна возвращать неправильные прогнозы, особенно вначале. Лучшая устойчивость к резким отклонениям зачастую приводит к тому, что ошибки из-за изменений сети не так сильно влияют на MAE, как на MSE. Это стабилизирует процесс обучения.

С другой стороны, одна из полезных особенностей MSE, которой нет в MAE, в том, что ее градиент затухает по мере приближения потерь к нулю. Это положительно сказывается на методах оптимизации, помогая достигнуть оптимума: низкие градиенты означают меньшие изменения в сети. Но, к счастью для нас, есть нечто среднее между MSE и MAE — функция потерь Хьюбера.

Потери Хьюбера, как и MSE, квадратично «наказывают» агент за ошибки, близкие к нулю. Но для крупных эта функция не полностью квадратична. Квадратичность (закругленность) есть там, где ошибки почти нулевые, а для тех, значение которых больше заранее определенного порогового, эта функция становится линейной (прямой). Наличие этих двух полезных свойств делает потери Хьюбера устойчивыми к резким отклонениям, как MAE, и дифференцируемыми рядом с 0, как MSE.



Функция потерь Хьюбера использует гиперпараметр δ , определяющий пороговое значение, по достижении которого потери превращаются из квадратичных в линейные (по сути, это переход от MSE к MAE). Если δ нулевой, вы получаете MAE, а если равен бесконечности — MSE. Обычно параметру δ присваивают значение 1, но имейте в виду, что ваши функция потерь, метод оптимизации и темпы обучения неочевидно взаимосвязаны. Если изменить что-то одно, вам придется корректировать и другие компоненты. Можете поэкспериментировать с блокнотом Jupyter для этой главы.

Любопытно, что функцию потерь Хьюбера можно реализовать как минимум двумя разными способами. Вы можете либо вычислить ее, как определено, либо вычислить потери MSE, а затем установить для всех градиентов, превышающих пороговое значение, фиксированное значение величины. Мы ограничиваем магнитуду градиентов. Первый способ зависит от того, какую платформу глубокого обучения вы используете, но проблема в том, что некоторые не дают доступа к гиперпараметру δ , тогда δ будет равен 1, что не всегда подходит и не всегда лучший вариант из возможных. Второй способ, который часто называют *ограничением потерь* или *ограничением градиентов*, отличается большей гибкостью, поэтому в блокноте я реализовал именно его.



Я знаю Python

DDQN с потерями Хьюбера

```
def optimize_model(self, experiences):
    states, actions, rewards, \
        next_states, is_terminals = experiences
    batch_size = len(is_terminals)

    (1) Сначала вычисляем цели и получаем фактические
        значения, как мы делали до использования двойного обучения
    <...>
    td_error = q_sa - target_q_sa

    (2) Как и прежде, вычисляем функцию
        потерь в виде среднеквадратической ошибки
    value_loss = td_error.pow(2).mul(0.5).mean()

    (3) Обнуляем оптимизатор и вычисляем
        градиенты, выполняя шаг назад
    self.value_optimizer.zero_grad()
    value_loss.backward()

    (4) Ограничиваем градиенты с помощью параметра max_gradient_norm.
        Он может принимать почти любое значение, но помните, что он влияет
        на другие гиперпараметры, такие как темп обучения
    torch.nn.utils.clip_grad_norm_(
        self.online_model.parameters(),
        self.max_gradient_norm)

    (5) В конце переводим оптимизатор на следующий шаг
    self.value_optimizer.step()
```

Важно отметить, что есть такое понятие, как *ограничение награды*, которое отличается от *ограничения градиента*. Это разные вещи: в одном случае мы имеем дело с наградой, а в другом — с погрешностями (потерями). Главное, не перепутать эти понятия с *ограничением Q-значений*, что, несомненно, будет ошибкой.

Помните, что мы пытаемся предотвратить чрезмерное увеличение градиентов. Для этого мы либо делаем потери линейными за пределами абсолютной пороговой погрешности TD, либо делаем градиент постоянным за пределами максимальной пороговой магнитуды.

В экспериментах со средой «перевернутый маятник» (их можно найти в блокноте) я реализовал потери Хьюбера через ограничение градиентов: вычислил MSE и затем ограничил градиенты. Но, как уже упоминалось, я сделал гиперпараметр, определяющий максимальные значения градиентов, бесконечным. Поэтому все фактически свелось к использованию старой доброй функции MSE. Но не стоит на этом останавливаться — экспериментируйте, исследуйте! Блокноты, которые я для вас подготовил, так же важны для учебного процесса, как и сама книга. Используйте их по своему усмотрению.



В деталях

Полный алгоритм двойной Q-сети (DDQN)

Алгоритмы DDQN и DQN почти идентичны, но между ними все же есть несколько различий. У DQN есть следующие особенности:

- аппроксимация функции ценности действий $Q(s, a; \theta)$;
- использование архитектуры «состояние на входе, значения на выходе» (узлы: 4, 512 128, 2);
- оптимизация Q-функции для аппроксимации оптимума, $q^*(s, a)$;
- использование нестратегических целей TD ($r + \gamma \max_{a'} Q(s', a'; \theta)$) для оценки политик.

Обратите внимание, что сейчас мы:

- используем регулируемую функцию потерь Хьюбера, которая в нашем случае фактически представляет среднеквадратическую ошибку (MSE), так как мы присвоили переменной `max_gradient_norm` значение `float('inf')`;
- применяем в качестве оптимизатора RMSprop с темпом обучения 0,0007. Заметьте, что раньше темп был равен 0,0005, так как в обычном DQN (без двойного обучения) при выборе темпа 0,0007 несколько начальных значений оказываются неудачными. А вот в DDQN лучшие результаты дает повышенный темп.

Но в DDQN мы по-прежнему используем:

- экспоненциально затухающую эпсилон-жадную стратегию (от 1,0 до 0,3 примерно за 20 000 шагов) на этапе улучшения;
- буфер воспроизведения размером от 320 до 50 000 выборок и с набором размером 64;
- целевую сеть, которая замораживается на 15 шагов, после чего полностью обновляется.

DDQN состоит из тех же трех основных этапов, что и DQN, таких как:

- сбор кортежа опыта $(S_t, A_t, R_{t+1}, S_{t+1}, D_{t+1})$ и добавление его в буфер воспроизведения;
- выбор из буфера случайного мини-набора и вычисление нестратегических целей TD для всего набора данных: $r + \gamma \max_{a'} Q(s', a'; \theta)$;
- корректировка функции ценности действий $Q(s, a; \theta)$ с помощью MSE и RMSprop.

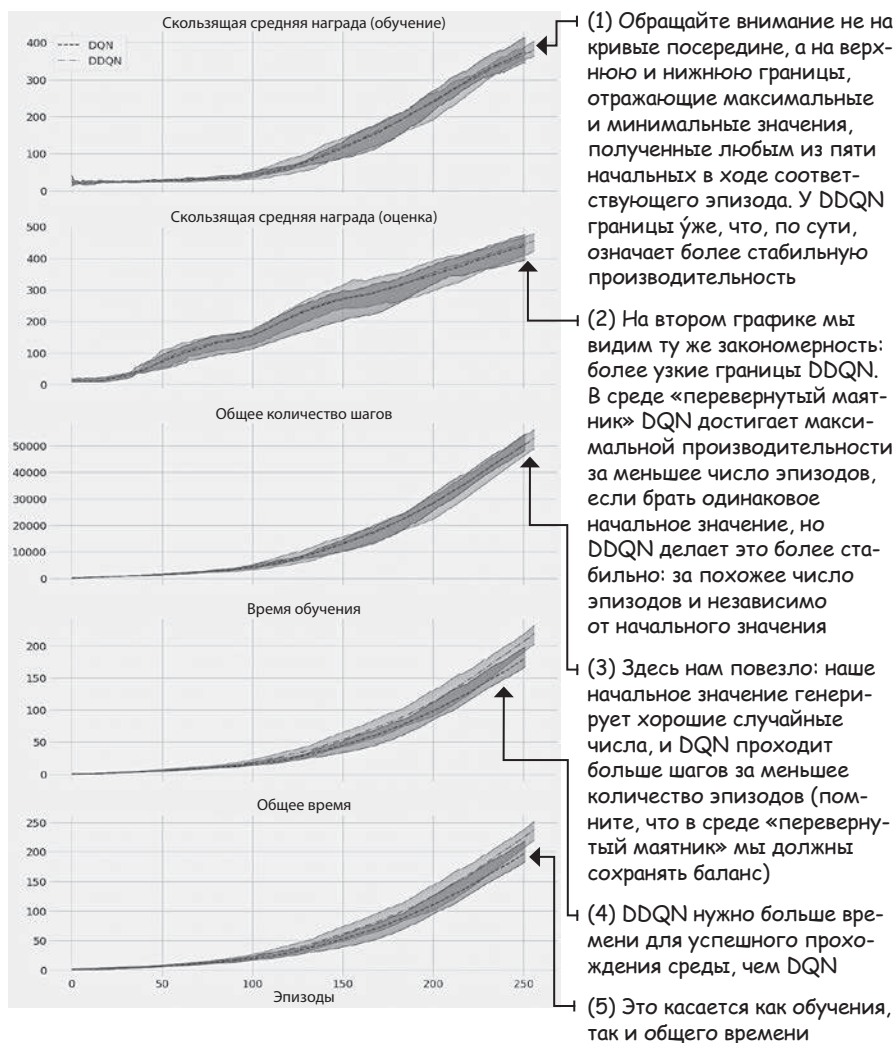
Как итог, у DDQN те же реализация и гиперпараметры, что и у DQN, только теперь мы используем двойное обучение, у которого чуть более высокий темп. Внедрение функции потерь Хьюбера ничего не меняет, так как мы «ограничиваем» градиенты максимальным значением, равным бесконечности, что равнозначно использованию MSE. Но во многих средах эта функция может пригодиться, поэтому не забывайте регулировать этот параметр.



Посчитаем

Алгоритм DDQN более стабилен по сравнению с NFQ и DQN

В среде «перевернутый маятник» производительность DQN и DDQN похожа. Но это простая среда с плавной функцией вознаграждения. В реальных задачах алгоритм DDQN должен всегда показывать лучшие результаты.



Что еще можно улучшить

Наш текущий метод ценностно ориентированного глубокого обучения неидеален, но он довольно неплох. DDQN превосходит человека во многих играх Atari. Чтобы воспроизвести эти результаты, нам пришлось бы изменить сеть так, чтобы она принимала в качестве ввода изображения (стек из четырех изображений, чтобы выводить из них направление и скорость движения), и откорректировать гиперпараметры.

Но если мы пойдем дальше, у нас останется как минимум еще несколько потенциальных улучшений, простых в реализации и положительно влияющих на производительность.

Для начала нам нужно пересмотреть текущую архитектуру сети. Сейчас наше представление Q -функции упрощенное.

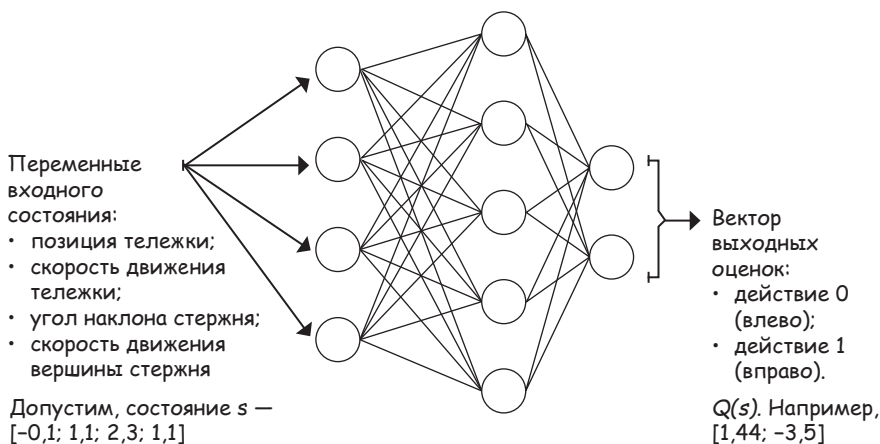


Освежим в памяти

Текущая архитектура нейронной сети

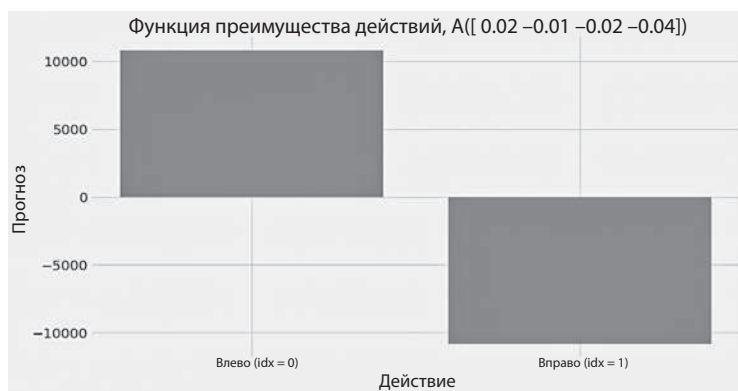
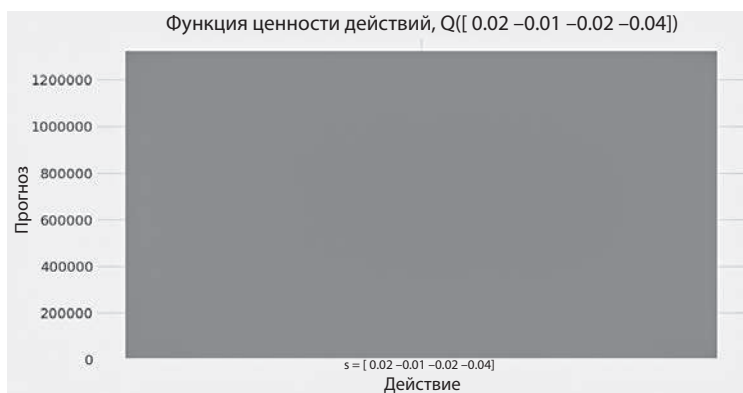
Мы буквально делаем обучение с подкреплением похожим на контролируемое. Но нам нужно освободиться от этого и мыслить нестандартно.

Архитектура «состояние на входе, значения на выходе»



Есть ли лучший способ представления Q -функции? Подумайте над этим, просматривая следующие диаграммы.

Ниже показаны гистограммы, отражающие ожидаемую функцию ценности действий Q , функцию ценности состояний V и функцию преимущества действий A для среды «перевернутый маятник» при условии, что стержень находится в положении, близком к вертикальному.



Взгляните на эти разные функции и значения и подумайте, как лучше всего спроектировать нейросеть, чтобы повысить эффективность использования данных. Подсказка: Q -значения в рамках одного состояния связаны между собой через V -функцию. То есть у функции ценности действий Q есть неразрывная связь с функцией ценности состояний V , так как оба действия в $Q(s)$ индексируются по одному состоянию s (в примере справа $s = [0,02; -0,01; -0,02; -0,04]$).

Вопрос в том, можно ли узнать что-то о $Q(s, 0)$ с помощью выборки $Q(s, 1)$? Посмотрите на диаграмму функции преимущества действий $A(s)$ и обратите внимание на то, насколько проще заметить в ней жадное действие для этих прогнозов в сравнении с диаграммой функции ценности действий $Q(s)$. Что нам это дает? В следующей главе мы рассмотрим архитектуру «*дуэльная сеть*» (dueling network), которая поможет использовать эти связи.

Еще один аспект, который можно улучшить, — наш способ выборки кортежей опыта из буфера воспроизведения. Сейчас мы делаем это равномерно случайно. Уверен, ваша интуиция подсказывает вам, что есть подход куда лучше. Это действительно так.

Люди не путешествуют по миру, беспорядочно запоминая факты в случайные моменты времени. Есть более методичный подход, в рамках которого разумные агенты «воспроизводят воспоминания». Я почти уверен, что мой пес гоняется во сне за кроликами. Разный опыт может иметь разное значение для достижения наших целей. Люди часто воспроизводят опыт, принесший им неожиданное удовольствие или боль. Это логично, и важно учитывать такой опыт, чтобы повторить его или избежать. Дальше мы рассмотрим способы определения приоритетов образца опыта для получения максимальной выгоды от каждого и познакомимся с приоритетным воспроизведением опыта (prioritized experience replay, PER).

Подведем итоги

В этой главе вы узнали о распространенных проблемах методов ценностно ориентированного глубокого обучения с подкреплением. То, что динамические данные нестационарны, независимы и равномерно распределены, как того ожидает большинство методов оптимизации, создает множество трудностей при использовании ценностно ориентированных подходов.

Вы научились стабилизировать методы ценностно ориентированного глубокого обучения с подкреплением разными способами, результаты которых подтверждены несколькими эмпирическими тестами. Мы подробно обсудили компоненты, которые стабилизируют ценностно ориентированные методы. В частности, вы узнали о преимуществах использования целевых сетей и буферов воспроизведения в алгоритме DQN (Nature DQN или Vanilla DQN). Вы убедились, что целевые сети делают цели неподвижными для оптимизатора, что положительно влияет на стабильность, хоть и в ущерб скорости схождения. Еще вы увидели, что буферы воспроизведения позволяют представить динамические данные так, будто они соответствуют IID, что служит источником

серьезных проблем в ценностно ориентированных методах бэггинга. Сочетание этих двух подходов делает алгоритм достаточно стабильным для эффективного решения нескольких задач DRL.

Но это далеко не предел совершенства для ценностно ориентированных методов. Вы реализовали довольно простое изменение, которые сильно повлияло на производительность в целом. Вы внедрили в агент DQN стратегию двойного обучения, что в сочетании с аппроксимацией функций дало нам алгоритм DDQN, который смягчает проблему слишком положительного прогнозирования в нестратегических ценностно ориентированных методах.

В дополнение к новым алгоритмам вы узнали о разных стратегиях исследования, которые можно использовать в ценностно ориентированных методах. Мы взяли уже знакомые нам экспоненциально затухающую эпсилон-жадную стратегию и softmax и применили их в контексте функций аппроксимации.

Мы рассмотрели разные функции потерь и узнали, какая из них лучше подходит для обучения с подкреплением и почему. Вы узнали, что функция потерь Хьюбера позволяет находить баланс между MSE и MAE с помощью единственного гиперпараметра и что она входит в число рекомендованных для использования в методах ценностно ориентированного глубокого обучения с подкреплением.

На этом этапе вы:

- понимаете, почему использование динамических данных для обучения нейросетей с оптимизаторами, рассчитанных на работы со стационарными, соответствующими требованиям IID, — проблема для ценностно ориентированных методов DRL;
- умеете выполнять задачи RL с непрерывными пространствами состояний с помощью более стабильных алгоритмов, дающих более однородные результаты;
- имеете представление о продвинутых методах ценностно ориентированного глубокого обучения с подкреплением и можете выполнять сложные задачи.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

- **#gdrl_ch09_tf01.** В этой и следующей главах мы тестируем алгоритмы только в среде «перевернутый маятник». Подберите несколько других и проверьте работу агентов в них. Например, среды «лунный посадочный модуль» и «горная

машина» можно найти по адресам <https://gym.openai.com/envs/#box2d> и https://gym.openai.com/envs/#classic_control. Внесли ли вы какие-то изменения (помимо гиперпараметров), чтобы адаптировать агенты для этих сред? Подберите единый набор гиперпараметров для решения всех задач: то есть вы используете один набор гиперпараметров и обучаете агент с нуля в каждой среде, а не один обученный агент, который хорошо работает во всех средах.

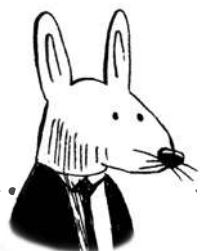
- **#gdr1_ch09_tf02.** В этой и следующей главах мы тестируем алгоритмы в непрерывных, но малоразмерных средах. Пример многомерной среды — игры Atari, доступные по адресу <https://gym.openai.com/envs/#atari> (те, в которых не упоминается RAM). Теперь попробуйте видоизменить сети, буфер воспроизведения и код алгоритмов из этой главы, чтобы агенты смогли проходить среды, основанные на изображениях. Это непростая задача, и обучение займет некоторое время, от многих часов до нескольких дней.
- **#gdr1_ch09_tf03.** Я уже писал, что ценностно ориентированные методы чувствительны к гиперпараметрам. На самом деле есть так называемый принцип «смертельной триады», согласно которому нейросети нельзя использовать вместе с бэггингом и нестратегическими методами. Изучите этот вопрос!
- **#gdr1_ch09_tf04.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdr1_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы реализуете архитектуру глубокой нейросети, использующей некоторые особенности методов ценностно ориентированного глубокого обучения с подкреплением.
- ✓ Создадите буфер воспроизведения, ранжирующий выборки в зависимости от их неожиданности.
- ✓ Разработаете агент, формирующий политику, близкую к оптимальной, за меньшее количество эпизодов, чем все агенты ценностно ориентированного глубокого обучения с подкреплением до этого.

Интеллект заключается в том, насколько эффективно вид делает то, что ему нужно для выживания.

*Чарльз Дарвин (Charles Darwin),
английский натуралист, геолог и биолог,
наиболее известен своим вкладом
в исследование эволюции*

Ранее мы занимались улучшением NFQ за счет реализации DQN и DDQN. Здесь мы продолжим оптимизировать рассмотренные ранее алгоритмы с помощью двух дополнительных подходов к улучшению методов ценностно ориентированного глубокого обучения с подкреплением. Но на этот раз в центре нашего внимания будет не стабильность (хотя она вполне может оказаться побочным эффектом), а эффективность использования выборок в DQN и других ценностно ориентированных методах DRL.

Сначала мы рассмотрим архитектуру функциональной нейросети, где представление Q-функции разделено на два потока: один аппроксимирует V-функцию, а другой — A-функцию. V-функции — это значения отдельных состояний, тогда как A-функции выражают расстояние от каждого действия к его V-функции.

Это полезно знать при разработке специализированных RL-архитектур, способных извлекать нужную информацию из выборок, генерируемых всеми действиями в заданном состоянии, и представлять ее в виде V-функции для этого состояния. Это значит, что отдельный кортеж опыта может помочь улучшить прогнозы значений всех действий в этом состоянии, что позволяет агенту эффективнее использовать выборки.

Второе улучшение, которое мы рассмотрим, относится к буферу воспроизведения. Как вы помните, стандартный такой буфер в DQN собирает выборки равномерно случайно. Важно понимать, что такие выборки помогают делать градиенты пропорциональными исходному распределению для генерации данных, благодаря чему обновления не смещаются. Но если бы у нас был метод назначения выборкам приоритетов, мы могли бы использовать наиболее перспективные из них для обучения. Итак, в этой главе мы познакомимся с новым методом сбора данных, позволяющим генерировать выборки, которые предположительно дают агенту максимальное количество информации для внесения реальных улучшений.

Дуэльная DDQN: архитектура нейросети, рассчитанная на обучение с подкреплением

Давайте теперь подробно обсудим эту специализированную архитектуру нейронной сети. *Дуэльная* сеть — это улучшение, которое относится только к архитектуре сети, но не к алгоритму: никаких изменений в алгоритм мы вносить не будем — наше внимание сосредоточено только на архитектуре. Это позволяет нам сочетать дуэльные сети с практически любыми улучшениями, предложенными за годы существования алгоритма DQN. Например, мы можем создать дуэльный агент DQN, двойной дуэльный агент DQN (или, как я его называю, дуэльную сеть DDQN) и т. д. Многие из этих модификаций внедряются без усилий, и в ближайшее время мы этим воспользуемся. Теперь реализуем дуэльную архитектуру, которая будет применяться в наших экспериментах, и попутно ее изучим.

Обучение с подкреплением отличается от контролируемого

В предыдущей главе мы сосредоточили наши усилия на том, чтобы сделать обучение с подкреплением больше похожим на контролируемое. За счет использования буфера воспроизведения мы сделали динамические данные, которые последовательно находил и собирал агент, похожими на независимый и равномерно распределенный набор данных, аналогичный тем, что часто встречаются в контролируемом обучении.

Мы сделали цели внешне более статическими, что тоже свойственно задачам контролируемого обучения. Это помогло стабилизировать процесс обучения, но задачи обучения с подкреплением — это отдельная категория, и при их выполнении этот факт лучше не игнорировать.

У агентов ценностно ориентированного глубокого обучения с подкреплением есть одно неочевидное свойство, которым мы воспользуемся, — то, как функции ценности зависят друг от друга. Мы можем извлечь пользу из того, что функции ценности состояний $V(s)$ и действий $Q(s, a)$ связаны между собой через функцию преимущества действий $A(s, a)$.



Освежим в памяти

Еще раз о функциях ценности

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

↑ (1) Напомню, что функция ценности действий политики — это ожидаемая выгода при выборе действия a в состоянии s с учетом продолжения следования этой политике

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

↑ (2) Функция ценности состояния s для заданной политики — это выгода, которую мы ожидаем получить в этом состоянии, продолжая следовать этой политике

$$a_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$$

↑ (3) Функция преимущества действий описывает разность между выбором действия a в состоянии s и выбором действия, по умолчанию предоставляемого политикой

$$\mathbb{E}_{a \sim \pi(s)} [a_{\pi}(s, a)] = 0$$

↑ (4) Бесконечная выборка политики для пары «состояние — действие» дает 0. Почему? Потому что нет никакого преимущества в выполнении действия по умолчанию

$$q_{\pi}(s, a) = v_{\pi}(s) + a_{\pi}(s, a)$$

← (5) Наконец, это то же уравнение преимущества, но записанное по-другому. Мы воспользуемся им чуть позже

Особенности методов ценностно ориентированного глубокого обучения с подкреплением

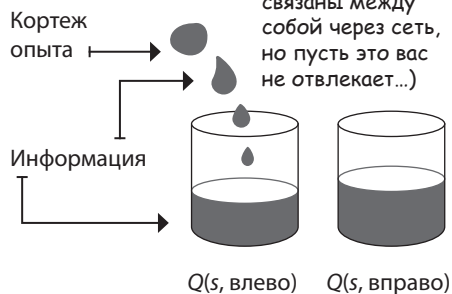
Функция ценности действий $Q(s, a)$ — это сумма функций ценности состояний $V(s)$ и преимущества действий $A(s, a)$. Значит, мы можем разложить Q -функцию на две части: одна общая для всех действий, а другая уникальна для каждого из них. Проще говоря, одна часть зависит от действия, а другая — нет.

Сейчас мы формируем Q -функцию отдельно для каждого действия, но это неэффективно. Конечно, при этом происходит небольшое обобщение, так как внутренние узлы сетей взаимосвязаны, поэтому информация разделяется между ними. Но при формировании $Q(s, a_1)$ мы игнорируем то, что та же информация может использоваться при формировании $Q(s, a_2)$, $Q(s, a_3)$ и остальных действий, доступных в состоянии s . Суть в том, что функция $V(s)$ общая для всех действий $a_1, a_2, a_3 \dots a_N$.

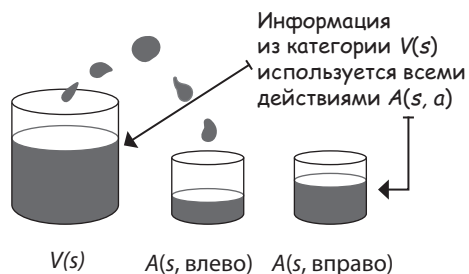
Эффективное использование выборок

(1) Аппроксимируя Q -функцию напрямую, мы извлекаем информацию из каждой выборки и помещаем ее в одну категорию

(Формально эти категории связаны между собой через сеть, но пусть это вас не отвлекает...)



(2) Повысить точность сети можно быстрее, если создать два отдельных потока: один для сбора общей информации ($V(s)$), а другой — для сбора различий между действиями $A(s, a_1)$ и $A(s, a_2)$



По сути

Функция ценности действий $Q(s, a)$ зависит от функции ценности состояний $V(s)$

Все сводится к тому, что ценность действий зависит от ценности состояний. Было бы неплохо этим воспользоваться. В конце концов, выбор худшего действия в хорошем состоянии лучше, чем выбор оптимального действия в плохом. Видите, как ценность действий зависит от ценности состояний?

Дуэльная сетевая архитектура использует эту зависимость функции ценности действий $Q(s, a)$ от функции ценности состояний $V(s)$ так, что каждое обновление улучшает прогноз V -функции, общей для всех действий.

Выгода от использования преимуществ

Рассмотрим среду «перевернутый маятник». Когда стержень в вертикальном положении, ценность действий «влево» и «вправо» почти одинакова. В этом идеальном состоянии неважно, какое решение вы примете (для чистоты эксперимента допустим, что тележка находится строго посередине колеи и что все векторы движения равны 0).

Но если стержень наклонится на 10° вправо, выбор действия будет важен. В этом состоянии агенту лучше всего толкнуть тележку вправо, чтобы компенсировать наклон. А вот движение влево было бы плохим выбором: оно бы лишь увеличило наклон.

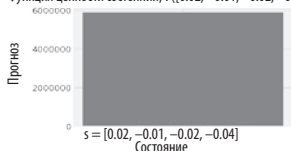
Именно это описывает функция преимущества действий $A(s, a)$: насколько лучшим будет выбор действия a в текущем состоянии s по сравнению со средним действием?

Связь между функциями ценности

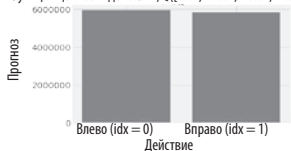
Состояние $s = [0.02, -0.01, -0.02, -0.04]$



Функция ценности состояний, $V([0.02, -0.01, -0.02, -0.04])$



Функция ценности действий, $Q([0.02, -0.01, -0.02, -0.04])$



Функция преимущества действий, $A([0.02, -0.01, -0.02, -0.04])$

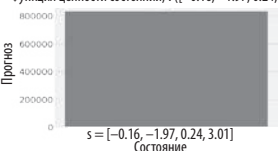


(1) Состояние слева довольно неплохое: стержень находится почти вертикально, а тележка — примерно по центру колеи. А вот состояние справа выглядит похуже: стержень заметно наклонен

Состояние $s = [-0.16, -1.97, 0.24, 3.01]$



Функция ценности состояний, $V([-0.16, -1.97, 0.24, 3.01])$



(2) Функция ценности состояний улавливает «положительность» ситуации: состояние слева в десять раз ценнее, чем справа (по крайней мере согласно хорошо обученному агенту)

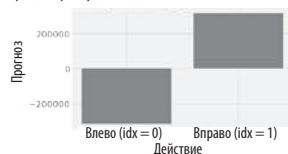
(3) Функция ценности действий не отражает эту связь напрямую, но помогает определить несколько благоприятных действий. Слева непонятно, что делать дальше, а справа очевидно, что тележку нужно двигать вправо

Функция ценности действий, $Q([-0.16, -1.97, 0.24, 3.01])$



(4) Функция преимущества действий тоже отражает эту «положительность». Но заметьте, что она намного четче выделяет благоприятные действия, чем функция ценности действий. Состояние слева довольно хорошо отражает это свойство

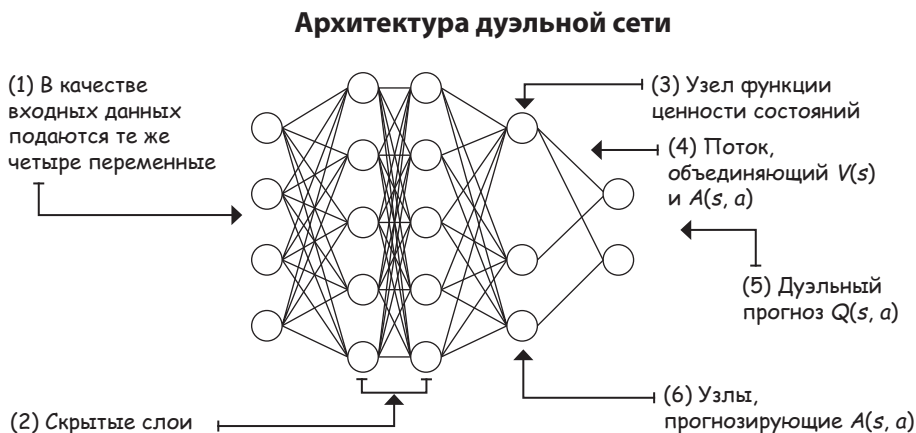
Функция преимущества действий, $A([-0.16, -1.97, 0.24, 3.01])$



Архитектура, рассчитанная на обучение с подкреплением

Архитектура дуэльной сети заключается в создании двух функций ценности: $V(s)$ и $A(s, a)$. Но перед разбиением сети важно убедиться, что она разделяет внутренние узлы. Например, если в качестве входных данных используются изображения, результаты свертывания должны разделяться, чтобы вместе с ними разделялись и слои для извлечения закономерностей. В среде «перевернутый маятник» мы разделяем скрытые слои.

Слой прямо перед выходными слоями делится на два потока: один для функции ценности состояний $V(s)$, другой для функции преимущества действий $A(s, a)$. Выходной слой V -функции всегда заканчивается одним узлом, так как значение состояния — это всегда отдельное число. А вот выходной слой Q -функции возвращает вектор, размер которого равен количеству действий. В среде «перевернутый маятник» выходной слой потока A -функции состоит из двух узлов, по одному для действий «влево» и «вправо».



0001

Немного истории

История создания архитектуры дуэльной сети

Архитектура дуэльной нейросети была предложена в 2015 году в научной работе *Dueling Network Architectures for Deep Reinforcement Learning*, опубликованной Зию Ваном (Ziyu Wang), который в то время проходил аспирантуру в Оксфордском университете. Это была, наверное, первая научная работа с описанием архитектуры глубокой нейронной сети, предназначенной специально для методов ценностно ориентированного глубокого обучения с подкреплением.

Сейчас Зию работает научным сотрудником в Google DeepMind, продолжая развивать область глубокого обучения с подкреплением.

Реализация дуэльной сети

Реализация дуэльной сети довольно проста. Я заметил, что ее можно разбить на два потока в любом месте после входного слоя, и все будет работать как положено. Можно даже представить себе конфигурацию с двумя сетями, но я не вижу в этом смысла. В целом я советую оставить общими как можно больше слоев и проводить разбиение только на два потока прямо перед выходным.



Я знаю Python

Реализация дуэльной сети

```
class FCDuelingQ(nn.Module):
    def __init__(self,
                  input_dim,
                  output_dim,
                  hidden_dims=(32, 32),
                  activation_fc=F.relu):
        super(FCDuelingQ, self).__init__()
        self.activation_fc = activation_fc
```

(1) Эта сеть похожа на обычную. Как и прежде, нам нужны переменные для количества узлов во входных и выходных слоях, форма скрытых слоев и функция активации

```
        self.input_layer = nn.Linear(input_dim,
                                      hidden_dims[0])
```

(2) Создаем входной слой и прикрепляем его к первому скрытому. Здесь `input_dim` — количество входных узлов, а `hidden_dims[0]` — количество узлов в первом скрытом слое. `nn.Linear` создает слой с вводом и выводом

```
        self.hidden_layers = nn.ModuleList()
        for i in range(len(hidden_dims)-1):
            hidden_layer = nn.Linear(
                hidden_dims[i], hidden_dims[i+1])
            self.hidden_layers.append(hidden_layer)
```

(3) Формируем скрытые слои в соответствии с переменной `hidden_dims`. Например, если она равна (64, 32, 16), будет создан слой с 64 входными узлами и 32 выходными, а после него слой с 32 входными и 16 выходными

```
        self.value_output = nn.Linear(hidden_dims[-1], 1)
        self.advantage_output = nn.Linear(
            hidden_dims[-1], output_dim)
```

(4) В конце создаем два выходных слоя, каждый из которых прикреплен к последнему скрытому. У `value_output` один выходной узел, а количество выходных узлов в `advantage_output` составляет `output_dim`. В среде «перевернутый маятник» это число равно 2

Реконструкция функции ценности действий

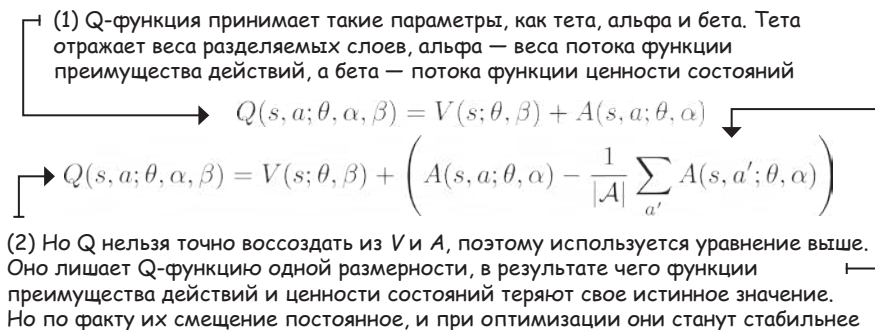
Хочу уточнить: дуэльная архитектура предназначена для создания новой сети, которая служит улучшенной версией предыдущей, но использует тот же метод управления. Наши изменения должны быть совместимы с ранее рассмотренными методами и не нарушать их работу. Мы хотим просто подставить новую нейросеть, и все.

Для этого нужно как-то агрегировать два полученных из сети вывода и воссоздать функцию ценности действий $Q(s, a)$, чтобы любой рассмотренный ранее метод мог использовать модель дуэльной сети. Так при использовании дуэльной архитектуры вместе с алгоритмом DDQN мы создаем агент DDQN, а из дуэльной сети и алгоритма DQN — агент DQN.



Я знаю математику

Уравнения агрегации дуэльной архитектуры



Но как объединить эти выходные значения? Кто-то из вас, наверное, думает, что их можно просто сложить. Это соответствовало бы определению, которое я представил. Но кто-то мог заметить, что $V(s)$ и $A(s, a)$ нельзя точно воссоздать только при наличии $Q(s, a)$. Судите сами: если прибавить число +10 к $V(s)$ и вычесть его из $A(s, a)$, получится та же функция $Q(s, a)$ с двумя разными значениями для $V(s)$ и $A(s, a)$.

В дуэльной архитектуре эта проблема решается путем вычитания среднего преимущества из агрегированного прогноза функции ценности действий $Q(s, a)$. В результате $V(s)$ и $A(s, a)$ смещаются на постоянную величину, а процесс оптимизации стабилизируется.

Несмотря на смещение прогнозов, относительное значение $A(s, a)$ не меняется, поэтому значение $Q(s, a)$ тоже корректное. И все это с использованием того же алгоритма управления. Это большой шаг вперед.



Я знаю Python

Прямое прохождение дуэльной сети

```
class FCDuelingQ(nn.Module):
    <...>
    def forward(self, state):
```

(1) Это тот же класс, который использовался прежде. Я убрал код построения сети для краткости

(2) При прямом проходе мы в первую очередь убеждаемся, что у ввода сети `state` подходящие тип и размерность. Так делается из-за того, что при обучении вводятся наборы состояний, а при взаимодействии — одиночные состояния. Иногда ввод может быть в виде векторов NumPy

```
        x = state
        if not isinstance(x, torch.Tensor):
            x = torch.tensor(x,
                              device=self.device,
                              dtype=torch.float32)
            x = x.unsqueeze(0)
```

(3) Итак, мы привели входную переменную `x` (опять же одиночное состояние или их набор) в тот вид, на который рассчитывает сеть. Далее передаем `x` входному слою, который принимает переменные `input_dim` и выводит `hidden_dim[0]`, которые будут пропущены через функцию активации

```
        x = self.activation_fc(self.input_layer(x))
```

(4) Этот вывод используется как ввод для первого скрытого слоя. Мы последовательно передаем переменную `x` каждому скрытому слою и функции активации. Эту переменную можно считать текущим состоянием пульсовой волны, которая проходит сеть насквозь, от входа к выходу

```
        for hidden_layer in self.hidden_layers:
            x = self.activation_fc(hidden_layer(x))
```

(5) Теперь у `x` есть значения, полученные из последнего скрытого слоя и его функции активации. Мы используем их как ввод для слоев `advantage_output` и `value_output`. Поскольку `v` представлен одиночным значением, которое прибавляется к `a`, мы его разворачиваем

```
        a = self.advantage_output(x)
        v = self.value_output(x)
        v = v.expand_as(a)
```

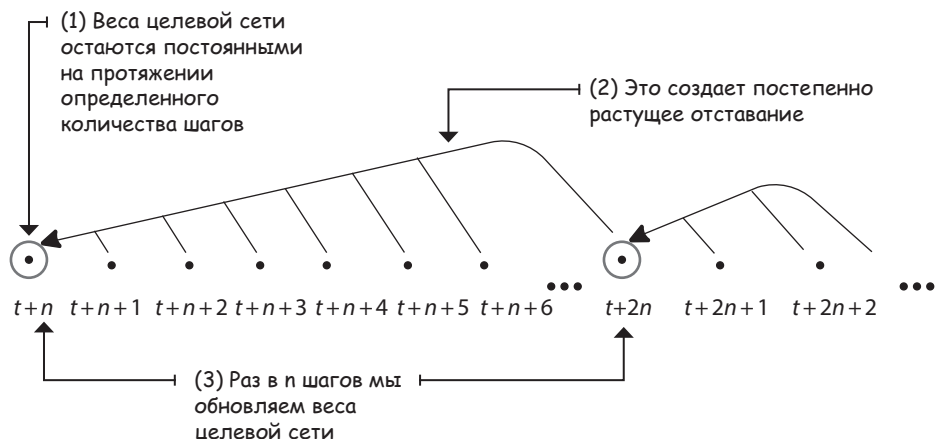
(6) В конце суммируем `v` и `a`, вычитая из полученного результата среднее значение `a`. Это будет наша функция $Q(s, \cdot)$ с прогнозами всех действий во всех состояниях

```
        q = v + a - a.mean(1, keepdim=True).expand_as(a)
        return q
```

Непрерывное обновление целевой сети

Сейчас наш агент использует целевую сеть, которая может устареть на несколько шагов, прежде чем получит большое обновление веса при синхронизации с динамической. В среде «перевернутый маятник» это около 15 шагов, но в сложных средах это число может достигать десятков тысяч.

Полное обновление целевой сети



У этого подхода есть несколько минусов. С одной стороны, мы замораживаем веса на несколько шагов и вычисляем прогнозы с помощью постепенно устаревающих данных. По завершении цикла обновления повышается вероятность того, что прогнозы не принесут никакой пользы при обучении. С другой стороны, обновление целевой сети происходит резко, что может изменить сразу весь ландшафт функции потерь. Такой стиль обновления выглядит одновременно слишком консервативным и агрессивным.

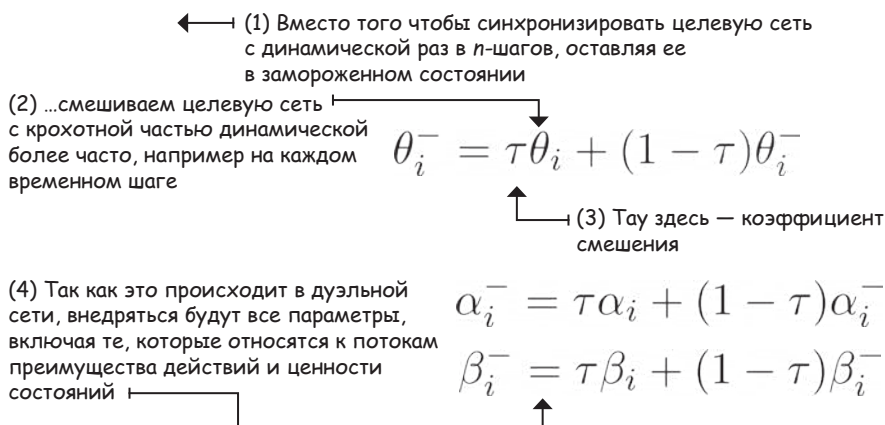
Это важно, потому что мы хотим предотвратить слишком быстрое изменение сети, которое вызывает нестабильность, и сохранить ее полезные свойства. Можно ли получить похожий результат как-то иначе? Может, целевую сеть лучше замедлить, а не заморозить?

Такая методика есть. Она называется *усреднением Поляка* и заключается во внедрении весов из динамической сети в целевую на каждом шаге. Иначе говоря, на каждом шаге мы создаем новую целевую сеть с небольшой долей динамических весов (новой информации), которая постепенно увеличивается на $\sim 1\%$. Так сеть всегда отстает, но уже намного меньше. Кроме того, мы можем обновлять ее на каждом шаге.



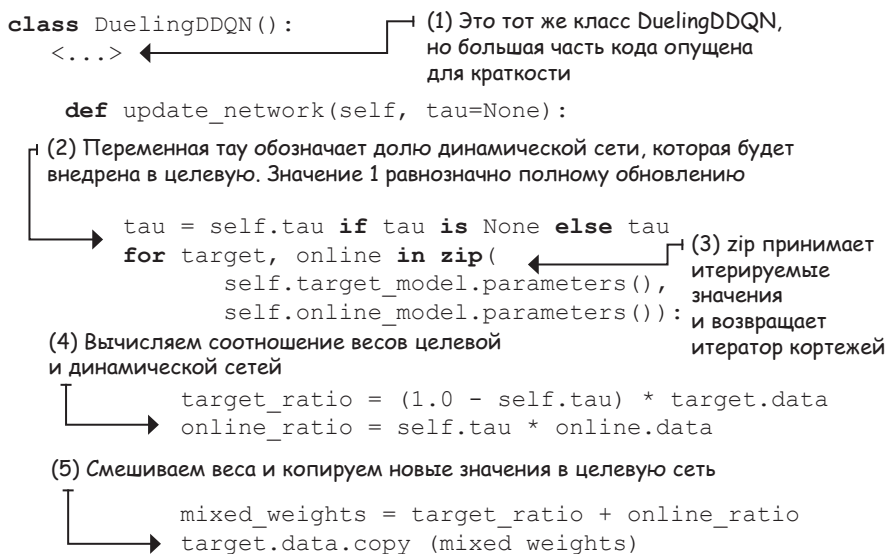
Я знаю математику

Усреднение Поляка



Я знаю Python

Смешивание весов целевой и динамической сетей



Преимущества дуэльной сети

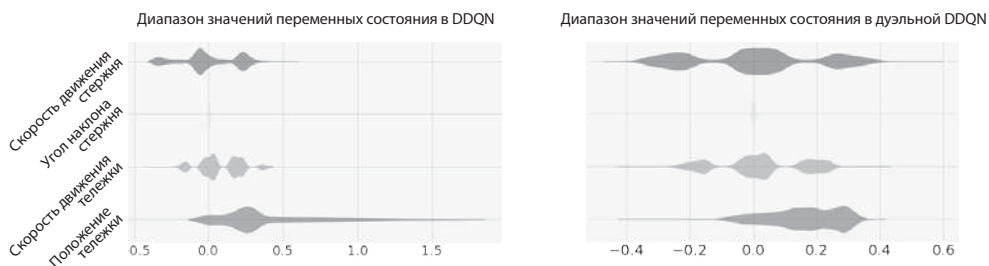
Как вы уже убедились, А-функции особенно полезны, когда есть много действий с похожими значениями. Строго говоря, дуэльная архитектура улучшает процесс оценки политик, особенно при наличии множества похожих оцененных действий. Благодаря дуэльной сети наш агент может быстрее и точнее сравнивать их, что довольно полезно в среде «перевернутый маятник».

Такие аппроксиматоры функций, как нейросети, могут ошибаться, и это нормально. В сети с архитектурой, которую мы использовали ранее, эти погрешности могут быть разными для всех пар «состояние — действие», так как все они вычисляются отдельно. Но, учитывая, что V-функция — это часть Q-функции, общая для всех действий в состоянии, применяя дуэльную архитектуру, мы можем снизить погрешность функций и вариативность погрешностей. Это связано с тем, что теперь погрешность в компоненте с самой большой величиной одинаковая для всех действий с похожими значениями (функцией ценности состояний $V(s)$).

Если дуэльная сеть положительно влияет на оценку политик агентом, то полностью обученный агент на основе дуэльной DDQN должен показывать лучшую результативность, чем DDQN при одинаковой ценности действий «влево» и «вправо». Я провел эксперимент, собрав состояния 100 эпизодов для агентов DDQN и дуэльной DDQN. Как подсказывает мне моя интуиция, если один агент лучше другого прогнозирует действия с похожими значениями, то его диапазон вдоль колеи должен быть более узким. Агент должен «понимать» разницу между движением влево и вправо, даже если стержень стоит в строго вертикальном положении. Внимание: я не проводил исследование абляции, но результаты моего эксперимента показывают, что агент на основе дуэльной DDQN действительно дает лучшие прогнозы в этих состояниях.

Пространство состояний, посещенных полностью обученными агентами в среде «перевернутый маятник»

(1) Я не стану делать из этого никаких выводов. Но обратите внимание на состояния, посещенные полностью обученными агентами DDQN и дуэльной DDQN для пяти начальных значений



(2) Слева видно, что полностью обученные агенты DDQN посещают позиции на колее с большим смещением вправо, вплоть до 1,5 единицы. В то же время агенты дуэльной DDQN, полностью обученные с теми же гиперпараметрами, остаются близко к центру. Возможно, агенты дуэльной DDQN лучше оценивают стратегии? Подумайте об этом и сами поэкспериментируйте



В деталях

Алгоритм дуэльной двойной глубокой Q-сети (дуэльной DDQN)

Дуэльная DDQN, DDQN и DQN почти идентичны, за исключением нескольких тонкостей. Я стараюсь свести различия между этими алгоритмами к минимуму, но и показать много разных способов их улучшения. Небольшое изменение всего нескольких гиперпараметров очень сильно повлияет на производительность этих алгоритмов, поэтому я не занимаюсь оптимизацией агентов. Но позвольте мне перечислить те аспекты, которые все еще остаются неизменными:

- сеть выводит функцию ценности действий $Q(s, a; \theta)$;
- мы оптимизируем функцию ценности действий для аппроксимации ее оптимального варианта $q^*(s, a)$;
- мы используем нестратегические цели TD ($r + \gamma \max_a Q(s', a'; \theta)$) для оценки политик;
- несмотря на применение регулируемых потерь Хьюбера, переменная $\max_gradient_norm$ все еще равна $\text{float}('inf')$, поэтому мы фактически используем MSE;
- в качестве оптимизатора используется RMSprop с темпом обучения 0,0007;
- этап улучшения основан на экспоненциально затухающей эпсилон-жадной стратегии (от 1,0 до 0,3 примерно за 20 000 шагов);
- этап оценки основан на стратегии жадного выбора действий;
- мы применяем буфер воспроизведения с диапазоном выборок 320–50 000 и набор размером 64.

Но кое-что изменилось:

- архитектура нейросети. Мы используем дуэльную сеть с архитектурой «состояние на входе, значения на выходе» (узлы: 4, 512 128, 1; 2, 2);
- если раньше целевая сеть замораживалась на 15 шагов и затем полностью обновлялась, то теперь мы используем усреднение Поляка: на каждом временном шаге смешиваем динамическую и целевую сети в соотношении 1 к 9, чтобы сформировать новые веса целевой сети.

Дуэльная DDQN — это то же, что и DDQN, только с другой сетью.

1. Сбор кортежа опыта $(S_t, A_t, R_{t+1}, S_{t+1}, D_{t+1})$ и добавление его в буфер воспроизведения.
2. Выбор из буфера случайного мини-набора и вычисление нестратегических целей TD для всего набора данных: $r + \gamma \max_a Q(s', a'; \theta)$.
3. Корректировка функции ценности действий $Q(s, a; \theta)$ с помощью MSE и RMSprop.

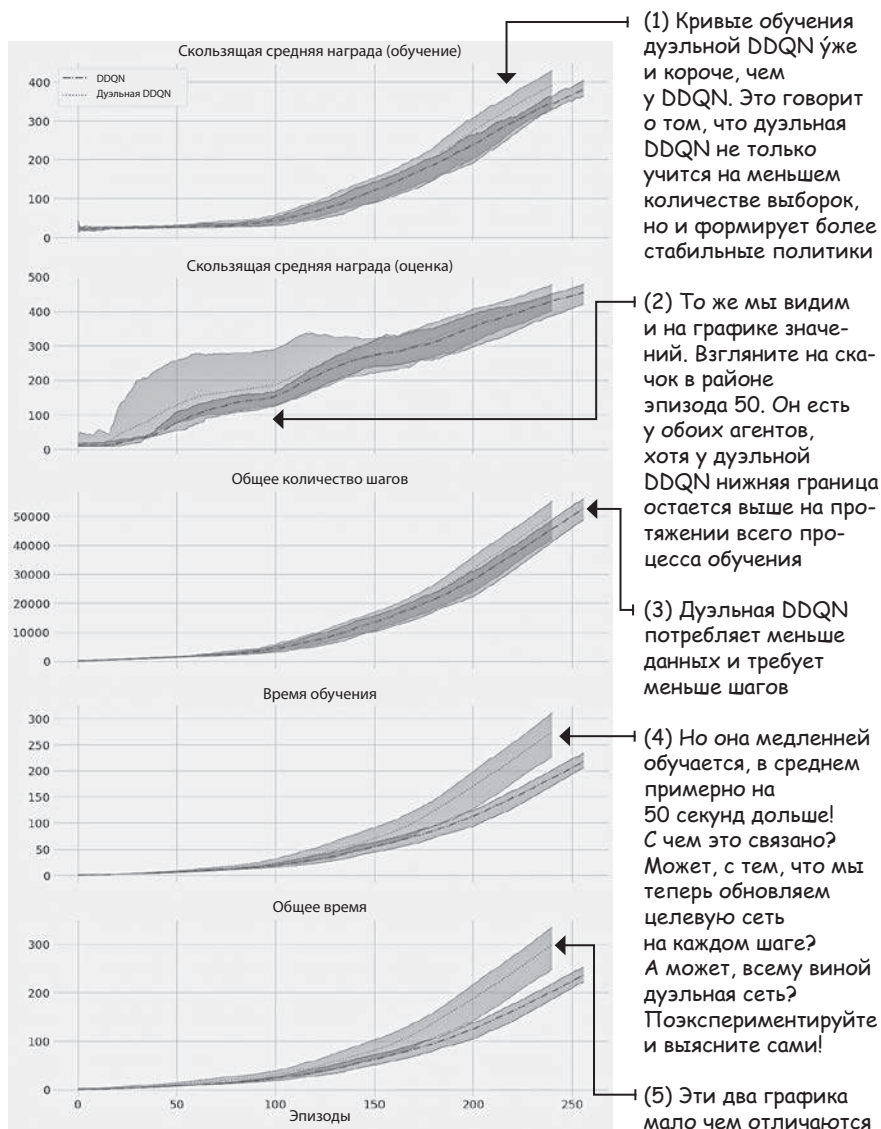
Обратите внимание на одну любопытную особенность: все эти улучшения похожи на детали конструктора, которые можно по-разному сочетать. Возможно, вы хотите попробовать дуэльную сеть DQN без двойного обучения? Может быть, вам хочется ограничить градиенты с помощью потерь Хьюбера или вы предпочитаете усреднение Поляка в соотношении 1 к 1 с обновлением раз в 5 шагов? Выбор за вами! Надеюсь, мой исходный код позволит вам свободно экспериментировать.



Посчитаем

Дуэльная DDQN использует данные эффективнее, чем все прошлые методы

У DDQN и дуэльной DDQN похожая производительность в среде «перевернутый маятник», хотя второй алгоритм чуть более эффективен, ведь ему нужно меньше выборок для прохождения среды. Но DDQN работает чуть быстрее.



PER: приоритетное воспроизведение полезного опыта

В этом разделе мы познакомимся с более разумным методом воспроизведения опыта. Наша цель — выделить для кортежей опыта ресурсы с наивысшим потенциалом для обучения. *Приоритетное воспроизведение опыта* (prioritized experience replay, PER) — это специализированный буфер воспроизведения, предназначенный именно для этого.

Более разумный способ воспроизведения опыта

Сейчас наш агент извлекает кортежи опыта из буфера воспроизведения равномерно случайно. Это верно как в математическом смысле, так и в целом. Но наша интуиция может подсказывать, что это плохой способ воспроизведения опыта, так как однородное случайное распределение выделяет ресурсы для бесполезных выборок. Вас может смутить то, что наш агент тратит время и вычислительную мощь на изучение того, что в текущем состоянии не может принести ему никакой пользы.

Но не будем спешить с выводами. Равномерно случайное распределение — это, конечно, не лучший выбор, но человеческая интуиция не всегда позволяет определить оптимальный сигнал обучения. Когда я впервые реализовал приоритетный буфер воспроизведения, еще до чтения научной работы о PER, моей первой мыслью было: «Я ведь хочу получать максимально высокую суммарную дисконтированную выгоду, поэтому повторять нужно только опыт с высокой наградой». Конечно же, это не сработало. Затем я вспомнил, что агентам нужен и отрицательный опыт: «Ага! Пусть мой агент повторяет опыт с наивысшей величиной наград! К тому же мне нравится использовать функцию abc! » Но это тоже не дало хороших результатов. Почему эти эксперименты провалились? Ведь если агент учится посещать полезные состояния, он должен повторять их чаще других, чтобы знать, как в них попасть. Логично, правда?



Аналогия от Мигеля

Человеческая интуиция и неустанное стремление к счастью

Я очень сильно люблю свою дочь и хочу, чтобы ее жизнь состояла только из хороших моментов. Родители понимают, о чем я.

Я заметил, что ей очень нравится шоколад, поэтому начал иногда баловать ее конфетами. Я делал это все чаще, но затем, не получив очередную сладость, она стала на меня обижаться.

Думаете, дело в чрезмерном поощрении? Правильно! Агентам (а может, и людям) нужно часто напоминать как о положительном, так и об отрицательном опыте. Но вместе с тем им нужен и обыденный опыт с небольшими наградами. В конце концов, каким бы ни был опыт, он не сделает обучение максимально эффективным, к чему мы и стремимся. Звучит парадоксально, не так ли?

Какой опыт можно считать важным

На самом деле мы хотим обучаться на опыте, который нас удивляет и обладает неожиданной ценностью: мы думали, что ценность будет одной, а она оказалась совсем другой. Такой опыт приближает нас к реальности. Мы по-своему смотрим на окружающий мир, ожидая того или иного исхода. Но когда возникает сильный разрыв между реальностью и ожиданиями, мы знаем, что это должно нас чему-то научить.

В обучении с подкреплением эта степень неожиданности определяется погрешностью TD! Если быть точнее, *абсолютной* погрешностью TD. Это разность между текущим прогнозом агента и целевым значением (ценностью). Текущий прогноз указывает на ожидаемую агентом ценность после определенного его поведения. Целевая ценность направляет агента к новому прогнозу той же пары «состояние — действие», заставляя его «взглянуть правде в глаза». Абсолютная разница между этими значениями показывает, насколько мы далеки, насколько неожиданный этот опыт и сколько новой информации мы получили, что делает ее хорошим индикатором возможностей для обучения.



Я знаю математику

Абсолютная погрешность TD в качестве приоритета

↓ (1) Я называю это целью дуэльной DDQN, чтобы подчеркнуть, что мы используем целевую сеть и дуэльную архитектуру. Но это можно назвать и просто целью TD

$$|\delta_i| = \left| r + \underbrace{\gamma Q(s', \arg\max_{a'} Q(s', a'; \theta_i, \alpha_i, \beta_i); \theta^-, \alpha^-, \beta^-)}_{\text{Цель дуэльной DDQN}} - \underbrace{Q(s, a; \theta_i, \alpha_i, \beta_i)}_{\text{Погрешность дуэльной DDQN}} \right|$$

Абсолютная погрешность дуэльной DDQN

Погрешность TD не идеальный показатель лучшей возможности для обучения, но это его самая разумная альтернатива. В реальности лучший критерий обучаемости находится внутри сети, за обновлениями параметров. Но вычислять градиенты для всех выборок в буфере воспроизведения на каждом временном шаге было бы непрактично. У погрешности TD есть важная особенность: механизм ее вычисления уже готов, и это по-прежнему хороший сигнал для выбора опыта, который стоит повторять.

Жадная приоритизация на основе погрешности TD

Представим, что погрешности TD используются для назначения приоритетов выборкам так:

- берем действие a в состоянии s и получаем новое состояние s' , награду r и флаг окончания d ;
- запрашиваем у сети прогноз текущего состояния $Q(s, a; \theta)$;
- вычисляем новое целевое значение для этой выборки как $target = r + \gamma \max_{a'} Q(s', a'; \theta)$;
- вычисляем абсолютную погрешность TD как $atd_err = abs(Q(s, a; \theta) - target)$;
- вставляем выборку в буфер воспроизведения в виде кортежа $(s, a, r, s', d, atd_err)$;
- извлекаем лучший опыт из буфера, отсортированного по atd_err ;
- обучаемся на этом опыте и повторяем все сначала.

Здесь есть сразу несколько проблем. Разберем их по очереди. Начнем с того, что погрешности TD вычисляются дважды: перед добавлением в буфер и при обучении сети. К тому же мы игнорируем то, что погрешности TD меняются вместе с сетью, так как они вычисляются с ее помощью. Но нельзя решить эту проблему обновлением всех погрешностей TD на каждом временном шаге. Это было бы слишком расточительно.

Обе проблемы можно решить за счет обновления погрешностей TD только для тех выборок, с помощью которых обновляется сеть (повторяемый опыт), и добавления погрешностей TD с наибольшей величиной в буфер, чтобы все они повторялись как минимум один раз.

Но такое решение порождает новые проблемы. Во-первых, если у нас нулевая погрешность TD в первом обновлении, то эта выборка вряд ли повторится. Во-вторых, при использовании аппроксиматоров функций уменьшение погрешностей происходит медленно. Это значит, что много обновлений сконцентрировано в небольшом подмножестве буфера воспроизведения. Еще не нужно забывать о том, что погрешности TD содержат много информационного шума.

Поэтому нам нужна стратегия для сбора опыта на основе погрешностей TD, но она должна быть стохастичной, а не жадной. Стохастически сэмплируя выборки с высоким приоритетом, мы можем гарантировать, что у каждой из них будет шанс на повторение и что вероятность их выбора будет монотонной по отношению к абсолютной погрешности TD.



По сути

Погрешности TD, приоритеты и вероятности

Важнейший вывод, который мы можем сделать: одних погрешностей TD недостаточно. Их нужно использовать для вычисления приоритетов, на основе которых можно вычислить вероятности.

Стохастическая выборка приоритетного опыта

Давайте обсудим, почему нам нужна стохастическая приоритизация. В средах с высокой степенью стохастичности обучение на опыте, собранном жадно и на основе погрешности TD, подвержено влиянию шума.

Погрешности TD зависят от одношаговой награды и функции ценности действий для следующего состояния: и то и другое может обладать высокой степенью стохастичности. В таких средах погрешности TD могут быть очень вариативны. Позволив агенту строго следовать погрешностям TD, можно попасть в неприятности. Мы не хотим, чтобы он заикливался на неожиданных ситуациях. Дополнительные источники шума для погрешностей TD — это нейросеть и использование аппроксиматоров крайне нелинейных функций, особенно в начале обучения, где самые большие погрешности. Если собирать опыт жадно с учетом только погрешностей TD, агент большую часть времени будет обучаться на выборках с потенциально большой величиной этих погрешностей.



По сути

Стохастическая выборка приоритетного опыта

Погрешности TD зашумлены и медленно уменьшаются. Мы не хотим прекращать повторение выборок, у которых из-за шума погрешность TD равна нулю, и не хотим заикливаться на искаженных выборках с большой погрешностью TD. И нам точно не нужно сосредотачиваться на выборках, у которых погрешность TD была высокой с самого начала.

0001 Немного истории

Создание буфера воспроизведения с приоритетным опытом

Научная работа *Prioritized Experience Replay* была представлена одновременно с появлением дуэльной архитектуры, которую в 2015 году предложили ребята из Google DeepMind.

Автор PER — Том Шоль (Tom Schaul), старший научный сотрудник в Google DeepMind. Том защитил свою докторскую диссертацию в 2011 году в Мюнхенском техническом университете. Спустя два года работы в Нью-Йоркском университете он присоединился к команде DeepMind Technologies, которая шесть месяцев спустя была куплена компанией Google и превратилась в подразделение, известное сегодня как Google DeepMind.

Том основной разработчик проекта PyBrain — модульной библиотеки машинного обучения для Python. Это один из первых проектов для реализации алгоритмов машинного обучения, обучения с подкреплением и оптимизации методом черного ящика. Он основной разработчик PyVGDL — высокоуровневого языка для описания компьютерных игр, основанного на pygame.

Пропорциональная приоритизация

Вычислим приоритеты для каждой выборки в буфере с учетом погрешностей TD. Сначала опыт можно собирать пропорционально абсолютным погрешностям TD. Возьмем абсолютную погрешность TD каждой выборки и прибавим к ней небольшую константу, эpsilon, чтобы у нулевых тоже был шанс на повторение.



Я знаю математику

Пропорциональная приоритизация

$$p_i = |\delta_i| + \epsilon$$

(1) Приоритет выборки i ... (2) ...равен абсолютной погрешности TD (3) Небольшая константа, эpsilon, чтобы не получить нулевой приоритет

Мы масштабируем значение приоритета, возводя его в степень альфа, гиперпараметра между 0 и 1, что позволяет нам балансировать между однородной и приоритетной выборками. Так мы можем выполнять стохастическую приоритизацию, как уже обсуждалось.

Если параметр альфа нулевой, все приоритеты становятся равными 1, то есть одинаковыми. Если параметр альфа равен 1, все значения становятся равными

абсолютной погрешности TD. Таким образом, приоритет пропорционален абсолютной погрешности TD — значение между ними смешивает две стратегии выборки.

Эти масштабированные приоритеты преобразуются в вероятности только после деления их значений на сумму всех приоритетов. Затем с помощью этих вероятностей можно будет извлекать выборки из буфера воспроизведения.



Я знаю математику

От приоритетов к вероятностям

(1) Вычисляем вероятности...

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

(2) ...возводя приоритеты в степень альфа, чтобы совместить однородное и приоритетное воспроизведение опыта

(3) А затем нормализуем результаты, чтобы полученная сумма вероятностей была равна 1

Приоритизация на основе ранжирования

Проблема пропорциональной приоритизации в том, что она чувствительна к резким отклонениям. Значит, выборки, погрешность TD у которых намного больше средней (фактически или из-за шума), извлекаются чаще, чем выборки с малой величиной. Такой побочный эффект обычно нежелателен.

Приоритизацию опыта можно проводить немного иначе, извлекая выборки с учетом их порядкового номера при сортировке по абсолютной погрешности TD.

Порядковый номер здесь обозначает позицию выборки в буфере, отсортированном по абсолютной погрешности TD в порядке убывания. Например, при использовании этого подхода порядковый номер 1 получает выборка с самой большой абсолютной погрешностью TD, выборка на втором месте будет иметь порядковый номер 2 и т. д.



Я знаю математику

Приоритизация на основе ранжирования

(1) Чтобы назначать приоритеты на основе ранжирования, мы вычисляем их как обратную величину порядкового номера соответствующей выборки

$$p_i = \frac{1}{rank(i)}$$

После ранжирования выборок по погрешности TD мы вычисляем их приоритеты в виде обратной величины их порядковых номеров. Затем, как и при пропорциональной стратегии, мы возводим их в степень альфа. Дальше на основе этих приоритетов, как и прежде, вычисляются вероятности с такой нормализацией значений, чтобы их сумма была равна 1.



По сути

Приоритизация на основе ранжирования

Если в пропорциональной приоритизации используется абсолютная погрешность TD и небольшая константа для учета выборок с нулевой погрешностью, то приоритизация на основе ранжирования состоит в применении обратного порядкового номера выборки после сортировки буфера по абсолютной погрешности TD в порядке убывания.

В обоих подходах приоритеты преобразуются в вероятности одним и тем же способом.

Отклонение при назначении приоритетов

Прогнозирование одного распределения с помощью другого создает отклонение в прогнозах. Поскольку мы делаем выборку на основе этих вероятностей, приоритетов и ошибок TD, нам нужно это учитывать.

Для начала разберемся подробнее. Распределение обновлений должно быть основано на распределении прогнозов. При обновлении Q -функции для состояния s и действия a важно понимать, что мы всегда обновляем и цели.

Цели — это выборки ожидаемых значений, поэтому награда и состояние на следующем шаге могут быть стохастичными: то есть при выборе действия a в состоянии s можно получить много разных наград.

Если игнорировать это и обновлять выборку чаще прогноза, у нас получится отклонение в сторону этого значения. Проблема усугубляется к концу обучения, когда наши методы становятся близки к сходимости.

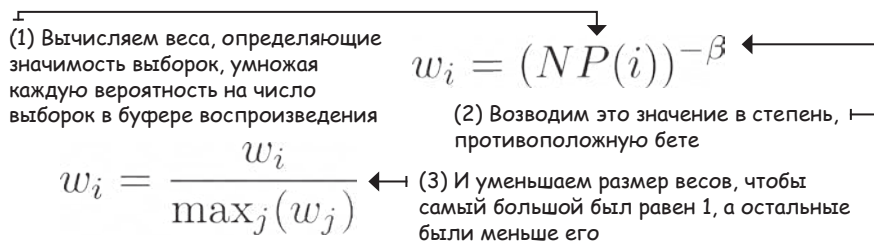
Для смягчения этого отклонения применяется *взвешенная выборка по значимости* — масштабирование погрешностей TD с помощью весов, основанных на вероятности каждой выборки.

Этот подход изменяет величину обновлений, чтобы казалось, что выборки получены из равномерного распределения.



Я знаю математику

Вычисление весов во взвешенной выборке по значимости



Для эффективного вычисления взвешенной выборки по значимости с помощью приоритетного буфера воспроизведения мы добавляем удобный гиперпараметр, бета, который позволяет регулировать степень коррекции. Когда он равен нулю, коррекция не выполняется, а когда равен 1, происходит полная коррекция смещения.

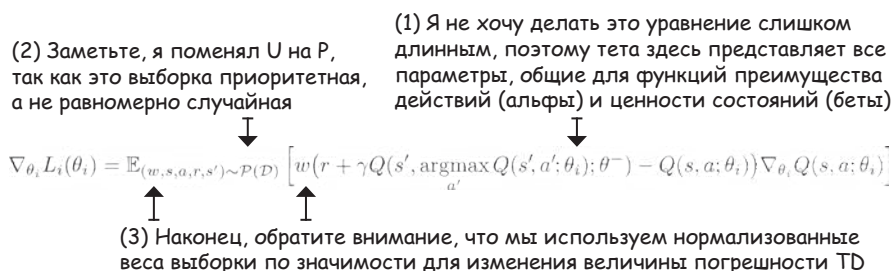
Кроме того, мы хотим нормализовать веса по их максимуму, чтобы максимальный вес был равен 1, а все остальные уменьшали величину погрешностей TD. Так погрешности не будут становиться слишком большими, что стабилизирует процесс обучения.

Веса в выборках по значимости используются в функции потерь. Вместо того чтобы применять погрешности TD прямо в обновлениях градиента, в методе PER мы умножаем их на веса выборок по значимости и уменьшаем, чтобы компенсировать несоответствие в распределении.



Я знаю математику

Дульная DDQN с обновлением градиента методом PER





Я знаю Python

Приоритетный буфер воспроизведения

```
class PrioritizedReplayBuffer():
    <...>
    def store(self, sample):
```

(1) Функция `store` в классе `PrioritizedReplayBuffer` выглядит довольно просто. Сначала вычисляется приоритет выборки. Помните, мы делаем его максимальным. В коде ниже по умолчанию используется число 1, которое затем заменяется максимальным значением

```
        priority = 1.0
        if self.n_entries > 0:
            priority = self.memory[
                :self.n_entries,
                self.td_error_index].max()
```

(2) Получив приоритет и выборку (опыт), добавляем их в память

```
        self.memory[self.next_index,
                    self.td_error_index] = priority
        self.memory[self.next_index,
                    self.sample_index] = np.array(sample)
```

(3) Увеличиваем переменную, обозначающую количество выборок в буфере. Важно убедиться, что размер буфера не превышает `max_samples`

```
        self.n_entries = min(self.n_entries + 1,
                               self.max_samples)
```

(4) Эта переменная обозначает индекс, по которому будет вставлена следующая выборка. Ее значение изменяется от `max_samples` до 0 и возвращается обратно

```
        self.next_index += 1
        self.next_index = self.next_index % self.max_samples
```

```
    def update(self, idxs, td_errors):
```

(5) Функция `update` принимает массив с идентификаторами выборок и новые значения погрешностей TD

```
        self.memory[idxs,
                    self.td_error_index] = np.abs(td_errors)
```

(6) Если данные собираются с использованием ранжирования, массив дополнительно сортируется. Заметьте, что массивы не очень хорошо подходят для реализации приоритетного буфера воспроизведения, из-за того что их сортировка зависит от количества выборок. Это плохо влияет на производительность

```
        if self.rank_based:
            sorted_arg = self.memory[:self.n_entries,
                                    self.td_error_index].argsort()[::-1]
            self.memory[:self.n_entries] = self.memory[
                sorted_arg]
```

```

class PrioritizedReplayBuffer():
    <...>
    def sample(self, batch_size=None):
        (1) Вычисляем batch_size, обновляем beta и убираем
            из entries обнуленные строки
            batch_size = self.batch_size if batch_size == None \
                                else batch_size
        self._update_beta()
        entries = self.memory[:self.n_entries]

        (2) Теперь вычисляем приоритеты. Если используется
            ранжирование, просто берем порядковый номер и прибавляем 1
            (эти номера были сохранены в функции update). Если
            приоритеты пропорциональные, суммируем абсолютную
            погрешность TD и небольшую постоянную эпсилон, чтобы
            избежать нулевых значений
            if self.rank_based:
                priorities = 1/(np.arange(self.n_entries) + 1)
            else: # proportional
                priorities = entries[:, self.td_error_index] + EPS

        (3) Теперь получаем вероятности: сначала внедряем
            приоритеты из равномерного распределения, а затем
            из вероятностного
            scaled_priorities = priorities**self.alpha
            pri_sum = np.sum(scaled_priorities)
            probs = np.array(scaled_priorities/pri_sum,
                                dtype=np.float64)

        (4) Вычисляем на основе вероятностей веса
            для выборки по значимости
            weights = (self.n_entries * probs)**-self.beta

        (5) Нормализуем веса. Максимальный будет равен 1
            normalized_weights = weights/weights.max()

        (6) Выбираем индексы выборок в буфере
            с помощью вероятностей
            idxs = np.random.choice(self.n_entries,
                                    batch_size, replace=False, p=probs)

        (7) Извлекаем выборки из буфера
            samples = np.array([entries[idx] for idx in idxs])

        (8) Наконец, возвращаем выборки, сгруппированные
            по идентификаторам, весам и кортежам опыта
            samples_stacks = [np.vstack(batch_type) for \
batch_type in np.vstack(samples[:, self.sample_index]).T]
            idxs_stack = np.vstack(idxs)
            weights_stack = np.vstack(normalized_weights[idxs])
            return idxs_stack, weights_stack, samples_stacks

```



Я знаю Python

Функция потерь для приоритетного буфера воспроизведения

```
class PER():
    <...>  (1) Как вы помните, это часть кода. Здесь показаны
           те фрагменты, которые я считаю важными

    def optimize_model(self, experiences):
        (2) Обратите внимание, что вместе с выборками теперь
           используются веса и идентификаторы

        idxs, weights, \
        (states, actions, rewards,
          next_states, is_terminals) = experiences

        <...>

        (3) Как и прежде, вычисляем целевые значения
        argmax_a_q_sp = self.online_model(next_states).max(1)[1]
        q_sp = self.target_model(next_states).detach()
        max_a_q_sp = q_sp[np.arange(batch_size), argmax_a_q_sp]
        max_a_q_sp = max_a_q_sp.unsqueeze(1)
        max_a_q_sp *= (1 - is_terminals)
        target_q_sa = rewards + (self.gamma * max_a_q_sp)

        (4) Запрашиваем текущие прогнозы: ничего нового
        q_sa = self.online_model(states).gather(1, actions)

        (5) Вычисляем погрешности TD, как и раньше
        td_error = q_sa - target_q_sa

        (6) Но теперь погрешности TD в функции потерь уменьшены с учетом их весов
        value_loss = (weights * td_error).pow(2).mul(0.5).mean()

        (7) Продолжаем оптимизацию. Здесь без изменений
        self.value_optimizer.zero_grad()
        value_loss.backward()
        torch.nn.utils.clip_grad_norm_(
            self.online_model.parameters(),
            self.max_gradient_norm)
        self.value_optimizer.step()

        (8) Обновляем приоритеты повторенного набора с использованием
        абсолютных погрешностей TD
        priorities = np.abs(td_error.detach().cpu().numpy())
        self.replay_buffer.update(idxs, priorities)
```

```

class PER():
    <...>
    def train(self, make_env_fn, make_env_kargs, seed, gamma,
              max_minutes, max_episodes, goal_mean_100_reward):
        <...>
        for episode in range(1, max_episodes + 1):
            <...>
            for step in count():
                state, is_terminal = \
                    self.interaction_step(state, env)
                <...>
                if len(self.replay_buffer) > min_samples:
                    experiences = self.replay_buffer.sample()
                    idxs, weights, samples = experiences
                    experiences = self.online_model.load(
                        samples)
                    experiences = (idxs, weights) + \
                                (experiences,)
                    self.optimize_model(experiences)
                    if np.sum(self.episode_timestep) % \
                        self.update_target_every_steps == 0:
                        self.update_network()

                    if is_terminal:
                        break

```

(1) Тот же класс PER, только с функцией обучения

(2) Внутри цикла обработки эпизодов

(3) Внутри цикла временных шагов

(4) Каждый временной шаг в ходе обучения

(5) Заметьте, как мы извлекаем выборки из буфера

(6) Извлекаем из выборок идентификаторы, веса и кортеж опыта. Обратите внимание на то, как мы загружаем переменные выборки в графический процессор

(7) Затем снова группируем переменные: это делается только для загрузки выборок в память графического процессора и подготовки их к обучению

(8) Оптимизируем модель (это функция из кода выше)

(9) Все остальное делаем как обычно



В деталях

Дуэльная DDQN с алгоритмом приоритетного буфера воспроизведения

Давайте в последний раз улучшим все методы ценностно ориентированного глубокого обучения с подкреплением, рассмотренные ранее. Оптимизация будет касаться буфера воспроизведения. Большинство гиперпараметров останутся такими же, как и в прошлых методах. Но давайте углубимся в детали. Вот что не поменяется:

- сеть выводит функцию ценности действий $Q(s, a; \theta)$;
- мы используем дуэльную сеть с архитектурой «состояние на входе, значения на выходе» (узлы: 4, 512 128, 1; 2, 2);
- мы оптимизируем функцию ценности действий для аппроксимации ее оптимального варианта $q^*(s, a)$;
- мы используем нестратегические цели TD ($r + \gamma \max_a Q(s', a; \theta)$) для оценки политик;
- несмотря на применение регулируемых потерь Хьюбера, переменная $\max_gradient_norm$ все еще равна $\text{float}('inf')$, поэтому мы фактически используем MSE;
- в качестве оптимизатора используется RMSprop с темпом обучения 0,0007;
- этап улучшения основан на экспоненциально затухающей эпсилон-жадной стратегии (от 1,0 до 0,3 приблизительно за 20 000 шагов);
- этап оценки основан на стратегии жадного выбора действий;
- целевая сеть, которая обновляется каждый временной шаг с помощью усреднения Поляка с τ (коэффициент смешения) 0,1;
- мы применяем буфер воспроизведения с диапазоном выборок от 320 до 50 000 и набор размером 64.

Вот что мы поменяли:

- погрешности TD корректируются с помощью взвешенной выборки по значимости (в результате меняется функция потерь);
- используется приоритетный буфер воспроизведения с пропорциональным назначением приоритетов, 10 000 выборок, значением альфа (степенью приоритетности в сравнении с однородной выборкой — 1 означает полную приоритетность), равным 0,6, значением бета (начальное значение бета, коррекции смещения, — 1 соответствует полной коррекции) 0,1 и скоростью ее оптимизации 0,99992 (для полной оптимизации нужно около 30 000 временных шагов).

Базовый алгоритм PER такой же, как у дуэльной DDQN, DDQN и DQN.

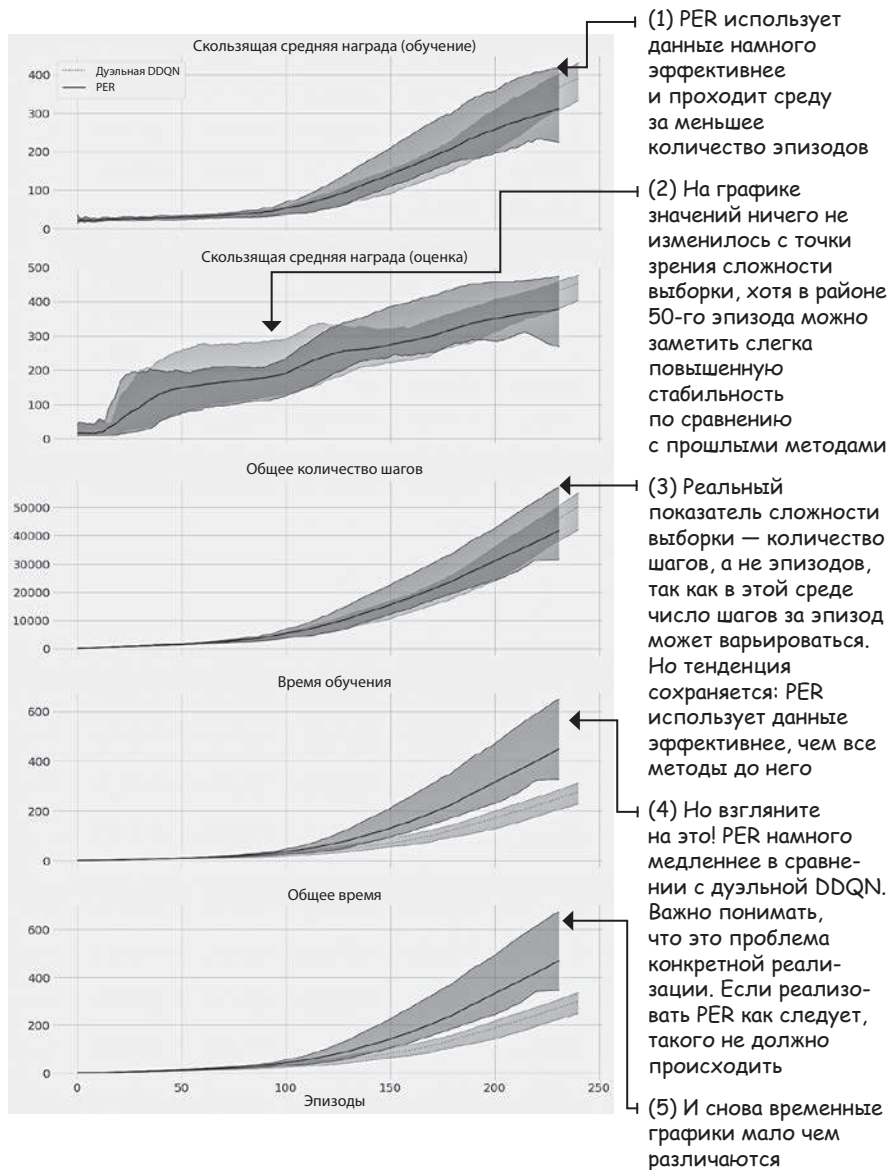
1. Сбор кортежа опыта $(S_t, A_t, R_{t+1}, S_{t+1}, D_{t+1})$ и добавление его в буфер воспроизведения.
2. Выбор из буфера случайного мини-набора и вычисление нестратегических целей TD для всего набора данных: $r + \gamma \max_a Q(s', a; \theta)$.
3. Корректировка функции ценности действий $Q(s, a; \theta)$ с помощью MSE и RMSprop.
4. Регулирование погрешностей TD в буфере воспроизведения.



Посчитаем

PER повышает эффективность использования данных еще сильнее

Приоритетный буфер воспроизведения использует меньше выборок, чем любой другой метод, рассмотренный ранее. И, как вы видите на графиках ниже, при этом все выглядит стабильнее... наверное.



Подведем итоги

Этой главой мы завершаем исследование ценностно ориентированных методов DRL. Здесь мы обсудили, как повысить эффективность использования данных в ценностно ориентированных методах. Вы познакомились с дуэльной архитектурой и узнали, как она использует особенности ценностно ориентированного RL за счет разделения $Q(s, a)$ на функции ценности состояний $V(s)$ и преимущества действий $A(s, a)$. Благодаря этому каждая выборка, участвующая в обновлении сети, может добавлять информацию для прогнозирования функции ценности состояний $V(s)$, общей для всех действий, что позволяет быстрее получить правильные прогнозы и упрощает выборку.

Мы обсудили и приоритетность выборок. Вы узнали, что погрешности TD — хороший критерий для создания приоритетов, на основе которых можно вычислять вероятности. Мы выяснили, что должны компенсировать изменение распределения ожидаемых оценок, которые прогнозируем. В связи с этим используется выборка по значимости — метод корректировки смещения.

В этой и предшествующих двух главах мы полностью погрузились в область ценностно ориентированного DRL. Все началось с простого подхода, NFQ. Мы сделали эту методику более стабильной, воспользовавшись улучшениями из DQN и DDQN. Затем мы позаботились об эффективности применения выборок с помощью дуэльной DDQN и PER. В итоге у нас получился довольно действенный алгоритм. Но у ценностно ориентированных методов тоже есть недостатки. Во-первых, они чувствительны к гиперпараметрам. Это общеизвестный факт, но вы можете сами в этом убедиться, изменив свои: большинство значений не будут подходить. Во-вторых, эти методы рассчитаны на взаимодействие с марковской средой, где состояния содержат всю информацию, необходимую агенту. Чем дальше мы уходим от бэггинга и ценностно ориентированных методов в целом, тем меньше это соответствует реальности. И последнее: сочетание бэггинга, нестратегического обучения и аппроксимации функций называют смертельной триадой, которая, как нам известно, вызывает отклонения. Но исследователи все еще не знают, как это предотвратить.

Я вовсе не утверждаю, что ценностно ориентированный метод DRL сильно уступает методам из следующих глав. У них тоже есть свои минусы. Главный вывод таков: отклонение — слабое место методов ценностно ориентированного глубокого обучения с подкреплением, и об этом важно помнить. Специалисты все еще выясняют, как с этим бороться, но хорошим практическим советом будет использование целевых сетей, буферов воспроизведения, двойного обучения, достаточно (но не слишком) низких темпов сбора данных и, возможно, чуточку терпения. Извините, но правила устанавливаю не я.

На этом этапе вы:

- умеете выполнять задачи RL с непрерывными пространствами состояний;
- знаете, как стабилизировать работу агентов ценностно ориентированного DRL;
- можете повысить эффективность использования выборок агентами ценностно ориентированного DRL.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

- **#gdrl_ch10_tf01.** Буферов воспроизведения из этой и двух прошлых глав достаточно для «перевернутого маятника» и других малоразмерных сред. Но позже вы заметите, что в более сложных средах приоритетный буфер вызывает трудности. Попробуйте самостоятельно переписать код всех буферов воспроизведения, чтобы повысить их скорость. Не подсматривайте чужие реализации. Можно заметить, что самый медленный этап в приоритетном буфере — сортировка выборок. Постарайтесь исправить это.
- **#gdrl_ch10_tf02.** При попытке реализации многомерных сред вроде игр Atari код буфера воспроизведения из этой и прошлых глав становится недопустимо медленным и совершенно непрактичным. Что же делать? Узнайте, как другие решают проблему, препятствующую внедрению приоритетных буферов. Поделитесь этой информацией и самостоятельно реализуйте соответствующие структуры данных. Изучите их и подробно напишите о том, чем они полезны.
- **#gdrl_ch10_tf03.** В этой и предыдущей главах вы познакомились с методами, способными выполнять задачи с многомерными и непрерывными пространствами состояний. Но что насчет пространств действий? Эти алгоритмы умеют выбирать только по одному действию за раз, и значения у этих действий дискретны, что сильно удручает. Но постойте, разве DQN-подобные методы рассчитаны только на задачи с дискретными пространствами действий, размером 1? Изучите этот вопрос и расскажите о результатах.
- **#gdrl_ch10_tf04.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdrl_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы познакомитесь с семейством методов глубокого обучения с подкреплением, которые умеют оптимизировать свою производительность напрямую, без функций ценности.
- ✓ Узнаете, как улучшить эти алгоритмы с помощью функции ценности.
- ✓ Реализуете алгоритмы DRL, способные использовать сразу несколько процессов для очень быстрого обучения.

Нет ничего лучше несчастья. В каждом поражении, каждой горе, каждой потере есть свое семя, собственный урок того, как улучшить свои результаты в следующий раз.

*Малкольм Икс (Malcolm X),
мусульманский духовный наставник
и американский правозащитник*

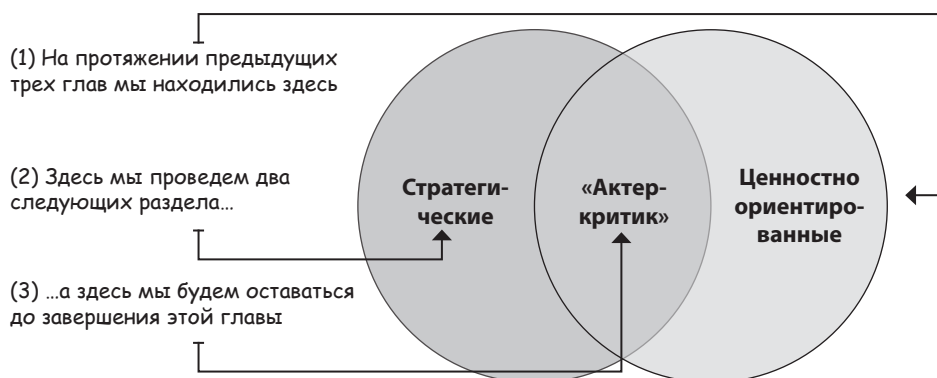
В книге мы исследовали методы, способные находить оптимальные и близкие к ним политики через формирование функций ценности. Но на самом деле нам нужны не эти функции, а именно политики.

В этой главе мы посмотрим, что находится на другом конце спектра, и обсудим промежуточные решения. Начнем с методов, оптимизирующих политики напрямую, *стратегических* или *методов градиента политик*. Они параметризуют политику и приводят ее к максимальной ожидаемой выгоде.

После знакомства с базовыми методами градиента политик мы сосредоточимся на промежуточных подходах для формирования как политик, так и функций ценности. Они называются «актер-критик», поскольку политику выбора действий можно считать актером, а функцию ценности, которая занимается ее оценкой, — критиком. Во многих тестах глубокого обучения с подкреплением методы «актер-критик» часто показывают лучшие результаты, чем ценностно ориентированные или методы градиента политик. Освоив этот подход, вы сможете выполнять более сложные задачи.

Эти методы сочетают в себе то, что вы узнали в прошлых трех главах, посвященных функциям ценности обучения, и то, что узнали в первой части этой главы, о политиках обучения. Методы «актер-критик» зачастую показывают непревзойденные результаты в целом ряде разных сред для тестирования производительности глубокого обучения с подкреплением.

Методы: стратегические, ценностно ориентированные и «актер-критик»



REINFORCE: обучение политик на основе результатов

В этом разделе мы начнем формулировать причины для использования стратегических методов. После введения мы обсудим несколько преимуществ, которых стоит от них ожидать. В конце будет представлен простейший алгоритм градиента политик *REINFORCE*.

Введение в методы градиента политик

В первую очередь я бы хотел подчеркнуть, что градиент политик, в отличие от ценностно ориентированных методов, пытается максимизировать производительность. Ценностно ориентированные методы в основном нацелены на то, чтобы научиться оценивать политики. Для этого они минимизируют потери — разность между прогнозируемыми и целевыми значениями. Наша цель — получить реальную функцию ценности действий заданной политики. Для этого мы параметризуем функцию ценности и минимизируем средне-квадратическую ошибку (разность между предсказанными и целевыми значениями). Заметьте, что у нас не было реальных целевых значений: вместо них методы бэггинга и Монте-Карло использовали прогнозируемую и фактическую выгоду.

Для сравнения: цель стратегических методов — максимизация производительности параметризованной политики, поэтому мы используем градиентный подъем (или выполняем обычный градиентный спуск при отрицательной производительности). Очевидно, что производительность агента равна ожидаемой совокупной дисконтированной награде от начального состояния, что ничем не отличается от ожидаемой функции ценности состояний из всех начальных состояний заданной политики.



Я знаю математику

Цели ценностно ориентированных и стратегических методов

(1) Цель ценностно ориентированных методов — минимизация функции потерь — среднеквадратичной разности между реальной и параметризованной Q-функцией



$$L_i(\theta_i) = \mathbb{E}_{s,a} \left[\left(q_\pi(s, a) - Q(s, a; \theta_i) \right)^2 \right]$$

$$J_i(\theta_i) = \mathbb{E}_{s_0 \sim p_0} \left[v_{\pi_{\theta_i}}(s_0) \right]$$



(2) Цель стратегических методов — максимизация мер производительности — реальной функции ценности параметризованной политики из всех начальных состояний



С акцентом на RL

Ценностно ориентированные и стратегические методы, методы градиента политик и «актер-критик»

Ценностно ориентированные методы. Алгоритмы, формирующие только функции ценности. Q-обучение, SARSA, DQN и др. — примеры таких методов.

Стратегические методы. Широкий спектр алгоритмов для оптимизации политик, включая оптимизацию методом черного ящика, такую как генетические алгоритмы.

Методы градиента политик. Методы выполнения задачи оптимизации на градиенте производительности параметризованной политики. С ними вы познакомитесь в этой главе.

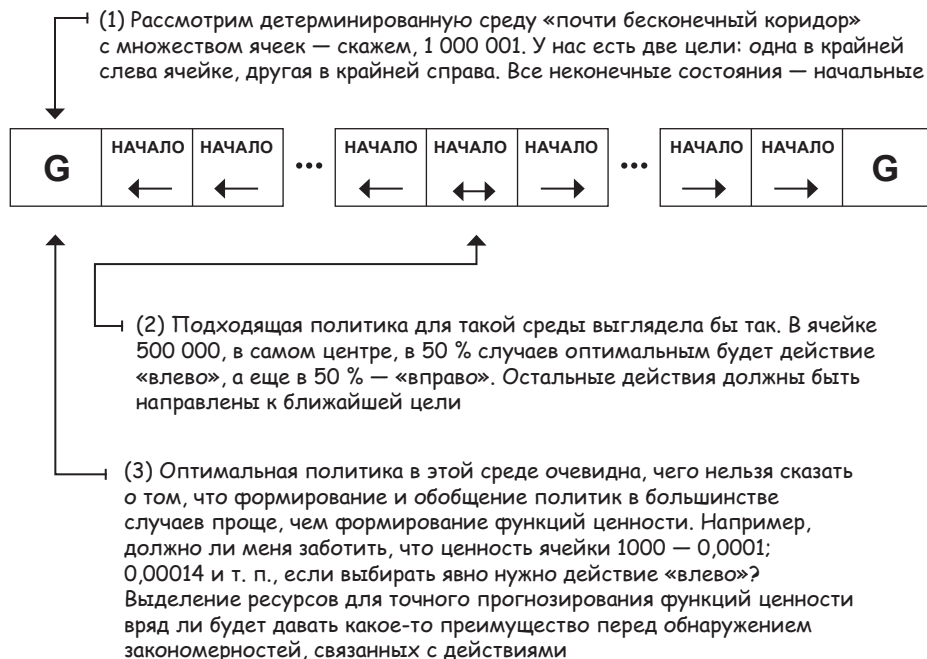
Методы «актер-критик». Методы, формирующие как политику, так и функцию ценности, в основном когда последняя формируется с помощью бэггинга и используется в качестве диапазона для градиента стохастических политик. С этими методами вы познакомитесь здесь и в следующей главе.

Преимущества методов градиента политик

Основное преимущество формирования параметризованных политик — то, что роль политики теперь может играть любая обучаемая функция. В ценностно ориентированных методах мы имели дело с дискретными пространствами действий, в основном потому, что мы вычисляли самые ценные из них. В многомерных пространствах искать максимальное значение может быть очень затратно. Более того, в случае с непрерывными пространствами действий ценностно ориентированные методы оказываются очень ограниченными.

А вот стратегические методы облегчают формирование стохастических политик. У этого есть ряд дополнительных преимуществ. Например, формирование стохастических политик обеспечивает лучшую производительность в частично наблюдаемых средах. Интуиция подсказывает, что возможность вычисления произвольных вероятностей или действий делает агент менее зависимым от марковского свойства. Например, если после наблюдения за небольшим количеством состояний агенту не удастся их различить. Как правило, лучшая стратегия — это случайное поведение с учетом определенных вероятностей.

Формирование политик может быть упрощенной и универсальной задачей



Последнее преимущество, о котором нужно упомянуть: поскольку политики параметризованы непрерывными значениями, вероятности действий плавно изменяются в зависимости от изученных параметров. Поэтому стратегические методы нередко отличаются лучшей сходимостью. Как вы помните из прошлых глав, ценностно ориентированным методам свойственны колебания и отклонения. Одна из причин этого в том, что крошечные изменения в пространстве функции ценности могут иметь серьезные последствия для пространства действий. Существенные различия в действиях могут приводить к появлению новых, совершенно необычных траекторий, снижая тем самым стабильность.

В ценностно ориентированных методах для изменения функции ценности используется агрессивный подход: мы берем максимальное ожидаемое Q-значение. Стратегические методы вместо этого используют градиент стохастических политик, которые всегда изменяют действия постепенно и плавно. Если следовать градиенту политики, то будет гарантирована сходимость по крайней мере к локальному оптимуму.



Я знаю Python

Стохастическая политика для дискретных пространств состояний

```

class FCDAP(nn.Module):
    def __init__(self,
                  input_dim,
                  output_dim,
                  hidden_dims=(32, 32),
                  init_std=1,
                  activation_fc=F.relu):
        super(FCDAP, self).__init__()
        self.activation_fc = activation_fc

        self.input_layer = nn.Linear(
            input_dim, hidden_dims[0])

        self.hidden_layers = nn.ModuleList()
        for i in range(len(hidden_dims)-1):
            hidden_layer = nn.Linear(
                hidden_dims[i], hidden_dims[i+1])
            self.hidden_layers.append(hidden_layer)

        self.output_layer = nn.Linear(
            hidden_dims[-1], output_dim)

    def forward(self, state):
        x = state
        if not isinstance(x, torch.Tensor):
            x = torch.tensor(x, dtype=torch.float32)
            x = x.unsqueeze(0)

        x = self.activation_fc(self.input_layer(x))

        for hidden_layer in self.hidden_layers:
            x = self.activation_fc(hidden_layer(x))

        return self.output_layer(x)

```

(1) FCDAP расшифровывается как fully connected discrete-action policy (полносвязная политика с дискретными действиями)

(2) Эти параметры позволяют указать полносвязную архитектуру, функцию активации и максимальную величину веса и смещения

(3) Функция `__init__` создает линейную связь между входными данными и первым скрытым слоем...

(4) ...а после создает связи между всеми скрытыми слоями

(5) В конце эта функция соединяет последний скрытый слой с выходными узлами, создавая выходной слой

(6) Здесь мы видим метод, который отвечает за прямое прохождение

(7) Прежде чем пропускать состояние через сеть, убедимся, что у него подходящая размерность и что его переменные нужных типов

(8) После этого передаем верно отформатированное состояние входному слою и затем пропускаем его через функцию активации

(9) Далее вывод первой активации проходит через цепочку скрытых слоев и соответствующих активаций

(10) В конце получаем вывод — логистическое распределение вероятностей действий


```

def full_pass(self, state):
    logits = self.forward(state)
    dist = torch.distributions.Categorical(logits=logits)
    action = dist.sample()
    logpa = dist.log_prob(action).unsqueeze(-1)
    entropy = dist.entropy().unsqueeze(-1)
    is_exploratory = action != np.argmax( \
        logits.detach().numpy())

    return action.item(), is_exploratory.item(), \
        logpa, entropy

def select_action(self, state):
    logits = self.forward(state)
    dist = torch.distributions.Categorical(logits=logits)
    action = dist.sample()
    return action.item()

def select_greedy_action(self, state):
    logits = self.forward(state)
    return np.argmax(logits.detach().numpy())

```

(11) Здесь мы выполняем полное прямое прохождение. Это удобная функция для получения вероятностей, действий и всего, что нужно для обучения

(12) Прямое прохождение возвращает логистическое распределение вероятностей действий

(13) Далее выбираем действие из вероятностного распределения

(14) Вычисляем логарифм вероятности этого действия и форматируем его для обучения

(15) Здесь вычисляется энтропия политики

(16) А здесь мы определяем, была ли выбранная политика исследовательской (для статистики)

(17) В конце возвращаем действие, которое можно передать среде напрямую, флаг, обозначающий, исследовательское ли это действие, логарифм его вероятности и энтропию политики

(18) Это вспомогательная функция на случай, если нам нужно только выбранное действие

(19) А это функция для выбора жадного действия согласно политике

Прямое обучение политик

Одно из главных преимуществ прямой оптимизации политик — это правильная цель. Мы формируем политику, которая сразу оптимизирует функцию ценности, не занимаясь ее изучением и не учитывая динамику среды. Как это возможно? Позвольте показать.



Я знаю математику

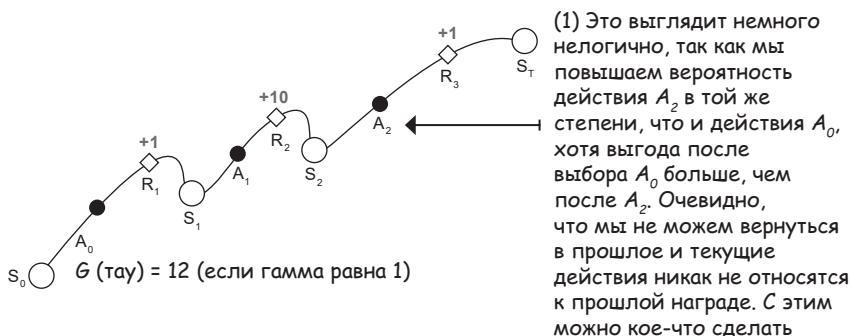
Вывод градиента политик

- (1) Для начала вспомним упрощенное уравнение цели, представленное ранее $\longrightarrow J(\theta) = \mathbb{E}_{s_0 \sim p_0} [v_{\pi_\theta}(s_0)]$
- (2) Мы знаем, что нам нужно найти градиент относительно этого показателя производительности $\longrightarrow \nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{s_0 \sim p_0} [v_{\pi_\theta}(s_0)]$
- (3) Чтобы уравнение было проще, τ будет обозначать полную траекторию $\longrightarrow \tau = S_0, A_0, R_1, S_1, \dots, S_{T-1}, A_{T-1}, R_T, S_T$
- (4) Так мы можем злоупотребить обозначениями и использовать функцию G для получения выгоды полной траектории $\longrightarrow G(\tau) = R_1 + \gamma R_2 + \dots + \gamma^{T-1} R_T$
- (5) Мы можем получить и вероятность траектории
- (6) Это вероятность начальных состояний, действия, перехода и т. д. В результате получается произведение всех вероятностей, которые делают эту траекторию возможной $\longrightarrow p(\tau|\pi_\theta) = p_0(S_0)\pi(A_0|S_0; \theta)P(S_1, R_1|S_0, A_0) \dots P(S_T, R_T|S_{T-1}, A_{T-1})$
- (7) После всех этих изменений обозначений мы можем сказать, что цель такова $\longrightarrow \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)] = \nabla_\theta \mathbb{E}_{s_0 \sim p_0} [v_{\pi_\theta}(s_0)]$
- (8) Теперь взгляните на метод прогнозирования градиентов ожиданий, известный как градиентная оценка функции вклада $\longrightarrow \nabla_\theta \mathbb{E}_x [f(x)] = \mathbb{E}_x [\nabla_\theta \log p(x|\theta) f(x)]$
- (9) С помощью этого тождества мы можем заменить значения и получить вот такое уравнение $\longrightarrow \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log p(\tau|\pi_\theta) G(\tau)]$
- (10) Обратите внимание на зависимость от вероятности траектории
- (11) Если заменить вероятность траектории, вычислить логарифм, записать произведения в виде суммы и дифференцировать по значению θ , всякая зависимость от переходной функции пропадет и у нас останется функция, с которой мы можем работать дальше $\longrightarrow \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi(A_t|S_t; \pi_\theta) G(\tau) \right]$

Сокращение дисперсии градиента политик

Было бы полезно уметь вычислять градиент политик без какой-либо информации о переходной функции среды. Этот алгоритм повышает логарифм вероятности всех действий вдоль траектории прямо пропорционально положительности полной выгоды. Проще говоря, мы собираем полную траекторию и вычисляем полную дисконтированную выгоду, а затем используем полученный результат для взвешивания логарифма вероятности каждого действия, выбранного на этой траектории: $A_t, A_{t+1} \dots A_{T-1}$.

Используем только награды, полученные в результате действий



Я знаю математику

Сокращение дисперсии градиента политик

(1) Это градиент, который мы пытаемся спрогнозировать в алгоритме REINFORCE (об этом дальше)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G_t(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right]$$

(2) Это просто значит, что мы собираем данные вдоль траектории

(3) Вычисляем выгоду для каждого шага траектории

(4) И используем это значение для взвешивания логарифма вероятности действия, выбранного на этом временном шаге

0001 Немного истории

Появление алгоритма REINFORCE

Рональд Дж. Уильямс (Ronald J. Williams) предложил семейство алгоритмов REINFORCE в 1992 году в научной работе *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning*.

В 1986 году была опубликована статья *Learning representations by back-propagating errors*, которую Уильямс написал в соавторстве с Джеффри Хинтоном и др. Она дала толчок развитию исследований в области искусственных нейронных сетей (artificial neural network, ANN) того времени.



Я знаю Python

REINFORCE

```
class REINFORCE():
    <...>

    def optimize_model(self):
        T = len(self.rewards)

        discounts = np.logspace(0, T, num=T, base=self.gamma,
                                endpoint=False)

        returns = np.array(
            [np.sum(discounts[:T-t] * self.rewards[t:]) \
             for t in range(T)])

        policy_loss = -(discounts * returns * \
                        self.logpas).mean()
```

(1) Это алгоритм REINFORCE. Часть кода была опущена для простоты. Полный исходный код можно найти в блокноте Jupyter к этой главе

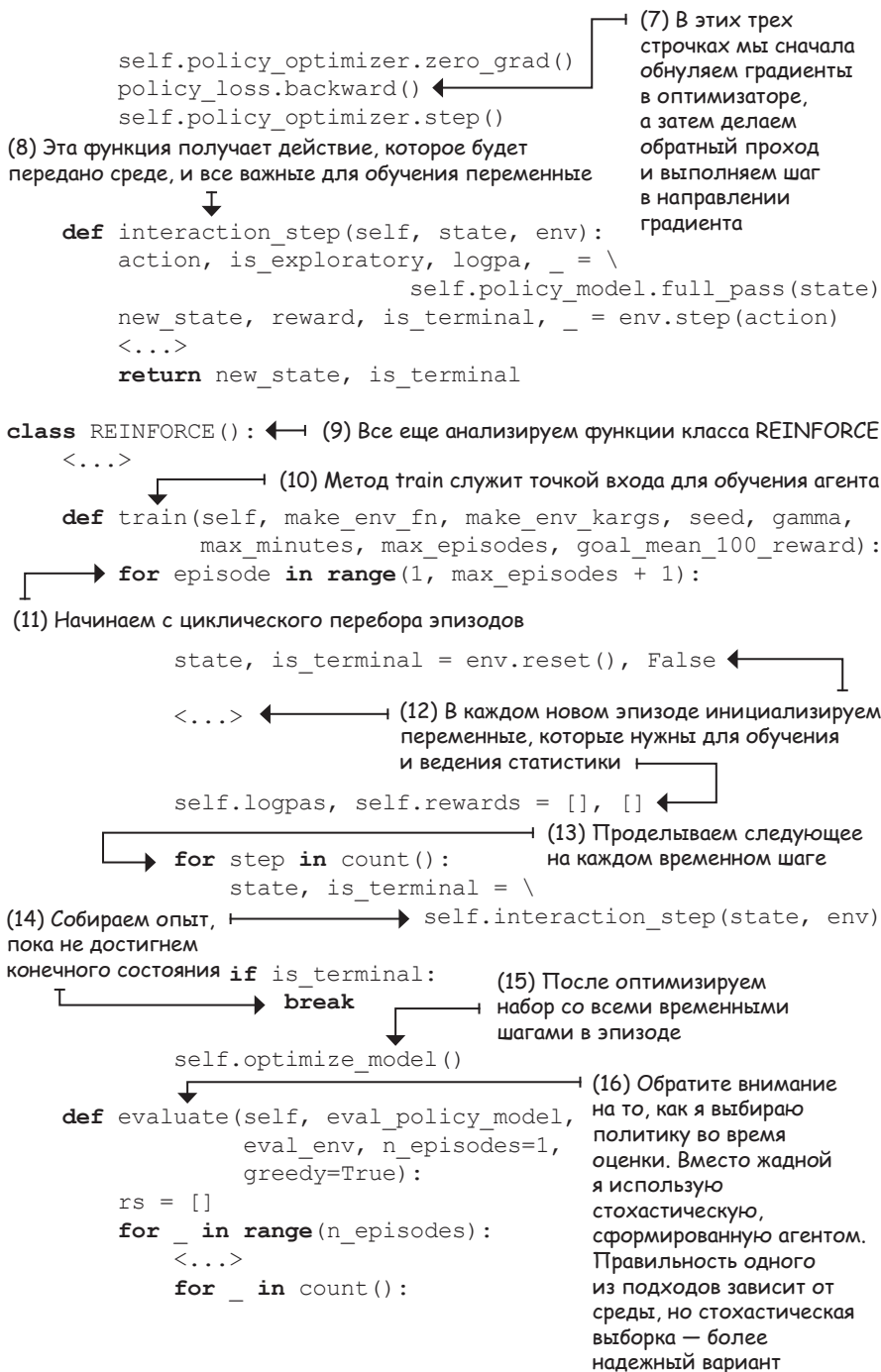
(2) Сначала вычисляем коэффициенты дисконтирования, как в методах Монте-Карло. Функция `logspace` с этими параметрами возвращает список значений гамма, по одному для каждого временного шага, например [1; 0,99; 0,9801...]

(3) Теперь находим сумму дисконтированной выгоды для всех временных шагов

(4) Подчеркиваю: это выгода для каждого временного шага в эпизоде, от начального состояния на шаге 0 до предпоследнего, $T - 1$

(5) Обратите внимание, что здесь мы используем математически правильное обновление градиента политик, что происходит нечасто. Дополнительный коэффициент дисконтирования подразумевает, что мы пытаемся оптимизировать ожидаемую дисконтированную выгоду от начального состояния, чтобы выгода, полученная позже в том же эпизоде, тоже была дисконтированной

(6) Это потеря политики: логарифм вероятности того, что выбранное действие будет взвешено с помощью выгоды. Поскольку PyTorch по умолчанию выполняет градиентный спуск и мы хотим максимизировать производительность, здесь используется отрицательное среднее, чтобы функция была перевернутой. Можете считать это градиентным подъемом по производительности. Кроме того, здесь учитываются дисконтированные градиенты политик, поэтому выгода умножается на коэффициенты дисконтирования



```

if greedy:
    a = eval_policy_model.\
        select_greedy_action(s)
else:
    a = eval_policy_model.select_action(s)
s, r, d, _ = eval_env.step(a)
<...>
return np.mean(rs), np.std(rs)

```

VPG: формирование функции ценности

Алгоритм REINFORCE, с которым вы познакомились ранее, хорошо работает в простых средах и дает гарантии сходимости. Но, так как для вычисления градиента используется полная выгода Монте-Карло, у этого алгоритма возникает проблема с дисперсией. В этом разделе мы обсудим несколько методов борьбы с этой проблемой, известных как *базовый градиент политик* (vanilla policy gradient) или *REINFORCE с направляющей* (REINFORCE with baseline).

Дальнейшее сокращение дисперсии градиента политик

REINFORCE — это базовый алгоритм, склонный к высокой дисперсии. Вы, наверное, помните об этом, но повторить не помешает. Накапливание случайных событий вдоль траектории, включая исходное состояние, выбранное из начального распределения (вероятности переходной функции, но в этой главе вместе со стохастическими политиками), — это элемент случайности, привносимый выбором действий. Вся эта стохастичность накапливается внутри выгоды, превращая ее в сильно искаженный сигнал, который сложно интерпретировать.

Для сокращения дисперсии вместо полной выгоды можно использовать частичную, меняя логарифм вероятности действий. Это улучшение мы уже внедрили. Но остается еще одна проблема: логарифм вероятности меняется прямо пропорционально изменению выгоды. Это значит, что, если мы получаем весомую положительную выгоду, вероятность приведших к ней действий значительно повышается. Если же выгода отрицательная, эти вероятности сильно снижаются.

Но представьте себе среду вроде «перевернутого маятника», где все награды и выгода положительные. Чтобы полностью отделить хорошие действия от лучших, нам потребуется много данных, иначе дисперсию будет сложно смягчить. Было бы удобно, если бы мы могли вместо искаженной выгоды использовать нечто позволяющее различать разные ценности действий в одном и том же состоянии. Ничего не напоминает?



Освежим в памяти

Использование прогнозируемых преимуществ в методах градиента политик

(1) Вспомните определение реальной функции преимуществ действий

$$\longrightarrow a_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$$

(2) Функция преимущества имеет примерно такой вид

$$\hookrightarrow A(S_t, A_t) \approx R_t + \gamma R_{t+1} + \dots + \gamma^{T-1} R_T - v_{\pi}(S_t)$$

(3) Не самый плохой прогноз здесь — выгода G_t за вычетом спрогнозированной выгоды, ожидаемой в этом состоянии.

Мы можем легко этим воспользоваться $\longrightarrow A(S_t, A_t) = G_t - V(S_t)$

Формирование функции ценности

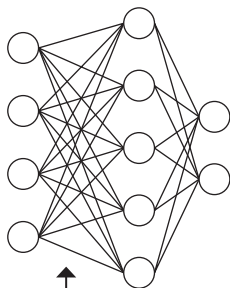
Как видно выше, мы можем сократить дисперсию градиента политик еще сильнее, используя прогноз функции преимущества действий вместо фактической выгоды. Это позволяет как бы сосредоточить результат вокруг нуля: у функций с результатами выше среднего рейтинг положительный, а у тех, что ниже среднего, — отрицательный. Первые снижают вероятности, а вторые их повышают.

Именно этим мы и займемся. Создадим две нейросети: одну для формирования политики, а другую — для формирования функции ценности состояний, V . После мы воспользуемся полученной функцией и выгодой для вычисления функции преимущества.

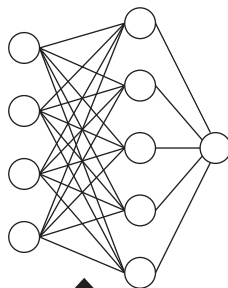
Две нейронные сети: одна для политики, другая для функции ценности

(1) Для среды «перевернутый маятник» используется та же сеть политики, что применялась в REINFORCE: входной слой с четырьмя узлами и двухузловой выходной. Подробнее об этом мы поговорим чуть позже

Сеть политики



Сеть функции ценности



(2) У используемой нами сети функции ценности тоже четырехузловой ввод, представляющий состояние, и выход с одним узлом, представляющий значение этого состояния. Сеть выводит выгоду, ожидаемую от входного состояния. Подробнее об этом позже

$\checkmark$ RL

С акцентом на RL

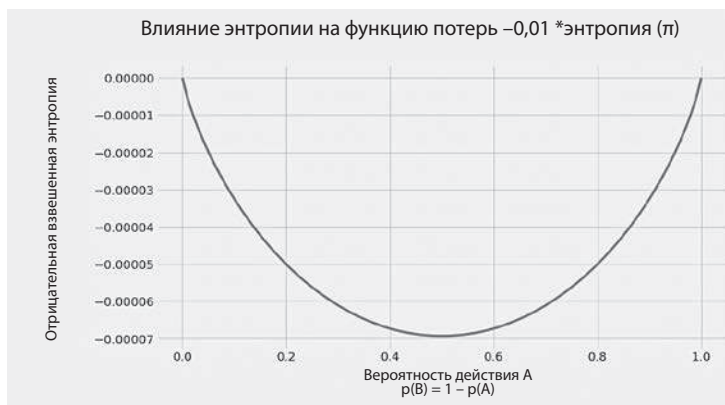
REINFORCE, базовый градиент политик, направляющие и «актер-критик»

Читатели, которые сталкивались с DRL, могут спросить: «Идет ли здесь речь о методе “актер-критик”?» Все сходится: мы формируем политику и функцию ценности. Но это как раз один из тех случаев, когда «акцент на RL» сбивает новичков с толку. И вот почему.

Во-первых, согласно одному из основателей RL, Ричу Саттону, методы градиента политик аппроксимируют градиент показателя производительности вне зависимости от того, формируют они приблизительную функцию ценности или нет. Но Дэвид Сильвер (David Silver), один из выдающихся деятелей в области DRL и бывший студент Саттона, с этим не согласен. Он утверждает, что стратегические методы не формируют дополнительно функцию ценности и что это делают только методы «актер-критик». Но далее Саттон объясняет, что к методам «актер-критик» относятся только те, которые формируют функцию ценности с помощью бэджинга, так как именно он добавляет к этой функции смещения, превращая ее в «критика». Мне нравится эта особенность: так, алгоритмы REINFORCE и VPG из этой книги не считаются методами «актер-критик». Но имейте в виду, что жаргон вариативен.

Поощрение исследования

Еще один незаменимый способ улучшения методов градиента политик — добавление элемента энтропии в функцию потерь. Под энтропией можно понимать много чего: от объема информации, который можно получить с помощью выборки из распределения, до количества возможных видов сортировки множества.



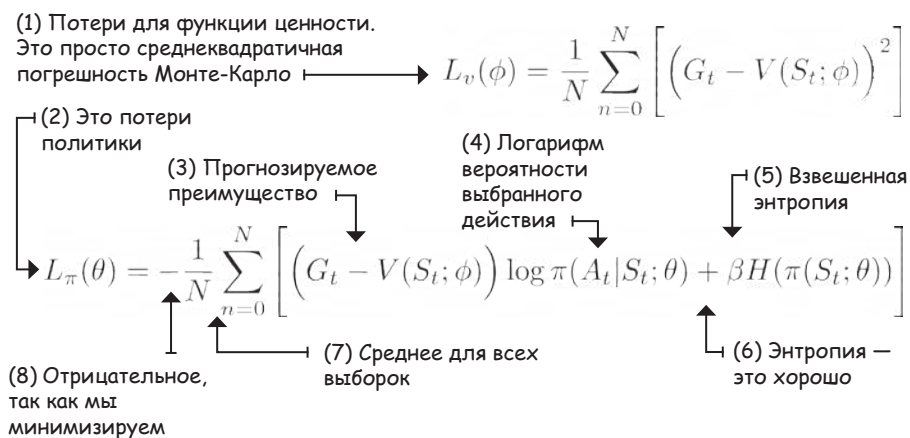
Мой взгляд на энтропию довольно прост. У равномерного распределения (с равномерно распределенными выборками) высокий уровень энтропии — если точнее, самый высокий из возможных. Например, если у вас есть две выборки, вероятность извлечения каждой из которых 50 %, энтропия будет максимально высокой для множества из двух выборок. Если у вас есть четыре выборки, каждая из которых извлекается с вероятностью 25 %, энтропия будет такой же — максимально высокой для множества из четырех выборок. Но если у вас есть две выборки, одна из которых извлекается в 100 % случаев, а другая — никогда, то энтропия будет наименьшей из возможных — всегда равной нулю. В PyTorch для вычисления энтропии вместо двоичного логарифма используется натуральный. Так происходит потому, что натуральный логарифм использует число Эйлера, e , что делает вычисления более «естественными». Но на практике результат всегда один. Энтропия в среде «перевернутый маятник», где предусмотрено два действия, находится в пределах между 0 и 0,6931.

В методах градиента политик отрицательная взвешенная энтропия прибавляется к функции потерь, чтобы стимулировать получение равномерно распределенных действий. В результате политика, где действия распределены наиболее равномерно (то есть обладают максимальной энтропией), способствует снижению уровня потерь. С другой стороны, схождение на одном действии (при нулевой энтропии) не уменьшает число потерь. Тогда агенту лучше сойтись на оптимальном действии.



Я знаю математику

Потери, используемые в VPG





Я знаю Python

Модель нейронной сети для функции ценности состояний

```

class FCV(nn.Module):
    def __init__(self,
                  input_dim,
                  hidden_dims=(32, 32),
                  activation_fc=F.relu):
        super(FCV, self).__init__()
        self.activation_fc = activation_fc

        self.input_layer = nn.Linear(input_dim,
                                      hidden_dims[0])

        self.hidden_layers = nn.ModuleList()
        for i in range(len(hidden_dims)-1):
            hidden_layer = nn.Linear(
                hidden_dims[i], hidden_dims[i+1])
            self.hidden_layers.append(hidden_layer)

        self.output_layer = nn.Linear(
            hidden_dims[-1], 1)

        def forward(self, state):
            x = state
            if not isinstance(x, torch.Tensor):
                x = torch.tensor(x, dtype=torch.float32)
                x = x.unsqueeze(0)

            x = self.activation_fc(self.input_layer(x))
            for hidden_layer in self.hidden_layers:
                x = self.activation_fc(hidden_layer(x))

            return self.output_layer(x)

```

(1) Эта нейросеть функции ценности состояний похожа на сеть Q-функции, которую мы использовали ранее

(2) Заметьте, что здесь вы можете менять гиперпараметры. Экспериментируйте!

(3) Создаем линейные соединения между входными узлами и первым скрытым слоем

(4) Здесь создаются соединения между скрытыми слоями

(5) Подключаем последний скрытый слой к выводу, состоящему из одного узла, который представляет значение состояния

(6) Это функция прямого прохода

(7) Форматирование ввода в соответствии с нашими ожиданиями

(8) Выполняем полный прямой проход...

(9) ...и возвращаем значение состояния



Я знаю Python

Базовый градиент политик (REINFORCE с направляющей)

```
class VPG():
    <...>

    def optimize_model(self):
        T = len(self.rewards)
        discounts = np.logspace(0, T, num=T, base=self.gamma,
                                endpoint=False)

        returns = np.array(
            [np.sum(discounts[:T-t] * self.rewards[t:]) for t in range(T)])

        value_error = returns - self.values
        policy_loss = -(
            discounts * value_error.detach() * self.logpas).mean()

        entropy_loss = -self.entropy_loss_weight * self.entropy_loss

        self.policy_optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(
            self.policy_model.parameters(),
            self.policy_model_max_grad_norm)

        self.policy_optimizer.step()

        value_loss = value_error.pow(2).mul(0.5).mean()
        self.value_optimizer.zero_grad()
        value_loss.backward()
        torch.nn.utils.clip_grad_norm_(
            self.value_model.parameters(),
            self.value_model_max_grad_norm)

        self.value_optimizer.step()
```

(1) Это алгоритм VPG. Я опустил много кода. Полную реализацию вы можете найти в блокноте Jupyter к этой главе

(2) Очень удобный способ вычисления суммы дисконтированных наград от временного шага 0 до T

(3) Заметьте, что этот цикл перебирает все шаги от 0 (1, 2, 3 и вплоть до конечного состояния T) и вычисляет выгоду состояния, равную сумме дисконтированных наград, начиная с этого состояния на временном шаге t и заканчивая конечным, T

(4) Находим погрешность ценности и используем ее для получения логарифма вероятности действий. Затем дисконтируем эти вероятности, чтобы сделать их совместимыми с градиентом политик. После используем отрицательное среднее

(5) Вычисляем энтропию и прибавляем дробь к потерям

(6) Теперь оптимизируем политику. Обнуляем оптимизатор, выполняем обратный проход и при желании ограничиваем градиент

(7) Переводим оптимизатор на шаг вперед

(8) В конце оптимизируем нейросеть функции ценности

АЗС: параллельное обновление политики

Метод VPG довольно хорошо себя проявляет в простых задачах: у него обычно нет смещения, так как в нем применяется несмещенная цель для формирования как политики, так и функции ценности. То есть он использует выгоду Монте-Карло — полную реальную выгоду, полученную в среде, без бэггинга. Единственный фактор, который может привести к смещению в этом алгоритме, — аппроксимация функций, которая по своей природе порождает это смещение. Но, поскольку ANN — это лишь направляющая для сокращения дисперсии реальной выгоды, смещения либо нет, либо оно получается совсем небольшим.

Но смещение в алгоритмах — это не всегда что-то плохое. Оно нередко добавляется для сокращения дисперсии. Так называемый *асинхронный алгоритм «актер-критик» с преимуществом* (asynchronous advantage actor-critic, АЗС) предусматривает для борьбы с дисперсией несколько дополнительных мер. Во-первых, он использует n -шаговую выгоду с бэггингом для формирования политики и функции ценности, во-вторых, у него есть сразу несколько «актеров» для параллельной генерации множества выборов. Давайте поближе с ним познакомимся.

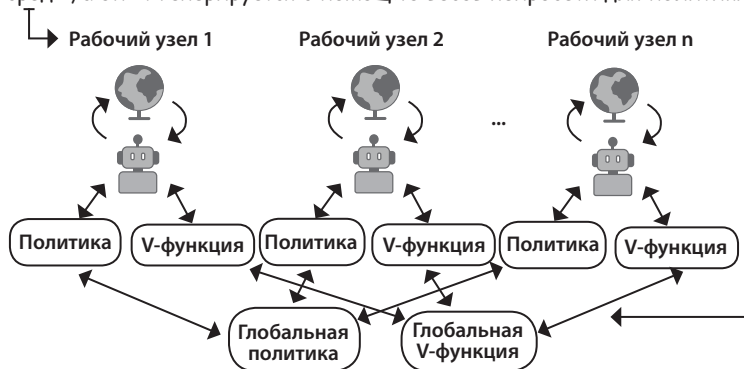
Использование рабочих узлов

Один из основных источников дисперсии алгоритмов DRL — это степень корреляции и нестационарности динамических выборов. В ценностно ориентированных методах применяется буфер воспроизведения для равномерного выбора мини-наборов, состоящих в основном из независимых и одинаково распределенных данных. К сожалению, использовать такое воспроизведение опыта для сокращения дисперсии можно только в нестратегических методах, так как стратегические агенты не могут повторно использовать данные, сгенерированные прошлыми политиками. Проще говоря, на каждом этапе оптимизации нужен свежий набор стратегических выборов.

Что можно предпринять вместо использования буфера воспроизведения в стратегических методах вроде алгоритмов градиента политик из этой главы? Мы можем создать несколько рабочих узлов, которые будут параллельно генерировать опыт, асинхронно обновляя политику и функцию ценности. Наличие в среде нескольких таких узлов, работающих параллельно, ослабляет корреляцию данных, применяемых для обучения, и снижает дисперсию алгоритма.

Асинхронное обновление модели

(1) В АЗС у нас много рабочих узлов для обучения. Каждый из них создает экземпляр среды, а опыт генерируется с помощью весов нейросети для политики и V-функции



(2) После сбора опыта каждый узел обновляет глобальную модель асинхронно, не координируя свои действия ни с одним из других узлов, а после перезагружает свои копии модели и продолжает работать



Я знаю Python

Логика рабочих узлов в АЗС

```
class АЗС():
    <...>
    def work(self, rank):
        local_seed = self.seed + rank
        env = self.make_env_fn(
            **self.make_env_kargs,
            seed=local_seed)
        torch.manual_seed(local_seed)
        np.random.seed(local_seed)
        random.seed(local_seed)
        nS = env.observation_space.shape[0]
        nA = env.action_space.n
        local_policy_model = self.policy_model_fn(nS, nA)
        local_policy_model.load_state_dict(
            self.shared_policy_model.state_dict())
```

(1) Это агент АЗС

(2) Как обычно, это все — фрагменты. Вы знаете, где искать рабочий код

(3) Это функция `work`, внутри которой каждый рабочий узел выполняет свой цикл

(4) Обратите внимание на то, как мы создаем уникальное исходное значение для каждого рабочего узла. Нам нужен разнообразный опыт

(5) Создаем для каждого узла среду с уникальным начальным значением

(6) Это же уникальное значение используется для PyTorch, NumPy и Python

(7) Удобные переменные

(8) Здесь мы создаем модель локальной политики. Заметьте, как мы инициализируем ее с помощью весов из сети общей политики, позволяющей нам периодически синхронизировать агенты

```

(9) Прodelываем то же с моделью ценности.
Важно: в выходных слоях параметр nA не нужен
local_value_model = self.value_model_fn(nS)
local_value_model.load_state_dict(
    self.shared_value_model.state_dict())
(10) Выполняем цикл обучения, пока рабочий
    узел не подаст сигнал о намерении его покинуть
while not self.get_out_signal:
    state, is_terminal = env.reset(), False
(11) Сбрасываем среду и присваиваем false флагу done или is_terminal
(12) Используем n-шаговую выгоду для формирования политики и функций ценности
    n_steps_start = 0
    logpas, entropies, rewards, values = [], [], [], []

    for step in count(start=1):
(13) Мы в цикле обработки эпизодов. Сначала нужно собрать шаг опыта
        state, reward, is_terminal, is_truncated, \
            is_exploratory = self.interaction_step(
                state, env, local_policy_model,
                local_value_model, logpas,
                entropies, rewards, values)

        if is_terminal or step - n_steps_start == \
            self.max_n_steps:
(14) Собираем максимум n-шагов. При попадании
            в терминальное состояние останавливаемся
            is_failure = is_terminal and not is_truncated
(15) Убеждаемся, что это действительно конечное состояние, а не результат
            сбрасывания временной оболочки
            next_value = 0 if is_failure else \
                local_value_model(state).detach().item()
(16) В случае отказа ценность следующего состояния равна 0,
            в противном случае выполняем бэггинг
            rewards.append(next_value)

(17) Смотрите! Я здесь схитрил и добавил к наградам next_value. Как вы
            скоро увидите, это позволило мне оставить код оптимизации из VPG
            почти без изменений. Запомните это
            self.optimize_model(
                logpas, entropies, rewards, values,
                local_policy_model, local_value_model)
(18) Теперь оптимизируем модель. Эта функция будет рассмотрена чуть позже

            logpas, entropies, rewards, values = [], [], [], []
            n_steps_start = step
(19) Сбрасываем переменные после
            этапа оптимизации и продолжаем

            if is_terminal:
(20) Выходим из цикла обработки
                break
                эпизодов, если это конечное состояние
<...>

```

Использование n -шаговых прогнозов

Обратите внимание, что я добавлял к последовательности наград значение следующего состояния, независимо от того, конечное оно или нет. Значит, переменная `rewards` содержит все награды на частичной траектории и прогноз ценности этого последнего состояния. Можете считать, что мы храним в одном и том же месте частичную и прогнозируемую остающуюся выгоды: первая отражает последовательность наград, а вторая — прогноз в виде одного числа. Единственная причина, почему это нельзя считать обычной выгодой, в том, что мы не используем дисконтированную сумму, но это тоже можно исправить.

Важно понимать, что это n -шаговая выгода из главы 5. Мы собираем награды на протяжении n шагов и затем выполняем бэггинг после n -го состояния или перед ним, если то, в которое мы попали, конечное — смотря что произойдет раньше.

У АЗС более низкая дисперсия n -шаговой выгоды по сравнению с Монте-Карло. Мы используем функцию ценности и для прогнозирования выгоды, с помощью которой обновляется политика. Как вы помните, бэггинг снижает дисперсию, но создает смещение. Так в нашем алгоритме градиента политик появился «критик». Добро пожаловать в мир методов «актер-критик».



Я знаю математику

Использование прогнозов на основе n -шагового бэггинга

(1) Раньше мы применяли для прогнозирования наших преимуществ полную выгоду

$$A(S_t, A_t; \phi) = G_t - V(S_t; \phi)$$

(2) Теперь же мы используем n -шаговую выгоду с бэггингом

$$A(S_t, A_t; \phi) = R_t + \gamma R_{t+1} + \dots + \gamma^n R_{t+n} + \gamma^{n+1} V(S_{t+n+1}; \phi) - V(S_t; \phi)$$

(3) Теперь для обновления вероятностей действий используется прогноз n -шаговой выгоды

$$L_{\pi}(\theta) = -\frac{1}{N} \sum_{n=0}^N \left[A(S_t, A_t; \phi) \log \pi(A_t | S_t; \theta) + \beta H(\pi(S_t; \theta)) \right]$$

(4) С помощью n -шаговой выгоды мы улучшаем прогноз функции ценности. Обратите внимание на бэггинг. Именно благодаря ему этот алгоритм относится к методам «актер-критик»

$$L_v(\phi) = \frac{1}{N} \sum_{n=0}^N \left[\left(R_t + \gamma R_{t+1} + \dots + \gamma^n R_{t+n} + \gamma^{n+1} V(S_{t+n+1}; \phi) - V(S_t; \phi) \right)^2 \right]$$



Я знаю Python

Этап оптимизации в A3C

```
class A3C():
    <...>

    def optimize_model(
        self, logpas, entropies, rewards, values,
        local_policy_model, local_value_model):

        T = len(rewards)
        discounts = np.logspace(0, T, num=T, base=self.gamma,
                                endpoint=False)

        returns = np.array(
            [np.sum(discounts[:T-t] * rewards[t:]) for t in range(T)])

        discounts = torch.FloatTensor(
            discounts[:-1]).unsqueeze(1)
        returns = torch.FloatTensor(returns[:-1]).unsqueeze(1)

        value_error = returns - values

        policy_loss = -(discounts * value_error.detach() * \
                        logpas).mean()
        entropy_loss = -entropies.mean()
        loss = policy_loss + self.entropy_loss_weight * \
                entropy_loss

        self.shared_policy_optimizer.zero_grad()
        loss.backward()

        torch.nn.utils.clip_grad_norm_(
            local_policy_model.parameters(),
            self.policy_model_max_grad_norm)
```

(1) A3C, функция оптимизации

(2) Сначала получаем длину награды. Помните, награды содержат значение бэзгинга

(3) Затем вычисляем все коэффициенты дисконтирования вплоть до $n + 1$

(4) Теперь это спрогнозированная n -шаговая выгода

(5) Прежде чем продолжать, нам нужно удалить лишний элемент и привести переменные к ожидаемому формату

(6) Вычисляем погрешности значений, вычитая ожидаемые значения из спрогнозированной выгоды

(7) Вычисляем потери как обычно

(8) Заметьте, теперь мы обнуляем общий оптимизатор политик, а после вычисляем потери

(9) Ограничиваем величину градиента


```

    for param, shared_param in zip(
        local_policy_model.parameters(),
        self.shared_policy_model.parameters()):
        (10) Взгляните. Здесь мы перебираем сетевые
        параметры всех локальных
        и общих политик
        if shared_param.grad is None:
            (11) Нам нужно скопировать
            каждый градиент
            из локальной
            модели в общую
            shared_param._grad = param.grad
        (12) После этого переходим к этапу оптимизации
        self.shared_policy_optimizer.step()
        (13) Затем загружаем общую модель в локальную
        local_policy_model.load_state_dict(
            self.shared_policy_model.state_dict())

        (14) Делаем то же самое, но с сетью ценности состояний. Вычисляем потери
        value_loss = value_error.pow(2).mul(0.5).mean()

        (15) Обнуляем общий оптимизатор значений
        self.shared_value_optimizer.zero_grad()
        value_loss.backward()
        (16) Передаем градиенты
        в обратном направлении
        torch.nn.utils.clip_grad_norm_(
            local_value_model.parameters(),
            self.value_model_max_grad_norm)
        (17) Затем ограничиваем их

        (18) Копируем все градиенты из локальной модели в общую
        for param, shared_param in zip(
            local_value_model.parameters(),
            self.shared_value_model.parameters()):
            if shared_param.grad is None:
                shared_param._grad = param.grad

        self.shared_value_optimizer.step()
        (19) Переводим
        оптимизатор
        на шаг вперед

        (20) В конце загружаем модель
        в локальную переменную
        local_value_model.load_state_dict(
            self.shared_value_model.state_dict())

```

Неблокирующие обновления модели

Один из важнейших аспектов алгоритма АЗС в том, что обновления сети в нем происходят асинхронно, без блокировки. Учитывая наличие общей модели, квалифицированные разработчики ПО обычно используют механизм блокировки, чтобы рабочие узлы не могли перезаписывать чужие обновления. Интересно, что в АЗС применяется стиль обновления Hogwild! («безудержный»), который не только отражает близкие к оптимальному показатели схождения, но и на порядок превосходит альтернативные методы блокировки.



Я знаю Python

Общий оптимизатор Адама

```
class SharedAdam(torch.optim.Adam):  ← (1) Нам нужно создать
    <...>                             оптимизатор Адама (и RMSprop
                                     в блокноте), который
                                     записывает внутренние
                                     переменные в разделяемую
                                     память. К счастью, PyTorch
                                     упрощает этот процесс

    for group in self.param_groups:
        for p in group['params']:
            state = self.state[p]
            state['step'] = 0
            state['shared_step'] = \
                torch.zeros(1).share_memory_()
            state['exp_avg'] = \
                torch.zeros_like(p.data).share_memory_()

    <...>  (2) Нам нужно только вызвать метод share_memory_
(3) Здесь показаны не все переменные      из переменных, которые должны быть общими
                                     для рабочих узлов

    def step(self, closure=None):
        for group in self.param_groups:
            for p in group['params']:
                if p.grad is None: continue
                self.state[p]['steps'] = \
                    self.state[p]['shared_step'].item()
                self.state[p]['shared_step'] += 1
            super().step(closure)  ← (4) Переопределяем функ-
                                     цию step, чтобы иметь
                                     возможность вручную
                                     инкрементировать пере-
                                     менную шага, которую
                                     не просто разместить
                                     в разделяемой памяти
                                     (5) В конце вызываем родительскую
                                     версию step
```

0001 Немного истории

История появления асинхронного алгоритма «актер-критик» с преимуществом (A3C)

Алгоритм A3C был впервые представлен Владом Мнихом и др. в научной работе *Asynchronous Methods for Deep Reinforcement Learning*. Если помните, Влад еще и автор агента DQN (научные работы 2013 и 2015 годов). Если DQN активизировал исследования в области DRL в целом, то алгоритм A3C привлек внимание к методам «актер-критик».

GAE: надежное прогнозирование преимущества

A3C снижает дисперсию целей за счет n -шаговой выгоды. Но, как вы помните из главы 5, есть более стабильный метод, объединяющий множество n -шаговых целей бэггинга в одну. Это делает цели еще устойчивее по сравнению с единой n -шаговой выгодой — λ -целью. *Обобщенное прогнозирование преимущества* (generalized advantage estimation, GAE) — это аналог λ -целей в TD(λ), только для преимуществ.

Обобщенное прогнозирование преимущества

GAE — это не агент как таковой, а способ прогнозирования целей для функции преимущества, который можно применять в большинстве методов «актер-критик». В GAE применяется экспоненциально взвешенное сочетание n -шаговых целей функции преимущества действий, точно так же как λ -цель — это экспоненциально взвешенное сочетание n -шаговых целей функции ценности состояний. Такие цели, которые регулируются так же, как и λ -цель, могут сильно снизить дисперсию прогнозов градиента политик за счет некоторого смещения.



Я знаю математику

Потенциальные способы оценки градиента политик

(1) В методах градиента политик и «актер-критик» мы пытаемся спрогнозировать такой градиент

$$g = \mathbb{E} \left[\sum_{t=0}^{\infty} \Psi_t \nabla_{\theta} \log \pi(A_t | S_t; \theta) \right]$$

(2) Пси можно заменить разными выражениями, прогнозирующими рейтинг с разной степенью дисперсии и смещения

(3) Это общая выгода, от шага 0 и до самого конца

$$\Psi_t = \sum_{t=0}^T \gamma^t R_t$$

(4) Но начинать можно и с текущего временного шага, как мы это делали в REINFORCE, двигаясь к концу эпизода

$$\Psi_t = \sum_{t'=t}^T \gamma^{t'-t} R_{t'}$$

(5) Как и в VPG, мы можем использовать направляющую, которой в нашем случае была функция ценности состояний

$$\Psi_t = \sum_{t'=t}^T \gamma^{t'-t} R_{t'} - b(S_t)$$

(6) В АЗС мы использовали n -шаговый прогноз преимущества, дающий самую низкую дисперсию

$$\Psi_t = a_{\pi}(S_t, A_t)$$

(7) Но мы могли бы воспользоваться и реальной функцией ценности действий...

$$\Psi_t = q_{\pi}(S_t, A_t)$$

(8) ...или даже остатком TD, который можно считать одношаговым прогнозом преимущества

$$\Psi_t = R_t + v_{\pi}(S_{t+1}) - v_{\pi}(S_t)$$



Я знаю математику

GAE — это стабильный прогноз функции преимущества

$$A^1(S_t, A_t; \phi) = R_t + \gamma V(S_{t+1}; \phi) - V(S_t; \phi) \longleftarrow (1) \text{ N-шаговые прогнозы преимущества...}$$

$$A^2(S_t, A_t; \phi) = R_t + \gamma R_{t+1} + \gamma^2 V(S_{t+2}; \phi) - V(S_t; \phi)$$

$$A^3(S_t, A_t; \phi) = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 V(S_{t+3}; \phi) - V(S_t; \phi)$$

...

$$A^n(S_t, A_t; \phi) = R_t + \gamma R_{t+1} + \dots + \gamma^n R_{t+n} + \gamma^{n+1} V(S_{t+n+1}; \phi) - V(S_t; \phi)$$

(2) ...которые можно смешать для получения аналога TD(λ), но для преимуществ

$$A^{GAE(\gamma, \lambda)}(S_t, A_t; \phi) = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

(3) Аналогично лямбда 0 означает одношаговый прогноз преимущества, а лямбда 1 — прогноз преимущества для бесконечного числа шагов

$$A^{GAE(\gamma, 0)}(S_t, A_t; \phi) = R_t + \gamma V(S_{t+1}; \phi) - V(S_t; \phi)$$

$$A^{GAE(\gamma, 1)}(S_t, A_t; \phi) = \sum_{l=0}^{\infty} \gamma^l R_{t+l} - V(S_t; \phi)$$



Я знаю математику

Потенциальные цели значений

(1) Заметьте, мы можем использовать несколько разных целей для обучения нейросети функции ценности состояний, применяемой для вычисления значений GAE

(2) Мы могли бы применить ожидаемую награду, как с выгодой Монте-Карло

$$y_t = \sum_{t'=t}^T \gamma^{t'-t} R_{t'}$$

$$y_t = R_t + \gamma R_{t+1} + \dots + \gamma^n R_{t+n} + \gamma^{n+1} V(S_{t+n+1}; \phi)$$

(3) N-шаговая цель бэггинга, включая цель TD

$$y_t = A^{GAE(\gamma, \lambda)}(S_t, A_t; \phi) + V(S_t; \phi)$$

(4) Или GAE, как прогноз TD(λ)

0001

Немного истории

Появление обобщенного прогнозирования преимущества

В 2015 году Джон Шульман (John Schulman) и др. опубликовали научную работу *High-dimensional Continuous Control Using Generalized Advantage Estimation*, где был предложен метод GAE.

Джон работает научным сотрудником в OpenAI. Он один из главных создателей GAE, TRPO и PPO — алгоритмов, с которыми вы познакомитесь в следующей главе. В 2018 году Джон был назван одним из лучших новаторов младше 35 лет за изобретение алгоритмов, которые по сей день считаются передовыми.



Я знаю Python

Этап оптимизации политик в GAE

```

class GAE():
    <...>
    def optimize_model(
        self, logpas, entropies, rewards, values,
        local_policy_model, local_value_model):

(2) Сначала создаем дисконтированную выгоду, как в A3C
        T = len(rewards)
        discounts = np.logspace(0, T, num=T, base=self.gamma,
                                endpoint=False)

        returns = np.array(
            [np.sum(discounts[:T-t] * rewards[t:]) for t in range(T)])

(3) Эти две строчки создают сначала массив NumPy со значениями всех
    состояний, а затем массив с  $(\gamma \times \lambda)^t$ . Лямбду часто обозначают
    как тау. Я тоже так делаю
        np_values = values.view(-1).data.numpy()
        tau_discounts = np.logspace(0, T-1, num=T-1,
                                    base=self.gamma*self.tau, endpoint=False)

(4) Эта строчка создает массив погрешностей TD:
     $R_t + \gamma \times \text{value}_{t+1} - \text{value}_t$ , от  $t = 0$  до  $T$ 
        advs = rewards[:-1] + self.gamma * \
                np_values[1:] - np_values[:-1]

(5) Здесь мы создаем обобщенные прогнозы
    преимущества (GAE), умножая коэффициенты
    дисконтирования тау на погрешности TD
        gaes = np.array(
            [np.sum(tau_discounts[:T-1-t] * advs[t:]) for t in range(T-1)])

    <...>
(6) Теперь используем GAE для вычисления
    потерь политики
        policy_loss = -(discounts * gaes.detach() * \
                        logpas).mean()

        entropy_loss = -entropies.mean()
        loss = policy_loss + self.entropy_loss_weight * \
                entropy_loss

(7) И продолжаем как обычно
        value_error = returns - values
        value_loss = value_error.pow(2).mul(0.5).mean()
    <...>

```

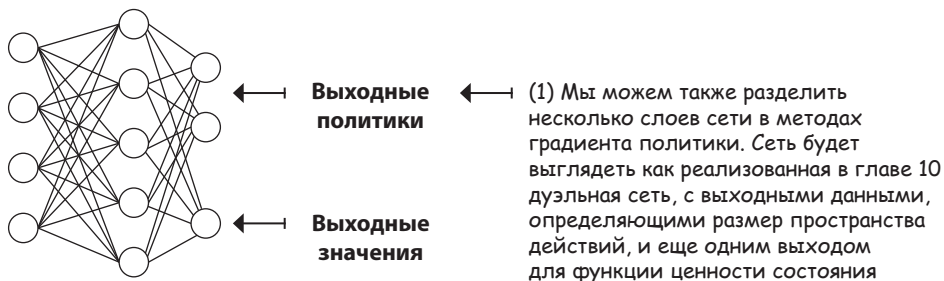
A2C: синхронное обновление политик

В A3C рабочие узлы обновляют нейросети асинхронно. Но это не объясняет высокую производительность алгоритма. «Актер-критик» с преимуществом (advantage actor-critic, A2C) — это синхронная версия A3C, которая, несмотря на меньшую цифру в названии, была предложена после A3C и показала свою эффективность, сравнимую с ней. В этом разделе мы исследуем A2C вместе с другими изменениями, которые можно внести в методы градиента политик.

Модель разделения весов

Текущий алгоритм можно изменить так, чтобы он использовал одну нейросеть как для политики, так и для функции ценности. Разделение модели может оказаться особенно полезным при обучении на изображениях, так как для извлечения закономерностей иногда нужно много вычислительных ресурсов. Но из-за потенциально разных масштабов обновлений политики и функции ценности этот подход может быть сложен в реализации.

Разделение весов между выходными политиками и значениями



Я знаю Python

Модель нейронной сети «актер-критик» с разделением веса

```
class FCAC(nn.Module): ← (1) Это полносвязная модель «актер-критик»
    def __init__(
        self, input_dim, output_dim,
        hidden_dims=(32,32), activation_fc=F.relu):
    (2) Это процесс создания экземпляра сети, похожий на тот,
    что мы видели в модели с независимой сетью
        super(FCAC, self).__init__()
        self.activation_fc = activation_fc
        self.input_layer = nn.Linear(input_dim, hidden_dims[0])
        self.hidden_layers = nn.ModuleList()
```

```

    for i in range(len(hidden_dims)-1):
        hidden_layer = nn.Linear(
            hidden_dims[i], hidden_dims[i+1])
        self.hidden_layers.append(hidden_layer)
    self.value_output_layer = nn.Linear(
        self.value_output_layer = nn.Linear(
            hidden_dims[-1], 1)
    self.policy_output_layer = nn.Linear(
        hidden_dims[-1], output_dim)

def forward(self, state):
    x = state
    if not isinstance(x, torch.Tensor):
        x = torch.tensor(x, dtype=torch.float32)
        if len(x.size()) == 1:
            x = x.unsqueeze(0)
    x = self.activation_fc(self.input_layer(x))
    for hidden_layer in self.hidden_layers:
        x = self.activation_fc(hidden_layer(x))
    return self.policy_output_layer(x), \
           self.value_output_layer(x)

def full_pass(self, state):
    logits, value = self.forward(state)
    dist = torch.distributions.Categorical(logits=logits)
    action = dist.sample()
    logpa = dist.log_prob(action).unsqueeze(-1)
    entropy = dist.entropy().unsqueeze(-1)
    action = action.item() if len(action) == 1 \
        else action.data.numpy()
    is_exploratory = action != np.argmax(
        logits.detach().numpy(), axis=int(len(state)!=1))
    return action, is_exploratory, logpa, entropy, value

def select_action(self, state):
    logits, _ = self.forward(state)
    dist = torch.distributions.Categorical(logits=logits)
    action = dist.sample()
    action = action.item() if len(action) == 1 \
        else action.data.numpy()
    return action

```

(3) Здесь формируется сеть, включая выходы оценок и политики, соединенные с последним скрытым слоем

(4) Прямой проход начинается с приведения ввода к ожидаемому типу и размерности

(5) Обратите внимание на вывод слоев политики и оценок

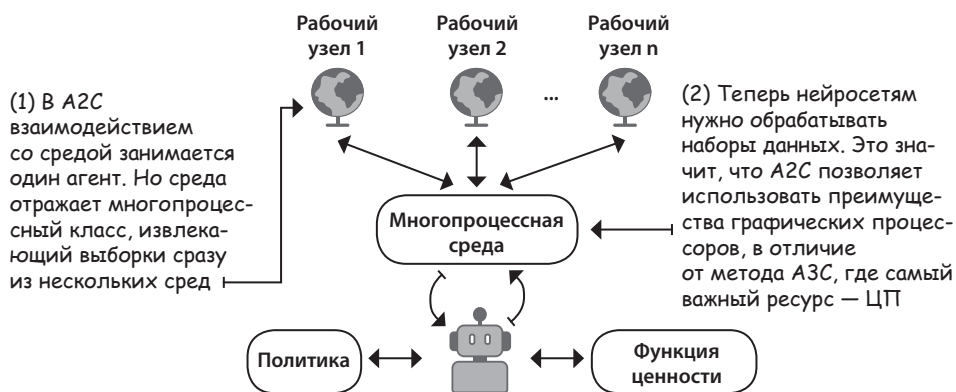
(6) Это удобная функция для одновременного получения логарифмических вероятностей, энтропий и других переменных

(7) Этот код выбирает действие/действия для заданного состояния или их набора

Упорядочение процесса обновления политики

Обновление нейросети в стиле Hogwild! может быть хаотичным, но внедрение механизма блокирования сильно снижает производительность АЗС. В А2С мы переносим рабочие узлы из агента в среду, а вместо нескольких узлов обучения используем один. Как оказалось, преимущество методов градиента политик в том, что рабочие узлы делятся накопленным опытом.

Синхронное обновление модели



Я знаю Python

Многопроцессорная оболочка среды

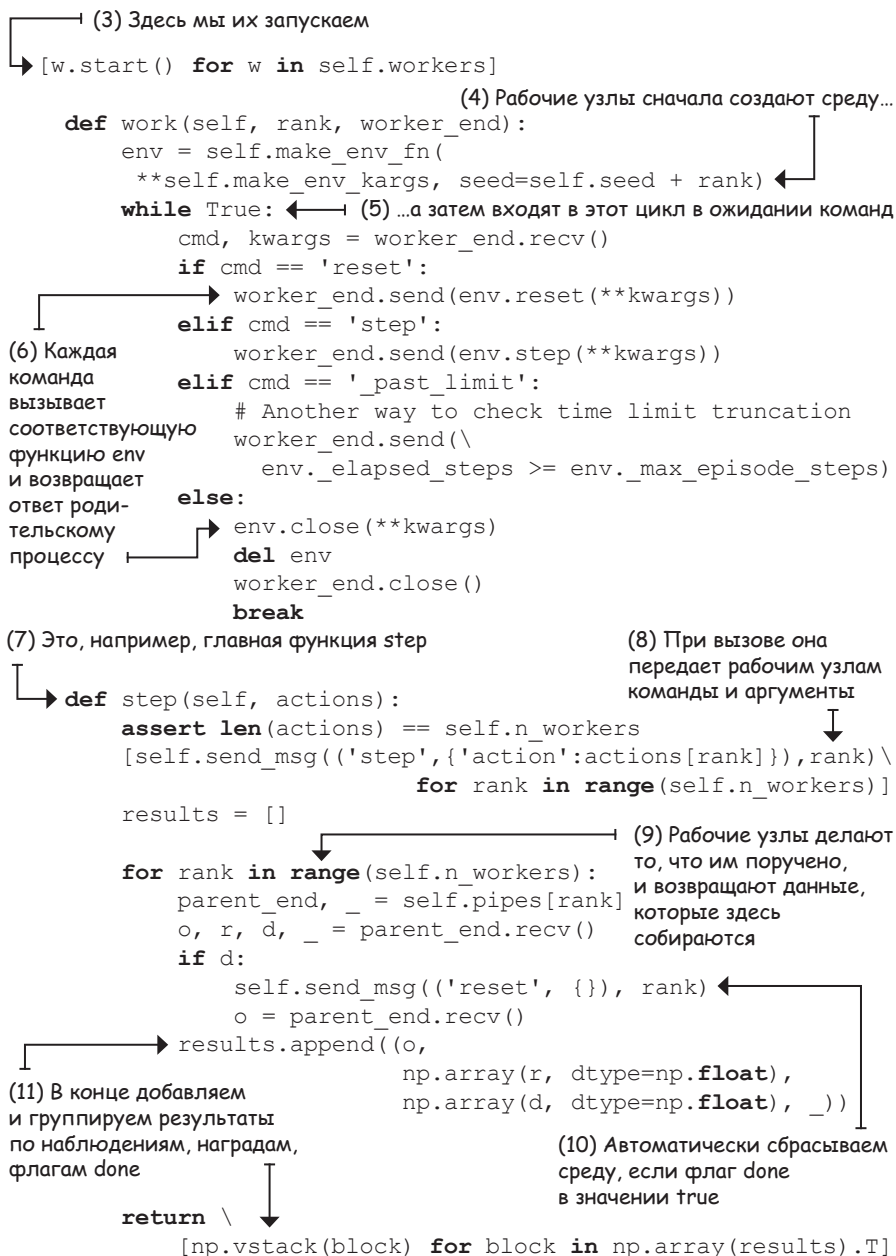
```
class MultiprocessEnv(object):
    def __init__(self, make_env_fn, make_env_kargs,
                 seed, n_workers):
        self.make_env_fn = make_env_fn
        self.make_env_kargs = make_env_kargs
        self.seed = seed
        self.n_workers = n_workers

        self.pipes = [
            mp.Pipe() for rank in range(self.n_workers)]
```

(1) Это класс многопроцессной среды, создающий сами рабочие узлы и каналы для взаимодействия с ними

```
        self.workers = [
            mp.Process(target=self.work,
                      args=(rank, self.pipes[rank][1])) \
                for rank in range(self.n_workers)]
```

(2) Здесь создаются рабочие узлы





Я знаю Python

Логика обучения A2C

```
class A2C():
```

```
    def train(self, make_envs_fn, make_env_fn,
              make_env_kargs, seed, gamma, max_minutes,
              max_episodes, goal_mean_100_reward):
```

(1) Так мы обучаемся в многопроцессной среде

```
        envs = self.make_envs_fn(make_env_fn,
                                  make_env_kargs, self.seed,
                                  self.n_workers)
        <...>
```

(2) Здесь создаются все векторизованные среды

(3) Здесь мы создаем единую модель «актер-критик», которая выводит политику и значения

```
        self.ac_model = self.ac_model_fn(nS, nA)
        self.ac_optimizer = self.ac_optimizer_fn(
            self.ac_model, self.ac_optimizer_lr)
```

(4) Смотрите, мы сбрасываем многопроцессную среду и получаем обратно стек состояний

```
        states = envs.reset()
```

```
        for step in count(start=1):
```

```
            states, is_terminals = \
                self.interaction_step(states, envs)
```

(5) Главное здесь то, что мы теперь работаем со стеками

```
            if is_terminals.sum() or \
                step - n_steps_start == self.max_n_steps:

                past_limits_enforced = envs._past_limit()
                failure = np.logical_and(is_terminals,
                                         np.logical_not(past_limits_enforced))
```

```
            next_values = self.ac_model.evaluate_state(
                states).detach().numpy() * (1 - failure)
```

(6) Но, по сути, все остается так же

```
            self.rewards.append(next_values)
            self.values.append(torch.Tensor(next_values))
            self.optimize_model()
            self.logpas, self.entropies = [], []
            self.rewards, self.values = [], []
            n_steps_start = step
```



Я знаю Python

Логика оптимизации модели в A2C

```

class A2C():
    def optimize_model(self):
        T = len(self.rewards)
        discounts = np.logspace(0, T, num=T, base=self.gamma,
                                endpoint=False)

        returns = np.array(
            [[np.sum(discounts[:T-t] * rewards[t:, w])
              for t in range(T)] \
             for w in range(self.n_workers)])

        np_values = values.data.numpy()
        tau_discounts = np.logspace(0, T-1, num=T-1,
                                     base=self.gamma*self.tau, endpoint=False)
        advs = rewards[:-1] + self.gamma * np_values[1:] \
              - np_values[:-1]

        gaes = np.array(
            [[np.sum(tau_discounts[:T-1-t] * advs[t:, w]) \
              for t in range(T-1)]
             for w in range(self.n_workers)])
        discounted_gaes = discounts[:-1] * gaes

        value_error = returns - values
        value_loss = value_error.pow(2).mul(0.5).mean()
        policy_loss = -(discounted_gaes.detach() * \
                        logpas).mean()

        entropy_loss = -entropies.mean()

        loss = self.policy_loss_weight * policy_loss + \
              self.value_loss_weight * value_loss + \
              self.entropy_loss_weight * entropy_loss

        self.ac_optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(
            self.ac_model.parameters(),
            self.ac_model_max_grad_norm)
        self.ac_optimizer.step()

```

(1) Так мы оптимизируем модель в A2C

(2) Обратите внимание на то, что мы теперь имеем дело с матрицами векторов, содержащих временные шаги каждого рабочего узла

(3) Некоторые операции работают точно так же, что довольно неожиданно

(4) А для некоторых нужно просто добавить цикл, чтобы охватить все узлы

(5) Так мы формируем единую функцию потерь

(6) В конце оптимизируем единую нейросеть



В деталях

Тестирование всех методов градиента политик в среде CartPole-v1

Чтобы показать алгоритмы градиента политик и упростить их сравнение с ценностно ориентированными методами из прошлых глав, я экспериментирую с теми же конфигурациями, что и раньше. А именно, со следующими.

REINFORCE:

- использует сеть политик с узлами 4-128-64-2, оптимизацию Адама и темп обучения 0,0007;
- в конце каждого эпизода обучается на выгоде Монте-Карло. Без направляющей.

VPG (REINFORCE с направляющей Монте-Карло):

- та же сеть политик, что и в REINFORCE, только теперь мы внедряем в функцию потерь параметр энтропии с весом 0,001 и ограничиваем градиент значением 1;
- теперь функция ценности используется как направляющая, а не как «критик». Это значит, что выгода MC используется без бэггинга, а функция ценности только уменьшает ее масштаб. Функция ценности формируется с использованием сети 4-256-128-1, оптимизатора RMSprop и темпа обучения 0,001. Градиент не ограничивается, хотя это можно было бы сделать.

A3C:

- сети политик и оценок обучаются точно так же;
- но теперь максимум раз в 50 шагов (или при попадании в конечное состояние) к выгоде применяется бэггинг. Это метод «актер-критик»;
- мы используем восемь рабочих узлов, каждый с копиями сетей, и выполняем обновления в стиле Hogwild!.

GAE:

- точно такие же гиперпараметры, как и в остальных алгоритмах;
- разница в том, что GAE добавляет гиперпараметр τ для дисконтирования преимуществ. Здесь τ равен 0,95. Обратите внимание: стиль агента предусматривает ту же n -шаговую логику бэггинга, из-за чего эта реализация GAE может оказаться не совсем чистой. Обычно одновременно обрабатываются наборы полных эпизодов. Но все же этот метод работает довольно хорошо.

A2C:

- а вот в A2C гиперпараметры отличаются. Прежде всего, у нас единая сеть: 4-256-128-3 (2 и 1). Обучение происходит с оптимизацией Адама, темпом 0,002 и градиентом, ограниченным до 1;
- политика, функция ценности и энтропия имеют вес 1,0, 0,6 и 0,001 соответственно;
- мы используем десятишаговый бэггинг, восемь рабочих узлов и τ , равный 0,95.

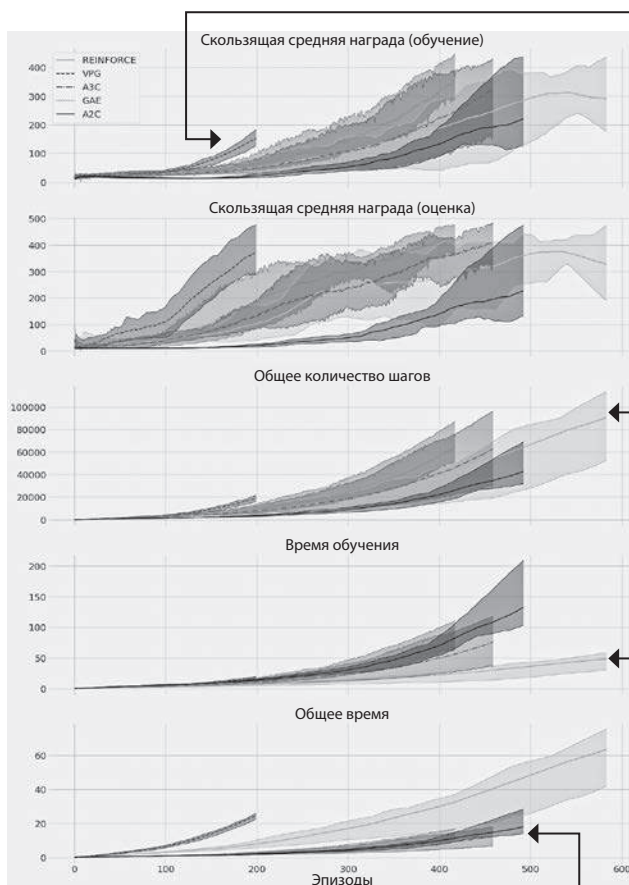
Эти алгоритмы не настраивались независимо друг от друга. Уверен, мы можем получить результаты и лучше.



Посчитаем

Методы градиента политик и «актер-критик» в среде CartPole-v1

(1) Чтобы вам было легче сравнивать стратегические и ценностно ориентированные методы, я тестировал все алгоритмы в среде «перевернутый маятник»



(2) Обратите внимание, что выборки VPG эффективней, чем у более сложных методов вроде A3C и A2C. Это объясняется тем, что последние два метода используют много рабочих узлов, из-за чего в самом начале для малейшего продвижения вперед нужно много данных

(3) Алгоритм REINFORCE сам по себе слишком неэффективный, что делает его непрактичным

(4) Но с точки зрения времени обучения REINFORCE явно потребляет меньше ресурсов. Еще обратите внимание на то, что алгоритмам с рабочими узлами нужна большая вычислительная мощность

(5) Любопытно, что с точки зрения общего времени параллельные методы оказываются невероятно быстрыми и проходят среду CartPole-v1 (500-шаговую версию) в среднем примерно за 10 секунд! Впечатляюще!

Подведем итоги

В этой главе мы исследовали методы градиента политик и «актер-критик». Вначале я рассказал, почему эти методы заслуживают внимания. Вы узнали, что настоящая цель методов RL — это формирование политики. Благодаря преследованию этой цели мы можем использовать стохастические политики, способные показывать лучшую производительность в частично наблюдаемых средах по сравнению с ценностно ориентированными методами. Вы узнали, что, хотя обычно мы изучаем стохастические политики, ничто не мешает нейросети сформировать детерминированную.

Знакомство с четырьмя новыми алгоритмами градиента политик началось с изучения REINFORCE и его простого подхода к улучшению политик. В качестве оценки (рейтинга) степени улучшения политики он позволяет использовать как полную, так и ожидаемую выгоду.

Затем мы обсудили базовый градиент политик, известный как REINFORCE с направляющей. В этом алгоритме функция ценности формируется с использованием выгоды Монте-Карло как цели. После этого функция играет роль направляющей, а не «критика». В VPG вместо бэггинга применяется ожидаемая награда (как в REINFORCE) и вычитается сформированная функция ценности, что позволяет снизить дисперсию градиента. Иначе говоря, в качестве рейтинга политики берется функция преимущества.

Вы познакомились с алгоритмом АЗС. В нем бэггинг выполняется как для формирования функции ценности, так и для оценки политики: мы используем n -шаговую выгоду для улучшения моделей. Помимо этого, мы применяем несколько обучаемых «актеров», каждый из которых выкатывает политику и оценивает выгоду, после чего модели политики и ценности обновляются в стиле Hogwild!. Другими словами, рабочие узлы обновляют неблокируемые модели.

Далее вы познакомились с алгоритмом GAE, который прогнозирует преимущества так же, как $TD(\lambda)$ прогнозирует λ -выгоду. GAE использует экспоненциально взвешенное сочетание всех n -шаговых преимуществ для получения более стабильного прогноза, который можно легко настраивать за счет более активного применения бэггинга (что приводит к смещению) или реальной выгоды (что повышает дисперсию).

В конце вы узнали, что, избавившись от асинхронности, присущей АЗС, можно получить сравнимые результаты без необходимости реализовывать собственные оптимизаторы. Так вы познакомились с алгоритмом А2С.

На этом этапе вы:

- знаете, в чем разница между ценностно ориентированными и стратегическими алгоритмами, методами градиента политик и «актер-критик»;
- можете самостоятельно реализовать базовые методы градиента политик и «актер-критик»;
- умеете настраивать алгоритмы градиента политик и «актер-критик» для прохождения разных сред.



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

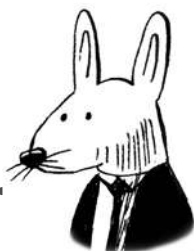
- **#gdr1_ch11_tf01.** Ранее я упоминал вымышленную среду «туманное озеро». Но вымышленная она лишь потому, что вы ее еще не реализовали. Попробуйте реализовать среды «туманное озеро» и «туманное замерзшее озеро». Убедитесь, что наблюдения, передаваемые агенту, отличаются от реального внутреннего состояния среды. Например, если агент находится в ячейке 3, внутреннее состояние остается неизвестным — он лишь знает, что попал в туманную ячейку. Тогда все туманные ячейки должны генерировать одно и то же наблюдение, чтобы агент не знал, где находится. После этого протестируйте агенты DRL, способные формировать только детерминированные политики (как в прошлых главах) и способные обучаться стохастическим (как в этой). Еще важно выполнять одинарное кодирование наблюдений, которые передаются нейросети. Создайте пакет Python с этой средой и блокнот Jupyter с интересными тестами и результатами.
- **#gdr1_ch11_tf02.** В этой главе мы все еще используем для тестов среду CartPole-v1, но, как вы знаете, сменить ее несложно. Для начала протестируйте те же агенты в похожих средах вроде LunarLander-v2 или MountainCar-v0. Похожими их делает то, что наблюдения в них малоразмерные и непрерывные, а действия — малоразмерные и дискретные. Затем поэкспериментируйте в других средах, с многомерными или непрерывными наблюдениями или действиями.
- **#gdr1_ch11_tf03.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>. #gdr1_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы познакомитесь с усовершенствованными методами глубокого обучения с подкреплением, которые по сей день остаются передовыми алгоритмическими достижениями в области DRL.
- ✓ Узнаете, как выполнять разные задачи глубокого обучения с подкреплением, от задач с непрерывными до задач с многомерными пространствами действий.
- ✓ Реализуете продвинутые методы «актер-критик» с нуля и приблизитесь к пониманию более сложных концепций, связанных с сильным искусственным интеллектом.

Критика может быть неприятной, но она необходима. Она выполняет ту же функцию, что и боль в человеческом теле, — обращает внимание на нездоровое положение дел.

*Уинстон Черчилль (Winston Churchill),
британский политик, армейский офицер,
писатель и премьер-министр Великобритании*

Ранее вы познакомились с другим, более прямым подходом к решению задач глубокого обучения с подкреплением. Вначале мы рассмотрели методы градиента политик, где агенты формировали политики, аппроксимируя их напрямую. В чистых методах градиента политик функции ценности не выступают промежуточными элементами — на самом деле они вообще не используются. Вместо этого мы сразу формируем стохастические политики.

Вскоре мы начали замечать, что функции ценности все еще могут играть важную роль, улучшая методы градиента политик. Так вы познакомились с методами «актер-критик», где агент формирует как политику, так и функцию ценности. Этот подход позволяет пользоваться преимуществами одной разновидности аппроксимации функций, избегая при этом недостатков другой. Например, в некоторых средах формирование политик может быть более простым, чем формирование достаточно точных функций ценности, так как связи в пространстве действий могут быть сильнее, чем связи между значениями. Но, несмотря на то что на получение точных значений состояния может уйти больше усилий, с помощью грубой аппроксимации можно снизить дисперсию цели градиента политик. Как вы уже знаете, формирование функции ценности и использование ее как направляющей для вычисления преимуществ может сильно снизить дисперсию целей, с помощью которых обновляется градиент политик. Более того, сокращение дисперсии обычно приводит к ускорению обучения.

Но в предыдущей главе функция ценности в основном выступала «критиком» для обновления стохастической политики. Для ее формирования использовались разные цели, а рабочие процессы распараллеливались разными способами. Но для создания политики на основе полученной функции ценности применялись примерно одинаковые подходы, и результат был примерно одинаковым, так как политика стохастическая. Ранее мы лишь слегка затронули вопрос использования сформированных политик и V-функций. В этой главе мы подробнее поговорим о методах «актер-критик» и займемся обучением наших агентов в четырех разных средах с повышенной сложностью: «маятник», «прыгун», «гепард» и «лунный посадочный модуль». Как вы вскоре увидите, у большинства этих сред непрерывные пространства действий, с которыми мы сталкиваемся впервые. Чтобы справиться с ними, мы изучим уникальные модели формирования политик.

Для прохождения этих сред мы сначала обсудим методы, формирующие детерминированные политики, которые при столкновении с определенным состоянием всегда возвращают одно и то же (предположительно оптимальное) действие. Мы исследуем ряд улучшений, благодаря которым алгоритмы градиента детерминированных политик до сих пор одни из передовых подходов к выполнению задач DRL. Далее речь пойдет о методах «актер-критик», использующих энтропию не в функции потерь, а в уравнении функции ценности, максимизирующих выгоду вместе с долгосрочной энтропией политики. В конце главы будут представлены алгоритмы с более стабильным этапом улучшения. Стабильность в них достигается за счет того, что политику можно обновлять лишь частично. Благодаря таким небольшим изменениям методы градиента политик демонстрируют устойчивые и монотонные улучшения производительности, что позволяет достичь высочайших результатов в нескольких тестовых средах DRL.

DDPG: аппроксимация детерминированной политики

Здесь мы исследуем *глубокий градиент по детерминированным политикам* (deep deterministic policy gradient, DDPG). Его можно считать приблизительной версией DQN или, еще лучше, DQN для непрерывных пространств действий. DDPG использует много методик из DQN: буфер воспроизведения для нестратегического формирования функций ценности состояний и целевые сети для стабилизации обучения. Но вместе с тем DDPG формирует политики, которые аппроксимируют оптимальные действия. Поэтому DDPG — метод градиента по детерминированным политикам, способный работать только с непрерывными пространствами действий.

Приемы в DDPG из DQN

Для начала важно сказать, что у DDPG та же архитектура, что и у DQN. Процесс обучения тоже похож: агент накапливает опыт динамически и сохраняет его в буфер воспроизведения. На каждом шаге он извлекает из буфера мини-набор, обычно с помощью равномерно случайной выборки. После на основе этого мини-набора агент вычисляет цель TD с бэггингом и формирует Q-функцию.

Основная разница между DQN и DDPG в том, что DQN использует целевую Q-функцию для получения жадного действия с помощью argmax , тогда как в DDPG применяется целевая функция детерминированной политики, которая обучается аппроксимации этого действия. Вместо использования argmax Q-функции следующего состояния, как в DQN, мы напрямую аппроксимируем лучшее действие в следующем состоянии с помощью функции политики. После в обоих методах мы используем это действие вместе с Q-функцией для получения максимального значения.



Я знаю математику

Цели функций ценности в DQN и DDPG

(1) Как вы помните, это функция потерь DQN для Q-функции. Она довольно простая

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(\mathcal{D})} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta_i) \right)^2 \right]$$

(2) Мы равномерно случайно извлекаем мини-набор из буфера $\mathcal{D}$

(3) Затем вычисляем цель TD с помощью награды и дисконтированного максимального значения следующего состояния согласно целевой сети

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(\mathcal{D})} \left[\left(r + \gamma Q(s', \text{argmax}_{a'} Q(s', a'; \theta^-); \theta^-) - Q(s, a; \theta_i) \right)^2 \right]$$

(4) Вы, наверное, помните и эту версию того же уравнения. Мы подставили argmax вместо $\max$

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(\mathcal{D})} \left[\left(r + \gamma Q(s', \mu(s'; \phi^-); \theta^-) - Q(s, a; \theta_i) \right)^2 \right]$$

(5) В DDPG мы извлекаем мини-набор так же, как и в DQN

(6) Но вместо argmax , согласно Q, мы формируем политику, μ

(7) μ формирует детерминированное жадное действие в заданном состоянии. Обратите внимание на то, что ϕ — это целевая сеть (—)



Я знаю Python

Сеть Q-функции в DDPG

```

class FCQV(nn.Module):
    def __init__(self,
                  input_dim,
                  output_dim,
                  hidden_dims=(32,32),
                  activation_fc=F.relu):
        super(FCQV, self).__init__()
        self.activation_fc = activation_fc

        self.input_layer = nn.Linear(input_dim, hidden_dims[0])
        self.hidden_layers = nn.ModuleList()
        for i in range(len(hidden_dims)-1):
            in_dim = hidden_dims[i]

            if i == 0:
                in_dim += output_dim

            hidden_layer = nn.Linear(in_dim, hidden_dims[i+1])
            self.hidden_layers.append(hidden_layer)

        self.output_layer = nn.Linear(hidden_dims[-1], 1)

    def forward(self, state, action):
        x, u = self._format(state, action)
        x = self.activation_fc(self.input_layer(x))

        for i, hidden_layer in enumerate(self.hidden_layers):
            if i == 0:
                x = torch.cat((x, u), dim=1)

            x = self.activation_fc(hidden_layer(x))

        return self.output_layer(x)

```

(1) Это сеть Q-функции в DDPG

(2) Здесь мы как обычно начинаем описывать архитектуру

(3) А это наше первое исключение. Мы увеличиваем размер первого скрытого слоя на размер вывода

(4) Обратите внимание, что вывод сети состоит из одного узла — значения пары «состояние — действие»

(5) Прямой проход начинается предсказуемо

(6) Действие и состояния связываются прямо в первом скрытом слое

(7) Затем все продолжается как раньше

(8) В конце возвращаем вывод

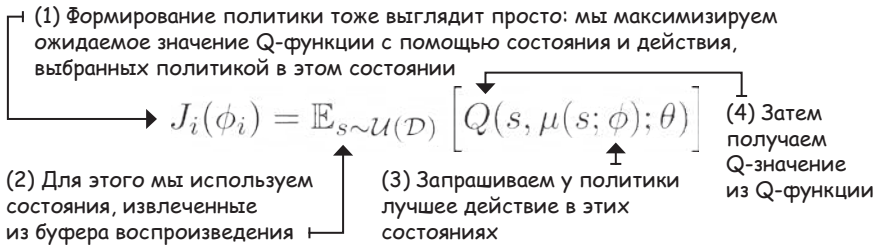
Формирование детерминированной политики

Теперь, чтобы этот алгоритм заработал, нам остается только добавить сеть политик. Нам нужна сеть, способная выбрать оптимальное действие в заданном состоянии. Она должна быть дифференцируемой для действий. Поэтому, чтобы обучение методом градиента политик было эффективным, действия должны быть непрерывными. Наша цель проста: использовать ожидаемое Q -значение, применяя сеть стратегий, μ . То есть агент должен попробовать найти действие, максимизирующее это значение. Заметьте, что на практике мы используем методы минимизации, поэтому минимизируем цель, обратную этой.



Я знаю математику

Цель детерминированной политики в DDPG



Заметьте, что вместо целевых сетей здесь используются динамические. Причем это относится как к политике (этап выбора действий), так и к функции ценности (этап ценности действий). Кроме того, учитывая, что нам нужно извлечь мини-набор состояний для формирования функции ценности, мы можем использовать их же для обучения сети политик.

0001 Немного истории

Появление алгоритма DDPG

Алгоритм DDPG был впервые представлен в научной работе *Continuous control with deep reinforcement learning*, опубликованной в 2015 году. Ее автор, Тимоти Лилликрап (Timothy Lillicrap) (и др.), работал научным сотрудником в Google DeepMind. Начиная с 2016 года он занимает должность штатного научного сотрудника и преподает в Университетском колледже Лондона.

В ходе своих исследований в DeepMind Тим опубликовал еще несколько научных работ, посвященных таким темам, как алгоритм A3C, AlphaGo, AlphaZero, Q-Prop, Starcraft II и др. Любопытный факт: до работы в области глубокого обучения с подкреплением Тим занимался не компьютерными науками, а когнитивистикой и системной психофизиологией.



Я знаю Python

Сеть детерминированной политики в DDPG

```
class FCDP(nn.Module):
    def __init__(self,
                  input_dim,
                  action_bounds,
                  hidden_dims=(32,32),
                  activation_fc=F.relu,
                  out_activation_fc=F.tanh):
        super(FCDP, self).__init__()

        self.activation_fc = activation_fc
        self.out_activation_fc = out_activation_fc
        self.env_min, self.env_max = action_bounds

        self.input_layer = nn.Linear(input_dim, hidden_dims[0])
        self.hidden_layers = nn.ModuleList()
        for i in range(len(hidden_dims)-1):
            hidden_layer = nn.Linear(hidden_dims[i],
                                     hidden_dims[i+1])
            self.hidden_layers.append(hidden_layer)

        self.output_layer = nn.Linear(hidden_dims[-1],
                                     len(self.env_max))

    def forward(self, state):
        x = self._format(state)
        x = self.activation_fc(self.input_layer(x))
        for hidden_layer in self.hidden_layers:
            x = self.activation_fc(hidden_layer(x))
        x = self.output_layer(x)

        x = self.out_activation_fc(x)

        return self.rescale_fn(x)
```

(1) Это сеть, которая используется в DDPG для формирования полносвязной детерминированной политики

(2) Обратите внимание, что в выходном слое поменялась функция активации. Мы используем метод `tanh`, чтобы уместить вывод в диапазон $(-1, 1)$

(3) Чтобы иметь возможность приводить вывод сети $(-1, 1)$ к ожидаемому диапазону, нам нужно получать минимальные и максимальные значения действий

(4) Знакомая архитектура: состояния на входе, действия на выходе

(5) Прямой проход тоже предскажем

(6) Ввод

(7) Скрытые слои

(8) Вывод

(9) Заметьте, что вывод активируется с помощью выходной функции активации

(10) Важно отметить, что действие в пределах от -1 до 1 приводится к диапазону, подходящему для конкретной среды. Функция здесь не показана, но ее детальное рассмотрение можно найти в блокноте Jupyter



Я знаю Python

Этап оптимизации модели в DDPG

```
def optimize_model(self, experiences):
    states, actions, rewards, \
        next_states, is_terminals = experiences
    batch_size = len(is_terminals)

    (2) На основе этого набора мы вычисляем цели с помощью спрогнозированного
    максимального значения следующего состояния для действий, выбранных
    нашей политикой, и с учетом значений от Q-функции
    argmax_a_q_sp = self.target_policy_model(next_states)
    max_a_q_sp = self.target_value_model(next_states,
                                         argmax_a_q_sp)
    target_q_sa = rewards + self.gamma * max_a_q_sp * \
        (1 - is_terminals)

    (3) Получаем прогнозы, вычисляем погрешность и потери. Обратите внимание на то, где используются
    целевая и динамическая сети
    q_sa = self.online_value_model(states, actions)
    td_error = q_sa - target_q_sa.detach()
    value_loss = td_error.pow(2).mul(0.5).mean()

    self.value_optimizer.zero_grad()
    value_loss.backward()
    torch.nn.utils.clip_grad_norm_(
        self.online_value_model.parameters(),
        self.value_max_grad_norm)
    self.value_optimizer.step()

    (4) Этап оптимизации тот же, что и в других сетях

    (5) Получаем действия в соответствии с прогнозом динамической политики
    для состояний, попавших в мини-набор. На основе этих действий
    прогнозируем значения с помощью динамической сети значений
    argmax_a_q_s = self.online_policy_model(states)
    max_a_q_s = self.online_value_model(states,
                                         argmax_a_q_s)
    policy_loss = -max_a_q_s.mean()

    (6) После этого получаем потери политики

    self.policy_optimizer.zero_grad()
    policy_loss.backward()
    torch.nn.utils.clip_grad_norm_(
        self.online_policy_model.parameters(),
        self.policy_max_grad_norm)
    self.policy_optimizer.step()

    (7) Обнуляем оптимизатор, выполняем обратный проход по функции потерь,
    ограничиваем градиенты и переводим оптимизатор на следующий шаг
```

Исследование с помощью детерминированных политик

В DDPG мы формируем детерминированные жадные политики, которые в идеальных условиях принимают состояние и возвращают оптимальное для него действие. Но когда политика не обучена, возвращаемые действия будут неточны, хоть и детерминированны. Как уже упоминалось, агентам приходится искать баланс между использованием имеющихся и сбором (исследованием) новых знаний. Но, повторюсь, так как агент DDPG формирует детерминированную политику, сбор данных будет нестратегическим. Представьте, что ваш агент упрямится и всегда выбирает одно и то же действие. Чтобы этого избежать, мы должны исследовать среду без использования политики. Поэтому в DDPG мы внедряем гауссовский шум в выбранные политикой действия.

Вы узнали об исследовании в нескольких агентах DRL. В NFQ, DQN и т. д. мы используем стратегии исследования, основанные на Q-значениях: получаем значения действий в заданном состоянии с помощью изученной Q-функции и исследуем на основе этих значений. В REINFORCE, VPG и т. д. мы применяем стохастические политики, поэтому этап исследования получается стратегическим: поскольку политика стохастическая (у нее есть элемент случайности), она сама занимается исследованием. В DDPG агент исследует среду, примешивая к действиям внешний шум, используя нестратегические методы исследования.



Я знаю Python

Этап исследования в градиентах детерминированных политик

```
class NormalNoiseDecayStrategy():
    def select_action(self, model, state, max_exploration=False):

        if max_exploration:
            noise_scale = self.high
        else:
            noise_scale = self.noise_ratio * self.high

        with torch.no_grad():
            greedy_action = model(state).cpu().detach().data
            greedy_action = greedy_action.numpy().squeeze()

            noise = np.random.normal(loc=0,
                                    scale=noise_scale,
                                    size=len(self.high))

            noisy_action = greedy_action + noise
            action = np.clip(noisy_action, self.low, self.high)

            self.noise_ratio = self._noise_ratio_update()

        return action
```

(1) С помощью этой функции политика выбирает действия

(2) Чтобы максимизировать исследование, мы установили шкалу шума на максимальное действие

(3) В противном случае уровень шума понижается

(4) Берем жадное действие прямо из сети

(5) Затем, используя уровень шума и нулевое среднее значение, находим гауссовский шум для этого действия

(6) Добавляем шум к действию и приводим его к нужному диапазону

(7) Обновляем график изменения шума. Он может быть непрерывным, линейным, экспоненциальным и т. д.

(8) В конце возвращаем действие

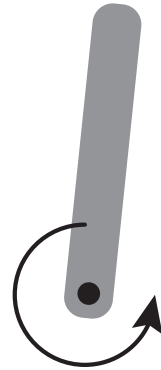


Конкретный пример

Среда «маятник»

Среда Pendulum-v0 — это перевернутый маятник, который агент должен поддерживать в вертикальном положении, прикладывая как можно меньше усилий. Пространство состояний — это вектор из трех переменных: $\cos(\theta)$, $\sin(\theta)$ и θ (точка), обозначающих косинус/синус угла наклона стержня и угловую скорость.

Пространство действий представлено единой непрерывной переменной в диапазоне от -2 до 2 , обозначающей направление, в котором прикладывается сила относительно шарнира. Шарнир — это черная точка внизу стержня. Действие — это сила, направленная либо по часовой стрелке, либо против нее.



Функция вознаграждения — это уравнение, основанное на значении угла, скорости и силе. Цель: поддерживать стержень в строго вертикальном положении без каких-либо усилий. На таком идеальном временном шаге агент получает самую высокую возможную награду — нулевую. На самом затратном шаге (с самой низкой наградой) агент может получить вознаграждение около -16 . Уравнение имеет следующий вид: $-(\theta^2 + 0.1 \times \dot{\theta}^2 + 0.001 \times a^2)$.

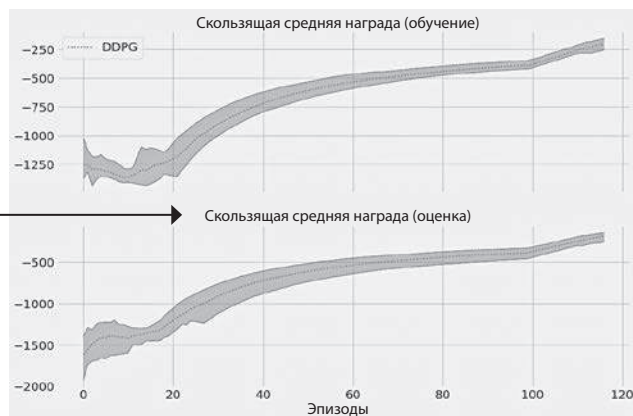
Это непрерывная задача, поэтому конечного состояния нет. Вместо него используется время ожидания 200 шагов, которое служит той же цели. Эта среда считается непроходимой, так как у нее нет целевой выгоды, но разумным пороговым значением будет -150 .



Посчитаем

DDPG в среде «маятник»

(1) Справа показано обучение агента DDPG до достижения награды -150 на этапе оценки. Здесь используется пять исходных значений, но график заканчивается вместе с эпизодами первого из них. Как видите, алгоритм работает хорошо и быстро. «Маятник» — простая среда



TD3: лучшие оптимизации для DDPG

Вот уже несколько лет DDPG — один из передовых методов глубокого обучения с подкреплением для задач управления. Но с момента его внедрения было предложено много улучшений, связанных с производительностью. В этом разделе мы обсудим набор оптимизаций, которые вместе составляют новый алгоритм *двойной DDPG* (TD3). У TD3 есть три основных отличия от DDPG. Во-первых, в нем применяется метод двойного обучения, как в двойном Q-обучении и DDQN, но с уникальной архитектурой, где есть сеть-близнец. Во-вторых, TD3 добавляет шум, и не только к передающимся среде действиям, но и к целевым, что делает сеть политики более устойчивой к ошибкам аппроксимации. И в-третьих, он откладывает обновление сети политик, ее целевой сети и целевой сети-близнеца, благодаря чему последняя обновляется чаще.

Двойное обучение в DDPG

В TD3 применяется особый вид сети Q-функции с двумя отдельными потоками, которые возвращают два разных прогноза искомой пары «состояние — действие». В большинстве случаев эти потоки независимы друг от друга, поэтому их можно считать раздельными сетями. Но в среде, ориентированной на изображения, лучше использовать общие слои признаков. Так сеть CNN могла бы извлекать общие признаки, что могло бы ускорить темп ее обучения. Но разделяемые слои обычно сложнее обучать, поэтому, прежде чем выбирать тот или иной подход, нужно поэкспериментировать.

В реализации ниже используется два совершенно отдельных потока, и единственное, что у них есть общего, — это оптимизатор. Как видно в функции потерь сети-близнеца, мы суммируем потери, полученные в обеих сетях, и затем используем их для оптимизации каждой из них.



Я знаю математику

Цель-близнец в TD3

(1) Потери сети-близнеца — это сумма среднеквадратических погрешностей каждого потока

$$J_i(\theta_i^a) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(\mathcal{D})} \left[(\text{TWLN}^{\text{target}} - Q(s, a; \theta_i^a))^2 \right]$$

$$J_i(\theta_i^b) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(\mathcal{D})} \left[(\text{TWLN}^{\text{target}} - Q(s, a; \theta_i^b))^2 \right]$$

(2) Вычисляем цель, используя минимум для обоих потоков. Это не полная цель TD3. Чуть позже мы ее дополним

$$\text{TWLN}^{\text{target}} = r + \gamma \min_n Q(s', \mu(s'; \phi^-); \theta^{n,-})$$

(3) Обратите внимание, что целевые сети используются для сетей политик и значений



Я знаю Python

Q-сеть-близнец в TD3

```
class FCTQV(nn.Module):
    def __init__(self,
                  input_dim,
                  output_dim,
                  hidden_dims=(32, 32),
                  activation_fc=F.relu):
        super(FCTQV, self).__init__()
        self.activation_fc = activation_fc
```

(1) Это полносвязная сеть-близнец для Q-значений. С ее помощью TD3 аппроксимирует Q-значения, используя дублируемые потоки

```
        self.input_layer_a = nn.Linear(input_dim + output_dim,
                                         hidden_dims[0])
        self.input_layer_b = nn.Linear(input_dim + output_dim,
                                         hidden_dims[0])
```

(2) Обратите внимание, что у нас два входных слоя. Напомню, что это две отдельные сети

```
        self.hidden_layers_a = nn.ModuleList()
        self.hidden_layers_b = nn.ModuleList()
        for i in range(len(hidden_dims)-1):
            hid_a = nn.Linear(hidden_dims[i], hidden_dims[i+1])
            self.hidden_layers_a.append(hid_a)
            hid_b = nn.Linear(hidden_dims[i], hidden_dims[i+1])
            self.hidden_layers_b.append(hid_b)
```

(3) Создаем для каждого из потоков скрытые слои

```
        self.output_layer_a = nn.Linear(hidden_dims[-1], 1)
        self.output_layer_b = nn.Linear(hidden_dims[-1], 1)
```

(4) В конце у нас получается два выходных слоя, каждый из которых состоит из одного узла, представляющего Q-значение

```
    def forward(self, state, action):
        x, u = self._format(state, action)
```

(5) Выполняем прямой проход, приводя ввод к формату, на который рассчитывает сеть

```
        x = torch.cat((x, u), dim=1)
        xa = self.activation_fc(self.input_layer_a(x))
        xb = self.activation_fc(self.input_layer_b(x))
```

(6) Затем объединяем состояние и действие и пропускаем их через каждый поток

(7) Проходим по всем скрытым слоям и их функциям активации

```

    for hidden_layer_a, hidden_layer_b in zip(
        self.hidden_layers_a, self.hidden_layers_b):
        xa = self.activation_fc(hidden_layer_a(xa))
        xb = self.activation_fc(hidden_layer_b(xb))

```

(8) Наконец, проходим через выходные слои и возвращаем их прямой вывод

```

    xa = self.output_layer_a(xa)
    xb = self.output_layer_b(xb)
    return xa, xb

```

(9) Это прямое прохождение потока Qa. Так мы получаем значения при вычислении целей для обновления политики

```

def Qa(self, state, action):
    x, u = self._format(state, action)

```

(10) Прежде чем пропускать входные значения через поток, форматируем и объединяем их

```

    x = torch.cat((x, u), dim=1)
    xa = self.activation_fc(self.input_layer_a(x))

```

(11) Затем пропускаем ввод через скрытые слои a...

```

    for hidden_layer_a in self.hidden_layers_a:
        xa = self.activation_fc(hidden_layer_a(xa))

```

(12) ...до достижения входного слоя, будто мы с самого начала имели дело с одной сетью

```

    return self.output_layer_a(xa)

```

Сглаживание целей для обновления политики

Как вы помните, чтобы улучшить этап исследования в DDPG, мы внедряем гауссовский шум в действия, которые передаются среде. В TD3 эта идея получила развитие: мы добавляем шум к действиям, которые используются не только на этапе исследования, но и при вычислении целей.

Формирование политики с шумными (искаженными) целями — это своеобразная регуляризация, так как теперь сеть вынуждена обобщать похожие действия. До этого сеть политик могла сходиться на неправильных действиях, ошибочно оцененных Q-функцией на ранних стадиях обучения. Новый метод позволяет это предотвратить. Шум распространяет значение по более широкому диапазону действий, чем прежде.



Я знаю математику

Сглаживание целей

(1) Рассмотрим функцию `clamp`, которая фактически закрепляет значение x между минимумом l и максимумом h

$$\text{clamp}(x, l, h) = \max(\min(x, h), l)$$

$$a^{l, \text{smooth}} = \text{clamp}(\mu(s'; \phi^-) + \text{clamp}(\epsilon, \epsilon_l), \epsilon_h, a_l, a_h)$$

(2) В TD3 мы сглаживаем действие за счет добавления ограниченного гауссовского шума E . Сначала мы выбираем значение E и фиксируем его между минимальной и максимальной величинами, установленными заранее. Добавляем этот шум к действию и помещаем это действие в диапазон значений, допустимых в текущей среде. В итоге получается сглаженное действие, которое используется дальше

$$\text{TD3}^{\text{target}} = r + \gamma \min_n Q(s', a^{l, \text{smooth}}; \theta^{n, -})$$



Я знаю Python

Этап оптимизации модели в TD3

(1) Для оптимизации моделей в TD3 мы используем мини-набор с кортежами опыта

```
def optimize_model(self, experiences):
    states, actions, rewards, \
        next_states, is_terminals = experiences
    batch_size = len(is_terminals)
```

(2) Получаем минимальное и максимальное значения, доступные в этой среде

```
with torch.no_grad():
    env_min = self.target_policy_model.env_min
    env_max = self.target_policy_model.env_max
    a_ran = env_max - env_min
    a_noise = torch.randn_like(actions) * \
        self.policy_noise_ratio * a_ran
```

(3) Получаем шум и приводим его к диапазону действий

(4) Получаем минимум и максимум для шума

```
n_min = env_min * self.policy_noise_clip_ratio
n_max = env_max * self.policy_noise_clip_ratio
```

(5) Ограничиваем шум

```
a_noise = torch.max(
    torch.min(a_noise, n_max), n_min)
```


(14) Но потери те же

```
└───> policy_loss = -max_a_q_s.mean()
```

(15) Это стандартный этап оптимизации политики

```
└───> self.policy_optimizer.zero_grad()
      policy_loss.backward()
      torch.nn.utils.clip_grad_norm_(
          self.online_policy_model.parameters(),
          self.policy_max_grad_norm)
      self.policy_optimizer.step()
```

Откладывание обновлений

Последняя оптимизация, которая отличает TD3 от DDPG, — откладывание обновлений сети политик и целевых сетей, благодаря чему динамическая сеть Q -функции обновляется чаще других. Задержка остальных обновлений приносит пользу, так как динамическая Q -функция показывает резкий перепад значений в начале обучения. Замедление политики так, чтобы она начинала обновляться только после нескольких обновлений функции ценности, позволяет последней сойтись на более точных значениях, прежде чем мы начнем использовать ее для формирования политики. Сети политик и целевые сети нужно обновлять в два раза реже, чем динамическую сеть Q -функции.

Еще один интересный аспект обновления политик — то, что мы должны применить один из потоков динамической модели значений, чтобы получить спрогнозированное Q -значение для действия, генерируемого политикой. В TD3 мы используем один из двух потоков — причем всегда один и тот же.

0001 Немного истории

История появления агента TD3

Алгоритм TD3 был предложен Скоттом Фудзимото (Scott Fujimoto) и др. в работе *Addressing Function Approximation Error in Actor-Critic Methods*, опубликованной в 2018 году.

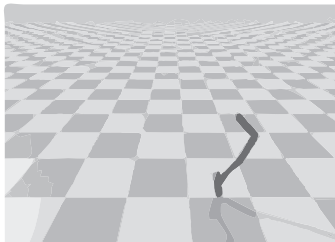
Скотт проходит аспирантуру в Макгиллском университете, работая над докторской диссертацией в области компьютерных наук под руководством профессоров Дэвида Мегера (David Meger) и Дойны Прекап (Doina Precup).



Конкретный пример

Среда «прыгун»

Среда «прыгун», которую мы здесь используем, — это открытая версия сред MuJoCo и Roboschool, основанных на системе Bullet Physics. MuJoCo — это физический движок с разными моделями и задачами, который широко применяется исследователями в области DRL, но требует лицензии. Если вы не студент, он может обойтись вам в копейечку. Roboschool — это проект компании OpenAI, который был призван создать открытые версии сред MuJoCo, но работа над ним была прекращена в пользу Bullet. Bullet Physics — это проект с открытым исходным кодом, представляющий многие из сред, которые можно найти в MuJoCo.



Среда HopperBulletEnv-v0 содержит вектор с 15 непрерывными переменными, из которых состоит пространство наблюдения. Они представляют разные суставы робота-прыгуна. В этой среде есть вектор с тремя непрерывными переменными в диапазоне от -1 до 1 , представляющими действия бедренного, коленного и голеностопного суставов. Обратите внимание, что каждое отдельно взятое действие содержит сразу три элемента. Агент должен сделать так, чтобы прыгун двигался вперед: это, как и минимальный расход энергии, поощряется функцией вознаграждения.



В деталях

Обучение агента TD3 в среде «прыгун»

В блокноте Jupyter к этой главе можно заметить, что агент обучается, пока его средняя награда за 100 эпизодов подряд не достигнет 1500. На самом деле рекомендуемое ограничение — 2500. Но, так как обучение проходит с пятью разными начальными значениями и каждый прогон занимает около часа, я решил сократить время на завершение блокнота за счет снижения порогового значения. Из анимированных изображений в блокноте видно, что даже с лимитом 1500 прыгун продвигается неплохо.

Имейте в виду: так как все реализации из этой книги выполняются одна за другой, на это уходит много времени. Оценивать производительность в каждом эпизоде не обязательно и в большинстве случаев не нужно. Для наших экспериментов это нормально, но, если вы хотите использовать этот код в проектах, советую избавиться от этой логики и оценивать производительность раз в 10–100 эпизодов.

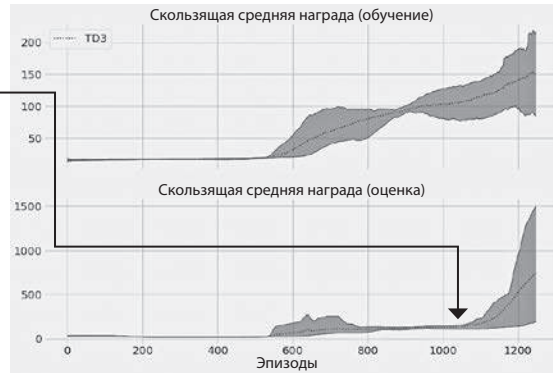
Обратите внимание и на детали реализации. Алгоритм TD3 из этой книги оптимизирует сети политик и значений по отдельности. Если вы захотите, например, использовать в обучении сети CNN, процесс свертывания лучше сделать общим и проводить оптимизацию сразу для всех сетей. Но опять же это потребовало бы дополнительной настройки.



Посчитаем

TD3 в среде «прыгун»

(1) TD3 работает хорошо, хотя эта среда довольно непростая. Как видите, после 1000 эпизодов производительность оценки немного повышается. В блокноте Jupyter можно найти анимированные изображения. Обратите внимание на то, как агент продвигается вперед. Наблюдать, как меняется производительность, всегда интересно



SAC: максимизация ожидаемой выгоды и энтропии

Прошлые два алгоритма, DDPG и TD3, нестратегические: они формируют детерминированные политики. Это значит, что метод использует опыт, сгенерированный одной политикой для оптимизации другой. В DDPG и TD3 используется буфер воспроизведения, содержащий опыт, сгенерированный несколькими ранними политиками. К тому же оптимизируется детерминированная политика (при каждом обращении возвращает одно и то же действие), поэтому оба эти метода исследуют среду нестратегически. В нашей реализации DDPG и TD3 внедряют гауссовский шум в векторы действий, которые передаются среде.

Для сравнения, агенты из предыдущей главы обучаются стратегически: они формируют стохастические политики, которые сами по себе вносят элемент случайности и занимаются исследованием среды. Для увеличения случайности в этих политиках мы добавили понятие энтропии в функцию потерь.

В этом разделе мы обсудим алгоритм «мягкий «актер-критик»» (soft actor-critic, SAC) — что-то среднее между этими двумя парадигмами. SAC — это нестратегический алгоритм, который похож на DDPG и TD3, но формирует не детерминированную, а стохастическую политику, как REINFORCE, A3C, GAE и A2C.

Добавление энтропии в уравнение Беллмана

Основная особенность SAC — то, что энтропия стохастической политики становится частью функции ценности, которую агент пытается максимизировать. Как вы увидите позднее, совместная максимизация ожидаемой совокупной награды и общей энтропии стимулирует более разнообразное поведение, но по-прежнему позволяет получить максимальную ожидаемую выгоду.



Я знаю математику

Агенту нужно максимизировать энтропию



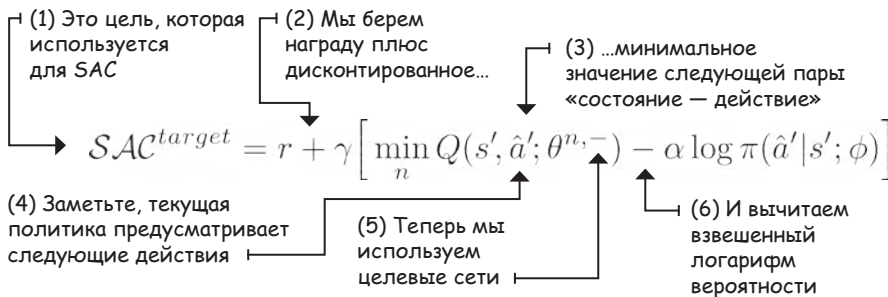
Формирование функции ценности действий

На практике SAC формирует функцию ценности так же, как и TD3: мы используем две сети, аппроксимирующие Q-функцию, и минимальный прогноз для большинства вычислений. Но есть и несколько различий. Во-первых, SAC оптимизирует каждую Q-функцию отдельно, что дает лучшие результаты. Мы возьмем это на вооружение. Во-вторых, мы добавляем энтропию к целевым значениям. И последнее: мы не выполняем сглаживание целевых действий напрямую, как в TD3. Остальное без изменений.



Я знаю математику

Цель функции ценности действий (агент обучается, вычисляя среднеквадратическую погрешность для этой цели)



Формирование политики

На этот раз для формирования стохастической политики мы применим «сжатую» гауссовскую, которая при прямом проходе возвращает среднее значение и стандартное отклонение. С помощью полученных результатов мы делаем выборку из гауссовского распределения, «сжимаем» значения с помощью функции гиперболического тангенса $\tanh$ и приводим их к диапазону, на который рассчитывает среда.

Для формирования политики используется прием *повторной параметризации* — перенос стохастичности из сети во входные данные. Так сеть получается детерминированной и легко обучаемой. Как вы увидите далее, этот прием просто и понятно реализован в PyTorch.



Я знаю математику

Цель политики (агент учится минимизировать цель, противоположную этой)



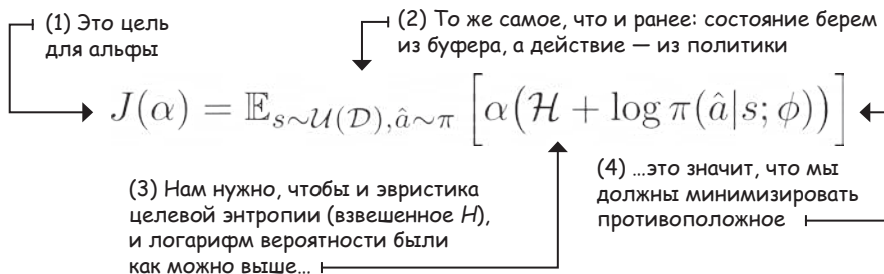
Автоматическая настройка коэффициента энтропии

Изюминка SAC в том, что альфа (коэффициент энтропии) может регулироваться автоматически. Этот алгоритм использует оптимизацию альфа-канала на основе градиента в направлении эвристической ожидаемой энтропии. Рекомендуемая целевая энтропия зависит от размерности пространства действий: от отрицательного произведения его вектора. С помощью этой энтропии мы можем автоматически оптимизировать альфу так, чтобы не нужно было изменять почти никаких гиперпараметров.



Я знаю математику

Функция цели для альфы (агент учится минимизировать цель, противоположную этой)



Я знаю Python

Гауссовская политика SAC

```
class FCGP(nn.Module):
    def __init__(self,
        <...>
        self.input_layer = nn.Linear(input_dim,
                                     hidden_dims[0])
        self.hidden_layers = nn.ModuleList()
        for i in range(len(hidden_dims)-1):
            hidden_layer = nn.Linear(hidden_dims[i],
                                     hidden_dims[i+1])
            self.hidden_layers.append(hidden_layer)
        self.output_layer_mean = nn.Linear(hidden_dims[-1],
                                           len(self.env_max))
        self.output_layer_log_std = nn.Linear(
            hidden_dims[-1],
            len(self.env_max))
```

(1) Это гауссовская политика, которую мы используем в SAC

(2) Все начинается так же, как и в других сетях, от ввода до скрытых слоев

(3) Но скрытые слои подключаются к двум потокам, один из которых возвращает среднее значение действия, а другой — логарифм стандартного отклонения

(4) Здесь вычисляется H , эвристика целевой энтропии

```
self.target_entropy = -np.prod(self.env_max.shape)
```

(5) Создаем переменную, присваиваем ей ноль и оптимизатор для регулирования `logalpha`

```
self.logalpha = torch.zeros(1,
                             requires_grad=True,
                             device=self.device)
self.alpha_optimizer = optim.Adam([self.logalpha],
                                   lr=entropy_lr)
```

(6) Функция `forward` выглядит знакомо

```
def forward(self, state):
    x = self._format(state)
    x = self.activation_fc(self.input_layer(x))
    for hidden_layer in self.hidden_layers:
        x = self.activation_fc(hidden_layer(x))
    x_mean = self.output_layer_mean(x)
    x_log_std = self.output_layer_log_std(x)
    x_log_std = torch.clamp(x_log_std,
                           self.log_std_min,
                           self.log_std_max)
    return x_mean, x_log_std
```

(7) Форматируем входные переменные и пропускаем их через всю сеть

(8) Ограничиваем логарифм std адекватным диапазоном: от -20 до 2

(10) При полном проходе получаем среднее значение и логарифм std

```
def full_pass(self, state, epsilon=1e-6):
    mean, log_std = self.forward(state)
```

(9) И возвращаем значения

(11) Получаем нормальное распределение с этими значениями

```
pi_s = Normal(mean, log_std.exp())
```

(12) Выборка r здесь выполняет прием с повторной параметризацией

```
pre_tanh_action = pi_s.rsample()
tanh_action = torch.tanh(pre_tanh_action)
```

(13) Сжимаем диапазон действия до $[-1, 1]$

```
action = self.rescale_fn(tanh_action)
```

(14) После этого приводим значения к диапазону, на который рассчитывает среда

```
log_prob = pi_s.log_prob(pre_tanh_action) - torch.log(
    (1 - tanh_action.pow(2)).clamp(0, 1) + epsilon)
```

(15) Меняем диапазоны для логарифма вероятности и среднего значения

```
log_prob = log_prob.sum(dim=1, keepdim=True)
return action, log_prob, self.rescale_fn(
    torch.tanh(mean))
```



Я знаю Python

Этап оптимизации в SAC

```
def optimize_model(self, experiences):
    states, actions, rewards, \
        next_states, is_terminals = experiences
    batch_size = len(is_terminals)

    current_actions, \
        logpi_s, _ = self.policy_model.full_pass(states)

    target_alpha = (logpi_s + \
        self.policy_model.target_entropy).detach()
    alpha_loss = -(self.policy_model.logalpha * \
        target_alpha).mean()

    self.policy_model.alpha_optimizer.zero_grad()
    alpha_loss.backward()
    self.policy_model.alpha_optimizer.step()

    alpha = self.policy_model.logalpha.exp()

    current_q_sa_a = self.online_value_model_a(
        states, current_actions)
    current_q_sa_b = self.online_value_model_b(
        states, current_actions)

    current_q_sa = torch.min(current_q_sa_a,
        current_q_sa_b)

    policy_loss = (alpha * logpi_s - current_q_sa).mean()

    ap, logpi_sp, _ = self.policy_model.full_pass(
        next_states)
```

(1) Это этап оптимизации в SAC

(2) Сначала извлекаем кортежи опыта из мини-набора

(3) Получаем текущие действия и логарифм вероятности состояния s

(4) Здесь вычисляются потери альфы, а здесь оптимизатор альфа переводится на следующий шаг

(5) Получаем текущее значение альфы

(6) Здесь мы получаем Q-значения, используя динамические модели, и $\hat{\alpha}$

(7) Затем используем минимальный прогноз Q-значения...

(8) ...и с его помощью вычисляем потери политики

(9) Чтобы вычислить потери Q-функции, получаем прогнозируемое следующее значение

(10) С помощью моделей целевых значений
вычисляем прогноз Q-значения для следующей
пары «состояние — действие»

```

└─> q_sap_a = self.target_value_model_a(next_states, ap)
    q_sap_b = self.target_value_model_b(next_states, ap)
└─> q_sap = torch.min(q_sap_a, q_sap_b) - \
                                             alpha * logpi_sp

```

(11) Получаем минимальный прогноз
Q-значения и коэффициент энтропии

```

└─> (12) Вычисляем цель, суммируя награду
      и дисконтированное минимальное значение
      следующего состояния с учетом энтропии
└─> target_q_sa = (rewards + self.gamma * \
                  q_sap * (1 - is_terminals)).detach()

```

(13) С помощью динамической модели получаем
спрогнозированные значения пары «состояние — действие»

```

└─> q_sa_a = self.online_value_model_a(states, actions)
    q_sa_b = self.online_value_model_b(states, actions)
└─> qa_loss = (q_sa_a - target_q_sa).pow(2).mul(0.5).mean()
    qb_loss = (q_sa_b - target_q_sa).pow(2).mul(0.5).mean()

```

(14) Вычисляем потери и оптимизируем каждую Q-функцию отдельно. Сначала a:

```

self.value_optimizer_a.zero_grad()
qa_loss.backward()
torch.nn.utils.clip_grad_norm_(
    self.online_value_model_a.parameters(),
    self.value_max_grad_norm)
self.value_optimizer_a.step()

```

(15) Затем b:

```

└─> self.value_optimizer_b.zero_grad()
    qb_loss.backward()
    torch.nn.utils.clip_grad_norm_(
        self.online_value_model_b.parameters(),
        self.value_max_grad_norm)
    self.value_optimizer_b.step()

```

(16) И в конце политики:

```

└─> self.policy_optimizer.zero_grad()
    policy_loss.backward()
    torch.nn.utils.clip_grad_norm_(
        self.policy_model.parameters(),
        self.policy_max_grad_norm)
    self.policy_optimizer.step()

```

0001 Немного истории

История появления агента SAC

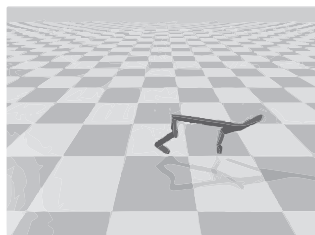
В 2018 году агент SAC предложил Туомас Хаарноя (Tuomas Haarnoja) в своей научной работе *Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor*. В момент публикации Туомас был аспирантом в Беркли и работал над своей докторской диссертацией под руководством профессоров Питера Аббееля (Pieter Abbeel) и Сергея Левина (Sergey Levine), а также был научным стажером в Google. В 2019 году Туомас стал научным сотрудником в Google DeepMind.



Конкретный пример

Среда «гепард»

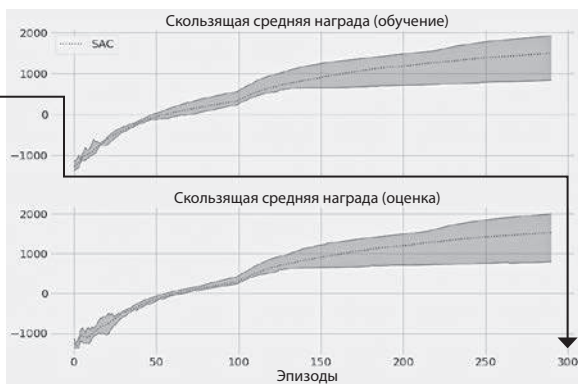
Пространство наблюдения в среде HalfCheetah-BulletEnv-v0 отображает суставы робота-гепарда в виде вектора с 26 непрерывными переменными. Действия представлены вектором с шестью непрерывными переменными от -1 до 1 . Задача агента — обеспечить движение гепарда вперед. Как и в случае с «прыгуном», этот процесс стимулируется функцией вознаграждения, которая поощряет минимальный расход энергии.



Посчитаем

SAC в среде «гепард»

(1) SAC проявляет себя в среде «гепард» довольно неплохо: всего за 300–600 эпизодов он осваивает управление роботом. Обратите внимание, что в этой среде рекомендуемая максимальная награда — 3000, но 2000 тоже дает хорошие результаты. Еще имейте в виду, что даже с заниженной наградой на обучение агента уходит несколько часов



РРО: ограничение этапа оптимизации

В этом разделе вы познакомитесь с алгоритмом «актер-критик» под названием *«проксимальная оптимизация политики»* (proximal policy optimization, РРО). По сути, он основан на той же архитектуре, что и алгоритм А2С: большую часть кода последнего можно будет использовать и в РРО. То есть мы можем взять сразу несколько параллельных сред, сгруппировать опыт по мини-наборам, получить прогнозы GAE с помощью «критика» и заняться обучением «актера» и «критика», как в А2С.

Важнейшее нововведение в РРО — это суррогатная целевая функция, позволяющая алгоритму политики выполнять несколько градиентных шагов в одном и том же мини-наборе выборок. Как вы уже знаете, А2С — стратегический метод, поэтому он не может повторно использовать выборки на этапах оптимизации. Как правило, стратегические методы должны отбрасывать образцы опыта сразу же при переводе оптимизатора на следующий шаг.

У РРО ограниченная функция цели, которая не дает политике меняться слишком сильно на этапе оптимизации. Такое осторожное улучшение позволяет не только избежать провалов в производительности, вызванных высокой дисперсией стратегических методов градиента политик, но и повторно использовать мини-наборы выборок с несколькими этапами оптимизации для каждого из них. Благодаря возможности повторного использования выборок РРО эффективнее применяет накопленный опыт в сравнении с другими стратегическими методами.

Использование той же архитектуры «актер-критик», что и в А2С

РРО можно считать улучшенной версией А2С. Я имею в виду, что у всех алгоритмов из этой главы, включая DDPG, TD3 и SAC, есть кое-что общее, но РРО, к примеру, нельзя назвать продолжением SAC. Алгоритм TD3 основан на DDPG, а SAC и TD3 разрабатывались параллельно, но вскоре после выхода первой версии автор SAC опубликовал вторую, содержащую несколько возможностей TD3. Итак, SAC не прямой потомок алгоритма TD3, но он обладает некоторыми его характеристиками. Если же говорить о РРО, то это улучшенная версия А2С, и мы частично воспользуемся кодом последнего: будем собирать в параллельных средах мини-наборы данных и применять GAE для получения целей политики.

0001 Немного истории

Появление агента РРО

Агент РРО был впервые представлен в 2018 году в научной работе Джона Шульмана и др. *Proximal Policy Optimization Algorithms*. Джон — научный сотрудник, соучредитель и один из лидеров команды обучения с подкреплением в OpenAI. Он защитил свою докторскую диссертацию в области компьютерных наук в Беркли под руководством Питера Аббея.

Группировка выборок

Одна из особенностей РРО, которой нет у A2C, — наличие возможности использования выборок. Мы можем собирать крупные наборы траекторий, как в NFQ, и «подгонять» модель под данные, оптимизируя ее снова и снова. Но есть способ лучше: создать буфер воспроизведения, из которого на каждом этапе оптимизации будет извлекаться самый крупный мини-набор. Это наделяет все мини-наборы стохастичностью, ведь выборки в них иногда отличаются, хотя в дальнейшем весь опыт, скорее всего, будет использован повторно.



Я знаю Python

Буфер воспроизведения эпизодов

```
class EpisodeBuffer():
    def fill(self, envs, policy_model, value_model):
        states = envs.reset()
```

(1) Это содержимое класса EpisodeBuffer

(2) Переменные для группировки информации о рабочих узлах

```
        we_shape = (n_workers, self.max_episode_steps)
        worker_rewards = np.zeros(shape=we_shape,
                                   dtype=np.float32)
        worker_exploratory = np.zeros(shape=we_shape,
                                       dtype=np.bool)
        worker_steps = np.zeros(shape=(n_workers),
                                 dtype=np.uint16)
        worker_seconds = np.array([time.time(),] * n_workers,
                                   dtype=np.float64)
```

(3) Входим в главный цикл, чтобы заполнить буфер

```
        buffer_full = False
        while not buffer_full and \
            len(self.episode_steps[self.episode_steps>0]) < \
            self.max_episodes/2:
```

(4) Сначала получаем текущие действия, логарифмы вероятностей и статистику

```
            with torch.no_grad():
                actions, logpas, \
                    are_exploratory = policy_model.np_pass(states)
                values = value_model(states)
```

(5) Передаем действия среде и получаем кортежи опыта

```
            next_states, rewards, terminals, \
                infos = envs.step(actions)
            self.states_mem[self.current_ep_idx,
                            worker_steps] = states
```

(6) Сохраняем кортежи в буфере воспроизведения

```

self.actions_mem[self.current_ep_idx,
                  worker_steps] = actions
self.logpas_mem[self.current_ep_idx,
                 worker_steps] = logpas

```

(7) Создаем по одной переменной для каждого рабочего узла
(рабочие узлы находятся внутри среды)

```

worker_exploratory[np.arange(self.n_workers),
                   worker_steps] = are_exploratory
worker_rewards[np.arange(self.n_workers),
               worker_steps] = rewards

```

(8) Вручную прерываем эпизоды, которым нужно слишком много шагов

```

for w_idx in range(self.n_workers):
    if worker_steps[w_idx] + 1 == self.max_episode_steps:
        terminals[w_idx] = 1
        infos[w_idx]['TimeLimit.truncated'] = True

```

(9) Проверяем, конечное ли состояние, и обрабатываем его заранее

```

if terminals.sum():
    idx_terminals = np.flatnonzero(terminals)
    next_values = np.zeros(shape=(n_workers))
    truncated = self._truncated_fn(infos)
    if truncated.sum():
        idx_truncated = np.flatnonzero(truncated)
        with torch.no_grad():
            next_values[idx_truncated] = value_model(\
                next_states[idx_truncated]).cpu().numpy()

```

(10) Проводим
бэггинг, пока
конечное
состояние
усечено

```

states = next_states
worker_steps += 1

```

(11) Обновляем переменную states
и увеличиваем счетчик шагов

```

if terminals.sum():
    new_states = envs.reset(ranks=idx_terminals)
    states[idx_terminals] = new_states

```

(12) При наличии конечных состояний
обрабатываем рабочие узлы

```

for w_idx in range(self.n_workers):
    if w_idx not in idx_terminals:
        continue

```

(13) По очереди
обрабатываем каждый
конечный рабочий узел

```
e_idx = self.current_ep_idxxs[w_idx]
T = worker_steps[w_idx]
self.episode_steps[e_idx] = T
```

(14) Собираем статистику, которая будет выводиться и анализироваться позже

```
self.episode_reward[e_idx] = worker_rewards[w_idx, :T].sum()
self.episode_exploration[e_idx] = worker_exploratory[\
                                                    w_idx, :T].mean()
self.episode_seconds[e_idx] = time.time() - \
                                                    worker_seconds[w_idx]
```

(15) Добавляем значение бэггинга к вектору наград. Вычисляем спрогнозированную выгоду

```
ep_rewards = np.concatenate((worker_rewards[w_idx, :T],
                               [next_values[w_idx]]))
ep_discounts = self.discounts[:T+1]
ep_returns = np.array(\
    [np.sum(ep_discounts[:T+1-t] * ep_rewards[t:]) \
     for t in range(T)])

self.returns_mem[e_idx, :T] = ep_returns
```

(16) Получаем спрогнозированные значения и снова добавляем значение бэггинга к вектору

```
ep_states = self.states_mem[e_idx, :T]
with torch.no_grad():
    ep_values = torch.cat((value_model(ep_states),
                           torch.tensor(\
                               [next_values[w_idx]]),
                           device=value_model.device,
                           dtype=torch.float32)))
```

(17) Здесь вычисляются и сохраняются в буфер обобщенные значения преимуществ

```
np_ep_values = ep_values.view(-1).cpu().numpy()
ep_tau_discounts = self.tau_discounts[:T]
deltas = ep_rewards[:-1] + self.gamma * \
    np_ep_values[1:] - np_ep_values[:-1]
gaes = np.array(\
    [np.sum(self.tau_discounts[:T-t] * deltas[t:]) \
     for t in range(T)])

self.gaes_mem[e_idx, :T] = gaes
```

```
worker_exploratory[w_idx, :] = 0
worker_rewards[w_idx, :] = 0
worker_steps[w_idx] = 0
worker_seconds[w_idx] = time.time()
```

(18) Начинаем сбрасывать все переменные worker_*, чтобы перейти к следующему шагу

(19) Узнаем, какой эпизод идет следующим,
и прерываем цикл, если эпизодов слишком много

```

    new_ep_id = max(self.current_ep_idx) + 1
    if new_ep_id >= self.max_episodes:
        buffer_full = True
    break

```

(20) Если буфер не заполнен, передаем
рабочему узлу идентификатор нового эпизода

```

    self.current_ep_idx[w_idx] = new_ep_id

```

(21) Если мы дошли до этих строк, значит эпизод полный,
поэтому мы обрабатываем память для выборки

```

    ep_idx = self.episode_steps > 0
    ep_t = self.episode_steps[ep_idx]

```

(22) Мы инициализируем сразу весь буфер, поэтому
нам нужно убрать из памяти все, что не является числом
(в пределах эпизода и временных шагов)

```

    self.states_mem = [row[:ep_t[i]] for i, \
        row in enumerate(self.states_mem[ep_idx])]
    self.states_mem = np.concatenate(self.states_mem)
    self.actions_mem = [row[:ep_t[i]] for i, \
        row in enumerate(self.actions_mem[ep_idx])]
    self.actions_mem = np.concatenate(self.actions_mem)
    self.returns_mem = [row[:ep_t[i]] for i, \
        row in enumerate(self.returns_mem[ep_idx])]
    self.returns_mem = torch.tensor(np.concatenate(\
        self.returns_mem), device=value_model.device)
    self.gaes_mem = [row[:ep_t[i]] for i, \
        row in enumerate(self.gaes_mem[ep_idx])]
    self.gaes_mem = torch.tensor(np.concatenate(\
        self.gaes_mem), device=value_model.device)
    self.logpas_mem = [row[:ep_t[i]] for i, \
        row in enumerate(self.logpas_mem[ep_idx])]
    self.logpas_mem = torch.tensor(np.concatenate(\
        self.logpas_mem), device=value_model.device)

```

(23) В конце извлекаем статистику для дальнейшего вывода

```

    ep_r = self.episode_reward[ep_idx]
    ep_x = self.episode_exploration[ep_idx]
    ep_s = self.episode_seconds[ep_idx]

```

```

    return ep_t, ep_r, ep_x, ep_s

```

(24) И возвращаем ее

Ограничение обновлений политики

Главная проблема обычного градиента политик в том, что даже небольшое изменение в пространстве параметров может привести к большой разнице в производительности. Именно из-за этой непропорциональности в методах градиента политик нужно применять низкие темпы обучения. Но даже это не всегда спасает от высокой дисперсии. Суть ограниченного РРО такова: ограничить цель так, чтобы на каждом временном шаге политика отдалялась не слишком сильно. Можете думать об этом как о тренере, который не дает своей команде слишком бурно реагировать на результаты. Команда применила новую тактику и выиграла? Отлично, но не нужно преувеличивать. Одна удачная игра не перевешивает результаты всего сезона. Продолжаем совершенствоваться постепенно.



Я знаю математику

Ограниченная цель политики

(1) Получение цели политики начинается с извлечения из буфера состояний, действий и значений GAE

(2) Далее вычисляется соотношение между новой и старой политиками, которое используется для получения цели

$$J(\phi, \phi^-) = \mathbb{E}_{(s, a, A^{GAE}) \sim \mathcal{U}(\mathcal{D}(\phi^-))} \left\{ \min \left[\frac{\pi(a|s; \phi)}{\pi(a|s; \phi^-)} A^{GAE}, \text{clamp} \left(\frac{\pi(a|s; \phi)}{\pi(a|s; \phi^-)}, 1 - \epsilon, 1 + \epsilon \right) A^{GAE} \right] \right\}$$

(3) Нам нужен минимум от взвешенного значения GAE...

(4) ...и той же цели, только с ограниченным соотношением

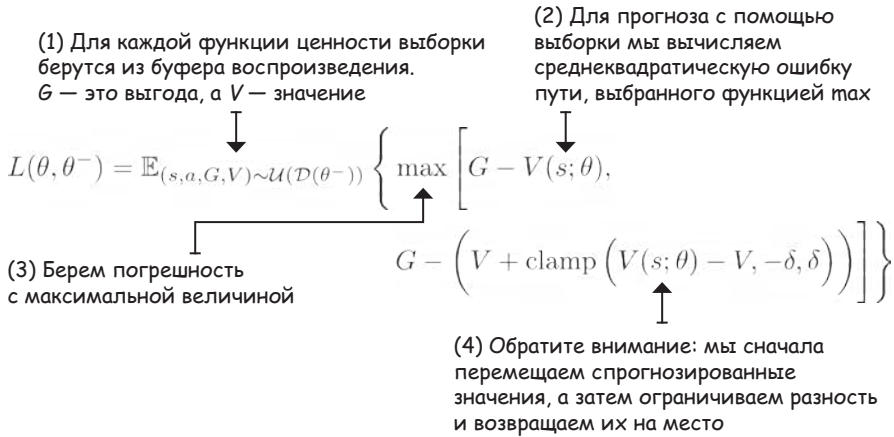
Ограничение обновлений функции ценности

Тот же принцип ограничения можно применить и к функции ценности: мы позволяем изменениям в пространстве параметров влиять на Q-значения только до определенной степени. Как можно догадаться, этот подход сглаживает дисперсию тех элементов, которые нас интересуют, независимо от того, насколько резко меняются параметры. Изменения в пространстве параметров могут быть любыми, а вот колебания производительности и значений хотелось бы сгладить.



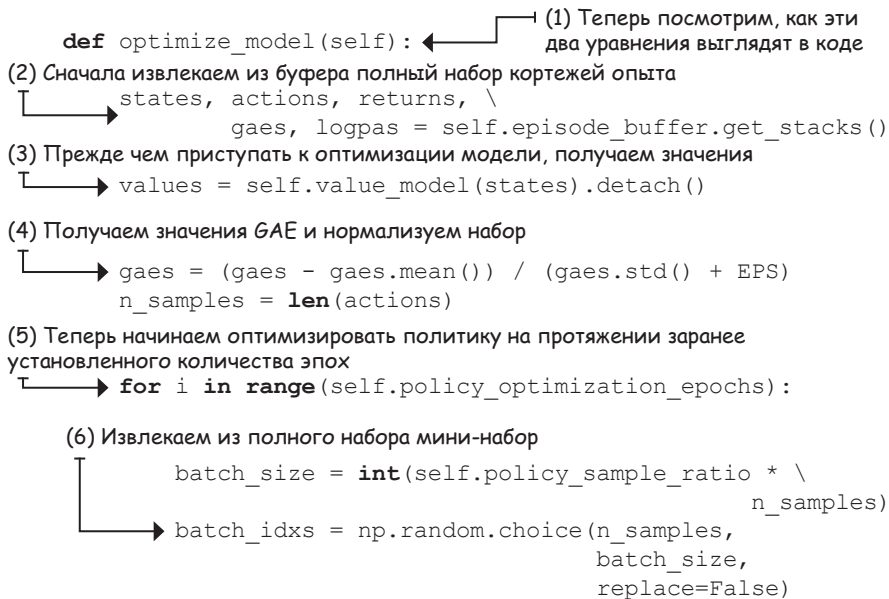
Я знаю математику

Потери функции ценности



Я знаю Python

Этап оптимизации PPO



(7) Для извлечения используются случайно выбранные индексы

```
states_batch = states[batch_idxs]
actions_batch = actions[batch_idxs]
gaes_batch = gaes[batch_idxs]
logpas_batch = logpas[batch_idxs]
```

(8) Для получения прогнозов используем динамическую модель

```
logpas_pred, entropies_pred = \
    self.policy_model.get_predictions( \
        states_batch, actions_batch)
```

(9) Здесь на основе логарифма вероятностей вычисляется их соотношение

```
ratios = (logpas_pred - logpas_batch).exp()
pi_obj = gaes_batch * ratios
```

(10) Вычисляем обычную и ограниченную цели

```
pi_obj_clipped = gaes_batch * ratios.clamp( \
    1.0 - self.policy_clip_range,
    1.0 + self.policy_clip_range)
pi_obj_clipped = gaes_batch * ratios.clamp( \
    1.0 - self.policy_clip_range,
    1.0 + self.policy_clip_range)
```

(11) С помощью отрицательной минимальной цели вычисляем потери

```
policy_loss = -torch.min(pi_obj,
    pi_obj_clipped).mean()
```

(12) Рассчитываем потери энтропии и взвешиваем их

```
entropy_loss = -entropies_pred.mean() * \
    self.entropy_loss_weight
```

(13) Обнуляем этап оптимизации и начинаем обучение

```
self.policy_optimizer.zero_grad()
(policy_loss + entropy_loss).backward()
torch.nn.utils.clip_grad_norm_( \
    self.policy_model.parameters(),
    self.policy_model_max_grad_norm)
self.policy_optimizer.step()
```

(14) После запуска оптимизатора мы используем этот полезный прием, чтобы повторная оптимизация выполнялась, только если новая политика находится в пределах исходной

```
with torch.no_grad():
    logpas_pred_all, _ = \
        self.policy_model.get_predictions(states,
            actions)
```

(15) Здесь вычисляется kl-расхождение двух политик

```
kl = (logpas - logpas_pred_all).mean()
```

(16) Если расхождение соответствует условию, выходим из цикла обучения

```
if kl.item() > self.policy_stopping_kl:
    break
```


(17) Начинаем вносить похожие обновления в функцию ценности

```

↓   for i in range(self.value_optimization_epochs):
    batch_size = int(self.value_sample_ratio * \
n_samples)

```

(18) Извлекаем мини-набор из полного набора, как мы это делали для политики

```

└───> batch_idx = np.random.choice(n_samples,
    batch_size,
    replace=False)

    states_batch = states[batch_idx]
    returns_batch = returns[batch_idx]
    values_batch = values[batch_idx]

```

(19) Получаем спрогнозированные моделью значения и вычисляем стандартные потери

```

└───> values_pred = self.value_model(states_batch)
    v_loss = (values_pred - returns_batch).pow(2)

```

(20) Вычисляем ограниченные спрогнозированные значения

```

└───> values_pred_clipped = values_batch + \
    (values_pred - values_batch).clamp( \
    -self.value_clip_range,
    self.value_clip_range)

```

(21) А затем — ограниченные потери

```

└───> v_loss_clipped = (values_pred_clipped - \
    returns_batch).pow(2)

```

(22) Используем среднеквадратическую ошибку максимума между стандартными и ограниченными потерями

```

└───> value_loss = torch.max(\
    v_loss, v_loss_clipped).mul(0.5).mean()

```

(23) Обнуляем оптимизатор, передаем обратно потери, ограничиваем градиент и переходим на следующий шаг

```

└───> self.value_optimizer.zero_grad()
    value_loss.backward()
    torch.nn.utils.clip_grad_norm_( \
    self.value_model.parameters(),
    self.value_model_max_grad_norm)

    self.value_optimizer.step()

```

(24) Мы можем выполнить что-то вроде ранней остановки, но для функции ценности

```

└───> with torch.no_grad():
    values_pred_all = self.value_model(states)

```

(25) По сути, мы проверяем среднеквадратическую ошибку значений, спрогнозированных новой и старой политиками

```

└───> mse = (values - values_pred_all).pow(2)
    mse = mse.mul(0.5).mean()
    if mse.item() > self.value_stopping_mse:
        break

```



Конкретный пример

Среда «посадочный лунный модуль» (LunarLander)

В отличие от других сред из этой главы LunarLander содержит дискретное пространство действий. Такие алгоритмы, как DDPG и TD3, работают только в средах с непрерывными действиями в виде единственной переменной, как в случае с «маятником», или вектора, как в случае с «прыгуном» и «гепардом». Агенты вроде DQN работают только в средах с дискретными пространствами действий наподобие «перевернутого маятника». У методов «актер-критик», таких как A2C и PPO, есть большой плюс: они позволяют использовать модели стохастических политик, совместимые почти с любым пространством действий.



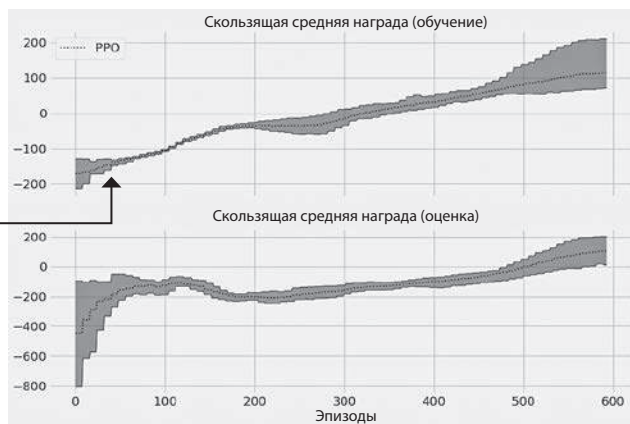
В этой среде на каждом шаге агенту нужно выбирать одно из четырех возможных действий: 0 (ничего не делать), 1 (включить левый двигатель), 2 (включить главный двигатель) или 3 (включить правый двигатель). Пространство наблюдений изображено в виде вектора с восьмью элементами, отражающими координаты, углы наклона, скорость движения и индикатор контакта с поверхностью. Функция вознаграждения зависит от расстояния до посадочной площадки и расхода топлива. Лимит вознаграждения для прохождения этой среды — 200, а максимальное число временных шагов — 1000.



Посчитаем

PPO в среде LunarLander

(1) Это простая среда, поэтому такой эффективный алгоритм, как PPO, проходит ее примерно за 10 минут. Вы можете заметить, что кривые не непрерывные. Дело в том, что в этом алгоритме этап оценки выполняется после обработки всех выборок в каждом эпизоде



Подведем итоги

В этой главе вы познакомились с продвинутыми методами «актер-критик» и DRL. Сначала вы узнали о DDPG, в которых формируется детерминированная политика, в связи с чем они используют нестратегические подходы к исследованию и подходы вне уравнения обновлений. Например, с помощью DDPG и TD3 мы вводим гауссовский шум в процесс выбора действий, позволяя детерминированным политикам стать исследовательскими.

Вы узнали, что TD3 привносит в DDPG три ключевых улучшения. Во-первых, TD3 использует методику двойного обучения, как в DDQN: она позволяет проводить перекрестную проверку прогнозов, получаемых из функции ценности, с помощью Q-сети-близнеца. Во-вторых, TD3 внедряет гауссовский шум не только в действия, передаваемые среде, но и в целевые, чтобы гарантировать, что политика не изучает действия на основе фиктивных оценок Q-значения. В-третьих, TD3 задерживает обновление сети политик, чтобы сеть значений успела получить более точные прогнозы, которые затем будут использоваться для изменения политики.

Далее мы рассмотрели метод максимизации энтропии SAC. Он заключается в максимизации совместной цели функции ценности и энтропии политики, что фактически выражается в получении самой высокой награды при самой разнообразной политике. Агент SAC, как DDPG и TD3, обучается нестратегически. Это значит, что все агенты могут повторно применять полученные выборки для улучшения политик. Но, в отличие от DDPG и TD3, SAC формирует стохастическую политику, которая может включать в себя этап исследования, который может быть стохастическим.

В конце вы изучили алгоритм PPO, более прямого потомка A2C. Это стратегический метод обучения, применяющий стратегический подход к исследованию. Но благодаря ограниченной цели, которая заставляет PPO осторожно оптимизировать формируемую политику, этот алгоритм может повторно использовать прошлый опыт на этапах улучшения.

Далее мы рассмотрим несколько направлений исследований, смежных с DRL, которые развивают область *сильного искусственного интеллекта* (artificial general intelligence, AGI). AGI — это попытка понять человеческий интеллект путем его воссоздания. Физик Ричард Фейнман (Richard Feynman) однажды сказал: «Я не могу понять то, что не могу создать». Было бы здорово раскрыть тайну человеческого интеллекта!

На этом этапе вы:

- понимаете принцип работы усовершенствованных алгоритмов «актер-критик» и связанных с ними приемов;
- можете реализовать продвинутые методы глубокого обучения с подкреплением и вносить в них свои улучшения, которыми можно будет поделиться;
- умеете применять передовые алгоритмы глубокого обучения с подкреплением в разных средах (надеюсь, даже в своих).



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

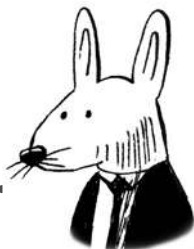
- **#gdr1_ch12_tf01.** Выберите среду с непрерывным пространством действий и протестируйте в ней все алгоритмы, с которыми познакомились в этой главе. Обратите внимание, что для этого вам придется внести изменения в PPO. Но сравнить работу этих агентов будет довольно полезно для вас.
- **#gdr1_ch12_tf02.** Добавьте алгоритм PPO в блокнот Jupyter из предыдущей главы. Протестируйте его в похожих средах и сравните результаты. Заметьте, что прежде, чем выполнять какие-либо обновления, эта реализация PPO буферизует некоторые выборки. Не забудьте отредактировать код или гиперпараметры, чтобы сделать сравнение честным. Как себя проявляет PPO? Обязательно проверьте этот алгоритм в среде сложнее «перевернутого маятника»!
- **#gdr1_ch12_tf03.** Есть и другие методы DRL для максимизации энтропии, такие как Q-обучение. Выберите из полного списка таких алгоритмов один и реализуйте его самостоятельно. Протестируйте свою реализацию и сравните ее с другими агентами, включая SAC. Напишите о плюсах и минусах таких методов.
- **#gdr1_ch12_tf04.** Протестируйте все алгоритмы из этой главы в среде с многомерным пространством наблюдений и непрерывным пространством действий. Например, в среде «автогонки» (<https://gym.openai.com/envs/CarRacing-v0/>). Подойдет любая похожая. Измените код агента так, чтобы он мог обучаться в таких средах.
- **#gdr1_ch12_tf05.** В каждой главе есть заключительный всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>.
#gdr1_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.



В этой главе

- ✓ Вы вспомните все изученные алгоритмы и познакомитесь с теми методами глубокого обучения с подкреплением, которые мы не исследовали подробно.
- ✓ Узнаете о продвинутых методиках глубокого обучения с подкреплением, сочетание которых повышает интеллект агента.
- ✓ Получите от меня совет о том, как следовать за своей мечтой и внести свой вклад в интереснейшие области ИИ и глубокого обучения с подкреплением.

Наша конечная цель в том, чтобы программы учились на своем опыте так же эффективно, как это делают люди.

*Джон Маккарти, основатель области
ИИ, автор языка Lisp*

В книге мы исследовали множество алгоритмов принятия решений и агентов обучения с подкреплением, от методов планирования из главы 3 до передовых агентов DRL из предыдущей главы. Основная цель этой книги — продемонстрировать все тонкости этих методик. Но я не могу охватить в одной книге всю область DRL, поэтому хочу дать вам несколько советов о том, куда двигаться дальше.

В этой главе основное внимание уделено нескольким вопросам. В первом разделе мы проведем краткий обзор всего материала из книги. Я хочу, чтобы вы свежим взглядом охватили приобретенные вами знания — это поможет вам определиться с направлением дальнейшего движения. Мы пройдемся по нескольким любопытным видам агентов, которые не удалось рассмотреть ранее. Но, несмотря на существование других алгоритмов, знайте, что в этой книге вы изучили самые базовые из них.

Я познакомлю вас с более глубокими направлениями исследований в области DRL, которые когда-то могут привести к возникновению сильного ИИ (artificial general intelligence, AGI). Я понимаю, что AGI у всех на слуху, и многие искажают значение этого термина. Это захватывающая и неоднозначная тема, которую нередко используют для привлечения внимания. Не давайте ввести себя в заблуждение и не отвлекайтесь. Сосредоточьте свое внимание на действительно важных вещах, которые находятся прямо перед вами, и двигайтесь навстречу своим целям, какими бы они ни были.

Я верю, что людям действительно по силам создать AGI, так как попытки это сделать не прекращаются по сей день. Изучение интеллекта и автоматизация заданий — это то, к чему мы стремились и над чем работали столетиями, и так будет всегда. Мы пытаемся раскрыть тайны интеллекта с помощью философии и самопознания, ищем ответы, анализируя свой разум. Мне кажется, что большинство исследователей в области ИИ сами своего рода философы: они применяют знания, приобретенные в ходе работы над обучением с подкреплением, для самосовершенствования и наоборот.

Людям нравится автоматизация: она стала возможной благодаря нашему интеллекту. Мы и дальше будем пытаться автоматизировать разные аспекты жизни. Кто-то может сказать, что AGI — это первый шаг к появлению человекоподобных роботов, которые захватят мир, но нам все еще не удастся научить агента проходить все игры Atari лучше человека. Отдельно взятый обученный агент не может играть во все игры, но мы можем отдельно выработать единый общий алгоритм. К обсуждению AGI нужно подходить осторожно.

В конце этой книги я расскажу вам, куда двигаться дальше. Меня часто спрашивают о применении DRL к нестандартным задачам, самодельным средам. Это моя специализация, поэтому я могу дать пару советов о том, как это лучше делать. Я немного расскажу о построении карьеры в этой области и оставлю прощальные пожелания. Осталась всего одна глава — давайте поднажмем!

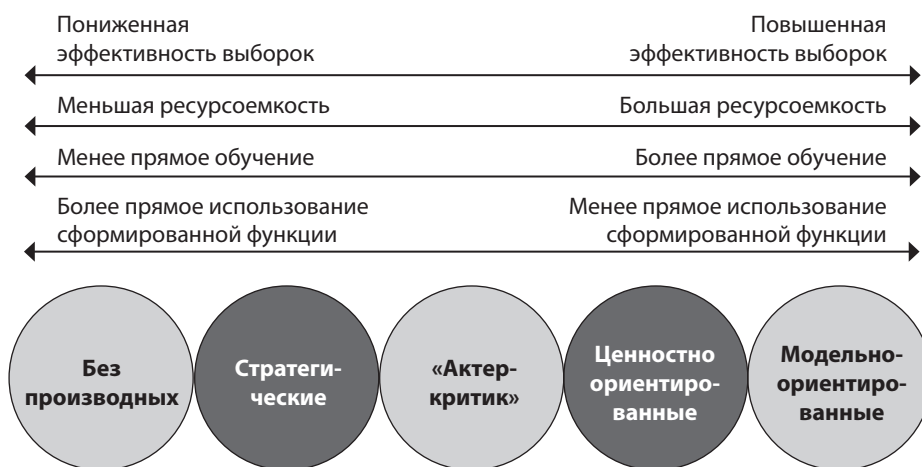
Важные темы, которые были рассмотрены, и те, которые остались без внимания

Эта книга охватывает большинство базовых аспектов DRL, от моделей MDP и их внутреннего устройства до продвинутых алгоритмов «актер-критик» и их обучения в сложных средах. Глубокое обучение с подкреплением — это активно развивающаяся область исследований, где ежемесячно публикуются новые алгоритмы. Темпы развития очень высоки, поэтому в одной книге невозможно как следует все объяснить.

К счастью, большинство концепций, которые мы не затронули, углубленные, и в большинстве случаев без них можно обойтись. Но это не значит, что они неважны: я настоятельно рекомендую вам продолжить изучение DRL. Можете рассчитывать на мою помощь, меня легко найти. Но пока что из всего, что не попало в эту книгу, я хотел бы остановиться на двух ключевых направлениях: методах модельно-ориентированного глубокого обучения с подкреплением и оптимизации без производных.

В этом разделе мы проведем краткий обзор алгоритмов и методов, с которыми вы познакомились в этой книге, и затронем два упомянутых выше подхода, о которых важно знать.

Сравнение разных алгоритмических подходов к глубокому обучению с подкреплением

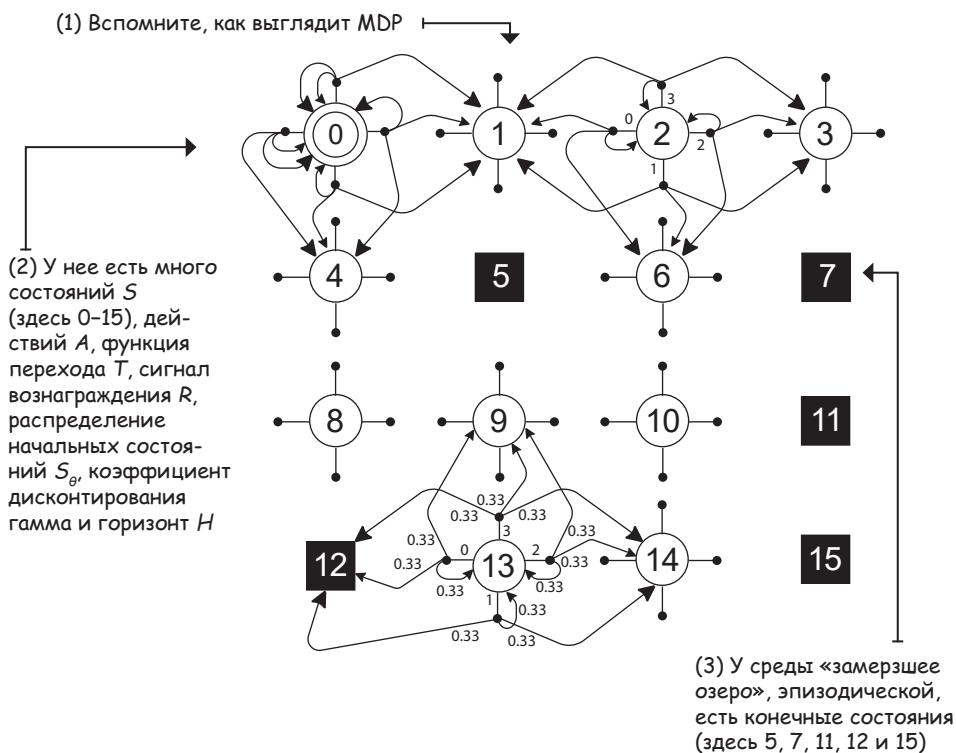


(1) В книге вы познакомились со всеми этими подходами к глубокому обучению с подкреплением: мы подробно рассмотрели ценностно ориентированные и стратегические методы, а также алгоритмы «актер-критик». Позже в этой главе будут представлены подходы, ориентированные на модели и не использующие производных

Марковский процесс принятия решений

Первые две главы были введением в область обучения с подкреплением и в процесс описания задач, которые мы пытаемся выполнить. Модели MDP — базовая концепция, у которой, несмотря на всю внешнюю простоту и ограниченность, широкие возможности. У этой области есть много других аспектов, которые мы могли бы исследовать. Я хочу, чтобы из всего этого вы вынесли для себя то, что задачи можно описывать в виде MDP. Займитесь этим: разделите задачу на состояния, наблюдения, действия и все остальные компоненты, которые превратили бы ее в MDP.

Функция перехода среды «замерзшее озеро»



Окружающий нас мир может показаться нестационарным и немарковским, но некоторые его аспекты можно преобразовать так, чтобы он принял вид MDP. Меняются ли вероятностные распределения в реальном мире или нам не хватает данных для определения фактических? Зависит ли будущее от прошлого или же пространство состояний настолько многомерно, что мы просто неспособны представить себе всю историю человечества как часть одного состояния?

Опять же попробуйте взять какую-то задачу и описать ее в рамках концепции MDP. Это может стать полезным упражнением, которое поможет вам применить DRL к своим задачам.

Методы планирования

В третьей главе мы обсуждали методы, которые помогают находить оптимальные политики для задач с доступными моделями MDP. Такие подходы, как итерация ценности и политик, последовательно вычисляют оптимальные функции ценности, что позволяет быстро извлекать оптимальные политики. Политика — это универсальный план.

Оценка политики «всегда влево» в среде SWF



(1) Здесь мы вычисляем Q-функцию после развертки полного состояния

Первая итерация

Н 0	0.0	0.0	0.0	0.0	НАЧАЛО 0.0	0.0	0.0	0.17	0.56	G 6
	1	2	3	4	5					

(2) Обратите внимание, что уже после первой итерации жадная политика по Q-функции стала лучше!

Вторая итерация

Н 0	0.0	0.0	0.0	0.0	НАЧАЛО 0.0	0.0	0.01	0.18	0.58	G 6
	1	2	3	4	5					

...

104-я итерация

Н 0	0.0	0.0	0.0	0.0	НАЧАЛО 0.01	0.01	0.03	0.04	0.24	0.63	G 6
	1	2	3	4	5						

(3) Как вы помните, именно это делает итерация ценности: улучшает процесс оценки усеченной политики

(4) Это полностью сошедшаяся функция ценности состояний для политики «всегда влево» — результат процесса оценки политики

Из этого раздела можно сделать два важных вывода. Во-первых, эти алгоритмы изолируют последовательные задачи принятия решений. Они требуют MDP, поэтому неопределенности нет, они ничего не усложняют, так как работают только в дискретных пространствах состояний и действий. Во-вторых, здесь есть общая закономерность: оценка поведения интересует нас в той же степени,

что и его улучшение. Я потратил много времени на то, чтобы это понять. Улучшение, оптимизация звучали интересней, поэтому я не обращал внимания на методы оценки. Но позже ко мне пришло понимание того, что при правильной оценке этап улучшения становится гораздо проще. Основная трудность обычно в том, чтобы дать политикам правильные и точные значения, но с MDP это делается довольно легко.

«Бандитские» методы

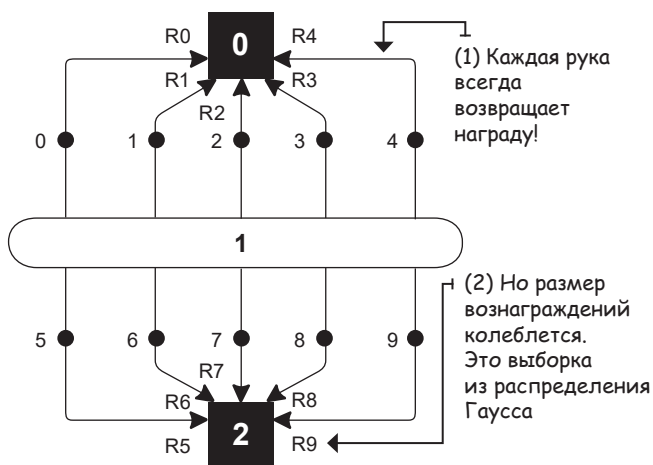
Глава 4 была посвящена обучению на оценочной обратной связи. Вы узнали об аспекте неопределенности обучения с подкреплением, убрав MDP. Мы прячем модель MDP, но делаем ее крайне простой: одно состояние, один шаг. Сложность в том, чтобы найти оптимальное действие или распределение действий за минимальное количество эпизодов, то есть минимизировать общие потери.

В главе 4 вы узнали о более эффективных способах поиска баланса между исследованием и эксплуатацией



Мы изучили несколько разных стратегий исследования и протестировали их в паре «бандитских» сред. Но моя цель в этой главе — показать, что неопределенность сама по себе создает проблемы, которые нужно рассматривать отдельно. Этой теме посвящено несколько книг, и, если вам интересно, ознакомьтесь с ними, потому что этот подход заслуживает особого внимания.

Десятируккие гауссовские бандиты



Здесь можно сделать следующий вывод: обучение с подкреплением сложнее методов планирования (см. главу 3) из-за отсутствия доступа к MDP. Без MDP нам приходится бороться с неопределенностью, исследуя среду. Стратегии исследования позволяют нашим агентам обучаться самостоятельно методом проб и ошибок. Именно это и делает эту область настолько захватывающей.

Табличное обучение с подкреплением

В главах 5, 6 и 7 мы сосредоточились сразу на двух аспектах обучения с подкреплением: последовательности и неопределенности. Последовательные задачи принятия решений в условиях неопределенности, представленные так, чтобы их было легче изучать (без крупных многомерных пространств состояний и действий), лежат в основе RL.

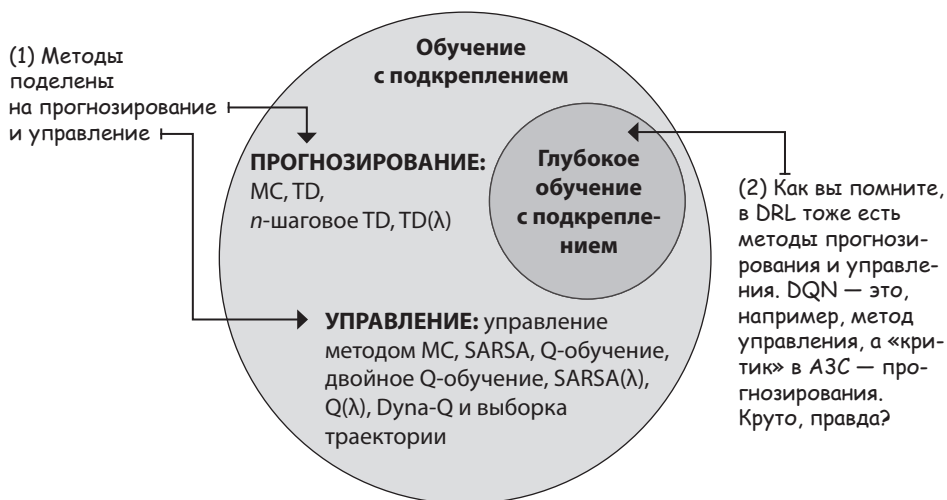
Глава 5 была посвящена оценке политик, глава 6 — их оптимизации, а глава 7 — продвинутым методам и того и другого. Я считаю это основой обучения с подкреплением. Уверенное понимание этих концепций поможет вам быстрее освоить DRL. Было бы заблуждением считать DRL чем-то отдельным от табличного обучения с подкреплением.

Ведущие лаборатории, занимающиеся исследованием DRL, нередко публикуют научные работы о решении задач с дискретными пространствами состояний и действий. В этом нет ничего постыдного. Это разумный подход — помните об этом, когда экспериментируете. Не беритесь сразу за самые сложные задачи: изолируйте запутанные аспекты и выработайте методы, которые позволят с ними справиться, а только потом повышайте уровень сложности.

В этих трех главах был рассмотрен широкий спектр алгоритмов. Мы обсудили такие подходы к оценке, как прогнозирование методом первого и каждого

визита Монте-Карло, временное разностное прогнозирование, n -шаговое TD и TD(λ). Мы не обошли стороной методы управления, включая управление методом первого и каждого визита Монте-Карло, SARSA, Q-обучение, двойное Q-обучение и более совершенные алгоритмы вроде SARSA(λ) и Q(λ) — оба с заменяющими и накопительными признаками. Мы уделили внимание и модельно-ориентированным подходам, таким как Дупа-Q и выборка траектории.

DRL — часть более широкой области обучения с подкреплением



Ценностно ориентированное глубокое обучение с подкреплением

Главы 8, 9 и 10 были посвящены особенностям методов ценностно ориентированного глубокого обучения с подкреплением. Мы затронули нейронную аппроксимацию Q, глубокие Q-сети (DQN), двойные глубокие Q-сети (DDQN), дуэльную архитектуру в DDQN (дуэльные DDQN) и приоритетное воспроизведение опыта (PER). Все началось с базового метода DQN, который мы постепенно улучшали. Все алгоритмы тестировались в среде «перевернутый маятник».

Есть и другие способы улучшить этот базовый алгоритм, и я советую вам их испробовать. Рассмотрите алгоритм Rainbow и внесите в DQN некоторые из оптимизаций, которые не вошли в эту книгу. Напишите об этом. Методики, которыми вы овладели в ходе реализации методов ценностно ориентированного глубокого обучения с подкреплением, — неотъемлемая часть других под-

ходов к DRL, включая формирование «критика» в методах «актер-критик». Вы можете открыть для себя много новых улучшений и методик. Не останавливайтесь.

Глубокое обучение с подкреплением, основанное на стратегических подходах и методах «актер-критик»

Глава 11 стала введением в стратегические подходы и методы «актер-критик». С первыми мы познакомились только на этом этапе, поэтому для их рассмотрения использовался простой алгоритм REINFORCE, который параметризует только политику. Поэтому ее аппроксимация происходила напрямую, без использования каких-либо функций ценности. В качестве сигнала для оптимизации в REINFORCE применялась выгода Монте-Карло, которую агент получал в ходе эпизода.

Мы исследовали алгоритм, формирующий функцию ценности для сокращения дисперсии выгоды MC, — базовый градиент политик (VPG). Это не совсем термин, и вместо него лучше было бы использовать *REINFORCE с направляющей*. Но важно отметить, что этот алгоритм, несмотря на формирование функции ценности, нельзя отнести к методам «актер-критик», так как функция в нем используется не как «критик», а как направляющая. Основной момент, который я хочу выделить, — то, что мы не используем функцию ценности для бэггинга и что у нее минимальное смещение, так как ее модель обучается на выгоде MC. За смещение в этом алгоритме ответственна только нейросеть.

Затем мы перешли к усовершенствованным методам «актер-критик», в которых применяется бэггинг: A3C (с n -шаговой выгодой), GAE (с разновидностью λ -выгоды для обновления политик) и A2C (с синхронным обновлением политик). Это продвинутые и надежные методы, которые широко используются до сих пор. Например, одно из главных преимуществ метода A3C (и его особенность) в том, что для работы ему нужен лишь ЦП: при отсутствии ГП он обучается быстрее остальных.

Продвинутые методы «актер-критик»

Несмотря на то что методы A3C, GAE и A2C относятся к категории «актер-критик», применение «критика» в них ничем не примечательно. В главе 12 мы исследовали методы, где использованию этого компонента уделяется особое внимание. Среди них можно выделить DDPG и TD3, которые лучше подходят для непрерывных пространств действий в качестве ценностно ориентированных методов, хотя многие относят их к группе «актер-критик». Например, если посмотреть, как A3C применяет компоненты «актер» и «критик», мы увидим много отличий от DDPG. В любом случае DDPG и TD3 — это передовые подходы, и при выполнении задач не так уж важно, к какой категории они относятся. Здесь

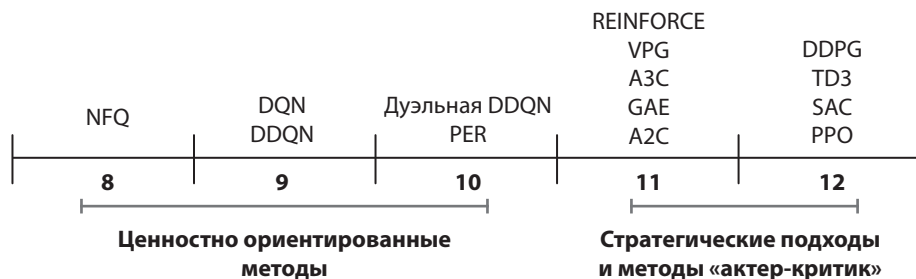
важно запомнить только то, что эти два метода могут работать лишь в средах с непрерывным пространством действий, тогда как их альтернативы вроде A3C совместимы как с непрерывными, так и с дискретными пространствами.

SAC — это отдельная тема. Мы рассматривали этот алгоритм после DDPG и TD3 только потому, что в нем используется много тех же методик. Но уникальным SAC делает его метод максимизации энтропии: функция ценности максимизирует не только выгоду, но и энтропию политики. Это многообещающие подходы, и я не удивлюсь, если на основе SAC появятся новые продвинутые методы.

В конце мы рассмотрели еще один интересный метод — PPO. Вы, наверное, заметили, что он относится к группе «актер-критик», потому что большая часть его кода была позаимствована из A3C. Основная особенность PPO — этап обновления политики. Если вкратце, то PPO улучшает политику понемногу: мы следим, чтобы эти изменения не были слишком большими. Можете считать это осторожной оптимизацией. PPO можно легко применять как к непрерывным, так и к дискретным пространствам действий. Этот алгоритм лежит в основе некоторых наиболее примечательных проектов в области DRL, таких как OpenAI Five, например.

В этих главах было рассмотрено много замечательных методов, но, что важнее, мы прошли по основам, благодаря которым вы будете лучше понимать эту область в дальнейшем. Многие алгоритмы основаны на методах из этой книги. Исключение составляют лишь методы модельно-ориентированного глубокого обучения с подкреплением и оптимизации без производных. В следующих двух разделах я расскажу о них подробнее, чтобы вы могли продолжить изучение DRL.

Алгоритмы DRL, рассмотренные в книге

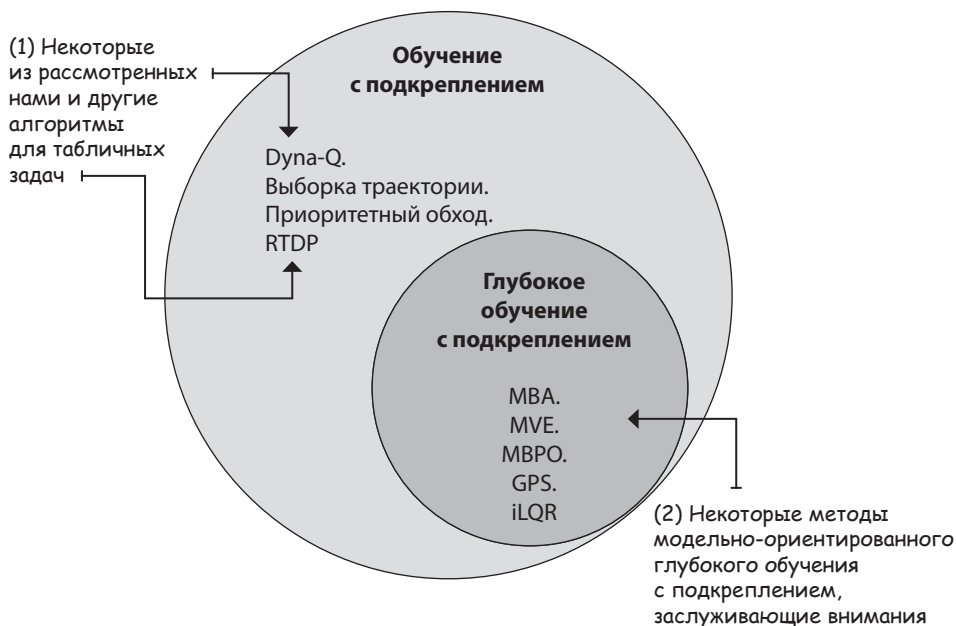


Модельно-ориентированное глубокое обучение с подкреплением

В главе 7 вы познакомились с методами модельно-ориентированного глубокого обучения с подкреплением, такими как Дупа-Q и выборка траектории. Их принцип работы заключен в их названии: мы используем методики глубокого обучения для формирования переходной функции и/или функции вознаграждения.

граждения и затем принимаем решения с учетом полученных результатов. Как и другие методы из главы 7, модельно-ориентированное глубокое обучение с подкреплением эффективно использует собранные данные (на самом деле модельно-ориентированный подход в этом смысле наиболее эффективный из всех существующих в RL).

Примечательные алгоритмы модельно-ориентированного обучения с подкреплением



Помимо эффективного использования данных, еще одно важное преимущество модельно-ориентированных методов — их универсальность. Построение динамики среды может помочь в решении разных смежных задач. Например, если вы обучаете агента управлению роботизированной рукой, чтобы он умел дотягиваться до объекта, ему будет легче научиться подбирать этот объект в дальнейшем, если он применит модельно-ориентированные методы для исследования реакции среды на его действия. Заметьте: здесь формирование модели функции вознаграждения не придает агенту универсальности. Но исследование того, как среда реагирует на движения руки, позволяет получить знания, полезные при выполнении других задач. Насколько я знаю, законы физики не менялись веками (к слову о медленно развивающихся областях науки!).

Есть еще несколько плюсов, о которых стоит упомянуть. Во-первых, изучение модели часто предполагает обучение под наблюдением, которое стабильнее и точнее, чем обучение с подкреплением. Во-вторых, с точной моделью среды мы

можем использовать для планирования теоретически обоснованные алгоритмы, такие как оптимизация траектории, модельно-прогнозируемое управление или даже эвристические поисковые методы вроде поиска по дереву Монте-Карло. И последнее: формирование модели в целом позволяет эффективнее использовать накопленный опыт, так как из среды извлекается большая часть информации, что открывает больше возможностей для принятия верных решений.

Но не все так гладко: у модельно-ориентированного обучения есть и свои сложности, о которых важно помнить при использовании этих методов. Прежде всего, моделирование динамики среды в дополнение к формированию политики и/или функции ценности требует больше вычислительных ресурсов. А если ограничиться только моделью динамики, погрешность, которую она будет накапливать, сделает ваш алгоритм непрактичным.

Не все аспекты динамики среды положительно и напрямую влияют на политику. Мы обсуждали эту проблему при рассмотрении методов, где политика формируется напрямую, без присвоения функции ценности. Представьте себе задачу, в которой вам нужно просто взять чашку и налить в нее что-то: если вы начнете изучать динамику жидкостей, их вязкость и текучесть, задача усложнится. Попытки моделирования среды сложнее, чем формирование политики напрямую.

Важно помнить, что моделям глубокого обучения нужно много данных. Как вы знаете, чтобы выжать максимум из глубокой нейросети, набор данных должен быть большим, что создает трудности для методов модельно-ориентированного DRL. Проблема усугубляется еще и тем, что в нейронных сетях сложно спрогнозировать и неопределенность модели. И, поскольку нейросеть пытается обобщать модель, какой бы неопределенной она ни была, вы можете получить совершенно негодные долгосрочные прогнозы.

Эта проблема ставит под сомнение лидерство модельно-ориентированных методов с точки зрения эффективного использования накопленного опыта: для получения полезной модели может потребоваться больше данных, чем для формирования хорошей политики в безмодельных методах. Но если у вас эта модель уже есть или вы можете ее получить отдельно от выполнения задачи, то она сможет применяться и в других средах. К тому же, если применять «поверхностные» модели, такие как гауссовский процесс или смешанная гауссовская модель, мы снова возвращаемся к тому, что модельно-ориентированные методы используют данные эффективней остальных.

Я не хочу, чтобы после прочтения этого раздела вы противопоставляли модельно-ориентированные и безмодельные методы, и не призываю вас их объединять, хотя иногда в результате может получиться что-то интересное. Работа инженера не поиск лучшего подхода, будь он ценностно ориентированным, стратегическим или методом «актер-критик». Не работайте молотком там, где нужна отвертка. Я лишь хотел объяснить, для чего лучше подходят те или иные виды алгоритмов. И только вам решать, как применять эти знания. Исследуйте, проводите интересные эксперименты — это важно. Но когда дело дойдет до выполнения задачи, тщательно обдумайте свой выбор.

Методы оптимизации без производных

Глубокое обучение — это изучение функции с помощью многоуровневых аппроксиматоров. Обычно оно проходит так. Сначала создается параметрическая модель, отражающая интересующую нас функцию. Затем мы определяем целевую функцию, чтобы знать, насколько ошибается модель в любой момент времени. После этого мы последовательно оптимизируем модель, определяя через обратное распространение, как изменить параметры. В итоге параметры обновляются с помощью градиентного спуска.

Обратное распространение и градиентный спуск — это практические алгоритмы оптимизации нейросетей. Они полезны для поиска самой низкой или высокой точки функции в заданном диапазоне: например, локального оптимума функции потерь или вознаграждения. Конечно, это не единственные и иногда не самые эффективные методы оптимизации параметрических моделей, вроде глубоких нейросетей.

Оптимизация без производных на основе генетических алгоритмов или эволюционных стратегий — это еще один способ улучшения моделей, который в последние годы стал популярен в сообществе DRL. Эти методы, известные также как безградиентные, методы черного ящика или нулевого порядка, не нуждаются в производных и могут пригодиться в ситуациях, когда плохо подходит градиентная оптимизация. Методы на основе градиентов показывают плохие результаты при оптимизации дискретных, прерывистых или, к примеру, многомодельных функций.

Методы без производных во многих случаях могут быть полезными и довольно простыми. Даже случайный подбор весов в нейросети может принести результат, если у вас достаточно вычислительных ресурсов. Основное преимущество таких методов в том, что они могут оптимизировать произвольные функции. Для работы им не нужны градиенты. Еще один плюс в том, что их легко распараллелить. Эти методы нередко применяются в задачах с сотнями или тысячами ЦП. Но за простотой распараллеливания скрывается низкая эффективность использования данных. Эти методы не учитывают последовательность задач обучения с подкреплением, теряя сведения, которые могли бы пригодиться при оптимизации.





Аналогия от Мигеля

Как работают методы без производных

Чтобы понять, чем отличаются градиентные методы от безградиентных, вспомните игру «холодно-горячо». Ту самую детскую игру, где один ребенок (охотник) должен найти какой-то предмет, а остальные знают, где он спрятан: если охотник приближается к предмету, дети кричат «горячо», а если удаляется от него — «холодно». В этой аналогии местоположение охотника обозначает параметры нейросети. Спрятанный предмет — это глобальный оптимум, выраженный либо самой низкой точкой функции потерь, которую нужно минимизировать, либо самой высокой точкой целевой функции, которую нужно максимизировать. Мы пытаемся оптимизировать расстояние между охотником и предметом. Для этого дети выкрикивают «холодно» или «горячо», меняя тем самым местоположение охотника.

Теперь мы подходим к самому интересному. Представьте, что дети не просто кричат «холодно» или «горячо», но еще и повышают голос, когда охотник приближается к предмету. Вы, наверное, знаете, что малыши легко перевозбуждаются и не умеют хранить секреты. Когда охотник слышит, что тихое «холодно» постепенно превратилось в громкое «горячо», он знает, что выбрал верное направление и эту «градиентную» информацию можно использовать для сокращения состояния. Этим и занимаются градиентные методы. Когда информация поступает непрерывно, то есть дети выкрикивают по нескольку раз в секунду, возгласы становятся громче или затихают в зависимости от расстояния, вы можете использовать эти изменения величины значений (то есть градиент) для поиска предмета. Отлично!

Но теперь представьте, что дети, направляющие охотника, не умеют или не хотят играть честно. Они могут давать неполную информацию. Например, когда охотник находится в определенном месте, они могут молчать. Так мы можем получить тихое «холодно», затем молчание, а затем снова тихое «холодно». Или представьте, что предмет находится за длинной стеной, прямо по центру, и даже если охотник приблизится к нему, то не сможет его достать с помощью градиентной информации. Так как он приблизился к цели, было бы правильно кричать «горячо», но предмет находится за стеной, вне зоны досягаемости. Во всех этих случаях градиентная оптимизация не самый оптимальный подход. И здесь приходят на помощь безградиентные методы.

Безградиентный подход может быть максимально прост. Охотник движется в случайном направлении, игнорируя поступающую «градиентную» информацию, затем прислушивается к детям и выбирает другую случайную позицию. Получив представление о нескольких (скажем, десяти) таких позициях, охотник выбирает из них тройку лучших и испытывает те, что по соседству, которые могут быть лучше. В этом случае градиентная информация бесполезна.

Эту аналогию можно развивать и дальше, но я остановлюсь. Все сводится к тому, что градиентные и безградиентные методы — это всего лишь стратегии достижения нужного нам результата. Их эффективность зависит от задачи, которую мы решаем.

Углубленные аспекты AGI

Ранее мы провели обзор ключевых идей глубокого обучения с подкреплением из этой книги и затронули две важные категории методов, которые не рассматривались здесь подробно. Но, как я уже упоминал, есть много других продвинутых концепций, которые, хоть и не нужны при знакомстве с DRL, играют важную роль в разработке сильного ИИ (AGI), в чем и заключается цель большинства исследователей ИИ.

Этот раздел мы начнем с того, что еще сильнее углубимся в тему AGI и обсудим несколько характеристик, которыми должны обладать агенты ИИ для выполнения задач, требующих высокого интеллекта. Я кратко объясню, что это за характеристики и для чего они нужны, чтобы вы могли продолжить изучение ИИ. Возможно, вам когда-то удастся внести свой вклад в одну из этих передовых областей исследования.

Еще раз, что такое AGI?

В этой книге вы видели много впечатляющих примеров агентов ИИ. То, что одна и та же компьютерная программа может научиться выполнять разные задачи, кажется удивительным. А когда мы перешли к средам посложнее, результаты оказались и вовсе головокружительными: AlphaZero учится играть в шахматы, го и сёги. OpenAI Five побеждает человеческие команды в игре Dota2. AlphaStar выигрывает у одного из лучших профессиональных игроков в StarCraft II. Это серьезные общие алгоритмы. Но проявляют ли они какие-то признаки сильного интеллекта? Чтобы ответить на этот вопрос, давайте сперва разберемся, что такое общий интеллект.

Сильный интеллект — это умение сочетать разные когнитивные способности для выполнения новых задач. Например, от AGI мы ожидаем, что компьютерная программа проявит общий интеллект. С этим разобрались. Теперь следующий вопрос: «Возможно ли, что какие-то алгоритмы из этой книги или даже передовые методы вроде AlphaZero, OpenAI Five и AlphaStar — это примеры сильного ИИ? Здесь не все так однозначно, но я бы сказал, что нет.

С одной стороны, многие из этих алгоритмов могут применять сразу несколько когнитивных способностей, включая восприятие и обучение, для выполнения новых задач, таких как игра Pong. Мы определили это свойство как положительное. Но один из самых слабых аспектов этих алгоритмов в том, что все их агенты плохо справляются с выполнением новых задач, если их специально для этого не обучить. Как правило, для получения хоть каких-то хороших результатов нужны миллионы выборок. Проще говоря, если вы учите агента DQN играть в Pong с помощью пикселей, он в итоге сможет превзойти человека, но при этом не будет иметь малейшего понятия о том, как играть в Breakout. Чтобы показать приличные результаты в этой игре, ему придется обучаться на миллионах кадров.

У людей такой проблемы нет. Я уверен, что, научившись играть в Pong, вы сможете за две секунды разобраться в Breakout, ведь в обеих играх нужно бить

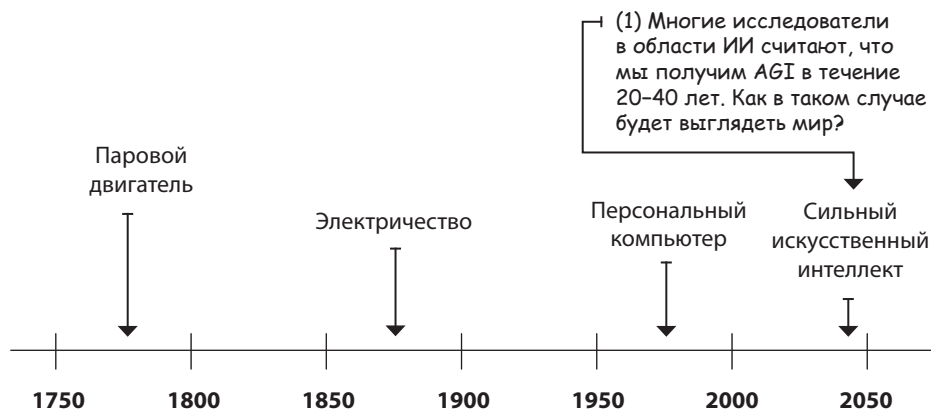
ракеткой по мячику. Но даже агент AlphaZero, программа, демонстрирующая невиданный уровень мастерства в нескольких совершенно разных настольных играх и способная побеждать профессиональных опытных игроков, никогда не сможет постирать ваши вещи.

Некоторые исследователи говорят, что их цель — создать системы ИИ, которые воспринимают информацию, обучаются, думают и даже чувствуют как люди. Даже одна мысль об этом будоражит воображение. Но есть ученые и с более приземленными целями. Создание ИИ, который мыслит как человек, — это не их первоочередная цель, если только это не обязательное условие для приготовления хорошего обеда. Возможно, опытный повар должен быть эмоциональным, кто знает? Суть в том, что одни люди хотят переложить на AGI рутинные задачи, а другие настроены более философски. Создание AGI может открыть путь к пониманию интеллекта как такового, к самопознанию, что само по себе было бы для человечества выдающимся достижением.

Но любой исследователь в области ИИ, независимо от его цели, согласился бы с тем, что нам нужны алгоритмы с более универсальными умениями. Прежде чем системы искусственного интеллекта смогут выполнять рутинные задачи, такие как стирка, готовка или мытье посуды, им нужно будет приобрести много дополнительных навыков. Интересно, что именно бытовые занятия вызывают у ИИ больше всего трудностей. Давайте рассмотрим несколько новейших областей исследований, которые расширяют границы возможного, чтобы заставить глубокое обучение с подкреплением и искусственный интеллект показывать признаки общего интеллекта.

В следующих разделах представлено несколько идей, которые могут вас заинтересовать, для дальнейшего изучения передовых методов DRL, приближающих область ИИ к интеллекту человеческого уровня. Чтобы охватить как можно больше информации, я уделю каждому из направлений лишь несколько предложений. Я хочу показать вам двери, а не открыть их для вас. Дальнейший выбор за вами.

Производственные революции



Продвинутые стратегии исследования

Одна из областей исследований, которая показывает впечатляющие результаты, связана с функцией получения награды. На страницах этой книги вам встречались агенты, которые обучаются на сигналах вознаграждения, но, как показывают недавние исследования, агенты могут обучаться и без них, что очень любопытно. Все это звучит заманчиво и может оказаться ключом к созданию человекоподобного интеллекта. Наблюдая за младенцем, можно заметить, что для обучения ему не нужны взрослые. Конечно, рано или поздно мы начинаем поощрять наших детей. Кто-то получает хорошую оценку, а кто-то похуже, у кого-то зарплата высокая, а у кого-то не очень. Но агенты не всегда преследуют награды, которые мы подготовили на их пути. Какова же функция вознаграждения для человека? Успешная карьера? Дети? Однозначного ответа нет.

Конечно, отказ от вознаграждения в обучении с подкреплением может казаться немного пугающим. Если у агента нет функции, которую ему нужно максимизировать, то как убедиться, что его цели совпадают с нашими? Как создать общий ИИ, пригодный для преследования человеческих целей? Возможно, для создания человекоподобного интеллекта мы должны дать агенту свободу выбора, позволив ему самому вершить свою судьбу? В любом случае я считаю это направление исследований одним из важнейших.

Обратное обучение с подкреплением

Обычно для формирования поведения предпочитают внедрять функцию вознаграждения, но есть и другие подходы. Например, имитация поведения человека может помочь в формировании политик с использованием меньшего числа выборок. Этому посвящено несколько смежных областей. *Поведенческое клонирование* — это применение методик контролируемого обучения для формирования политики на основе примеров, показанных человеком. Как можно догадаться по названию, агент занимается не размышлениями, а обобщением. Смежная область — *обратное обучение с подкреплением*. Это определение функции вознаграждения из демонстрационных примеров. Здесь мы уже не просто копируем поведение, а изучаем намерения другого агента. *Определение намерений* может стать мощным инструментом для достижения разных целей. Например, если взять многоагентное обучение с подкреплением как в состязательном, так и в совместном режиме, то информация о целях других агентов может пригодиться. Если стремления, о которых мы узнали, противоречат нашим собственным целям, мы можем выработать стратегии для противодействия, пока не поздно.

Но обратное обучение с подкреплением позволяет агентам формировать новые политики. Методику, где функция вознаграждения заимствуется у другого агента (или человека), а затем из нее формируется политика, многие называют *обучением на демонстрации* или *ученичеством* (apprenticeship learning). При знакомстве с обратным RL важно обратить внимание на то, что функция вознаграждения часто более лаконична, чем оптимальная политика. Попытки изучения функции вознаграждения уместны в разных ситуациях. Формирование

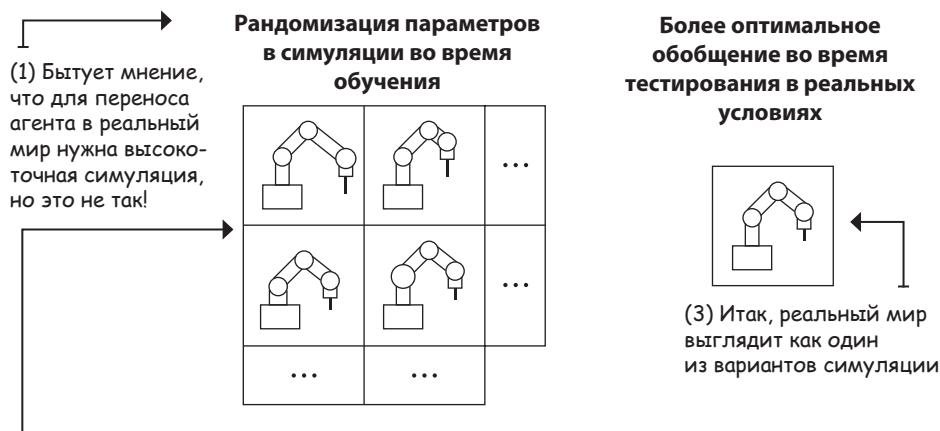
политик на основе демонстраций еще называют *имитационным обучением*, при котором функция вознаграждения выводится либо перед использованием политики или прямого поведенческого клонирования. Имитационное обучение часто применяется для инициализации агента с помощью приемлемой политики. Например, если агенту нужно обучаться на случайном поведении, на формирование хорошей политики уйдет много времени. Суть в том, что имитация человека, даже если это не самый подходящий метод, может привести к получению оптимальных политик с меньшим количеством взаимодействий со средой. Но это не всегда так: у политик, заранее обученных на примерах от человека, может быть нежелательное смещение, что усложнит для агентов процесс их улучшения.

Перенос обучения

Вы, наверное, заметили, что агентов, обученных в одной среде, нельзя перенести в другую. Агенты обучения с подкреплением — это общие алгоритмы: один и тот же агент может обучаться в разных средах. Но у агентов нет общего интеллекта, и то, чему они учатся, нельзя перенести напрямую в новые среды.

Перенос обучения, или *трансферное обучение* (transfer learning), — это область исследований, где изучаются способы переноса знаний из набора сред в новую среду. Например, один из подходов, который может быть вам интуитивно понятен, если у вас есть опыт глубокого обучения, называется *тонкой настройкой* (fine-tuning). Подобно тому как в контролируемом обучении мы повторно используем веса заранее обученной сети, агенты, обученные в похожих средах, могут повторно использовать в другой задаче признаки, извлеченные сверточными слоями. Если среды как-то взаимосвязаны, как в случае с играми Atari, несколько признаков могут оказаться переносимыми. В некоторых средах переносить можно даже политики.

Перенос знаний из симуляции в реальность широко востребован



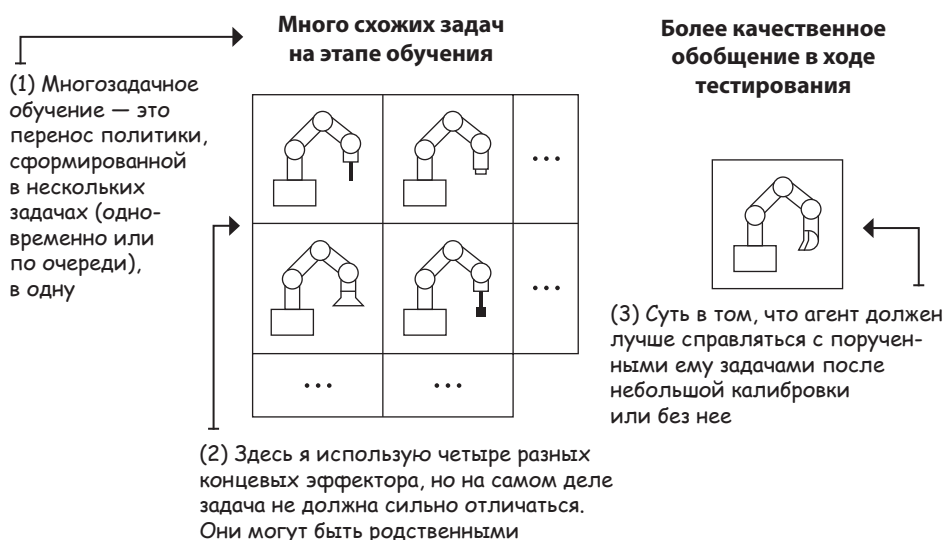
(2) Для этой цели лучше подойдет гибкая симуляция, позволяющая рандомизировать параметры в ходе обучения. Так агент будет делать более качественные обобщения

Общее направление исследований, посвященное обучению агентов более универсальным навыкам, называется *переносом обучения*. С его помощью нередко переносят в реальные условия и политики, сформированные в симуляции (sim-to-real). Перенос знаний вида sim-to-real востребован в робототехнике, где обучение агентов для управления роботами может быть непростым, дорогостоящим и опасным занятием. К тому же он не так хорошо масштабируется, как обучение в симуляции. Часто сначала приходится обучать в условиях симуляции, а затем переносить полученную политику в реальный мир. Бытует заблуждение, что для этого симуляция должна быть высокоточной, но исследования показывают обратное. Переносимость агента зависит от разнообразия наблюдений. Такие методики, как рандомизация параметров, очень быстро развиваются и показывают многообещающие результаты.

Многозадачное обучение

Есть еще одно родственное направление исследований — *многозадачное обучение*, где перенос обучения рассматривается немного иначе. Цель этой области — обучение не на одной, а на нескольких задачах с последующим переносом знаний в новую среду. Здесь пригодятся методы модельно-ориентированного обучения с подкреплением. Например, в робототехнике обучение робота разным задачам может помочь агенту сформировать более устойчивую модель динамики среды. Он узнает, что такое сила тяжести, как перемещаться вправо или влево и т. д. Чем бы ни занимался агент, сформированная модель динамики может быть перенесена в новую задачу.

Многозадачное обучение состоит из обучения нескольким связанным задачам и тестирования новой



Обучение по плану

Распространенный сценарий использования многозадачного обучения — разбиение одной задачи на несколько отсортированных по уровню сложности. Здесь у агента есть план, следуя которому он постепенно учится выполнять все более сложные задачи. *Обучение по плану* может быть полезно при создании сценариев. Если вам нужно сгенерировать для агента среду, которую он должен пройти, начинать нужно с простейшего сценария с частыми вознаграждениями. Это позволит агенту быстро продвигаться к цели и подтвердит, что ваша среда работает хорошо. Позже вы можете повысить сложность, вводя более редкие вознаграждения. Прodelав это для нескольких сценариев, вы сможете создавать план обучения, которым может пользоваться ваш агент. Потом он сможет обучаться во все более сложных средах и, если все будет хорошо, быстрее выработать желаемое поведение.

Метаобучение

Еще одна очень захватывающая область исследований — *метаобучение*. Если подумать, мы вручную пишем код для агентов, которые потом изучают много разных задач. В какой-то момент мы сами становимся слабым местом этого процесса. Разработай мы агент, который обучается не решению сложных задач, а тому, как учиться самостоятельно, весь процесс обучения стал бы автономным: ну, не совсем, но это был бы шаг в верном направлении. Овладение навыками самообучения позволяет использовать опыт, полученный при выполнении разных задач, для более эффективного самостоятельного обучения. Звучит логично. Метаобучение породило другие направления исследований, такие как автоматическое обнаружение методов оптимизации и архитектур нейросетей. Обратите на это внимание.

Иерархическое обучение с подкреплением

Часто мы можем обнаружить, что разрабатываем среды, в которых есть многоуровневые проблемы. Например, если нам нужно, чтобы агент нашел лучшую высокоуровневую стратегию, но при этом мы даем ему для действий низкоуровневые команды управления, то агент должен научиться переходить из низкоуровневого пространства действий в высокоуровневое. Понятно, что для большинства агентов существует иерархия политик. Когда я что-то планирую, я делаю это в высокоуровневом пространстве действий. Я думаю о том, как сходить в магазин, а не о движении моих рук по пути туда. *Иерархическое обучение с подкреплением* позволяет агентам создавать внутри себя иерархию действий для выполнения долгосрочных задач. В результате агент больше не мыслит командами «влево-вправо», а сосредоточен на выборе направления движения.

Многоагентное обучение с подкреплением

Мир не был бы таким захватывающим, если бы в нем был всего один агент. В этом типе обучения с подкреплением уделяется внимание методикам, в которых агент узнает, что, помимо него, есть и другие. Одна из главных трудностей здесь в том, что эти агенты обучаются вместе с вашим, меняя тем самым свое поведение. В результате этих изменений наши наблюдения становятся неустойчивыми, так как знания, получаемые агентом, немедленно устаревают, что усложняет процесс обучения.

Есть один интересный подход к совместному *многоагентному обучению с подкреплением*: использование методов «актер-критик», в которых «критик» применяет во время обучения всю информацию о состояниях, принадлежащую всем агентам. Преимущество здесь в том, что ваши агенты учатся работать совместно через «критика», после чего мы можем тестировать политику в более реалистичном пространстве наблюдений. Разделение всего пространства состояний может показаться неправдоподобным, но к этому можно относиться как к командным тренировкам, во время которых можно все. Например, футболист может сказать своим сокомандникам, что собирается сделать забег по флангу после какого-то условленного движения. Во время тренировки можно практиковаться, обладая всеми нужными данными, а после в ходе тестирования использовать выработанную стратегию в условиях ограниченного владения информацией.

У такого способа обучения есть еще одна привлекательная особенность: его можно рассматривать как общий случай мультиагентного RL. Как так? Подумайте, как несколько агентов принимают решения на разных уровнях. Многоуровневая структура подобна организационной в большинстве компаний. Руководство ставит высокоуровневые цели на несколько лет вперед, а остальные работники пытаются их достичь из месяца в месяц, изо дня в день. Те, кто находится наверху, ставят цели тем, кто внизу. За хорошую работу всех агентов вознаграждается вся система.

Конечно, многоагентное обучение с подкреплением может работать не только в совместном, но и в состязательном режиме, который будет еще интереснее. Люди часто считают конкуренцию и противников чем-то неприятным, но многоагентное обучение с подкреплением предполагает, что соперники — это лучший способ для самосовершенствования. В основе многих недавних историй успеха в области многоагентного RL лежат методы обучения, предусматривающие наличие соперников. Это могут быть как ранние версии того же агента (как в игре с самим собой), так и целое распределение других агентов в виде турнира, полученное после проведения всех состязаний (выживают только лучшие агенты). Соперники нередко позволяют нам развиваться, и, как бы мы к этому ни относились, они могут быть нужны для достижения оптимального поведения.

Объяснимый ИИ, безопасность, справедливость и этические стандарты

Есть несколько других важных областей исследований, которые, хоть и не служат прямым толчком к развитию человеческого интеллекта, являются основополагающими для успешных разработки, развертывания и внедрения решений ИИ.

Объяснимый искусственный интеллект — это попытка сделать агенты понятнее для человека. Мотивация здесь очевидна. Суд может допросить любого, кто нарушил закон. Но модели машинного обучения не рассчитаны на то, чтобы их можно было объяснить. Чтобы обеспечить высокие темпы внедрения ИИ в обществе, исследователи должны позаботиться о том, чтобы вопрос объяснимости стоял не так остро. Честно говоря, я считаю, что это не обязательно. Если ИИ дает мне точные прогнозы цен на фондовом рынке, меня не заботит, может он объяснить свое поведение или нет. Но в вопросах жизни и смерти, в которые вовлечены люди, ситуация может быстро выйти из-под контроля.

Еще одна область, которой нужно уделять больше внимания, — это безопасность. Катастрофические провалы, которые нередко случаются с системами ИИ, выглядят очевидными для людей. К тому же он подвержен атакам, которые не страшны человеку. Нам нужно удостовериться, что мы знаем, как развернутые системы ИИ будут реагировать на те или иные ситуации. Сейчас у нас нет возможности применить к искусственному интеллекту классический процесс проверки и утверждения (*validation and verification, V&V*) программных подходов, и это серьезная преграда на пути внедрения ИИ.

Важный аспект, который мы не затронули, — справедливость. Мы должны начать задумываться о том, что контролирует ИИ. Если компания создаст ИИ, который будет повышать прибыль в ущерб обществу, то какой тогда смысл в технологиях искусственного интеллекта? Нечто похожее уже происходит в сфере рекламы. Ведущие компании применяют ИИ для увеличения дохода путем манипуляций. Допустимо ли это? А ведь он становится все лучше и лучше. Но какой в этом смысл, если это вредит людям? Такие вопросы требуют серьезного рассмотрения.

Наконец, этические стандарты ИИ — еще одна проблема, которая недавно привлекла внимание благодаря Монреальской декларации об ответственном развитии ИИ. В ней перечислено десять этических принципов для ИИ, который служит интересам общества, а не только коммерческим компаниям.

Это несколько важнейших направлений, о которых важно помнить, когда будете готовы внести свой вклад.

Что дальше?

Этот раздел означает, что книга подошла к концу. Но это должно стать лишь началом или продолжением вашего участия в развитии областей ИИ и DRL. Здесь я хотел не только познакомить вас с основами DRL, но и сделать вас частью нашего большого сообщества. От вас нужна лишь решимость двигаться дальше. Перед вами открывается много возможностей, и в этом разделе я бы

хотел предложить вам несколько идей для уверенного старта. Нужно понимать, что мир — это хор, для которого важен каждый голос и талант. Примите то, чем вы одарены, развивайте это, насколько возможно, и внесите свой вклад, не щадя сил. Я могу поделиться своими соображениями, но только вам решать, что делать дальше: миру нужен ваш голос.

Как использовать DRL для выполнения нестандартных задач

У алгоритмов RL есть невероятно крутое свойство, о котором вы должны помнить, изучая другие виды агентов. Большинство агентов RL могут выполнить любую задачу на ваш выбор, если вы сформулируете ее в виде верной модели MDP, как в главе 2. Если задаться вопросом: «Что может выполнить алгоритм X или Y?» — ответ будет следующим: «То же, что и любой другой». В этой книге мы сосредоточились на нескольких алгоритмах, но при некотором изменении гиперпараметров все представленные здесь агенты могут пройти много других сред. Способность работать в произвольных средах интересует многих, но для полноценного рассмотрения этой темы нужна отдельная книга. Я советую ознакомиться с примерами из Интернета. Скажем, в средах Atari используется эмулятор Stella. Он обменивается со средой действиями и наблюдениями в виде изображений. Аналогично MuJoCo и система симуляции Bullet Physics служат бэкендом, лежащим в основе сред с непрерывным управлением. Посмотрите на то, как работают эти среды.

Обращайте внимание на то, как наблюдения передаются из симуляции в среду, а затем агенту и как его действия передаются среде, а затем системе симуляции. Это распространенный подход. Но если вы хотите создать свою среду, посмотрите, как это делают другие, и уже потом делайте сами. Возможно, вы хотите создать среду, где агент будет учиться инвестировать в фондовый рынок? Подумайте о том, у каких платформ есть API-интерфейс, который позволит вам это сделать. Позже вы можете создавать с помощью этого API разные среды. Например, в одной можно будет покупать акции, в другой — опционы и т. д. У продвинутых методов DRL есть много потенциальных применений, и досадно, что у нас в распоряжении совсем немного качественных сред. Вклад в развитие этой области, безусловно, приветствуется. Если вы не можете найти подходящую среду, попробуйте создать свою и затем поделиться ею со всем миром.

Дальнейшие шаги

Без сомнений, вы многому научились. Но если взглянуть на вещи в целом, то новых знаний, которые вы можете приобрести, намного больше. В мире еще очень много нового и неизведанного. Как видите, перед сообществом ИИ стоит непростая задача — понять, как работает разум.

На самом деле даже другие области, такие как психология, философия, экономика, лингвистика, исследование операций, теория управления и т. п., преследуют ту же цель, но с разных точек зрения и пользуются разной терминологией. Суть в том, что все эти направления только бы выиграли от понимания того, как работает разум, как люди принимают решения и как эти решения оптимизировать. Ниже вы найдете несколько идей о том, куда двигаться дальше.

Для начала определитесь с тем, что вас мотивирует, что привлекает ваше внимание, какие у вас амбиции. Некоторые полностью поглощены жадой исследований: узнавать новое о человеческом разуме довольно увлекательно. Другие хотят сделать мир лучше. Найдите свою мотивацию, какой бы она ни была, — найдите то, что вами движет. Если вы не привыкли к чтению научных работ, процесс не будет доставлять вам удовольствие, пока вы не определитесь с мотивацией. Когда вам это удастся, вы должны оставаться спокойными, скромными и откровенными. Ваша мотивация должна помочь вам сосредоточиться и усердно работать над достижением своих целей. Не позволяйте вдохновению стоять на вашем пути. Научитесь удерживать его в своем сердце, двигаясь вперед. Наша способность концентрироваться всегда под угрозой со стороны разных отвлекающих факторов. Каждые 15 минут на мой смартфон гарантированно приходят новые уведомления. Мы привыкли думать, что это нормально. Нам нужно вернуть контроль над своими жизнями и научиться как следует концентрироваться на том, что нам интересно, что мы любим. Тренируйте свое внимание.

Во-вторых, ищите баланс между приобретением новых знаний и участием в развитии области, не забывая при этом отдыхать. Как вы думаете, что будет, если на протяжении следующего месяца я буду потреблять 5000 калорий в день, а сжигать 1000? Или наоборот: потреблять 1000, а сжигать 5000? Или же представьте, что я спортсмен, который ежедневно потребляет и сжигает 5000 калорий, тренируясь семь дней в неделю? Все это нанесет серьезный ущерб здоровью. То же самое происходит и с разумом: некоторым кажется, что, прежде чем что-то сделать, нужно годами обучаться, поэтому они читают, смотрят видео, но никак не применяют свои знания. Есть и люди, которым не нравится читать научные труды — в конце концов, они уже реализовали агент DQN и написали об этом статью в блоге. Их знания быстро устаревают, и им недостает пищи для размышлений. Но даже те, кому удастся сбалансировать эти два аспекта, могут не находить времени для того, чтобы расслабиться, провести время в кругу семьи и как следует подумать. Это тоже неправильно. Найдите баланс между тем, что вы потребляете, тем, чем вы делитесь, и тем, как отдыхаете. Мы интеллектуальные спортсмены. Если нам не удастся применить большую часть изученной информации, наш разум может «потерять форму», мы станем вялыми и слишком медлительными. Если тратить все время на написание статей, не занимаясь исследованиями, наш текст устареет, станет скучным и сухим. Если же не выделить достаточно времени для отдыха, это плохо скажется на долгосрочном планировании.

Невозможно знать все на свете. Мы изучаем разум, и информации на эту тему очень много. Будьте избирательны и благоразумны, когда ищете, что почитать. Кто автор? Чем он примечателен? Вы можете прочесть книгу, но не понять, как применить на практике полученную информацию. Почаще делитесь своими знаниями. Вы должны уметь объяснить то, что выучили. Выражение «не нужно изобретать велосипед» только собьет вас с толку. Не бойтесь пробовать делать что-то самостоятельно: без этого не обойтись. По прочтении очередной книги вас может посетить отличная идея, которую, как вы позже узнаете, уже предлагали до вас. В этом нет ничего постыдного: важно двигаться вперед, а не ждать момента прозрения, который решит все проблемы человечества.

Я слышал, как Рич Саттон сказал: «То, что для вас очевидно, — это ваш самый важный вклад». Но, избегая «изобретения велосипеда», вы можете не поделиться «очевидными для вас» идеями, посчитав их недостойными внимания. Я не говорю, что вам нужно публиковать научную работу о новом алгоритме, который вы только что придумали и который подозрительно похож на Q-обучение. Я просто хочу, чтобы вы продолжали экспериментировать, не опасаясь того, что ваша работа может оказаться «бесполезной». Читайте, исследуйте, вносите свой вклад, и пусть все идет своим чередом. Это цикл, поток, поэтому просто продолжайте двигаться вперед.

Третий и последний совет: станьте частью прогресса, погрузитесь с головой в то, что делаете. Ваши мечты позволяют вам развиваться, но, развиваясь, вы сможете воплотить свои мечты. Не следуйте слепо за другими, а делайте то, что интересно вам. Относитесь критически к своим идеям, экспериментируйте, собирайте данные, пытайтесь разобраться в результатах и абстрагируйтесь от них. Не искажайте свои эксперименты, выявляйте факты. Активно занимаясь каким-то одним направлением долгое время, вы начнете в нем разбираться, и это хорошо. Область ИИ настолько широка, что досконально овладеть всеми ее аспектами невозможно. Но, достаточно долго преследуя свои интересы и прислушиваясь к своей интуиции, вы и сами не заметите, как станете уделять больше времени каким-то определенным вещам. Не останавливайтесь. Некоторые из нас чувствуют себя уверенней, оставаясь в зоне комфорта: когда ответов на интересующие нас вопросы нет, нам хочется вернуться обратно на твердую почву. Но это вовсе не значит, что стоит перестать их задавать. Глупых вопросов не бывает, ведь каждый из них — ключ к разгадке тайны. Продолжайте спрашивать. Продолжайте играть в эту игру и наслаждайтесь процессом.

Заявите о себе прямо сейчас!

Если вы дочитали эту книгу, не теряйте времени. Поработайте над чем-нибудь и поделитесь этим с сообществом. Напишите статью о понравившихся вам алгоритмах, которые не были здесь рассмотрены, или исследуйте передовые идеи, которые мы обсудили в этой главе, и расскажите о своих находках. Снимите видеоролик и поделитесь им со всем миром. Станьте частью движения. Давайте узнаем, что такое разум, давайте вместе создадим разумные системы. Лучшего времени для этого не придумать.

Подведем итоги

Вот и все! У вас получилось! На этом книга заканчивается. Что делать дальше, решать вам.

В первой главе я дал глубокому обучению с подкреплением следующее определение: «Глубокое обучение с подкреплением — это подход к ИИ на основе машинного обучения, направленный на создание компьютерных программ, способных выполнять задачи, требующие интеллекта. Отличительная черта

программ DRL — обучение методом проб и ошибок на основе обратной связи, которая является одновременно последовательной, оценочной и выборочной за счет использования мощной аппроксимации нелинейных функций».

Я писал, что, если по прочтении этой книги вы сможете вернуться к этому определению и полностью его понять, для меня это будет успехом. Я отметил, что вы должны объяснить, почему была выбрана именно такая формулировка и что каждое из этих слов означает в контексте глубокого обучения с подкреплением.

Достиг ли я своей цели? Понимаете ли вы это определение? Теперь ваша очередь выдать награду агенту, написавшему эту книгу. Как бы вы оценили этот проект: -1 , 0 или $+1$? Подобно агентам DRL, я обучаюсь на обратных связях, какими бы они ни были. С нетерпением жду вашей рецензии и комментариев. А моя работа на этом закончена.

В заключительной главе мы провели краткий обзор всего, чему учит эта книга, обсудили основные методы, которые в нее не вошли, и передовые идеи, которые могут повлиять на создание агентов с сильным искусственным интеллектом.

На прощание хотел бы поблагодарить вас за возможность поделиться своими взглядами на область глубокого обучения с подкреплением. Призываю вас не останавливаться: сосредоточьтесь на повседневной работе, подумайте о том, чем вы хотите заняться дальше, учитывая ваши уникальные таланты и способности.

На этом этапе вы:

- полностью понимаете, что такое глубокое обучение с подкреплением. Вы знакомы с деталями важнейших методов DRL, от базовых до продвинутых;
- знаете, куда двигаться дальше, понимая, как приобретенные вами знания относятся к областям глубокого обучения с подкреплением и ИИ;
- готовы показать нам свои знания, уникальные таланты и способности. Сделайте так, чтобы сообщество RL вами гордилось!



Не стыдно затвитить

Делитесь своими успехами

Вот несколько идей о том, как вывести полученные вами знания на новый уровень. Если хотите, можете поделиться своими результатами со всем миром. И не забывайте знакомиться с успехами других читателей. Это отличный шанс, и я надеюсь, что вы им воспользуетесь.

- **#gdrl_ch13_tf01.** Реализуйте методы модельно-ориентированного глубокого обучения с подкреплением.
- **#gdrl_ch13_tf02.** Реализуйте методы безградиентного глубокого обучения с подкреплением.

- **#gdrl_ch13_tf03.** Реализуйте многоагентную среду или агент для нее и поделитесь результатом.
- **#gdrl_ch13_tf04.** Воспользуйтесь продвинутыми методами глубокого обучения, не рассмотренными в этой книге, чтобы улучшить результаты агентов DRL. Например, вариационные автокодировщики (variational autoencoders, VAE) — многообещающий подход к сжатию пространства наблюдений, который позволяет ускорить процесс обучения. Или, может, вам приглянулись какие-то другие методы DL?
- **#gdrl_ch13_tf05.** Составьте список ресурсов по продвинутым методикам для разработки сильного ИИ (независимо от того, упоминались они здесь или нет).
- **#gdrl_ch13_tf06.** Используя свой любимый алгоритм, относящийся к методам разработки AGI, упомянутым в этой главе, создайте блокнот Jupyter и напишите об этом подробную статью.
- **#gdrl_ch13_tf07.** Составьте список интересных готовых сред.
- **#gdrl_ch13_tf08.** Создайте свою среду — что-то уникальное, к чему вы неравнодушны. Это может быть оболочка ИИ для прохождения какой-то игры, торговли на фондовом рынке и т. д.
- **#gdrl_ch13_tf09.** Обновите свое резюме, пришлите его мне, и я опубликую его в «Твиттере». Не забудьте указать несколько проектов, над которыми вы работали (относящихся к DRL, конечно).
- **#gdrl_ch13_tf10.** В каждой главе есть заключительный, всеохватывающий хештег. Можете использовать его для обсуждения любых других тем из этой главы, над которыми вы работали. Лучшее домашнее задание — то, которое вы сами себе задаете. Не забудьте поделиться тем, что вы решили исследовать самостоятельно, и полученными результатами.

Напишите твит со своими результатами, отметьте меня: @mimoralea (чтобы я ретвитнул) — и укажите подходящий хештег из приведенного выше списка, чтобы все заинтересованные могли найти эту информацию. Используйте эту возможность, чтобы пообщаться, внести свой вклад и проявить себя. Мы вас ждем!

Пример твита:

Привет, @mimoralea! Я написал статью со списком ресурсов, посвященных глубокому обучению с подкреплением. Можете найти ее по ссылке <ссылка>.
#gdrl_ch01_tf01

Я обязательно ретвитну вашу статью и помогу другим найти ее.

Мигель Моралес
Грокаем глубокое обучение с подкреплением

Перевел с английского Р. Волошко

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературные редакторы	<i>Ю. Зорина, Т. Сажина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>Е. Павлович, Н. Терех</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».
Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 07.2023. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 23.05.23. Формат 70×100/16. Бумага офсетная. Усл. п. л. 37,410. Тираж 1200. Заказ 0000.